



Генеративный ИИ на практике: трансформеры и диффузионные модели

Омар Сансевьеро, Педро Куэнка,
Аполинарио Пассос, Джонатан Уитакер

Hands-On Generative AI with Transformers and Diffusion Models

*Omar Sanseviero, Pedro Cuenca,
Apolinário Passos, and Jonathan Whitaker*

O'REILLY®

**Омар Сансевьеро, Педро Куэнка,
Аполинарио Пассос, Джонатан Уитакер**

Генеративный ИИ на практике: трансформеры и диффузионные модели

Астана
«АЛИСТ»
2026

УДК 004.89
ББК 32.973.26-018.1
Г34

Г34 Генеративный ИИ на практике: трансформеры и диффузионные модели:
пер. с англ. — Астана: АЛИСТ, 2026. — 400 с.: ил.

ISBN 978-601-12-6514-0

Книга представляет собой исчерпывающее практическое руководство по современному генеративному искусственному интеллекту. Она последовательно проводит читателя от основ представления информации до передовых методов создания изображений, текста и аудио с помощью открытых моделей. Подробно разбираются ключевые архитектуры: трансформеры и автоэнкодеры, CLIP, диффузионные модели и Stable Diffusion. Существенная часть книги посвящена трансферному обучению, включая тонкую настройку языковых моделей и моделей для генерации изображений. Подробно описаны креативные приложения, генерация аудио и рассмотрены стремительно развивающиеся направления в этой области. Книга имеет ярко выраженную практическую направленность, содержит пошаговые инструкции, проекты для самостоятельной работы, упражнения и задачи для закрепления материала.

Для специалистов-практиков по генеративному ИИ

УДК 004.89
ББК 32.973.26-018.1

© 2026 ALIST LLP

Authorized Russian translation of the English edition of *Hands-On Generative AI with Transformers and Diffusion Models* ISBN 9781098149246 © 2025 Omar Sanseviero, Pedro Cuenca, Apolinário Passos, and Jonathan Whitaker. This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

Авторизованный перевод с английского языка на русский издания *Hands-On Generative AI with Transformers and Diffusion Models* ISBN 9781098149246 © 2025 Omar Sanseviero, Pedro Cuenca, Apolinário Passos, Jonathan Whitaker.

Перевод опубликован и продается с разрешения компании-правообладателя O'Reilly Media, Inc.

ISBN 978-1-098-14924-6 (англ.)
ISBN 978-601-12-6514-0 (каз.)

© Omar Sanseviero, Pedro Cuenca, Apolinário Passos, Jonathan Whitaker, 2025
© Издание на русском языке ТОО "АЛИСТ", 2026

Оглавление

Отзывы о книге	11
Предисловие	13
Для кого эта книга	13
Предварительная подготовка.....	14
Чему вы научитесь	14
Как следует читать эту книгу	15
Требования к программному и аппаратному обеспечению.....	15
Условные обозначения, используемые в книге	16
Использование примеров кода	17
Комплект цветных изображений.....	17
Актуальность книги.....	17
Благодарности.....	17
 ЧАСТЬ I. ИСПОЛЬЗОВАНИЕ ОТКРЫТЫХ МОДЕЛЕЙ.....	21
 Глава 1. Введение в генеративные медиа	23
Генерация изображений	24
Генерация текста	26
Создание аудио	28
Этические и социальные последствия	28
Где мы были раньше и как обстоят дела сейчас	29
Как создаются генеративные ИИ-модели	30
Заключение.....	31
 Глава 2. Трансформеры	33
Языковая модель в действии	34
Токенизация текста	34
Прогнозирование вероятностей	37
Генерация текста	40
Генерализация с нулевым выстрелом (Zero-shot).....	49
Генерализация с несколькими выстрелами (Few-shot)	51
Блок трансформера.....	53
Генеалогия модели-трансформера	55
Задачи «последовательность-последовательность»	55
Модели, имеющие только энкодер	57
Сила предварительного обучения	60
Краткий обзор трансформеров	63
Ограничения	65
Помимо текста	66

Создание проекта: использование языковой модели для генерации текста	70
Заключение.....	70
Вопросы.....	72
Практика.....	72
Глава 3. Сжатие и представление информации	75
Автоэнкодеры	77
Подготовка данных	78
Энкодер	80
Декодер.....	83
Обучение.....	84
Исследование латентного пространства.....	89
Визуализация латентного пространства.....	93
Вариационные автоэнкодеры	97
Энкодеры и декодеры VAE	98
Выборка из распределения энкодера.....	99
Обучение VAE.....	102
Использование VAE для генеративного моделирования	110
CLIP	111
Контрастные потери.....	111
Использование CLIP, шаг за шагом.....	113
Классификация изображений с нулевым выстрелом с помощью CLIP	118
Конвейер классификации изображений с нулевым выстрелом	120
Варианты использования CLIP	121
Альтернативы CLIP.....	122
Время проекта: семантический поиск изображений	122
Заклучение.....	124
Вопросы.....	125
Задачи	126
Глава 4. Модели диффузии.....	127
Итеративное уточнение — ключ к пониманию моделей диффузии.....	128
Обучение моделей диффузии	131
Данные.....	132
Добавление шума	134
UNet	135
Обучение	137
Выборка.....	139
Оценка	140
Графики шума.....	142
Зачем надо добавлять шум?	143
Начинаем с простого	144
Математика	146
Влияние входного разрешения и масштабирования	151
Подробный разбор: UNet и альтернативы.....	153
Простая модель UNet.....	154
Улучшение UNet	157
Альтернативные архитектуры.....	158
Подробный разбор: цели диффузии.....	159
Время проекта: обучение собственной модели диффузии	161

Заключение.....	162
Вопросы.....	162
Задачи.....	163
Глава 5. Stable Diffusion и обусловленная генерация.....	165
Добавление контроля: модели с условиями.....	165
Подготовка данных.....	166
Создание модели, обусловленной классом.....	168
Обучение модели.....	169
Выборка.....	171
Повышение эффективности: Latent Diffusion.....	174
Stable Diffusion: подробнее о компонентах.....	175
Энкодер текста.....	176
Вариационный автоэнкодер (VAE).....	178
UNet.....	181
Stable Diffusion XL.....	183
FLUX, Stable Diffusion 3 и генерация видео.....	185
Руководство без классификаторов.....	185
Собираем все вместе: аннотированный цикл выборки.....	187
Открытые данные, открытые модели.....	190
Закат LAION-5B.....	191
Альтернативы.....	192
Добросовестное и коммерческое использование.....	192
Время проекта: создание интерактивной демо-модели с помощью Gradio.....	193
Заклучение.....	194
Вопросы.....	195
Задачи.....	195

ЧАСТЬ II. ПЕРЕНОС ОБУЧЕНИЯ ДЛЯ ГЕНЕРАТИВНЫХ МОДЕЛЕЙ..... 197

Глава 6. Тонкая настройка языковых моделей.....	199
Классификация текста.....	200
Определение набора данных.....	201
Определение типа модели.....	202
Выбор хорошей базовой модели.....	203
Предварительная обработка набора данных.....	204
Определение оценочных метрик.....	206
Обучение модели.....	208
Все еще актуально?.....	216
Генерация текста.....	217
Выбор правильной генеративной модели.....	217
Обучение генеративной модели.....	221
Инструкции.....	225
Краткое введение в адаптеры.....	230
Краткое введение в квантование.....	234
Собираем все вместе.....	237
Более глубокое погружение в оценку.....	243
Время проекта: поисково-дополненная генерация.....	246
Заклучение.....	247
Вопросы.....	249
Задачи.....	249

Глава 7. Тонкая настройка Stable Diffusion	251
Тонкая настройка полной модели Stable Diffusion	251
Подготовка набора данных	252
Тонкая настройка модели	254
Вывод модели	257
DreamBooth	259
Подготовка набора данных	261
Сохранение предыдущих знаний	261
Подготовка модели с помощью DreamBooth	262
Вывод	263
Обучение LoRA	264
Предоставление для Stable Diffusion новых возможностей	267
Inpainting	267
Дополнительные входы для специальных обусловливающих	267
Время проекта: самостоятельное обучение модели SDXL с помощью DreamBooth и LoRA	268
Заключение	269
Вопросы	270
Задачи	270
ЧАСТЬ III. ДВИГАЕМСЯ ДАЛЬШЕ	271
Глава 8. Творческое применение моделей text-to-image	273
Преобразование изображения в изображение	273
Inpainting	275
Взвешивание промптов и редактирование изображений	277
Взвешивание и слияние промптов	277
Редактирование диффузионных изображений с помощью Semantic Guidance	280
Редактирование реальных изображений с помощью инверсии	283
Редактирование с помощью LEDITS++	284
Редактирование реальных изображений с помощью тонкой настройки через инструкции	286
ControlNet	288
Изображения в качестве промптов и вариации изображений	291
Вариации изображений	291
Изображения в качестве промптов	293
Перенос стиля	293
Дополнительный контроль	295
Время проекта: ваш творческий холст	296
Заключение	296
Вопросы	297
Глава 9. Генерация аудио	299
Аудиоданные	301
Осциллограмма	305
Спектрограммы	306
Преобразование речи в текст с использованием архитектур на основе трансформеров	314
Техники на основе энкодеров	315
Техники на основе энкодер-декодера	319
От модели к конвейеру	323
Оценка	325

Генерация аудио	331
Генерация звука с помощью моделей Sequence-to-Sequence	332
Выход за рамки генерации речи с помощью Bark	337
AudioLM и MusicLM	339
AudioGen и MusicGen	342
Audio Diffusion и Riffusion	343
Dance Diffusion	346
Подробнее о моделях диффузии для генерации звука	347
Оценка систем генерации звука	348
Что дальше?	348
Время проекта: сквозная диалоговая система	349
Заключение	350
Вопросы	352
Задачи	353
 Глава 10. Быстро развивающиеся направления в области генеративного ИИ	355
Оптимизация предпочтений	355
Длинные контексты	357
Смесь экспертов	360
Оптимизации и квантование	362
Данные	364
Одна модель, чтобы править всеми	365
Компьютерное зрение	365
Компьютерное зрение 3D	368
Генерация видео	369
Мультимодальность	370
Сообщество	373
 Приложение А. Инструменты с открытым исходным кодом	375
Стек Hugging Face	375
Данные	376
Обертки	377
Локальный вывод	377
Инструменты развертывания	378
 Приложение В. Требования к памяти для моделей LLM	379
Требования к памяти вывода	379
Требования к памяти для обучения	380
Для дополнительного чтения	380
 Приложение С. Сквозная генерация, дополненная поиском	381
Обработка данных	381
Эмбединги документов	383
Извлечение	384
Генерация	385
RAG на уровне производства	387
 Предметный указатель	389
 Об авторах	393
 Об изображении на обложке	395

Отзывы о книге

Это важное техническое руководство, которое предоставляет понятные практические инструкции по внедрению стабильной диффузии и тонкой настройке языковых моделей. Оно обязательно должно быть на полке любого разработчика ИИ.

*Вики Рейзелман (Vicki Reyzelman),
главный архитектор решений ИИ, Mave Sparks*

Как всеобъемлющее и практическое руководство для тех, кто стремится освоить генеративный ИИ, эта книга сочетает теорию с реальными приложениями. От основ языковых моделей и методов диффузии до сложных тем, таких как тонкая настройка и создание приложений для преобразования текста в изображение, авторы предоставляют действенный код Python и четкие инструкции, которые позволяют читателям создавать и внедрять инновации и оставаться впереди в этой быстро развивающейся области. Их опыт и практический подход делают эту книгу бесценным ресурсом как для новичков, так и для опытных практиков.

– Анил Суд (Anil Sood), старший менеджер, Ernst & Young US

Это бесценное руководство демистифицирует генеративный ИИ, сочетая идеи с методами и примерами на практике, которые охватывают различные области. Она обязательна для прочтения тем, кто интересуется будущим ИИ.

*– Вишвеш Рави Шримали (Vishwesh Ravi Shrimali),
инженер в автомобильной промышленности*

Эта книга — невероятно хорошо составленное руководство, которое делает сложные ИИ-концепции доступными для широкого круга читателей. Авторы вносят ясность в трансформеры и модели диффузии, что делает книгу фантастической для тех, кто хочет по-настоящему понять фундаментальные строительные блоки, управляющие сегодняшним генеративным ИИ.

*– Сай М. Вуппалапати (Sai M Vuppalapati),
менеджер по продуктам платформ данных и области ИИ/МО, Tubi TV*

Эта книга — настоящая находка для тех, кто интересуется потенциалом контента, созданного с помощью ИИ. Сосредоточиваясь на решении актуальных проблем реальной жизни, она умело объединяет сложные концепции и делает

генеративный ИИ доступным как для энтузиастов, так и для профессионалов.

Она обязательна к прочтению для тех, кто готов погрузиться в эту динамичную область и исследовать силу генеративного ИИ.

– *Липи Дипакши Патнаик (Lipi Deepaakshi Patnaik), старший разработчик программного обеспечения, Zeta*

Эта книга — именно то, что вам нужно, чтобы начать работу с генеративным ИИ: от подробных объяснений до содержательных советов и упражнений для самостоятельного выполнения. В ней есть все. Это отличное руководство для тех, кто хочет научиться использовать, адаптировать и оценивать модели генеративного ИИ.

– *Люба Эллиотт (Luba Elliott), куратор по ИИ-искусству, elluba.com*

Омар, Педро, Аполинарио и Джонатан представляют впечатляющее сочетание технической глубины и интуитивно понятных руководств, позволяя читателям воплощать инновационные решения генеративного ИИ в жизнь с ясным пониманием и четкими намерениями. Благодаря понятным объяснениям трансформеров и моделей диффузии, их глубокой разработке и применению в тексте, изображениях и аудио они делают сложный мир ИИ доступным и действенным. Эта работа вооружает следующее поколение новаторов, чтобы они уверенно ориентировались в технических, этических и практических проблемах генеративного ИИ.

– *Адитья Гозль (Aditya Goel), консультант по ИИ*

Эта книга отлично подходит для тех, кто начинает свой путь с генеративного ИИ.

Авторы проводят нас по этой сложной теме простым и интуитивно понятным способом.

– *Зигмунт Леник (Zygmunt Lenyk), инженер-исследователь, Odyssey*

Данная книга предлагает всеобъемлющее и доступное руководство по основным концепциям и приложениям генеративного ИИ. Авторы искусно охватывают основные темы, от трансформеров и моделей диффузии до творческих приложений, что делает ее обязательной к прочтению для тех, кто хочет освоить технологии генеративного ИИ.

– *Гоурав Сингх Баис (Gourav Singh Bais), старший специалист по данным, старший автор технического контента, Allianz Services*

Это основное руководство для разработчиков по освоению инструментов и концепций, лежащих в основе крупнейшей революции ИИ за последнее десятилетие. Это настолько серьезный конкурент моей собственной книге, что я боюсь за наши гонорары!

– *Льюис Танстолл (Lewis Tunstall), инженер по машинному обучению в Hugging Face, соавтор «Natural Language Processing with Transformers»*

Предисловие

Генеративный ИИ — революционная технология, которая быстро шагнула от лабораторных демонстраций к реальным приложениям, повлияв на миллиарды людей. Она может создавать новый контент (изображения, текст, аудио, видео и многое другое), изучая закономерности на основе существующих данных, тем самым повышая креативность, дополняя информацию или помогая выполнять многие задачи. Например, модель генеративного ИИ, обученная на музыке, может сочинять новые мелодии, в то время как обученная на тексте может генерировать истории или даже программный код.

Эта книга не только для экспертов — она для всех, кто хочет узнать об этой увлекательной новой области. Мы не будем сосредотачиваться на создании моделей с нуля или погружении в сложную математику. Вместо этого мы будем использовать существующие модели для решения реальных задач, помогая вам сформировать прочное представление о том, как работают эти методы, и предоставляя основу для дальнейшего изучения.

Мы надеемся, что практический подход поможет вам быстро и эффективно приступить к работе с генеративным ИИ. Вы узнаете, как использовать предварительно обученные модели, адаптировать их для своих нужд и генерировать с их помощью новые данные. Вы также научитесь оценивать качество сгенерированных данных и исследовать этические и социальные проблемы, которые могут возникнуть при использовании генеративного ИИ. Это знакомство позволит вам оставаться в курсе новых разработок моделей и поможет определить области, которые вы, возможно, захотите изучить более глубоко.

Для кого эта книга

Учитывая новости, которые вы могли видеть о генеративных ИИ, а также о продуктах, которые они создают, вполне нормально испытывать волнение или беспокойство. Если вам интересно, как программы могут генерировать изображения, как обучить модель вести ленту социальных сетей в вашем стиле или как глубже понять нейросети вроде ChatGPT, эта книга для вас. С помощью генеративного ИИ мы можем делать все это и многое другое, например:

- ◆ писать выдержки из новостных статей;
- ◆ создавать изображения по их словесному описанию;
- ◆ улучшать качество изображений;

- ♦ транскрибировать аудиозаписи встреч;
- ♦ создавать синтетическую речь, похожую на конкретный голос;
- ♦ включать новые темы или стили в модели при генерации изображений (например, создать изображение «вашего кота, одетого как астронавт»).

Независимо от причины, по которой вы решили узнать о генеративном ИИ, эта книга станет хорошим проводником в этом путешествии.

Предварительная подготовка

В этой книге предполагается, что вы уверенно программируете на Python и имеете базовые знания о машинном обучении, включая такие фреймворки, как PyTorch или TensorFlow. Наличие практического опыта работы с моделями обучения необязательно, но будет полезным дополнением для более глубокого понимания содержания. Следующие ресурсы дают хорошую основу для тем, рассматриваемых в этой книге:

- ♦ *«Hands-On Machine Learning with Scikit-Learn, Keras u TensorFlow, 2nd ed.»* — Орельен Жерон (Aurélien Géron), издательство O'Reilly.
- ♦ *«Deep Learning for Coders with fastai and PyTorch»* — Джереми Говард (Jeremy Howard) и Сильвен Гатгер (Sylvain Gugger), издательство O'Reilly.

Если вас это пугает, не волнуйтесь. Книга призвана улучшить вашу интуицию и предоставить практический подход, который поможет вам начать работу.

Чему вы научитесь

Эта книга разделена на три части:

- ♦ В *части I «Использование открытых моделей»* вы познакомитесь с основными строительными блоками генеративного ИИ. Вы узнаете, как применять предварительно обученные модели для генерации текста и изображений. Эта часть поможет вам изучить основы данной области и понять общую картину.
- ♦ *Часть II «Перенос обучения для генеративных моделей»* посвящена тонкой настройке ИИ. Она демонстрирует способы, которые позволяют взять существующие модели и адаптировать их к вашим потребностям. Мы расскажем, как обучить модель диффузии новой концепции, как настроить модель трансформера, чтобы она могла классифицировать текст или участвовать в диалогах, а также как подойти к изучению передовых методов работы с большими моделями на ограниченном оборудовании. Не стоит волноваться, если вы впервые читаете о трансформерах или моделях диффузии, скоро вы о них узнаете.
- ♦ В *части III «Двигаемся дальше»* идеи из предыдущих частей будут расширены и дополнены. Вы научитесь генерировать данные в новых модальностях, таких как аудио, и проявлять креативность при создании новых приложений. После прочтения этой книги у вас будет четкое понимание методов и приемов, на которых строятся генеративные приложения.

Как следует читать эту книгу

Мы разработали книгу так, чтобы ее можно было читать по порядку, но при этом главы являются максимально автономными, чтобы вы могли быстро переходить к тем частям, которые вас больше всего интересуют. Многие идеи, изложенные здесь, применимы к разным модальностям, поэтому даже если вас интересует только одна конкретная область (например, генерация изображений), вам все равно может быть полезно просмотреть другие главы.

Мы включили упражнения и примеры кода на протяжении всей книги, чтобы помочь вам освоить материал. Постарайтесь выполнять их по мере продвижения и попробуйте адаптировать их к своим задачам, где это возможно. Самостоятельная практика поможет вам глубже понять материал.

В заключениях к большинству глав перечислены дополнительные ресурсы. Мы рекомендуем изучить их, чтобы улучшить понимание областей, затронутых в книге. Вам не нужно читать их, прежде чем перейти к новой главе. Вы можете вернуться позже, когда будете готовы подробнее изучить интересующие вас темы.

Требования к программному и аппаратному обеспечению

Чтобы извлечь максимальную пользу из этой книги, мы настоятельно рекомендуем выполнять примеры кода по мере чтения. Экспериментируйте с кодом, внося изменения и исследуя различные сценарии, чтобы улучшить свое понимание. Работа с трансформерами и моделями диффузии может быть ресурсоемкой, поэтому желательно иметь доступ к компьютеру с графическим процессором NVIDIA. Несмотря на то что это требование не является обязательным, оно значительно ускорит время обучения.

Вы можете использовать любую из облачных вычислительных сред, таких как Google Colaboratory и Kaggle Notebooks. Следуйте инструкциям, чтобы настроить свою среду и не отставать от нас:

◆ *Использование Google Colab*

Большая часть кода из примеров должна работать на любом экземпляре Google Colab. Мы рекомендуем использовать оборудование с GPU для примеров, имеющих обучающие циклы.

◆ *Локальный запуск кода*

Чтобы запустить код на компьютере, создайте виртуальную среду Python 3.10, используя предпочитаемый вами метод. Например, вы можете сделать это с помощью conda следующим образом:

```
conda create -n genaibook python=3.10
conda activate genaibook
```

Для оптимальной производительности мы рекомендуем использовать графический процессор¹, совместимый с CUDA.

Если вы не знаете, что такое CUDA, не волнуйтесь, мы объясним это в книге. Мы будем использовать множество вспомогательных утилит и функций. Чтобы получить к ним доступ, установите пакет `genaibook`.

```
pip install genaibook
```

Эта команда установит библиотеки, необходимые для запуска трансформеров и моделей диффузии, а также PyTorch, matplotlib, numpy и другие необходимые компоненты.

Все примеры кода и дополнительные материалы можно найти в репозитории книги на GitHub. Вы можете запустить их интерактивно в Jupyter Notebooks. Репозиторий будет регулярно обновляться последними ресурсами.

Условные обозначения, используемые в книге

В этой книге использованы следующие типографские условные обозначения:

◆ *Курсив*

Обозначает новые термины, URL-адреса, адреса электронной почты.

◆ Моноширинный шрифт

Используется для листингов кода, имен файлов, их расширений а также внутри абзацев для ссылки на элементы программы, такие как имена переменных или функций, базы данных, типы данных, переменные среды, операторы и ключевые слова.



Обозначает советы или рекомендации.



Обозначает примечания.



Обозначает предупреждения или предостережения

¹ Вместо графического процессора вы также можете использовать устройство *MPS* (*Metal Performance Shaders*), которое может работать на компьютерах Mac с процессором Apple Silicon, но мы не проводили тщательного тестирования данной конфигурации.

Использование примеров кода

Дополнительные материалы (примеры кода, упражнения и т. д.) доступны для загрузки по адресу <https://oreil.ly/handsOnGenAICode>.

Если у вас возникнет технический вопрос или проблема с использованием кода, отправьте электронное письмо по адресу support@oreilly.com.

Эта книга поможет вам решить ваши задачи. Если код предложен здесь, вы можете использовать его в своих программах и документации. Вам не нужно обращаться к нам за разрешением, если вы не собираетесь воспроизводить значительную часть какого-либо примера. Для пояснения: написание программы, которая использует несколько фрагментов из листинга какого-либо кода, не требует разрешения. Продажа или распространение примеров из книг O'Reilly требует разрешения. Ответы на вопросы с помощью цитат и примеров из этой книги не требуют разрешения. Включение значительного количества кода в документацию вашего продукта требует разрешения.

Мы ценим, но, как правило, не требуем указывать авторство. Обычно оно включает название, автора, издателя и ISBN. Например: «*Hands-On Generative AI with Transformers and Diffusion Models*» Омара Сансевьеро, Педро Куэнки, Аполинариу Пассоса и Джонатана Уитакера (издательство O'Reilly). Copyright 2025 Omar Sanseviero, Pedro Cuenca, Apolinário Passos u Jonathan Whitaker, 978-1-098-14924-6. Если вы считаете, что использование вами примеров кода выходит за рамки добросовестного использования или разрешений, указанных выше, свяжитесь с нами по адресу permissions@oreilly.com.

Комплект цветных изображений

Цветные версии рисунков для важных изображений доступны по ссылке https://files.alistbooks.kz/3143_Hands_On_AI.zip.

Актуальность книги

Понятие *State of the art* (SOTA) используется для описания наивысшего уровня производительности, достигнутого в настоящее время в определенной задаче или области. Касательно генеративного ИИ SOTA постоянно меняется по мере разработки новых моделей и открытия новых методов.

Эта книга даст вам прочную базу, но к тому времени, как вы ее прочтете, скорее всего, будут выпущены новые модели, которые превзойдут те, что мы здесь обсуждаем.

Благодарности

Мы хотели бы выразить нашу глубочайшую благодарность невероятной команде O'Reilly, особенно Джилл Леонард (Jill Leonard), за ее удивительное руководство и поддержку на протяжении всего процесса. Особая благодарность Николь Баттер-

филд (Nicole Butterfield), Карен Монтгомери (Karen Montgomery), Кейт Даллеа (Kate Dullea), Грегори Хайману (Gregory Hyman) и Кристен Браун (Kristen Brown) за их бесценные советы и вклад, от первоначального обзора до создания прекрасной обложки и иллюстраций. Мы глубоко признательны нашим техническим рецензентам: Вишвешу Рави Шримали (Vishwesh Ravi Shrimali), Дэвиду Мерцу (David Mertz), Липи Дипакши Патнаик (Lipi Deeraakshi Patnaik), Любе Эллиотт (Luba Elliott), Анилу Суду (Anil Sood), Саю Вуппалапати (Sai Vuppalarati), Ранджите Бхаттачарья (Ranjeeta Bhattacharya), Раджату Дубею (Rajat Dubey), Брайану Бишофу (Bryan Bischof), Владиславу Билаю (Vladislav Bilay), Гураву Сингху Баису (Gourav Singh Bais), Адитье Гоэль (Aditya Goel), Лакшманану Сету Санкаранараянану (Lakshmanan Sethu Sankaranarayanan), Зигмунту Ленику (Zygmunt Lenyk), Юсефу Хосну (Youssef Hosn), Вики Рейзелман (Vicki Reyzelman), Льюису Танстолу (Lewis Tunstall), Саяку Полу (Sayak Paul) и Вайбхаву Шриваставу (Vaibhav Srivastav). Их проникательные отзывы сыграли важную роль в создании этой книги.

Мы также хотели бы выразить нашу благодарность команде Hugging Face за их вдохновение и сотрудничество, в частности Клементине Фурье (Clementine Fourrier) за ее идеи по оценке моделей, Санчиту Ганди (Sanchit Gandhi) за его руководство по темам, связанным с генерацией звуков, а также Леандро фон Верра (Leandro von Werra) и Льюису Танстолу (Lewis Tunstall) за помощь в процессе написания книги. Команда Hugging Face продолжает вдохновлять нас своей гениальностью и добротой, помогая воплотить этот проект в жизнь.

Сердечная благодарность бесчисленным друзьям, соавторам и участникам, которые сформировали экосистему с открытым исходным кодом, быть частью которой мы очень гордимся. Мы благодарны всему сообществу машинного обучения (МО) за продвижение исследований, инструментов и ресурсов, которые составляют сердце этого руководства. Вся работа была выполнена в среде Jupyter Notebooks, и мы хотим выразить особую благодарность Джереми Ховарду (Jeremy Howard), Хамелу Хусейну (Hamel Husain) и всем участникам *Quarto* и *nbdev* за то, что сделали это возможным.

Джонатан

Я очень благодарен сообществу исследователей и хакеров, которые делятся своими идеями и продвигают вперед то, что возможно. Джереми Ховарду (Jeremy Howard), Танишку Абрахаму (Tanishq Abraham) и остальной команде *fastdiffusion*, которые собрались вместе, чтобы узнать все возможное об этих идеях. И моим замечательным соавторам, без которых эта книга не могла бы появиться!

Аполлинарио

Я благодарен моим соавторам Омару, Педро и Джонатану за совместное создание этой книги. Объединение технологического образования и творчества стало забавной задачей. Я благодарю своих друзей, которые понимают и поддерживают меня, даже когда я просто прихожу «потусоваться» с ноутбуком, и моих коллег из Hugging Face за постоянную поддержку.

Педро

Писать книгу очень весело, но это требует жертв от людей, которых ты любишь. Мне очень повезло, что меня поддержала Мария Хосе (Maria Jose), моя спутница жизни. Она облегчила работу над книгой, а когда я застрял, помогала мне здравым смыслом, что, честно говоря, совсем не распространено. Я извиняюсь перед мамой и папой за то, что всегда беру с собой ноутбук, когда приезжаю, перед сыном Пабло (Pablo) за то, что исследовал Хайрул или Эорзею не так много, как нам бы хотелось, и перед сыном Хавьером (Javier) за то, что иногда слишком много говорю о работе и слишком мало о жизни. Они лучшие.

Меня действительно вдохновляют мои замечательные соавторы. Я восхищаюсь ими, уважаю их и не могу поверить, как мне повезло учиться у них каждый день. Это касается и ребят из Hugging Face, чей энтузиазм и скромность создают первичный бульон, в котором все происходит, и ребят из открытого сообщества машинного обучения в целом, чья работа всегда продвигает эту область, но не всегда получает заслуженное признание.

Спасибо.

Омар

Мишель (Michelle), спасибо за твое поощрение в течение всего этого процесса, за все мозговые штурмы и твою поддержку в течение последних двух лет. Я бы не смог завершить этот проект без тебя. Походы снова на повестке дня!

Спасибо моим родителям, Ане (Ana) и Уолтеру (Walter), за то, что вы с самого начала привили мне любовь к книгам и помогли мне стать тем человеком, кем я являюсь сейчас.

Наконец, я хочу поблагодарить моих замечательных соавторов — Педро, Поли и Джонатана. Это путешествие было по-настоящему веселым, и я так благодарен, что мы достигли этого вместе.

ЧАСТЬ I

Использование открытых моделей

Введение в генеративные медиа

Генеративные модели стали очень популярны в последние годы. Если вы читаете эту книгу, вероятно, вы когда-то взаимодействовали с ними. Возможно, вы использовали ChatGPT для генерации текста, делали перенос стилей в таких приложениях, как Instagram¹, или смотрели видео с *дипфейками* (Deepfake), которые попали в заголовки различных новостей. Все это примеры генеративных моделей в действии.

В этой книге мы рассмотрим мир генеративных моделей. Начнем с основ для двух их семейств (трансформеры и модели диффузии), а затем перейдем к более сложным темам. Вы познакомитесь с типами генеративных моделей, узнаете, как они работают и как их использовать. В данной главе вы прочтете часть истории, как наша компания к этому пришла, и увидите возможности, предлагаемые некоторыми моделями, которые мы рассмотрим более подробно на протяжении всей книги.

Итак, что же такое *генеративное моделирование*? По сути, это обучение модели генерировать новые данные, которые напоминают ее обучающие наборы информации. Например, если мы обучим модель на ряде изображений с кошками, затем мы сможем использовать ее, чтобы генерировать новые рисунки кошек, которые похожи на исходный набор. Это мощная идея, и она имеет широкий спектр применения, от генерации новых изображений и видео до написания текста в определенном стиле.

В этой книге вы узнаете о популярных инструментах, которые упрощают использование существующих генеративных моделей. Мир *машинного обучения* (Machine Learning, ML) предлагает множество открытых моделей, обученных на больших наборах данных, доступных для использования любым человеком. Обучение их с нуля может быть дорогостоящим и трудоемким процессом, но модели с открытым доступом предоставляют практичную и эффективную альтернативу. Они могут генерировать новые данные, классифицировать существующую информацию и адаптироваться для новых приложений. Одним из самых популярных мест для поиска моделей с открытым доступом является Hugging Face — платформа с более чем двумя миллионами моделей для многих ML-задач, включая генерацию изображений.

¹ Соцсеть запрещена на территории Российской Федерации (принадлежит компании Meta).

Генерация изображений

В качестве примера библиотеки с открытым исходным кодом мы рассмотрим `diffusers`. Она предоставляет доступ к современным (SOTA) моделям диффузии. Это мощный и простой набор инструментов, который позволяет быстро загружать и обучать диффузионные модели.

Перейдя в `Hugging Face Hub` и отфильтровав модели, которые генерируют изображения на основе текстового запроса (`text-to-image`), мы можем найти некоторые популярные варианты, такие как `Stable Diffusion` или `Stable Diffusion XL (SDXL)`. Мы будем использовать `Stable Diffusion 1.5` — модель диффузии, способную генерировать высококачественные изображения. На ее веб-сайте вы можете прочитать *карточку модели* — важный документ, который обеспечивает ее доступность для обнаружения и воспроизводимость (повторяемость). Там вы можете прочитать о том, как она обучалась, о предполагаемых вариантах использования и многом другом.

Теперь, когда у нас есть модель (`Stable Diffusion`) и инструмент для ее использования (`diffusers`), мы можем сгенерировать наше первое изображение. Обратите внимание, при загрузке моделей их нужно отправить на определенное устройство, например `CPU` (`cpu`), `GPU` (`cuda` или `cuda:0`) или `Mac-оборудование` под названием `Metal` (`mps`). Библиотека `genaibook`, о которой мы упоминали в Предисловии, имеет *функцию полезности* для выбора подходящего устройства в зависимости от того, где вы запускаете код. Например, следующий фрагмент назначит `cuda` переменной устройства, если оно имеет `GPU`:

```
from genaibook.core import get_device
```

```
device = get_device()
print(f"Using device: {device}")
```

```
Using device: cuda
```

Далее мы загрузим `Stable Diffusion 1.5`. Библиотека `diffusers` предоставляет удобную высокоуровневую оболочку под названием `StableDiffusionPipeline`, которая идеально подходит для нашего варианта использования. Не стоит беспокоиться обо всех ее параметрах на данный момент, мы пока берем во внимание основные моменты:

- ♦ Существует много моделей с архитектурой `Stable Diffusion`, поэтому нам нужно указать ту, которую мы хотим применить. Мы будем использовать `stable-diffusion-v1-5/stable-diffusion-v1-5` — зеркало оригинальной `Stable Diffusion 1.5`, выпущенное компанией `RunwayML`.
- ♦ Нам необходимо указать *точность* (о ней вы узнаете позже), с которой будет загружена модель. На высоком уровне модели состоят из множества параметров (их миллионы или даже миллиарды). Каждый из них — это число, полученное во время обучения, и они могут храниться с разными уровнями точности. Другими словами, нам может понадобиться больше бит для хранения модели. Большая точность позволяет хранить больше информации, но также требует

больше памяти и вычислений. С другой стороны, мы можем использовать более низкую точность, установив параметр `torch_dtype=float16`, и использовать меньше памяти (по умолчанию `float32`). При выполнении вывода модели использование `float16` обычно является вполне приемлемым².

Первый запуск этого кода может занять некоторое время, поскольку конвейер (`StableDiffusionPipeline`) загружает модель размером в несколько гигабайт. Если вы выполните его во второй раз, он повторно загрузит модель только в том случае, если произошли какие-либо изменения в удаленном репозитории (`Hugging Face Hub`), где размещена модель. Библиотеки `Hugging Face` хранят модель локально в кеше, что значительно ускоряет последующие загрузки.

```
import torch
from diffusers import StableDiffusionPipeline

pipe = StableDiffusionPipeline.from_pretrained(
    "stable-diffusion-v1-5/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
    variant="fp16",
).to(device)
```

Теперь, когда модель загружена, мы можем определить *промпт* (**prompt**) — текстовый ввод, который будет передан в модель, которая затем сгенерирует наше первое изображение. Попробуйте ввести следующий промпт (результат изображен на рис. 1.1):

```
prompt = "a photograph of an astronaut riding a horse"
pipe(prompt).images[0]
```

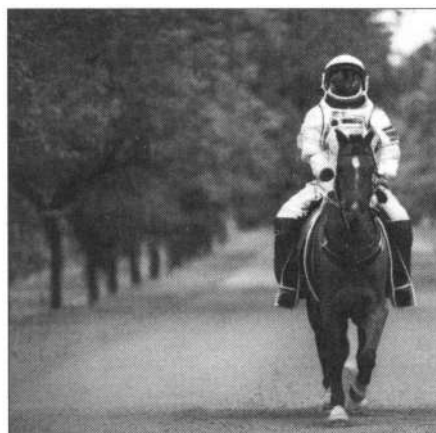


Рис. 1.1. Первое сгенерированное изображение на основе промпта

² Вас может заинтересовать параметр `variant`. Для некоторых моделей можно обнаружить, что есть несколько контрольных точек с разной точностью. При указании `torch_dtype=float16` мы загружаем модель по умолчанию (`float32`) и преобразуем ее в `float16`. Указывая также `variant="fp16"`, мы загружаем меньшую контрольную точку, уже сохраненную в точности `float16`, что требует вдвое меньше пропускной способности и места в хранилище для ее загрузки. Проверьте модель, которую вы хотите использовать, чтобы узнать, есть ли несколько вариантов ее точности.

Восхитительно! С помощью пары строк кода мы сгенерировали новое изображение. Попробуйте поиграть с промптом и сгенерировать другие картинки.

Вы можете заметить две вещи. Во-первых, запуск одного и того же кода каждый раз будет генерировать разные рисунки. Это происходит потому, что процесс диффузии является *стохастическим* по своей природе: он использует *случайность* для генерации изображений. Но мы можем контролировать ее, установив *начальное значение*:

```
import torch
torch.manual_seed(0)
```

Во-вторых, сгенерированные изображения не идеальны. Они могут иметь артефакты, быть размытыми или вообще не соответствовать промпту. Мы рассмотрим эти ограничения и то, как улучшить качество сгенерированных рисунков, в последующих главах:

- ♦ В *главах 4 и 5* мы рассмотрим все компоненты, лежащие в основе моделей диффузии, а также способы перехода от текста к новым изображениям. Там описаны такие фундаментальные методы, как *автоэнкодеры (AutoEncoder)*, которые будут представлены в *главе 3*. Модели диффузии могут обучаться эффективному представлению входных данных и сокращать требования к вычислениям для построения диффузионных и других генеративных моделей.
- ♦ В *главе 7* вы узнаете, как обучить Stable Diffusion новым темам. Например, мы можем научить ее концепции «моя собака», чтобы генерировать изображения животного в новых сценариях (например, «моя собака посещает Луну»).
- ♦ В *главе 8* мы покажем, как модели диффузии можно использовать не только для генерации изображений. Например, их можно применять для редактирования рисунков с помощью промптов или заполнять пустые места на картинках.

Генерация текста

Так же как `diffusers` удобна при работе с моделями диффузии, популярная библиотека `transformers` чрезвычайно полезна для запуска моделей на основе трансформеров и адаптации к новым вариантам использования. Она предоставляет стандартизированный интерфейс для широкого спектра задач: генерация текста, обнаружение объектов на изображениях и транскрибирование аудиофайла в текст.

Библиотека `transformers` предоставляет различные уровни абстракций. Например, если вас не интересуют все внутренние компоненты, проще всего использовать конвейер, который абстрагирует всю обработку, необходимую для получения прогноза. Его можно создать, вызвав функцию `pipeline()` и указав, какую задачу необходимо решить, например классификацию текста (`text-classification`).

```
from transformers import pipeline

classifier = pipeline("text-classification", device=device)
classifier("This movie is disgustingly good!")

[{'label': 'POSITIVE', 'score': 0.9998536109924316}]
```

Модель верно предположила, что тон входного текста был позитивным (POSITIVE). По умолчанию конвейер классификации текста «под капотом» использует анализ тональности, но мы также можем указать другие модели классификации текста, например на основе трансформеров.

Аналогичным образом можно переключить задачу с классификации на генерацию текста (text-generation), с помощью которой возможно создать новый текст на основе входного промпта. По умолчанию конвейер использует модель GPT-2. При генерации трансформер использует максимальное количество слов по умолчанию, поэтому не удивляйтесь, если вывод может быть «усечен» (позже вы узнаете, как это изменить).

```
from transformers import set_seed
```

```
# Установка начального значения гарантирует, что мы будем получать одни и те же #
результаты каждый раз, когда запускаем этот код
set_seed(10)
```

```
generator = pipeline("text-generation", device=device)
prompt = "It was a dark and stormy"
generator(prompt)[0]["generated_text"]
```

```
"It was a dark and stormy year, and my mind went blank," says the 27-year-old,
who has become obsessed with art, poetry and music since moving to France.
"I don't really know why, but there are things
```

Несмотря на то что GPT-2 не является хорошей моделью по сегодняшним меркам, она дает начальное понимание о возможностях трансформеров касательно генерации (при использовании небольшой модели). Те же концепции можно применить к таким моделям, как Llama или Mistral — одним из самых мощных моделей с открытым доступом (на момент написания). На протяжении всей книги мы будем соблюдать баланс между качеством и размером моделей. Обычно более крупные из них имеют более качественные генерации. В то же время мы хотим, чтобы люди с обычными компьютерами или доступом к бесплатным сервисам, вроде Google Colab, могли создавать новые данные, просто запустив код:

- ♦ Глава 2 расскажет вам, как работают модели трансформеров «под капотом». Мы подробно опишем различные их типы, а также как их использовать для генерации текста.
- ♦ Глава 6 научит вас, как обучить модели трансформеров на пользовательских данных для различных вариантов использования. Это позволит создать разговорные модели, подобные ChatGPT или Gemini. Мы также расскажем об эффективных подходах, которые позволят обучать модели трансформеров на пользовательском компьютере.

Создание аудио

Генеративные модели не ограничиваются изображениями и текстом. Они могут генерировать видео, короткие песни, синтетическую устную речь, планы строения белков и многое другое.

Глава 9 глубоко погрузит вас в задачи, связанные со звуком, которые можно решить с помощью ML, например расшифровку совещаний или создание звуковых эффектов. На данный момент мы можем ограничиться уже знакомым конвейером трансформеров и использовать уменьшенную версию MusicGen — модели, выпущенной компанией Meta³ для генерации музыки, обусловленной текстом.

```
pipe = pipeline("text-to-audio", model="facebook/musicgen-small",
               device=device)
data = pipe("electric rock solo, very intense")

print(data)

{'audio': array([[0.12342193, 0.11794732, 0.14775363, ..., 0.0265964 ,
                  0.02168683, 0.03067675]]), 'sampling_rate': 32000}
```

Поскольку у нас нет возможности распечатать аудиофайл прямо в книге, позже вы узнаете, в какой форме представляются аудиоданные и что это за числа в коде выше. Лучшая альтернатива — открыть код просмотрщиком в блокноте или сохранить аудио в файл, который затем можно воспроизвести с помощью приложения. Например, для этого можно использовать `IPython.display()`.

```
import IPython.display as ipd

display(ipd.Audio(data["audio"][0], rate=data["sampling_rate"]))
```

Этические и социальные последствия

Несмотря на то что генеративные модели предлагают замечательные возможности, их широкое распространение поднимает волнующие вопросы этики и влияния на общество. Важно помнить о них, когда вы исследуете возможности генеративных моделей. Вот несколько ключевых областей:

◆ Конфиденциальность и разрешение

Способность моделей генерировать реалистичные изображения и видео на основе небольшого количества данных формирует значительные проблемы для конфиденциальности. Например, синтетические изображения настоящих людей на основе их реальных изображений поднимают вопросы об использовании персональных данных без согласия. Это также увеличивает риск создания дипфейков,

³ Компания Meta признана экстремистской и ее деятельность запрещена на территории Российской Федерации. — Пер.

которые могут быть использованы, чтобы распространять дезинформацию или причинять вред отдельным лицам.

♦ *Предвзятость и справедливость*

Генеративные модели обучаются на больших наборах информации, которые могут содержать «предвзятые» данные. Такие ошибки могут наследоваться и усиливаться (мы рассмотрим это в *главе 2*). Например, наборы с такими данными, используемые для обучения, могут генерировать стереотипные или дискриминационные изображения. Важно рассмотреть возможность смягчения этих предвзятостей и гарантировать, что модели используются справедливо и этично.

♦ *Регулирование*

Учитывая потенциальные риски, связанные с генеративными моделями, растет потребность в механизмах нормативного надзора и подотчетности, чтобы обеспечить ответственное развитие. Это включает в себя требования прозрачности, этические принципы и правовые рамки для решения проблемы неправильного использования генеративных моделей.

Важно подходить к генеративным моделям с вдумчивым и этичным мышлением. По мере изучения их возможностей мы также рассмотрим этические последствия и то, как использовать их ответственно.

Где мы были раньше и как обстоят дела сейчас

Исследования и разработка генеративных моделей начались десятилетия назад с создания систем, основанных на правилах. По мере увеличения вычислительной мощности и доступности данных модели эволюционировали. Теперь они способны использовать статистические методы и машинное обучение. С появлением *глубокого обучения (Deep Learning)* как мощной парадигмы и прорывами в областях распознавания изображений и речи генеративные модели значительно продвинулись вперед. Несмотря на то что они были изобретены десятилетия назад, *сверточные (Convolutional Neural Networks, CNN)* и *рекуррентные (Recurrent Neural Networks, RNN)* нейронные сети стали широко популярны в последнее десятилетие. CNN произвели революцию в задачах обработки изображений, а RNN принесли возможности последовательного моделирования данных, позволяя решать такие задачи, как перевод и генерация текста.

Внедрение *генеративно-сопоставительных сетей (Generative Adversarial Networks, GAN)* Яном Гудфеллоу (Ian Goodfellow) в 2014 году, а также таких их вариантов, как *глубокие сверточные GAN (DCGAN)* и *условные GAN*, открыло новую эру генеративных моделей. GAN использовались для создания высококачественных изображений и применялись для переноса стиля, позволяя пользователям применять художественные стилистики к своим изображениям с поразительной реалистичностью. Несмотря на свою мощь, диффузионные модели превзошли GAN в последние годы.

Аналогично RNN были инструментом для моделирования языка. Однако трансформеры, включая архитектуры типа GPT, достигли наивысшей производительности в *обработке естественного языка (Natural Language Processing, NLP)*. Эти модели продемонстрировали замечательные возможности в таких задачах, как понимание языка, генерация текста и машинный перевод. GPT, в частности, стал чрезвычайно популярен из-за своей способности создавать связный и контекстно релевантный текст. Вскоре после этого появилась огромная волна генеративных языковых моделей.

Область генеративного ИИ стала более доступной, чем когда-либо, из-за быстрого расширения исследований, ресурсов и разработок в последние годы. Заинтересованное растущее ИИ-сообщество, богатая экосистема с открытым исходным кодом и исследования, облегчающие развертывание, привели к появлению широкого спектра приложений и вариантов использования. С 2023 года появилось новое поколение моделей, которые могут создавать высококачественные изображения, текст, код, видео и многое другое (ChatGPT, DALL·E, Imagen, Stable Diffusion, Llama, Mistral и многие другие).

Как создаются генеративные ИИ-модели

Обычно создание моделей сводится к большим затратам или открытому исходному коду. Самые впечатляющие генеративные модели за последнюю пару лет были созданы влиятельными исследовательскими лабораториями в крупных частных компаниях. OpenAI разработала ChatGPT, DALL·E и Sora; Google создала Imagen, Bard и Gemini; а Meta создала Llama и Code Llama.

Существует различная степень открытости при выпуске этих моделей. Некоторые из них можно использовать через определенные пользовательские интерфейсы, к другим есть доступ через API разработчиков, а третьи просто объявлены как исследовательские отчеты без какого-либо публичного доступа. В некоторых случаях могут сделать выпуск кода и весов моделей. Их обычно называют *релизами с открытым исходным кодом*, потому что это основные аспекты, необходимые для запуска модели на вашем оборудовании. Однако часто они скрываются по стратегическим причинам.

В то же время постоянно растущее, энергичное и восторженное ИИ-сообщество использует модели с открытым исходным кодом в качестве материала для своего творчества. Все типы практиков, включая исследователей, инженеров, мастеров и любителей, опираются на работы друг друга и предлагают новые решения и умные идеи, которые продвигают область вперед, одно достижение за другим. Некоторые из них попадают в теоретический корпус знаний, из которого черпают энтузиазм другие исследователи, и на основе их работ через некоторое время появляются новые впечатляющие модели.

Большие модели, даже скрытые, служат вдохновением для тех, чья работа приносит плоды. Этот цикл работает только потому, что некоторые модели имеют открытый исходный код и могут использоваться сообществом. Компании, которые выпускают подобные модели, делают это не из альтруистических соображений, а по

тому, что они находят экономическую ценность в такой стратегии. Предоставляя код и модели, они получают общественное внимание, а вместе с ним и исправления ошибок, новые идеи, производные архитектуры моделей или даже новые наборы данных, которые хорошо работают с выпущенными экземплярами. Поскольку все эти вклады основаны на опубликованных активах, данные компании могут быстро их принять во внимание и, таким образом, двигаться быстрее, чем они действовали бы самостоятельно. Когда Meta выпустила Llama, одну из самых популярных языковых моделей (**Language Model, LM**), вокруг нее органически выросла процветающая экосистема.

Как устоявшиеся, так и новые компании, включая Meta, Stability AI (Stable Diffusion) или Mistral AI, приняли различные степени открытости как часть своей бизнес-стратегии. Это так же закономерно, как и стратегия конкурирующих компаний, которые предпочитают хранить свои коммерческие секреты за закрытыми дверями (даже если они могут черпать информацию из открытого сообщества). На этом этапе мы хотели бы уточнить, что релизы моделей редко бывают действительно *открытыми*. Исходного кода, как и весов, недостаточно для полного понимания системы ML. Они являются лишь конечным результатом процесса обучения. Для точного воспроизведения существующей модели требуется исходный код, используемый при обучении (не только код моделирования или код вывода), режим и параметры обучения и, что особенно важно, все данные, используемые для этого. Ничего из этого, особенно данные, обычно не публикуется. Если бы был доступ к таким деталям, общественность могла бы понять, как работают модели, изучить предубеждения, которые могут на них влиять, и лучше оценить их сильные стороны и ограничения. Доступ к весам и коду дает несовершенную оценку всех этих знаний, фактические данные были бы намного лучше. Вдобавок ко всему, даже когда модели публикуются, они часто выпускаются со специальной лицензией, которая не соответствует определению открытого исходного кода, определенного в *Open Source Initiative*. Это не значит, что модели бесполезны или что компании не делают ничего хорошего, выпуская их. Но это является важным контекстом, который следует иметь в виду, и одной из причин, по которой часто говорят «открытый доступ» вместо «открытый исходный код».

Как бы то ни было, нет наилучшего времени для создания генеративных моделей или работы с ними. Вам не нужно быть инженером в первоклассной исследовательской лаборатории, чтобы придумать идеи для решения задач, которые вас интересуют, или внести свой вклад в эту область. Мы надеемся, что вы найдете эти страницы полезными в своем изучении.

Заключение

Надеюсь, после создания ваших первых изображений, текста и аудио вы будете рады узнать, как работают диффузия и трансформеры «под капотом», как адаптировать их для новых сценариев использования и как применять их в различных творческих приложениях. Несмотря на то что в этой главе основное внимание было уделено инструментам поверхностно, мы заложили прочную основу для понимания, как работают генеративные модели, когда отправимся дальше.

Трансформеры

Последние достижения в области генеративного ИИ связаны с появлением в 2017 году класса моделей, которые называются *трансформерами* (**Transformer**). Они стали известны благодаря *большим языковым моделям* (**Large Language Model, LLM**), таким как Llama и GPT-4, которые ежедневно используются сотнями миллионов пользователей. Трансформеры стали основой для современных приложений: от чат-ботов и поисковых систем до машинного перевода и резюмирования контента. Помимо работы с текстом, они показали хорошие результаты во многих областях, например компьютерное зрение, генерация музыки и *фолдинг белков* (**Protein folding**)¹. В этой главе мы рассмотрим основные идеи, лежащие в основе трансформеров, и то, как эти модели работают. Мы уделим особое внимание одному из самых распространенных сценариев применения — *языковому моделированию*.

Прежде чем углубиться в детали, сначала разберемся, что такое языковое моделирование. По своей сути *языковая модель* (**Language model, LM**) — это вероятностная модель, которая учится предсказывать следующее слово — *токен* (**Token**) — в последовательности на основе предыдущих или окружающих слов. Это позволяет зафиксировать базовую структуру и закономерности языка, что, в свою очередь, дает возможность генерировать реалистичный и связный текст. Например, если у вас есть предложение «Я начал свой день с еды», LM с высокой долей вероятности предскажет, что следующим словом будет «завтрак».

Каким образом трансформеры вписываются в эту картину? Они разработаны для эффективной обработки долгосрочных зависимостей и сложных отношений между словами. Представьте, что вы хотите резюмировать новостную статью, которая может быть большого объема (сотни или даже тысячи слов). Традиционные LM (вроде RNN) плохо работают с длинными контекстами, поэтому могут пропустить важные детали в начале статьи. Однако трансформеры показывают хорошие результаты в подобных задачах. Помимо высококачественных генераций, они имеют и другие свойства, такие как эффективное распараллеливание обучения, масштабируемость и передача знаний, что делает их популярными и подходящими для выполнения множества задач. В основе поведения таких моделей лежит механизм,

¹ Фолдинг белка — процесс спонтанного сворачивания полипептидной цепи в уникальную естественную пространственную структуру. Механизм сворачивания белков до конца не изучен. Экспериментальное определение трехмерной структуры белка часто очень сложно и дорого, однако аминокислотная последовательность белка обычно известна. Поэтому ученые пытаются использовать различные методы (в том числе нейросети), чтобы предсказать пространственную структуру белка из его аминокислотной последовательности. — Пер.

называемый *самовниманием* (**self-attention**), который позволяет взвешивать важность каждого слова в контексте всей последовательности (предложения).

Чтобы помочь вам сформировать понимание, как работают LM, по мере прочтения мы будем использовать примеры кода, которые будут демонстрировать, как можно взаимодействовать с существующими моделями.

Языковая модель в действии

В этом подразделе мы рассмотрим существующую (предварительно обученную) модель-трансформер, чтобы получить общее представление, как они работают. За последние годы различные компании, исследовательские лаборатории и открытые сообщества выпустили тысячи открытых моделей, которые вы можете свободно использовать.

Модель, которую мы рассмотрим, является небольшой в сравнении с другими существующими экземплярами и может быть запущена непосредственно на вашем оборудовании. Но учтите, что те же принципы применимы к более крупным и мощным моделям, которые были выпущены с тех пор. Вот несколько хороших примеров:

◆ GPT-2 (137M)

Эта модель попала в заголовки новостей в 2019 году благодаря своим (на тот момент) впечатляющим возможностям генерации текста. Несмотря на то что она имеет небольшой размер и является нестандартной по сегодняшним меркам, GPT-2 хорошо демонстрирует, как в целом работают LM.

◆ Qwen2 (494M)

Это модель компании Alibaba. Она относится к семейству моделей Qwen, которое также включает различные модели, имеющие более 500 миллионов (а некоторые более 100 миллиардов) параметров.

◆ SmolLM (135M)

Это модель компании Hugging Face, обученная на данных очень высокого качества. Ее авторы также выпустили вариации с 135 миллионами, 360 миллионами и 1,7 миллиарда параметров.

Глава 6 предоставит больше информации о выборе подходящей модели для вашего сценария использования.

Токенизация текста

Рассмотрим генерацию текста на основе начальных входных данных. Например, дана фраза «it was a dark and stormy», и нам нужно ее продолжить. Модели не могут получать текст напрямую в качестве входных данных. Чтобы ввести текст в модель, мы должны сначала найти способ превратить последовательность символов в числа. Этот процесс называется **токенизацией** (**Tokenizing**), и он является важным шагом в любом конвейере NLP.

Самое простое решение — разбить текст на отдельные символы и присвоить каждому уникальный числовой идентификатор (рис. 2.1). Такая схема может быть полезна для языков, вроде китайского, где каждый символ несет в себе много информации. Но, например, в английском языке словарь токенов будет небольшим, и в выводе мы получим неизвестные токены (символы, которые не были найдены во время обучения). Данный метод требует достаточного количества токенов для представления строки, что плохо сказывается на производительности и стирает часть структуры текста, а следовательно, и его смысл. Это является недостатком данного метода в плане его *точности*. Каждый символ несет мало информации, что затрудняет изучение базовой структуры текста.



Рис. 2.1. Идентификаторы соответствуют положению букв в алфавите: каждая буква имеет свой собственный идентификатор, причем его значение одинаково для всех экземпляров одной и той же буквы

Существует и другой подход — разбить текст на отдельные слова (рис. 2.2). Несмотря на то что это позволяет вместить больше смысла в один токен, данный метод также имеет свои недостатки. Поскольку мы имеем дело с большим количеством неизвестных слов (например, опечатки или сленг), необходимо учитывать разные формы одного и того же слова (например, «бежать», «бежит» и «бегущий»). В конечном итоге мы можем получить очень большой словарь, который будет превышать полмиллиона слов для таких языков, как английский.



Рис. 2.2. Одно и то же слово всегда имеет один и тот же идентификатор

Современные стратегии токенизации балансируют между этими двумя крайностями, разбивая текст на *подслова*, которые отражают как его структуру, так и значение, при этом сохраняя возможность обрабатывать неизвестные слова и различные формы одного и того же слова (рис. 2.3). Символам, которые обычно встречаются вместе (например, наиболее часто встречающееся слово), можно назначить один токен, представляющий целое слово или группу слов. Длинные и сложные слова или их варианты с множеством склонений можно разбить на несколько токенов, каждый из которых будет представлять значимую их часть.



Рис. 2.3. В этом примере слово «llama» разделено на два токена, поскольку оно, вероятно, не является распространенным среди данных, использовавшихся при создании токенизатора

Не существует единого наилучшего метода пометить слова в предложении. Каждая языковая модель имеет собственный токенизатор. Различие между ними заключается в количестве поддерживаемых токенов и стратегии токенизации. Например, токенизатор GPT-2 в среднем выдает 1,3 токена на слово.

Рассмотрим, как токенизатор модели Qwen обрабатывает предложение. Для начала подключим библиотеку `transformers`, чтобы его загрузить. Затем пропустим входной текст (промпт) через токенизатор, чтобы закодировать строку в числа (токены). Также в целях демонстрации мы будем использовать метод `decode()`, чтобы преобразовать каждый идентификатор обратно в соответствующий ему токен:

```
from transformers import AutoTokenizer

# Используйте идентификатор модели, которую хотите применить
# GPT-2 "openai-community/gpt2"
# Qwen "Qwen/Qwen2-0.5B"
# SmolLM "HuggingFaceTB/SmolLM-135M"

prompt = "It was a dark and stormy"
tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen2-0.5B")
input_ids = tokenizer(prompt).input_ids
input_ids

[2132, 572, 264, 6319, 323, 13458, 88]

for t in input_ids:
    print(t, "\t:", tokenizer.decode(t))

# Вывод
2132 : It
572 : was
264 : a
6319 : dark
323 : and
13458 : storm
88 : y
```

Как показано в выводе, токенизатор разбил входную строку на ряд токенов и назначил каждому из них уникальный идентификатор. Большинство слов представлены одним токеном, но в случае с Qwen и GPT-2 слово «stormy» разбито на два токена: один для «storm» (включая пробел перед словом) и один для суффикса «y». Это позволяет модели распознать, что «stormy» связано с «storm» и что суффикс «y» часто используется для превращения существительных в прилагательные. Однако токенизатор SmolLM не разбивает ни одно из слов в этом конкретном предложении. Каждая модель обычно сопряжена со своим собственным токенизатором, поэтому всегда используйте «правильный» токенизатор. Все три модели, рассматриваемые в этом подразделе, имеют словари, которые насчитывают от 50 000 до 150 000 токенов, что позволяет им представлять практически любой входной текст.



Несмотря на то что мы обычно говорим об обучении токенизаторов, это не имеет ничего общего с обучением модели. Последнее является *стохастическим* (недетерминированным) явлением по своей природе, тогда как при обучении токенизаторов используется статистический процесс, который определяет, какие под слова лучше всего выбрать для определенного набора данных. То, как выбирать под слова, является проектным решением алгоритма токенизации. Поэтому обучение токенизаторов является детерминированным. Мы не будем погружаться в различные стратегии токенизации, но один из самых популярных подходов к разбиению слов на под слова — это *кодирование байтовых пар на уровне байтов* (**Byte-level Byte-Pair Encoding, BPE**), используемое в GPT-2, WordPiece и SentencePiece.

Прогнозирование вероятностей

GPT-2, Qwen и SmolLM обучены как *каузальные языковые модели* (**Causal Language Model**), также известные как *авторегрессивные*. Это означает, что они предсказывают следующий токен в последовательности, учитывая предыдущие. Библиотека `transformers` имеет высокоуровневые инструменты, которые позволяют использовать подобные модели для генерации текста или быстрого выполнения других задач. Чтобы понять, как модели делают свои прогнозы, проверим их в моделировании языка. Начнем с загрузки модели.

```
from transformers import AutoModelForCausalLM

model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen2-0.5B")
```

Обратите внимание на классы `AutoTokenizer` и `AutoModelForCausalLM`. Библиотека `transformers` поддерживает сотни моделей и соответствующих им токенизаторов. Вместо того чтобы изучать имя каждого из них (а также классы непосредственно моделей), мы будем использовать автоматические классы `AutoTokenizer` и `AutoModelFor*`. Нам нужно лишь указать, какую задачу необходимо решить, например классификация (`AutoModelForSequence Classification`) или обнаружение объектов (`AutoModelForObjectDetection`). В случае Qwen2 мы будем использовать класс, соответствующий задаче казуального моделирования языка. Трансформер выберет подходящий класс по умолчанию на основе конфигурации модели, например `Qwen2Tokenizer` и `Qwen2ForCausalLM`.

Если пропустить токенизированное предложение из предыдущего подраздела через модель, мы получим результат с 151 936 значениями для каждого токена во входной строке.

```
# Мы снова токенизируем, но на этот раз указываем токенизатору,
# чтобы он возвращал тензор PyTorch, который ожидает модель,
# а не список целых чисел
input_ids = tokenizer(prompt, return_tensors="pt").input_ids

outputs = model(input_ids)
outputs.logits.shape # Вывод для каждого входного токена

torch.Size([1, 7, 151936])
```

Первое измерение (**Dimension**) в выходных данных — это количество партий (1, потому что мы только что прогнали одну последовательность через модель). Второе измерение — длина входной последовательности или количество токенов в ней (в данном случае 7). Третье измерение — размер словаря. Мы получили список из ~150 000 чисел для каждого токена в исходной последовательности. Это необработанные выходные данные модели. Они называются *логитами* (**Logit**) и соответствуют токенам в словаре. Для каждого входного токена модель предсказывает вероятность, с которой каждый токен в словаре может продолжить последовательность до конкретной точки. В нашем примере модель предскажет логиты для «It», «It was», «It was a» и т. д. Более высокие значения логита означают, что модель считает соответствующий токен более вероятным продолжением последовательности. Таблица 2.1 показывает входные последовательности, наиболее вероятный идентификатор и соответствующий ему токен.



Логиты — это необработанные выходные данные модели (список чисел, например [0.1, 0.2, 0.01, ...]). Мы можем использовать их для выбора наиболее вероятного токена, чтобы продолжить последовательность. Однако мы также можем преобразовывать их в вероятности, что мы и сделаем дальше.

Таблица 2.1. Наиболее вероятные токены для продолжения входных последовательностей согласно модели Qwen2

Входная последовательность	Идентификатор наиболее вероятного следующего токена	Соответствующий токен
It	374	is
It was	264	a
It was a	2244	great
It was a dark	323	and
It was a dark and	13458	storm
It was a dark and storm	88	y
It was a dark and stormy	3729	(выясним следующее слово)

Сосредоточимся на логитах для всего промпта и попробуем предсказывать следующее слово последовательности. Мы можем найти индекс токена с наибольшим значением, используя метод `argmax()`.

```
final_logits = model(input_ids).logits[0, -1] # Последний набор логитов
final_logits.argmax() # Индекс токена с наибольшим значением
```

```
tensor(3729)
```

Значение 3729 соответствует токenu, который считается наиболее вероятным для строки «It was a dark and stormy». Расшифровав его идентификатор, мы можем узнать, какое слово модель считает логичным для продолжения последовательности.

```
tokenizer.decode(final_logits.argmax())
```

```
' night'
```

Таким образом, «night» — наиболее вероятный токен. И это имеет смысл, учитывая то, что мы предоставили в качестве входных данных. Модель учится обращать внимание на другие токены, используя *самовнимание*. Оно является фундаментальным строительным блоком трансформеров. Интуитивно самовнимание позволяет модели определить, какой вклад вносит каждый токен в значение последовательности.



Модели-трансформеры содержат много слоев самовнимания, каждый из которых специализируется на каком-то конкретном аспекте ввода. В отличие от эвристических систем, данные особенности изучаются прямо во время обучения, а не определяются заранее.

Далее мы выясним, какие еще токены могут быть потенциальными кандидатами. Выберем 10 лучших значений с помощью метода `topk()`:

```
import torch
```

```
top10_logits = torch.topk(final_logits, 10)
for index in top10_logits.indices:
    print(tokenizer.decode(index))
```

```
#Вывод
night
evening
day
morning
winter
afternoon
Saturday
Sunday
Friday
October
```

Нам необходимо преобразовать логиты в вероятности, чтобы лучше понять, насколько модель «уверена» в каждом прогнозе. Это можно сделать с помощью операции `softmax()`, сравнив каждое значение со всеми другими предсказаниями и нормализовав их так, чтобы все числа в сумме давали 1. В коде ниже этот метод используется для вывода десяти самых вероятных токенов и связанных с ними вероятностей в соответствии с моделью².

```
top10 = torch.topk(final_logits.softmax(dim=0), 10)
for value, index in zip(top10.values, top10.indices):
    print(f"{tokenizer.decode(index):<10} {value.item():.2%}")
```

² В коде мы используем «<» для выравнивания токена по левому краю, значение «10» для указания ширины поля и «.2%» для форматирования вероятности в проценты с двумя десятичными знаками.

```
#Вывод
night 88.71%
evening 4.30%
day 2.19%
morning 0.49%
winter 0.45%
afternoon 0.27%
Saturday 0.25%
Sunday 0.19%
Friday 0.17%
October 0.16%
```

Прежде, чем двигаться дальше, поэкспериментируйте с кодом. Можете попробовать следующие варианты:

◆ *Измените несколько слов*

Попробуйте изменить прилагательные («dark» и «stormy») во входной строке и посмотрите, как меняются прогнозы. Меняется ли предсказанное ранее слово «night»? Как меняются вероятности?

◆ *Измените входную строку*

Попробуйте использовать разные входные строки и проанализируйте, как меняются прогнозы. Согласны ли вы с результатами?

◆ *Грамматика*

Что произойдет, если вы предоставите строку, которая не является грамматически правильной? Как модель с этим справится? Посмотрите на вероятности наиболее подходящих прогнозов.

Генерация текста

Теперь, когда нам известны предсказания для следующего токена в последовательности, мы можем сгенерировать текст, многократно возвращая прогнозы обратно в модель. Для этого мы вызовем функцию `model(ids)`, сгенерируем новый идентификатор токена, добавим его в список и снова вызовем функцию. Для более удобной генерации слов авторегрессивные модели-трансформеры обладают методом `generate()`, который идеально подходит для этого. Рассмотрим пример.

```
output_ids = model.generate(input_ids, max_new_tokens=20)
decoded_text = tokenizer.decode(output_ids[0])
```

```
print("Input IDs", input_ids[0])
print("Output IDs", output_ids)
print(f"Generated text: {decoded_text}")
```

```
#Вывод
Input IDs tensor([ 2132,  572,  264,  6319,  323, 13458,  88])
Output IDs tensor([ 2132,  572,  264,  6319,  323, 13458,  88, 3729,
                   13,  576, 12884,  572,  6319,  323,  279, 9956,  572, 1246,
                   2718, 13,  576, 11174,  572, 50413, 1495,  323,  279])
```

Generated text: It was a dark and stormy night. The sky was dark and the wind was howling. The rain was pouring down and the

После запуска метод `model()` вернул список логитов для каждого токена в словаре (151 936). Затем мы рассчитали вероятности и выбрали наиболее подходящий токен. Метод `generate()` абстрагирует эту логику. Он делает несколько прямых проходов по модели, повторно предсказывает следующий токен и добавляет его к входной последовательности. Также он предоставляет идентификаторы токенов конечной последовательности, включая как входные, так и новые токены. Затем с помощью метода `tokenizer.decode()` нам достаточно преобразовать идентификаторы обратно в текст.

Существует множество возможных стратегий для генерации текста. Рассмотренный выше подход, где мы выбираем наиболее вероятный токен, называется *жадным декодированием* (рис. 2.4). Несмотря на простоту метода, иногда он может приводить к неоптимальным результатам, особенно при генерации более длинных текстовых последовательностей. Жадное декодирование может быть проблематичным, поскольку оно не учитывает общую вероятность предложения, сосредоточиваясь непосредственно на следующем слове. Например, если взять начальное слово «Sky», а в качестве последующих вариантов предположить «blue» или «rockets», данная стратегия может отдать предпочтение сочетанию «Sky blue», поскольку «blue» изначально кажется более вероятным после «Sky». Однако такой подход может упустить из виду более связную и более вероятную общую последовательность, например «Sky rockets soar». Поэтому жадное декодирование иногда приводит к менее оптимальной генерации текста.

Подход, описанный выше, позволяет выбирать лишь один токен за раз. Существуют методы, например *лучевой поиск* (рис. 2.5), которые исследуют несколько воз-

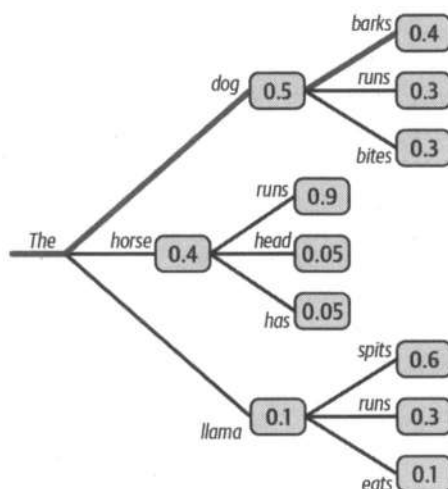


Рис. 2.4. Пример жадного декодирования: модель генерирует последовательность «The dog barks», потому что «dog» является наиболее вероятным вторым токеном (значение 0,5) после «The», а «barks» — наиболее вероятным токеном после «The dog»

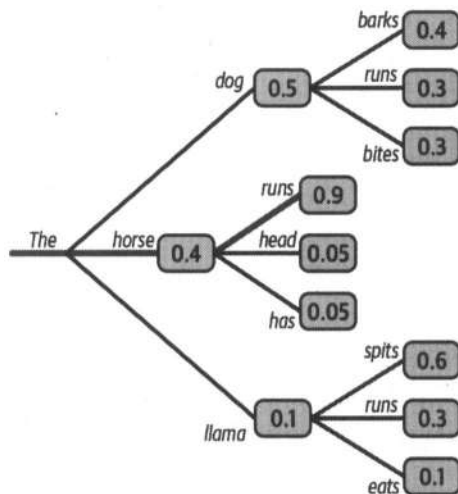


Рис. 2.5. В примере лучевого поиска модель находит более вероятную последовательность: «The dog barks» имеет общую вероятность $0,5 \times 0,4 = 0,2$, а «The horse runs» — $0,4 \times 0,9 = 0,36$

можных продолжений последовательности и возвращают наиболее вероятный вариант. Данная стратегия (пример кода находится ниже) сохраняет самые подходящие номера предполагаемых прогнозов (`num_beams`) во время генерации и выбирает наиболее вероятную из них.

```

beam_output = model.generate(
    input_ids,
    num_beams=5,
    max_new_tokens=30,
)

print(tokenizer.decode(beam_output[0]))

```

Вывод

It was a dark and stormy night. The wind was howling, and the rain was pouring down.
The sky was dark and gloomy, and the air was filled with the

В зависимости от модели вы можете получить несколько похожих результатов. Есть параметры (хотя они и используются нечасто), которые мы можем контролировать, чтобы генерации были менее повторяющимися. Рассмотрим два параметра:

◆ `repeat_penalty`

Устанавливает, насколько сильно нужно «штрафовать» модель за уже сгенерированные токены, чтобы избежать повторения. Хорошее значение по умолчанию 1,2.

◆ `bad_words_ids`

Содержит список токенов, которые не следует генерировать (например, чтобы избежать оскорбительных слов).

Рассмотрим, чего можно добиться, наказывая за повторение.

```
beam_output = model.generate(
    input_ids,
    num_beams=5,
    repetition_penalty=2.0,
    max_new_tokens=38,
)

print(tokenizer.decode(beam_output[0]))

# Вывод
```

```
It was a dark and stormy night. The sky was filled with thunder and lightning,
and the wind howled in the distance. It was raining cats and dogs,
and the streets were covered in puddles of water.
```

Как часто бывает в машинном обучении, выбор стратегии генерации зависит от разных обстоятельств. Лучевой поиск, например, хорошо работает, когда желаемая длина текста в некоторой степени предсказуема. Он дает хорошие результаты при резюмировании (обобщении) или переводе текста, но не при открытой его генерации, где длина вывода может сильно варьироваться, что приводит к повторению. Несмотря на то что мы способны запретить модели избегать повторений, это может привести к ухудшению ее работы. Также обратите внимание, что лучевой поиск работает медленнее жадного декодирования, поскольку ему необходимо выполнять вывод для нескольких лучей одновременно, что может стать проблемой в больших моделях.

Когда мы используем жадное декодирование и лучевой поиск, мы заставляем модель генерировать текст с распределением последующих слов с высокой вероятностью, как показано на рис. 2.6³. Интересно то, что высококачественный естественный язык не соответствует подобному распределению. Человеческий текст, как правило, менее предсказуем. Авторы статьи «*The Curious Case of Neural Text Degeneration*» предполагают, что человеческий язык «не одобряет» предсказуемые слова, поскольку люди оптимизируют свою речь, чтобы не утверждать очевидное. В статье предлагается метод, называемый *выборкой ядра* (Nucleus sampling или Top-p sampling).

Прежде чем обсуждать этот метод, сначала разберемся с обычной *выборкой* (Sampling). Мы определяем следующее слово с помощью выборки из распределения вероятностей предполагаемых токенов. Это означает, что данный процесс генерации не является детерминированным. Например, следующими возможными токенами в нашем примере будут «night» (60%), «day» (35%) и «apple» (5%), и вместо того, чтобы взять «night» (что соответствует жадному декодированию), мы выполним выборку из распределения. Другими словами, с вероятностью в 5% будет выбран токен «apple», даже если он имеет низкую вероятность и приводит к бессмыслен-

³ В статистике *распределение* — это способ описать, как распределены значения переменной (как часто они встречаются).

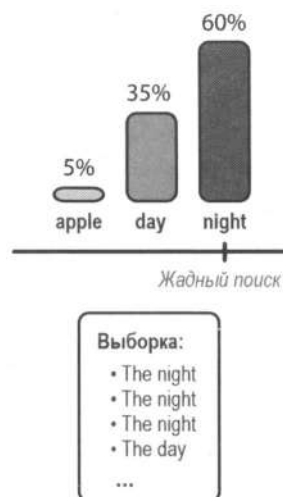


Рис. 2.6. Жадное декодирование всегда будет выбирать в качестве следующего наиболее вероятный токен, в то время как выборка ядра определяет следующий токен с помощью выборки из распределения вероятностей

ному выводу. Выборка позволяет избежать создания повторяющегося текста, что приводит к более разнообразным генерациям.

Выборка в трансформерах выполняется с помощью параметра `do_sample`:

```
from transformers import set_seed

#Установка начального значения гарантирует, что мы будем получать
#одни и те же результаты при каждом запуске этого кода
set_seed(70)

sampling_output = model.generate(
    input_ids,
    do_sample=True,
    max_new_tokens=34,
    top_k=0, # We'll come back to this parameter
)

print(tokenizer.decode(sampling_output[0]))

#Вывод
It was a dark and stormy night. Kevin said he was going to stay up all
night, staring at the cloudless stars, wondering, what if I lost my dream.
He'd been teasing her about
```

Мы можем манипулировать распределением вероятностей до того, как выполним из него выборку, делая его более «выпуклым» или «плоским» с помощью параметра *температуры* — `temperature`. Если его значение выше 1, то случайность распределения увеличивается и его можно использовать, чтобы поощрить генерацию менее вероятных токенов. Значение от 0 до 1 уменьшает случайность, увеличивая

вероятность наиболее предположительных токенов, и позволяет избежать предсказаний, которые могут быть слишком неожиданными. Значение, равное 0, перемещает внимание на наиболее предполагаемый токен, что эквивалентно жадному декодированию (рис. 2.7).

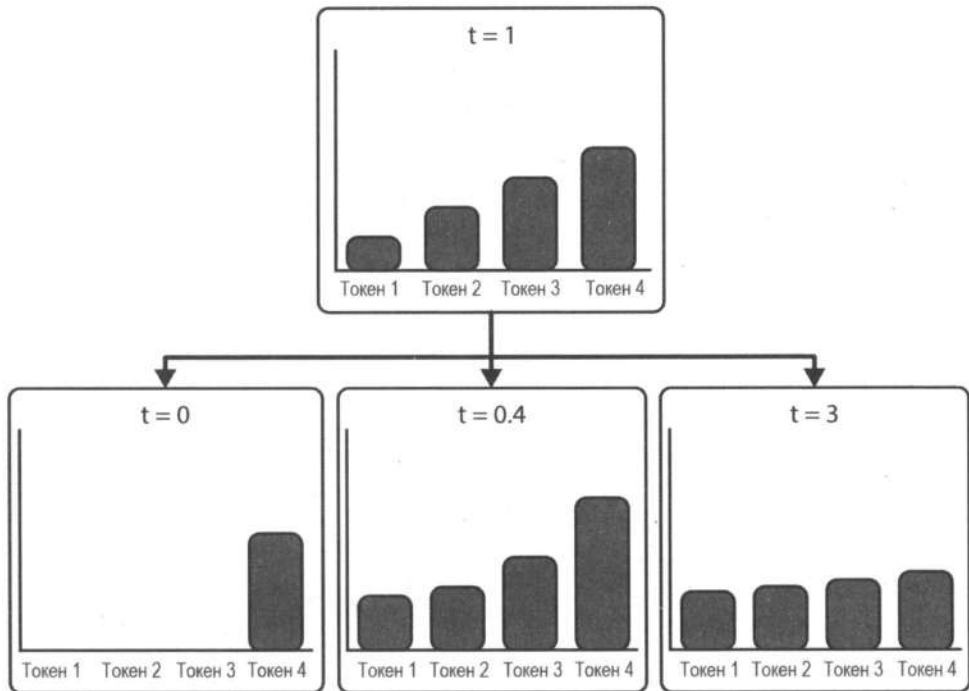


Рис. 2.7. Влияние параметра `temperature` на распределение вероятности токена

Сравните влияние параметра `temperature` на сгенерированный текст в примере ниже.

```
sampling_output = model.generate(
    input_ids,
    do_sample=True,
    temperature=0.4,
    max_new_tokens=40,
    top_k=0,
)

print(tokenizer.decode(sampling_output[0]))
```

#Вывод

It was a dark and stormy night in 1878. The only light was the moon,
and the only sound was the distant roar of the thunder. The only thing
that could be heard was the sound of the storm

```
sampling_output = model.generate(
    input_ids,
```

```

do_sample=True,
temperature=0.001,
max_new_tokens=40,
top_k=0,
)

print(tokenizer.decode(sampling_output[0]))

```

#Вывод

It was a dark and stormy night. The sky was dark and the wind was howling.
The rain was pouring down and the lightning was flashing. The sky was dark
and the wind was howling. The rain was pouring down

```

sampling_output = model.generate(
    input_ids,
    do_sample=True,
    temperature=3.0,
    max_new_tokens=40,
    top_k=0,
)

print(tokenizer.decode(sampling_output[0]))

```

#Вывод

It was a dark and stormy 清晨一步

女人们都 BL 咻任何时候 مت زب attendees*sinruitment لاuresindi
ambassadors eventData 原来是 exalENCE hemisphere worldsw.Anyar 久◆ Sous dapat
HVişisdia.inventory emptiedfuncpping {\Sex

Первый текст является наиболее последовательным. Второй вариант, который использует очень низкое значение `temperature`, имеет повторы (он похож на жадное декодирование). Третий вариант с чрезвычайно высоким значением `temperature` вовсе дает бессмысленный текст.

Вероятно, вы могли заметить параметр `top_k`. Он соответствует методу, который называется *выборка Top-K (Top-K Sampling)*. Это простой подход, в котором рассматриваются только K наиболее вероятных последующих токенов. Например, используя значение `top_k = 5`, метод сначала отфильтрует пять наиболее вероятных токенов и перераспределит вероятности так, чтобы они в сумме давали 1.

```

sampling_output = model.generate(
    input_ids,
    do_sample=True,
    max_new_tokens=40,
    top_k=5,
)

print(tokenizer.decode(sampling_output[0]))

```

#Вывод

It was a dark and stormy night in New York. The city was on the brink
of a violent storm. The sky above was painted with a mix of bright red and orange.
It was a sign, but the storm had arrived

Результат стал лучше, но есть проблема. Она заключается в том, что количество соответствующих кандидатов (наиболее вероятных токенов) на практике может сильно различаться. Если мы определим $\text{top_k} = 5$, некоторые распределения все равно будут включать токены с очень низкой вероятностью, в то время как другие распределения будут состоять только из токенов с высокой вероятностью.

Последняя стратегия генерации, которую мы рассмотрим, — это *выборка Тор-р* (**Top-p sampling**, также известная как *выборка ядра*). Вместо того чтобы выбирать K слов с самой высокой вероятностью, мы будем использовать все наиболее подходящие слова, кумулятивная вероятность которых превышает заданное значение. Если мы используем $\text{top_p} = 0,94$, сначала будут отобраны наиболее подходящие слова, которые в совокупности имеют вероятность 0,94 или выше (рис. 2.8). Затем вероятность перераспределяется и среди них выполняется обычная выборка. Проверим это на практике.

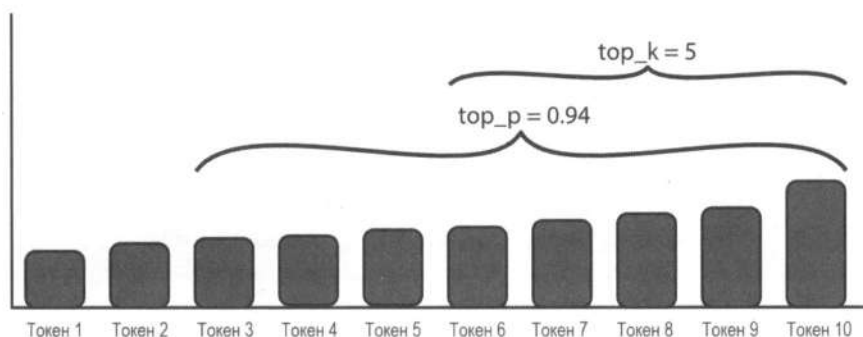


Рис. 2.8. Влияние параметров top_k и top_p на распределение вероятности токенов:
при $\text{top_k} = 5$ рассматриваются только пять наиболее вероятных токенов;
при $\text{top_p} = 0,94$ в рассмотрение включаются все токены,
кумулятивная вероятность которых достигает 0,94

В трансформерах мы можем использовать выборку Тор-р и менять вероятность с помощью параметра top_p .

```
sampling_output = model.generate(
    input_ids,
    do_sample=True,
    max_new_tokens=40,
    top_p=0.94,
    top_k=0,
)

print(tokenizer.decode(sampling_output[0]))
```

#Вывод

It was a dark and stormy night in the skies of Morrowind, and a particularly ruthless fighter had decided that these careless tourists at Carrabine should be dealt with with maximum cruelty. The chief of this important operation was appointed by

На практике обычно используются методы как *Top-K*, так и *Top-p*. Их можно даже комбинировать, чтобы отфильтровывать слова с низкой вероятностью, но при этом они будут иметь больший контроль над генерацией. Проблема со стохастическими методами заключается в том, что сгенерированный текст не обязательно бывает связным.

К текущему моменту мы рассмотрели три метода генерации: жадное декодирование, лучевой поиск и выборку (с параметрами `temperature`, `top_k` и `top_p`, обеспечивающими дополнительный контроль). Если вас не удивляют текущие результаты, учтите, что это модели с несколькими сотнями миллионов параметров. Тот факт, что они могут генерировать связный текст, уже впечатляет. Новые модели имеют миллиарды или даже сотни миллиардов параметров и обучаются с использованием более качественных данных, поэтому вы можете переключиться на модель больше и современнее, что приведет к еще более интересным результатам. В Интернете вы можете поискать интерактивные онлайн-демонстрации, чтобы наглядно увидеть, как параметры `temperature`, `top_k` и `top_p` влияют на распределение генераций.

Если вы хотите поэкспериментировать, вот несколько советов:

- ◆ Попробуйте различные значения параметров. Как увеличение количества лучей влияет на качество генерации? Что произойдет, если уменьшить или увеличить значение `top_p`?
- ◆ Один из подходов к уменьшению повторений в лучевом поиске — «штрафы» за *n-граммы* (последовательности из *n* слов). Их можно настроить с помощью `no_repeat_ngram_size`, что позволит избежать повторения одной и той же *n*-граммы. Например, если вы используете `no_repeat_ngram_size = 4`, генерация никогда не будет содержать именно четыре последовательных слова.
- ◆ *Top-K* может привести к отбрасыванию высококачественных токенов, а *Top-p* может привести к включению токенов с низкой вероятностью. Для более динамичного подхода вы можете использовать подход *Min-P*, который умножает параметр `min_p` на вероятность верхнего токена, а затем включает только токены выше этого процента. Другими словами, метод определяет динамический порог на основе вероятности верхнего токена.
- ◆ Попробуйте более новый метод — *контрастный поиск*. Он может генерировать длинный, связный вывод, избегая повторений. Это достигается с помощью учета как вероятностей, предсказанных моделью, так и сходства с контекстом. Его можно контролировать с помощью параметров `penalty_alpha` и `top_k`.

Генерация — это активная область исследований, и в новых работах появляются различные предложения, например более сложная фильтрация. Мы кратко обсудим их в последней главе. Не существует правила, которое работало бы для всех моделей, поэтому всегда важно экспериментировать с различными методами.

Генерализация с нулевым выстрелом (Zero-shot)

Генерация текста — превосходный способ применить трансформеры на практике, но написание фейковых статей о единорогах — не та причина, по которой они так популярны⁴. Чтобы хорошо предсказывать следующий токен, эти модели должны знать достаточно много о мире. Мы можем воспользоваться этим при выполнении различных задач. Например, вместо того, чтобы обучать модель, предназначенную для перевода, можно использовать достаточно мощную языковую модель. Например:

#Перевод предложения с английского на французский

Translate the following sentence from English to French:

Input: The cat sat on the mat.

Translation:

Я набрал этот пример в Copilot, и он любезно предложил вариант «Le chat était assis sur le tapis» в качестве перевода. Это прекрасная иллюстрация того, как модель может выполнять задачи, для которых она не была обучена. Чем мощнее модель, тем больше задач она может выполнять без дополнительного обучения. Такая гибкость делает трансформеры довольно популярными в последние годы.

Чтобы проверить это на практике, попробуем использовать Qwen в задаче классификации. Мы будем определять обзоры фильмов как положительные или отрицательные — классическая задача в области NLP. Для этого применим подход *нулевого выстрела* (Zero-shot), чтобы сделать пример интереснее. Он подразумевает, что мы не станем предоставлять модели никаких помеченных данных (примеров). Вместо этого мы предоставим ей промпт с текстом обзора и попросим определить настроение в нем.

Использовать генеративную модель в качестве классификатора можно несколькими способами. Для начала вставим обзор фильма в шаблон промпта, который предоставит контекст для модели. Промпт дает указание просто вернуть настроение обзора (вывод можно также ограничить только как положительный или отрицательный). Альтернативный способ, особенно полезный в небольших моделях, вроде GPT-2 или Qwen, заключается в том, чтобы узнать прогноз для следующего токена и определить, какое значение имеет большую вероятность: положительное или отрицательное. Воспользуемся этим подходом и найдем идентификаторы, соответствующие токенам.

#Проверяем идентификаторы токенов на наличие слов « positive» и « negative»

#(обратите внимание на пробел перед словами)

tokenizer.encode(" positive"), tokenizer.encode(" negative")

#Вывод

{[6785], [8225]}

⁴ Первым примером, посвященном релизу GPT-2, была известная выдуманная новость о единорогах.

Получив идентификаторы, мы можем выполнить вывод с помощью модели и сгенерировать метку на основе вероятностей.

```
def score(review):
    """
    Эта функция предсказывает, является ли отзыв положительным или отрицательным,
    используя несколько умных промптов. Она просматривает логиты для токенов
    « positive» и « negative» и возвращает метку с наивысшей оценкой.
    """

    prompt = f"""Question: Is the following review positive or
        negative about the movie?
        Review: {review} Answer:"""

    input_ids = tokenizer(prompt, return_tensors="pt").input_ids ❶
    final_logits = model(input_ids).logits[0, -1] ❷
    if final_logits[6785] > final_logits[8225]: ❸
        print("Positive")
    else:
        print("Negative")
```

Немного поясним код:

- ❶ Токенизируем запрос.
- ❷ Получаем логиты для каждого токена в словаре. Обратите внимание, что мы используем `model()`, а не `model.generate()`, т. к. первый метод возвращает логиты для каждого токена в словаре, в то время как второй возвращает логиты только для выбранного токена.
- ❸ Проверяем, какой логит выше, для положительного токена или для отрицательного.

Данный классификатор с нулевым выстрелом можно опробовать на нескольких фейковых отзывах, чтобы оценить, как он работает.

```
score("This movie was terrible!")
```

```
#Вывод
```

```
Negative
```

```
score("That movie was great!")
```

```
#Вывод
```

```
Positive
```

```
score("A complex yet wonderful film about the depravity of man") #Ошибка
```

```
#Вывод
```

```
Negative
```

В репозитории GitHub для этой книги вы найдете набор помеченных данных с обзорами, а также код, который позволяет оценить точность данного подхода. Можете ли вы настроить шаблон промпта, чтобы улучшить производительность модели?

Можете ли вы придумать другие задачи, которые можно было бы выполнить с использованием аналогичного подхода?

Возможности нулевого выстрела в последних релизах некоторых моделей стали переломным моментом. По мере улучшения модели могут выполнять больше задач прямо «из коробки», что делает их более доступными и простыми в использовании, а также снижает потребность в специализированных моделях для каждой конкретной задачи.

Генерализация с несколькими выстрелами (Few-shot)

Нулевой выстрел (где модель обучается без примеров) — не единственный способ заставить мощные языковые модели выполнять произвольные задачи. Также существует подход, который известен как *несколько выстрелов* (**Few-shot**). В нем мы предоставляем модели несколько готовых примеров, а затем просим ее дать аналогичные ответы. Вместо того чтобы обучать модель, мы показываем несколько правильных ответов, чтобы повлиять на генерацию, увеличивая тем самым вероятность, что текст продолжения будет следовать той же структуре, что и наш промпт. Попробуем выполнить это на практике. Помимо готовых примеров, мы также предоставим модели краткое описание, что она должна делать (например, «Перевести с английского на испанский»). Это поможет реализовать более качественные генерации.

```
prompt = """\n
#Переведи с английского на испанский
Translate English to Spanish:

English: I do not speak Spanish.
Spanish: No hablo español.

English: See you later!
Spanish: ;Hasta luego!

English: Where is a good restaurant?
Spanish: ¿Dónde hay un buen restaurante?

English: What rooms do you have available?
Spanish: ¿Qué habitaciones tiene disponibles?

English: I like soccer
Spanish: ""
inputs = tokenizer(prompt, return_tensors="pt").input_ids
output = model.generate(
    inputs,
    max_new_tokens=10,
)

print(tokenizer.decode(output[0]))
```

#Вывод

Translate English to Spanish:

English: I do not speak Spanish.

Spanish: No hablo español.

English: See you later!

Spanish: ¡Hasta luego!

English: Where is a good restaurant?

Spanish: ¿Dónde hay un buen restaurante?

English: What rooms do you have available?

Spanish: ¿Qué habitaciones tiene disponibles?

English: I like soccer

Spanish: Me gusta el fútbol

English:

В этом коде мы формулируем задачу, которую хотим выполнить, и приводим четыре примера, чтобы задать контекст для модели. Таким образом, данная задача по генерализации состоит из четырех шагов. Затем мы просим модель сгенерировать больше текста, чтобы он следовал шаблону, и предоставить запрошенный перевод.

Вот несколько дополнительных идей для самостоятельного изучения:

- ◆ Будет ли это работать с меньшим количеством примеров?
- ◆ Будет ли это работать без описания задачи?
- ◆ Будет ли это работать с другими задачами?
- ◆ Какие результаты могут дать GPT-2 и SmolLM в данных условиях?



Модель GPT-2, учитывая ее размер и процесс обучения, не очень хороша в задачах с несколькими выстрелами, и еще хуже показывает себя в задачах генерализации с нулевым выстрелом. Тогда логично задаться вопросом, как мы смогли использовать ее для классификации в предыдущем подразделе? Мы немного схитрили: мы не учитывали текст, сгенерированный моделью, а просто проверяли, была ли вероятность для «positive» больше, чем вероятность для «negative». Понимание того, как модели работают «под капотом», поможет создать мощные приложения даже с небольшими экземплярами. Не бойтесь исследовать.

GPT-2 и Qwen 0.5B — это примеры базовых моделей. Некоторые из них изначально имеют возможность решения задач с помощью методов *zero-shot* и *few-shot*, которые мы можем использовать при выводе. Существует еще один подход, который заключается в *тонкой настройке (Fine-tuning)* модели. Мы берем базовую модель и продолжаем обучать ее немного дольше на данных, специфичных для конкретной задачи. При этом нам могут быть не всегда нужны экстремальные возможности генерализации, демонстрируемые самыми мощными моделями в мире. Если вы

хотите решить конкретную задачу, дешевле и лучше будет настроить и развернуть меньшую специализированную модель.

Также важно отметить, что базовые модели *не являются разговорными*. Вы можете написать хороший промпт, который поможет создать целого чат-бота с помощью базовой модели, но зачастую гораздо более удобно настроить саму базовую модель с помощью разговорных данных, тем самым улучшив ее диалоговые возможности. Именно это мы и сделаем в *главе 6*. Последние релизы различных LLM, как правило, включают не только базовую модель, но и официальную версию с разговорными возможностями. В случае с Qwen — это модель Qwen2-0.5B-Instruct.

Блок трансформера

Теперь мы можем перейти к архитектурной схеме моделей генерации языка на основе трансформеров (рис. 2.9).

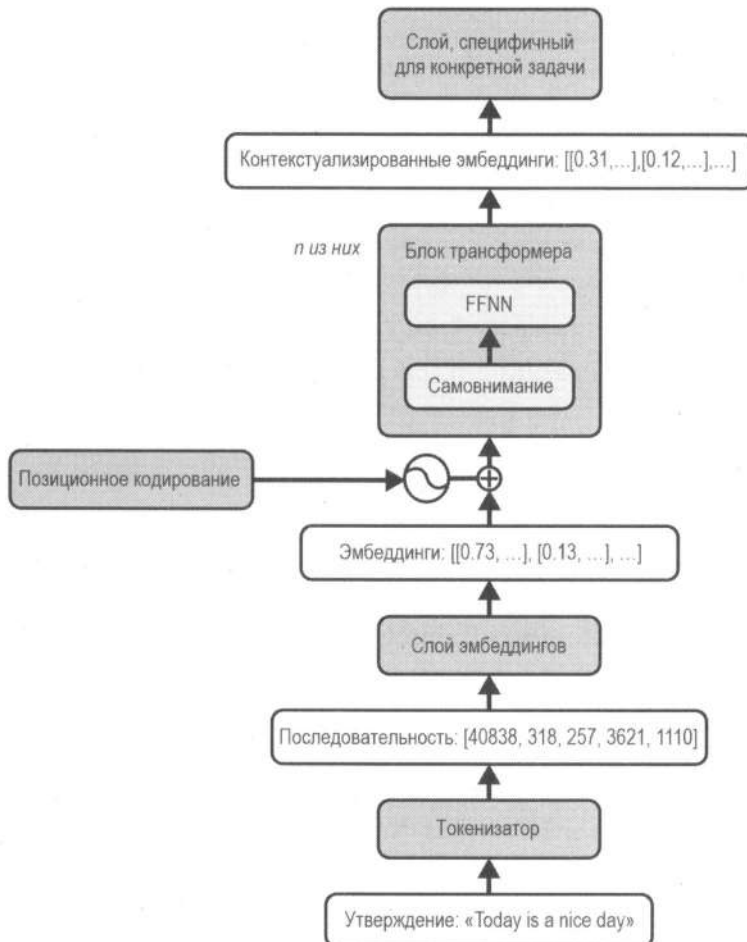


Рис. 2.9. Архитектура языковой модели на основе трансформера

В число задействованных компонентов высокого уровня входят следующие:

◆ *Токенизация*

Входной текст разбивается на отдельные токены. Каждый из них имеет соответствующий идентификатор, используемый, чтобы индексировать эмбединги токенов.

◆ *Эмбединги входных токенов*

Токены представлены в виде векторов, называемых эмбедингами (**Embedding**). Они служат числовыми представлениями, которые фиксируют основную информацию каждого токена. Векторы можно представить в виде длинного списка чисел, где каждое число соответствует определенному аспекту значения токена. Во время обучения модель учится сопоставлять каждый токен с соответствующим ему эмбедингом. Эмбединг всегда будет одинаковым для конкретного токена, независимо от его положения во входной последовательности.

◆ *Позиционное кодирование*

Модель-трансформер не имеет понятия о порядке, поэтому нам нужно дополнить эмбединги токенов необходимой информацией. Это делается через добавление позиционного кодирования к эмбедингам токенов. Оно представляет собой набор векторов, которые кодируют положение каждого токена во входной последовательности. Это позволяет модели различать токены, основываясь на их положении, что может быть полезно, поскольку один и тот же токен, появляющийся в разных местах, может иметь разные значения.

◆ *Блоки трансформеров*

Блок трансформера — это ядро данной модели. Сила трансформеров заключается в возможности наложения нескольких блоков друг на друга, что позволяет модели изучать более сложные и абстрактные отношения между входными токенами. Блок состоит из двух основных компонентов:

• *Механизм самовнимания*

Он позволяет модели взвешивать важность каждого токена в контексте всей последовательности и помогает понимать отношения между токенами во входных данных. Механизм самовнимания является ключом к способности трансформера обрабатывать долгосрочные зависимости и сложные отношения между словами, а также помогает генерировать связный и соответствующий контексту текст.

• *FFNN*

Выходной сигнал самовнимания передается через нейронную сеть прямого распространения (**Feed-Forward Neural Network, FFNN**), которая дополнительно уточняет представление входной последовательности.

◆ *Контекстуализированные эмбединги*

Вывод блока трансформера представляет собой набор контекстных эмбедингов, которые фиксируют отношения между токенами во входной последователь-

ности. В отличие от входных эмбеддингов, которые фиксированы для каждого токена, контекстные эмбеддинги обновляются на каждом уровне модели, основываясь на отношениях между токенами. Эмбеддинги фиксируют богатую и сложную семантическую информацию о токене в контексте, в котором он появляется.

♦ Прогноз

Этот слой обрабатывает финальное представление в окончательный вывод в зависимости от задачи. В случае генерации текста подразумевается наличие линейного слоя, сопоставляющим контекстные эмбеддинги со словарем, за которым следует операция `softmax` для прогнозирования следующего токена в последовательности.

Мы рассмотрели упрощенное представление архитектуры трансформера. Погружение во внутренние механизмы того, как работает самовнимание, или в структуру блока трансформера выходит за рамки этой книги. Однако знание архитектуры на высоком уровне может быть полезным для понимания, как работают эти модели в целом и как их можно применять к различным задачам. Подобная архитектура позволяет трансформерам достигать беспрецедентной производительности в различных задачах и областях.

Генеалогия модели-трансформера

В начале главы мы экспериментировали с Qwen касательно авторегрессивной генерации текста. Данная модель является примером трансформера на основе *декодера* (**Decoder**). Она имеет один стек блоков трансформера, которые обрабатывают входную последовательность. На сегодняшний день такой подход популярен, но за последние годы были разработаны и другие архитектуры. В этом подразделе представлена краткая генеалогия моделей трансформеров.

Задачи «последовательность-последовательность»

В оригинальной статье о трансформерах (*«Attention Is All You Need»*) использовалась архитектура *энкодер-декодер* (**Encoder-Decoder**), которая представлена на рис. 2.10. Несмотря на то что она была популярна до 2023 года, в большинстве исследовательских лабораторий ее заменили *декодеры*.

Статья была сосредоточена на машинном переводе в качестве примера для задачи типа *последовательность-последовательность* (**sequence-to-sequence**). Наилучшие результаты в то время были достигнуты с помощью рекуррентных нейросетей (RNN), которые использовали различные методы, такие как *долгая краткосрочная память* (**Long Short-term Memory, LSTM**) или *управляемые рекуррентные блоки* (**Gated Recurrent Unit, GRU**). Статья продемонстрировала лучшие результаты, сосредоточившись исключительно на методе *внимания*, и показала, что масштабируемость и обучение были намного проще. Такие важные факторы, как отличная производительность, стабильное обучение и легкая масштабируемость, стали причиной того, что популярность трансформеров взлетела, и они были адаптированы

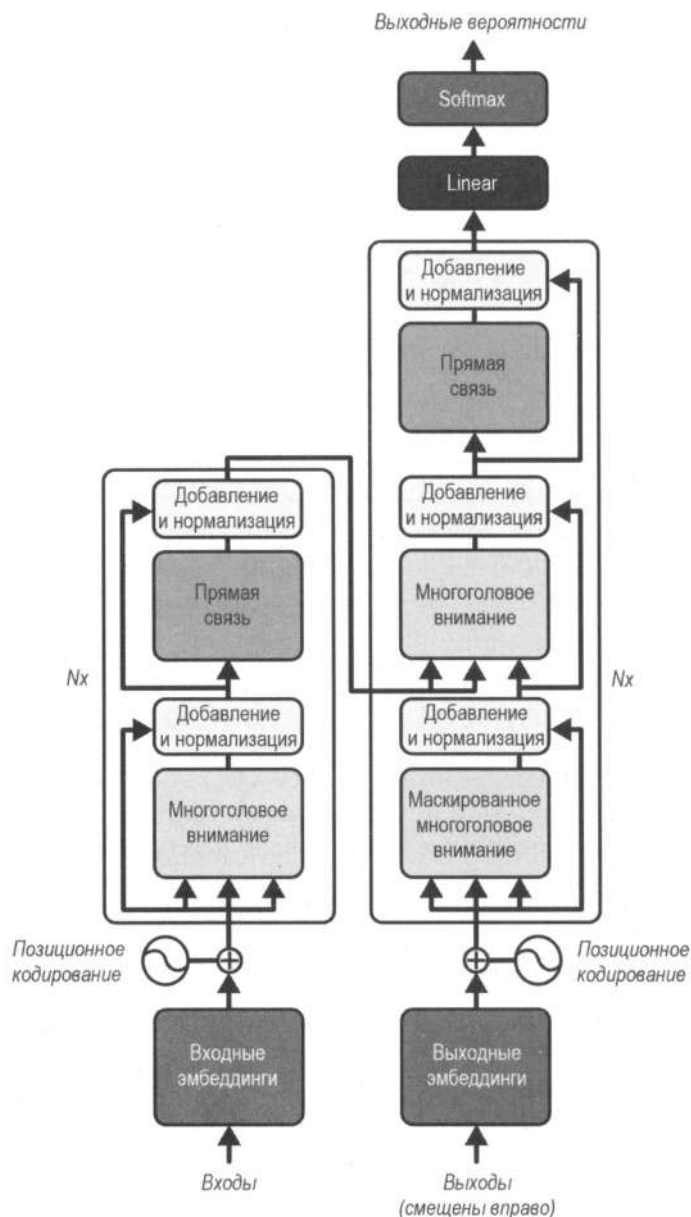


Рис. 2.10. Архитектура энкодер-декодер

для выполнения множества задач (о чем более подробно мы расскажем в следующем подразделе).

В моделях энкодер-декодер (как и в оригинальной модели из статьи) один стек блоков трансформера, называемый **энкодером (Encoder)**, перерабатывает входную последовательность в набор расширенных представлений, которые затем подаются в другой стек блоков трансформера — **декодер**. Последний декодирует их в выходную последовательность. Такой подход к преобразованию одной последователь-

ности в другую называется *sequence-to-sequence* (или *seq2seq*) и хорошо подходит для задач вроде перевода, резюмирования или ответов на вопросы.

Предположим, вы пропускаете английское предложение через энкодер модели-переводчика, которая генерирует расширенные эмбединги, фиксирующие значение ввода. Затем декодер генерирует соответствующее предложение на французском языке, используя эти эмбединги. Генерация происходит в декодере по одному токену за раз, как мы видели при выводе последовательностей ранее в этой главе. Однако прогнозы для каждого последующего токена дополняются не только предыдущими токенами в генерируемой последовательности, но и выходными данными энкодера.

Механизм, посредством которого вывод со стороны энкодера включается в стек декодера, называется *перекрестным вниманием* (**Cross-attention**). Оно похоже на самовнимание, за исключением того, что каждый токен на входе (последовательность, обрабатываемая декодером) обращает внимание на контекст из энкодера, а не на другие токены в своей последовательности. Слои перекрестного внимания чередуются с слоями самовнимания, что позволяет декодеру использовать как контексты в своей последовательности, так и информацию из энкодера.

После статьи, описанной выше, существующие *seq2seq*-модели (например, Marian NMT), начали включать эти методы в качестве центральной части своей архитектуры. Новые модели были разработаны уже с использованием этих идей. Особенно стоит отметить модель **BART (Bidirectional and Auto-Regressive Transformers)**. Во время предварительной подготовки она искажает входные последовательности и пытается восстановить их в выходных данных декодера. После этого она настраивается на другие задачи генерации, такие как перевод или резюмирование, используя обширные представления последовательностей, достигнутые во время предварительного обучения. Также обратите внимание, что искажение входных данных является одной из ключевых идей, лежащих в основе *моделей диффузии*, о которых мы поговорим в *главе 4*.

Модели, имеющие только энкодер

Как уже обсуждалось ранее, модель-трансформер изначально была основана на архитектуре *энкодер-декодер*, которая применялась в моделях *BART* или *T5*. Помимо этого, энкодер или декодер могут обучаться и использоваться независимо друг от друга, что приводит к появлению целых семейств различных трансформеров. В начале главы мы уже рассмотрели авторегрессию, а также модели, которые имеют только декодер. Они специализируются на генерации текста с использованием описанных нами методов и дают впечатляющую производительность (ее уже продемонстрировали ChatGPT, Claude, Llama и Gemma).

Модели, имеющие только энкодер, с другой стороны, специализируются на получении расширенных представлений из текстовых последовательностей и могут использоваться в таких задачах, как классификация или подготовка семантических эмбедингов для множества документов, которая используется в системах поиска.

Наиболее известной моделью с энкодером на борту, вероятно, является *BERT*. В ней было введено *моделирование замаскированного языка*, которое позже позаимствовали и применили в *BART*.

Обычное языковое моделирование может предсказывать следующий токен, учитывая значения предыдущих, — это то, что мы сделали с Qwen. Такая модель учитывает только контекст слева от определенного токена. Существует и другой подход, используемый в моделях с энкодерами, который называется *моделированием замаскированного языка* (**Masked Language Modeling, MLM**). Оригинальный MLM (предложенный в статье «*BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*») позволяет предварительно обучить модель, чтобы она могла «заполнять пробелы». Учитывая входной текст, мы случайным образом *маскируем* некоторые токены, и модель должна предсказать их (рис. 2.11). В отличие от обычного моделирования языка, MLM использует как последовательность слева, так и справа от замаскированного токена. Это помогает создавать сильные представления входного текста. «Под капотом» таких моделей используются энкодеры — часть архитектуры трансформера.

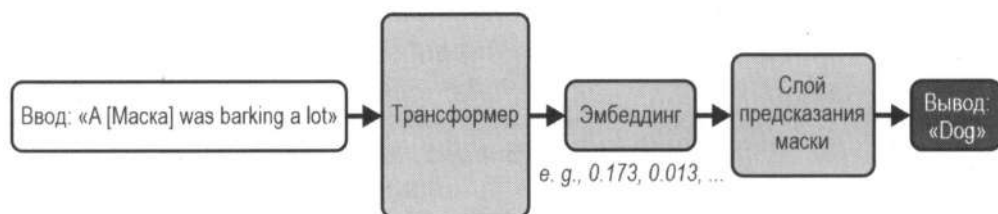


Рис. 2.11. Модели с энкодерами выводят семантические эмбединги, которые можно использовать для решения задач вроде прогнозирования токена в середине последовательности

Мы только что обсудили архитектуры *декодер* и *энкодер-декодер*. Часто возникает вопрос: зачем использовать модель энкодер-декодер для задач перевода, если одного декодера (Qwen и Llama) вполне достаточно, чтобы показывать хорошие результаты. Дело в том, что модели энкодер-декодер предназначены для перевода всей входной последовательности в выходные данные, что делает их хорошо подходящими для этой задачи. Модели, имеющие только декодер, фокусируются исключительно на прогнозировании следующего токена в последовательности. Изначально они (например, GPT-2) были меньше приспособлены к обучению с нулевым выстрелом, чем их более поздние релизы (например, GPT-4), но это было связано не только с отсутствием энкодера. Улучшение возможностей обучения с нулевым выстрелом в более продвинутых моделях (опять же, GPT-4) также обусловлено большими обучающими данными, лучшими методами обучения и увеличенными размерами моделей. Несмотря на то что энкодеры в моделях *seq2seq* играют решающую роль в понимании полного контекста входных последовательностей, прогресс моделей, где есть только энкодер, сделал их более эффективными и универсальными (даже для задач, традиционно подходящих под модели *seq2seq*).

Рассмотрим небольшой пример с кодом. Вместо того чтобы задействовать классы `AutoModel` и `AutoTokenizer`, использованные ранее, рассмотрим API более высокого

уровня из библиотеки `transformers`, который известен как `pipeline`. Он позволяет легко загружать модель для конкретной задачи. Этот API берет на себя всю предварительную и последующую обработку, а значит, является отличным способом быстро опробовать модель.

```
from transformers import pipeline

fill_masker = pipeline("fill-mask", model="bert-base-uncased")
fill_masker("The [MASK] is made of milk.")

[{'score': 0.19546695053577423,
  'token': 9841,
  'token_str': 'dish',
  'sequence': 'the dish is made of milk.'},
 {'score': 0.1290755718946457,
  'token': 8808,
  'token_str': 'cheese',
  'sequence': 'the cheese is made of milk.'},
 {'score': 0.10590697824954987,
  'token': 6501,
  'token_str': 'milk',
  'sequence': 'the milk is made of milk.'},
 {'score': 0.04112089052796364,
  'token': 4392,
  'token_str': 'drink',
  'sequence': 'the drink is made of milk.'},
 {'score': 0.03712352365255356,
  'token': 7852,
  'token_str': 'bread',
  'sequence': 'the bread is made of milk.'}]
```

Что происходит «под капотом»? Энкодер получает входную последовательность и генерирует контекстуализированное представление для каждого токена. Это представление имеет форму вектора чисел, который фиксирует значение токена в контексте всей последовательности. За энкодером обычно следует слой, специфичный для задачи, который использует эти представления для выполнения различных сценариев, например классификация, ответы на вопросы или моделирование замаскированного языка (рис. 2.11). Энкодер обучен генерировать представления, которые полезны для задач, требующих хорошего понимания входных данных.

Помимо моделей «только энкодер», «только декодер» и энкодер-декодер компании и исследовательские лаборатории выпустили большое количество других открытых и закрытых языковых моделей, вроде GPT-4, Mistral, Falcon, Llama, Qwen, Yi, Claude, Bloom, Gemma и пр. На рис. 2.12 представлена неполная генеалогия моделей-трансформеров, демонстрирующая их влияние на NLP по состоянию на 2024 год.

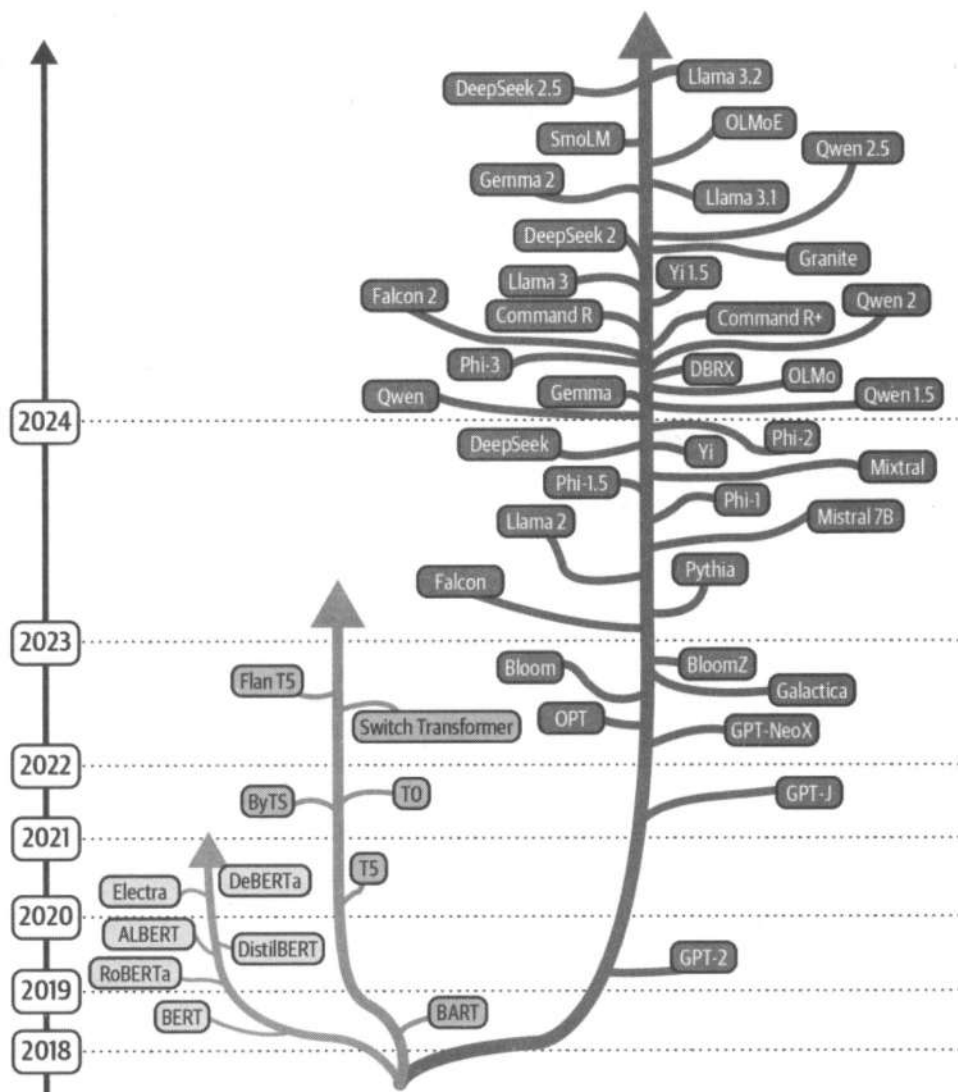


Рис. 2.12. Хронология выпуска открытых моделей «только энкодер» (красный цвет (ветвь слева)), «энкодер-декодер» (зеленый цвет (ветвь в центре)) и «только декодер» (синий цвет (ветвь справа))

Сила предварительного обучения

Доступ к существующим моделям весьма полезен. Трансформеры демонстрируют наилучшую производительность во многих языковых задачах, вроде классификации текста, машинного перевода и ответов на вопросы на основе исходного текста. Почему они работают так хорошо?

Во-первых, дело в механизме внимания. Он позволяет обрабатывать длинные последовательности и изучать долгосрочные отношения. Иначе говоря, трансформеры могут оценивать релевантность одних токенов для других.

Во-вторых, у этих моделей есть способность масштабироваться. Их архитектура реализована так, чтобы ее можно было оптимизировать для *распараллеливания*. Исследования показали, что трансформеры могут масштабироваться для обработки наборов данных высокой сложности и большого объема. Несмотря на то что изначально их архитектура была разработана для текста, она может быть достаточно гибкой, чтобы поддерживать различные типы данных и обрабатывать нестандартный ввод.

В-третьих, эти модели имеют возможность *предварительного обучения* и *тонкой настройки*. Традиционные подходы к задачам, вроде классификации, ограничены доступностью маркированных данных. Модель необходимо было обучать с нуля на большом корпусе готовых примеров, пытаясь при этом предсказывать метку напрямую из входного текста. Такой подход часто называют *контролируемым обучением* или *обучением с учителем* (**Supervised learning**). Однако он имеет существенный недостаток — для эффективного результата требуется большой объем помеченных данных. Это является проблемой, поскольку маркировка данных — дело дорогостоящее и требующее большого количества времени. К тому же во многих случаях данных может даже вообще не быть.

Чтобы удовлетворить эту потребность, исследователи начали искать способ предварительного обучения моделей на существующих данных, которые затем можно было бы настроить (скорректировать) для конкретной задачи. Такой подход известен как *трансферное обучение* (**Transfer learning**) и является основой современного ML во многих областях, таких как NLP и *компьютерное зрение* (**Computer Vision**). Первоначальные работы в области NLP были сосредоточены на поиске наборов данных, специфичных для конкретной задачи, чтобы можно было предварительно обучить языковую модель. Но такой подход, как **ULMFiT (Universal Language Model Fine-tuning for Text Classification)**, показал, что предварительное обучение даже на универсальном тексте (например, статья из Википедии) может дать впечатляющие результаты, когда модель настроена на другие задачи, вроде анализа настроений или ответов на вопросы. Это подготовило почву для появления трансформеров, которые показали отличные результаты для обучения расширенным представлениям языка.

Идея предварительного обучения заключается в том, чтобы научить модель на большом наборе немаркированных данных, а затем настроить ее на новую задачу, для которой потребуется гораздо меньшее количество маркированных данных (рис. 2.13). До применения к NLP трансферное обучение уже показывало хорошие результаты в сверточных сетях (CNN), которые составляют основу современного компьютерного зрения. В этом случае сначала мы обучаем большую модель с огромным количеством маркированных изображений. Благодаря этому процессу она изучает общие признаки, которые можно использовать для другой, но похожей задачи. Например, мы можем предварительно обучить модель на тысячах классов, а затем настроить ее для распознавания, есть ли на изображении, например, хот-дог.

С трансформерами дело движется еще дальше благодаря самоконтролируемому предварительному обучению. Мы можем предварительно обучить модель на больших немаркированных текстовых данных. Каким образом? Чтобы ответить на этот

вопрос, порассуждаем о казуальных моделях, таких как GPT. Она умеет предсказывать, какой токен будет следующим. Что ж, тогда нам не нужны никакие метки для получения обучающих данных. Имея корпус текста, мы можем маскировать токены после последовательности и обучать модель предсказывать их. Как и в случае с компьютерным зрением, предварительное обучение дает модели осмысленное представление базового текста. Затем мы можем точно настроить ее для выполнения другой задачи, например генерации текста в авторском стиле. Предполагая, что модель уже изучила представление языка, тонкая настройка потребует гораздо меньше данных, чем если бы она обучалась с нуля.



Рис. 2.13. Предварительное обучение базовой модели может потребовать значительного количества ресурсов, однако тонкая настройка существующей модели под новую задачу обходится значительно дешевле

Для многих задач обширное представление входных данных важнее, чем возможность предсказать следующий токен. Например, если вы хотите тонко настроить модель, чтобы прогнозировать настроение обзоров на фильмы, моделирование замаскированного языка (MLM) в этой задаче будут более подходящим. Такие модели, как GPT-2, предназначены, чтобы оптимизировать генерации текста, а не создавать мощные его представления. С другой стороны, модели вроде BERT идеально подходят для подобной задачи. Как упоминалось ранее, последний слой модели-энкодера выводит плотное представление входной последовательности, которое называется эмбедингом. Затем его можно использовать, добавив небольшую простую сеть поверх энкодера и тонко настроив модель для определенной задачи. В качестве конкретного примера мы можем добавить простой линейный слой поверх выходных данных энкодера BERT, чтобы спрогнозировать настроение в тексте. Мы можем использовать этот подход для решения широкого спектра задач:

◆ Классификация токенов

Позволяет определить каждую сущность в предложении, например человека, местоположение или организацию.

◆ Извлечение ответа на вопрос

Например, дано задание: ответить на конкретный вопрос, а ответ извлечь из ввода.

◆ Семантический поиск

Функции, генерируемые энкодером, могут быть полезны для построения поисковой системы. Например, дана база данных из ста документов. Мы можем вычислить эмбединги для каждого из них, а затем сравнить входные эмбединги с эмбедингами документов во время вывода, таким образом идентифицируя наиболее подходящий документ в базе.

Этот список является неполным. Он также может включать в себя задачи по определению сходства текста, обнаружению аномалий, связыванию именованных сущностей, рекомендательные системы и классификацию документов.

Попробуем использовать модель на основе BERT, которая была настроена для классификации последовательностей, чтобы определить, является ли настроение текста положительным или отрицательным. Снова воспользуемся API pipeline, чтобы загрузить модель и выполнить классификацию.

```
from transformers import pipeline

classifier = pipeline(
    "text-classification",
    model="distilbert/distilbert-base-uncased-finetuned-sst-2-english",
)

classifier("This movie is disgustingly good!")
```

#Вывод

```
[{'label': 'POSITIVE', 'score': 0.9998536109924316}]
```

Эта модель может анализировать отзывы и прогнозировать их настроение, как мы делали ранее в разделах «Генерализация с нулевым выстрелом» и «Генерализация с несколькими выстрелами». В разделе «Практика» показано, как оценивать подобные модели классификации, а также как сравнивать подходы «нулевого выстрела» и «тонкой настройки».

Краткий обзор трансформеров

Мы обсудили три типа архитектур:

◆ Архитектура на основе энкодера

Модели-энкодеры, такие как BERT, DistilBERT и RoBERTa, идеально подходят для задач, требующих понимания всего ввода целиком⁵. Они выводят контек-

⁵ DistilBERT — это модель меньшего размера, которая сохраняет 95% исходной производительности BERT, имея на 40% меньше параметров. RoBERTa — это мощная модель на основе BERT, которая обучается дольше с помощью различных гиперпараметров.

стуализированные эмбединги, которые фиксируют значение входной последовательности. Затем мы можем добавить небольшую сеть поверх них и обучить модель для новой конкретной задачи, которая опирается на семантическую информацию.

♦ *Архитектура на основе декодера*

Модели вроде GPT-2, Qwen, Gemma и Llama идеально подходят для генерации нового текста.

♦ *Архитектура энкодер-декодер*

Модели, такие как BART и T5, отлично подходят для задач, требующих генерации новых предложений на основе заданных входных данных, например резюмирование или перевод.

Вероятно, вы можете подумать, что со всеми этими задачами можно справиться, например, с помощью ChatGPT или Llama. И это правда. Учитывая огромный и растущий объем обучающих данных, вычислительной мощности и оптимизаций обучения, качество генеративных моделей значительно улучшилось. Их возможности обучения с нулевым выстрелом значительно увеличились по сравнению с тем, что было несколько лет назад.

Существуют две основные точки зрения на этот счет. С одной стороны, с учетом ресурсов тонкая настройка под конкретную задачу даст лучшие результаты, чем использование универсальной предварительно обученной модели. Например, вы хотите использовать модель GPT, чтобы сгенерировать диалоги персонажей игры в реальном времени, тогда предварительная тонкая настройка с помощью похожих данных может повысить производительность. Или, например, вам нужна модель, которая извлекает сущности из набора со статьями по химии, тогда имеет смысл тонкая настройка на основе энкодера с соответствующими текстами по заданной теме.

С другой стороны, с появлением высококачественных недорогих универсальных моделей, которые хорошо справляются с множеством задач, некоторые исследователи утверждают, что тонкая настройка может быть ненужной в большинстве сценариев. Вместо этого быстрое проектирование может быть более эффективным и дешевым подходом.

Модели *seq2seq* изначально были успешными, потому что они могут кодировать входные последовательности переменной длины в эмбединги, которые суммируют входную информацию. Затем декодер использует этот контекст, чтобы генерировать выходные данные. В последнее время модели, имеющие только декодер, приобрели популярность из-за своей простоты, масштабируемости, эффективности и возможности распараллеливания. На практике в зависимости от задачи используются разные их типы. Не существует единой «золотой» модели, которая подходит для всего.

Имея более чем миллион открытых вариантов, вы можете задаться вопросом, какой из них использовать. Глава 6 поможет вам сориентироваться в этом вопросе и предоставит рекомендации, как выбрать правильную модель для вашей задачи, а также как настроить ее под конкретные потребности.

Ограничения

На этом этапе вы можете задаться вопросом о возможных минусах трансформеров. Кратко рассмотрим некоторые ограничения:

♦ *Большой размер моделей*

Исследования показывают, что более крупные модели работают лучше. Несмотря на интерес, это также вызывает опасения. Во-первых, некоторые из самых мощных моделей требуют десятки миллионов долларов США для обучения, и это только на вычислительную мощность. Во-вторых, использование таких объемов вычислительной мощности может иметь экологические последствия: долгая непрерывная работа GPU потребляет много электроэнергии. В-третьих, даже если некоторые из этих моделей имеют открытый исходный код, для их запуска может потребоваться множество устройств с GPU.

♦ В главе 6 мы рассмотрим некоторые методы использования подобных LLM, даже если у вас под рукой нет подобной техники. Даже в этом случае их развертывание в средах с ограниченными ресурсами часто является проблемой.

♦ *Последовательная обработка*

Как вы помните из подраздела о декодерах, для каждого нового токена нам приходилось обрабатывать все предыдущие. Генерация 10 000-го токена в последовательности займет значительно больше времени, чем генерация исходного. В терминах компьютерной науки трансформеры имеют квадратичную сложность времени обработки относительно длины входных данных. Это означает, что по мере увеличения длины ввода время, необходимое для обработки, растет квадратично, что затрудняет их масштабирование для очень длинных документов, а также усложняет использование этих моделей в реальном времени. Несмотря на то что трансформеры преуспевают во многих задачах, их вычислительные потребности должны быть тщательно рассмотрены и оптимизированы при использовании на практике. Тем не менее на текущий момент проведено много исследований по повышению их эффективности для чрезвычайно длинных последовательностей, например умные методы кеширования, а также кольцевое и бесконечное внимание.

♦ *Фиксированный размер входных данных*

Максимальное количество токенов, которое могут обрабатывать трансформеры, зависит от базовой модели. Оно называется контекстным окном, и его важно учитывать при выборе предварительно обученной модели. Вы не можете просто передать целые энциклопедии трансформерам, ожидая, что они смогут их обработать, хотя эта тенденция быстро меняется. Несмотря на то что некоторые модели могут обрабатывать лишь не более 512 токенов, в настоящее время все больше распространяются варианты, которые могут обрабатывать до 32 000 токенов. Новые методы позволяют масштабировать их до сотен тысяч или даже миллионов токенов. Например, Llama 3.1 может обрабатывать 131 000 токенов, что примерно сопоставимо с объемом этой книги.

◆ Ограниченная возможность интерпретации

Трансформаторы часто критикуют за отсутствие интерпретируемости⁶.

Все эти ограничения активно исследуются. На текущий момент уже изучено, как обучать и запускать модели с меньшей вычислительной мощностью (например, QLoRA, которую мы рассмотрим в главе 6), ускорять генерацию (например, с помощью Flash Attention и вспомогательной генерации), делать возможными неограниченные размеры ввода (например, RoPE и снижение внимания) и интерпретировать механизмы внимания.

Одной из больших проблем является наличие предубеждений в моделях. Если обучающие данные, используемые для предварительной подготовки трансформеров, содержат предубеждения, модель может выучить их и сохранить. Это большая проблема в ML, но она особенно актуальна для трансформеров. Вернемся к конвейеру `fill-mask`. Допустим, мы хотим предсказать наиболее вероятную профессию. Как вы можете увидеть в следующем примере, результаты различаются в зависимости от пола: слова «man» (мужчина) и «woman» (женщина).

```
unmasker = pipeline("fill-mask", model="bert-base-uncased")
result = unmasker("This man works as a [MASK] during summer.")
print([r["token_str"] for r in result])

result = unmasker("This woman works as a [MASK] during summer.")
print([r["token_str"] for r in result])
```

#Вывод

```
['farmer', 'carpenter', 'gardener', 'fisherman', 'miner']
['maid', 'nurse', 'servant', 'waitress', 'cook']
```

Почему так происходит? Для обеспечения предварительного обучения исследователям обычно требуются большие объемы данных, поэтому они стараются извлечь весь контент, который могут найти. Он может быть любого качества, включая «токсичную» информацию (которая может быть в некоторой степени отфильтрована). Базовая модель может в конечном итоге «укоренить» и «увечить» эти предубеждения при тонкой настройке. Аналогичные опасения существуют для разговорных моделей, где они могут генерировать «токсичный» контент, извлеченный из набора данных предварительного обучения.

Помимо текста

Трансформеры используются для многих задач, где данные представлены в виде текста. Яркий пример — генерация кода. Вместо того чтобы обучать языковую модель с помощью английского текста, мы можем использовать те же принципы, рассмотренные ранее, и научить ее автоматически дополнять код. Другой пример —

⁶ Среди направлений исследований в этой области все большую популярность приобретает использование разреженных автоэнкодеров (**Sparse AutoEncoder**) для извлечения интерпретируемых признаков из трансформеров.

использование трансформеров для ответов на вопросы, например, из электронной таблицы.

Поскольку модели-трансформеры показали большой успех в задачах, связанных с текстом, в некоторых сообществах возник значительный интерес к адаптации их для других областей. Это привело к использованию их для таких задач, как распознавание изображений, сегментация, обнаружение объектов, понимание видео и т. д. (рис. 2.14).

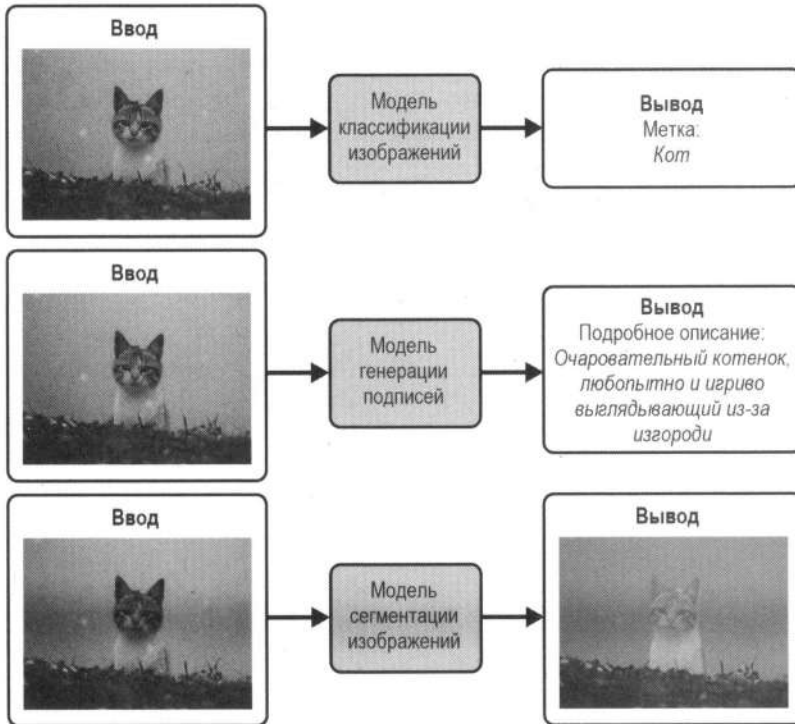


Рис. 2.14. Модели-трансформеры можно использовать для таких задач, как классификация изображений, обнаружение объектов и сегментация изображений

CNN широко использовались в качестве моделей SOTA для большинства задач компьютерного зрения. Появление *трансформеров зрения (Vision Transformer, ViT)* в последние годы побудило многих людей к исследованиям, как решать зрительные задачи с помощью внимания и методов на основе трансформеров. В архитектуре ViT авторы не отказываются от использования CNN полностью: в конвейере обработки изображений сверточные сети извлекают *карты признаков* изображения для обнаружения высокоуровневых граней, текстур и других шаблонов. Затем полученные карты делятся на фиксированные по размеру непересекающиеся участки. Их можно обрабатывать как последовательность токенов, поэтому механизм внимания позволяет изучать взаимосвязи между участками в разных местах.

К сожалению, ViT требуют больше данных (300 миллионов изображений) и вычислений, чем CNN, чтобы они могли давать хорошие результаты. В последние годы

было проведено много работ по их изучению. Например, модель DeiT смогла использовать архитектуру на основе трансформеров с наборами данных среднего размера (1,2 миллиона изображений) благодаря использованию методов *аугментации* и *регуляризации*, распространенных в CNN. Такие модели, как DETR, SegFormer и Swin Transformer, продвинули область еще дальше, благодаря возможности поддерживать множество задач, вроде классификации изображений, обнаружения объектов, сегментации изображений, классификации видео, понимания документов, восстановления изображений, сверхвысокого разрешения и пр.

Мощным примером моделей-трансформеров, которые работают с изображениями, является *классификация изображений с нулевым выстрелом*. В отличие от традиционных классификаторов, которые обучаются на фиксированном наборе классов, классификация изображений с нулевым выстрелом позволяет указывать классы прямо во время вывода. Это обеспечивает гибкость применения одной модели для множества задач классификации изображений (даже для тех, для которых она явно не обучалась). Чтобы продемонстрировать это на практике (рис. 2.15), загрузим изображение с помощью библиотеки PIL — широко используемого инструмента для предварительного обучения в задачах со зрением.

```
import requests
from PIL import Image

from genaibook.core import SampleURL

#Скачиваем изображение и загружаем его с помощью библиотеки PIL
url = SampleURL.CatExample
image = Image.open(requests.get(url, stream=True).raw)
image
```



Рис. 2.15. Изображение, загруженное с помощью библиотеки PIL

Далее применим высокоуровневый pipeline, чтобы использовать модель для данной задачи.

```
pipe = pipeline(
    "zero-shot-image-classification", model="openai/clip-vit-base-patch32"
) ❶
```



```

labels = ["cat", "dog", "zebra"] ❷
pipe(image, candidate_labels=labels) ❸
image

[{'score': 0.9936687350273132, 'label': 'cat'},
 {'score': 0.006043245084583759, 'label': 'dog'},
 {'score': 0.0002880473621189594, 'label': 'zebra'}]

```

Немного поясним код:

- ❶ Загружаем модель `openai/clip-vit-base-patch32`.
- ❷ Определяем классы, которые хотим использовать во время вывода.
- ❸ Передаем изображение и метки через конвейер, чтобы получить прогнозы.

Модели трансформеров также можно использовать для задач, связанных со звуком (рассмотрим в главе 9), например транскрибирование аудиофайлов и генерация синтетической речи или музыки. «Под капотом» они имеют те же фундаментальные механизмы предварительного обучения и внимания, но каждая область предполагает использование определенных типов данных, требующих их модификаций и разного подхода к решению задач.

Также существуют и другие направления, в которых применяются и исследуются трансформеры, например:

♦ Графы

Отличным чтением для вводного ознакомления является статья «Introduction to Graph Machine Learning» Клементины Фурье (Clémentine Fourier). Использование трансформеров для построения графов все еще носит исследовательский характер, но ранние результаты все равно поражают. Примерами являются: прогнозирование токсичности молекул, прогнозирование эволюции систем или генерация новых возможных молекул.

♦ 3D-данные

Выполнение сегментации данных, которые могут быть представлены в 3D-формате. Например, в виде облака точек LiDAR для автономного вождения или КТ-скана для сегментации изображений органов. Еще один пример — оценка шести степеней свободы объекта, что может быть полезно в робототехнических приложениях.

♦ Временные ряды

Анализ цен на акции или выполнение прогнозирования погоды.

♦ Мультимодальность

Некоторые трансформеры могут использоваться для обработки или вывода нескольких типов данных (например, текста, изображений и аудио) вместе. Это открывает новые возможности, где вы можете генерировать речь, текст или изображения и при этом иметь единую модель для обработки. Еще один пример — визуальные ответы, когда модель может отвечать на вопросы по предоставленным изображениям.

Создание проекта: использование языковой модели для генерации текста

Мы использовали метод `generate()` в подразделе «Генерация текста» для выполнения различных методов декодирования. Чтобы лучше понять, как он работает, пришло время реализовать его самостоятельно. Вы будете использовать `generate()` в качестве примера, но реализуете его с нуля.

Ваша цель — заполнить код в следующей функции. Вместо того чтобы использовать метод `model.generate()`, идея состоит в том, чтобы итеративно вызывать метод `model()`, передавая предыдущие токены в качестве входных данных. Вам нужно реализовать жадное декодирование при `do_sample = False`, выборку при `do_sample = True` и выборку *Тор-К* при `do_sample = True` и `top_k != None`. Эта задача сложная, поэтому не волнуйтесь, если вы найдете решение не сразу. Мы предлагаем вам начать с реализации жадного декодирования, а затем на его основе продолжить писать код.

```
def generate(
    model, tokenizer, input_ids, max_length=50, do_sample=False, top_k=None
):
    """Сгенерируйте последовательность без использования model.generate()

    Аргументы:
    model: модель, которая будет использоваться для генерации
    tokenizer: токенизатор, который будет использоваться для генерации
    input_ids: входящие ID
    max_length: максимальная длина последовательности
    do_sample: Следует ли использовать выборку
    top_k: количество токенов для выборки
    """
    #Здесь будет ваш код
    #Начните с самого простого подхода – жадного декодирования
    #Добавьте обычную выборку, а затем выборку top-k
```

Заключение

Теперь вы знаете, как загружать и использовать трансформеры для различных задач. В этой главе мы также рассказали, как трансформеры моделируют последовательности данных вроде текста и как это умение позволяет им исследовать ценные представления, которые можно использовать для генерации или классификации новых последовательностей. По мере увеличения масштаба моделей растут и их возможности — до такой степени, что массивные модели с сотнями миллиардов параметров теперь могут выполнять множество задач, которые ранее считались невыполнимыми для компьютеров.

Существуют мощные, предварительно обученные модели, которые можно изменять под определенные области и сценарии использования благодаря тонкой на-

стройке. Тенденция к созданию более крупных и универсальных моделей изменила подход к их использованию. Экземпляры, ориентированные на конкретные задачи, часто вытесняются универсальными LLM. Большинство людей теперь взаимодействуют с этими моделями через API, хостинги, локальные развертывания или напрямую через удобные пользовательские интерфейсы на основе чатов. В то же время появление больших и мощных моделей с открытым доступом, таких как Llama, вызвало сильную мотивацию среди исследователей и практиков запускать высококачественные модели непосредственно на потребительских компьютерах, что позволяет создавать решения, ориентированные на конфиденциальность. Также в последние годы появились новые подходы к обучению, которые позволяют людям настраивать эти модели без больших вычислительных ресурсов. Глава 6 описывает это более подробно и погружает читателя как в традиционные, так и в новые методы тонкой настройки.

Несмотря на то что в дальнейшем мы более подробно углубимся в тему обучения трансформеров, изучение их внутренних частей (например, математики, лежащей в основе механизмов внимания) или предварительное обучение модели с нуля выходят за рамки этой книги. К счастью, есть прекрасные ресурсы, чтобы узнать об этом подробнее:

- ◆ *«The Illustrated Transformer»* Джея Аламмара (Jay Alammar) — прекрасное наглядное пособие, подробно и интуитивно объясняющее модели трансформеров.
- ◆ *«Natural Language Processing with Transformers»* Льюиса Танстола (Lewis Tunstall) (издательство O'Reilly) — будет полезно, если вы хотите глубже погрузиться во внутренние механизмы тонкой настройки моделей-трансформеров под множество конкретных задач.
- ◆ Hugging Face предоставляет бесплатный курс с примерами кода, который научит, как решать различные задачи NLP.

Если вы хотите глубже изучить семейство моделей GPT, мы предлагаем ознакомиться со следующими статьями:

- ◆ *«Improving Language Understanding by Generative Pre-training»*

Это первая статья про GPT, опубликованная в 2018 году Алеком Рэдфордом (Alec Radford). В ней была представлена идея модели на основе трансформера, предварительно обученной на большом корпусе текста, для изучения общих языковых представлений, а также ее тонкая настройка на конкретных нисходящих задачах. Кроме того, в статье показано, что модель достигла лучших результатов на нескольких тестах, которые касались понимания естественного языка в то время.

- ◆ *«Language Models Are Unsupervised Multitask Learners»*

В этой статье, опубликованной в 2019 году Алеком Рэдфордом, была представлена GPT-2 — модель на основе трансформера с 1,5 миллиарда параметров, предварительно обученная на большом корпусе веб-текста. В статье также показано, что GPT-2 может хорошо справляться с различными задачами на естественном языке без тонкой настройки, вроде генерации текста, резюмирования, перевода, понимания прочитанного и рассуждений на основе здравого смысла.

В заключение в статье обсуждались потенциальные этические и социальные последствия крупномасштабных LM.

♦ «*Language Models Are Few-Shot Learners*»

Эта статья, опубликованная в 2020 году Томом Б. Брауном (Tom B. Brown), показывает, что масштабирование языковых моделей значительно улучшает их способность выполнять новые задачи с помощью всего лишь нескольких примеров или простых инструкций без тонкой настройки или градиентных обновлений. В статье также представлена GPT-3 — авторегрессивная языковая модель с 175 миллиардами параметров, которая демонстрирует высокую производительность при работе со многими наборами данных в задачах обработки естественного языка.

Вопросы

1. Какова роль механизма внимания в генерации текста?
2. В каких случаях предпочтительнее использовать токенизатор на основе символов?
3. Что произойдет, если вы используете токенизатор, отличный от того, который используется с моделью?
4. Каков риск использования параметра `no_repeat_ngram_size` при генерации? (Подсказка: подумайте о названиях городов.)
5. Что произойдет, если вы объедините лучевой поиск и выборку?
6. Представьте, что вы используете большую языковую модель (LLM), которая генерирует код в редакторе с помощью выборки. Какое значение параметра температуры (`temperature`) будет удобнее: низкое или высокое?
7. В чем важность тонкой настройки и чем она отличается от генерации с нулевым выстрелом?
8. Объясните различия, а также сценарии применения трансформеров на основе энкодеров, декодеров и энкодер-декодеров.

Вы можете найти ответы на эти вопросы и решения следующих задач в репозитории книги на GitHub.

Практика

1. **Резюмирование.** Используйте модель резюмирования (можно использовать `pipeline("summarization")`), чтобы выделить основную информацию из текста. Как это соотносится с результатами использования нулевого выстрела (Zero-shot)? Можно ли превзойти получившиеся результаты, предоставив модели несколько выстрелов (Few-shot)?
2. **Анализ настроений.** В дополнительном материале по нулевому выстрелу мы вычисляем некоторые метрики с помощью классификации. Исследуйте исполь-

зование модели-энкодера `distilbert-baseuncased-finetuned-sst-2-english`, которая может выполнять анализ настроений. Какие результаты у вас получились?

3. **Семантический поиск.** Вам нужно создать систему часто задаваемых вопросов (FAQ). Трансформеры — это мощные модели, которые могут измерять семантическое сходство текста. В то время как энкодеры обычно выводят эмбединги для каждого токена, трансформеры выводят эмбединги для всего ввода, что позволяет определить, имеют ли два текста схожие значения. Рассмотрим простой пример с использованием библиотеки `sentence_transformers`.

```
from sentence_transformers import SentenceTransformer, util

sentences = ["I'm happy", "I'm full of happiness"]
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")

#Вычисляем эмбединги для обоих списков
embedding_1 = model.encode(sentences[0], convert_to_tensor=True)
embedding_2 = model.encode(sentences[1], convert_to_tensor=True)

util.pytorch_cos_sim(embedding_1, embedding_2)

tensor([[0.6003]], device='cuda:0')
```

Напишите список из пяти вопросов и ответов по теме. Ваша цель — создать систему, которая, получив новый вопрос, может дать пользователю наиболее вероятный ответ. Как вы можете использовать трансформеры, чтобы решить эту задачу? Дополнительный материал к этой книге содержит решение, но, хоть это и сложно, мы предлагаем вам сначала попробовать решить задачу самостоятельно, прежде чем искать в нем ответ.

Мощная техника, называемая *поисково-дополненной генерацией (Retrieval-Augmented Generation, RAG)*, объединяет генерацию текста и эмбединги для извлечения соответствующих документов. Приложение C полностью демонстрирует пример того, как построить конвейер RAG. Перед этим мы предлагаем прочитать главу 6, в которой представлена тонкая настройка, а также как ее использовать для адаптации моделей к конкретным задачам.

Сжатие и представление информации

В этой главе описаны модели и методы машинного обучения, которые помогут изучить эффективное представление данных в задачах, связанных с изображениями, видео или текстом. Почему это важно? *Представления* позволяют сократить объем информации, которую нужно хранить и обрабатывать, сохраняя при этом основные характеристики данных. Подробные представления позволяют обучать модели для определенных задач, а их компактность — снизить вычислительные требования для обучения и работы с моделями, требующими больших объемов данных. Например, обучение на векторных эмбедингах изображения может быть более эффективным и выразительным, чем обучение непосредственно на его пикселях.

Традиционные методы сжатия, например ZIP или JPEG, ориентированы на определенные типы данных и используют специально разработанные алгоритмы для уменьшения размеров файлов. Несмотря на то что они эффективны в своих задачах, им не хватает гибкости и адаптивности для рассматриваемых в книге сценариев. Например, ZIP может отлично сжимать общие данные без потерь, выявляя и кодируя повторяющиеся шаблоны, а JPEG обеспечивает значительное уменьшение размера изображения за счет отбрасывания менее заметной визуальной информации. Однако эти методы не имеют механизма обучения на данных, которые они сжимают, и не могут автоматически адаптироваться к различным типам контента или оптимизироваться под конкретные задачи, помимо уменьшения размера. Именно здесь модели машинного обучения могут быть полезны.

Сначала рассмотрим *автоэнкодеры (AutoEncoder)*¹ — семейство ML-моделей, состоящих из *энкодера*, который сжимает данные, и *декодера*, который затем реконструирует их с помощью представлений. Энкодер изучает основные характеристики данных, на которых ему нужно сосредоточиться, что позволяет декодеру восстанавливать информацию (левая часть рис. 3.1). Такой подход к обучению позволяет автоматически создавать алгоритмы для сжатия, не полагаясь на традиционные методы. Сжатие информации (даже с потерями) полезно само по себе, но есть еще несколько интересных вещей, которые мы можем использовать, как только у нас будет компактное представление набора данных.

¹ Перевод «автокодировщик» предпочтителен в академических текстах, учебниках и документации. Перевод «автоэнкодер» чаще встречается в статьях, блогах и разговорной речи. В этой книге здесь и далее мы будем использовать транслитерацию «автоэнкодер». — *Пер.*

Если система обучена правильно и декодер может восстановить оригинал, значит, представления «уловили» необходимую информацию. Таким образом, работа с ними эквивалентна работе с оригиналами, но при этом она требует гораздо меньше памяти и вычислений. Это один из ключевых аспектов проектирования таких моделей, как *Stable Diffusion*. Как будет видно в главе 5, мы можем генерировать и обрабатывать объемные изображения, но большая часть вычислений происходит в *латентном пространстве* (Latent space)², где находятся представления.

Поскольку обученные представления захватывают основную информацию, мы можем разделить энкодер и декодер, как только автоэнкодер будет обучен, и использовать энкодер в качестве компонента извлечения признаков. Добавление небольшой сети поверх выходов энкодера (как показано на центральной части рис. 3.1) позволяет обучать модель для различных задач, например, классификация текста или изображений. Эти небольшие сети работают не со всем входным изображением, а с основными характеристиками (признаками), полученными энкодером.

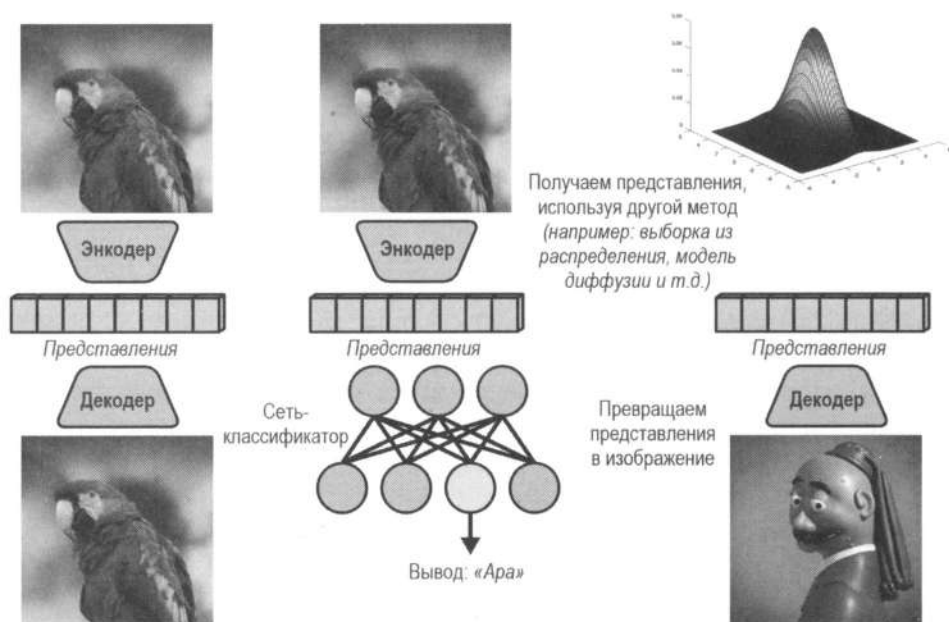


Рис. 3.1. Эффективные методы представления данных (слева) можно использовать для других задач, таких как классификация (посередине), или для создания нового контента (справа)

Мы также можем кодировать различные типы данных в одни и те же представления латентного пространства. Как мы знаем из главы 2, языковые *seq2seq*-модели используют архитектуру «энкодер-декодер» для выполнения широкого спектра задач, вроде перевода или резюмирования текста. Несмотря на то что при проектировании

² Латентное пространство — это понятие в глубоком обучении, которое относится к преобразованному, часто низкоразмерному представлению данных, которое захватывает их существенные особенности и базовые закономерности. Это абстрактное пространство позволяет моделям эффективнее обрабатывать сложные данные, фокусируясь на наиболее значимых признаках. — *Пер.*

таких систем следует учитывать много деталей, одним из ключевых моментов является то, что работа энкодера заключается в захвате основных признаков, которые несут достаточно семантической информации о входном тексте. Это работает и в других методах. Например, модель, генерирующая описания изображений, переводит представления этих изображений в текстовые описания, используя латентное пространство в качестве внутренних рабочих данных.

Другой пример использования автоэнкодеров — генеративное моделирование (правая часть рис. 3.1). После того как мы обучили пару «энкодер-декодер», мы можем отбросить энкодер и сгенерировать новые данные с помощью выборки из случайного распределения в латентном пространстве. Это основа *вариационных автоэнкодеров* (**Variational AutoEncoder, VAE**). Пример, как это можно реализовать, будет рассмотрен далее в этой главе.

Мы будем использовать изображения, чтобы продемонстрировать, как работают автоэнкодеры и VAE, но эти методы можно применять и к любым другим данным. В последнем разделе этой главы мы рассмотрим, как системы *мультимодального репрезентативного обучения* (**Multimodal representation learning**)³, такие как CLIP, позволяют заполнить пробел между текстом и изображениями и могут использоваться в очень интересных сценариях (семантический поиск, фильтрация данных, генерация текста в изображение и многое другое).

Автоэнкодеры

Автоэнкодеры состоят из двух частей, соединенных вместе, — энкодера и декодера (схематически представлены на рис. 3.2). Оба компонента обучаются вместе таким образом, чтобы энкодер мог создавать промежуточные представления, которые затем декодер будет использовать для повторной генерации входных данных. Если обучение проходит успешно, автоэнкодер может извлекать ключевые признаки из входных данных.



Рис. 3.2. Архитектура автоэнкодера

³ Мультимодальное репрезентативное обучение — это раздел репрезентативного обучения, направленный на интеграцию и интерпретацию информации из различных модальностей (видов информации), таких как текст, изображения, аудио или видео. Главная цель — научить систему понимать, как разные модальности соотносятся друг с другом, и использовать это понимание для решения более сложных задач, чем те, что можно выполнить с помощью одного типа данных. — Пер.

Подготовка данных

В этом разделе мы создадим простой автоэнкодер с использованием набора данных MNIST. MNIST — это классический набор данных, состоящий из 70 000 черно-белых изображений рукописных цифр с низким разрешением (28×28). Мы загрузим его из распределения набора данных, размещенного на Hugging Face. Чтобы сделать это, мы воспользуемся библиотекой `datasets`, которая предоставляет унифицированный API для доступа к тысячам наборов данных любого типа. Подробности о том, как она работает, сейчас не важны, просто обратите внимание, что библиотека позаботится о загрузке и кешировании данных для последующего их использования. Она разделена на две части: обучающий набор с 60 000 изображений и тестовый набор с оставшимися 10 000 изображений.

```
from datasets import load_dataset

mnist = load_dataset("mnist")
mnist
DatasetDict({
  train: Dataset({
    features: ['image', 'label'],
    num_rows: 60000
  })
  test: Dataset({
    features: ['image', 'label'],
    num_rows: 10000
  })
})
```

Как вы можете видеть, набор данных имеет столбцы `image`, который содержит рукописные изображения, и `label`, который содержит число, представленное изображением. Поскольку мы собираемся обучить автоэнкодер сжимать и реконструировать изображения, нам пока не нужны данные меток (`label`). Мы будем подавать на вход энкодеру пакеты случайных выборок, а задача декодера — воссоздать изображения, близкие к исходным. Поскольку входные данные содержат все необходимое (без опоры на внешнюю помеченную информацию), процесс обучения автоэнкодера можно назвать примером *самообучения* (**Self-supervised learning**).

Несмотря на то что сейчас мы игнорируем метки, позже мы будем использовать их для визуализации. Перед обучением модели рассмотрим набор данных (рис. 3.3).

```
mnist["train"]["image"][1]
```



Рис. 3.3. Символ из набора данных с индексом 1

Изображения с разрешением всего 28×28 очень малы по сегодняшним меркам. Мы воспользуемся вспомогательной функцией `show_images()`, чтобы сделать их разрешение более высоким. Эта функция заимствована из Python-библиотеки `matplotlib`, которая по умолчанию использует высококонтрастную цветовую палитру для представления монохромных изображений (рис. 3.4).

```
from genaibook.core import show_images

show_images(mnist["train"]["image"][:4])
```

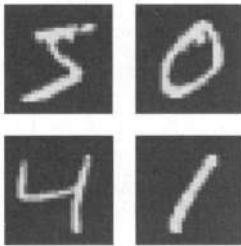


Рис. 3.4. Пример работы библиотеки `matplotlib`



Рис. 3.5. Изменяем настройки библиотеки `matplotlib` так, чтобы мы получили черные цифры на белом фоне

Поскольку оригиналы являются черно-белыми, настроим `matplotlib` так, чтобы она использовала только серые цвета. Выбираем «инвертированный серый» (`gray_r`), чтобы получить черные числа на белом фоне (рис. 3.5). Обратите внимание, что в оригиналах все наоборот, пиксели с числовыми данными — белые, а фон — черный (представлен нулями).

```
import matplotlib as mpl

mpl.rcParams["image.cmap"] = "gray_r"
show_images(mnist["train"]["image"][:4])
```

В следующем примере мы преобразовываем изображения в тензоры `PyTorch` и перетасовываем обучающий набор данных. Мы используем функцию `ToTensor()` из библиотеки `torchvision`⁴, чтобы преобразовать входные пиксели в диапазоне $[0, 255]$ в тензоры `PyTorch` от 0 до 1. Мы не применяем никаких других манипуляций. Для удобства функция `show_images()` также может рисовать тензоры, представляющие изображения.

```
from torchvision import transforms

def mnist_to_tensor(samples):
    t = transforms.ToTensor()
    samples["image"] = [t(image) for image in samples["image"]]
    return samples
```

⁴ Как мы увидим позже, параметр `transforms` из библиотеки `torchvision` представляет собой набор общих процедур преобразования, конвертации, дополнения и манипуляции касательно изображений.

```
mnist = mnist.with_transform(mnist_to_tensor)
mnist["train"] = mnist["train"].shuffle(seed=1337)
```

Проверим одно изображение из набора и убедимся, что входные пиксели находятся в диапазоне от 0 до 1 (рис. 3.6).



Рис. 3.6. Проверка работы библиотеки: входные пиксели находятся в диапазоне от 0 до 1

С помощью следующего кода мы создадим класс `DataLoader` в PyTorch для подготовки обучающих данных. Поскольку обучение автоэнкодера — это самоконтролируемый процесс, мы будем работать со столбцом `image` из набора данных и игнорировать метки из столбца `label`. Позже мы вернемся и используем их для визуализации результатов. `DataLoader` является абстракцией, чья основная задача — сопоставлять входные данные, что подразумевает сбор и объединение отдельных выборок в обучающие пакеты одной формы. В нашем случае все изображения имеют одинаковый размер и, следовательно, все тензоры имеют одинаковую форму, поэтому `DataLoader` объединит их вместе. В более сложных случаях `DataLoader` способен справляться с нерегулярными формами входных данных, используя такие стратегии, как *заполнение* или *усечение*.

```
from torch.utils.data import DataLoader

bs = 64
train_dataloader = DataLoader(mnist["train"]["image"], batch_size=bs)
```

Энкодер

Сначала создадим определение модели для кодирующей части нашего автоэнкодера. Поскольку мы работаем с изображениями, логичный выбор — использовать сверточные слои, которые хорошо улавливают их особенности. Мы могли бы рассмотреть множество других альтернатив для этого: линейные слои, блоки трансформеров, остаточные пропускные соединения и т. д. Но мы будем использовать простой сверточный энкодер, основанный на *сверточном автоэнкодере* из библиотеки `pythae`. Это отличная отправная точка для исследования.



Сверточные слои представляют собой набор небольших 2D-фильтров, многократно применяемых к различным областям входного изображения. Они могут обнаруживать такие повторяющиеся структуры, как линии или круглые области. Традиционно 2D-фильтры использовались в цифровой обработке, где они тщательно создавались вручную, чтобы соответствовать определенным особенностям входных изображений. Главное отличие сверточных слоев заключается в том, что фильтры не подготавливаются заранее — вместо этого сверточные слои изучают их в ходе

процесса обучения. Накладывая несколько сверточных слоев, модель может постепенно извлекать более абстрактные признаки из входного изображения, обучая фильтры, которые эффективно решают задачу. Существует область интерпретации и объяснений, цель которой — визуализировать и понять внутреннюю работу слоев модели. Она показывает, что фильтры, изученные нейронными сетями, иногда напоминают классические, разработанные вручную для обнаружения краев, цветов или контуров.

Для более глубокого изучения сверточных нейронных сетей мы рекомендуем прочитать главу 13 книги *«Deep Learning for Coders with fastai and PyTorch»* авторов Джереми Ховарда (Jeremy Howard) и Сильвена Гаттера (Sylvain Gugger) издательства O'Reilly.

Поскольку мы будем складывать несколько сверточных слоев, напомним простую вспомогательную функцию `conv_block()` для их создания. Она будет выполнять 2D-свертку, а после добавлять слой пакетной нормализации и нелинейность (для этого примера мы будем использовать функцию активации `ReLU`). Во время обучения пакетная нормализация использует среднее значение и стандартное отклонение текущего пакета для нормализации входных данных, чтобы они оставались в предсказуемом диапазоне, что в большинстве случаев приводит к более плавному и быстрому обучению.

Реализуем функцию.

```
from torch import nn

def conv_block(in_channels, out_channels, kernel_size=4, stride=2, padding=1):
    return nn.Sequential(
        nn.Conv2d(
            in_channels,
            out_channels,
            kernel_size=kernel_size,
            stride=stride,
            padding=padding,
        ),
        nn.BatchNorm2d(out_channels),
        nn.ReLU(),
    )
```

Энкодер представляет собой последовательность сверточных слоев. Каждый слой постепенно снижает разрешение изображения, одновременно увеличивая количество каналов представлений до 1024. В конце мы добавим линейный слой, чтобы создать 16-мерные векторные представления. Комментарии в методе `forward()` показывают, как преобразуется форма входных данных по мере их прохождения через слои.

```
class Encoder(nn.Module):
    def __init__(self, in_channels):
        super().__init__()
        self.conv1 = conv_block(in_channels, 128)
        self.conv2 = conv_block(128, 256)
```

```

self.conv3 = conv_block(256, 512)
self.conv4 = conv_block(512, 1024)
self.linear = nn.Linear(1024, 16)

def forward(self, x):
    x = self.conv1(x) #(размер пакета, 128, 14, 14)
    x = self.conv2(x) #(размер пакета, 256, 7, 7)
    x = self.conv3(x) #(размер пакета, 512, 3, 3)
    x = self.conv4(x) #(размер пакета, 1024, 1, 1)
    #Сохраняем размер пакета при выравнивании
    x = self.linear(x.flatten(start_dim=1)) # (bs, 16)
    return x

```

Теперь необходимо проверить, что мы можем пропустить наши изображения через энкодер. Они представлены в виде вектора [1, 28, 28], поскольку содержат только один канал (черных или белых) пиксельных данных. Однако обратите внимание, что мы запрограммировали наш энкодер обычным способом (мы могли бы использовать его и для трехканальных изображений).

```
mnist["train"]["image"][0].shape
```

```
torch.Size([1, 28, 28])
```

Далее выберем и поместим одно изображение в пакет, создав новое измерение с индексацией None. Нам также нужно установить энкодер в режим eval. Он настраивает модель на вывод, а не на обучение. Если мы этого не сделаем, последний слой BatchNorm2d потерпит неудачу, поскольку он получит тензор [1, 1024, 1, 1] и не сможет вычислить среднее значение и стандартное отклонение одной выборки⁵.

```
in_channels = 1
```

```
x = mnist["train"]["image"][0][None, :]
```

```
encoder = Encoder(in_channels).eval()
```

```
encoded = encoder(x)
```

```
encoded.shape
```

```
torch.Size([1, 16])
```

Модель энкодера работает. Она преобразует изображения с разрешением 28×28 (по 784 пикселя каждое) в векторы, которые содержат всего 16 чисел. Если мы сможем эффективно обучить энкодер, то его представления будут иметь гораздо меньшую размерность, чем исходные пиксельные данные. Конечно, представления на текущий момент не имеют смысла, т. к. модель еще не обучена.

⁵ В режиме eval слой BatchNorm2d применяет среднее значение и стандартное отклонение, полученные из всех мини-пакетов во время обучения. Поскольку мы еще не обучили модель, это будут случайные данные, но нас интересует только то, работает ли определение модели. Во время реального обучения мы будем использовать размер пакета больше единицы, и BatchNorm2d будет работать нормально.

```
encoded
```

```
tensor([[[-0.0145, -0.0318, -0.0109, 0.0080,
          -0.0218, 0.0305, 0.0183, -0.0294,
          0.0075, 0.0178, -0.0161, -0.0018,
          0.0208, -0.0079, 0.0215, 0.0101]],
        grad_fn=<AddmmBackward0>)
```

Посмотрим, сможет ли энкодер обработать пакет из 64 изображений.

```
batch = next(iter(train_dataloader))
encoded = Encoder(in_channels=1)(batch)
batch.shape, encoded.shape

(torch.Size([64, 1, 28, 28]), torch.Size([64, 16]))
```

Этот код завершает нашу модель энкодера, которая преобразует изображения в промежуточные представления. Теперь перейдем к декодеру.

Декодер

Декодер начинает работу с латентного представления, полученного энкодером (векторы с 16 измерениями), и превращает его в изображения исходного размера.

Архитектура декодера не обязательно должна быть обратной по отношению к энкодеру. Она может быть представлена чем угодно, что «понимает» представления и может переводить их в изображения. В нашем случае мы создадим более-менее симметричную относительно энкодера сеть: применим *транспонированные свертки* для увеличения разрешения, одновременно уменьшая количество каналов, пока не достигнем желаемого выходного разрешения 28×28 пикселей⁶.

Добавим транспонированные свертки с линейным слоем для создания тензоров из 16×384 ($1024 \times 4 \times 4$) пикселей. Форма линейного слоя будет преобразована в разрешение 4×4 ($[1024, 4, 4]$), которое послужит входом для первой транспонированной свертки. После этого мы постепенно уменьшим каналы и увеличим разрешение, пока не достигнем исходной формы изображения. Есть и другие способы добиться того же, но помните, что входные данные — это плоский вектор с 16 каналами, а выходные данные должны состоять из 1 канала и 28×28 пикселей.

Реализуем декодер.

```
def conv_transpose_block(
    in_channels,
    out_channels,
    kernel_size=3,
    stride=2,
```

⁶ Транспонированная свертка работает так же, как и обычная. Но вместо входных 2D-данных она применяется к их увеличенной версии (двумерные входные данные заполняются нулями между строками и столбцами). Это приводит к формированию выходной 2D-матрицы, которая становится больше входной после применения фильтра.

```

padding=1,
output_padding=0,
with_act=True,
):
    modules = [
        nn.ConvTranspose2d(
            in_channels,
            out_channels,
            kernel_size=kernel_size,
            stride=stride,
            padding=padding,
            output_padding=output_padding,
        ),
    ]
    if with_act: # Controlling this will be handy later
        modules.append(nn.BatchNorm2d(out_channels))
        modules.append(nn.ReLU())
    return nn.Sequential(*modules)

class Decoder(nn.Module):
    def __init__(self, out_channels):
        super().__init__()
        self.linear = nn.Linear(
            16, 1024 * 4 * 4
        ) # note it's reshaped in forward
        self.t_conv1 = conv_transpose_block(1024, 512)
        self.t_conv2 = conv_transpose_block(512, 256, output_padding=1)
        self.t_conv3 = conv_transpose_block(256, out_channels, output_padding=1)
    def forward(self, x):
        bs = x.shape[0]
        x = self.linear(x) # (bs, 1024*4*4)
        x = x.reshape((bs, 1024, 4, 4)) # (bs, 1024, 4, 4)
        x = self.t_conv1(x) # (bs, 512, 7, 7)
        x = self.t_conv2(x) # (bs, 256, 14, 14)
        x = self.t_conv3(x) # (bs, 1, 28, 28)
        return x

decoded_batch = Decoder(x.shape[0])(encoded)
decoded_batch.shape

torch.Size([64, 1, 28, 28])

```

Обучение

На текущий момент мы создали класс `Encoder`, который уменьшает размерность входных данных, и класс `Decoder`, который реконструирует представления до исходного изображения. Помимо того, что оба эти компонента инициализируются слу-

чайными весами, в данный момент между собой они не связаны. Нам нужно обучить их совместно, чтобы они оба понимали одни и те же латентные представления.

Для этого создадим класс `AutoEncoder`, который последовательно будет передавать входные данные через энкодер и декодер. Мы обучим его минимизировать разницу между восстановленным изображением на выходе и исходным. Если нам это удастся, вывод будет похож на ввод.

Этот процесс важен, поскольку он позволяет сжимать данные, но он станет еще полезнее, когда мы сможем использовать оба компонента по отдельности после обучения. Это позволит нам делать много интересных вещей в следующих главах. Например, мы сможем использовать энкодер для преобразования произвольных изображений в их сжатые представления, которые затем могут использоваться в качестве входных данных другими моделями. Мы также сможем использовать декодер для генерации новых изображений, похожих на те, что есть в обучающем наборе.

Начнем обучение нашего `AutoEncoder`.

```
class AutoEncoder(nn.Module):
    def __init__(self, in_channels):
        super().__init__()
        self.encoder = Encoder(in_channels)
        self.decoder = Decoder(in_channels)

    def encode(self, x):
        return self.encoder(x)

    def decode(self, x):
        return self.decoder(x)

    def forward(self, x):
        return self.decode(self.encode(x))

model = AutoEncoder(1)
```

Мы можем использовать библиотеку `torchsummary`, чтобы вывести суммарную информацию о модели. Она покажет количество параметров и выходную форму каждого слоя. Этот инструмент полезен для проверки, что модель определена правильно, а также для понимания ее архитектуры.

```
import torchsummary

torchsummary.summary(model, input_size=(1, 28, 28), device="cpu")
```

```
-----
Layer (type)          Output Shape          Param #
-----
      Conv2d-1         [-1, 128, 14, 14]      2,176
    BatchNorm2d-2      [-1, 128, 14, 14]       256
        ReLU-3         [-1, 128, 14, 14]         0
-----
```

Conv2d-4	[-1, 256, 7, 7]	524,544
BatchNorm2d-5	[-1, 256, 7, 7]	512
ReLU-6	[-1, 256, 7, 7]	0
Conv2d-7	[-1, 512, 3, 3]	2,097,664
BatchNorm2d-8	[-1, 512, 3, 3]	1,024
ReLU-9	[-1, 512, 3, 3]	0
Conv2d-10	[-1, 1024, 1, 1]	8,389,632
BatchNorm2d-11	[-1, 1024, 1, 1]	2,048
ReLU-12	[-1, 1024, 1, 1]	0
Linear-13	[-1, 16]	16,400
Encoder-14	[-1, 16]	0
Linear-15	[-1, 16384]	278,528
ConvTranspose2d-16	[-1, 512, 7, 7]	4,719,104
BatchNorm2d-17	[-1, 512, 7, 7]	1,024
ReLU-18	[-1, 512, 7, 7]	0
ConvTranspose2d-19	[-1, 256, 14, 14]	1,179,904
BatchNorm2d-20	[-1, 256, 14, 14]	512
ReLU-21	[-1, 256, 14, 14]	0
ConvTranspose2d-22	[-1, 1, 28, 28]	2,305
BatchNorm2d-23	[-1, 1, 28, 28]	2
ReLU-24	[-1, 1, 28, 28]	0
Decoder-25	[-1, 1, 28, 28]	0

=====

Total params: 17,215,635

Trainable params: 17,215,635

Non-trainable params: 0

Input size (MB): 0.00

Forward/backward pass size (MB): 2.86

Params size (MB): 65.67

Estimated Total Size (MB): 68.54

С помощью следующего кода мы создадим простой цикл обучения, который будет многократно проходить по обучающим данным и использовать постоянную скорость обучения. Чтобы сосредоточиться на главном, мы не будем делать запуск проверок на тестовом наборе (но вам рекомендуется сделать это для практики). Мы будем использовать популярную библиотеку `tqdm`, чтобы отобразить прогресс, но не станем описывать ее здесь для краткости (больше примеров мы увидим в других главах). Не обязательно понимать все, что происходит, но обратите внимание на некоторые высокоуровневые операции:

1. Загрузка пакета из класса `DataLoader`.
2. Получение прогнозов модели.
3. Расчет потерь относительно исходных изображений.
4. Выполнение оптимизации (`optimizer`) для обновления весов модели.

```

import torch
from matplotlib import pyplot as plt
from torch.nn import functional as F
from tqdm.notebook import tqdm, trange

from genaibook.core import get_device

num_epochs = 10
lr = 1e-4

device = get_device()
model = model.to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=lr, eps=1e-5)

losses = [] #Список для хранения значений потерь для построения графиков
for _ in (progress := trange(num_epochs, desc="Training")):
    for _, batch in (
        inner := tqdm(enumerate(train_dataloader), total=len(train_dataloader))
    ):
        batch = batch.to(device)

        #Проходим через модель и получаем реконструированные изображения
        preds = model(batch)

        #Сравниваем прогноз с исходными изображениями
        loss = F.mse_loss(preds, batch)

        #Отображаем потери и сохраняем для построения графика
        inner.set_postfix(loss=f"{loss.cpu().item():.3f}")
        losses.append(loss.item())

        #Обновляем параметры модели с помощью оптимизатора optimizer
        #на основе потерь
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
    progress.set_postfix(loss=f"{loss.cpu().item():.3f}", lr=f"{lr:.0e}")

```

Построим кривую потерь, чтобы увидеть, как прошло обучение (рис. 3.7).

```

plt.plot(losses)
plt.xlabel("Step")
plt.ylabel("Loss")
plt.title("AutoEncoder - Training Loss Curve")
plt.show()

```

Мы не проводили проверку во время цикла обучения, но мы можем увидеть, как AutoEncoder справляется с тестовым набором. Использование визуальных данных — отличный способ обучения, поскольку мы можем наблюдать конечные результаты.

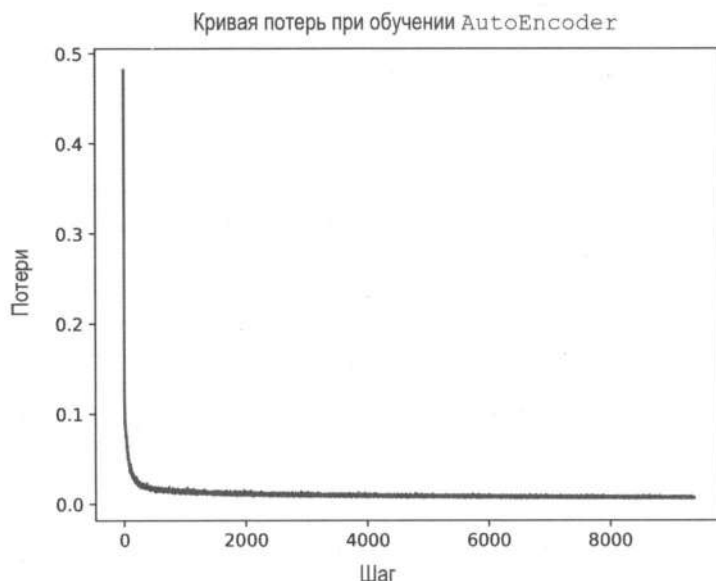


Рис. 3.7. Кривая потерь при обучении автоэнкодера

Мы создадим пакет с 16 выборками из тестового набора, пропустим их через обученные энкодер и декодер, а затем отобразим результат. Также мы создадим оценочный класс `DataLoader`.

```
eval_bs = 16
eval_dataloader = DataLoader(mnist["test"]("image"), batch_size=eval_bs)
```

Мы используем `model.eval()`, чтобы перевести модель в режим оценки (в нашем случае он отключит обновления `BatchNorm`), и менеджер контекста `inference_mode`, чтобы отключить вычисление градиента.

```
model.eval()
with torch.inference_mode():
    eval_batch = next(iter(eval_dataloader))
    predicted = model(eval_batch.to(device)).cpu()
```

Теперь, когда у нас есть прогнозы, выведем исходные изображения и их восстановленные версии (рис. 3.8).

```
batch_vs_preds = torch.cat((eval_batch, predicted))
show_images(batch_vs_preds, imsize=1, nrow=2)
```

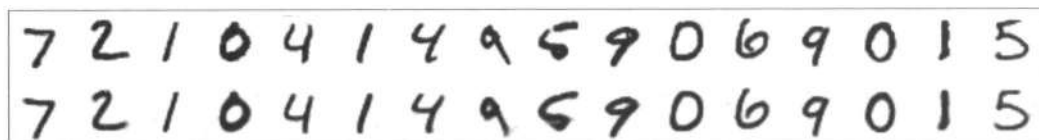


Рис. 3.8. Сравнение исходных изображений, которые мы загрузили в энкодер, и их восстановленные версии, которые выдал декодер

Результаты выглядят довольно хорошо. Первая строка представляет исходные изображения MNIST, а вторая — реконструированные версии из нашего автоэнкодера. Помните, что числа во второй строке являются приближениями оригиналов, полученными декодером из кратких векторных представлений.

На этом этапе вам необходимо убедиться, что вы поняли рассмотренные концепции. Попробуйте получить более высокие результаты восстановленных изображений. Вот несколько идей для экспериментов:

- ♦ Постепенно уменьшайте скорость обучения.
- ♦ Попробуйте разные размеры пакетов.
- ♦ Используйте сигмоидальную функцию в конце декодера, чтобы сделать конечные значения пикселей либо черными, либо белыми. Обратите внимание, что входные данные находятся в диапазоне от 0 до 1, поэтому сигмоидальный вывод должен соответствовать этому диапазону.
- ♦ Поиграйте с глубиной или топологией сети.

Мы также предлагаем поэкспериментировать с циклом обучения: добавьте и запишите в логи оценку потерь во время обучения.

Исследование латентного пространства

Одним из важных гиперпараметров автоэнкодера является количество измерений, которые используются для представления закодированных входных данных. Мы произвольно выбрали 16, и результаты показывают, что этого достаточно для представления широкого спектра нарисованных чисел.

В нашем следующем эксперименте мы будем использовать всего два измерения для представления векторов в латентном пространстве. Мы запрограммируем энкодер сжать как можно больше информации о входных изображениях всего в два числа с плавающей точкой и выясним, достаточно ли этого для восстановления исходных данных. Кроме того, использование двух измерений очень удобно для визуализации, поскольку после обучения новой модели мы сможем нарисовать несколько интересных графиков в двумерном пространстве, которые помогут лучше понять то, что мы делаем.

Мы немного реорганизуем наш код, внося следующие изменения:

- ♦ Включим размерность латентного пространства в качестве гиперпараметра.
- ♦ Используем контейнер (`nn.Sequential`) для сверточных слоев, чтобы упростить настройку глубины сети, если мы захотим поэкспериментировать с этим позже.
- ♦ Заменяем функцию активации после окончательной свертки декодера сигмоидальной функцией. Нам нужно, чтобы декодер производил либо белые, либо черные пиксели. Сигмоидальная функция активации подходит для этой цели лучше, чем `ReLU`. Это связано с тем, что она сжимает вывод до диапазона (0, 1). Это тот же диапазон, что имеют пиксели на изображениях.

Также это отличный момент, чтобы поместить цикл обучения внутрь функции.



Не пытайтесь создать код с множеством опций и параметров изначально. Лучше начать с простого рабочего кода и постепенно дополнять его по мере необходимости.

На рис. 3.9 показан график, который отображает разницу между работой ReLU и сигмоидальной функцией. Используя сигмоиду в качестве функции активации, мы можем гарантировать, что вывод после каждого слоя будет лежать в том же диапазоне, что используют входные изображения, — $(0, 1)$. Это не является строго необходимым условием для обучения сети, мы просто пытаемся помочь ей, поскольку знаем желаемый выходной диапазон.

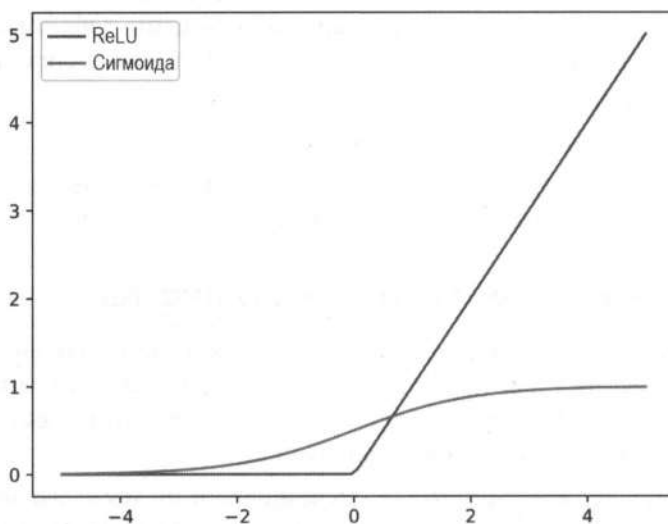


Рис. 3.9. Сравнение функций активации ReLU и сигмоиды

Переработанный код выглядит следующим образом.

```
class Encoder(nn.Module):
    def __init__(self, in_channels, latent_dims):
        super().__init__()

        self.conv_layers = nn.Sequential(
            conv_block(in_channels, 128),
            conv_block(128, 256),
            conv_block(256, 512),
            conv_block(512, 1024),
        )

        self.linear = nn.Linear(1024, latent_dims)

    def forward(self, x):
        bs = x.shape[0]
        x = self.conv_layers(x)
        x = self.linear(x.reshape(bs, -1))
        return x
```

```

class Decoder(nn.Module):
    def __init__(self, out_channels, latent_dims):
        super().__init__()

        self.linear = nn.Linear(latent_dims, 1024 * 4 * 4)
        self.t_conv_layers = nn.Sequential(
            conv_transpose_block(1024, 512),
            conv_transpose_block(512, 256, output_padding=1),
            conv_transpose_block(
                256, out_channels, output_padding=1, with_act=False
            ),
        )
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        bs = x.shape[0]
        x = self.linear(x)
        x = x.reshape((bs, 1024, 4, 4))
        x = self.t_conv_layers(x)
        x = self.sigmoid(x)
        return x

```

Новая версия класса `AutoEncoder` почти идентична предыдущей: она вызывает декодер на выходах энкодера. Класс просто принимает дополнительный аргумент `latent_dims`, чтобы мы могли указать желаемую размерность представлений.

```

class AutoEncoder(nn.Module):
    def __init__(self, in_channels, latent_dims):
        super().__init__()
        self.encoder = Encoder(in_channels, latent_dims)
        self.decoder = Decoder(in_channels, latent_dims)

    def encode(self, x):
        return self.encoder(x)

    def decode(self, x):
        return self.decoder(x)

    def forward(self, x):
        return self.decode(self.encode(x))

```

Цикл обучения остался прежним, но мы поместили его внутрь функции, чтобы использовать повторно и вызывать при необходимости.

```

def train(model, num_epochs=10, lr=1e-4):
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, eps=1e-5)

    model.train() # Put model in training mode
    losses = []

```

```

for _ in (progress := trange(num_epochs, desc="Training")):
    for _, batch in (
        inner := tqdm(
            enumerate(train_dataloader), total=len(train_dataloader)
        )
    ):
        batch = batch.to(device)

        #Проходим через модель и получаем другой набор изображений
        preds = model(batch)

        #Сравниваем результат с исходными изображениями
        loss = F.mse_loss(preds, batch)

        #Отображаем потери и сохраняем их для построения графика
        inner.set_postfix(loss=f"{loss.cpu().item():.3f}")
        losses.append(loss.item())

        #Обновляем параметры модели с помощью оптимизатора optimizer
        #на основе потерь
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()

    progress.set_postfix(loss=f"{loss.cpu().item():.3f}", lr=f"{lr:.0e}")
return losses

```

Обучение класса AutoEncoder происходит всего с помощью двух латентных переменных (рис. 3.10):

```

ae_model = AutoEncoder(in_channels=1, latent_dims=2)
ae_model.to(device)

losses = train(ae_model)

plt.plot(losses)
plt.xlabel("Step")
plt.ylabel("Loss")
plt.title("Training Loss Curve (two latent dimensions)")
plt.show()

```

Для сравнения мы еще раз загрузим обученную модель и посмотрим на некоторые изменения (рис. 3.11).

```

ae_model.eval()
with torch.inference_mode():
    eval_batch = next(iter(eval_dataloader))
    predicted = ae_model(eval_batch.to(device)).cpu()

batch_vs_preds = torch.cat((eval_batch, predicted))
show_images(batch_vs_preds, imsize=1, nrows=2)

```

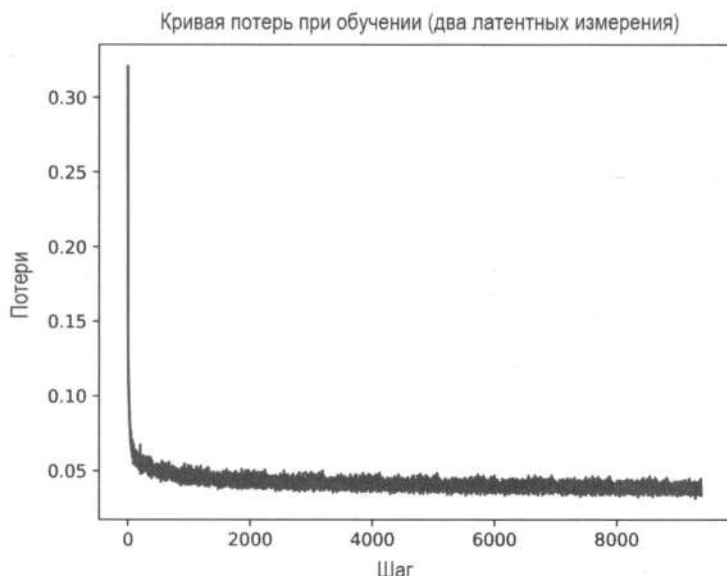



Рис. 3.10. График обучения AutoEncoder всего с двумя латентными измерениями

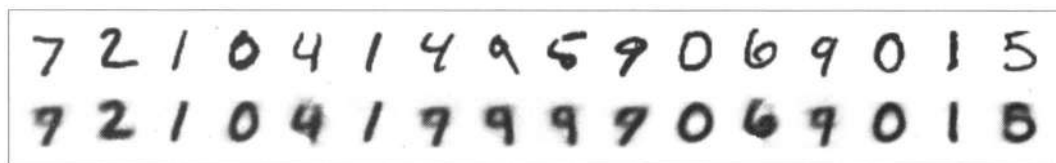


Рис. 3.11. Сравнение входных изображений с их восстановленными версиями после прохода через модель, которая использует всего 2 измерения (2 числа с плавающей точкой) для представления данных

Результаты не так хороши, как раньше, но следует помнить, что теперь мы используем всего два числа с плавающей точкой для представления рукописных изображений размером 28×28 . Можно увидеть некоторую путаницу с числами «4», «5» и «9», но в целом восстановленные изображения очень похожи на исходные.

Визуализация латентного пространства

Мы использовали всего два измерения для латентного пространства, чтобы визуализировать его структуру. Теперь визуализируем все закодированные векторы из тестового набора данных, используя столбец `label` для назначения разных цветов каждому классу (рис. 3.12). Первое значение закодированных векторов будет отображаться на оси X , а второе — на оси Y .

```
images_labels_dataloader = DataLoader(mnist["test"], batch_size=512)
import pandas as pd

df = pd.DataFrame(
    {
        "x": [],
```

```

    "y": [],
    "label": [],
  }
}

for batch in tqdm(
    iter(images_labels_dataloader), total=len(images_labels_dataloader)
):
    encoded = ae_model.encode(batch["image"].to(device)).cpu()
    new_items = {
        "x": [t.item() for t in encoded[:, 0]],
        "y": [t.item() for t in encoded[:, 1]],
        "label": batch["label"],
    }
    df = pd.concat([df, pd.DataFrame(new_items)], ignore_index=True)

plt.figure(figsize=(10, 8))

for label in range(10):
    points = df[df["label"] == label]
    plt.scatter(points["x"], points["y"], label=label, marker=".")

plt.legend();

```

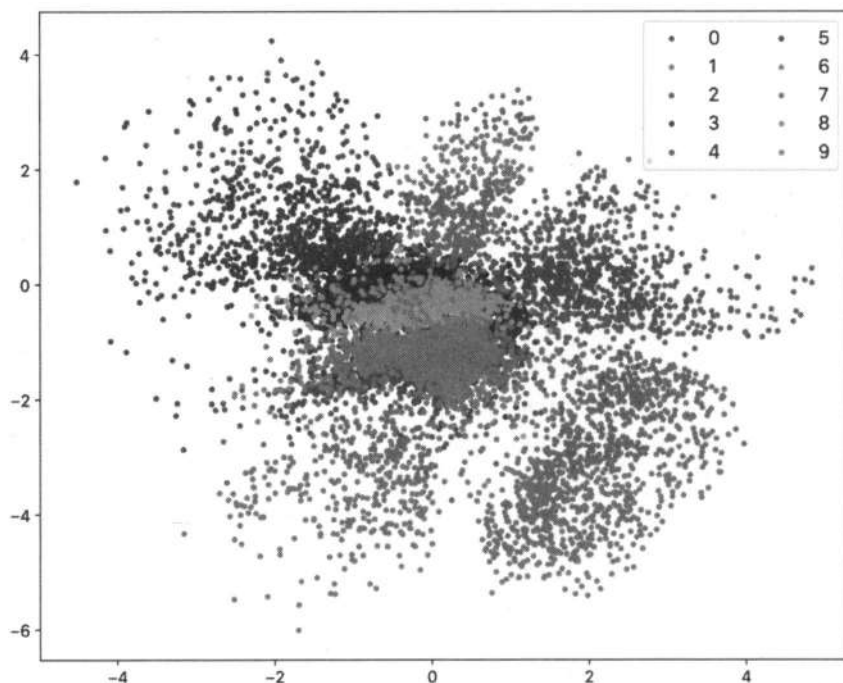


Рис. 3.12. Визуализация латентного пространства

AutoEncoder хорошо справился с разделением различных областей латентного пространства для каждого изображения в нашем наборе данных. Обратите внимание, изображения, представляющие число 0 (темно-синие точки), находятся далеко от представлений числа 1 (оранжевые точки). Во время обучения мы не использовали информацию о метках изображений, но даже в этом случае точки данных автоматически группировались в различных областях в соответствии с их визуальными особенностями. Также стоит отметить, что этот процесс происходил совершенно без ограничений, поэтому нет никаких гарантий относительно формы или структуры латентного пространства.

Таким образом, латентное пространство достаточно обширно, чтобы захватить релевантные признаки изображений в нашем наборе данных, но все еще не ясно, как мы можем использовать его в генеративных целях. В идеале, чтобы генерировать новые изображения, похожие на оригиналы, нам следует отказаться от энкодера и предоставить декодеру случайные выборки из латентного пространства. Однако мы можем увидеть некоторые проблемы на графике:

- ◆ Пространство, занимаемое представлением, разбросано во всех направлениях.
- ◆ В центре много пересечений и больших областей пустого пространства.
- ◆ График несимметричен: отрицательные значения по оси Y используются чаще, чем положительные.

Это затрудняет выбор релевантных областей в латентном пространстве, которые могли бы привести к отличным генерациям. Рассмотрим пример генерации изображения с помощью декодера. Начнем с генерации случайных латентных выборок (обычно обозначаются z):

```
N = 16 #Генерируем 16 точек
z = torch.rand((N, 2)) * 8 - 4
```

Затем мы визуализируем сгенерированные латентные выборки, наложенные поверх представления латентного пространства, которое мы показывали ранее (рис. 3.13).

```
plt.figure(figsize=(10, 8))

for label in range(10):
    points = df[df["label"] == label]
    plt.scatter(points["x"], points["y"], label=label, marker=".")

plt.scatter(z[:, 0], z[:, 1], label="z", marker="s", color="black")
plt.legend();
```

В завершение декодер сгенерирует изображения из только что созданных нами латентных выборок (рис. 3.14).

```
ae_decoded = ae_model.decode(z.to(device))
show_images(ae_decoded.cpu(), imsize=1, nrows=1, suptitle="AutoEncoder")
```

Сгенерированные изображения разумно использовать там, где выборки близки к одной из областей, выделенных моделью в латентном пространстве. Но когда они лежат за пределами этих областей, то они не так убедительны. Также обратите

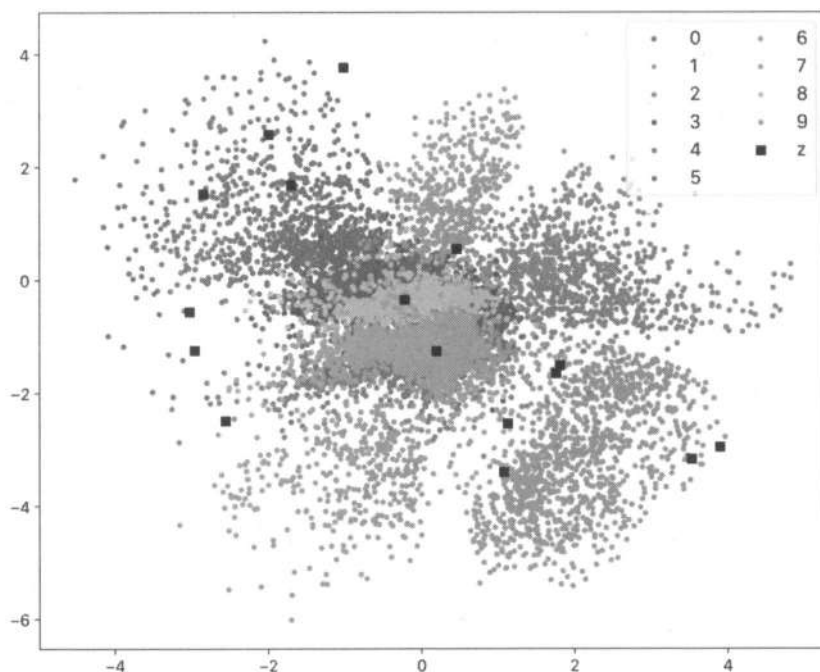


Рис. 3.13. Визуализация сгенерированных латентных выборок, наложенных поверх представления латентного пространства



Рис. 3.14. Изображения, сгенерированные из созданных латентных выборок

внимание, что некоторые числа будут представлены заново, поскольку назначенные им области в латентном пространстве больше по размеру.

В следующем подразделе мы обсудим, как можно использовать другой тип автоэнкодера, чтобы навести порядок в латентном пространстве, и как это может упростить генерацию.

Прежде чем двигаться дальше, рассмотрите несколько упражнений, которые вы можете выполнить сейчас (или в конце главы), чтобы закрепить свое понимание концепции:

1. Насколько хорошо работает генерация, если модель обучена с 16 латентными измерениями?
2. Обучите модель снова с теми же параметрами, которые мы использовали ранее (просто запустите код, показанный в главе), но с другой инициализацией случайных чисел⁷ и заново визуализируйте латентное пространство. Есть вероятность, что формы и структура будут другими. Это ожидаемо? Почему?

⁷ Вы можете использовать `torch.manual_seed(num)`, чтобы указать начальное значение.

3. Насколько хороши признаки изображений, извлеченные энкодером? Отбросьте часть с декодером и постройте классификатор чисел поверх энкодера. Обучите пару линейных слоев с использованием нелинейности между ними. Последний линейный слой должен выводить вектор с 10 измерениями, представляющими 10 меток в наборе данных. Обучите только эти слои, не обновляя веса энкодера. Какую точность вы можете получить? Сравните модель с 16 латентными измерениями с моделью, которая имеет всего два измерения.

Вариационные автоэнкодеры

В предыдущем разделе мы рассмотрели, как простой автоэнкодер может изучать эффективные представления входных данных в латентном пространстве меньшей размерности. Автоэнкодер может верно кодировать и восстанавливать (декодировать) любую информацию. Он отлично подходит для извлечения признаков или представления имеющихся данных, но не для генерации новых.

Как обсуждалось ранее, причина этого кроется в том, что автоэнкодер не умеет разделять представления на согласованные части латентного пространства. Как мы уже могли убедиться, представления схожих входных данных обычно сгруппированы близко друг к другу, но есть значительное количество пересечений и пустых областей, а также существенная изменчивость в объеме латентного пространства, отведенного каждому классу. Если мы выберем случайную точку в латентном пространстве и пропустим ее через декодер, мы не сможем точно предсказать, какой результат получится.

Вариационные автоэнкодеры (Variational AutoEncoder, VAE) решают эту проблему, изучая распределение вероятностей для каждого признака в латентном пространстве. Вместо сопоставления входных данных с определенными точками VAE представляют каждую функцию с помощью *гауссовского распределения*⁸, фиксируя изменчивость этого признака в данных.

Рассмотрим набор, состоящий из изображений нескольких пород собак и кошек. Мы не знаем, какие признаки будут извлечены энкодером, но мы могли бы представить, что некоторые из них будут использоваться для представления таких характеристик, как пятна на шерсти, глаза, уши, ноги или хвосты. Признаки могут в значительной степени перекрываться среди всех изображений в наборе (у всех этих животных два уха, четыре лапы и хвост), но также они имеют некоторую изменчивость в том, как выглядят уши у собак по сравнению с тем, как они выглядят у кошек. Признак, отображающий, например, ухо, можно представить с помощью гауссовского распределения, которое охватывает всю изменчивость, при этом среднее значение распределения представляет собой среднюю форму уха животного. Если мы отойдем от среднего в разных направлениях, то получим непрерывный

⁸ Распределение Гаусса, также называемое *нормальным распределением*, имеет форму колоколообразной кривой, в которой большинство точек сгруппировано вокруг среднего значения и гораздо меньше — в крайних точках.

и однородный переход к различным формам ушей, которые могут появляться у разных пород.

Такой подход (рис. 3.15) создает более структурированное латентное пространство, где выборка из распределений позволяет нам генерировать новые, правдоподобные экземпляры. Как и в случае с автоэнкодерами, информация о классе обычно не используется в VAE⁹. Посмотрим, как мы можем закодировать и обучить их.

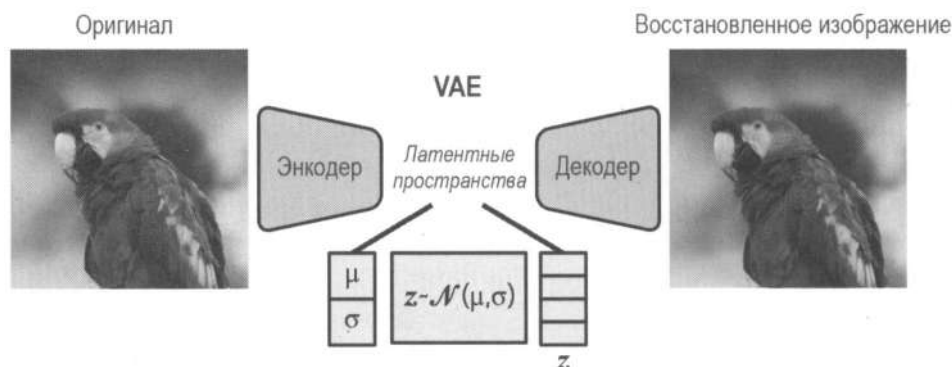


Рис. 3.15. VAE изучают гауссовские представления признаков и описывают их через средние значения и дисперсии; точки z в скрытом пространстве выбираются из предсказанных гауссовских распределений

Энкодеры и декодеры VAE

Энкодеры VAE очень похожи на базовые энкодеры из предыдущего раздела. В нашем примере мы использовали несколько сверточных слоев и один линейный, чтобы спроектировать желаемый размер латентного представления.

Для создания нашего первого энкодера VAE мы будем использовать ту же архитектуру. Единственное отличие — вместо линейного слоя для прогнозирования латентного пространства изображения мы будем использовать линейные слои для изучения распределения. Распределение характеризуется двумя параметрами — *средним значением* и *дисперсией*, поэтому нам понадобятся два линейных слоя¹⁰.

С точки зрения кода вот как выглядит наш первый энкодер VAE.

```
class VAEEncoder(nn.Module):
    def __init__(self, in_channels, latent_dims):
        super().__init__()

        self.conv_layers = nn.Sequential(
            conv_block(in_channels, 128),
```

⁹ Семейство VAE, которое называется *условные вариационные автоэнкодеры* (Conditional Variational Autoencoder, C-VAE), использует информацию о классе для дальнейшего разделения распределений в латентном пространстве, сохраняя при этом гауссовское представление признаков. Это упрощает генерацию выборок, напоминающих конкретный класс, который нас интересует.

¹⁰ На самом деле, технически это не совсем дисперсия, и вскоре мы это увидим.

```

        conv_block(128, 256),
        conv_block(256, 512),
        conv_block(512, 1024),
    )

    #Определяем полностью связанные слои для среднего значения
    #и логарифмической дисперсии
    self.mu = nn.Linear(1024, latent_dims)
    self.logvar = nn.Linear(1024, latent_dims)

    def forward(self, x):
        bs = x.shape[0]
        x = self.conv_layers(x)
        x = x.reshape(bs, -1)
        mu = self.mu(x)
        logvar = self.logvar(x)
        return (mu, logvar)

```

Если сравнить этот фрагмент кода с примером энкодера из предыдущего раздела, различия будут минимальными. Мы используем два линейных слоя вместо одного, чтобы вычислить два разных значения из одного и того же представления, извлеченного сверточными слоями, и возвращаем значения в метод `forward()`.

Оба слоя представляют среднее значение и дисперсию распределения вероятностей. Однако изначально это просто два идентичных линейных слоя. Задача состоит в том, чтобы они научились представлять то, что мы хотим в процессе обучения.

Перед тем как начать, обратите внимание, что вычисленное значение `mu` представляет среднее значение, а `logvar` — логарифм дисперсии. Мы используем `mu`, поскольку именно греческая буква μ часто используется в математической нотации для обозначения среднего значения нормального распределения. Вместо прямого вывода дисперсии мы используем ее логарифм (`logvar`), поскольку это дает некоторую «численную устойчивость» (мы объясним это позже).

Что касается декодера, нам не нужно вносить в него никаких изменений. Разница между VAE и простым автоэнкодером заключается в способе нахождения точки в латентном пространстве для представления входного элемента. Но цель декодера остается прежней: при заданной точке (z) показать пиксели, кодированное представление которых наиболее похоже на z . В случае автоэнкодера z является линейной проекцией признаков, извлеченных сверточными слоями. Когда мы используем энкодер VAE, мы получаем нормальное распределение, а затем делаем из него выборку, чтобы получить z . Следовательно, мы можем использовать тот же класс `Decoder` из предыдущего раздела, но нам нужно немного изменить модель, чтобы сделать выборку из распределения.

Выборка из распределения энкодера

После модификации энкодер VAE будет возвращать среднее значение и дисперсию нормального распределения, которые соответствуют представлениям входных дан-

ных. Чтобы получить декодированный вывод, мы должны сделать выборку из этого распределения, как показано в следующем фрагменте.

```
class VAE(nn.Module):
    def __init__(self, in_channels, latent_dims):
        super().__init__()
        self.encoder = VAEEncoder(in_channels, latent_dims) # (1)
        self.decoder = Decoder(in_channels, latent_dims)

    def encode(self, x):
        #Возвращаем mu, log_var
        return self.encoder(x)

    def decode(self, z):
        return self.decoder(z)

    def forward(self, x):
        #Получаем параметры нормального (гауссовского) распределения
        mu, logvar = self.encode(x) # (2)

        #Делаем выборку из распределения
        std = torch.exp(0.5 * logvar) # (3)
        z = self.sample(mu, std) # (4)

        #Декодируем латентную точку в пиксельное пространство
        reconstructed = self.decode(z) # (5)

        #Возвращаем восстановленное изображение, а также mu и logvar
        #чтобы мы могли вычислить потерю распределения
        return reconstructed, mu, logvar # (6)

    def sample(self, mu, std):
        #Ре-параметризация
        #Выборка из N(0, I), перевод и масштабирование
        eps = torch.randn_like(std) # (7)
        return mu + eps * std
```

Поясним моменты, обозначенные цифрами:

- ❶ Мы используем измерения `latent_dims` для представления среднего значения и логарифмической дисперсии распределений.
- ❷ Энкодер вычисляет две переменные: среднее значение и логарифмическую дисперсию.
- ❸ Вычисляем стандартное отклонение от логарифмической дисперсии.
- ❹ Делаем выборку из распределения с использованием вычисленных среднего значения и стандартного отклонения.
- ❺ Декодер преобразует выборку в изображение.

- ❻ Мы возвращаем не только реконструированное изображение, но и среднее значение и логарифмическую дисперсию.
- ❼ Мы выполняем «трюк репараметризации»: делаем выборку из стандартного нормального распределения, а затем переводим ее и масштабируем.

На текущий момент мы использовали общие понятия, чтобы объяснить, как работают VAE. Сейчас мы сделаем небольшой экскурс и коснемся пары статистических концепций, которые позволят пересмотреть предыдущий код с использованием более точной терминологии. Можете смело пропустить этот раздел или вернуться к нему позже. Для применения или обучения VAE необязательно понимать их, но это может быть полезно, если вы хотите углубиться в тему и изучить больше информации по ней.

Во-первых, обратите внимание, что мы применяем многомерные гауссовские распределения, а не только действительные одномерные нормальные кривые. В нашем примере мы использовали переменную `latent_dims` как для среднего значения, так и для дисперсии. Мы могли бы взять произвольное количество измерений, например 16, которые мы применяли для нашего первого автоэнкодера, или 2 для более простой визуализации. Действительное (одномерное) нормальное распределение обозначается как $N(\mu, \sigma^2)$, где μ — среднее значение распределения, а σ — стандартное отклонение, которое является квадратным корнем дисперсии σ^2 . Одной из полезных характеристик нормальных распределений является то, что все они могут быть выражены в терминах стандартного нормального распределения, среднее значение которого равно 0, а дисперсия — 1, путем перевода и масштабирования:

$$N(\mu, \sigma^2) = \mu + \sigma N(0, 1).$$

Это означает, что для получения выборки из произвольного нормального распределения $N(\mu, \sigma^2)$ мы можем сделать выборку из $N(0, 1)$, затем умножить на σ и добавить μ . Это называется трюком репараметризации, и он будет весьма полезен, когда мы рассмотрим модели диффузии.

Многомерные гауссовские распределения называются также *многовариационными*. Они по-прежнему могут быть определены двумя параметрами, с той лишь разницей, что μ — это вектор, а σ — это *ковариационная матрица*, теперь обозначаемая как Σ . Следовательно, распределение теперь можно определить как $N(\mu, \Sigma)$. Если распределение независимо во всех измерениях (каждая переменная не коррелирует с другими и имеет одинаковую дисперсию), оно называется *изотропным*. В изотропном многовариационном гауссовском распределении ковариационная матрица Σ является *диагональной*: в ней все элементы на диагонали равны и могут быть выражены как $\sigma^2 I$, где I — единичная матрица. Стандартное многовариационное гауссовское распределение затем можно выразить как $N(0, I)$.

Наш предыдущий пример VAE моделирует многовариационное изотропное гауссовское распределение, поскольку нет причин полагать, что координаты выборки зависят друг от друга (и так действительно проще). Это означает, что мы можем использовать трюк репараметризации, чтобы сделать выборку из стандартного распределения, а затем перевести ее и масштабировать, чтобы получить вектор латентного пространства, который мы будем декодировать.

Репараметризация используется не только для удобства, это важная часть обучения. Когда мы используем выражение $\mu + \text{eps} * \text{std}$, где eps — выборка из стандартного распределения, градиенты относительно входных данных модели не зависят от стохастического процесса (выборка из распределения) и, следовательно, могут быть вычислены. Это позволяет обучать модель с помощью знакомых методов градиентного спуска, которые мы используем для любой нейронной сети.

Наша модель предсказывает логарифм дисперсии вместо самой дисперсии, что повышает «численную устойчивость» и облегчает обучение. Математически нет никакой разницы, что из этого вычислять. Но на практике дисперсия всегда является положительным числом, обычно близким к 0. И у модели нет никаких оснований выдавать положительные малые значения, когда мы начинаем обучение. Кроме того, числа представлены в формате с плавающей точкой, что затрудняет распознавание значений, расположенных очень близко друг к другу. Взяв логарифм, мы получаем два преимущества:

- ♦ Мы расширяем диапазон приемлемых значений до $-\infty$, поэтому модель имеет гораздо больше свободы для выражения результатов с помощью значений с плавающей точкой.
- ♦ Мы гарантируем, что дисперсия всегда будет положительной, потому что она является экспонентой ($\log\text{var}$).



Распределения Гаусса часто используются во многих других областях ML. Они нужны для удобства, поскольку их математические характеристики хорошо известны. В главе 4 мы рассмотрим их использование, чтобы смоделировать искажения шума, что является неотъемлемой частью моделей диффузии.

Обучение VAE

Ключом к обучению VAE является *функция потерь* (**Loss function**). В разделе «Автоэнкодеры» функция потерь, которую мы использовали, измеряла разницу между восстановленными и исходными изображениями. Нам по-прежнему важно, чтобы восстановленные изображения максимально были похожи на оригиналы, но теперь мы введем второй фактор для потерь, чтобы наложить ограничение на VAE, о котором мы говорили: нам нужно, чтобы признаки (более или менее) следовали гауссовскому распределению.

Чтобы достичь этого, мы будем использовать *дивергенцию Кульбака–Лейблера* (**Kullback–Leibler Divergence, KLD**) между распределениями, также известную как *относительная энтропия*. Это способ измерить, насколько одно распределение вероятностей отличается от другого. В случае многовариационных изотропных гауссовских распределений KLD можно вычислить следующим образом:

$$D_{KL}[N(\mu, \sigma^2) \| N(0, 1)] = -\frac{1}{2} \sum (1 + \log(\sigma^2) - \mu^2 - \sigma^2).$$

Чтобы объединить два фактора потерь, мы создадим функцию `vae_loss()`, которая будет получать исходные изображения и выходные данные от энкодера и выполнять следующее:

- ◆ Вычислять потери при восстановлении как *среднеквадратичную ошибку (Mean Squared Error, MSE)* между пикселями, сгенерированными декодером, и исходными изображениями. Этот фактор потерь идентичен тому, который мы использовали для обучения автоэнкодеров.
- ◆ Вычислять KLD по уравнению, которое мы только что представили.
- ◆ Складывать оба предыдущих значения. Мы могли бы сделать одно из них больше, чтобы сбалансировать точность восстановления и соответствие гауссовскому распределению. Сейчас мы их суммируем, но поэкспериментировать с балансом — отличный сценарий, который стоит попробовать самостоятельно.

Функция потерь возвращает три значения: общие потери, потери восстановления и KLD. Для обучения нам нужны только общие потери, но мы также будем отслеживать и другие параметры для визуализации и анализа.

```
def vae_loss(batch, reconstructed, mu, logvar):
    bs = batch.shape[0]

    #Потери восстановления из пикселей — по 1 на изображение
    reconstruction_loss = F.mse_loss(
        reconstructed.reshape(bs, -1),
        batch.reshape(bs, -1),
        reduction="none",
    ).sum(dim=-1)

    #Потери KLD на входное изображение
    kl_loss = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp(), dim=-1)

    #Объединяем обе потери и получаем среднее значение по изображениям
    loss = (reconstruction_loss + kl_loss).mean(dim=0)

    return (loss, reconstruction_loss, kl_loss)
```

Теперь, когда потери определены, мы можем перейти к обучению модели. Мы будем использовать общую потерю, чтобы обновлять веса, но мы также будем отслеживать потерю восстановления и KLD, чтобы понять, как обучается модель.

```
def train_vae(model, num_epochs=10, lr=1e-4):
    model = model.to(device)
    losses = {
        "loss": [],
        "reconstruction_loss": [],
        "kl_loss": [],
    }

    model.train()
    optimizer = torch.optim.AdamW(model.parameters(), lr=lr, eps=1e-5)
    for _ in (progress := trange(num_epochs, desc="Training")):
```

```

for _, batch in (
    inner := tqdm(
        enumerate(train_dataloader), total=len(train_dataloader)
    )
):
    batch = batch.to(device)

    #Проходим через модель
    reconstructed, mu, logvar = model(batch)

    #Вычисляем потери
    loss, reconstruction_loss, kl_loss = vae_loss(
        batch, reconstructed, mu, logvar
    )

    #Отображаем потери и сохраняем их для построения графика
    inner.set_postfix(loss=f"{loss.cpu().item():.3f}")
    losses["loss"].append(loss.item())
    losses["reconstruction_loss"].append(
        reconstruction_loss.mean().item()
    )
    losses["kl_loss"].append(kl_loss.mean().item())

    #Обновляем параметры модели на основе общих потерь
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    progress.set_postfix(loss=f"{loss.cpu().item():.3f}", lr=f"{lr:.0e}")
return losses
vae_model = VAE(in_channels=1, latent_dims=2)
losses = train_vae(vae_model, num_epochs=10, lr=1e-4)

```

Проанализируем три вида потерь, которые мы сохранили во время обучения:

- ◆ *Потери восстановления* — показывают, насколько выходные изображения похожи на оригиналы.
- ◆ *KLD* — показывает, насколько хорошо признаки следуют гауссовскому распределению.
- ◆ *Общие потери* — сумма двух предыдущих значений.

Построим график потерь при обучении VAE (рис. 3.16).

```

for k, v in losses.items():
    plt.plot(v, label=k)
plt.legend();

```

Общая потеря доминирует над потерей восстановления, поскольку ее величина намного больше KLD. Построим график компонентов потерь отдельно, чтобы увидеть, как они изменяются во время обучения (рис. 3.17).

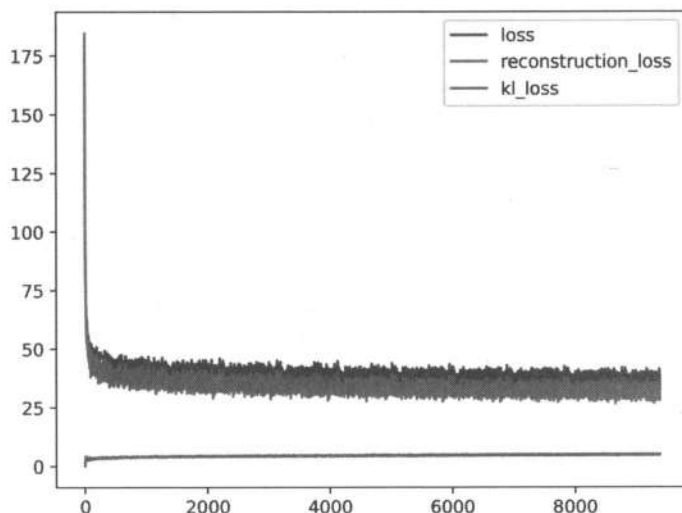


Рис. 3.16. График потерь (KLD, потери восстановления и общие потери)

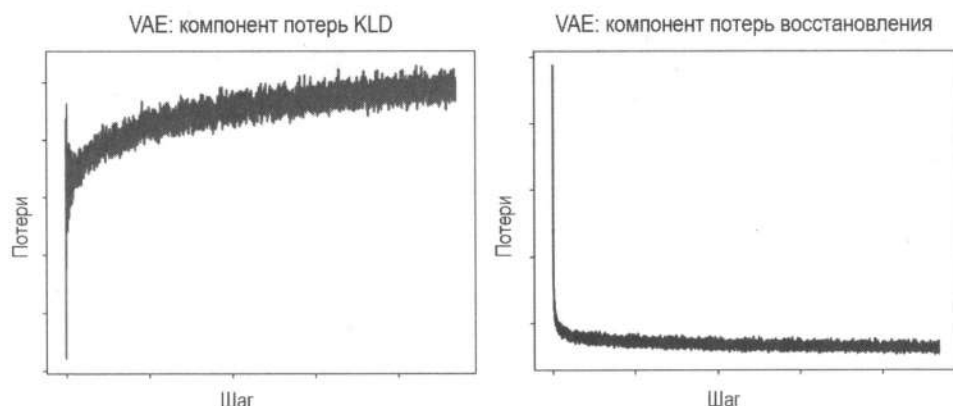


Рис. 3.17. График KLD-потерь и потерь восстановления во время обучения

Обратите внимание, график KLD имеет своеобразную форму. Он резко возрастает в начале обучения, затем уменьшается, а после снова медленно увеличивается. Почему так происходит? Мы считаем, что концептуально это фазы, через которые проходит KLD при обучении:

- ◆ Когда начинается обучение, энкодер и декодер VAE инициализируются случайными весами, и модель ничего не знает о входных данных. Поэтому ввод напоминает случайное распределение, следовательно, KLD очень низкий.
- ◆ Когда сеть увидела несколько первых пакетов данных, восстановление будет низкого качества, и уже не будет случайным, поэтому KLD резко возрастает.
- ◆ На ранних этапах обучения модель знает достаточно, чтобы представить средние характеристики входных данных. KLD заставляет модель выдавать выходные данные, которые становятся все больше и больше похожи на гауссовское распределение, уменьшая при этом KLD.

- ◆ Поскольку энкодер и декодер учатся выдавать более точные представления, становится сложнее улучшить их качество и при этом соответствовать гауссовскому распределению. Потери восстановления доминируют, а KLD увеличивается, но между ними есть баланс. Если бы мы увеличили «важность» KLD в потерях, мы могли бы достичь лучшего соответствия распределению Гаусса, но это привело бы к представлениям худшего качества.

Восстановим некоторые изображения из набора MNIST и визуализируем результаты (рис. 3.18).

```
vae_model.eval()
with torch.inference_mode():
    eval_batch = next(iter(eval_dataloader))
    predicted, mu, logvar = (v.cpu() for v in vae_model(eval_batch.to(device)))
batch_vs_preds = torch.cat((eval_batch, predicted))
show_images(batch_vs_preds, imsize=1, nrow=2)
```

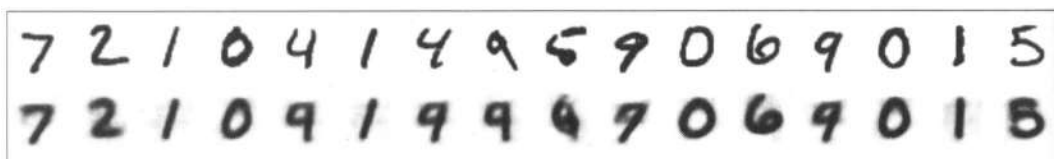


Рис. 3.18. Сравнение восстановленных изображений с оригиналами MNIST для модели VAE

Результаты хуже, чем у обычного автоэнкодера, потому что модель не только должна научиться кодировать входные изображения, но и быть ограничена в плане попыток избежать слишком большого отклонения от нормального распределения.

Если мы построим график средних значений нормального распределения, закодированного моделью для тестового набора, мы получим более качественный результат, чем в случае с автоэнкодером, как показано на рис. 3.19. Результаты теперь лучше центрированы вокруг нуля и не уходят так далеко, как в автоэнкодере. Области, занимаемые разными классами, имеют схожий размер, хотя все еще есть пересечения между похожими на вид числами.

```
df = pd.DataFrame(
    {
        "x": [],
        "y": [],
        "label": [],
    }
)

for batch in tqdm(
    iter(images_labels_dataloader), total=len(images_labels_dataloader)
):
    mu, _ = vae_model.encode(batch["image"].to(device))
    mu = mu.to("cpu")
    new_items = {
        "x": [t.item() for t in mu[:, 0]],
```

```

    "y": [t.item() for t in mu[:, 1]],
    "label": batch["label"],
}
df = pd.concat([df, pd.DataFrame(new_items)], ignore_index=True)

plt.figure(figsize=(10, 8))

for label in range(10):
    points = df[df["label"] == label]
    plt.scatter(points["x"], points["y"], label=label, marker=".")

plt.legend();

```

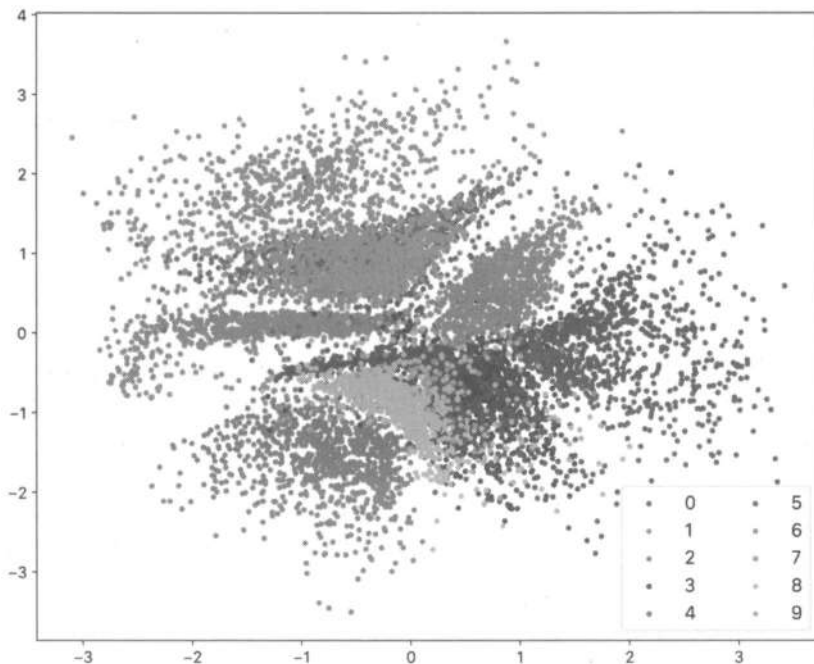


Рис. 3.19. График средних значений нормального распределения, закодированного моделью VAE для тестового набора

Главное преимущество, когда мы пытаемся подогнать энкодер к нормальному распределению, заключается в том, что теперь мы можем выбирать случайные данные из распределения и получать изображения, которые похожи на входной набор данных. Посмотрим, как это работает для автоэнкодера и VAE (рис. 3.20).

```

z = torch.normal(0, 1, size=(10, 2))
ae_decoded = ae_model.decode(z.to(device))
vae_decoded = vae_model.decode(z.to(device))

show_images(ae_decoded.cpu(), imsize=1, nrows=1)
show_images(vae_decoded.cpu(), imsize=1, nrows=1)

```

Мы выбираем чистые случайные данные из нормального распределения, а затем используем декодеры из AutoEncoder (верхний ряд) и VAE (нижний ряд), чтобы показать, как эти точки будут реконструированы. Конечно, мы увидим разные результаты, поскольку автоэнкодер и VAE обучались отдельно и выделяли разные части латентного пространства каждому классу.

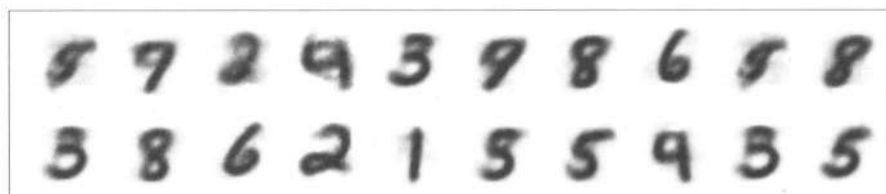


Рис. 3.20. Сравнение результатов работы автоэнкодера и VAE

Результаты от VAE больше похожи на числа, чем результаты автоэнкодера. Это связано с тем, что процесс обучения VAE побуждал энкодер не слишком сильно отклоняться от нормального распределения, в то время как у автоэнкодера такого ограничения не было. Запустите код выборки несколько раз и понаблюдайте за ним самостоятельно.

Попробуйте выполнить забавное упражнение: покажите, как представления меняются, когда мы перемещаемся в 2D-сетку через латентное пространство. Мы можем зафиксировать вертикальную линию в точке $x = -0,8$, как это сделано на рис. 3.21, и исследовать различные точки на этой линии.

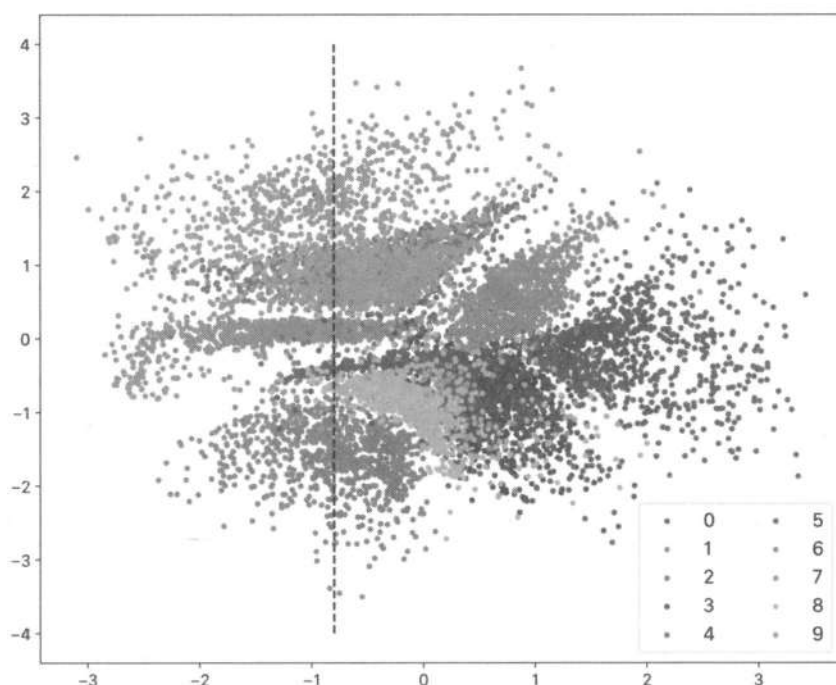


Рис. 3.21. Латентное пространство VAE, которое фокусируется на выборках при $x = -0,8$

Если мы выберем точки от $y=-2$ до $y=2$, то увидим, что реконструкции соответствуют областям латентного пространства, которые представляют различные числа (рис. 3.22).

```
import numpy as np

with torch.inference_mode():
    inputs = []
    for y in np.linspace(-2, 2, 10):
        inputs.append([-0.8, y])
    z = torch.tensor(inputs, dtype=torch.float32).to(device)
    decoded = vae_model.decode(z)
    show_images(decoded.cpu(), imsize=1, nrows=1)
```

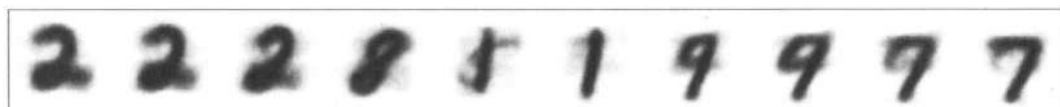


Рис. 3.22. В области от $y = -2$ до $y = 2$ видно, что реконструкции соответствуют областям латентного пространства

Расширим идею исследования латентного пространства до 2D-сетки (рис. 3.23). Это визуальное представление того, чему научилась модель (при этом интересно наблюдать, что переходы являются не слишком «безумными»)¹¹.

```
inputs = []
for x in np.linspace(-2, 2, 20):
    for y in np.linspace(-2, 2, 20):
        inputs.append([x, y])
z = torch.tensor(inputs, dtype=torch.float32).to(device)
decoded = vae_model.to(device).decode(z)
show_images(decoded.cpu(), imsize=0.4, nrows=20)
```

Вот несколько упражнений для погружения в эти темы:

1. Когда мы обучали VAE, мы добавили потери восстановления и KLD. Однако оба параметра имеют разные масштабы. Что произойдет, если мы придадим больше важности одному из них по сравнению с другим? Проведите несколько экспериментов и попробуйте объяснить результаты.
2. VAE, который мы исследовали в этом разделе, использует только два измерения для представления среднего значения и логарифма распределения. Попробуйте повторить подобное исследование, используя 16 измерений.
3. Люди способны легко определять нереалистичные черты лица. Попробуйте обучить автоэнкодер и VAE на наборе, который содержит лица, и проанализируйте

¹¹ Это довольно грубый способ показать изученное многообразие модели. Чтобы изучить лучший способ отображения этого результата и попробовать дополнительные эксперименты, мы рекомендуем вам ознакомиться с репозиторием `pytorch-mnist-vae` на GitHub от Джеки Лунга (Jackie Loong).

результаты их работы. Вы можете начать с набора *Frey Face* — это однородный набор монохромных изображений лица одного и того же человека с разными выражениями (мимикой). Также вы можете попробовать свои силы при обучении модели на наборе *CelebFaces Attributes*, который можно скачать с Hugging Face Hub. Еще один интересный пример — набор данных *Oxford pets*, также доступный в Hugging Face Hub.

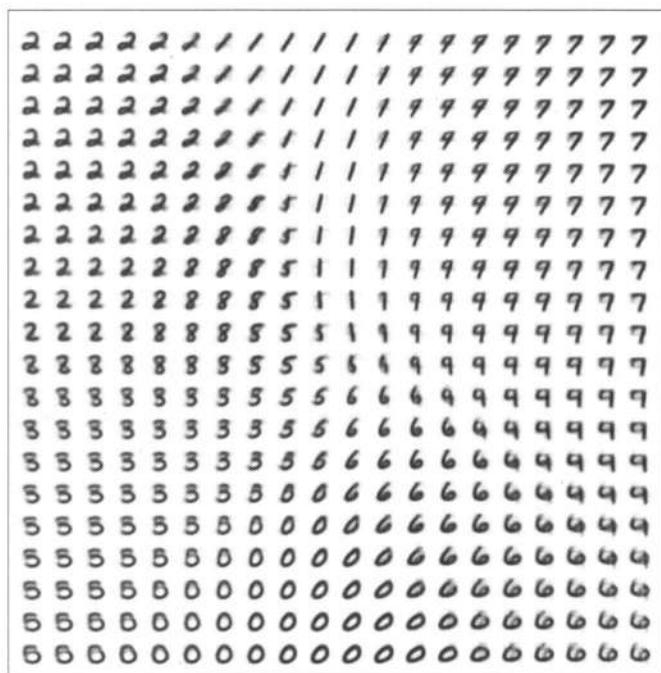


Рис. 3.23. Исследование латентного пространства в 2D-сетке

Использование VAE для генеративного моделирования

Обучение энкодера, ограниченного нормальным распределением, является ключевым пониманием VAE и одним из краеугольных камней генеративного моделирования. С помощью автоэнкодера мы можем изучать эффективные представления данных. Тем не менее нет никакой гарантии, что латентное пространство, изученное моделью, поможет нам генерировать новые изображения, которые напоминают исходные, будучи нацеленными на изучение распределений, VAE позволяют генерировать новые правдоподобные изображения, просто работая со случайными точками в латентном пространстве. Модели диффузии продвигают идею выборки из случайного шума на шаг дальше, включая в процесс итеративное уточнение. Мы подробно обсудим это в следующих главах.

CLIP

До этого момента мы фокусировались на данных, представленных только изображениями. С помощью нейросети *CLIP* (**C**ontrastive **L**anguage-**I**mage **P**re-training) мы можем отойти от автоэнкодеров и VAE и рассмотреть технику сопоставления изображений с текстом, которая похожа на методы, которые мы изучили ранее в этой главе. Так же как в автоэнкодерах и VAE, мы создаем обширные представления из входных данных, но метод CLIP отличается в том плане, что он может работать как с изображениями, так и с текстом одновременно.

Наш набор данных теперь будет состоять из двух модальностей: изображений и текстовых описаний, характеризующих эти изображения. CLIP-модели могут измерять, насколько точно текст описывает содержимое изображения для произвольной пары «текст-изображение», которую мы предоставляем. Ключевым моментом, как обычно, является функция потерь.

Контрастные потери

Модель CLIP была представлена компанией OpenAI в 2021 году. Изначально она была частью инструментария, разработанного для создания DALL-E — модели преобразования текста в изображение, которая покорила мир. Несмотря на то что она была закрытой (веса модели все еще остаются закрытыми), CLIP, наоборот, имела открытый исходный код. Это была необычная новость, поскольку возможность связывать изображения с текстом открывала впечатляющий функционал. С тех пор CLIP и подобные ей модели стали незаменимыми инструментами во многих областях, например в *генеративном ландшафте*.

CLIP использует функцию потерь, которая называется *контрастной потерей* (**Contrastive loss**). Механизм ее работы схематически показан на рис. 3.24. Набор обучающих данных состоит из миллионов изображений с соответствующими описаниями. Для каждой пары «изображение-текст» мы кодируем изображение с помощью любого энкодера, чтобы получить вектор эмбедингов в латентном пространстве. Каждое из полей на рисунке ($I_1, I_2, \dots, I_N$) представляет собой векторы эмбедингов для разных изображений. Текст также кодируется, обычно с помощью трансформера, подобного тем, что мы видели в главе 2. Важно то, что мы используем энкодеры таким образом, чтобы размеры векторов для изображений и текста были одинаковыми. Поэтому мы можем вычислить внутреннее произведение (скалярное произведение) между эмбедингами текста и изображений, чтобы определить, насколько они близки.

Обучение осуществляется с помощью большого количества пар «изображение-текст» в одном пакете. Мы вычисляем скалярные произведения всех эмбедингов изображений в одном пакете со всеми эмбедингами текста в другом пакете и пытаемся максимизировать произведение элементов, изначально составляющих одну пару (синяя диагональ на рис. 3.24), минимизируя остальные. Таким образом, похожие тексты и изображения будут представлены векторами, близкими друг к другу

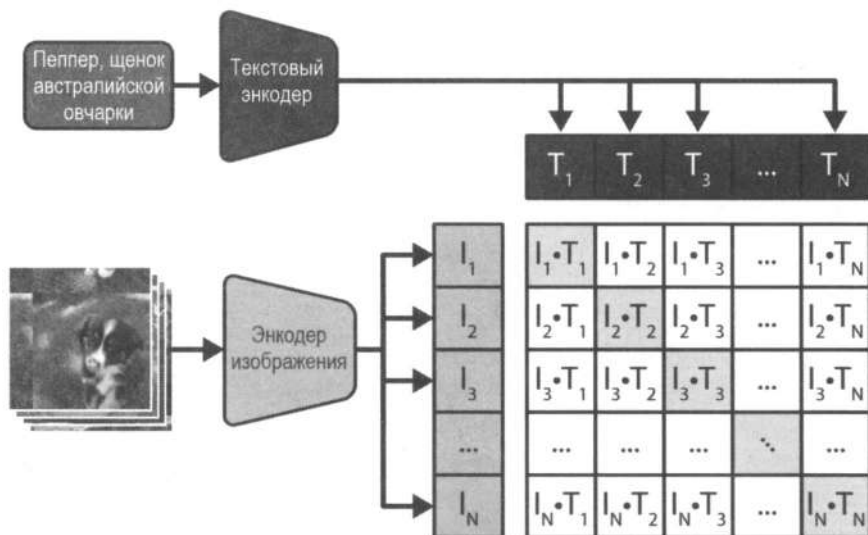


Рис. 3.24. CLIP (адаптировано из изображения OpenAI)

в латентном пространстве, в то время как разные пары будут находиться в разных его областях.

Зачем использовать скалярное произведение?

Если вы не изучали математический анализ ранее, то в векторных пространствах действует следующее важное соотношение:

$$A \cdot B = |A| |B| \cos(\theta).$$

Выражение означает, что скалярное произведение двух векторов равно произведению их длин на угол между ними. Это можно показать, используя закон косинусов из евклидовой геометрии и определение скалярного произведения как $A \cdot B = \sum_{i=1}^N a_i b_i$. В нашем обсуждении мы не упомянули, что векторные эмбединги нормализованы к единичной длине, поэтому их скалярное произведение равно углу между векторами. Это называется *косинусным сходством*, и оно определяет близость между двумя векторами. Несмотря на то что векторы имеют много измерений, скалярное произведение является простым скаляром (т. е. действительным числом), который можно использовать в качестве оценки сходства.

Обучение CLIP требует огромных объемов данных и большого количества вычислений. Изначальные модели, выпущенные OpenAI, использовали собственный набор данных из 400 миллионов пар «изображение-текст» и большие размеры пакета (32 000 пар). С тех пор было предпринято множество попыток обучить как сами CLIP, так и им подобные модели, включая поддержку дополнительных модальностей вроде аудио:

♦ OpenCLIP

Это открытая реализация CLIP. Она использовалась для обучения нескольких моделей с различными наборами данных, разрешениями изображений и размерами моделей.

◆ CLAP (Contrastive Language-Audio Pretraining)

Позволяет получать представления аудио вместо изображений. Ее можно использовать для обучения моделей, которые генерируют аудиоданные (рассмотрим в главе 9).

Использование CLIP, шаг за шагом

Далее мы воспользуемся предварительно обученной моделью, чтобы получить представление о том, как работают CLIP, и увидеть несколько сценариев, где их можно применять. В следующем примере мы задействуем `clip-vit-large-patch14`, одну из моделей компании OpenAI. Она использует *ViT (Vision Transformer)* в качестве энкодера изображений (существуют и другие версии, использующие сверточную архитектуру ResNet). Также существуют вариации модели разного размера. Вы можете поэкспериментировать с некоторыми из них, чтобы увидеть, как они работают на вашем оборудовании с учетом таких характеристик, как скорость, память и качество.

Рассмотрим фотографию на рис. 3.25 (взята из бесплатного ресурса Pixabay), на которой изображен очаровательный львенок, смотрящий на нас из-за ветки дерева. Посмотрим, как мы можем использовать ее вместе с CLIP.



Рис. 3.25. Фотография милого львенка, который находится за веткой

Пока что мы использовали трансформеры только для загрузки текста, но их можно использовать для работы и с другими данными. Аналогично использованию `GPT2LMHeadModel` из предыдущей главы, мы попробуем применить `CLIPModel`.

```
import requests
from PIL import Image
from transformers import CLIPModel, CLIPProcessor

from genaibook.core import SampleURL

clip = CLIPModel.from_pretrained("openai/clip-vit-large-patch14").to(device)
processor = CLIPProcessor.from_pretrained("openai/clip-vit-large-patch14")
```

```
url = SampleURL.LionExample
image = Image.open(requests.get(url, stream=True).raw)
```

Загруженная модель clip содержит два компонента: модель зрения для кодирования изображений и текстовую модель для кодирования текста. Обработчик CLIPProcessor, который можно рассматривать как эквивалент токенизатора, подготавливает входные данные, чтобы они соответствовали этапам предварительной обработки, используемым во время обучения (изменение размера изображения, нормализация и т. д.). Это необходимо для того, чтобы входные данные, которые мы предоставляем для вывода, имели те же характеристики, что и данные, которые модель видела во время обучения. Сначала необходимо обработать изображение.

```
image_inputs = processor(images=image, return_tensors="pt")
pixel_values = image_inputs["pixel_values"]
pixel_values.shape, pixel_values.min(), pixel_values.max()

(torch.Size([1, 3, 224, 224]), tensor(-1.7923), tensor(2.0179))
```

Изображение было приведено к форме квадрата размером 224×224 пикселей и нормализовано. Вы можете проверить обработчик изображений на предмет полного набора примененных преобразований. А именно мы использовали *центральное кадрирование* (выбрали блок квадратной формы вокруг центра изображения, а затем уменьшили его до размера 224×224 пикселей). Этот метод используется как для пейзажей (обрезается левая и правая части изображения), так и для портретов (обрезается верхняя и нижняя части). Помните, если некоторые из объектов, которые вы хотите исследовать, находятся в обрезаемых областях, это может привести к потере информации.

```
processor.image_processor

CLIPImageProcessor {
  "crop_size": {
    "height": 224,
    "width": 224
  },
  "do_center_crop": true,
  "do_convert_rgb": true,
  "do_normalize": true,
  "do_rescale": true,
  "do_resize": true,
  "image_mean": [
    0.48145466,
    0.4578275,
    0.40821073
  ],
  "image_processor_type": "CLIPImageProcessor",
  "image_std": [
    0.26862954,
    0.26130258,
    0.27577711
  ],
}
```

```

"resample": 3,
"rescale_factor": 0.00392156862745098,
"size": {
    "shortest_edge": 224
}
}

```

Проверим, выдержит ли наша фотография львенка центральное кадрирование (рис. 3.26).

```

width, height = image.size
crop_length = min(image.size)

left = (width - crop_length) / 2
top = (height - crop_length) / 2
right = (width + crop_length) / 2
bottom = (height + crop_length) / 2

cropped = image.crop((left, top, right, bottom))
cropped

```



Рис. 3.26. Изображение львенка после применения центрального кадрирования

Объект на фотографии полностью сохранен. В зависимости от данных вам может потребоваться обрезать исходные изображения до передачи их в обработчик, чтобы обеспечить видимость объекта.

Далее мы получим вектор эмбедингов из предварительно обработанных изображений. Для этого мы будем использовать модель зрения, сохраненную внутри экземпляра `clip`. Этот подкомпонент иногда называют *башней зрения*.

```

with torch.inference_mode():
    output = clip.vision_model(pixel_values.to(device))
    image_embeddings = output.pooler_output
    image_embeddings.shape

torch.Size([1, 1024])

```

Башня зрения возвращает словарь с последними латентными состояниями и выводом пулера (**Pooler**). Этот вектор с формой `[1, 1024]` представляет собой результат процесса кодирования.

Далее рассмотрим языковую часть модели. Мы выполним аналогичный процесс, чтобы получить эмбединги для двух текстовых промптов: «фотография льва» и «фотография зебры». Наша цель — сравнить косинусное сходство между эмбедингами изображений и эмбедингами каждого промпта. Описание льва должно лучше соответствовать изображению.

```
prompts = [
    "a photo of a lion",    # Изображение льва
    "a photo of a zebra",  # Изображение зебры
]

#Отступ гарантирует, что все входные данные будут иметь одинаковую длину
text_inputs = processor(text=prompts, return_tensors="pt", padding=True)

{'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1],
                           [1, 1, 1, 1, 1, 1, 1]]),
 'input_ids': tensor([[49406, 320, 1125, 539, 320, 5567, 49407],
                      [49406, 320, 1125, 539, 320, 22548, 49407]])}
```

Обработка текста токенизирует входные строки так же, как в предыдущей главе. Теперь мы можем передать токенизированные текстовые входные данные через компонент языковой башни модели, чтобы получить эмбединги промпта.

```
text_inputs = {k: v.to(device) for k, v in text_inputs.items()}

with torch.inference_mode():
    text_output = clip.text_model(**text_inputs)

text_embeddings = text_output.pooler_output
text_embeddings.shape

torch.Size([2, 768])
```

На выходе мы получаем два вектора, используя пакет из двух входных промптов. Однако каждый вектор имеет 768 измерений, в то время как эмбединги изображения имеют 1024 измерения. Помните, для вычисления скалярного произведения двух векторов они должны иметь одинаковое количество измерений. Поэтому нам потребуется дополнительный этап, о котором мы не упоминали ранее. Вместо того чтобы выбирать модели энкодеров, которые создают одинаковые измерения, мы можем взять произвольные энкодеры для текста и изображения, а затем вычислить проекцию на векторы с одинаковой размерностью. Эти проекции уже исследованы в процессе обучения CLIP и являются частью оболочки модели `clip`, которую мы скачали ранее.

В этом случае изученные проекции представляют собой просто линейные слои, которые сопоставляют свои входы с векторами с 768 измерениями. Таким образом,

у нас уже есть одна проекция для компонента текста, а другая — для компонента зрения.

```
print(clip.text_projection)
print(clip.visual_projection)

Linear(in_features=768, out_features=768, bias=False)
Linear(in_features=1024, out_features=768, bias=False)

with torch.inference_mode():
    text_embeddings = clip.text_projection(text_embeddings)
    image_embeddings = clip.visual_projection(image_embeddings)
text_embeddings.shape, image_embeddings.shape
(torch.Size([2, 768]), torch.Size([1, 768]))
```

Мы почти готовы вычислить косинусные сходства. Нам просто нужно помнить об использовании нормализованных векторов с единичными длинами, чего мы можем достичь с помощью масштабирования наших эмбеддингов и деления их на соответствующие значения. Затем мы можем вычислить два скалярных произведения одновременно, используя умножение матриц.

```
text_embeddings = text_embeddings / text_embeddings.norm(
    p=2, dim=-1, keepdim=True
)
image_embeddings = image_embeddings / image_embeddings.norm(
    p=2, dim=-1, keepdim=True
)

similarities = torch.matmul(text_embeddings, image_embeddings.T)
similarities

tensor([[0.2171],
        [0.1888]], device='cuda:0')
```

Во время обучения модель интерпретирует косинусные сходства как логиты, которые затем подаются в классификатор потерь *перекрестной энтропии* (**Cross-entropy**). Он, в свою очередь, предсказывает метку для каждой пары «изображение-текст». Поскольку при обучении использовались пакеты размером 32, метки представляют собой каждую из 32 768 позиций в пакете. Если вы снова посмотрите на изображение в начале раздела, то увидите, что после обучения модели процесс перекрестной энтропии выберет класс T_1 для I_1 , класс T_2 для I_2 и т. д.

Однако есть еще один момент. Поскольку векторы нормализованы, логиты могут находиться только в диапазоне $[-1, 1]$, который из-за ограничений формата с плавающей точкой может не иметь достаточного динамического диапазона для захвата распределений вероятностей по категориям 32 тысяч элементов. Авторы модели использовали обучаемый параметр *температуры* (*temperature*) для масштабирования логитов, чтобы иметь более широкий диапазон. Однако они также обрезали максимум шкалы до значения 100, чтобы обеспечить числовую стабильность. Во

всех обучающих запусках они обнаружили, что шкала всегда достигала максимального значения. Поэтому можно сделать вывод, что в выходных данных CLIP использует масштабный коэффициент, равный 100, перед тем, как интерпретировать логиты в качестве вероятностей.

Применим эту поправку к нашему предыдущему фрагменту кода, а затем преобразуем масштабированные логиты сходства в вероятности. Данные вероятности будут показывать, насколько модель уверена в том, что изображение соответствует одному из двух текстовых описаний, которые мы использовали.

```
similarities = 100 * torch.matmul(text_embeddings, image_embeddings.T)
similarities.softmax(dim=0).cpu()
```

```
tensor([[0.9441],
        [0.0559]])
```

Модель сопоставляет промпт «фотография льва» и изображение с уверенностью в 94,4%.

Классификация изображений с нулевым выстрелом с помощью CLIP

Процесс, которому мы следовали в предыдущем разделе, представляет собой подробное пошаговое руководство для решения задачи, которая называется *классификация с нулевым выстрелом*, с помощью CLIP. К счастью, библиотеки программного обеспечения, такие как `transformers` или `OpenCLIP`, предоставляют более высокие уровни абстракции, которые значительно упрощают процесс.

Почему это называется классификацией с нулевым выстрелом?

Классификация — одна из основных задач ML, в которой, имея точку данных и набор предопределенных классов, нужно оценить вероятность того, что точка соответствует одному из классов. Согласно этому определению набор классов должен быть зафиксирован заранее, поэтому обучаемая нами модель будет знать только об этих классах и ни о чем другом. Например, набор ImageNet содержит изображения, распределенные по 20 000 классов. В течение многих лет конкурс *ImageNet Large Scale Visual Recognition Challenge (ILSVRC)* был тестовым эталоном для систем компьютерного зрения, целью которого была классификация объектов, принадлежащих 1000 классам набора ImageNet. В 2012 году глубокая CNN, известная как AlexNet, легко выиграла соревнование того года. Это положило начало революции, когда показатели точности стали увеличиваться из года в год по мере разработки новых и лучших моделей глубокого обучения.

Классификация с нулевым выстрелом подразумевает собой способность модели правильно распределять данные без явного обучения. В *главе 2* мы наблюдали пример, где классифицировали отзывы с нуля, используя LM, и предыдущий раздел является еще одной яркой демонстрацией этой возможности. CLIP была обучена сопоставлять пары «изображение-текст», но мы также можем использовать такое поведение для классификации, если создадим описания, способные разумно характеризовать изображения. Например, если мы хотим классифицировать кошек и собак, наши промпты должны быть

примерно следующими: «Фото кошки» и «Фото собаки». Модель будет сопоставлять наилучший промпт для предоставленного нами изображения и выдавать результат классификации, который нам нужен.

Повторим пример из предыдущего раздела с API transformers. Загрузим модель CLIP, обработчик и тестовое изображение так же, как мы делали это и раньше.

```
clip = CLIPModel.from_pretrained("openai/clip-vit-large-patch14").to(device)
processor = CLIPProcessor.from_pretrained("openai/clip-vit-large-patch14")
```

```
image = Image.open(requests.get(SampleURL.LionExample, stream=True).raw)
```

Мы используем обработчик, чтобы вычислить входные данные для промптов изображения и текста одновременно. Мы также можем удобно вызвать CLIP-модель с полным набором входных данных для извлечения логитов (масштабированных косинусных сходств) между изображением и каждым промптом. Задействуем еще несколько промптов, чтобы сделать задачу интереснее.

```
prompts = [
    "a photo of a lion",
    "a photo of a zebra",
    "a photo of a cat",
    "a photo of an adorable lion cub",
    "a puppy",
    "a lion behind a branch",
]
inputs = processor(
    text=prompts, images=image, return_tensors="pt", padding=True
)
inputs = {k: v.to(device) for k, v in inputs.items()}

outputs = clip(**inputs)
logits_per_image = outputs.logits_per_image
probabilities = logits_per_image.softmax(dim=-1)

probabilities = probabilities[0].cpu().detach().tolist()

for prob, prompt in sorted(zip(probabilities, prompts), reverse=True):
    print(f"{100*prob: =2.0f}%: {prompt}")

89%: a photo of an adorable lion cub # Фото очаровательного львенка
9%: a lion behind a branch # Лев за веткой
2%: a photo of a lion # Фото льва
0%: a photo of a zebra # Фото зебры
0%: a photo of a cat # Фото кота
0%: a puppy # Щенок
```

Аналогичным способом мы можем предоставить несколько изображений и промптов в одном и том же пакете ввода и получить все вероятности классификации

одновременно. Попробуйте поисследовать и адаптировать данный пример к вашему варианту использования.

Конвейер классификации изображений с нулевым выстрелом

Теперь, когда вы знаете, как работает CLIP и как использовать ее для классификации изображений с нулевым выстрелом, рассмотрим API еще более высокого уровня для простоты и удобства. Мы применим абстракцию `pipeline`, которую уже рассматривали в *главе 1*. В том случае мы продемонстрировали, как использовать ее для задач классификации текста, но мы также можем применить ее ко многим другим задачам, включая классификацию изображений с нулевым выстрелом.

Чтобы создать экземпляр конвейера, нужно просто дать ему имя задачи (в данном случае `zero-shot-imageclassification`) и указать модель, которую хотим использовать.

```
from transformers import pipeline
classifier = pipeline(
    "zero-shot-image-classification",
    model="openai/clip-vit-large-patch14",
    device=device,
)
```

Конвейер позаботится обо всех деталях за нас: токенизация, предварительная обработка изображений и постобработка логитов. Нам просто нужно вызвать экземпляр конвейера с изображением, которое мы хотим классифицировать, и набором меток-кандидатов. Конвейер вернет словарь, который удобно отсортирован и содержит все оценки, связанные с предоставленными метками.

```
scores = classifier(
    image,
    candidate_labels=prompts,
    hypothesis_template="{}",
)
```

Аргумент `hypothesis_template` — это строка Python, которая применяется к каждой метке-кандидату, чтобы построить текстовый промпт для классификации. Если мы опустим этот аргумент, конвейер автоматически будет использовать промпт «This is a photo of a \{\}», что в принципе подходит для форматирования меток классов, указанных по их имени (например, "cat" или "lion"). Поскольку мы уже создали промпты, которые хорошо работают с CLIP, применим «\{\}», чтобы использовать метки нетронутыми.

```
{'label': 'a photo of an adorable lion cub',
 'score': 0.886413037776947},
{'label': 'a lion behind a branch', 'score': 0.09321863204240799},
{'label': 'a photo of a lion', 'score': 0.018809959292411804},
{'label': 'a photo of a zebra', 'score': 0.0011134858941659331},
{'label': 'a photo of a cat', 'score': 0.0004198708338662982},
{'label': 'a puppy', 'score': 2.4912407752708532e-05}}
```

Варианты использования CLIP

Изначальная задача, для решения которой была разработана CLIP, — это классификация изображений с нулевым выстрелом. Результаты впечатляют: модель достигает производительности, аналогичной конкурентам, которые были обучены для классификации изображений из набора ImageNet, без использования его меток при обучении. Как следствие, производительность остается одинаково высокой для многих других наборов данных, без необходимости их тонкой настройки.

Как мы уже показали в предыдущих разделах, основой для решения задач классификации изображений с нулевым выстрелом является возможность вычислять сходства между произвольным изображением и промптом. Это становится возможным, поскольку и изображения, и текстовые эмбединги охватывают основную семантику данных. Способ, согласно которому работает CLIP, позволил сообществу исследователей использовать его для многих других задач, а не только для классификации с нулевым выстрелом.

Возможность вычислить сходство между текстом и изображениями позволяет реализовать такие задачи, как *семантический поиск*, который позволяет находить фотографии на основе описаний их содержимого на естественном языке или изображения, похожие на предоставленный готовый пример. В *главе 2* мы представляли похожую задачу — создание FAQ-системы (часто задаваемые вопросы) с помощью вычисления сходств между эмбедингами выводов LM, — но она была ограничена текстовыми данными. В конце этой главы мы предлагаем решить задачу, которая задействует CLIP для семантического поиска.

CLIP также можно применять, чтобы получать расширенные эмбединги для последующих задач. Например, некоторые модели преобразования текста в изображение (text-to-image) используют CLIP, чтобы получить семантически насыщенные представления промптов, предоставленных пользователем.

Модель также подходит в качестве важного инструмента для различных сценариев генерации. Руководство, разработанное Райаном Мердоком (Ryan Murdock), Кэтрин Кроусон (Katherine Crowson) и другими, использует CLIP в качестве функции потерь, чтобы направлять градиенты модели к желаемому представлению (выраженному промптом). Этот метод породил «творческий взрыв» в сообществе генеративного моделирования. Позже обусловливание CLIP стало важным в таких моделях, как Stable Diffusion, которые мы рассмотрим в *главе 5*.

Возможности оценки, которые предоставляет CLIP, также используются для фильтрации и оценки массивных наборов данных «изображение-текст» в наборах вроде LAION. Использование этих моделей позволяет создателям наборов выбирать пары, в которых сходство между изображением и описанием дает высокую оценку, и отбрасывать остальные. Это позволяет создавать и совершенствовать наборы порядка миллиардов элементов, которые можно использовать для создания лучших моделей, способных еще точнее совершенствовать наборы.

Альтернативы CLIP

Поскольку CLIP широко используются в разных отраслях и исследованиях, многие исследователи активно пытаются сделать концепции, представленные в моделях этого типа, лучше и быстрее, а также адаптировать их к другим задачам. Со стороны ИИ-сообщества было приложено много усилий, чтобы сделать их исходный код открытым и легко воспроизводимым. Репозиторий OpenCLIP предоставляет открытый исходный код, который позволяет реализовать подобные модели, а также ряд других архитектур. Используя кодовую базу OpenCLIP и огромный набор данных, о котором мы упоминали в предыдущем разделе, команда LAION обучила очень мощные модели различных размеров и сделала все контрольные точки доступными для использования любым разработчиком.

Чем лучше модель понимает текст в парах «изображение-текст», тем лучше она работает. Например, BLIP, CoCa и CapPa демонстрируют, что для генерации описаний (подробных описаний того, что представляет собой изображение) могут создаваться модели, способные генерировать превосходные представления изображений и решать широкий спектр зрительно-языковых задач. Использование альтернативной функции потерь (сигмоидальная функция, используемая в SigLIP, вместо нормализации softmax) позволяет упростить обучение и устранить проблему, где модель требует огромных размеров пакета для эффективного обучения.

Другое многообещающее направление — создание меньших, более быстрых моделей, которые могут работать на персональных компьютерах и мобильных устройствах. Например, таким представителем является MobileCLIP от Apple, которая сопоставима с производительностью CLIP-моделей от OpenAI, имея при этом гораздо меньшие размеры. Еще один стартап Apple — Data Filtering Networks — призван улучшить качество наборов данных «изображение-текст», а также повысить качество обучения с его помощью.

Как вы увидите в оставшейся части этой книги, CLIP является важным компонентом систем генерации изображений. Исследование более надежных, эффективных и быстрых моделей, подобных ей, позволяет нам с большим оптимизмом смотреть в будущее.

Время проекта: семантический поиск изображений

Настало время реализовать интересный проект — семантическая поисковая система по локальной коллекции изображений. Она даст возможность искать фото в собственной библиотеке, просто описывая их содержимое (например, «собака прыгает в воду жарким летним днем» или «женщина с зонтиком идет по оживленной улице»), вместо того, чтобы пытаться самому вспомнить, где они хранятся. Для проекта вы можете применить любой другой набор данных с изображениями, который вам нравится, но использование контента, который что-то для вас значит, будет полезнее и позволит вам в полной мере оценить, насколько хорошо работает система.

Вот несколько шагов для выполнения проекта, которые мы предлагаем:

1. Выберите модель, которая может создавать эмбединги как для изображений, так и для текстовых описаний. Можно использовать и другие альтернативы, но CLIP будет отличным началом. Выберите семейство моделей, где в наличии есть экземпляры разных размеров. Так вы сможете их легко заменить друг на друга. Это даст вам возможность оценить в полной мере работу небольших моделей, которые способны работать быстрее, а также убедиться, насколько увеличивается производительность в более крупных моделях.
2. Найдите достаточное количество фотографий и скопируйте их в папку на вашем компьютере. Несколько сотен или тысяч экземпляров должно быть достаточно.
3. Напишите цикл для создания эмбедингов из ваших фотографий с использованием выбранной модели:
 - Считайте фотографии с диска. Обрежьте и/или измените их размер так, чтобы они были одинаковыми, и создайте пакет. Вы можете использовать класс `DataLoader` от PyTorch, как мы уже делали это для циклов обучения в данной главе. Предварительную обработку вы можете сделать вручную с помощью `torchvision.transforms` или использовать встроенный обработчик, если он существует. Выберите размер пакета, который подходит для вашего оборудования.
 - Пропустите каждый пакет через компонент модели, который отвечает за кодирование изображений (также известный как модель зрения или башня зрения). Используйте режим вывода (не нужно вычислять градиенты, т. к. вы ничего не будете обучать).
 - Получите эмбединги из выходных данных и сохраните их на диск. В результате у вас будет многомерный вектор для каждого файла изображения. Вы можете преобразовать их в массивы `numpy` и записать все в один файл. Не забудьте сохранить имена или пути к исходным фотографиям. Они понадобятся вам для извлечения фотографий позже.
4. На данном этапе у вас есть массив векторов в формате `numpy`. Теперь вы можете использовать текстовую часть модели для выполнения запросов:
 - Напишите функцию, которая получает входной промпт и генерирует вектор эмбедингов, используя текстовую башню модели.
 - Вычислите косинусное сходство между этим вектором и всеми векторами в таблице эмбедингов изображений. Если у вас достаточно оперативной памяти, вы можете сделать это, просто применив операцию `matmul` в PyTorch.
 - Отсортируйте результаты и выберите лучшие.
 - Найдите изображения, связанные с наилучшими баллами, и визуализируйте их. Соответствуют ли они промпту, который вы использовали?

Дополнительные задания:

- ♦ Попробуйте найти изображения, которые похожи на другие (т. е. выполните семантический поиск на основе входного изображения, а не текстового описания).

- ◆ Если у вас много фотографий, могут возникнуть проблемы с вычислением оценок. Как можно решить эту проблему? Существуют ли какие-либо фреймворки или сервисы, которые могут в этом помочь? Сколько фотографий нужно, чтобы достичь предела вычислительной мощности вашего компьютера?
- ◆ Предварительно обученные модели ничего не знают о субъектах, которые есть на ваших фотографиях. Что можно сделать, чтобы иметь возможность искать по именам или местам?
- ◆ Если вы любите авантюры, можете использовать MobileCLIP, чтобы запустить поисковую систему на своем телефоне. Это уже сам по себе большой вызов.

Заключение

Эта глава показала, каким образом можно применять сжатые представления входных данных для захвата основных характеристик элементов набора и как они могут быть эффективно использованы во многих дополнительных задачах. Вначале мы рассмотрели классические автоэнкодеры, которые служат для кодирования входных выборок в латентном пространстве с уменьшенной размерностью и последующего восстановления исходных точек данных из латентных представлений.

Используя компоненты автоэнкодера (энкодер и декодер) отдельно, мы можем создавать новые приложения, которые способны не только восстанавливать данные из представлений. Например, энкодер можно использовать в качестве инструмента, который способен извлекать признаки данных. Поскольку он умеет изучать основные характеристики входного набора, мы можем применять его для обучения других систем, таких как классификаторы, чьи входные данные являются латентными представлениями.

Мы также рассмотрели, как можно использовать декодер в генеративных целях. Если латентное пространство является представлением исходного набора данных, возникает вопрос, можем ли мы перейти к произвольным точкам в нем и посмотреть, какой вывод мы получим? Автоэнкодеры имеют некоторые ограничения при решении данной задачи из-за того, каким образом они обучаются.

Эти ограничения можно обойти с помощью VAE — особого вида автоэнкодеров, который старается достичь более качественных представлений в латентном пространстве. В таких моделях латентные признаки «стремятся» максимально соответствовать нормальному распределению вероятностей, благодаря чему мы можем сделать выборку из желаемого распределения, чтобы получить новые случайные латентные признаки. Если мы передадим эти признаки декодеру, мы сможем генерировать точки данных, которые будут выглядеть так, как будто они почти без изменений пришли из исходного набора. Это важный результат для генеративных приложений. Факт, что латентное пространство является компактным представлением данных, лежит в основе генеративных систем, таких как Stable Diffusion. Как мы увидим более подробно в *главе 5*, мы можем проводить вычисления в латентном пространстве, представленном изображениями, а не необработанными данными. Это позволяет обучить высококачественные системы генерации, которые способны работать быстро и эффективно на пользовательском оборудовании.

В завершение мы изучили CLIP — очень важную модель, разработанную и опубликованную компанией OpenAI. Она кодирует данные изображений и текста в одно и то же латентное пространство. Модель открывает новые возможности, без которых решение рассматриваемых задач было бы довольно сложным. Например, имея несколько предложений, можно измерить, какое из них лучше всего соответствует изображению. И наоборот, имея несколько изображений, мы можем выбрать то, которое лучше всего соответствует описанию. CLIP была опубликована как часть проекта по генерации изображений, который называется DALL-E и разработан также компанией OpenAI. Модель произвела революцию в генеративных исследованиях. Альтернативы, подобные CLIP, являются ключевыми компонентами Stable Diffusion и других моделей преобразования текста в изображение, но они применяются и для многих других приложений: поиск изображений на естественном языке, семантический поиск, извлечение изображений, похожих по содержанию или стилю, и многое другое.

Все это возникло из первоначального понимания, что изучение того, как сжимать данные, эквивалентно самому изучению данных. Многие вариации этой концепции используются в различных методах извлечения и представления данных: CNN, трансформеры или сочетание лучшего из них обоих с такими системами, как VQGAN. Мы лишь кратко изложили мотивы и идеи, лежащие в основе этих блоков, в надежде, что они окажутся полезными, чтобы сориентировать вас в этой обширной и интересной области.

Вопросы

Большинство из этих вопросов похоже на те, что мы рассмотрели в главе. В зависимости от вашего стиля обучения, вы можете ответить на них по мере прохождения главы или попробовать после прочтения всей книги:

1. Как работает генерация, если модель автоэнкодера обучена с 16 латентными измерениями? Можете ли вы сравнить генерации между моделью с 16 латентными измерениями и моделью всего с 2 измерениями?
2. Обучите модель снова с теми же параметрами, которые мы уже использовали (просто запустите код, показанный в главе), но с другой инициализацией случайных чисел и визуализируйте латентное пространство. Стоит ли ожидать, что формы и структура будут другими? Какова вероятность этого? Почему?
3. Насколько хороши признаки изображений, извлеченные энкодером? Исследуйте этот вопрос, обучив числовой классификатор «поверх» энкодера.
4. Когда мы обучали VAE, мы добавили потери восстановления и KLD. Однако обе функции имеют разные масштабы. Что произойдет, если мы придадим больше «важности» одной из них по сравнению с другой? Можете ли вы провести несколько экспериментов и объяснить результаты?
5. Модель VAE, которую мы обучили, использует только два измерения для представления среднего значения и логарифма распределения. Можете ли вы повторить подобное исследование, используя 16 измерений?

Вы можете найти решения и ответы на эти вопросы и решения последующих задач в репозитории книги на GitHub.

Задачи

1. Использование BLIP-2 для поиска.

Практический проект по семантическому поиску изображений довольно сложен, но вот вам еще одна идея для рассмотрения. Можно ли использовать модель BLIP-2 для задач на определение сходства, как мы делали это с CLIP в данной главе? Как бы вы это реализовали и как это соотносится с CLIP? Какие еще задачи можно решить с помощью BLIP-2?

2. Обучение собственного автоэнкодера или VAE.

Люди при рассмотрении лиц легко могут определить нереалистичные черты. Попробуйте обучить автоэнкодер и VAE для набора данных, который содержит лица, и посмотрите, как будут выглядеть результаты. Вы можете начать с набора данных *Frey Face*, который использовался в разделе по VAE. Это однородный набор монохромных лиц одного и того же человека с разными выражениями. Или, если вы хотите глубже продвинуться в решении этой задачи, можете попробовать свои силы на наборе данных *CelebFaces Attributes*, который также размещен на Hugging Face Hub. Еще одним интересным примером может стать набор данных с домашними животными Оксфорда, также доступный на Hugging Face Hub.

Модели диффузии

Генерация изображений стала широко популярна после того, как в 2014 году Иэн Гудфеллоу (Ian Goodfellow) представил *генеративно-сопоставительные сети* (**Generative Adversarial Net, GAN**). Ключевые идеи, заложенные в них, стали основой для дальнейшего развития и привели к появлению большого семейства моделей, которые могли быстро генерировать высококачественные изображения. Однако, несмотря на свой успех, при создании GAN-сетей возникали сложности, поскольку нужно было учитывать множество параметров, а также им требовалась помощь в эффективном решении задач генерализации. Такие ограничения привели к тому, что параллельно с GAN активно стали изучаться *модели диффузии* (**Diffusion Model**) — класс моделей, которые переопределили дальнейшую судьбу высококачественной генерации изображений.

В конце 2020 года модели диффузии вызвали ажиотаж в мире машинного обучения. Исследования показали, как можно использовать их для генерации изображений более высокого качества, чем могли это делать GAN. Затем последовал поток исследовательских работ, затрагивающих различные улучшения и модификации, которые еще больше повысили качество генераций. К концу 2021 года такие модели, как GLIDE, смогли продемонстрировать невероятные результаты в задачах преобразования текста в изображение (text-to-image). А спустя еще несколько месяцев они стали популярны наравне с DALL-E 2 и Stable Diffusion. Эти модели позволили любому человеку легко генерировать изображения, просто вводя текстовое описание того, что он хотел бы увидеть.

В этой главе мы подробно рассмотрим, как работают модели диффузии, и расскажем ключевые идеи, которые делают их такими мощными. Также мы сгенерируем изображения с помощью существующих доступных моделей, чтобы вы поняли, как они работают, а затем обучим свои собственные экземпляры, чтобы понять их еще лучше. Область генерации изображений быстро развивается, поэтому темы, затронутые здесь, дадут вам прочную основу, чтобы вы могли уверенно изучать ее дальше. Мы расширим и укрепим эти знания в *главах 5, 7 и 8*.

Если рассматривать на высоком уровне, концепция моделей диффузии заключается в следующем: на вход приходят изображения, размытые *шумом*, и модель учится его устранять, выводя четкое изображение. Набор обучающих данных, как правило, содержит изображения с разным количеством шума (на входе даже может быть чистый шум). Во время вывода модель сначала выдает изображение с чистым шумом, которое соответствует обучающему распределению. Затем она выполняет несколько итераций, корректируя себя, и в конце мы получаем генерации высокого качества.

Итеративное уточнение — ключ к пониманию моделей диффузии

Что делает модели диффузии такими мощными? Методы, рассмотренные ранее (VAE или GAN), позволяют генерировать вывод с помощью одного прямого прохода по всей цепочке преобразований данных в модели. Это означает, что конвейер должен делать все правильно с первой попытки. Если он совершит ошибку, то не сможет вернуться и исправить ее. Модели диффузии генерируют результат, используя множество итераций на разных шагах. Такое постоянное уточнение позволяет исправлять ошибки на предыдущих этапах и постепенно улучшать результат. Чтобы проиллюстрировать это, на рис. 4.1 показан пример модели диффузии в действии.



Рис. 4.1. Прогрессивный процесс шумоподавления

Загрузить предварительно обученную модель можно с помощью библиотеки `diffusers` из хранилища Hugging Face. Она предоставляет высокоуровневый конвейер, который можно использовать для создания изображений напрямую. Мы загрузим экземпляр `ddpm-celebahq-256`. Это одна из первых доступных моделей диффузии, генерирующих изображения. Она была обучена с помощью *CelebA-HQ* — популярного в то время набора высококачественных данных со знаменитостями. Поэтому изображения в выводе будут максимально на них похожи. С помощью этой модели мы сгенерируем изображение из шума (рис. 4.2).

```
import torch
from diffusers import DDIMPipeline

from genaibook.core import get_device

# Настраиваем устройство для работы с нашим GPU или CPU
device = get_device()

# Загружаем конвейер
image_pipe = DDIMPipeline.from_pretrained("google/ddpm-celebahq-256")
image_pipe.to(device)

# Образец изображения
image_pipe().images[0]
```



Рис. 4.2. Изображение, сгенерированное моделью ddpm-celebahq-256

Конвейер не показывает, что происходит «под капотом», поэтому мы углубимся в его механизм самостоятельно. При запуске кода вы можете заметить, что генерация заняла 1000 шагов. Это значит, что конвейер выполнил 1000 уточнений (и прямых проходов), чтобы получить финальное изображение. Один из главных недостатков моделей диффузии по сравнению с GAN — им требуется сделать много итераций, чтобы в результате получить изображение высокого качества, что делает их медленными.

Мы можем воссоздать данный процесс шаг за шагом, чтобы лучше понять, что происходит внутри. Вначале мы инициализируем нашу выборку x с помощью пакета из четырех случайных изображений (иначе говоря, мы задаем некоторый случайный шум). Затем мы выполняем 30 шагов, чтобы постепенно убрать шум на входном изображении, и в конце получаем выборку из реального распределения.

Сгенерируем несколько изображений. На левой части рис. 4.3 вы можете наблюдать входные данные на каждом шаге (начиная со случайного шума). Прогноз модели для конечных изображений находится на правой части рис. 4.3. Результаты на нулевом шаге (первая строка справа) пока не очень впечатляют. Поэтому мы немного изменяем значение x на входе на небольшую величину в направлении прогноза (уменьшаем шум) и снова пропускаем его через модель (шаг 10, левая часть рис. 4.3). В результате мы получаем вывод, который чуть лучше, чем на предыдущем шаге, и можем использовать его для следующего шага и т. д. Так постепенно мы делаем более качественный прогноз. При достаточном количестве шагов модель может создавать впечатляюще реалистичные изображения.

```
from genaibook.core import plot_noise_and_denoise
```

```
#Случайная начальная точка — это пакет из 4 изображений
```

```
#Каждое изображение является 3-канальным (RGB)
```

```
image = torch.randn(4, 3, 256, 256).to(device)
```

```
#Устанавливаем определенное кол-во шагов диффузии
```

```
image_pipe.scheduler.set_timesteps(num_inference_steps=30)
```

```

#Проходим по временным шагам выборки
for i, t in enumerate(image_pipe.scheduler.timesteps):
    #Получаем прогноз с учетом текущего значения x и временного шага t
    #Поскольку мы выполняем вывод, нам не нужно рассчитывать градиенты,
    #поэтому мы можем использовать функцию torch.inference_mode()
    with torch.inference_mode():
        #Нам нужно передать временной шаг t, чтобы модель знала, на каком
        #временном шаге она находится в данный момент
        #Мы узнаем об этом больше в следующих разделах
        noise_pred = image_pipe.unet(image, t)["sample"]

    #Вычисляем, как должен выглядеть обновленный x с помощью scheduler
    scheduler_output = image_pipe.scheduler.step(noise_pred, t, image)

    #Обновляем x
    image = scheduler_output.prev_sample

    #Периодически отображаем как x, так и прогнозируемые изображения
    #с уменьшением шума
    if i % 10 == 0 or i == len(image_pipe.scheduler.timesteps) - 1:
        plot_noise_and_denoise(scheduler_output, i)

```

Не волнуйтесь, если этот кусок кода выглядит пугающе. Мы объясним, как все это работает позже. Пока сосредоточьтесь на самой концепции.



Рис. 4.3. Пример генерации изображений по шагам

Модели, которые итеративно убирают шумы из входных данных, могут быть применены к широкому спектру задач. В этой главе мы сосредоточимся на *безусловной* (не ограниченной условиями) генерации изображений, похожих на обучающие данные. Например, мы можем обучить модель на наборе, который представлен фотографиями бабочек, чтобы она могла генерировать новые высококачественные данные. Она не сможет создавать изображения, отличные от элементов ее обучающего набора, поэтому не стоит ожидать, что она будет генерировать, например, динозавров.

В *главе 5* мы глубоко погрузимся в модели диффузии, обусловленные текстом, но также важно отметить, что мы можем применять их и к другим видам данных. Модели диффузии могут работать с аудио, видео, текстом, 3D-объектами, белковыми структурами и многим другим. Несмотря на то что большинство из них реализовано с использованием метода итеративного шумоподавления, существуют также другие подходы, например *искажение* (всегда используется в сочетании с итеративным уточнением).

Обучение моделей диффузии

В этом разделе мы обучим модель диффузии с нуля, чтобы лучше понять, как она работает. Начнем с применения компонентов из библиотеки *diffusers*. По мере продвижения мы постепенно «демонстрируем», как работает каждый из них.

Обучение модели диффузии является относительно простым процессом по сравнению с другими генеративными моделями. Чтобы реализовать это, мы должны много раз выполнить следующие процессы:

1. Загрузить несколько изображений из обучающих данных.
2. Добавить шум в разных количествах. Помните, мы хотим, чтобы модель могла хорошо оценить, как «исправить» (удалить шум) и очень шумные изображения, и изображения, близкие к идеальным. Поэтому нам нужен набор данных с разным количеством шума.
3. Подать входные данные вместе с шумом в модель.
4. Оценить, насколько хорошо модель справляется с шумоподавлением входных данных.
5. Использовать полученную информацию для обновления весовых коэффициентов модели.

Чтобы сгенерировать новые изображения, нам нужно изначально взять полностью случайный ввод и многократно пропустить его через модель, обновляя на каждой итерации на небольшую величину, основанную на прогнозах. Некоторые методы выборки могут помочь оптимизировать этот процесс, позволяя создавать качественные изображения за минимальное количество шагов.

Данные

Для этого примера мы будем использовать набор изображений из Hugging Face Hub, а именно коллекцию из 1000 изображений бабочек. Позже, в разделе «*Время проекта: обучение собственной модели диффузии*», вы увидите, как использовать свои данные. Теперь загрузим набор данных с бабочками.

```
from datasets import load_dataset
```

```
dataset = load_dataset("huggan/smithsonian_butterflies_subset", split="train")
```

Прежде чем использовать данные для обучения, мы должны подготовить их. Изображения обычно представляются в виде сетки пикселей. В отличие от предыдущей главы, где мы использовали оттенки серого, наши текущие изображения будут цветными. Пиксели представлены значениями от 0 до 255 для каждого из трех основных цветов (красный, зеленый и синий). Чтобы обработать их и подготовить к обучению, нужно сделать следующее:

1. Сделать изображения фиксированного размера. Это условие является необходимым, поскольку модель ожидает, что все изображения будут иметь одинаковые размеры.
2. Необязательно: выполнить *аугментацию* — «прирастить» некоторые области, случайным образом перевернув изображения по горизонтали, что делает модель более надежной и позволит обучаться с большим количеством данных. Аугментация (рис. 4.4) является распространенной практикой в задачах компьютерного зрения, поскольку она помогает модели лучше обобщать невидимые данные. «Перевоорачивание» — это всего лишь один из методов аугментации, но существуют и другие, например смещение, масштабирование и вращение.

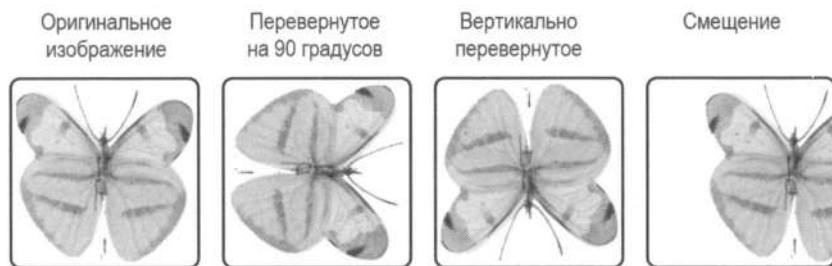


Рис. 4.4. Аугментация позволяет создать больше данных из обучающего набора, улучшая их генерализацию

3. Преобразовать данные в тензоры PyTorch. Они представляют значения цвета как числа с плавающей точкой от 0 до 1. Входные данные модели всегда должны быть отформатированы как многомерные матрицы или тензоры.
4. Нормализовать данные так, чтобы среднее значение было равно 0, а диапазон всех значений варьировался от -1 до 1 . Это обычная практика при обучении моделей глубокого обучения, поскольку она помогает делать это быстрее и эффективнее.

Мы можем определить наши преобразования с помощью `torchvision.transforms`¹.

```
from torchvision import transforms

image_size = 64

#Определяем преобразования
preprocess = transforms.Compose(
    [
        #Меняем размер
        transforms.Resize((image_size, image_size)),
        #Переворачиваем случайным образом (аугментация данных)
        transforms.RandomHorizontalFlip(),
        #Преобразуем в тензор (0, 1)
        transforms.ToTensor(),
        #Нормализуем, приводя к диапазону (-1, 1)
        transforms.Normalize([0.5], [0.5]),    ]
)
```

Библиотека `datasets` предоставляет удобный метод `set_transform()`. Он позволяет указывать преобразования, которые применяются к данным на лету по мере их использования. Теперь мы можем поместить набор данных в пакет с помощью класса `DataLoader`. Это утилита загрузки, которая делает итерацию по пакетам легкой, упрощая код обучения.

```
def transform(examples):
    examples = [preprocess(image) for image in examples["image"]]
    return {"images": examples}

dataset.set_transform(transform)
batch_size = 16

train_dataloader = torch.utils.data.DataLoader(
    dataset, batch_size=batch_size, shuffle=True
)
```

Мы можем убедиться, что это сработало, загрузив пакет и проверив изображения. Ниже на рис. 4.5 представлен пример пакета из обучающего набора².

```
from genaibook.core import show_images

batch = next(iter(train_dataloader))

#При нормализации мы преобразовали (0, 1) в (-1, 1)
#Теперь мы преобразуем обратно к (0, 1) для отображения
show_images(batch["images"][:8] * 0.5 + 0.5)
```

¹ `torchvision` — это библиотека PyTorch, которая предоставляет широкий спектр инструментов для работы с изображениями. В книге мы будем использовать ее только для тех преобразований данных, которые касаются предварительной обработки.

² Чтобы вывести красивых бабочек, мы использовали изображения размером более чем 64×64 вместо пикселизованных.



Рис. 4.5. Пример пакета из обучающего набора

Добавление шума

Искажение данных с помощью постепенного добавления шума — это наиболее распространенный подход, при котором мы вносим разное количество шума в обучающие данные. Наша цель — обучить модель, чтобы она надежно подавляла шум независимо от его количества на входе. Подаваемый на вход шум регулируется *графиком шума (Noise schedule)*, который является важным компонентом моделей диффузии. Разные исследовательские работы и подходы решают эту проблему по-разному.

Мы рассмотрим один распространенный подход, основанный на работе «*Denoising Diffusion Probabilistic Models*». В библиотеке `diffusers` добавление шума обрабатывается классом `Scheduler`, который принимает пакет изображений и список временных шагов, а затем определяет, как создать зашумленные версии этих изображений. Позже в этой главе мы исследуем математические аспекты, стоящие за этим процессом, а пока посмотрим, как это работает на практике. Следующий фрагмент кода применяет постепенно увеличивающееся количество шума к каждому из входных изображений (рис. 4.6).

```
from diffusers import DDPMSScheduler
```

```
#Подробнее о beta_start и beta_end мы расскажем в следующих разделах
```

```
scheduler = DDPMSScheduler(
    num_train_timesteps=1000, beta_start=0.001, beta_end=0.02
)
```

```
#Создаем тензор с 8 равномерно распределенными значениями от 0 до 999
```

```
timesteps = torch.linspace(0, 999, 8).long()
```

```
#Загружаем 8 изображений из набора данных
#и добавляем к ним весь шум, количество которого постепенно увеличивается
x = batch["images"][:8]
noise = torch.rand_like(x)
noised_x = scheduler.add_noise(x, noise, timesteps)
show_images((noised_x * 0.5 + 0.5).clip(0, 1))
```

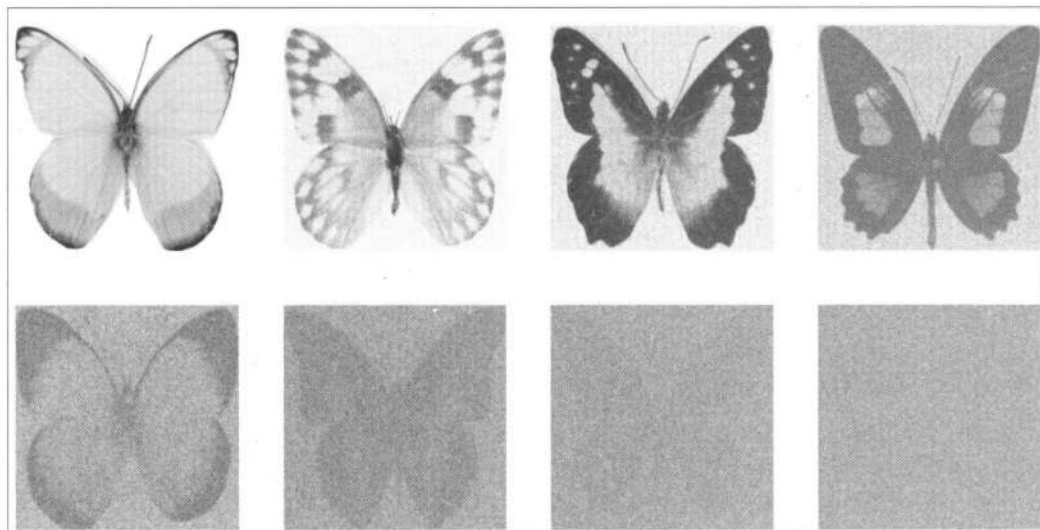


Рис. 4.6. Постепенно увеличиваем количество шума и добавляем его к изображениям в пакете

Во время обучения мы выбираем временные шаги случайным образом. График шума принимает некоторые параметры (*beta_start* и *beta_end*), которые нужны, чтобы определить, сколько шума должно присутствовать на конкретном временном шаге. Мы рассмотрим графики шума более подробно далее в этой главе.

UNet

UNet — это сверточная сеть, созданная для задач наподобие сегментации изображений, где желаемый выход имеет ту же форму, что и вход. Например, она применяется в медицине для визуализации различных анатомических структур.

Как показано на рис. 4.7, UNet состоит из ряда слоев *понижающей дискретизации* (также называется *понижающей выборкой*), которые сокращают объем входных данных. Затем следует ряд слоев *повышающей дискретизации* (повышающая выборка), которые снова увеличивают объем входных данных. Слои понижающей дискретизации обычно сопровождаются *пропускными соединениями* (**Skip connection**), которые связывают выходы слоев понижающей дискретизации со входами слоев повышающей дискретизации. Это позволяет вторым включать более мелкие детали из ранних слоев, сохраняя важную информацию с высоким разрешением во время процесса шумоподавления.

Архитектура UNet, предоставляемая библиотекой *diffusers*, более продвинута, чем ее исходная версия, предложенная в 2015 году, поскольку она содержит такие до-

полнения, как *блоки внимания* и *остаточные блоки*. Мы рассмотрим их подробнее позже, но основная идея здесь заключается в том, что UNet может принимать входные данные и создавать прогноз той же формы. В моделях диффузии вход может быть представлен изображениями с шумом, а выход — предсказанным шумом. Имея данную информацию, мы теперь можем очистить входное изображение от шума.

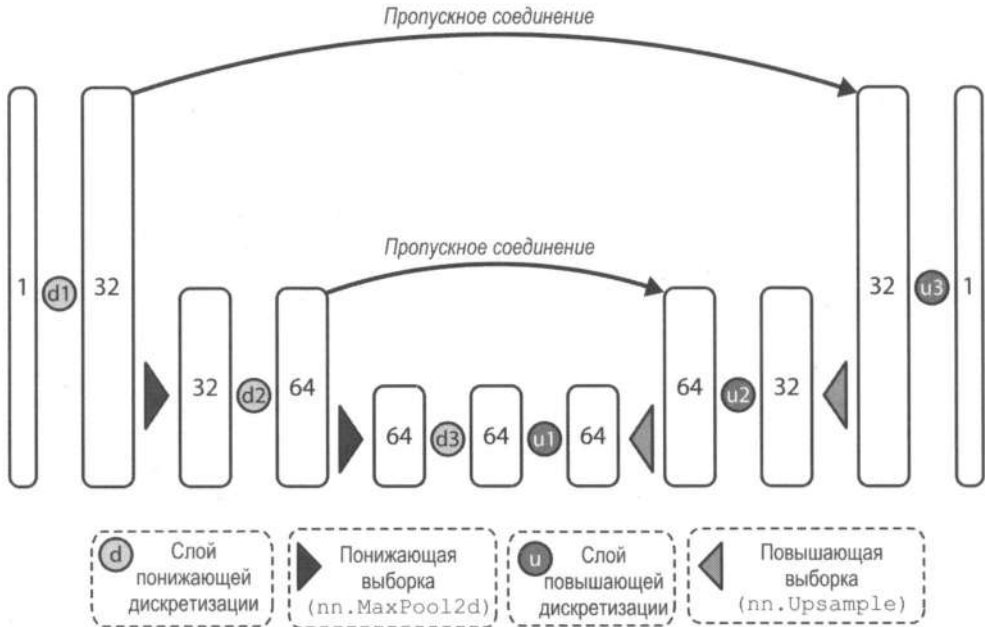


Рис. 4.7. Архитектура упрощенной сети UNet

Мы можем создать UNet и пропустить через нее наш пакет изображений с шумом следующим образом.

```
from diffusers import UNet2DModel
```

```
model = UNet2DModel(
    in_channels=3, #3 канала для изображений RGB
    sample_size=64, #Указываем наш размер ввода
    #Количество каналов на блок влияет на размер модели
    block_out_channels=(64, 128, 256, 512),
    down_block_types=(
        "DownBlock2D",
        "DownBlock2D",
        "AttnDownBlock2D",
        "AttnDownBlock2D",
    ),
    up_block_types=("AttnUpBlock2D", "AttnUpBlock2D", "UpBlock2D", "UpBlock2D"),
).to(device)
```

```
#Передаем пакет данных, чтобы убедиться, что он работает
with torch.inference_mode():
    out = model(noised_x.to(device), timestep=timesteps.to(device)).sample

print(noised_x.shape)
print(out.shape)

torch.Size([8, 3, 64, 64])
torch.Size([8, 3, 64, 64])
```

Обратите внимание, выходной сигнал имеет ту же форму, что и входной, а это именно то, что нам нужно.

Обучение

Теперь, когда у нас есть готовые данные, мы можем обучить модель. На каждом этапе мы делаем следующее:

1. Загружаем пакет изображений.
2. Добавляем шум к изображениям. Его количество зависит от временных шагов: чем их больше, тем больше шума. Нам нужно, чтобы модель могла подавлять шум как в большом количестве шума, так и в малом, поэтому мы будем инициализировать количество шума случайными значениями. Для этого выберем случайное количество временных шагов.
3. Загружаем изображения с шумом в модель.
4. Рассчитываем потери с помощью `mse`. Это распространенная функция потерь для задач регрессии, которая также включает прогнозирование шума модели UNet. Она измеряет среднеквадратичную разницу между прогнозируемыми и истинными значениями и сильнее штрафует большие ошибки. В модели UNet функция `mse` вычисляется между прогнозируемым и фактическим шумом, помогая модели генерировать более реалистичные изображения за счет минимизации потерь. Иногда она называется просто *шум* или *эпсилон* (т. е. разница).
5. Изучаем потери и обновляем веса модели с помощью оптимизатора.

Ниже показано, как все это выглядит в коде. Обратите внимание, обучение занимает некоторое время.

```
from torch.nn import functional as F

num_epochs = 50 #Сколько прогонов данных нам следует сделать?
lr = 1e-4 #Какую скорость обучения следует использовать?
optimizer = torch.optim.AdamW(model.parameters(), lr=lr)
losses = [] #Здесь хранятся значения потерь для последующей визуализации

#Обучаем модель (это занимает некоторое время)
for epoch in range(num_epochs):
    for batch in train_dataloader:
```

```

#Загружаем входные изображения
clean_images = batch["images"].to(device)

#Выбираем шум для добавления к изображениям
noise = torch.randn(clean_images.shape).to(device)

#Выбираем случайный временной шаг для каждого изображения
timesteps = torch.randint(
    0,
    scheduler.config.num_train_timesteps,
    (clean_images.shape[0],),
    device=device,
).long()

#Добавляем шум к чистым изображениям в соответствии
#с величиной шума на каждом временном шаге
noisy_images = scheduler.add_noise(clean_images, noise, timesteps)

#Получаем прогноз модели для шума
#Модель также использует временной шаг в качестве входных данных
#для дополнительного условия
noise_pred = model(noisy_images, timesteps, return_dict=False)[0]

#Сравниваем прогноз с реальным шумом
loss = F.mse_loss(noise_pred, noise)

#Сохраняем потери для последующего построения графика
losses.append(loss.item())

#Обновляем параметры модели на основе потерь с помощью оптимизатора
loss.backward()
optimizer.step()
optimizer.zero_grad()

```

Теперь, когда модель обучена, построим график потерь (рис. 4.8).

```

from matplotlib import pyplot as plt

plt.subplots(1, 2, figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(losses)
plt.title("Training loss")
plt.xlabel("Training step")

plt.subplot(1, 2, 2)
plt.plot(range(400, len(losses)), losses[400:])
plt.title("Training loss from step 400")
plt.xlabel("Training step");

```

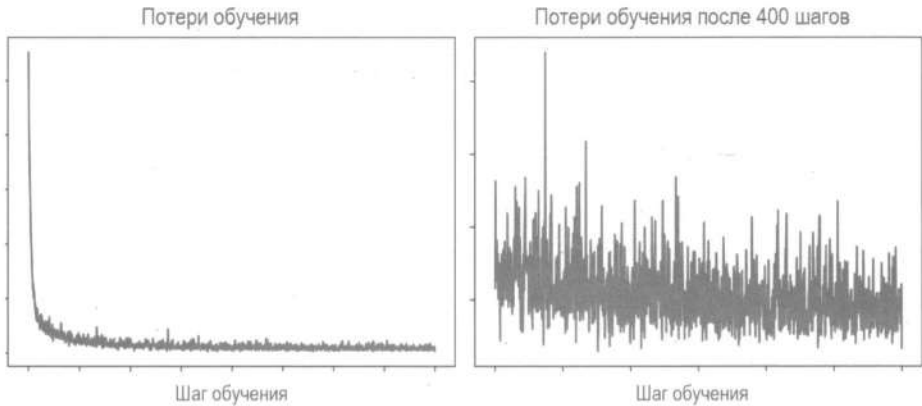


Рис. 4.8. График потерь при обучении

Кривая потерь на рис. 4.8 слева показана на всех шагах, а справа — после первых 400 шагов. Кривая потерь имеет тенденцию к снижению по мере того, как модель учится подавлять шумы на изображении. График демонстрирует достаточный уровень шума — потери не очень стабильны. Это происходит потому, что каждая итерация использует разное количество временных шагов при шумоподавлении. Глядя на `mse`, трудно сказать, будет ли эта модель хороша для прогнозов шума, поэтому мы перейдем к следующему разделу и посмотрим, насколько хорошо она это делает.

Выборка

Теперь, когда у нас есть модель, реализуем вывод и сгенерируем несколько изображений. Библиотека `diffusers` использует идею конвейеров, чтобы объединить все компоненты, необходимые для генерации с помощью модели диффузии. Для тестирования UNet мы можем использовать конвейер, который мы только что обучили. Несколько генераций показаны на рис. 4.9.¹

```
pipeline = DDIMPipeline(unet=unet, scheduler=scheduler)
ims = pipeline(batch_size=4).images
show_images(ims, nrow=1)
```

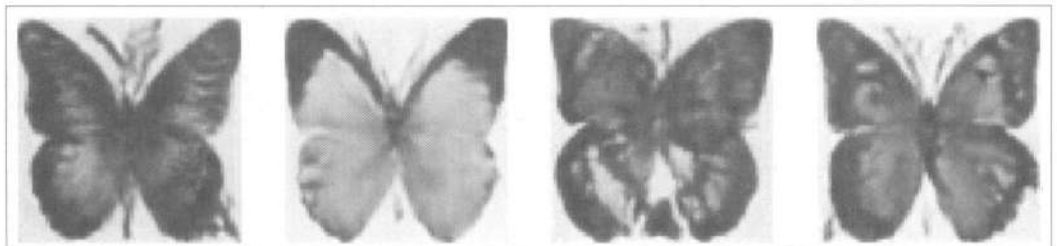


Рис. 4.9. Генерации модели Unet: изображения пока выглядят пикселизированными

¹ Изображения сгенерированы с помощью обученной модели с разрешением 64·64 и масштабированы, поэтому они будут выглядеть пикселизированными.

Перекладывание работы по созданию выборок на конвейер не показывает нам, что происходит «под капотом». Сделаем простой цикл выборки, показывающий, как модель постепенно «уточняет» входное изображение (рис. 4.10) на основе кода в методе `call()` из конвейера.

```
# Случайная начальная точка (4 случайных изображения):
sample = torch.randn(4, 3, 64, 64).to(device)

for t in scheduler.timesteps:
    # Получаем прогнозы модели
    with torch.inference_mode():
        noise_pred = model(sample, t)["sample"]

    # Обновляем выборку на каждом шаге
    sample = scheduler.step(noise_pred, t, sample).prev_sample

show_images(sample.clip(-1, 1) * 0.5 + 0.5, nrows=1)
```

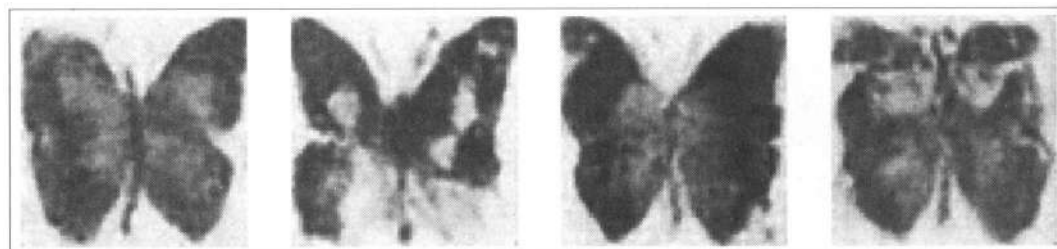


Рис. 4.10. Модель постепенно делает входное изображение более четким

Это тот же код, который мы использовали в начале главы, чтобы продемонстрировать идеи итеративного уточнения. Теперь вы можете лучше понять, что здесь происходит. Если вы посмотрите на реализацию конвейера `DDIMPipeline` из библиотеки `diffusers`, вы увидите, что логика очень схожа с логикой на нашей реализации в предыдущем фрагменте.

Первый ввод является полностью случайным. Затем модель уточняет его за несколько шагов. Каждый шаг — это небольшое обновление ввода, основанное на прогнозе модели для шума на каждом временном шаге. Пока что мы все еще абстрагируемся от сложности, стоящей за вызовом функции `pipeline.scheduler.step()`. Позже мы подробнее рассмотрим различные методы выборки и то, как они работают.

Оценка

Оценка генеративных моделей сложна, поскольку является субъективной по своей природе. Например, если задан входной промпт «изображение кошки в солнцезащитных очках», в результате мы можем получить множество генераций, которые будут потенциально подходящими. Существует распространенный подход, кото-

рый заключается в объединении качественной оценки (например, когда непосредственно люди сравнивают генерации) и количественных метрик, которые обеспечивают основу для этих оценок, но не обязательно соответствуют высокому качеству изображения.

Начальное расстояние Фреше (Fréchet Inception Distance, FID) позволяет оценивать производительность генеративных моделей. FID показывает, насколько похожи два набора данных. Используя предварительно обученную нейросеть (пример показан на рис. 4.11), FID измеряет, насколько близко сгенерированные выборки соответствуют реальным, сравнивая статистику между картами признаков, извлеченными из обоих наборов данных. Чем ниже оценка, тем лучше качество и реалистичность сгенерированных изображений, созданных моделью. Оценки FID популярны из-за их способности предоставлять «объективную» метрику сравнения для различных типов генеративных сетей, не полагаясь на человеческое суждение.

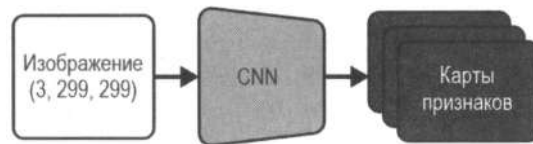


Рис. 4.11. Схема предварительно обученной сети в составе FID

Несмотря на удобство оценок FID, следует помнить о некоторых важных моментах (которые могут быть справедливы и для других метрик оценки):

- ◆ Оценки FID предназначены для сравнения двух распределений. Поэтому предполагается, что у нас есть доступ к исходному набору данных для сравнения. Кроме того, вы не можете рассчитать FID для одной генерации. Если изображение одно, то невозможно рассчитать его оценку.
- ◆ Оценка FID для конкретной модели зависит от количества выборок, используемых для расчета, поэтому при сравнении моделей нужно убедиться, что обе переданные оценки рассчитаны с использованием одинакового количества выборок. Обычно для этого используют 50 000 экземпляров, хотя для экономии времени вы можете использовать меньшее их количество при разработке и выполнить полную оценку только после того, как будете готовы опубликовать результаты.
- ◆ FID может быть чувствителен ко многим факторам. Например, другое количество шагов вывода приведет к совершенно другой оценке. Выбор графика шума (в данном случае DDPM) также влияет на FID.
- ◆ При расчете FID изображения приводятся к размеру 299×299. Это делает показатель менее полезным в качестве метрики для изображений с чрезвычайно низким или высоким разрешением. Также есть небольшие различия между тем, как изменение размера обрабатывается различными фреймворками глубокого обучения, что может привести к небольшим различиям в оценке.

- ♦ Сеть, используемая в качестве экстрактора признаков для FID, обычно представляет собой модель, обученную под задачу классификации на наборе ImageNet⁴. При генерации изображений после обучения на других наборах признаки, изученные моделью, могут быть менее полезными. Более точный подход — сначала обучить сеть классификации на данных, соответствующих конкретной теме, но это затрудняет сравнение оценок между различными подходами и методами. На данный момент ImageNet является стандартным выбором.
- ♦ Если вы сохраняете сгенерированные выборки для последующей оценки, их формат и алгоритм сжатия могут повлиять на оценку FID. По возможности избегайте формата JPEG (изображения низкого качества).

Даже если учесть все эти моменты, оценки FID являются лишь грубой мерой и не полностью отражают все нюансы того, что делает изображения более «реальными». Оценка генеративных моделей — это активная область исследований. Стандартные метрики, такие как *Kernel Inception Distance* (KID) и *Inception Score*, имеют схожие с FID проблемы. Вы можете смело их пробовать, чтобы получить представление, как одна модель работает относительно другой. Также обратите внимание на фактические изображения, сгенерированные каждой моделью, чтобы лучше понять, как они сравниваются.

Качество изображений, измеряемое FID или KID, является лишь одной из метрик, которую мы можем использовать, чтобы оценить производительность моделей, преобразовывающих текст в изображение. Существует комплексный подход, который называется *Holistic Evaluation of Text-to-Image Models* (HEIM). Он «пытается» учесть дополнительные желаемые характеристики моделей text-to-image, например соблюдение инструкций, оригинальность, способность к рассуждению, многоязычие, отсутствие предвзятости и токсичности и пр.

Человеческие предпочтения являются золотым стандартом качества относительно того, что в конечном итоге является довольно субъективной областью. Например, набор данных Parti Prompts содержит 1600 промптов различной сложности и категорий и позволяет сравнивать модели text-to-image, похожие на те, что мы рассмотрим в главе 5.

Графики шума

В предыдущем примере во время обучения мы добавили к изображениям разное количество шума. Для этого мы выбрали случайный временной шаг от 0 до 1000, а затем всю работу доверили графику шума. Во время вывода мы также использовали график, чтобы он предоставил нам информацию о том, какие временные шаги использовались и как выполнялся переход от одного шага к другому с учетом прогнозов модели.

⁴ ImageNet — один из самых популярных наборов данных для компьютерного зрения. Он содержит миллионы изображений в тысячах категорий, что делает его популярным инструментом для обучения и оценки базовых моделей.

Количество добавляемого шума — важный аспект, который может кардинально повлиять на производительность модели. В этом разделе мы увидим, почему это важно, и рассмотрим различные подходы, применяемые на практике.

Зачем надо добавлять шум?

В начале этой главы мы упоминали, что ключевая идея моделей диффузии — это итеративное уточнение. Поэтому в процессе обучения мы добавляем искажения разной степени к входным данным, чтобы во время вывода получить хороший результат как для максимально зашумленных изображений, так и для почти идеальных. Модель итеративно устраняет шум с изображений.

Пока что мы фокусировались на одном конкретном виде искажения — *гауссовский шум*. Он соответствует нормальному распределению, которое мы наблюдали в *главе 3*. Большее количество этого шума находится в районе среднего значения, а по мере продвижения к крайним областям его количество уменьшается⁵. Одна из причин, почему мы его используем, кроется непосредственно в основе моделей диффузии, поскольку они предполагают именно его применение. Если мы используем другой тип шума, технически это уже не является диффузией.

Однако исследовательская работа «*Cold Diffusion: Inverting Arbitrary Image Transforms Without Noise*» продемонстрировала, что нам не обязательно ограничиваться этим методом искажения только ради теоретического удобства. Авторы показали (рис. 4.12), что подход, подобный диффузии, работает для многих других методов

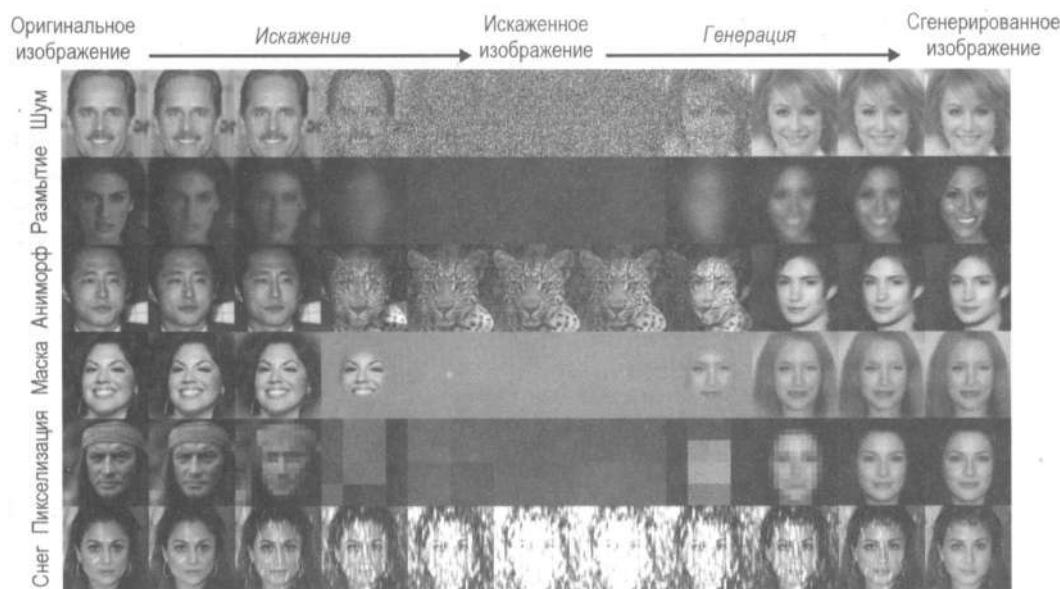


Рис. 4.12. Общие принципы диффузии работают и для других типов искажений, а не только для гауссовского шума (изображения адаптированы из статьи «*Cold Diffusion*»)

⁵ Гауссовский шум мы добавляем с помощью функции `torch.randn_like()`.

искажения. Это означает, что вместо шума мы можем применять и другие преобразования. Например, такие модели, как Muse, MaskGIT и Paella, применяют *маскировку случайных токенов* или *замену* в качестве эквивалентных методов искажения.

Тем не менее добавление шума остается самым популярным подходом по нескольким причинам:

- ♦ Мы можем легко контролировать количество шума, обеспечивая плавный переход от идеального изображения к полностью искаженному. Но это не касается уменьшения разрешения изображения, поскольку может привести к *дискретным* переходам.
- ♦ У нас может быть много допустимых случайных начальных точек для вывода, в отличие от методов, которые могут иметь ограниченное количество возможных начальных (полностью искаженных) состояний, таких как полностью черное изображение или изображение в один пиксель.

Поэтому в качестве метода искажения на данный момент мы будем использовать шум. Далее мы добавим его к нашим изображениям.

Начинаем с простого

У нас есть несколько изображений (x), и нам нужно добавить к ним случайный шум. Для этого сгенерируем чистый гауссовский шум тех же размеров, что и входные изображения с помощью функции `torch.rand_like()`.

```
x = next(iter(train_dataloader))["images"][:8]
noise = torch.rand_like(x)
```

Один из способов добавить разные объемы шума — это *линейная интерполяция* (`lerp`) между изображениями и шумом на некоторую величину. Данная функция плавно переходит от исходного изображения x к чистому шуму, когда значение меняется от 0 до 1.

```
def corrupt(x, noise, amount):
    #Изменяем amount так, чтобы модель правильно работала с исходными данными
    amount = amount.view(-1, 1, 1, 1) # убеждаемся, что шум транслируется

    #Смешиваем исходные данные и шум
    return (
        x * (1 - amount) + noise * amount
    ) #Эквивалентно x.lerp(noise, amount)
```

Теперь посмотрим, как это работает на пакете данных, где уровень шума варьируется от 0 до 1 (рис. 4.13).

```
amount = torch.linspace(0, 1, 8)
noised_x = corrupt(x, noise, amount)
show_images(noised_x * 0.5 + 0.5)
```

Модель работает, как нам нужно: плавный переход от исходного изображения к чистому шуму. Наш график шума использует подход *непрерывного времени*, где

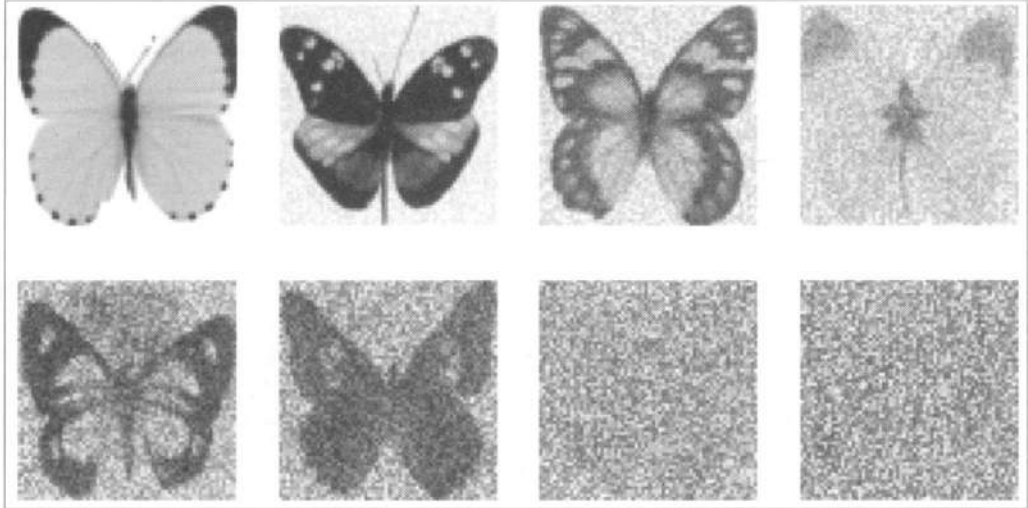


Рис. 4.13. Работа модели, когда уровень шума варьируется от 0 до 1

значение шума непрерывно проходит весь путь на временной шкале от 0 до 1. Другие подходы предполагают использование *дискретного времени*, где график шума меняется по заданным временным шагам, которые представляют собой большие целые числа. Мы обернем нашу функцию в класс, который будет преобразовывать непрерывное время в дискретные временные шаги и соответствующим образом добавлять шум (рис. 4.14).

```
class SimpleScheduler:
    def __init__(self):
        self.num_train_timesteps = 1000

    def add_noise(self, x, noise, timesteps):
        amount = timesteps / self.num_train_timesteps
        return corrupt(x, noise, amount)

scheduler = SimpleScheduler()
timesteps = torch.linspace(0, 999, 8).long()
noised_x = scheduler.add_noise(x, noise, timesteps)
show_images(noised_x * 0.5 + 0.5)
```

Теперь у нас есть некоторый результат, и мы можем напрямую сравнить его с работой графиков шума из библиотеки `diffusers`, например `DDPMScheduler` (рис. 4.15), который мы использовали во время обучения.

```
scheduler = DDPMScheduler(beta_end=0.01)
timesteps = torch.linspace(0, 999, 8).long()
noised_x = scheduler.add_noise(x, noise, timesteps)
show_images((noised_x * 0.5 + 0.5).clip(0, 1))
```

Если сравнить результаты нашего графика шума с результатами `DDPMScheduler`, можно заметить, что они не совпадают, но являются достаточно похожими. Это дает возможность обучить модель с помощью нашего графика шума.

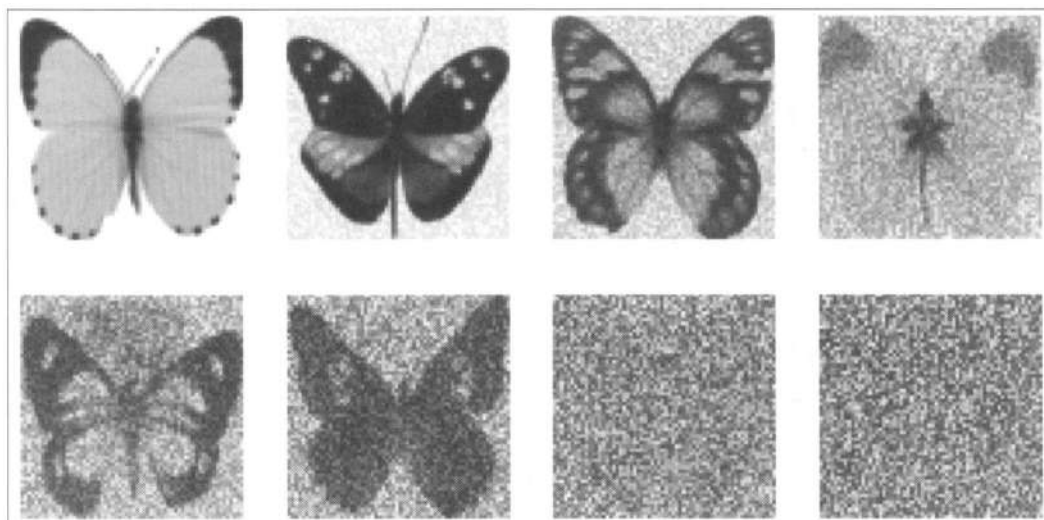


Рис. 4.14. Работа функции линейной дискретизации, когда она обернута в класс, который позволяет добавлять шум дискретно

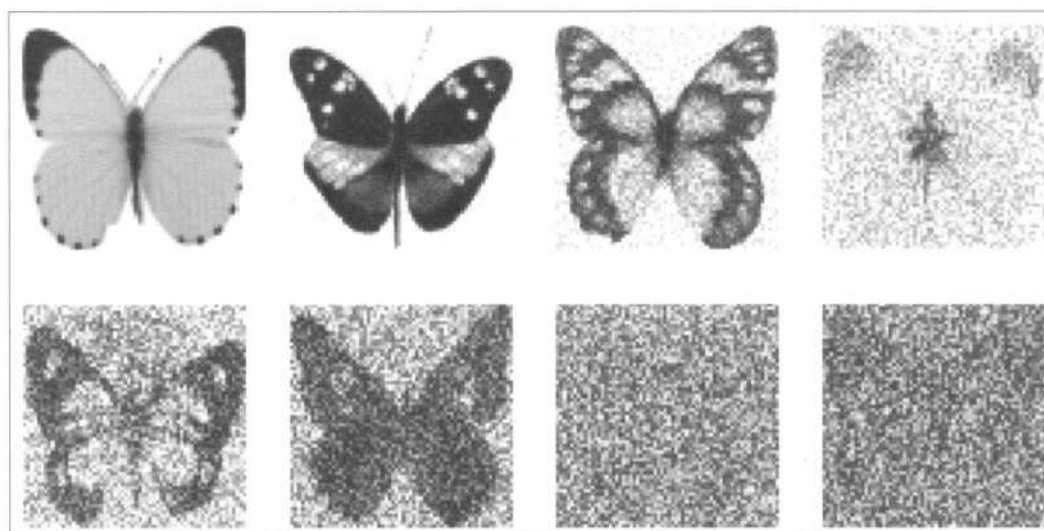


Рис. 4.15. Результат работы графика шума DDPMScheduler

Математика

В этом разделе мы погрузимся в математические основы, которые позволят понять, как мы добавляем шум к исходным изображениям. Следует помнить, что в литературе существует множество обозначений и подходов. Например, в некоторых работах график шума параметризуется непрерывно, таким образом, значение t меняется от 0 (шум отсутствует) до 1 (полный шум), как мы это сделали в нашей функции искажения. В других работах используется подход с дискретным временем, в котором временные шаги представлены целыми числами и изменяются от 0 до некото-

рого большого числа t (обычно 1000). Выбирая между этими двумя методами, можно выполнить преобразование так, как мы это делали с нашим классом `SimpleScheduler`. Далее мы будем придерживаться подхода с дискретным временем.

Хорошей отправной точкой для более глубокого погружения в математические аспекты могут послужить исследовательская работа «*Denoising Diffusion Probabilistic Models*» или статья «*The Annotated Diffusion Model*». Если вы считаете, что этот раздел содержит слишком много глубокой информации, можете сосредоточиться на высокоуровневых концепциях и вернуться к математике позже.

Мы начнем с определения, как сделать один шаг шума, чтобы перейти от временного шага $t - 1$ к t . Как упоминалось ранее, идея состоит в том, чтобы добавить некоторый гауссовский шум (ϵ). Он имеет единичную дисперсию, которая управляет разбросом значений шума. Добавляя его к изображению на каждом шаге, мы постепенно искажаем исходное изображение, что является ключевой частью процесса обучения модели диффузии.

$$x_t = x_{t-1} + \epsilon.$$

Чтобы контролировать количество шума, добавляемого на каждом шаге, мы введем параметр β_t . Он определяется для всех временных шагов t и указывает, сколько шума должно быть добавлено на каждом из них. Другими словами, x_t представляет собой смесь x_{t-1} и некоторого случайного шума, масштабированного с помощью β_t . Это позволяет нам постепенно увеличивать количество шума, добавляемого к изображению, по мере продвижения по временным шагам, что является ключевой частью процесса обучения моделей диффузии.

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon.$$

Далее мы можем определить процесс добавления шума как распределение, где шум x_t имеет среднее значение $\sqrt{1 - \beta_t} x_{t-1}$ и дисперсию β_t . Это распределение помогает нам более точно моделировать процесс добавления шума. Формула в виде распределения выглядит следующим образом:

$$q(x_t | x_{t-1}) = N(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I).$$

Мы определили распределение для выборки x , обусловленной предыдущим значением. Чтобы получить вход с шумом на временном шаге t , мы могли бы начать с $t = 0$ и многократно применять по одному шагу, что было бы очень неэффективно. Вместо этого мы используем формулу для перехода к любому временному шагу t за раз, выполнив «трюк репараметризации». Идея состоит в том, чтобы предварительно вычислить график шума, который определяется значениями β_t . Затем мы определим $\alpha_t = 1 - \beta_t$ и $\bar{\alpha}_t$ как кумулятивное произведение всех значений α до времени t , что можно выразить как $\bar{\alpha}_t := \prod_{s=1}^t \alpha_s$. Используя эти инструменты и обозначения, переопределим распределение и способ выборки в конкретное время. Новое распределение $q(x_t | x_{t-1})$ имеет среднее значение $\bar{\alpha}_t x_{t-1}$ и дисперсию $(1 - \bar{\alpha}_t)I$.

$$q(x_t | x_{t-1}) = N(x_t; \bar{\alpha}_t x_{t-1}, (1 - \bar{\alpha}_t)I).$$

(Часть задач в конце главы затрагивают «трюк репараметризации».)

Теперь мы можем сделать выборку изображения с шумом на временном шаге t , используя следующую формулу:

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon.$$

Уравнение для x_t показывает, что шумный вход на временном шаге t представляет собой комбинацию исходного изображения x_0 (масштабированного на величину $\sqrt{\bar{\alpha}_t}$) и шума ϵ (масштабированного на величину $\sqrt{1 - \bar{\alpha}_t}$). Обратите внимание, теперь мы можем вычислить выборку напрямую, не проходя по всем предыдущим временным шагам, что делает ее гораздо более эффективной для обучения моделей диффузии.

В библиотеке `diffusers` значения $\bar{\alpha}_t$ хранятся в графике `scheduler.alphas_cumprod`. Зная это, мы можем построить график коэффициентов масштаба для исходного изображения x_0 и шума ϵ по разным временным шагам для данного графика шума. Библиотека `diffusers` позволяет контролировать начальное (`beta_start`) и конечное (`beta_end`) значения, а также как они будут изменяться (например, линейно — `beta_schedule="linear"`). График `DDPMScheduler` на рис. 4.16 описывает количество шума (оранжевая линия), добавленное к входному изображению (синяя линия). Мы видим, что шум увеличивается в масштабе по мере увеличения временных шагов, как и ожидалось.

```
from genaibook.core import plot_scheduler
```

```
plot_scheduler(
    DDPMScheduler(beta_start=0.001, beta_end=0.02, beta_schedule="linear")
)
```

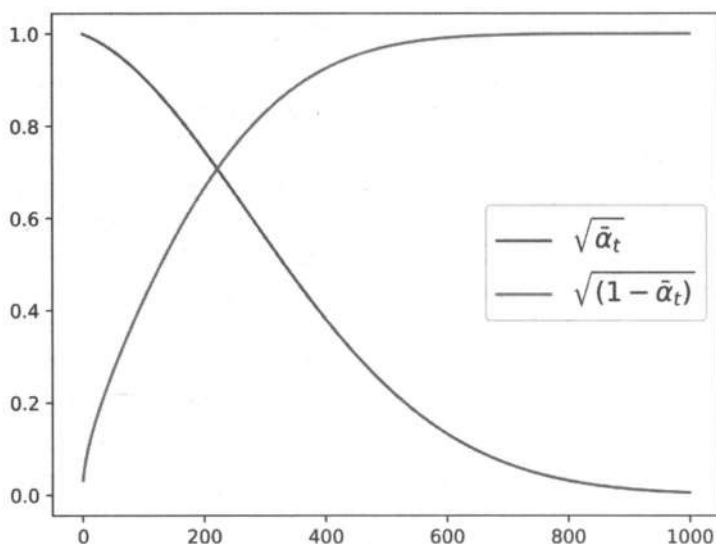


Рис. 4.16. Работа графика `DDPMScheduler`

Класс `SimpleScheduler` просто линейно смешивает исходное изображение и шум. Мы можем в этом убедиться, построив график коэффициентов масштабирования (рис. 4.17), эквивалентных $\sqrt{\alpha_t}$ и $\sqrt{(1 - \alpha_t)}$ в случае DDPM.

```
plot_scheduler(SimpleScheduler())
```

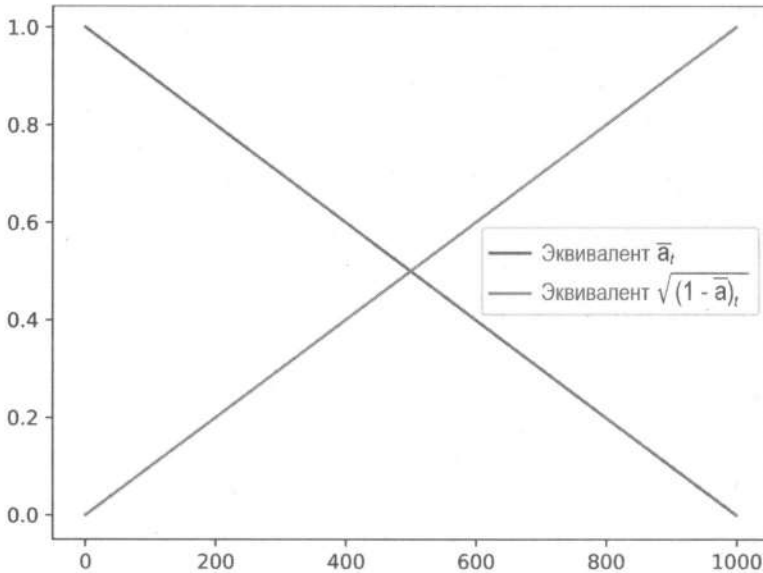


Рис. 4.17. Работа графика `SimpleScheduler`

Хороший график шума позволяет гарантировать, что модель получит смесь изображений с разными уровнями шума. Наилучший выбор будет отличаться в зависимости от данных обучения. Визуализируйте еще несколько вариантов и обратите внимание на следующее:

- ◆ Слишком низкое значение `beta_end` означает, что мы никогда не исказим изображение полностью, следовательно, модель не получит ничего похожего на случайный шум в качестве отправной точки для вывода.
- ◆ Чрезвычайно высокое значение `beta_end` означает, что большинство временных шагов тратится на почти полный шум, что приведет к плохой производительности обучения.
- ◆ Разные `beta`-графики дают разные кривые. Косинусный график популярен, поскольку он плавно переходит от исходного изображения к шуму.

Далее визуализируем на рис. 4.18 сравнение различных графиков шума `DDPMScheduler` с множеством гиперпараметров и `β`-графиков.

```
fig, (ax) = plt.subplots(1, 1, figsize=(8, 5))
plot_scheduler(
    DDPMScheduler(beta_schedule="linear"),
    label="default schedule",
    ax=ax,
```

```

    plot_both=False,
)
plot_scheduler(
    DDPMScheduler(beta_schedule="squaredcos_cap_v2"),
    label="cosine schedule",
    ax=ax,
    plot_both=False,
)
plot_scheduler(
    DDPMScheduler(beta_start=0.001, beta_end=0.003, beta_schedule="linear"),
    label="Low beta_end",
    ax=ax,
    plot_both=False,
)
plot_scheduler(
    DDPMScheduler(beta_start=0.001, beta_end=0.1, beta_schedule="linear"),
    label="High beta_end",
    ax=ax,
    plot_both=False,
)

```

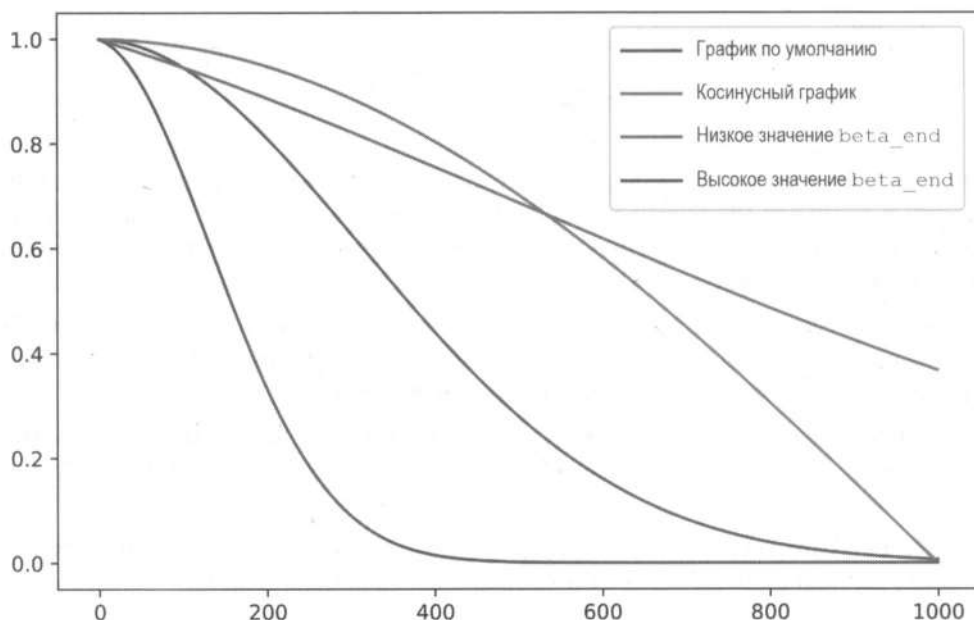


Рис. 4.18. Сравнение различных графиков DDPMScheduler с множеством гиперпараметров и β -графиков



Все представленные здесь графики называются *сохраняющими дисперсию* (Variance preserving, VP). Это означает, что дисперсия входных данных модели поддерживается близкой к 1 по всему графику. Вы также можете столкнуться с термином *взрывающая дисперсия* (Variance exploding, VE), где шум добавляется к исходному изображению в разных количествах (что приводит к высокодисперс-

ным входным данным). SimpleScheduler почти соответствует графику VP, но дисперсия не всегда сохраняется из-за линейной интерполяции.

Важность воздействия хорошо зашумленных изображений на модель (включая чистый шум, который подается на ввод в самом начале) была исследована в статье под названием «*Common Diffusion Noise Schedules and Sample Steps are Flawed*». Она показала, что некоторые модели диффузии не могли генерировать слишком яркие или слишком темные изображения, поскольку графики обучения не охватывали все состояния. Как и во многих связанных с диффузией темах, постоянный поток новых статей направлен на исследование графиков шума, поэтому к тому времени, как вы это прочтете, скорее всего, появится новая обширная коллекция вариантов, которую можно попробовать на практике.

Влияние входного разрешения и масштабирования

Два аспекта графиков шума, которые до недавнего времени по большей части мы игнорировали, — это размер входных данных и их масштабирование. Во многих исследовательских работах тестирование потенциальных графиков шума происходит на небольших наборах данных и при низком разрешении, а затем для обучения финальных моделей разработчики используют более эффективные графики шума на изображениях с большим разрешением. При таком подходе можно заметить проблему, если добавить одинаковое количество шума к двум изображениям разного размера, как показано на рис. 4.19.

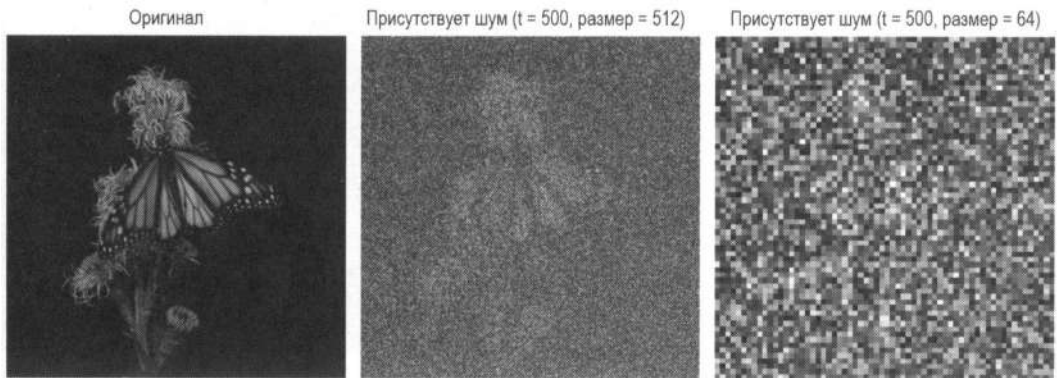


Рис. 4.19. Применение одинакового количества входного шума к изображениям с разным разрешением

Изображения с высоким разрешением, как правило, содержат много избыточной информации. Поэтому даже если один пиксель искажен шумом, окружающие пиксели содержат достаточно данных для восстановления. Для изображений с низким разрешением, где один пиксель может содержать много полезной информации, это не работает. Добавление того же количества шума к изображению с низким разрешением приведет к гораздо более сильному искажению, чем добавление эквивалентного количества шума к изображению с высоким разрешением.

Две независимые исследовательские работы в начале 2023 года тщательно изучили этот эффект. В каждой из них использовались новые идеи для обучения моделей,

способных генерировать выходные данные с высоким разрешением. В статье «*Simple diffusion: End-to-end diffusion for high resolution images*» был представлен метод корректировки шума на основе размера входных данных, что позволило соответствующим образом изменить график (оптимизированный под изображения с низким разрешением) для нового целевого разрешения. В другой статье («*On the Importance of Noise Scheduling for Diffusion Models*») были проведены аналогичные эксперименты и отмечен еще один критически важный аспект — *масштабирование* входных данных, а именно как мы представляем наш ввод. Если изображения представлены как числа с плавающей точкой от 0 до 1, они будут иметь меньшую дисперсию, чем шум (обычно имеет единичную дисперсию). Таким образом, соотношение сигнал/шум будет ниже для заданного уровня шума, чем если бы изображения были представлены как числа с плавающей точкой от -1 до 1 (как мы делали в предыдущем примере обучения) или как-то еще. Масштабирование входных изображений меняет соотношение сигнал/шум, поэтому его корректировка является еще одним способом настроить модель при обучении на более крупных изображениях. В этой статье, по сути, масштабирование входных данных рекомендуется как простой способ адаптации обучения для различных размеров изображений. Также график шума может быть настроен в зависимости от разрешения, но тогда сложнее найти его оптимальные значения, поскольку задействовано множество гиперпараметров. На рис. 4.20 можно увидеть эффект масштабирования входных данных.

```
import numpy as np
from genaibook.core import load_image, SampleURL

scheduler = DDPMScheduler(beta_end=0.05, beta_schedule="scaled_linear")
image = load_image(
    SampleURL.DogExample,
    size=(512, 512),
    return_tensor=True,
)

t = torch.tensor(300) #Временной шаг, к которому мы приближаемся
scales = np.linspace(0.1, 1.0, 4)

images = [image]
noise = torch.randn_like(image)
for b in reversed(scales):
    noised = (
        scheduler.add_noise(b * (image * 2 - 1), noise, t).clip(-1, 1) * 0.5 + 0.5
    )
    images.append(noised)

show_images(
    images[1:],
    nrow=1,
    titles=[f"Scale: {b}" for b in reversed(scales)],
    figsize=(15, 5),
)
```

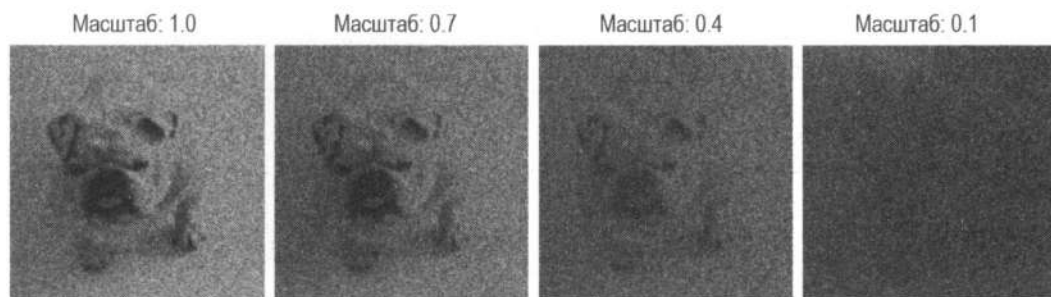


Рис. 4.20. Эффект масштабирования входных данных

Как вы видите, ко всем изображениям применен одинаковый входной шум, соответствующий шагу $t = 300$, но при этом входные данные умножены на разные масштабные коэффициенты. Шум более заметен там, где масштаб имеет большее влияние на изображение. Масштаб также уменьшает динамический диапазон (дисперсию), что приводит к более затемненным входным данным⁶.

Подробный разбор: UNet и альтернативы

Рассмотрим фактическую модель, которая делает важные прогнозы. Она должна быть способна принимать зашумленное изображение и выводить его шум, тем самым позволяя убирать искажение входного изображения. Для этого ей необходима способность принимать изображение произвольного размера, а затем выводить его того же размера. Кроме того, она должна быть способна делать точные прогнозы на уровне пикселей, одновременно захватывая информацию об изображении более высокого уровня. Популярным решением здесь являются модели с архитектурой UNet, которая была изобретена в 2015 году для сегментации медицинских изображений и с тех пор стала популярным выбором для множества задач.

Подобно автоэнкодерам и VAE, которые мы рассмотрели в предыдущей главе, модели UNet состоят из ряда блоков понижающей и повышающей дискретизации. Первые отвечают за уменьшение размера изображения, а вторые — за увеличение размера изображения. Блоки понижения частоты дискретизации обычно включают ряд сверточных слоев, за которыми следует непосредственно слой понижения частоты дискретизации или *пуллинга*⁷. Блоки повышения частоты дискретизации аналогично обычно включают ряд сверточных слоев, за которыми следует слой повышения частоты дискретизации или транспонированной свертки. Транспонированный сверточный слой — это особый тип сверточного слоя, который увеличивает размер изображения (а не уменьшает его).

⁶ В этом режиме входные изображения нормализуются до передачи в модель, чтобы не снижать дисперсию столь радикально.

⁷ **Пулинг (Pooling)** — это метод выбора информации, которую необходимо сохранить при понижении частоты дискретизации выходных данных с предыдущего слоя. К распространенным стратегиям относятся *пулинг среднего значения*, который уменьшает пакет до его среднего значения, или *пулинг максимального значения*, который выбирает максимальное значение в конкретном пакете. Пулинг применяется независимо ко всем каналам входного тензора.

Обычные автоэнкодеры и VAE — плохой выбор для данной задачи. Они не способны делать достаточно хорошие и точные прогнозы на уровне пикселей, поскольку должны реконструировать изображения из низкоразмерного латентного пространства. В UNet блоки понижения и повышения частоты дискретизации связаны пропускными соединениями, которые позволяют информации напрямую перетекать из понижающих блоков в повышающие блоки. Это позволяет модели делать точные прогнозы на уровне пикселей, а также захватывать более высокоуровневую информацию об изображении в целом.

Простая модель UNet

Чтобы лучше понять структуру UNet, построим простую сеть с нуля. На рис. 4.21 показана базовая архитектура UNET.

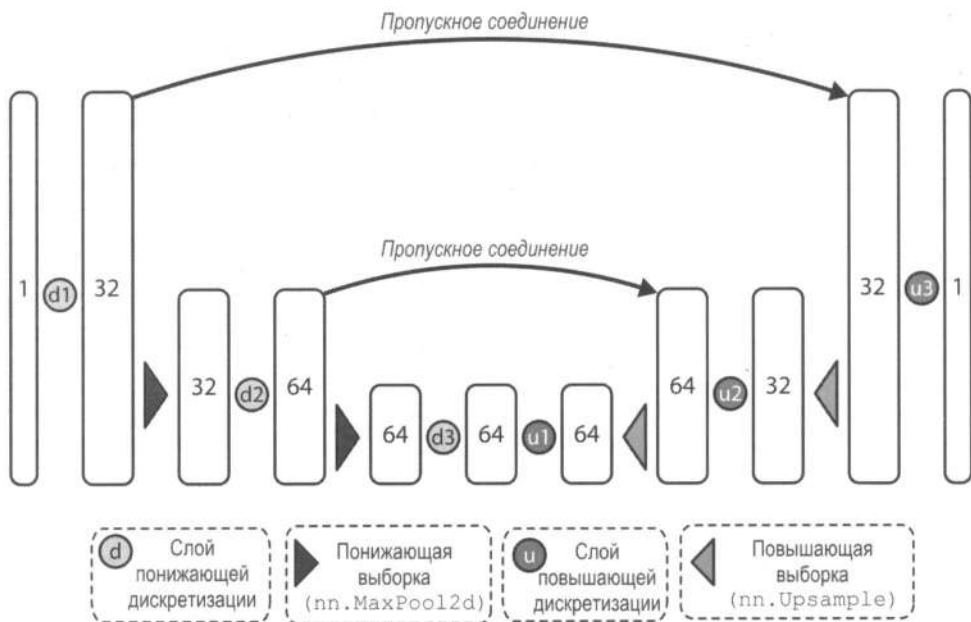


Рис. 4.21. Архитектура базовой сети Unet

Мы разработаем модель, работающую с одноканальными изображениями (например, в оттенках серого), которую можно использовать, чтобы построить модель диффузии для таких наборов данных, как MNIST. Мы будем использовать три слоя для понижения дискретизации и еще три для повышения. Каждый слой будет состоять из свертки, за которой следует функция активации и шаг повышения или понижения дискретизации, в зависимости от того, осуществляют они кодирование или декодирование. Существует несколько способов реализовать пропускные соединения. Один из вариантов, который мы будем использовать, заключается в добавлении вывода из понижающего блока к соответствующему вводу повышающего блока. Есть также и другой способ — объединить вывод блока понижения с вводом

блока повышения. Мы даже можем добавить несколько дополнительных слоев в пропускные соединения.

Пока что мы не будем усложнять все и применим первый вариант. В виде кода эта модель выглядит следующим образом.

```
from torch import nn

class BasicUNet(nn.Module):
    """Минимальная реализация UNet"""

    def __init__(self, in_channels=1, out_channels=1):
        super().__init__()
        self.down_layers = nn.ModuleList(
            [
                nn.Conv2d(in_channels, 32, kernel_size=5, padding=2),
                nn.Conv2d(32, 64, kernel_size=5, padding=2),
                nn.Conv2d(64, 64, kernel_size=5, padding=2),
            ]
        )
        self.up_layers = nn.ModuleList(
            [
                nn.Conv2d(64, 64, kernel_size=5, padding=2),
                nn.Conv2d(64, 32, kernel_size=5, padding=2),
                nn.Conv2d(32, out_channels, kernel_size=5, padding=2),
            ]
        )

        #Применяем функцию активации SiLU, которая хорошо себя
        #зарекомендовала благодаря различным свойствам (гладкость,
        #немонотонность и т. д.)
        self.act = nn.SiLU()
        self.downscale = nn.MaxPool2d(2)
        self.upscale = nn.Upsample(scale_factor=2)

    def forward(self, x):
        h = []
        for i, l in enumerate(self.down_layers):
            x = self.act(l(x))
            #Для всех слоев, кроме третьего (последнего) слоя понижения
            if i < 2:
                #Сохраняем вывод для пропускных соединений
                h.append(x)
                #Уменьшение масштаба готово для следующего слоя
                x = self.downscale(x)

        for i, l in enumerate(self.up_layers):
            #Для всех слоев, кроме первого слоя повышения
```

```

    if i > 0:
        #Увеличиваем масштаб
        x = self.upscale(x)
        #Извлекаем сохраненный вывод (пропускные соединения)
        x = h.pop()
        x = self.act(1(x))

    return x

```

Если взять входное изображение в оттенках серого в форме [1, 28, 28], его путь через модель будет следующим:

1. Изображение проходит через блок уменьшения масштаба. Первый слой (двумерная свертка с 32 фильтрами) придает ему форму [32, 28, 28].
2. Затем изображение уменьшается с помощью пуллинга максимального значения, что придает его форме вид [32, 14, 14]. Набор данных MNIST содержит белые числа на черном фоне (где черный цвет представлен числом 0), поэтому мы используем пуллинг максимального значения, чтобы выбрать самые большие значения и, таким образом, сосредоточиться на самых ярких пикселях⁸.
3. Изображение проходит через второй блок уменьшения масштаба (двумерная свертка с 64 фильтрами), который придает ему форму [64, 14, 14].
4. После еще одного уменьшения масштаба форма принимает вид [64, 7, 7].
5. В блоке уменьшения масштаба есть третий слой, но на этот раз уменьшения масштаба не происходит, потому что мы уже используем достаточно маленькие блоки 7×7. Это позволяет сохранить форму [64, 7, 7].
6. Затем мы выполняем весь описанный выше процесс, но в обратном порядке, увеличивая масштаб до [64, 14, 14], затем до [32, 14, 14] и, наконец, до [1, 28, 28].

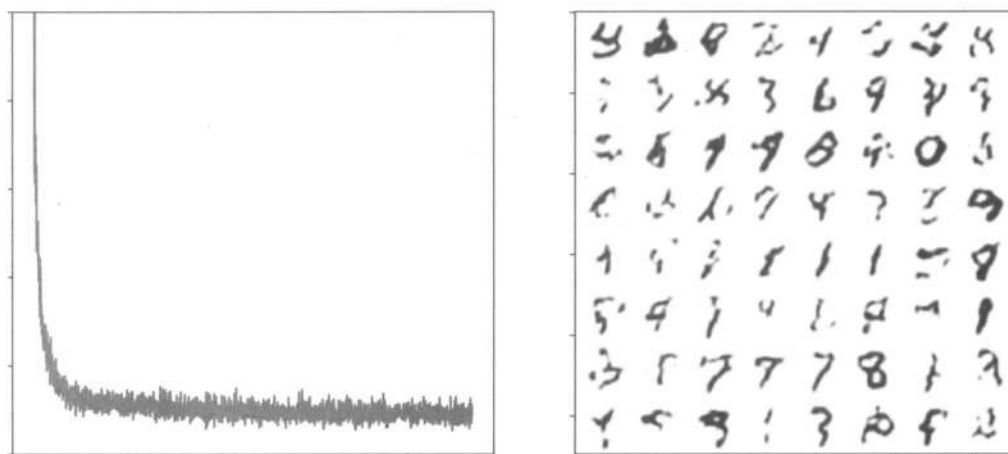


Рис. 4.22. График потерь и примеры генераций базовой сети UNet

⁸ Для наглядности мы демонстрируем MNIST в виде черных цифр на белом фоне, но в обучающем наборе данных все наоборот.

Модель диффузии, обученная с помощью данной архитектуры на наборе MNIST, теперь умеет создавать выборки, которые показаны на рис. 4.22 (код этого процесса опущен для краткости изложения, но включен в дополнительный материал книги).

Улучшение UNet

Сеть UNet отлично подходит для простых сценариев. Отсюда возникает вопрос, как мы можем улучшить ее, чтобы выполнять более сложные задачи? Ниже приведены некоторые варианты:

◆ Добавить больше параметров

Это можно сделать, используя несколько сверточных слоев в каждом блоке, применяя большее количество фильтров в каждом сверточном слое или делая сеть глубже.

◆ Добавить нормализацию (например, пакетную)

Пакетная нормализация может помочь модели обучаться быстрее и надежнее, давая гарантию, что выходные данные каждого слоя будут центрироваться вокруг 0 и иметь стандартное отклонение, равное 1.

◆ Добавить регуляризацию (например, Dropout)

Dropout помогает предотвратить переобучение (**Overfitting**), что важно при работе с небольшими наборами данных⁹.

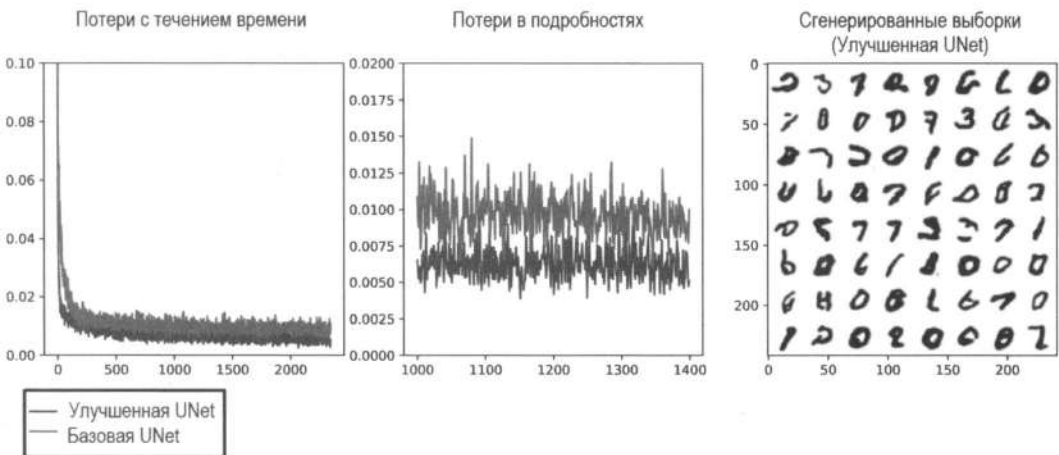


Рис. 4.23. Графики потерь и примеры генераций сети UNet из библиотеки *diffusers*, реализованной с некоторыми улучшениями в сравнении с базовой архитектурой

⁹ **Dropout** (от англ. «выпадение») — метод регуляризации искусственных нейронных сетей. Он помогает предотвратить переобучение, которое происходит, когда модель изучает шум в обучающих данных, а не базовые закономерности. Метод случайным образом «выключает» (делает неактивными) определенную долю нейронов (они «выпадают» из процесса) в слое сети на каждом шаге обучения. Это означает, что выход выбранного нейрона устанавливается в ноль и не участвует в прямом и обратном распространении ошибки. — Пер.

◆ *Добавить внимание*

Введение слоев самовнимания позволит модели фокусироваться на разных частях изображения в разное время, что может помочь сети UNet выучить более сложные функции. Добавление слоев внимания, как в трансформерах, также позволит увеличить количество обучаемых параметров. Недостатком здесь является то, что слои внимания гораздо более требовательны к вычислениям, чем обычные сверточные слои при более высоких разрешениях, поэтому они обычно используются только при низких разрешениях (например, блоки с более низким разрешением в UNet).

Для сравнения на рис. 4.23 показаны результаты реализации сети UNet, обученной на наборе MNIST, с помощью библиотеки *diffusers*. Библиотека позволяет реализовать все улучшения, упомянутые выше.

Альтернативные архитектуры

Недавно ИИ-сообществом было предложено несколько альтернативных архитектур для моделей диффузии (рис. 4.24).

К таким архитектурам относятся:

◆ *Трансформеры*

Исследовательская работа «Scalable Diffusion Models with Transformers» демонстрирует, что архитектура на основе трансформеров позволяет превосходно обучать модели диффузии. Однако требования к вычислениям и памяти остаются проблемой при работе с очень высокими разрешениями.

◆ *UViT*

Архитектура UViT направлена на то, чтобы получить «лучшее из обоих миров» (UNet и трансформеры). В ней средние слои UNet заменены на большой стек блоков трансформера. Ключевым моментом здесь является то, что если большую часть вычислений сосредоточить на блоках с низким разрешением, то это позволит более эффективно обучить модели диффузии с высоким разрешением. Для очень высоких разрешений выполняется дополнительная предварительная обработка с помощью так называемого вейвлет-преобразования (**Wavelet transform**), чтобы уменьшить пространственное разрешение входного изображения, сохраняя при этом как можно больше информации через дополнительные каналы, что, опять же, сокращает объем вычислений, затрачиваемых на более высокие пространственные разрешения.

◆ *Рекуррентные интерфейсные сети (RIN)*

В работе «Scalable Adaptive Computation for Iterative Generation» показана работа такого подхода. Сначала входы высокого разрешения сопоставляются с более управляемым и низкоразмерным латентным представлением, которое затем обрабатывается стеком блоков трансформера, прежде чем изображение будет декодировано. Кроме того, в этой работе представлена идея рекуррентности, когда в модель передается информация из предыдущего шага обработки. Это может быть полезно при итеративном улучшении.

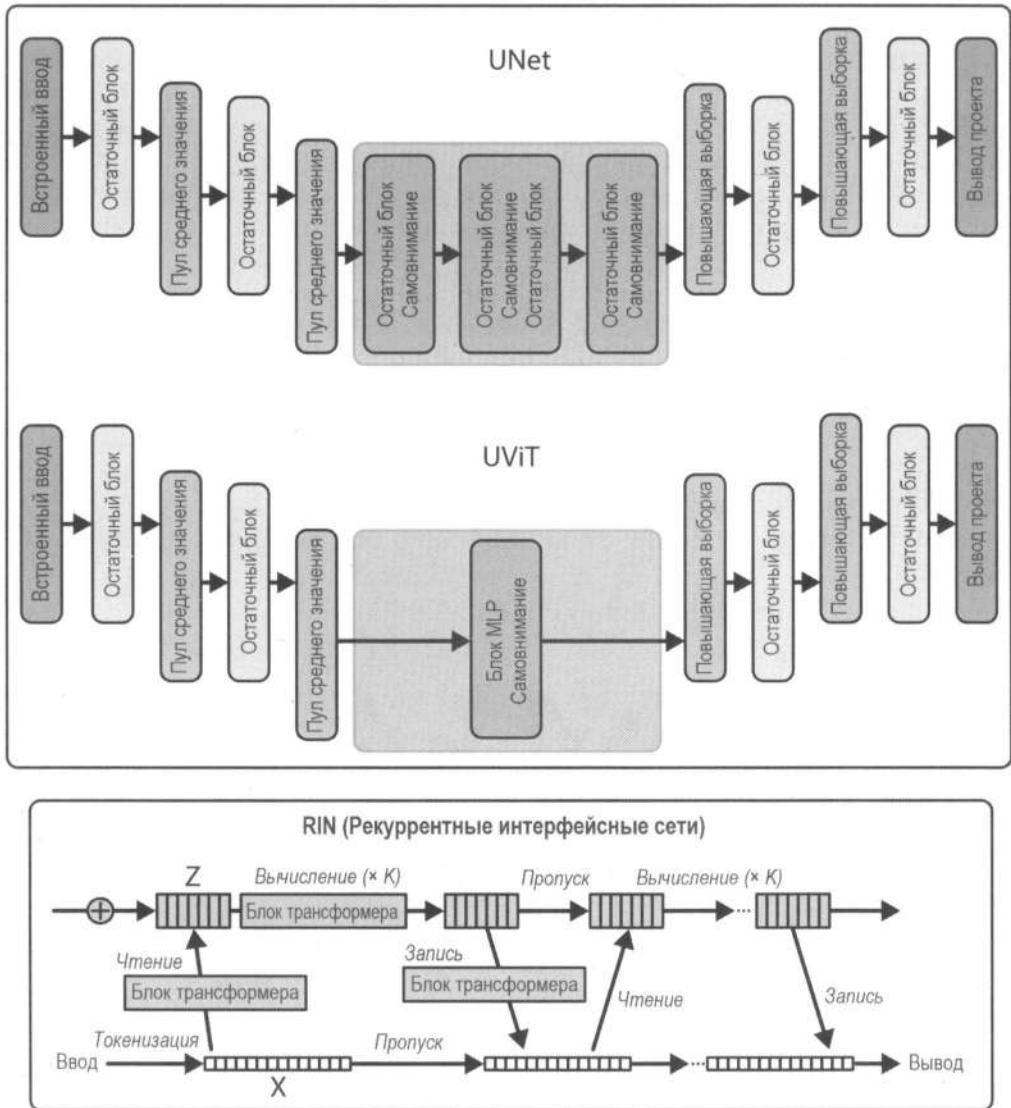


Рис. 4.24. Сравнение сетей UNet и UViT, а также сеть RIN

Высококачественные модели диффузии, основанные на трансформерах, включают таких представителей, как Flux, Stable Diffusion 3, PixArt-Σ и Sora (работает с текстом и видео). Пока что сложно сказать, какие подходы будут более эффективными в будущем: UNet на основе трансформеров или гибридные вроде UViT и RIN.

Подробный разбор: цели диффузии

На данный момент мы рассмотрели модели диффузии, которые могут принимать на вход искаженные данные и способны убирать из них шум. На первый взгляд, можно предположить, что вполне логичная цель прогнозирования — получить на вы-

ходе изображение без шума (назовем его x_0). Однако на уровне кода мы также сравнили модель прогнозирования с шумом единичной дисперсии, который использовался для создания версии изображения с шумом (часто называемой *эпсилон*, ϵ ps). Математически они кажутся идентичными, поскольку, если нам известны шум и временной шаг, мы можем вывести x_0 , и наоборот. Несмотря на то что это утверждение верно, выбор целевых данных имеет некоторое влияние на то, насколько велики будут потери на разных временных шагах и, следовательно, на то, какие уровни шума модель лучше всего будет учиться подавлять. Для модели предсказать шум проще, чем напрямую предсказать целевые данные. Это связано с тем, что шум на каждом шаге соответствует уже знакомому нам распределению, и предсказать разницу между двумя шагами часто бывает проще, чем предсказать абсолютные значения целевых данных.

Чтобы получить некоторое представление, визуализируем разные целевые данные на разных временных шагах. На рис. 4.25 входное изображение и случайный шум (первые две строки) являются одинаковыми, но искаженные изображения в третьей строке имеют разное количество добавленного шума в зависимости от временного шага.

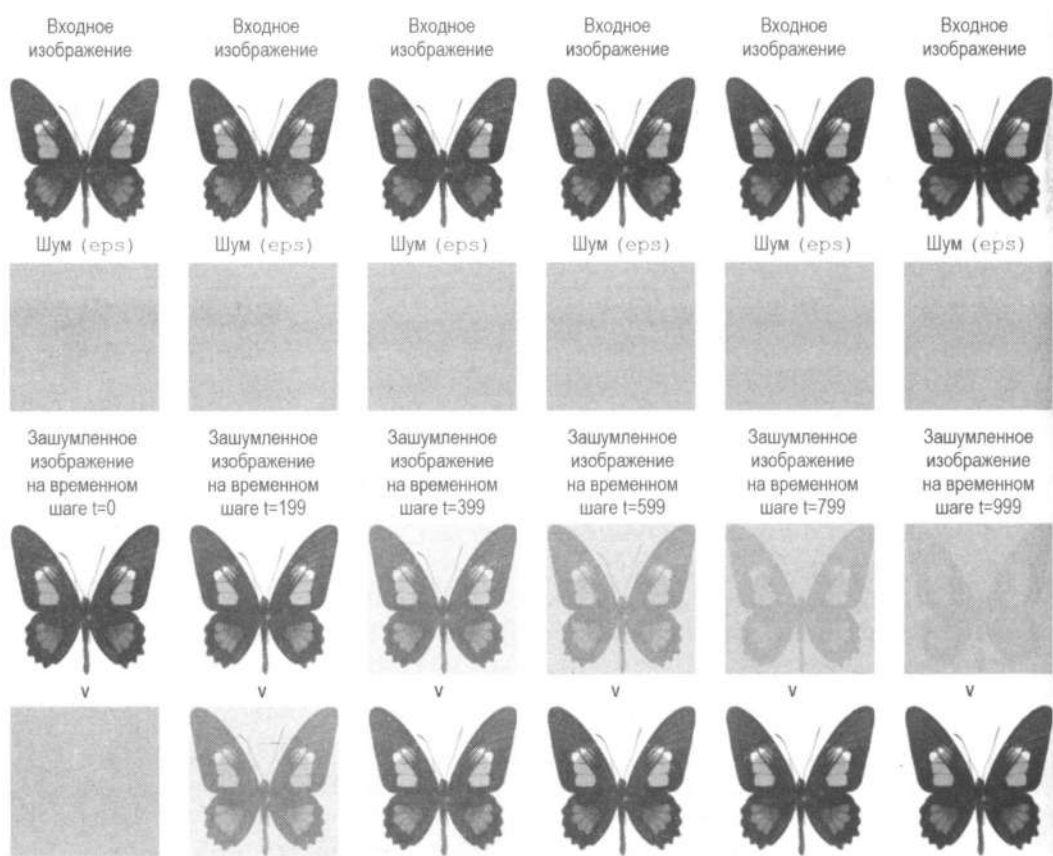


Рис. 4.25. Сравнение целей ϵ ps, x_0 и v : ϵ ps пытается предсказать шум, добавляемый на каждом временном шаге, x_0 подавляет шум на изображении, а v использует смесь обоих подходов

При крайне низком уровне шума целевая функция x_0 тривиально проста (зашумленное изображение почти такое же, как входное), в то время как точное предсказание шума практически невозможно. Аналогично при крайне высоком уровне шума целевая функция ϵ также проста (зашумленное изображение почти равно добавленному чистому шуму), в то время как точное предсказание изображения без шума практически невозможно. Если мы используем x_0 , при обучении более низким уровням шума будет придаваться меньший вес.

Ни один из вариантов не является идеальным, поэтому введем дополнительные целевые функции, которые позволяют модели предсказывать сочетание x_0 и ϵ на разных временных шагах. Скорость v , показанная на последней строке рис. 4.25, является одной из них и определяется как $v = \sqrt{\alpha} \cdot \epsilon + \sqrt{1 - \alpha} \cdot x_0$. Целевая функция ϵ остается одним из наиболее предпочтительных подходов, но важно также знать о ее недостатках и существовании других целевых функций.



Группа исследователей из NVIDIA работала над объединением различных формулировок моделей диффузии в последовательную структуру с четким разделением вариантов проектирования. Они смогли выявить изменения в процессах выборки и обучения, что привело к повышению производительности. Сейчас она известна как *k-диффузия*. Если вам интересно узнать больше о различных целевых функциях, масштабировании и нюансах моделей диффузии, мы рекомендуем прочитать статью «*Elucidating the Design Space of Diffusion-Based Generative Models*» для более глубокого изучения.

Время проекта: обучение собственной модели диффузии

Пришло время вам обучить свою безусловную модель диффузии. Как и раньше, вы научите ее генерировать новые изображения. Главным аспектом этого проекта является создание или поиск хорошего набора данных, который вы сможете использовать.

Если вы хотите использовать существующий набор, хорошей отправной точкой будет отфильтровать наборы данных по классификации изображений из Hugging Face Hub и выбрать какой-нибудь из них по вашему вкусу. Один из главных вопросов, на который вам предстоит ответить: какую часть набора данных вы хотите использовать для обучения? Будете ли вы использовать весь набор, как и прежде, чтобы модель генерировала цифры? Или вы будете использовать определенный класс набора (например, «кошки», чтобы получить экспертную модель по кошкам)? Или вы будете использовать подмножество набора данных (например, только изображения с определенным разрешением)?

Если вы хотите вместо этого загрузить новый набор данных, первым шагом будет поиск информации и доступ к ней. Чтобы поделиться набором, наиболее простой подход — применить функцию `ImageFolder` из библиотеки `datasets`. Затем вы можете загрузить набор в Hugging Face Hub и использовать его в своем проекте.

Получив данные, подумайте о шагах предварительной обработки, определении модели и цикле обучения. Вы можете использовать код из этой главы в качестве отправной точки и изменить его в соответствии с вашим набором данных.

Заключение

В этой главе мы обучили модель диффузии с нуля и подробно изучили каждый ее компонент. Мы провели обучение с помощью сети UNet, которая получает искаженные изображения в качестве ввода и может предсказать их шум. При обучении мы добавляли шум разной величины в соответствии со случайным числом временных шагов. Одна из проблем, с которой мы столкнулись, заключалась в том, что для добавления шума на большом количестве шагов (например, 900) нам нужно было бы выполнить большое количество итераций. Чтобы исправить это, мы использовали «трюк репараметризации», который позволяет напрямую получать шумный вход на определенном временном шаге. Модель обучалась минимизировать разницу между прогнозами шума и фактическим входным шумом. Для вывода мы выполнили итеративный процесс, в котором модель уточняла начальные случайные входные данные. Вместо того чтобы сохранять окончательное предсказание одного шага диффузии, мы итеративно изменяли входные данные x на небольшую величину в направлении прогноза. Это одна из причин, по которой вывод с помощью моделей диффузии становится медленным и является одним из его главных недостатков по сравнению с моделями типа GAN.

Мир диффузии быстро меняется. Уже сейчас существует множество улучшений, таких как графики шума, модели, методы обучения и т. д. В этой главе ключевое внимание уделено основам, которые позволяют нам перейти к *обусловленной генерации* (например, созданию изображений, заданных входным промптом) и предоставляют базу для более глубокого погружения в мир диффузии. Некоторые материалы в этой главе могут помочь вам лучше узнать эту область.

Для дополнительного чтения мы предлагаем следующие материалы:

- ♦ Пост «*Annotated Diffusion Model*» содержит технический обзор графика DDPM.
- ♦ Рецензия «*What are Diffusion Models?*» на Github автора Лилиан Вэнг (Lilian Weng) отлично подходит для более глубокого погружения в математику.
- ♦ Статья «*Denoising Diffusion Probabilistic Models*».
- ♦ Работа графика шума Karras по унификации формулировок моделей диффузии.
- ♦ Статья «*Simple Diffusion: End-to-End Diffusion for High Resolution Images*» объясняет, как настроить график выборки для разных размеров.

Вопросы

1. Объясните алгоритм диффузионного вывода.
2. Какова роль графика шума?

3. Какие характеристики важно отслеживать при создании обучающего набора данных изображений?
4. Почему в примерах мы случайным образом переворачиваем обучающие изображения?
5. Как можно оценить генерации моделей диффузии?
6. Как значения `beta_end` влияют на процесс диффузии?
7. Почему мы используем UNets вместо VAE в качестве основной модели для диффузии?
8. Какие преимущества и проблемы возникают при добавлении компонентов трансформеров (например, слои внимания или архитектуры на основе трансформера) к моделям диффузии?

Ответы на эти вопросы и решения последующих задач вы можете найти в репозитории книги на GitHub.

Задачи

1. *Репараметризация.* Покажите, что

$$x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon$$

эквивалентно

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon.$$

Обратите внимание, это не тривиальный пример, и он не требует использования моделей диффузии. Мы рекомендуем ознакомиться с руководством «*A Beginner's Guide to Diffusion Models: Understanding the Basics and Beyond*».

Также важно знать, как объединить два гауссиана. Если у вас есть два гауссиана с разной дисперсией, $N(\mu_1, \sigma_1^2)$ и $N(\mu_2, \sigma_2^2)$, результирующий гауссиан будет $N(\mu_1 + \mu_2, \sigma_1^2 + \sigma_2^2)$.

2. *График шума DDIM.* В этой главе мы использовали график DDPM, который может требовать сотни или тысячи итераций для достижения высококачественных результатов. Недавние исследования показали, что возможно достижение хороших генераций с минимальным количеством шагов (вплоть до одного или двух). Библиотека `diffusers` содержит несколько графиков шума, среди которых есть `DDIMScheduler`, описанный в работе «*Denoising Diffusion Implicit Models*». Создайте с помощью него несколько изображений. При использовании `DDPMScheduler` в этой главе выборка заняла у нас 1000 шагов. Сколько шагов вам потребуется для генерации изображений похожего качества с помощью `DDIMScheduler`? Поэкспериментируйте с переключением между графиками.

Stable Diffusion и обусловленная генерация

В предыдущей главе мы рассмотрели модели диффузии и идею итеративного уточнения. В конце мы создали собственную модель и сгенерировали изображения, но ее обучение заняло много времени. Кроме того, у нас не было контроля над генерациями.

В этой главе мы рассмотрим модели, обусловленные текстом, которые могут эффективно генерировать изображения на основе описаний. В качестве примера мы изучим Stable Diffusion. Однако прежде, чем перейти к ее изучению, мы рассмотрим, как работают *условные* модели в целом, а также как появились современные модели, способные преобразовывать текст в изображение.

Добавление контроля: модели с условиями

Прежде чем приступить к генерации изображений из текстовых описаний, сперва начнем с чего-то более простого. Рассмотрим, как можно управлять выводом нашей модели, чтобы он соответствовал определенным условиям, типам или классам. Для этого мы можем использовать метод, называемый *обуславливанием* (**Conditioning**). Он заключается в том, чтобы «попросить» модель сгенерировать не просто любое изображение, а такое, которое соответствовало бы предопределенному классу. В таком контексте обуславливание позволяет управлять выводом в процессе генерации, предоставляя дополнительную информацию вроде метки или промпта.

Обуславливание — это простая, но эффективная концепция. Для начала внесем некоторые изменения в модель диффузии, которую мы использовали в *главе 4*. Во-первых, вместо «бабочек» мы будем использовать набор Fashion MNIST, данные в котором распределены по классам. Он содержит тысячи изображений одежды, которые связаны с метками из 10 классов. Во-вторых, что особенно важно, мы предоставим модели два входных параметра вместо одного: изображение (как и прежде) и метку класса, к которой оно относится. Таким образом, мы ожидаем, что модель изучит ассоциации между ними. Это поможет ей понять отличительные особенности каждого предмета одежды друг от друга (свитеров, ботинок и др.).

Обратите внимание, это не является задачей классификации. Нам не нужно, чтобы модель сообщала, к какому классу принадлежит изображение. Нам необходимо, чтобы она выполняла ту же задачу, что и в *главе 4*, — генерировала правдоподобные изображения, которые выглядели бы похожими на обучающий набор. Важное отличие здесь — мы предоставляем модели дополнительную информацию об изображениях. Функция потерь и стратегия обучения останутся прежними.

Подготовка данных

Поскольку для решения нам нужен набор, элементы в котором разделены на отдельные классы, наборы, использующиеся в задачах классификации с помощью компьютерного зрения, идеально для этого подходят. Например, мы могли бы использовать ImageNet, где миллионы изображений разделены на 1000 классов. Однако обучение на нем займет очень много времени. Самый разумный шаг — использовать набор меньшего размера. Так мы можем убедиться, что все работает, как ожидалось. Цикл обратной связи при этом останется достаточно коротким, чтобы мы могли быстро выполнить итерации уточнения и быть уверенными, что все делаем верно.

Набор Fashion MNIST разработан компанией Zalando и имеет открытый исходный код. Он отлично нам подходит, поскольку имеет нужные характеристики: компактный размер, черно-белые изображения и разбиение элементов по десяти классам. Отличие от обычного набора MNIST заключается в том, что классы соответствуют разным типам одежды (вместо цифр), а изображения содержат больше деталей, чем простые рукописные цифры.

Как и в *главе 3*, нам нужно настроить библиотеку `matplotlib` для использования инвертированных серых цветов, чтобы они соответствовали набору Fashion MNIST. Рассмотрим несколько примеров (рис. 5.1).

```
import matplotlib as mpl
from datasets import load_dataset

from genaibook.core import show_images

mpl.rcParams["image.cmap"] = "gray_r"

fashion_mnist = load_dataset("fashion_mnist")
clothes = fashion_mnist["train"]["image"][:8]
classes = fashion_mnist["train"]["label"][:8]
show_images(clothes, titles=classes, figsize=(4, 2.5))
```

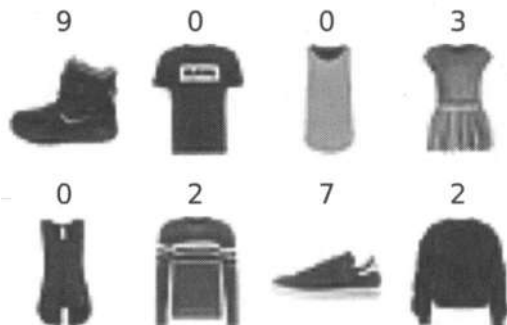


Рис. 5.1. Несколько примеров из набора Fashion MNIST

Класс 0 соответствует футболкам, 2 — свитерам, а 9 — ботинкам. Далее подготовим набор и класс `DataLoader` аналогично тому, как мы это делали в *главе 4*. Но в этот раз мы также включим информацию о классах изображений в качестве входных данных. У нас нет необходимости менять размер изображений, но мы все равно увеличим их с 28×28 до 32×32 . Это позволит сохранить исходное качество и поможет UNet делать лучшие прогнозы¹.

Дополнительно мы применим **заполнение (Padding)**. Оно поможет избежать проблем, когда преобразование данных обрежет или исказит информацию на краях изображений.

```
import torch
from torchvision import transforms

preprocess = transforms.Compose(
    [
        #Случайный поворот (аугментация данных)
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(), #Конвертируем в тензор (0, 1)
        transforms.Pad(2), #Добавляем по 2 пикселя с каждой стороны
        transforms.Normalize([0.5], [0.5]), #Приводим к виду (-1, 1)
    ]
) ❶

def transform(examples): ❷
    images = [preprocess(image) for image in examples["image"]]
    return {"images": images, "labels": examples["label"]} ❸

train_dataset = fashion_mnist["train"].with_transform(transform) ❹

train_dataloader = torch.utils.data.DataLoader(
    train_dataset, batch_size=256, shuffle=True
) ❺
```

В этом коде мы делаем следующее:

- ❶ Определяем ряд преобразований (переворот, преобразование в тензор, заполнение и нормализация), которые будут применены к изображениям.
- ❷ Обработываем пакет изображений с помощью указанных преобразований.
- ❸ Возвращаем словарь, содержащий обработанные изображения и соответствующие им метки.

¹ В *главе 4* мы изменяли размер изображений с бабочками, поскольку они были очень большими (512×283). Мы уменьшали их размер, чтобы ускорить обучение. Сейчас они достаточно маленькие и не требуют изменения, но мы все равно увеличим их до 32×32 , чтобы было удобно использовать степени числа 2, которые более предпочтительны при работе каскадных слоев UNet.

- ➊ Загружаем часть набора, которая служит для обучения. Использование метода `with_transform` гарантирует, что элементы будут возвращены после применения преобразований.
- ➋ Создаем класс `DataLoader`, который будет создавать пакеты и перемешивать данные, упрощая код.

Создание модели, обусловленной классом

Модель `UNet` из библиотеки `diffusers` способна принимать пользовательскую информацию об обусловливании. Создадим модель, похожую на ту, что мы использовали в главе 4, но добавим аргумент `num_class_embeds` в конструктор `UNet`. Он будет сообщать модели, что мы хотели бы использовать метки классов в качестве дополнительного условия. Так как в наборе `Fashion MNIST` десять классов, аргумент будет иметь вид `num_class_embeds=10`.

```
from diffusers import UNet2DModel

model = UNet2DModel(
    in_channels=1, #1 канал для изображений в оттенках серого
    out_channels=1,
    sample_size=32,
    block_out_channels=(32, 64, 128, 256),
    num_class_embeds=10, #Включаем обусловливание с помощью классов
)
```

Чтобы делать прогнозы с помощью этой модели, мы должны передать метки классов в качестве дополнительных входных данных в метод `forward()`.

```
x = torch.randn((1, 1, 32, 32))
with torch.inference_mode():
    out = model(x, timestep=7, class_labels=torch.tensor([2])).sample
out.shape

torch.Size([1, 1, 32, 32])
```



Модель из главы 4 также можно считать обусловленной, поскольку мы передавали в нее временной шаг. Ожидалось, что это дополнительное знание (насколько далеко мы продвинулись в процессе диффузии) поможет ей генерировать более реалистичные изображения.

Временной шаг и метка класса превращаются в эмбединги, которые модель использует во время прямого прохода. На всех этапах они проецируются на измерения, количество которых соответствует количеству каналов в каждом слое. Затем они добавляются к выходам каждого слоя. Таким образом, информация об обусловливании подается в каждый блок `UNet`, что дает модели достаточно возможностей научиться эффективно ее использовать.

Эмбединги эффективны в диффузионных моделях, поскольку они обеспечивают компактное и плотное представление информации об условиях вроде временных шагов и меток классов, которые модель может легко интегрировать по всей архи-

текстуре UNet. Гибкость эмбедингов также позволяет эффективно обрабатывать различные типы условий, которые могут быть непрерывными (временные шаги), категориальными (метки классов) или даже текстовыми (промпты).

Обучение модели

Добавление шума работает так же хорошо на изображениях в оттенках серого, как и на изображениях с бабочками. Рассмотрим влияние шума по мере увеличения временных шагов при шумоподавлении (рис. 5.2).

```
from diffusers import DDPMStochasticScheduler

scheduler = DDPMStochasticScheduler(
    num_train_timesteps=1000, beta_start=0.001, beta_end=0.02
)
timesteps = torch.linspace(0, 999, 8).long()
batch = next(iter(train_dataloader))

#Загружаем 8 изображений из набора данных и
#добавляем к ним увеличивающееся количество шума
x = batch["images"][0].expand([8, 1, 32, 32])
noise = torch.randn_like(x)
noised_x = scheduler.add_noise(x, noise, timesteps)
show_images((noised_x * 0.5 + 0.5).clip(0, 1))
```

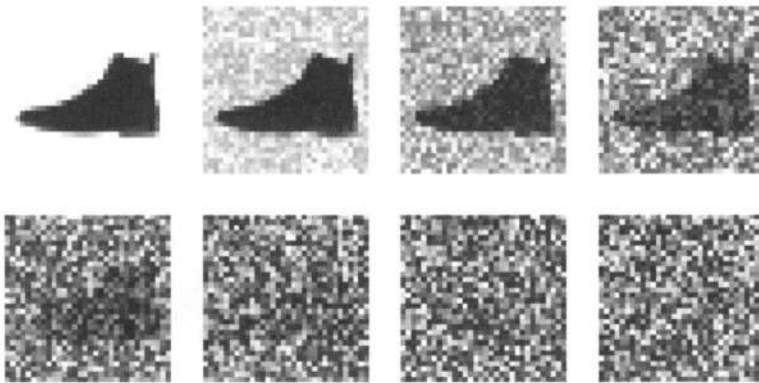


Рис. 5.2. Влияние шума по мере увеличения временных шагов при шумоподавлении

Цикл обучения будет почти таким же, как в главе 4, за исключением того, что теперь мы передаем метки классов. Обратите внимание, это просто дополнительная информация для модели, и она никак не влияет на то, как определяется функция потерь. Мы также используем метод `torch`, чтобы отобразить индикацию выполнения.

Теперь мы можем начать обучение. Не пугайтесь следующего кода, он похож на то, что мы делали с безусловной генерацией. Рекомендуем сравнить их для наглядности и лучшего понимания того, что мы делаем. Попробуйте найти все различия.

Скрипт ниже выполняет следующее:

1. Загружает пакет изображений и соответствующие им метки (используя `train_dataloader` и `tqdm` для итераций по пакету).
2. Добавляет шум к изображениям на основе временного шага (используя график шума `scheduler.add_noise()`).
3. Передает искаженные изображения в модель вместе с метками классов для обучения (используя функцию `model()`).
4. Вычисляет потери.
5. Возвращает потери и обновляет веса модели с помощью оптимизатора.

```
from torch.nn import functional as F
from tqdm import tqdm

from genaibook.core import get_device

#Инициализируем график шума
scheduler = DDPMsScheduler(
    num_train_timesteps=1000, beta_start=0.0001, beta_end=0.02
)

num_epochs = 25
lr = 3e-4
optimizer = torch.optim.AdamW(model.parameters(), lr=lr, eps=1e-5)
losses = [] # To store loss values for plotting

device = get_device()
model = model.to(device)

#Обучаем модель (это займет какое-то время)
for epoch in (progress := tqdm(range(num_epochs))):
    for step, batch in (
        inner := tqdm(
            enumerate(train_dataloader),
            position=0,
            leave=True,
            total=len(train_dataloader),
        )
    ):
        #Загружаем входные изображения и классы
        clean_images = batch["images"].to(device)
        class_labels = batch["labels"].to(device)

        #Выбираем шум для добавления к изображениям
        noise = torch.randn(clean_images.shape).to(device)
```

```

#Выбираем случайный временной шаг для каждого изображения
timesteps = torch.randint(
    0,
    scheduler.config.num_train_timesteps,
    (clean_images.shape[0],),
    device=device,
).long()

#Добавляем шум к чистым изображениям в соответствии с величиной
#шума на каждом временном шаге
noisy_images = scheduler.add_noise(clean_images, noise, timesteps)

#Получаем прогноз модели для шума
#Обратите внимание на использование class_labels
noise_pred = model(
    noisy_images,
    timesteps,
    class_labels=class_labels,
    return_dict=False,
)[0]

#Сравниваем прогноз с реальным шумом
loss = F.mse_loss(noise_pred, noise)

#Обновляем отображение потерь
inner.set_postfix(loss=f"{loss.cpu().item():.3f}")

#Сохраняем потери для последующего построения графика
losses.append(loss.item())

#Обратный проход и оптимизация
loss.backward()
optimizer.step()
optimizer.zero_grad()

```

После завершения обучения мы можем построить график потерь (рис. 5.3), чтобы увидеть, как работает модель.

```

import matplotlib.pyplot as plt

plt.plot(losses)

```

Выборка

Теперь, передавая метку желаемого класса, можно генерировать изображения (начиная со случайного шума и постепенно его уменьшая).

```

def generate_from_class(class_to_generate, n_samples=8):
    sample = torch.randn(n_samples, 1, 32, 32).to(device)

```

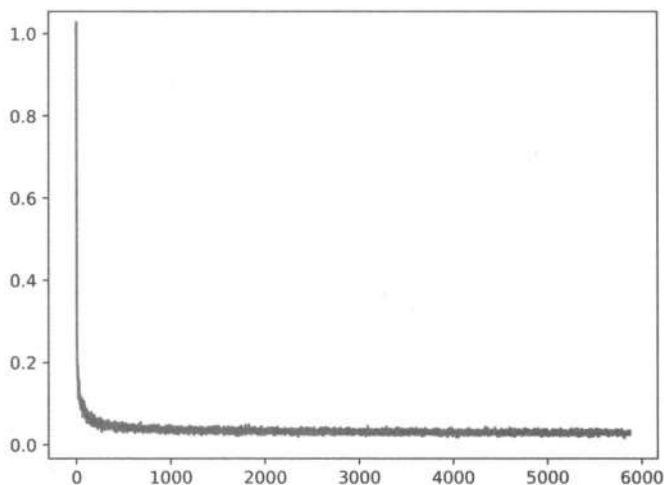


Рис. 5.3. График потерь

```
class_labels = [class_to_generate] * n_samples
class_labels = torch.tensor(class_labels).to(device)

for _, t in tqdm(enumerate(scheduler.timesteps)):
    #Получаем прогноз модели
    with torch.inference_mode():
        noise_pred = model(sample, t, class_labels=class_labels).sample

    #Обновляем выборку с каждым шагом
    sample = scheduler.step(noise_pred, t, sample).prev_sample
    return sample.clip(-1, 1) * 0.5 + 0.5
```

Например, мы можем сгенерировать класс изображений, который соответствует футболкам (класс «0»), как показано на рис. 5.4.

```
images = generate_from_class(0)
show_images(images, nrow=2)
```

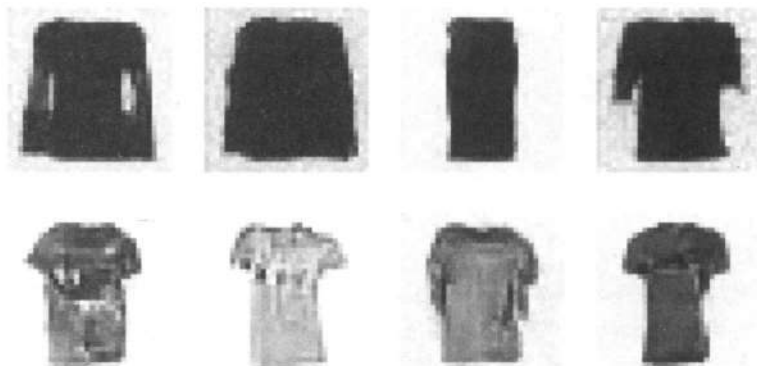


Рис. 5.4. Несколько сгенерированных футболок (класс «0»)

Аналогичным образом генерируем кроссовки (класс «7»), как показано на рис. 5.5.

```
images = generate_from_class(7)
show_images(images, nrows=2)
```

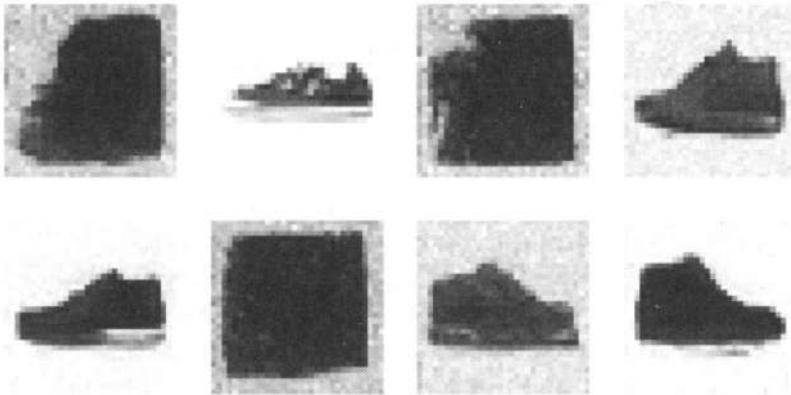


Рис. 5.5. Несколько сгенерированных кроссовок (класс «7»)

Или ботинки, которые соответствуют классу «9» (рис. 5.6).

```
images = generate_from_class(9)
show_images(images, nrows=2)
```



Рис. 5.6. Несколько сгенерированных ботинок (класс «9»)

Как вы можете видеть, сгенерированные изображения все еще содержат некоторый шум. Их можно было бы улучшить, если бы мы более подробно исследовали архитектуру модели, выполнили настройку гиперпараметров и обучались дольше. В любом случае модель узнала формы разных типов одежды и поняла, что класс «9» выглядит иначе, чем класс «0». Если выразиться иначе, модель «привыкла», что цифра «9» соответствует ботинкам. Когда мы просим ее сгенерировать изображение и предоставляем цифру «9», она отвечает изображениями ботинок.

Повышение эффективности: Latent Diffusion

Вам может показаться, что раз мы способны обучить условную модель, нам просто нужно масштабировать ее и обусловить текстом вместо меток классов. Это не совсем так. По мере роста размера изображений растет и вычислительная мощность, необходимая для работы с ними. Это особенно заметно при использовании самовнимания, где количество операций растет квадратично относительно количества входных данных. Изображение размером 128×128 имеет в четыре раза больше пикселей, чем изображение размером 64×64 . Следовательно, оно требует в 16 раз больше памяти и вычислений в слое самовнимания. Это является проблемой при генерации изображений с высоким разрешением.

Модели класса Latent Diffusion² пытаются смягчить эту проблему, используя отдельный вариационный автоэнкодер (рис. 5.7). Как мы видели в главе 2, вариационные автоэнкодеры (VAE) могут сжимать изображения до меньшего пространственного измерения. Изображения, как правило, содержат большой объем избыточной информации. При наличии достаточного количества обучающих данных VAE могут научиться создавать представления гораздо меньшего размера, а затем реконструировать их в изображения с высокой точностью. VAE, используемый в Stable Diffusion, принимает трехканальные изображения и создает четырехка-

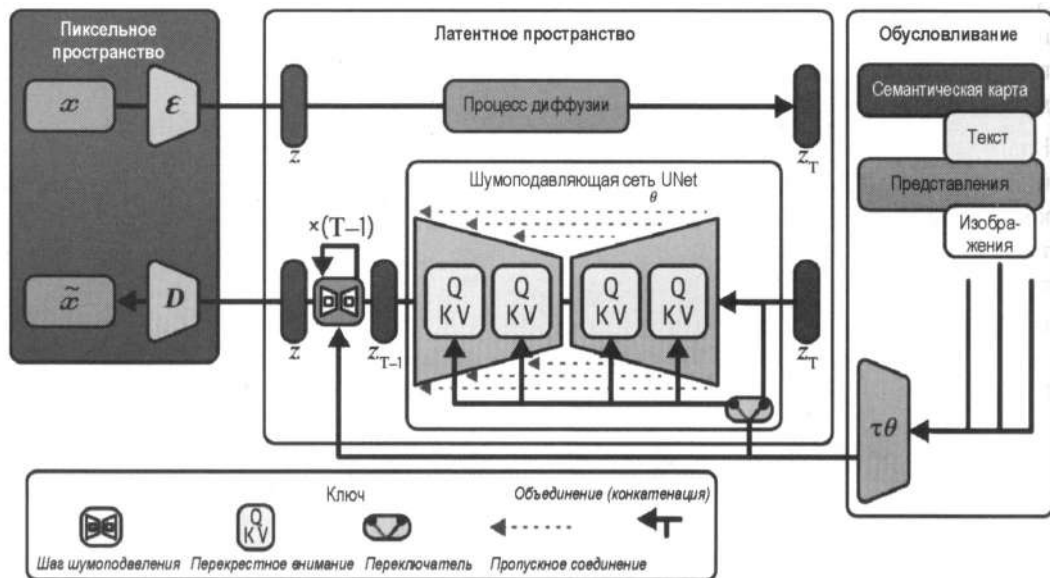


Рис. 5.7. Процесс работы Latent Diffusion: обратите внимание на энкодер и декодер VAE слева для перевода между пиксельным и латентным пространствами

² Latent Diffusion — это класс моделей, которые генерируют новые данные, преобразуя латентные представления (сжатые версии данных), сохраняющие важные особенности и исключая шум. В отличие от традиционных моделей, которые напрямую генерируют выходные данные из входных, Latent Diffusion работают в сжатом латентном пространстве. — Пер.

нальное латентное представление с коэффициентом уменьшения 8 для каждого пространственного измерения. Входное изображение размером 512×512 (786 432 значения) будет сжато до латентного изображения размером $4 \times 64 \times 64$ (16 384 значения).

Применяя процесс диффузии к меньшим латентным представлениям, а не к изображениям с полным разрешением, мы можем добиться множества преимуществ (меньшее использование памяти, меньшее количество слоев, необходимых UNet, более быстрое время генерации и т. д.) и при этом сохранить возможность декодировать результат обратно в изображение с высоким разрешением. Это значительно снижает стоимость обучения и запуска моделей.

Работа, в которой впервые была представлена эта идея, называется «*Latent Diffusion Models*». Она продемонстрировала мощь этой техники, где модели обуславливаются картами сегментации, метками классов и текстом. Впечатляющие результаты привели к дальнейшему сотрудничеству между такими организациями, как LAION, StabilityAI, RunwayML и EleutherAI. Это позволило обучить более мощную модель, известную как Stable Diffusion¹.

Stable Diffusion: подробнее о компонентах

Stable Diffusion — это модель типа Latent Diffusion, которая может генерировать изображения, обусловленные текстовыми промптами. Ее также можно использовать для выполнения многих других задач вроде модификации изображений, о чем мы подробнее расскажем в главе 9.

Благодаря популярности Stable Diffusion появились сотни веб-сайтов и приложений, которые позволяют использовать ее для создания изображений без наличия каких-либо технических знаний. Она также очень хорошо поддерживается библиотекой *diffusers*, которая позволяет делать выборку изображений (рис. 5.8), используя удобный конвейер, как мы делали в главе 1.

```
from diffusers import StableDiffusionPipeline

pipe = StableDiffusionPipeline.from_pretrained(
    "stable-diffusion-v1-5/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
    variant="fp16",
).to(device)

pipe("Watercolor illustration of a rose").images[0]
```

В этом подразделе мы рассмотрим все компоненты, которые позволяют Stable Diffusion делать отличные генерации.

¹ LAION и EleutherAI — некоммерческие организации, ориентированные на открытое машинное обучение. StabilityAI — одна из компаний, которая больше всего продвигала машинное обучение с открытым доступом. RunwayML — компания, создающая инструменты на базе искусственного интеллекта для творческих приложений.



Рис. 5.8. Пример работы Stable Diffusion

Энкодер текста

Как Stable Diffusion понимает текст? Ранее мы уже объясняли, как подача дополнительной информации в UNet позволяет иметь некоторый контроль над типами генерируемых изображений. В случае Stable Diffusion дополнительной подсказкой является текстовый промпт. Во время вывода мы даем описание изображения, которое хотим сгенерировать, и немного чистого шума в качестве отправной точки, а модель делает все возможное, чтобы понизить уровень шума случайных входных данных до уровня, который соответствует описанию.

Чтобы это все могло работать, нам нужно создать числовое представление текста, которое фиксировало бы соответствующую информацию о том, что она описывает. Для этого мы будем использовать *текстовый энкодер*, преобразующий входную строку в текстовые эмбединги, которые затем подаются в UNet вместе с временным шагом и зашумленными латентными представлениями, как показано на рис. 5.9.

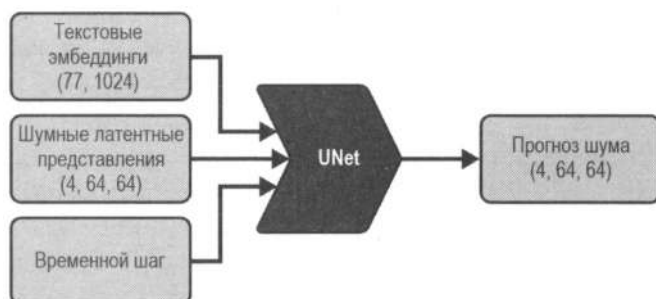


Рис. 5.9. UNet может быть обусловлена несколькими входными данными, такими как временной шаг, метка класса или эмбединги текста

Stable Diffusion использует предварительно обученную модель трансформера на основе CLIP, представленную в *главе 3*. Текстовый энкодер — это модель трансформера, которая принимает последовательность токенов и создает вектор для

каждого из них, как показано на рис. 5.10. В первых версиях (Stable Diffusion 1–1.5), где использовался оригинальный CLIP от OpenAI, текстовый энкодер сопоставлялся с 768-мерным вектором. Поскольку исходный набор данных в CLIP неизвестен, некоторые представители ИИ-сообщества обучили версию с открытым исходным кодом, которая называется OpenCLIP. Stable Diffusion 2 использует текстовый энкодер уже из OpenCLIP, он генерирует 1024-мерные векторы для каждого токена.

Вместо того чтобы объединить векторы всех токенов в одно представление, мы сохраним их отдельно и будем использовать в качестве условия для UNet. Это позволит модели использовать информацию в каждом токене отдельно, а не только общее значение промпта. Поскольку мы извлекаем текстовые эмбединги из внутреннего представления модели CLIP, их часто называют *скрытыми состояниями энкодера*. Далее мы подробнее рассмотрим, как работает энкодер текста «под капотом». Этот процесс аналогичен для моделей, которые мы обсуждали в главе 2.

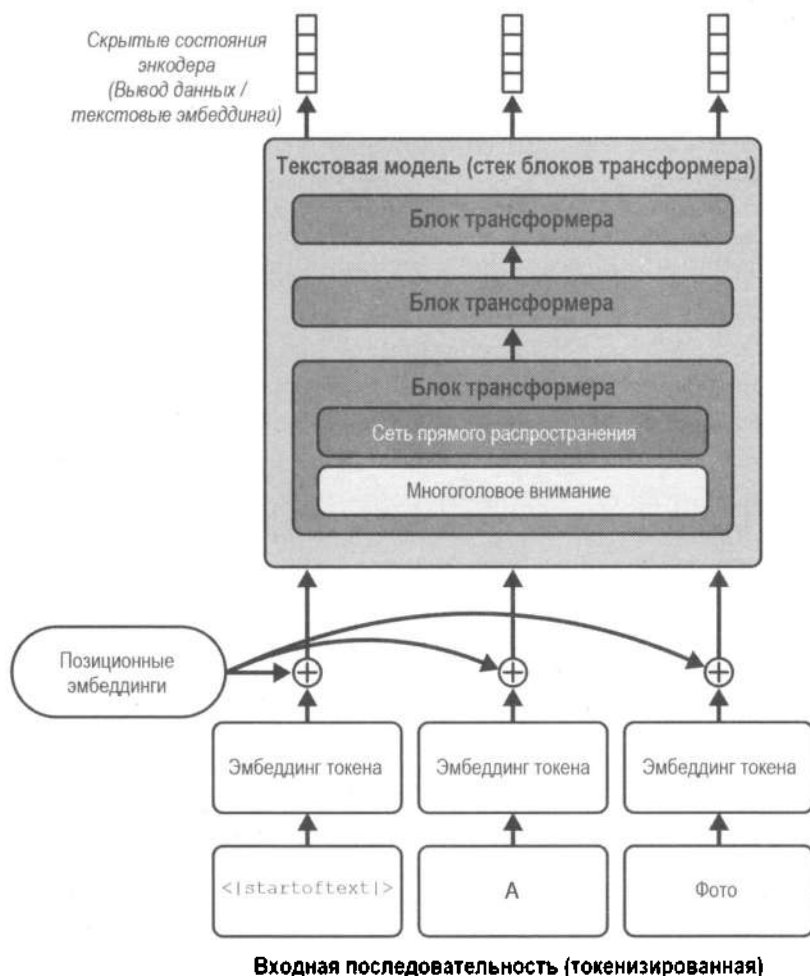


Рис. 5.10. Процесс кодирования текста преобразует входной промпт в набор текстовых эмбедингов (скрытых состояний энкодера), которые затем могут быть переданы в качестве условий в UNet

Первый шаг кодирования текста — токенизация, которая преобразует набор символов в последовательность чисел. В следующем примере видно, как токенизация фразы работает с помощью токенизатора Stable Diffusion. Каждому токени в промпте назначается уникальный номер (например, «photograph» имеет номер 8853 в словаре токенизатора). Существуют также специальные токены, которые предоставляют дополнительный контекст, например где заканчивается предложение.

```
prompt = "A photograph of a puppy"
```

```
#Превращаем текст в последовательность токенов
```

```
text_input = pipe.tokenizer(
    prompt,
    return_tensors="pt",
)
```

```
#Выводим каждый токен и соответствующий ему идентификатор
```

```
for t in text_input["input_ids"][0]:
    print(t, pipe.tokenizer.decoder.get(int(t)))
```

```
tensor(49406) <|startoftext|>
tensor(320) a</w>
tensor(8853) photograph</w>
tensor(539) of</w>
tensor(320) a</w>
tensor(6829) puppy</w>
tensor(49407) <|endoftext|>
```

После того как текст токенизирован, мы можем пропустить его через энкодер, чтобы получить окончательные эмбединги, которые будут переданы в UNet.

```
text_embeddings = pipe.text_encoder(text_input.input_ids.to(device))[0]
print("Text embeddings shape:", text_embeddings.shape)
```

```
Text embeddings shape: torch.Size([1, 77, 768])
```

Вариационный автоэнкодер (VAE)

Вариационные автоэнкодеры (рис. 5.11) необходимы, чтобы мы могли сжимать изображения в меньшее латентное представление и восстанавливать их обратно. Они являются важнейшим компонентом Stable Diffusion. Мы не будем вдаваться в подробности обучения, но некоторые особенности их архитектуры стоит упомянуть. В дополнение к обычным потерям восстановления и KL-дивергенции (описанным в главе 3) VAE используют дополнительную *функцию потерь дискриминатора на основе локальных фрагментов (патчей)*⁴, чтобы помочь модели генериро-

⁴ **Дискриминатор (Discriminator)** — это «оценщик» (или «критик») в генеративно-сопоставительных сетях (GAN). Его цель — отличить сгенерированные данные от реальных. Он оценивает, насколько искусственно сгенерированы данные, и пытается найти подделку. Дискриминатор действует на уровне не всего изображения,

вать правдоподобные детали и текстуры на изображениях. Это помогает избежать размытых выходных данных, типичных для прошлых поколений VAE. Как и текстовые энкодеры, VAE обычно обучаются отдельно, а затем используются в качестве «замороженного» компонента во время обучения и процесса выборки модели диффузии.



Рис. 5.11. Архитектура VAE

Загрузим изображение и посмотрим, как оно выглядит после сжатия и восстановления с помощью VAE. Сначала посмотрим на оригинал (рис. 5.12).

```
from genaibook.core import load_image, show_image, SampleURL
```

```
im = load_image(
    SampleURL.LlamaExample,
    size=(512, 512),
)
show_image(im);
```

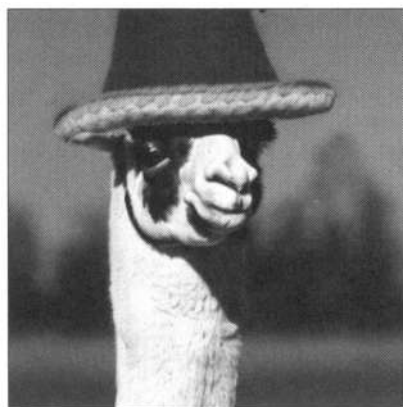


Рис. 5.12. Исходное изображение перед подачей в энкодер VAE

Пропустим изображение через VAE (рис. 5.13).

```
with torch.inference_mode():
    #Обрабатываем изображение
    tensor_im = transforms.ToTensor()(im).unsqueeze(0).to(device) * 2 - 1 ❶
    tensor_im = tensor_im.half() ❷
```

а отдельных локальных фрагментов (патчей) размером $N \times N$. Потери дискриминатора на основе патчей (в оригинале **patch-based discriminator loss**) — это показатель способности дискриминатора правильно классифицировать реальные данные как настоящие, а сгенерированные — как поддельные. — Пер.

```
#Кодируем изображение
latent = pipe.vae.encode(tensor_im) ❸

#Делаем выборку из латентного распределения
latents = latent.latent_dist.sample() ❹
latents = latents * 0.18215 ❺
latents.shape

torch.Size([1, 4, 64, 64])
```

В этом коде происходит следующее:

- ❶ Мы преобразуем изображение в соответствии с входными ожиданиями VAE. Мы делаем его тензором и добавляем измерение с помощью оператора `unsqueeze(0)`, чтобы сделать его пакетом с размером 1. Затем мы нормализуем изображение до диапазона $[-1, 1]$, чтобы оно соответствовало диапазону ввода VAE.
- ❷ Мы преобразуем изображение из формата `float32` в `float16`, поскольку конвейер, который мы загрузили, имеет точность `torch.float16`.
- ❸ Мы передаем изображение через энкодер VAE. Как нам уже известно из главы 3, VAE выводит распределение, из которого мы можем сделать выборку.
- ❹ Получаем доступ к объекту распределения, созданному энкодером VAE, чтобы сгенерировать из него выборку.
- ❺ Масштабируем латентный вектор до фиксированного размера 0,18215. Разработчики Stable Diffusion ввели данный коэффициент масштабирования для гарантии, что латентное пространство будет иметь дисперсию, близкую к 1. Это соответствует приближительному масштабу шума, добавленного в процессе диффузии. Доступ к нему можно получить в `vae.config.scaling_factor`.

Исходные данные представляют собой трехканальное изображение размером 512×512 (786 432 значения). VAE сжимает его в четырехканальное латентное представление размером 64×64 (16 384 значения). Построим график каждого из каналов в латентном представлении (рис. 5.13).

```
show_images(
    [l for l in latents[0]],
    titles=[f"Channel {i}" for i in range(latents.shape[1])],
    ncols=4,
)
```

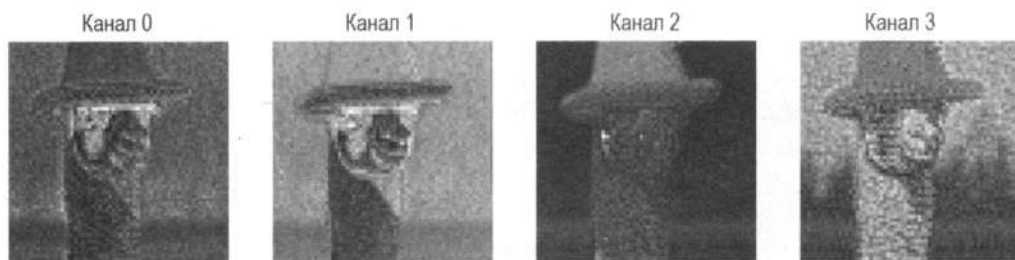


Рис. 5.13. График каждого из 4 каналов в латентном представлении

Теперь, когда изображение закодировано в латентное представление, мы можем декодировать его обратно. В идеале результат должен быть идентичен оригиналу. На практике VAE потенциально может вносить некоторые шумы и артефакты. Посмотрим, как выглядит декодированное изображение (рис. 5.14).

```
with torch.inference_mode():
    image = pipe.vae.decode(latents / 0.18215).sample
    image = (image / 2 + 0.5).clamp(0, 1)
    show_image(image[0].float())
```

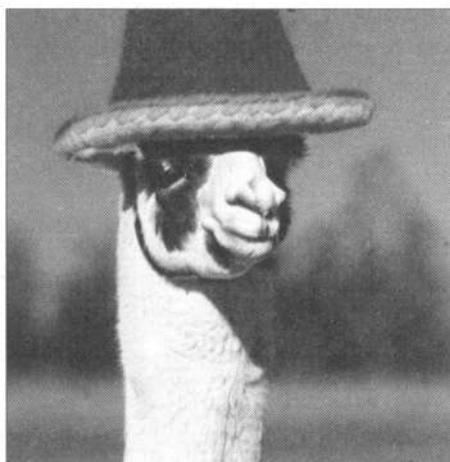


Рис. 5.14. Изображение, пропущенное через VAE

При генерации изображений с нуля мы создаем случайный набор латентных представлений в качестве отправной точки. Далее мы итеративно их уточняем, чтобы сгенерировать выборку. Затем декодер VAE декодирует конечные латентные представления в изображение, которое мы можем просмотреть. Энкодер используется только в том случае, если подразумевается использование существующего изображения (мы рассмотрим это в главе 8).

UNet

Сеть UNet, используемая в модели Stable Diffusion, похожа на ту, что мы использовали в главе 4 для генерации изображений. Но вместо того, чтобы принимать трехканальное изображение в качестве входных данных, она принимает четырехканальное латентное представление. Эмбединги временных шагов подаются так же, как и обусловленность класса из примера в начале этой главы. Помимо этого, наша UNet также должна принимать эмбединги текста в качестве дополнительного условия. Кроме того, сеть «напичкана» слоями перекрестного внимания. Каждое пространственное местоположение в UNet может соответствовать различным токенам при обусловливании текста, получая соответствующую информацию из промпта. На рис. 5.15 показано, как текстовое обусловливание (а также обусловливание на основе временного шага) подается в разных точках.

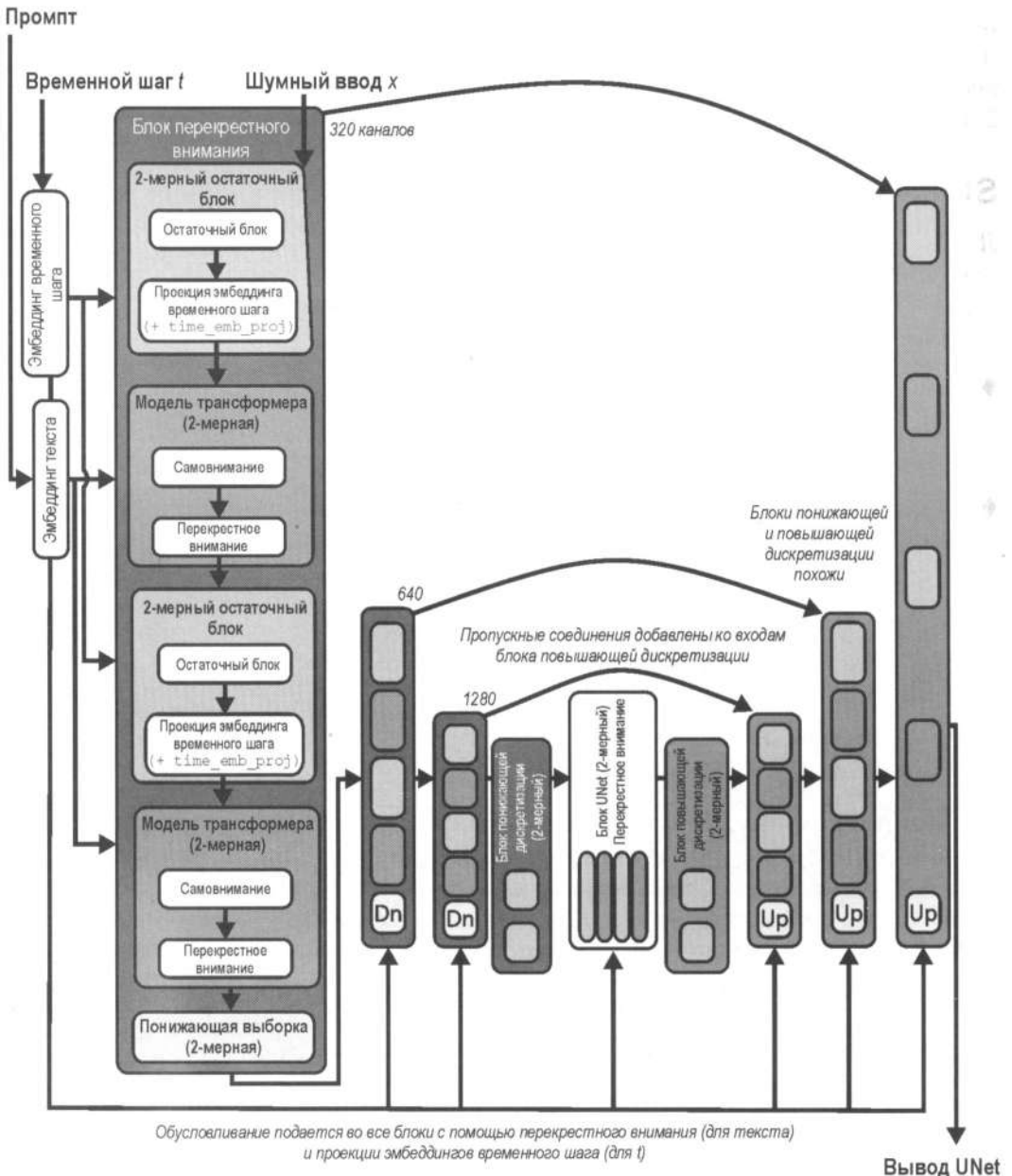


Рис. 5.15. Архитектура обусловленной UNet

Слева на рис. 5.15 вы можете найти входные данные модели. Шумный ввод x в этом случае будет четырехканальным латентным представлением. Временной шаг и промпт также подаются в модель (в форме эмбедингов) в разных точках архитектуры. Как и во всех предыдущих версиях UNet, модель имеет ряд слоев, которые понижают дискретизацию входных данных, а затем повышают ее до исходного размера. Архитектура также имеет пропускные соединения, которые

позволяют модели получать доступ к информации из более ранних блоков. UNet из Stable Diffusion версий 1 и 2 имеет около 860 миллионов параметров. UNet в более поздней версии — Stable Diffusion XL (SDXL) — имеет их еще больше (около 2,6 миллиарда), а также она использует дополнительную информацию об условиях.

Stable Diffusion XL

Летом 2023 года была выпущена новая улучшенная версия Stable Diffusion — Stable Diffusion XL (SDXL). Она использует те же принципы, описанные в этой главе, но с различными улучшениями всех компонентов системы. Некоторые из самых интересных изменений:

- ◆ *Энкодер текста большего размера для захвата лучших представлений промптов*

Он использует выходные данные двух текстовых энкодеров и объединяет эмбединги.

- ◆ *Обусловливание всего*

В дополнение к временному шагу (который несет информацию о количестве шума) и текстовым эмбедингам, SDXL использует следующие дополнительные сигналы обусловливания:

- *Исходный размер изображения*

Вместо того чтобы отбрасывать изображения малого размера в обучающем наборе (они составляют почти 40% от общего объема данных, используемых для обучения SDXL), они масштабируются и используются во время обучения. Однако модель также получает информацию о размерах изображений, которые ей подаются. Таким образом, она узнает, что артефакты масштабирования не должны быть частью больших изображений, и получает поощрения для получения лучшего качества во время вывода.

- *Координаты обрезки*

Входные изображения обычно случайным образом обрезаются во время обучения, поскольку все изображения в пакете должны иметь одинаковый размер. Это может привести к нежелательным эффектам, например к обрезке голов людей или полному удалению объектов с изображения, даже если они могут быть описаны в промпте. После обучения, если мы запросим необрезанное изображение, установив координаты обрезки на (0, 0), модель с большей вероятностью создаст объекты, которые центрированы в кадре.

- *Целевое соотношение сторон*

После начального предварительного обучения на изображениях квадратной формы SDXL был точно настроен на работу с различными соотношениями сторон. Информация об исходном соотношении сторон использовалась в качестве другого сигнала обусловливания. Как и в других сценариях, это позволяет генерировать гораздо более реалистичные изображения ландшафтов и портретов с меньшим количеством артефактов, чем раньше.

◆ *Большее разрешение*

SDXL разработана для создания изображений с разрешением 1024×1024 пикселей (или неквадратных изображений с общим количеством пикселей около 10 242). Как и соотношение сторон, эта возможность была достигнута на этапе тонкой настройки.

◆ *UNet примерно в три раза больше*

Контекст перекрестного внимания стал больше, чтобы можно было учитывать увеличение количества обусловленности.

◆ *Улучшенный VAE*

VAE в SDXL использует ту же архитектуру, что и исходный в Stable Diffusion, но он обучен на большем размере пакета и использует метод экспоненциального скользящего среднего (**Exponential Moving AVERAGE, EMA**) для обновления весов.

◆ *Модель Рефайнера*

В дополнение SDXL имеет модель рефайнера (**Refiner**), которая работает на том же латентном пространстве, что и базовая модель. Однако она обучалась на высококачественных изображениях только в течение первых 20% графика шума. Это означает, что рефайнер «знает», как генерировать изображения с небольшим количеством шума и создавать высококачественные текстуры и детали.

Многие из этих методов уже изучены другими исследователями и разработчиками благодаря тому, что исходный Stable Diffusion является открытым. SDXL объединяет многие из описанных идей, чтобы добиться изображений впечатляющего качества (рис. 5.16), при этом затраты на работу модели стали ниже, а использование памяти увеличилось.

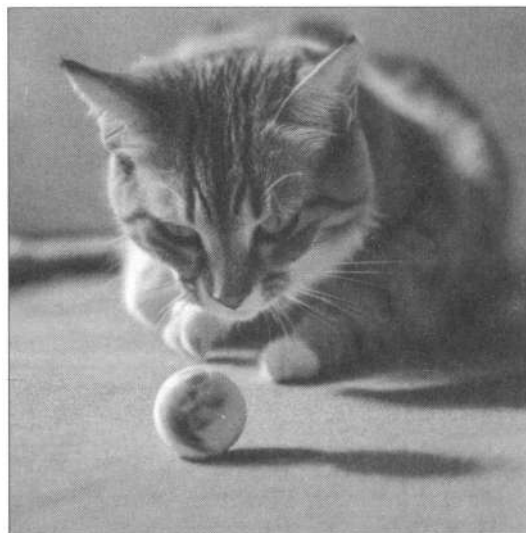


Рис. 5.16. Изображение, созданное с помощью SDXL

FLUX, Stable Diffusion 3 и генерация видео

Развитие моделей диффузии активно продвигается. В июне 2024 года компания StabilityAI выпустила Stable Diffusion 3, а в августе 2024 года компания Black Forest Labs выпустила семейство моделей Flux. Принципы, лежащие в основе этих моделей, аналогичны описанным в этой главе. Однако они содержат иной график шума (называется *rectified flow matching*⁵) и другую архитектуру (диффузионный трансформер вместо UNet).

Оба семейства моделей доступны для работы с изображениями разных размеров и поддерживают разные разрешения. Также они имеют улучшенные возможности быстрого понимания и рендеринга текста. Эту архитектуру можно масштабировать и для генерации видео (семейство моделей CogVideoX).

Все эти экземпляры демонстрируют, что рассмотренные нами принципы (в частности, обусловливание) являются отличными инструментами для управления поведением генеративных моделей, а также, что релизы с открытым исходным кодом могут ускорить исследования в области ИИ.

Руководство без классификаторов

Несмотря на все усилия сделать обусловливание текста максимально полезным, модель при составлении своих прогнозов по умолчанию по-прежнему склонна полагаться в первую очередь на шумное входное изображение, а не на промпт. В некоторой степени это имеет смысл: многие описания лишь отдаленно связаны с соответствующими им изображениями, поэтому модель учится не слишком сильно полагаться на них. Однако такое поведение нежелательно при генерации новых изображений. Если модель не следует промпту, мы можем получить данные, не соответствующие нашему описанию.

Чтобы изменить это, мы должны использовать *руководство* — любой метод, который помогает обеспечивать больший контроль над процессом выборки. Один из вариантов, который мы могли бы применить, — это изменить функцию потерь в пользу какого-либо направления, например если мы хотим, чтобы генерации стали определенного цвета. Это поможет измерить, насколько далеко в целом мы находимся от целевого цвета. Другой альтернативой является использование моделей вроде CLIP или классификаторов, которые оценивают результаты и включают сигналы потерь в процесс генерации. Используя CLIP, мы могли бы сравнить разницу между текстом промпта и сгенерированными эмбедами изображения и направить процесс диффузии, чтобы минимизировать эту разницу.

Еще одной альтернативой является *руководство без классификатора (Classifier-free guidance, CFG)*. Оно объединяет генерации условных и безусловных моделей

⁵ *Rectified flow matching* — метод, который упрощает траекторию генерации данных в генеративных моделях на основе дифференциальных потоков (*Flow Matching*). Цель — улучшить качество генерации и эффективность выборки, особенно в задачах, связанных с переносом данных между двумя эмпирически наблюдаемыми распределениями. — Пер.

диффузии. Во время обучения текстовое обусловливание иногда остается пустым. Тогда модель учится подавлять шум изображений без какой-либо текстовой информации (безусловная генерация). Затем мы делаем два прогноза во время вывода: один с текстовым промптом и один без. Мы можем использовать разницу между ними, чтобы создать окончательный единый прогноз, который продвинет наш вывод еще ближе к нашей цели, указанной прогнозом (обусловленным текстом) в соответствии с коэффициентом масштабирования (масштаб руководства). Нам необходимо, чтобы в итоге генерация изображения лучше соответствовала промпту.

Чтобы задействовать руководство, мы должны изменить прогноз шума, сделав некоторые манипуляции, например `noise_pred = noise_pred_uncond + guide_scale * (noise_pred_text - noise_pred_uncond)`. Это небольшое изменение работает на удивление хорошо и позволяет гораздо лучше контролировать генерации. Мы углубимся в детали реализации позже в этой главе, а пока что посмотрим, как его использовать. Проверьте результаты промпта «An oil painting of a collie in a top hat» с CFG-масштабами, равными 1, 2, 4 и 12 (слева направо на выходном изображении, рис. 5.17).

```
images = []
prompt = "An oil painting of a collie in a top hat"
for guidance_scale in [1, 2, 4, 12]:
    torch.manual_seed(0)
    image = pipe(prompt, guidance_scale=guidance_scale).images[0]
    images.append(image)

from genaibook.core import image_grid

image_grid(images, 1, 4)
```



Рис. 5.17. Результаты работы модели с промптом, который реализован с CFG-масштабами 1, 2, 4 и 12 (слева направо)

Как вы можете видеть, более высокие значения приводят к результатам, которые лучше соответствуют описанию, но слишком высокие значения могут начать перенасыщать изображение. В статье «*Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding*» от Google Imagen говорится, что это связано с тем, что текущие прогнозы превышают границы $[-1, 1]$, в которых обучается модель. Поскольку модель диффузии работает итеративно, ей приходится иметь дело с входными данными, значения которых не видны во время обучения, что может

привести к перенасыщению, «постеризации» и даже сбоям в генерации вывода. В работе исследователи предложили метод, который называется *динамическим порогом*, чтобы удерживать значения в пределах диапазона и значительно улучшать качество при высоких масштабах CFG.

Собираем все вместе: аннотированный цикл выборки

Теперь, когда вы знаете, что делает каждый компонент, объединим их, чтобы сгенерировать изображение. Ниже приведены настройки, которые мы будем использовать.

```
#Некоторые настройки
prompt = [
    "Acrylic palette knife painting of a flower"
] #Что мы хотим сгенерировать
height = 512 #Высота по умолчанию для Stable Diffusion
width = 512 #Ширина по умолчанию для Stable Diffusion
num_inference_steps = 30 #Количество шагов шумоподавления
guidance_scale = 7.5 #Масштаб для руководства без классификатора (CFG)
seed = 42 #Начальное значение для генератора случайных чисел
```

Первый шаг — закодировать текстовый промпт «Acrylic palette knife painting of a flower». Поскольку мы планируем использовать CFG, создадим два набора текстовых эмбеддингов: один для промпта, а другой будет пустой строкой, которая является безусловным вводом. Несмотря на то что здесь мы будем использовать безусловный ввод, данная настройка обеспечивает большую гибкость. Например, мы можем сделать следующее:

◆ Закодировать негативный промпт вместо пустой строки

Добавление негативного промпта позволяет направлять модель, избегая движения в определенном направлении. В упражнении 6 к этой главе вы сможете попробовать негативные промпты.

◆ Объединить несколько промптов с разным весом

Вес промпта позволяет увеличивать или уменьшать акцент на определенные его части. Вы узнаете больше об этом в главе 8.

Реализуем создание текстовых эмбеддингов.

```
#Токенизируем ввод
text_input = pipe.tokenizer(
    prompt,
    #Заполняем до максимальной длины, чтобы обеспечить одинаковую
    #форму входных данных
    padding="max_length",
    return_tensors="pt",
)
```

```
#Делаем то же самое для безусловного ввода (пустая строка)
uncond_input = pipe.tokenizer(
    "",
    padding="max_length",
    return_tensors="pt",
)

#Пропускаем оба эмбединга через текстовый энкодер
with torch.inference_mode():
    text_embeddings = pipe.text_encoder(text_input.input_ids.to(device))[0]
    uncond_embeddings = pipe.text_encoder(uncond_input.input_ids.to(device))[0]
```

```
#Объединяем оба набора текстовых эмбедингов
text_embeddings = torch.cat([uncond_embeddings, text_embeddings])
```

Далее мы создаем случайные начальные латентные представления и настраиваем график шума на желаемое количество шагов вывода.

```
#Подготавливаем график шума
pipe.scheduler.set_timesteps(num_inference_steps)

#Подготавливаем случайные начальные латентные представления
latents = (
    torch.randn(
        (1, pipe.unet.config.in_channels, height // 8, width // 8),
    )
    .to(device)
    .half()
)
latents = latents * pipe.scheduler.init_noise_sigma
```

Затем мы проходим по каждому шагу выборки. Получив прогноз на каждом шаге, мы используем его для обновления латентных представлений.

```
for t in pipe.scheduler.timesteps:
    #Создаем две копии латентных представлений, чтобы они соответствовали
    #двум текстовым эмбедингам (безусловному и условному)
    latent_input = torch.cat([latents] * 2)
    latent_input = pipe.scheduler.scale_model_input(latent_input, t)

    #Предсказываем остаточный шум для безусловных и условных
    #латентных представлений
    with torch.inference_mode():
        noise_pred = pipe.unet(
            latent_input, t, encoder_hidden_states=text_embeddings
        ).sample

    #Разделяем прогноз на безусловную и условную версии
    noise_pred_uncond, noise_pred_text = noise_pred.chunk(2)
```



```
#Выполняем руководство без классификатора (CFG)
noise_pred = noise_pred_uncond + guidance_scale * (
    noise_pred_text - noise_pred_uncond
)

#Обновляем латентные представления для следующего временного шага
latents = pipe.scheduler.step(noise_pred, t, latents).prev_sample
```

Обратите внимание на шаг CFG. Наше окончательное предсказание шума — `noise_pred_uncond + guide_scale * (noise_pred_text - noise_pred_uncond)`. Эта строка «уводит» предсказание от безусловной версии к обусловленной (на основе промпта). Попробуйте изменить масштаб руководства, чтобы лучше изучить, как это влияет на вывод.

К концу цикла латентные представления должны соответствовать правдоподобному изображению, которое соответствует промпту. Последний шаг — декодировать латентные представления в изображение с помощью VAE, чтобы мы могли увидеть результат (рис. 5.18).

```
#Масштабируем и декодируем латентные представления с помощью VAE
latents = 1 / pipe.vae.config.scaling_factor * latents
with torch.inference_mode():
    image = pipe.vae.decode(latents).sample
image = (image / 2 + 0.5).clamp(0, 1)

show_image(image[0].float());
```

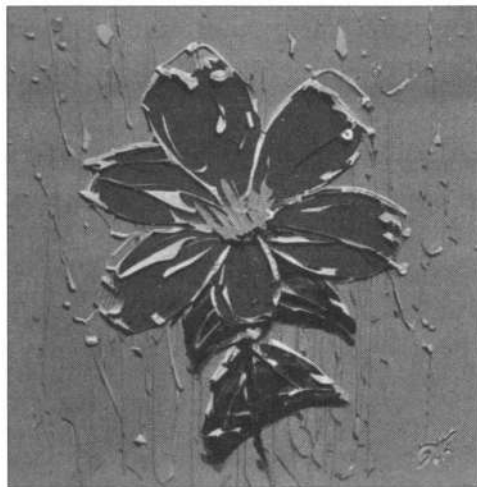


Рис. 5.18. Результат работы модели: изображение хорошо соответствует промпту

Если вы изучите исходный код в `StableDiffusionPipeline`, то заметите, что код, рассмотренный выше, очень похож на метод `call()`, используемый конвейером. Наша версия показывает, что «за кулисами» не происходит ничего магического. Вы можете использовать это как подсказку, когда мы столкнемся с дополнительными

конвейерами, которые добавляют различные «трюки» к нашей реализации модели (ее можно считать прочной основой для дальнейшего изучения).

Открытые данные, открытые модели

Набор данных LAION-5B включает более 5 миллиардов изображений и соответствующих им подписей (пары «изображение-подпись»). Для его создания были взяты все URL-адреса изображений, найденные в CommonCrawl⁶, а затем с помощью CLIP отобраны из них только пары «изображение-подпись» с высокой степенью сходства между текстом и изображением.

Этот набор был создан ML-сообществом, которое увидело необходимость в том, чтобы он находился в открытом доступе. До этого лишь несколько исследовательских лабораторий в крупных компаниях имели доступ к подобным данным. Эти организации держали детали своих разработок в тайне, что делало невозможным проверку или воспроизведение их результатов. Создание общедоступного источника с индексами URL-адресов и описаний позволило отдельным небольшим ИИ-сообществам и организациям обучать собственные модели и проводить исследования, которые без этого были бы невозможны.

Первая версия Latent Diffusion была одной из таких моделей. Ее обучали на предыдущей версии набора данных LAION с 400 миллионами пар «изображение-текст» от компании Computer Vision Group. Релиз модели, обученной с помощью открытой LAION, ознаменовал событие, когда надежная модель преобразования текста в изображение стала доступна всему исследовательскому сообществу.

Успех Latent Diffusion показал потенциал такого подхода, поэтому он также был реализован в последующей версии Stable Diffusion, чье обучение занимало значительное время работы GPU. Даже используя открытый набор LAION, лишь немногие могли позволить себе инвестировать время в подобное. Поэтому публичный выпуск весов и кода модели стали важным событием. Это был первый раз, когда мощная модель, имеющая возможности, равные лучшим альтернативам с закрытым исходным кодом, стала доступна всем.

Общедоступность сделала Stable Diffusion популярной для исследователей и разработчиков, изучающих подобные технологии. В последние годы появились сотни исследовательских работ, которые основаны на базовой модели, но при этом они привносят новые возможности или инновационные способы улучшить ее скорость и качество. Помимо этого, модель стала популярна среди интернет-сообщества в целом. Кто-то (не обязательно имеющий опыт в машинном обучении) даже находчиво умудрялся «взламывать» ее, чтобы использовать в своих творческих сценариях или оптимизировать для более быстрого вывода (рис. 5.19).

Бесчисленные стартапы нашли способы интегрировать эти быстро совершенствующиеся модели в свои продукты, порождая целую экосистему новых приложений.

⁶ CommonCrawl — это открытое хранилище данных, полученных с помощью скраппинга (Scraping). Оно работает аналогично тому, как Google индексирует сайты в Интернете для своего поиска.



Рис. 5.19. В области преобразования текста в изображение творчество также получило бурное развитие

Время после появления Stable Diffusion показало, насколько сильно влияние открытого обмена подобными технологиями. По качеству модель стала конкурентоспособна с такими коммерческими альтернативами того времени, как DALL-E и Midjourney. Множество разработчиков-любителей придумали собственные улучшения для нее. Мы надеемся, что этот пример вдохновит других последовать ему и поделиться своей работой со всеми в будущем. Некоторые улучшения и дополнительные методы настройки моделей мы рассмотрим в *главах 7 и 8*.



Помимо обучения Stable Diffusion, набор данных LAION-5B использовался во многих других исследовательских работах. Одним из примеров является OpenCLIP. Это попытка ИИ-сообщества обучить высококачественные CLIP-модели, имеющие открытый исходный код, воспроизводить качество, аналогичное исходному. CLIP полезны для многих задач, наподобие поиска изображений или классификации с нулевым выстрелом. Наличие «прозрачности» в данных, используемых для обучения, также позволяет исследовать влияние масштабирования моделей на способность правильно воспроизводить результаты и делать исследования более доступными.

Закат LAION-5B

Огромный успех генеративных моделей, которые преобразуют текст в изображение, и коммерческих приложений, основанных на них, вызвал множество опасений относительно источников, откуда были взяты данные, а также их содержание.

Набор включает миллионы URL-адресов, указывающих на изображения, которые могут содержать защищенные авторским правом материалы, например фотографии, произведения искусства, комиксы и иллюстрации. Исследования показали, что этот набор также включает в себя конфиденциальную личную информацию, например персональные медицинские данные, которые были доступны в Интернете.

Использование такого набора для обучения дает возможность воспроизводить контент, который усугубляет общественные предубеждения, поскольку их можно использовать для производства откровенного материала для взрослых. Кроме того, такие модели могут генерировать контент, похожий на обучающие данные. Это

может привести к появлению данных, чрезвычайно похожих на защищенный авторским правом материал. Однако, поскольку обучение происходит на открытых наборах, такие «предубеждения» и проблемный контент можно изучать, анализировать и смягчать. Более поздние исследования показывают, что набор LAION-5B не способен помочь отфильтровать явно незаконный материал, касающийся насилия и безопасности детей, что привело к его запрету.

Альтернативы

На смену ушедшим наборам приходят другие общедоступные альтернативные варианты, такие как COYO-700M и DataComp-1B. Изображения в них получены тем же способом, что и в LAION (скраппинг⁷), но они не содержат запрещенного контента, который привел бы к их немедленной деактивации. Наборы по-прежнему содержат те же проблемы «предвзятости» данных и материалы, защищенные авторским правом, которые были упомянуты в предыдущем разделе. CommonCanvas — набор данных меньшего масштаба (70 миллионов пар «изображение-текст»), но он содержит исключительно изображения с открытой лицензией Creative Commons.

Добросовестное и коммерческое использование

В законодательстве некоторых стран есть пункты, затрагивающие добросовестное использование открытых наборов данных касательно использования авторского права для исследовательских целей. В других странах уже есть прецеденты, которые расцениваются как благоприятные в отношении использования подобных данных для обучения ML-моделей. Но как относиться к тому, что модель, обученная на подобных материалах, используется в коммерческих целях? Это сложный вопрос, и в настоящее время он рассматривается в судах нескольких юрисдикций в США и Европе с точки зрения закона об авторском праве, добросовестного использования наборов в исследовательских целях, конфиденциальности, экономического влияния инструментов ИИ на рабочие места и пр. У нас нет ответа на данный вопрос, но подобная юридическая серая зона отдаляет исследовательское ИИ-сообщество и сообщество разработчиков ПО с открытым исходным кодом от использования открытых наборов данных. Для Stable Diffusion XL набор данных, используемый для обучения, является закрытым, несмотря на то, что веса модели имеют открытый исходный код.

Создание нового большого набора «текст-изображение», в котором согласие, безопасность и лицензирование были бы в центре внимания, стало бы отличным ресурсом для исследований и позволило создавать новые коммерческие приложения, которые имели бы правовую определенность. Наборы данных CommonCanvas имеют хороший потенциал в этом направлении.

⁷ Скраппинг (Scrapping) — это технология получения веб-данных путем извлечения их со страниц веб-ресурсов. Скраппинг может быть сделан вручную пользователем компьютера, однако термин обычно относится к автоматизированным процессам, реализованным с помощью кода, который выполняет GET-запросы на целевой сайт. — *Илер*.

Время проекта: создание интерактивной демо модели с помощью Gradio

До этого момента мы были сосредоточены на запуске трансформеров и моделей диффузии с применением библиотек с открытым исходным кодом. Это дает нам большую гибкость и контроль над моделями, но также требует много усилий для их настройки и запуска. Реальность такова, что большинство людей не умеют писать код, но могут быть заинтересованы в изучении моделей и их возможностей.

В этом проекте мы создадим простую демо модель, которая позволит пользователям генерировать изображения из текстовых промптов с помощью Stable Diffusion. Такие модели позволяют легко демонстрировать себя широкой аудитории, а также делают работу и исследования более доступными.

Существует много способов создать демо. Вы можете использовать HTML, JavaScript и CSS, однако это подразумевает наличие некоторого опыта веб-разработки и не является простым процессом. Библиотеки с открытым исходным кодом, такие как streamlit и gradio, упрощают создание интерактивных демо моделей с использованием Python. В этой главе мы будем использовать gradio, поскольку она является очень простой и лаконичной.

Gradio можно запустить где угодно — в Python IDE, Jupyter Notebook, Google Colab или облачной среде, наподобие Hugging Face Spaces. Самый простой способ создать проект — использовать класс Interface, который имеет три ключевых аспекта:

◆ inputs

Содержит ожидаемые типы входных данных, например текстовые промпты или изображения.

◆ outputs

Содержит ожидаемые типы выходных данных, например сгенерированные изображения.

◆ fn

Это функция, которая вызывается, когда пользователь взаимодействует с демо-моделью. Здесь и происходит «магия». В ней вы можете запустить любой код, включая трансформеры или модели диффузии.

Рассмотрим пример (рис. 5.20).

```
import gradio as gr

def greet(name):
    return "Hello " + name

demo = gr.Interface(fn=greet, inputs="text", outputs="text")

demo.launch()
```

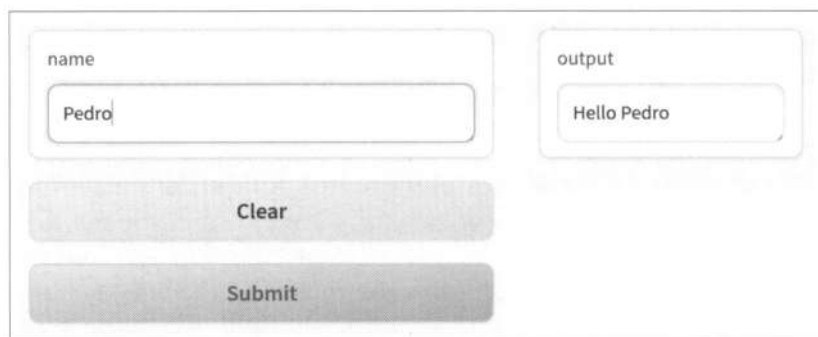
The image shows a web-based user interface for a demo application. It consists of a light gray rectangular container. Inside, on the left, is a form with a label 'name' above a text input field containing the word 'Pedro'. To the right of this is another form with a label 'output' above a text field containing 'Hello Pedro'. Below the 'name' input field is a wide, light gray button labeled 'Clear'. At the bottom of the container is a wide, dark gray button labeled 'Submit'.

Рис. 5.20. Работа демо модели

Теперь ваша очередь. Создайте простую демо модель, которая позволит пользователям генерировать изображения из текстовых промптов с помощью Stable Diffusion. Вы можете использовать код из предыдущего раздела в качестве отправной точки. Как только у вас получится запустить модель, добавьте больше функций, чтобы сделать ее интерактивной и интересной. Например, вы можете реализовать следующее:

- ◆ Добавить ползунок для управления масштабом руководства.
- ◆ Добавить дополнительное текстовое поле для ввода негативного промпта.
- ◆ Добавить заголовок и описание, чтобы пользователи понимали, что делает ваша демо модель.

Если вам нужна помощь, можете ознакомиться с официальной документацией и кратким руководством по gradio.

Заключение

В этой главе мы показали, как обусловливание дает новые способы управлять генерацией изображений в моделях диффузии. Вы увидели, как текстовый энкодер может обусловливать модель диффузии, обрабатывающую текстовые промпты, обеспечивая мощные возможности преобразования текста в изображение. Также мы продемонстрировали, как все это можно объединить в модели Stable Diffusion, углубившись в цикл выборки и показав, как работают вместе различные компоненты.

В главе 7 вы узнаете, как тонко настроить Stable Diffusion, чтобы добавить новые знания или возможности в модель. Например, вы увидите, как с помощью фотографий вашего питомца Stable Diffusion может изучить саму концепцию «ваш питомец» и генерировать изображения с ним в новых сценариях, например «ваш питомец на Луне».

В главе 8 мы покажем некоторые возможности, которые вы можете добавить к моделям диффузии, чтобы вывести их за рамки простой генерации изображений. Например, мы рассмотрим метод *закрашивания*, который позволяет маскировать часть

изображения, а затем заполнять (восстанавливать) ее. В *главе 8* также будут изучены методы редактирования изображений с помощью промптов.

Вопросы

1. Чем отличается процесс обучения обусловленной модели диффузии от обучения безусловной модели, особенно с точки зрения входных данных и используемой функции потерь?
2. Как внедрение временного шага влияет на качество и эволюцию генерации изображений в процессе диффузии?
3. Объясните разницу между Latent Diffusion и нормальной диффузией. Каковы компромиссы использования Latent Diffusion?
4. Как текстовый промпт включается в модель?
5. В чем разница между руководством на основе модели и руководством без классификатора? В чем преимущество руководства без классификатора?
6. Какой эффект дает использование негативного промпта? Поэкспериментируйте с ним, используя конвейер `pipe(..., negative_prompt="")`. Как можно управлять генерацией изображений с помощью Stable Diffusion?
7. Допустим, вы хотите удалить все белые шляпы из любого сгенерированного изображения. Как для этого можно использовать негативные промпты? Сначала попробуйте реализовать это с помощью конвейера высокого уровня. Затем попробуйте адаптировать сквозной пример вывода (подсказка: требуется изменить лишь случайную часть обусловливания без классификатора).
8. Что произойдет в SDXL, если вы будете использовать (256, 256) вместо (1024, 1024) в качестве условия для исходного размера изображения? Что произойдет, если вы будете использовать координаты обрезки, отличные от (0, 0)? Можете ли вы объяснить почему?

Вы можете найти ответы на эти вопросы и решения для задач ниже в репозитории книги на GitHub.

Задачи

Обусловливание. Допустим, мы хотим склонить модель генерировать изображения в определенном цвете, например синем. Как это сделать? Первый шаг — определить функции обусловливания, которые мы хотели бы минимизировать (в данном случае потеря цвета).

```
def color_loss(images, target_color=(0.1, 0.5, 0.9)):
    """При заданном целевом цвете (R, G, B) нам нужно вернуть потерю,
    насколько далеко в среднем пиксели изображения находятся от этого цвета"""
    #Сопоставляем целевой цвет to (-1, 1)
    target = torch.tensor(target_color).to(images.device) * 2 - 1
```

```
#Получаем форму, подходящую для работы с изображениями (b, c, h, w)
target = target[None, :, None, None]

#Средняя абсолютная разница между пикселями изображения и целевым цветом
error = torch.abs(images - target).mean()
return error
```

Учитывая данную функцию потерь, напишите цикл выборки (обучение не требуется), который изменяет x в направлении функции потерь (для упрощения решения мы рекомендуем использовать безусловный класс `DDPMPipeline` из главы 4).

ЧАСТЬ II

Перенос обучения для генеративных моделей

Тонкая настройка языковых моделей

В главе 2 мы изучили, как работают языковые модели, а также как они используются для генерации текста и классификации последовательностей. Мы убедились, что они могут быть полезны во многих задачах без дальнейшего обучения благодаря правильным промптам и возможностям, которые дает генерация с нулевым выстрелом. Мы также изучили некоторые предварительно обученные модели. В этой главе мы обсудим, как можно улучшить производительность языковых моделей в определенных задачах с помощью тонкой настройки на собственных данных.

Несмотря на то что предварительно обученные модели демонстрируют замечательные возможности, их обучение, которое носит универсальный характер, может не подойти для определенных задач или областей. В таких случаях тонкая настройка часто используется для адаптации модели к нюансам, характерным для определенного набора данных или задачи. Например, в области медицинских исследований модели, предварительно обученные на общих данных, не будут работать хорошо по умолчанию. Их необходимо настраивать на определенных узко направленных наборах, чтобы они могли эффективно решать конкретные медицинские вопросы, например генерировать релевантный текст или осуществлять помощь в извлечении информации из документов. Другой пример — разговорные (диалоговые) модели. Несмотря на то что предварительно обученные большие модели могут генерировать связный текст, они обычно не очень хорошо подходят для общения с пользователем или следования инструкциям. Но мы можем обучить их на наборе, содержащем повседневные разговоры и неформальные языковые структуры, адаптируя модель для вывода увлекательного диалогового текста (например, как тот, который генерирует ChatGPT).

Цель этой главы — дать вам прочную основу, чтобы вы могли выполнять тонкую настройку больших языковых моделей (LLM). Мы рассмотрим следующие направления:

- ♦ Классификация заголовков текста с использованием тонко настроенной модели-энкодера.
- ♦ Понимание роли моделей-энкодеров в современной эпохе больших языковых моделей.
- ♦ Создание текста в определенном стиле с использованием модели-декодера.
- ♦ Решение множества задач с помощью одной модели благодаря тонкой настройке через инструкции.

- ♦ Методы тонкой настройки, которые имеют эффективные параметры и позволяют обучать модели с менее производительными GPU.
- ♦ Методы, которые позволяют выполнять вывод с меньшими вычислительными затратами.

Классификация текста

Прежде чем погрузиться в генеративные модели, неплохо было бы понять общий смысл тонкой настройки предварительно обученной модели. Начнем с классификации последовательностей, где модель назначает класс заданным входным данным. Классификация последовательностей — одна из классических областей ML. С ее помощью можно решать такие задачи, как обнаружение спама, распознавание настроений, классификация намерений, обнаружение поддельного контента и многие другие. Несмотря на то что это является простой задачей, которую можно решить с помощью универсальной языковой модели, она также служит хорошей отправной точкой, чтобы вы могли понять сам процесс тонкой настройки и его этапы.

Мы выполним тонкую настройку модели, которая классифицирует темы коротких аннотаций в новостных статьях. Как вы вскоре убедитесь, тонкая настройка требует гораздо меньше вычислений и количества данных, чем обучение с нуля. Типичный процесс показан на рис. 6.1 и имеет следующие шаги:

1. Определение набора данных для задачи.
2. Определение типа модели (энкодер, декодер или энкодер-декодер).
3. Выбор хорошей базовой модели, которая соответствует требованиям.
4. Предварительная обработка данных.

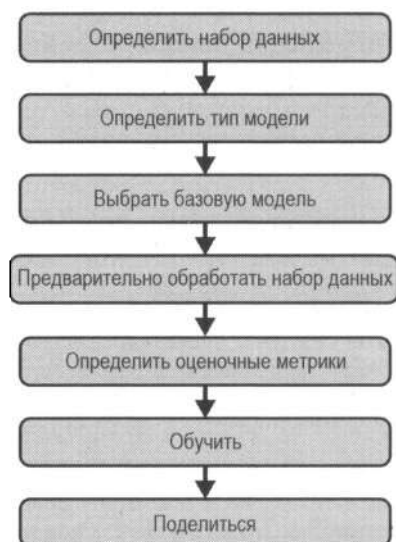


Рис. 6.1. Обычные этапы процесса тонкой настройки

5. Определение метрик оценки.
6. Обучение модели.
7. Общий доступ к модели (сделать ее открытой).

Определение набора данных

На этом этапе происходит адаптация предварительно обученной языковой модели общего назначения для работы в качестве текстового классификатора, а именно мы показываем ей категории, которые она должна выявлять. Поэтому нам необходимо использовать маркированный набор данных для классификации последовательностей. В зависимости от задачи и варианта использования вы можете выбрать открытый (публичный) или закрытый (коммерческий) вариант. Хорошие наборы можно найти на таких ресурсах, как Hugging Face Datasets, Kaggle, Zenodo и Google Dataset Search. Поскольку их насчитывается множество сотен тысяч, нам нужна помощь в поиске подходящего для нашего варианта использования. Для этого мы можем использовать фильтрацию на Hugging Face.

Среди наиболее популярных вариантов есть AG News — это известный некоммерческий набор данных, используемый для сравнительного анализа моделей касательно классификации текста.



Возможно, когда-нибудь вам захочется поделиться своим набором со всеми. Для этого вы можете загрузить его как репозиторий. Библиотека datasets имеет встроенную поддержку распространенных типов данных (аудио, изображения, текст, CSV, JSON, pandas DataFrames и др.).

Первым делом нам нужно исследовать данные. Как показано в коде ниже, набор содержит два столбца (один с текстом и один с меткой). Он предоставляет 120 000 обучающих выборок. Этого более чем достаточно для тонкой настройки, которая требует гораздо меньше данных по сравнению с предварительной подготовкой. Нескольких тысяч примеров должно быть достаточно, чтобы получить хорошую базовую модель.

```
from datasets import load_dataset

raw_datasets = load_dataset("fancyzhx/ag_news")
raw_datasets

DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 120000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 7600
  })
})
```

Рассмотрим, как выглядит конкретный пример.

```
raw_train_dataset = raw_datasets["train"]
raw_train_dataset[0]

{'label': 2,
 'text': 'Wall St. Bears Claw Back Into the Black (Reuters) Reuters '
        '- Short-sellers, Wall Street's dwindling\\band of "
        'ultra-cynics, are seeing green again.'}
```

Первая выборка содержит текст и метку, которая имеет значение 2. Чтобы выяснить, к какому классу относится это значение, мы можем проверить набор данных `features` и его поле `label`.

```
print(raw_train_dataset.features)

{'label': ClassLabel(names=['World',
                           'Sports',
                           'Business',
                           'Sci/Tech'],
                    id=None),
 'text': Value(dtype='string', id=None)}
```

Таким образом мы имеем: метка «0» означает мировые новости, «1» — новости спорта, «2» — новости бизнеса, а «3» — наука и технологии. Разобравшись с этим, определим, какую модель использовать.

Определение типа модели

Мы повторим действия в *главе 2*, а именно выберем один из трех типов трансформеров, в зависимости от задачи, которую мы пытаемся решить. Напомним:

◆ Энкодеры

Они способны получать богатые семантические представления из последовательностей, фиксируя значение входных данных, которые могут использоваться для различных задач, и полагаясь на семантическую информацию входных данных (например, идентификация сущностей в тексте или классификация последовательностей). Поверх эмбеддингов может быть добавлена небольшая сеть для обучения определенной нисходящей задаче.

◆ Декодеры

Они предназначены для генерации новых последовательностей вроде текста. Декодеры принимают входные данные (часто эмбеддинги или контекст) и производят связные выходные последовательности, что делает их идеальными для задач генерации текста.

◆ Энкодер-декодеры

Эти модели хорошо подходят для задач, требующих преобразования одной входной последовательности в другую, например в машинный перевод или

резюмирование. Энкодер обрабатывает входные данные, а декодер генерирует соответствующие выводы.

Относительно нашей задачи (классификация кратких аннотаций новостных статей) у нас есть три возможных подхода:

1. Нулевой выстрел или несколько выстрелов

Мы можем использовать высококачественную предварительно обученную модель, объяснить задачу (например, «классифицировать по этим четырем категориям») и позволить модели сделать все остальное. Такой подход не требует какой-либо тонкой настройки и очень распространен в настоящее время среди мощных предварительно обученных моделей (одна модель может решать множество задач, формулируя их как задачи генерации текста).

2. Модель генерации текста

Здесь мы выполняем тонкую настройку, чтобы генерировать метки (например, «бизнес») с учетом новостной статьи на входе. Здесь мы могли бы использовать модель либо с декодером, либо с энкодер-декодером.

3. Модель энкодера с классификатором

Здесь мы можем выполнять тонкую настройку с помощью добавления простой классификационной сети (называемой также *classification head*) к эмбедингам. Этот подход способен дать нам специализированную и эффективную модель, адаптированную к конкретному варианту использования, что делает ее благоприятным выбором для нашей задачи.

Исходя из этого, мы выберем третий подход.

Выбор хорошей базовой модели

Требования к нашей модели:

- ◆ Она имеет архитектуру на основе энкодера.
- ◆ Она достаточно мала, чтобы можно было выполнить тонкую настройку за несколько минут на пользовательском GPU.
- ◆ Она имеет надежные результаты на этапе предварительной подготовки.
- ◆ Она может обрабатывать короткие последовательности текста.

Несмотря на то что модель BERT уже устарела, она отлично подходит в качестве базовой архитектуры энкодера для тонкой настройки. Учитывая, что нам нужно обучить модель быстро и с использованием небольшой вычислительной мощности, мы можем выбрать вариацию DistilBERT. Она на 40% меньше и на 60% быстрее BERT, а также имеет почти те же возможности. Располагая базовой моделью, теперь мы можем настроить ее для решения множества задач, например для ответов на вопросы или классификации текста.

Помимо BERT и DistilBERT, в качестве базовых моделей можно использовать множество других. Мы не будем углубляться в каждую из них, но важно знать, что они существуют. Вот некоторые примеры: RoBERTa, ALBERT, Electra, DeBERTa,

Longformer, LuKE, MobileBERT и Reformer. Каждая из них имеет собственную процедуру обучения и построена на основе оригинальной модели BERT. Выбор зависит от ваших конкретных требований, но в нашем случае DistilBERT является хорошей отправной точкой, учитывая вычислительные требования. На момент написания книги наиболее свежей моделью является DeBERTa.

Предварительная обработка набора данных

Каждая языковая модель имеет свой собственный токенизатор. Чтобы выполнить тонкую настройку DistilBERT, мы должны быть уверены, что токенизация набора и предварительная подготовка модели осуществлялись с помощью одного токенизатора. Например, мы можем использовать класс `AutoTokenizer`, чтобы загрузить соответствующий токенизатор, а затем определить функцию, которая будет токенизировать пакет выборок. Ожидается, что все входные данные в пакете для библиотеки `transformers` будут иметь одинаковую длину. Поэтому используя команду `padding=True`, мы добавим нули к элементам набора, чтобы все они имели тот же размер, что и самый длинный из них.

Важно отметить, что модели-трансформеры имеют максимальный размер контекста — это наибольшее количество токенов, которые языковая модель может использовать для прогнозов. Для DistilBERT этот предел составляет 512 токенов, поэтому не пытайтесь применять ее, например, для целых книг. К счастью, большинство наших данных представляют собой небольшие аннотации, но некоторые все равно могут превышать этот предел. Чтобы обойти это ограничение, мы будем использовать команду `truncation=True`. Она обрежет все выборки, чтобы они соответствовали длине контекста. Однако такой подход имеет нюанс: обрезание текста означает, что некоторая потенциально полезная информация может быть потеряна.

Обработка длинных контекстов с помощью трансформеров является активной областью исследований. Для сценариев, когда применяются модели на основе энкодера, а контекст имеет большой размер, вы можете попробовать несколько стратегий:

- ◆ Использовать специализированную модель-трансформер с длинным контекстом, например Longformer.
- ◆ Разделить текст на более мелкие сегменты и отдельно их обработать.
- ◆ Использовать подход *скользящего окна* (**Sliding window**) для обработки текста по частям.
- ◆ Резюмировать текст в качестве шага предварительной обработки, а затем уже передать его в модель.

Какую стратегию выбрать, зависит от задачи и модели. Например, если вы хотите понять общее «настроение» книги, лучше использовать фрагментацию и анализировать части книги по отдельности. Если вы хотите классифицировать тему длинной статьи, вы можете сначала резюмировать ее, а затем классифицировать получившуюся выжимку из текста.

Далее токенизируем два пакета данных, чтобы проверить вывод.

Функция `tokenize_function()` получает пакет данных, токенизирует их с помощью токенизатора `DistilBERT` и обеспечивает равномерную длину, заполняя и усекая данные по мере необходимости. Как вы можете сами проверить: например, первый элемент короче второго, поэтому у него есть несколько дополнительных токенов с идентификатором 0 в конце. Все нули соответствуют токену `[PAD]`, который игнорируется во время вывода. Кроме того, стоит отметить, что маска внимания для этого пакета также имеет 0 в конце. Это гарантирует, что модель будет обращать внимание только на фактические токены.

Теперь мы можем использовать метод `map()` для токенизации всего набора данных. Эта функция применяется к каждому элементу набора параллельно.

```
tokenized_datasets = raw_datasets.map(tokenize_function, batched=True)
tokenized_datasets
```

```
DatasetDict({
  train: Dataset({
    features: ['text', 'label', 'input_ids', 'attention_mask'],
    num_rows: 120000
  })
  test: Dataset({
    features: ['text', 'label', 'input_ids', 'attention_mask'],
    num_rows: 7600
  })
})
```

Определение оценочных метрик

В дополнение к мониторингу потерь во время обучения хорошей практикой является определение некоторых нисходящих метрик, чтобы лучше оценивать и мониторить производительность модели. Воспользуемся библиотекой `evaluate`. Это удобный инструмент со стандартизированным интерфейсом для различных метрик, выбор которых зависит от задачи. Для классификации последовательностей могут подойти следующие варианты:

◆ Точность (*Accuracy*)

Метрика представляет собой долю правильных прогнозов из всего их количества. Она дает общее представление о совокупной производительности модели. Эта метрика хорошо подходит для сбалансированных наборов, и ее легко интерпретировать. Однако она может вводить в заблуждение при использовании для несбалансированных данных.

◆ Прецизионность (*Precision*)

Это отношение положительных прогнозов, которые помечены как верные, ко всем положительным прогнозам. Иначе говоря, какая доля прогнозов, выделенных как положительные, действительно является положительными. Метрика помогает понять точность истинно положительных прогнозов модели. Прецизионность следует использовать, когда стоимость ложных положительных результатов высока, например при обнаружении спама.

◆ Полнота (*Recall*)

Метрика указывает долю фактических положительных прогнозов, которые были правильно предсказаны моделью. Она отражает способность модели захватывать все положительные случаи, и ее показатель будет ниже, если ложных отрицательных случаев много. Полноту следует использовать, когда стоимость ложных отрицательных результатов высока, например при медицинской диагностике.

◆ Оценка F1 (F1 score)

Это гармоническое среднее¹ между прецизионностью и полнотой, представляющее сбалансированную меру, которая учитывает как ложноположительные, так и ложноотрицательные результаты и штрафует сильные расхождения между прецизионностью и полнотой. F1 часто используется для несбалансированных наборов данных и является хорошей метрикой по умолчанию для задач классификации.

Библиотека `evaluate` предоставляет атрибут `description` и метод `compute()` для получения метрик с учетом меток и прогнозов модели.

```
import evaluate

accuracy = evaluate.load("accuracy")
print(accuracy.description)
print(accuracy.compute(references=[0, 1, 0, 1], predictions=[1, 0, 0, 1]))

('Accuracy is the proportion of correct predictions among the total '
 'number of cases processed. It can be computed with:'
 'Accuracy = (TP + TN) / (TP + TN + FP + FN)'
 'Where:'
 'TP: True positive'
 'TN: True negative'
 'FP: False positive'
 'FN: False negative')
('accuracy': 0.5)
```

Далее определим функцию `compute_metrics()`. Она, учитывая прогноз (содержащий как метку, так и прогнозы), возвращает словарь с точностью и оценкой F1. Оценивая модель во время обучения, мы автоматически используем эту функцию для отслеживания прогресса.

```
f1_score = evaluate.load("f1")

def compute_metrics(pred): ❶
    labels = pred.label_ids
    preds = pred.predictions.argmax(-1) ❷

    #Вычисляем точность и оценку F1
    acc_result = accuracy.compute(references=labels, predictions=preds)
    acc = acc_result["accuracy"] ❸

    f1_result = f1_score.compute(
        references=labels, predictions=preds, average="weighted"
    )
```

¹ Гармоническое среднее — это тип среднего значения, полезный при работе с отношениями, поскольку он придает больший вес более низким значениям.

```
f1 = f1_result["f1"] ❹

return {"accuracy": acc, "f1": f1} ❺
```

Рассмотрим код подробнее:

- ❶ Метод `compute_metrics()` ожидает экземпляр `EvalPrediction`. `EvalPrediction` — это служебный класс, используемый классом `Trainer`, который содержит метки и прогнозы модели для выборки.
- ❷ Мы используем аргумент `argmax`, чтобы получить предсказанный класс с наибольшей вероятностью.
- ❸ Мы используем загруженную метрику `accuracy`, чтобы вычислить оценку точности между метками и прогнозами. Напомним, что `accuracy` выводит словарь с помощью ключа `accuracy`.
- ❹ Повторяем те же действия с оценкой F1. Поскольку у нас есть несколько классов, мы используем аргумент `weighted=True`. Это означает, что мы вычисляем значения F1 для каждого класса, а затем усредняем их, взвешивая по количеству истинных экземпляров для каждого класса.
- ❺ В завершение мы возвращаем обе метрики в виде словаря.

Обучение модели

DistilBERT — это модель на основе энкодера. Если мы будем работать с ней «как есть», то получим только эмбединги (как мы это делали в *главе 2*), поэтому ее нельзя использовать напрямую. Чтобы иметь возможность классифицировать текстовые последовательности, нам нужно «прикрутить» классификатор (классификационную сеть) поверх базового трансформера.

При тонкой настройке мы не будем использовать фиксированные эмбединги. Все параметры модели, исходные веса и классификатор можно обучить. Для этого требуется, чтобы классификатор был дифференцируемым. Он будет принимать эмбединги в качестве входных и выходных вероятностей классов (рис. 6.2). Но зачем нам нужно обучать веса? Обучая все параметры, мы помогаем сделать эмбединги более полезными для нашей конкретной задачи.

Несмотря на применение простой сети прямого распространения, мы можем использовать и более сложные сети в качестве классификатора или даже классические модели, такие как Logistic Regression и Random Forests (в этом случае мы используем модель как экстрактор признаков и замораживаем веса). Использование простого слоя работает хорошо. Это является вычислительно эффективным решением и наиболее распространенным подходом.



Если вы занимались переносом обучения в компьютерном зрении, то, вероятно, знакомы с концепцией заморозки весов базовой модели. Она редко применяется в NLP, поскольку цель здесь — сделать внутренние языковые представления более полезными для последующих задач. В компьютерном зрении часто приходится замораживать некоторые слои, поскольку признаки, изученные базовой моделью, носят более общий характер и являются полезными для многих сценариев исполь-

зования. Например, некоторые слои захватывают признаки (например, края или текстуры), которые широко применимы в задачах компьютерного зрения. Замораживать или размораживать слои нужно в зависимости от размера набора данных, объема вычислений и сходства между предварительной подготовкой и тонкой настройкой. Далее в этой главе вы узнаете о методе, который называется *адаптер*. Он позволяет работать с замороженными LLM.

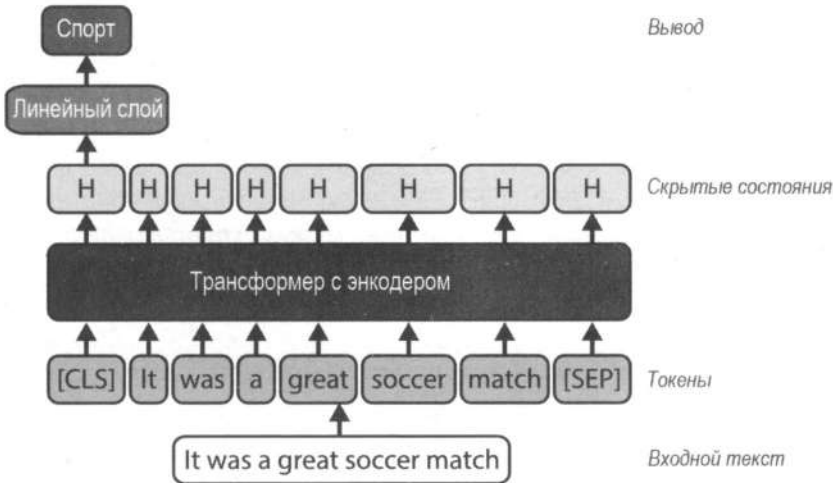


Рис. 6.2. Схема модели BERT с классификатором: на практике эмбединги (соответствующие классификации), используются как объединенные и могут применяться для задач классификации

Чтобы обучить модель с классификатором, можно загрузить ее с помощью функции `AutoModelForSequenceClassification`. Она выполняет два действия:

- ◆ Загружает указанную модель (в данном случае `DistilBERT`) без маской языковой классификационной сети. Эта часть является энкодером, который выводит эмбединги для каждого токена. Также она выводит объединенные эмбединги, содержащие информацию обо всей последовательности.
- ◆ Добавляет случайно инициализированную классификационную сеть поверх базовой модели (представленной линейным слоем), которая получает объединенные эмбединги и выводит вероятности классов.

Посмотрим, как это реализуется.

```
import torch
from transformers import AutoModelForSequenceClassification

from genaibook.core import get_device

device = get_device()
num_labels = 4
model = AutoModelForSequenceClassification.from_pretrained(
    checkpoint, num_labels=num_labels
).to(device)
```

```
('Некоторые веса DistilBertForSequenceClassification не были инициализированы
из контрольной точки модели distilbert-base-uncased
и инициализированы заново: ["classifier.bias",
    "classifier.weight", "pre_classifier.bias",
    "pre_classifier.weight"]
Возможно, вам следует ОБУЧИТЬ эту модель на последующей задаче, чтобы
иметь возможность использовать ее для прогнозирования и вывода')
```

При выполнении вы получите предупреждение о новой инициализации некоторых весов. Это логично, поскольку у вас появилась новая сеть и ее нужно обучить.

Инициализировав модель, ее наконец-то можно обучить. Для этого доступны различные подходы. Если вы знакомы с PyTorch, то можете написать свой цикл обучения. В качестве альтернативы мы будем использовать высокоуровневый класс `Trainer` из библиотеки `transformers`, который значительно упрощает цикл обучения.

Но сперва нам нужно определить класс `TrainingArguments`, как показано во фрагменте кода ниже. С помощью него мы зададим гиперпараметры, используемые для обучения (например, скорость обучения и снижение веса), определим количество выборок в пакете, установим интервалы оценки и определим, хотим ли мы поделиться нашей моделью с экосистемой, поместив ее в Hub. Мы не будем изменять гиперпараметры, поскольку значения по умолчанию работают хорошо. Тем не менее мы рекомендуем вам изучить их и поэкспериментировать с ними. Класс `Trainer` — это надежный и гибкий инструмент.

```
from transformers import TrainingArguments
```

```
batch_size = 32 #Вы можете изменить это значение в зависимости от своего GPU
training_args = TrainingArguments(
    "classifier-chapter4",
    push_to_hub=True, ❶
    num_train_epochs=2, ❷
    eval_strategy="epoch", ❸
    per_device_train_batch_size=batch_size, ❹
    per_device_eval_batch_size=batch_size,
)
```

В этом коде мы делаем следующее:

- ❶ Определяем, нужно ли отправлять модель в Hugging Face Hub каждый раз при ее сохранении. Вы можете изменить частоту сохранения с помощью параметра `save_strategy`, который по умолчанию выполняется каждые несколько сотен шагов.
- ❷ Определяем общее количество эпох для выполнения. Эпоха — это полный проход по обучающим данным.
- ❸ Определяем, когда выполнять оценку модели на проверочном наборе. По умолчанию она выполняется каждые 500 шагов, но если указать другой интервал, например в целую эпоху, то оценка будет выполняться в конце каждого прохода.

- ❶ Определяем размер пакета на одно ядро для обучения. Вы можете уменьшить его, если на вашем GPU мало памяти. Кроме того, вы можете использовать параметр `auto_find_batch_size=True`, чтобы определить максимальный размер пакета, подходящего для вашего GPU².

Теперь у нас есть все необходимое:

- ◆ предварительно обученная и готовая к настройке модель с соответствующим модулем;
- ◆ аргументы обучения;
- ◆ функция для вычисления метрик;
- ◆ набор данных для обучения и оценки;
- ◆ токенизатор.

Набор данных AG News содержит 120 000 элементов. Этого достаточно, чтобы обеспечить хорошие начальные результаты. Для быстрого первого обучения мы будем использовать 10 000 элементов набора, но вы можете экспериментировать с этим числом (чем больше данных, тем результат должен быть лучше). Обратите внимание, мы по-прежнему будем использовать весь тестовый набор.

```
from transformers import Trainer
```

```
#Перемешиваем набор и выбираем 10 000 элементов для обучения
shuffled_dataset = tokenized_datasets["train"].shuffle(seed=42)
small_split = shuffled_dataset.select(range(10000))
```

```
#Инициализируем класс Trainer
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    compute_metrics=compute_metrics,
    train_dataset=small_split,
    eval_dataset=tokenized_datasets["test"],
    tokenizer=tokenizer,
)
```

Когда все готово и класс `Trainer` инициализирован, настало время обучения.

```
trainer.train()
```

В табл. 6.1 представлены данные о потерях, метриках оценки и скорости обучения.

Обучение должно занять совсем немного времени. Итоговая оценочная точность и значение F1 близки к 92%, что вполне приемлемо, особенно учитывая, что мы используем менее 10% доступных обучающих данных. Потери оценки уменьша-

² Удобным инструментом для оценки объема видеопамати, необходимого для выполнения вывода и обучения, является *Model Memory Calculator*.

Таблица 6.1. Метрики обучения и оценки для тонкой настройки DistilBERT на основе набора данных AG News

Метрика	Значение в эпоху 1	Значение в эпоху 2
eval_loss	0.2624	0.2433
eval_accuracy	0.9117	0.9184
eval_f1	0.9118	0.9183
eval_runtime	15.2709	14.5161
eval_samples_per_second	497.678	523.557
eval_steps_per_second	15.585	16.396
train_runtime	-	213.9327
train_samples_per_second	-	93.487
train_steps_per_second	-	2.926
train_loss	-	0.2714

ются между эпохами, что и было нашей целью. Если вы хотите поделиться итоговой моделью с другими, необходимо в конце добавить команду `push_to_hub`.

```
trainer.push_to_hub()
```

Вам может показаться, что использование класса `Trainer` является чем-то вроде «черного ящика». Но, по сути, он просто создает обычные циклы обучения PyTorch, подобные тем, что мы делали для обучения простых моделей диффузии в главе 3. Написание подобного цикла с нуля выглядит примерно так.

```
from transformers import AdamW, get_scheduler

optimizer = AdamW(model.parameters(), lr=5e-5) ❶
lr_scheduler = get_scheduler("linear", ...) ❷

for epoch in range(num_epochs): ❸
    for batch in train_dataloader: ❹
        batch = {k: v.to(device) for k, v in batch.items()} ❺
        outputs = model(**batch)
        loss = outputs.loss ❻
        loss.backward()

        optimizer.step() ❼
        lr_scheduler.step()
        optimizer.zero_grad()
```

В этом коде происходит следующее:

- ❶ Оптимизатор сохраняет текущее состояние модели и обновляет параметры на основе градиентов.

- ❷ График определяет, как в процессе изменяется скорость обучения.
- ❸ Выполняется итерация по всем данным в течение нескольких эпох.
- ❹ Выполняется итерация по всем пакетам обучающих данных.
- ❺ Пакет перемещается на устройство и происходит запуск модели.
- ❻ Выполняется вычисление потерь и их обратное распространение.
- ❼ Происходит обновление параметров модели, настройка скорости обучения и сброс градиентов до нуля.

Класс `Trainer` отвечает за все: оценка и прогнозирование, отправка моделей в Hub, возможность обучения на нескольких GPU, сохранение мгновенных контрольных точек, ведение журнала и многое другое.

Если вы отправили модель в Hub, другие пользователи смогут получить к ней доступ с помощью инструментов `AutoModel` или `pipeline()`. Рассмотрим пример.

#Используем функцию `pipeline()` в качестве помощника высокого уровня
`from transformers import pipeline`

```
pipe = pipeline(
    "text-classification",
    model="genaiabook/classifier-chapter4",
    device=device,
)
pipe(
    """The soccer match between Spain and
    Portugal ended in a terrible result for Portugal."""
)
```

```
[{'label': 'Sports', 'score': 0.8631355166435242}]
```

Прогноз получился верный. При первой попытке вы можете получить метку `LABEL_1` вместо `Sports`. Это связано с тем, что модель не знает внутренних названий меток. Чтобы обновить их, нужно изменить файл `config.json`, добавив сопоставления `id2label` и `label2id`. Это сделает прогнозы более удобными для восприятия и интерпретации.

Рассмотрим метрики подробнее. Для получения прогнозов можно использовать методы `Trainer.predict` или `pipe.predict`. Первый возвращает объект `PredictionOutput`, содержащий прогнозы, идентификаторы меток и метрики, а второй возвращает список словарей с прогнозами и соответствующими метками. Рассмотрим первые три примера текстов с соответствующими им прогнозами и метками. Прогон нескольких выборок через сеть всегда важен для обеспечения корректной работы.

#Получаем прогноз для всех выборок
`model_preds = pipe.predict(tokenized_datasets["test"]["text"])`

#Получаем метки набора данных
`references = tokenized_datasets["test"]["label"]`

```
#Получаем список имен меток
label_names = raw_train_dataset.features["label"].names

#Выводим результаты для первых 3 выборок
samples = 3
texts = tokenized_datasets["test"]["text"][:samples]
for pred, ref, text in zip(model_preds[:samples], references[:samples], texts):
    print(f"Predicted {pred['label']}; Actual {label_names[ref]}")
    print(text)

('Predicted Business; Actual Business; Fears for T N pension after '
'talks Unions representing workers at Turner Newall say they are '
'"disappointed' after talks with stricken parent firm Federal "
'Mogul.'
'\n'
'Predicted Sci/Tech; Actual Sci/Tech; The Race is On: Second '
'Private Team Sets Launch Date for Human Spaceflight (SPACE.com) '
'SPACE.com - TORONTO, Canada -- A second\team of rocketeers '
'competing for the #36;10 million Ansari X Prize, a contest '
'for\privately funded suborbital space flight, has officially '
'announced the first\launch date for its manned rocket.'
'\n'
'Predicted Sci/Tech; Actual Sci/Tech; Ky. Company Wins Grant to '
'Study Peptides (AP) AP - A company founded by a chemistry '
'researcher at the University of Louisville won a grant to develop '
'a method of producing better peptides, which are short chains of '
'amino acids, the building blocks of proteins.')
```

Прогноз соответствует эталону, и метка имеет смысл. Теперь перейдем к метрикам.

В задачах классификации с помощью машинного обучения *матрица ошибок* (также называется *матрицей путаницы*) служит таблицей, которая суммирует эффективность модели и отображает количество истинно положительных, истинно отрицательных, ложноположительных и ложноотрицательных прогнозов. Для многоклассовой классификации матрица становится квадратной, а размер ее равен количеству классов, где каждая ячейка представляет число экземпляров для комбинации меток и предсказанных классов. Строки указывают на фактические (истинные) классы, а столбцы — на предсказанные. Матрицу можно нормализовать так, чтобы сумма значений каждой строки равнялась 1. Это упростит интерпретацию эффективности для различных классов. Анализ данной матрицы дает представление о сильных и слабых сторонах модели в различении конкретных классов.

Воспользуемся функцией `evaluate` для загрузки и вычисления матрицы ошибок, а также функцией `ConfusionMatrixDisplay` из библиотеки `sklearn` для ее визуализации. Матрица ошибок поможет понять, где модель допускает путаницу и какие классы сложнее предсказать. Например, взглянув на следующую матрицу, можно быть уверенным, что статьи о бизнесе часто путаются со статьями о науке и технике (рис. 6.3).

```

import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

#Преобразуем предсказанные метки в id
label_to_id = {name: i for i, name in enumerate(label_names)}
pred_labels = [label_to_id(pred["label"]) for pred in model_preds]

#Вычисляем матрицу ошибок
confusion_matrix = evaluate.load("confusion_matrix")
cm = confusion_matrix.compute(
    references=references, predictions=pred_labels, normalize="true"
)["confusion_matrix"]

#Строим график матрицы ошибок
fig, ax = plt.subplots(figsize=(6, 6))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=label_names)
disp.plot(cmap="Blues", values_format=".2f", ax=ax, colorbar=False)
plt.title("Normalized confusion matrix")
plt.show()

```

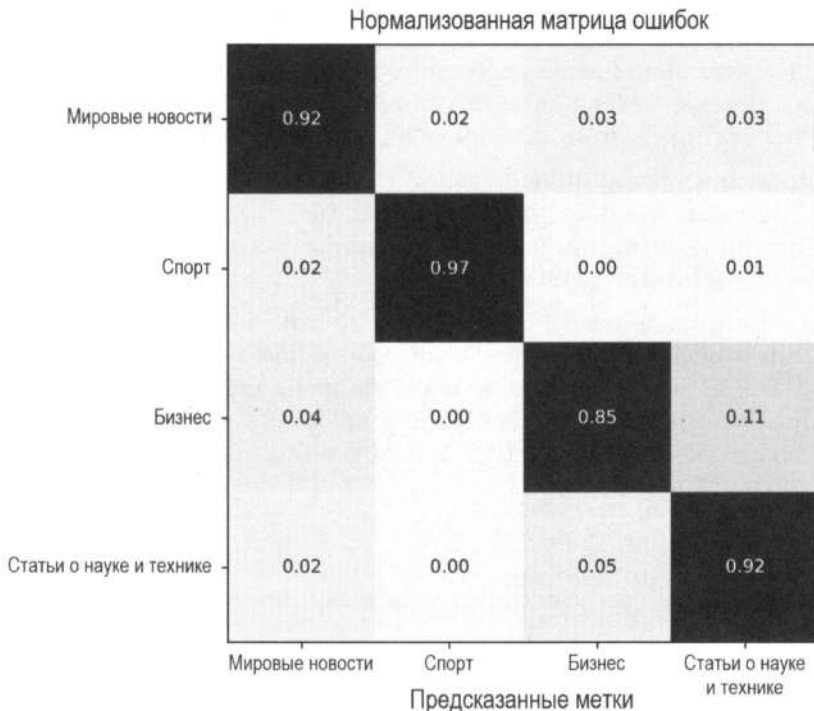


Рис. 6.3. Статьи о бизнесе часто путаются со статьями о науке и технике

Все еще актуально?

Обучение модели-энкодера для классификации текста может показаться странной затеей в эпоху ChatGPT. Проще попросить уже готовую модель классифицировать текст по категориям. Несмотря на то что сложно превзойти удобство быстрых универсальных моделей, в некоторых случаях небольшие, индивидуальные классификаторы оказываются более полезными, особенно в приложениях, где скорость и эффективность играют ключевую роль.

Примером такого использования является подготовка обучающих данных для крупнейших современных LLM. Она предполагает обработку огромных объемов текста. В статье «*The Llama 3 Herd of Models*» авторы применяют так называемую *фильтрацию качества на основе модели*³, при которой классификатор качества обучается и затем используется для оценки документов с целью дальнейшей фильтрации. Авторы статьи также утверждают: «Мы используем *DistilRoberta*, чтобы генерировать оценки качества для каждого документа из соображений эффективности». Обучающими данными для этой модели являются прогнозы Llama 2. Было бы чрезвычайно затратно использовать Llama 2 непосредственно на данных объемом в несколько триллионов токенов, которые требуется отфильтровать. Аналогичный подход использовался командой Phi-3 и авторами FineWeb для отбора максимально образовательного контента для своей платформы FineWeb Edu. Авторы написали отличную статью в блоге о своих рецептах и важности качественной фильтрации.

Еще одно полезное применение небольших и быстрых моделей на основе энкодеров — это получение эмбедингов для поисковых систем, которые затем используются для поиска документов, максимально отвечающих запросу.

Еще один пример использования — *guardrailing*⁴, когда небольшая модель используется для проверки входных данных на предмет вредоносности или выходных данных на предмет безопасности. Во всех этих случаях скорость и эффективность небольшой модели играют ключевую роль.

С учетом всего вышесказанного на сегодняшний день наблюдается тенденция отказа от небольших специализированных моделей в пользу более эффективных универсальных. Тем не менее возможности для настройки остаются (отсюда и эта книга), поэтому далее мы обсудим, как можно применить аналогичный процесс для генерации текста (найти или создать набор данных, выбрать модель, определить метрики оценки и обучить модель).

³ *Model-based quality filtering* — это метод оценки и фильтрации контента на базе метрик качества, основанный на использовании различных типов моделей. Подход позволяет выявлять контент низкого качества на основе лингвистических паттернов, которые модель изучает на обучающих данных, а также отбирать релевантные признаки из большого набора, оставляя только те, которые являются наиболее информативными и соответствующими задаче. — Пер.

⁴ *Guardrailing (guardrails)* — это внедрение мер безопасности, направленных на ограничение поведения генеративных систем в рамках безопасных, этических и юридических границ. Цель — предотвратить непредвиденные последствия, например вредоносный контент (оскорбления, насилие), дезинформацию и сфабрикованные данные, предвзятость (дискриминационный язык или несправедливые рекомендации), нарушения конфиденциальности. — Пер.

Генерация текста

Теперь, когда модель-энкодер настроена для классификации, мы можем перейти к генерации данных. Наши метки представлены списком групп («Мировые новости», «Спорт», «Бизнес» и «Наука/Технологии»). С помощью обучения и тонкой настройки мы решаем задачу прогнозирования токенов, где метки являются текстовыми выходными данными.

Например, если нам нужно генерировать код, то нам необходимо собрать значительное количество данных и обучить модель с нуля. Для получения приемлемых результатов потребуется много вычислений, что может занять несколько недель или месяцев.

Вместо того чтобы обучать модель с нуля для генерации открытого текста, мы можем настроить существующую модель. Такой подход позволяет использовать уже имеющиеся знания модели о языке, значительно сокращая потребность в обширных данных и вычислительных мощностях. Например, используя публикации в социальных сетях, можно обучить модель генерировать новые посты в вашем уникальном стиле.

С метками или без меток?

В задаче прогнозирования следующего токена нам не нужно явно маркировать данные, как это было при классификации. Модель учится делать прогнозы на основе входного текста. Это позволяет создавать большие наборы данных прямо из Интернета.

С другой стороны, существует новое семейство методов *Reinforcement Learning with Human Feedback (RLHF)*, которое позволяет управлять выходными данными модели, предоставляя обратную связь. Это особенно полезно для разговорных моделей, где можно указывать предпочтения или корректировать выходные данные. Именно поэтому многие чат-боты имеют кнопки «Нравится»/«Не нравится» или отображают рядом сгенерированный текст, чтобы пользователи могли выбрать лучший вариант. Даже в этом случае оптимизация предпочтений — это лишь один из компонентов процесса обучения, и модель все равно должна обучаться на данных без учителя. Мы подробнее обсудим RLHF в главе 10.

Попробуем настроить модель для генерации новостей в определенном стиле, например о бизнесе. Мы можем использовать тот же набор данных (AG News). Начнем с фильтрации всех выборок с меткой «business» (где она имеет значение 2) и удаления ненужного столбца.

```
filtered_datasets = raw_datasets.filter(lambda example: example["label"] == 2)
filtered_datasets = filtered_datasets.remove_columns("label")
```

Выбор правильной генеративной модели

На этом этапе важно решить, какую базовую модель использовать. Поскольку целью является генерация, мы будем использовать модель на основе декодера. Из тысяч доступных вариантов нужно выбрать тот, который соответствует требова-

ниям. Обсудим некоторые ключевые факторы, которые могут повлиять на наше решение:

◆ *Размер модели*

Локальное развертывание большой модели (например, с 60 миллиардами параметров) на пользовательском компьютере нецелесообразно. Выбор размера зависит от таких факторов, как ожидаемое время вывода, производительность оборудования и требования к развертыванию. Позже в этой главе мы рассмотрим методы, позволяющие запускать модели с большим количеством параметров, используя те же вычислительные ресурсы.

◆ *Обучающие данные*

Производительность тонко настроенной модели коррелирует с тем, насколько точно обучающие данные базовой модели соответствуют вашим данным вывода. Например, тонкая настройка модели для генерации кода более эффективна, если изначально выбрать модель, предварительно обученную именно на коде. Важно учитывать специфику источников данных, особенно если не все авторы раскрывают свои собственные разработки. Вполне логично, что для генерации, например, корейского текста вам потребуется использовать что-то аналогичное (многоязычную модель или модель, обученную именно на корейском языке), но никак не модель, генерирующую английский текст. Не все создатели раскрывают свои источники данных, и это может затруднить их определение.

◆ *Длина контекста*

Разные модели имеют разные ограничения на длину контекста. Например, если она составляет 1024, модель будет учитывать последние 1024 токена для прогнозирования. Чтобы генерировать длинный текст, соответственно, вам нужна модель с большой длиной контекста. Мы рассмотрим способы работы с ними позже в этой книге.

◆ *Лицензия*

Аспект лицензирования имеет решающее значение при выборе базовой модели. Необходимо учитывать, соответствует ли модель вашим требованиям. Лицензии могут быть коммерческими или некоммерческими, а также существует различие между лицензиями с открытым исходным кодом и лицензиями с открытым доступом. Понимание этого необходимо, чтобы обеспечить соблюдение юридических рамок и ограничений по использованию. Некоторые авторы разрешают коммерческое использование, но при этом они могут определять сценарии, в которых модель не должна применяться. В иных случаях лицензия может ограничивать использование выходных данных (например, запрещать использовать вывод одной модели для обучения другой).

Оценка генеративных моделей по-прежнему остается сложной задачей. Для этого существуют *бенчмарки* (**benchmark**), но они оценивают только отдельные их аспекты и служат лишь индикаторами для различных возможностей моделей. В таблице Open LLM Leaderboard от Hugging Face собраны метрики касательно

возможностей тысяч моделей. Она позволяет фильтровать их по размеру и типу. Однако важно отметить, что бенчмарки являются инструментами только для систематического сравнения, и окончательный выбор модели всегда должен основываться на ее эффективности в вашей реальной задаче. Многие модели, представленные в таблице, не ориентированы на диалог, и, следовательно, их не следует использовать для разговорных сценариев. Выбор зависит от вашего варианта использования, и делать его, основываясь лишь на одной метрике, не рекомендуется.

Рассмотрим некоторые из них:

♦ *MMLU-Pro* (знания)

Это бенчмарк, который содержит 12 000 вопросов. Каждый из них включает отрывок текста и 10 вариантов ответа. Вопросы касаются математики, физики, экономики, психологии, бизнеса и других дисциплин.

♦ *GPQA* (комплексные знания)

Это небольшой бенчмарк, состоящий из сложных вопросов с несколькими вариантами ответов по физике, химии и биологии уровня магистратуры. Вопросы разработаны экспертами в соответствующих областях (имеющими или получающими докторскую степень) и сложны для неспециалистов и для квалифицированных специалистов из других областей (даже с доступом к Интернету)⁵.

♦ *MuSR* (многошаговое рассуждение)

Бенчмарк состоит из сложных задач, каждая из которых содержит около тысячи слов. Он представляет сложность для моделей с кратким контекстом. Задачи могут включать в себя создание детективных историй, распределение команд и размещение объектов.

♦ *MATH* (решение задач)

Содержит более 12 000 задач из экзаменов по математике для старшей школы. Он имеет различные уровни сложности, и в таблицу LLM Leaderboard включены только самые сложные (5-й уровень).

♦ *BBH* (смешанный тип)

Этот бенчмарк содержит 23 сложных задания, требующих многошагового рассуждения. Они охватывают алгоритмические и арифметические рассуждения (например, булевы выражения, геометрические фигуры и навигацию), понимание естественного языка (например, распознавание сарказма и упорядочивание прилагательных), знание мира (например, понимание спортивных дисциплин и рекомендации фильмов) и рассуждения (обнаружение ошибок перевода). Бенчмарк коррелирует с предпочтениями человека.

⁵ Специалисты, имеющие или получающие докторскую степень в области, отличной от заданной, отвечали на вопросы, имея неограниченное время и полный доступ к Интернету (за исключением случаев, когда речь шла о магистрах права). Им платили 10 долларов за попытку ответить на каждый вопрос и бонус в размере 30 долларов за правильный ответ. В среднем они тратили на каждый вопрос 37 минут, и даже при этом точность ответа составляла 34%.

Таблица 6.2. Подборка популярных предварительно подготовленных LLM с открытым доступом и их результаты в рейтинге Open LLM Leaderboard

Модель	Создатель	Размер	Обучающие данные	Производительность по Open LLM	Длина контекста	Размер словарного запаса	Лицензия
GPT-2	OpenAI	117 млн 380 млн 812 млн 1.6 млрд	Не изданы До 40 ГБ текста, полученные скрапplingом	6.51 5.81 5.48 4.98	1,024	50 257	MIT
GPT-Neo	EleutherAI	125 млн 1.3 млрд 2.7 млрд	The Pile 300 млрд токенов 380 млрд токенов 420 млрд токенов	4.38 5.33 6.34	2,048	50 257	MIT
Falcon	TII UAE	7 млрд 11 млрд 40 млрд 180 млрд	Частично выпущены Усовершенствованы Сайт построен на базе CommonCrawl 1,5 трлн токенов 1 трлн токенов 3,5 трлн токенов	5.1 13.78 11.33 N/A	8,192	65 024	Apache 2.0 (7 млрд и 40 млрд) Пользовательская (11 млрд и 180 млрд)
Llama 2	Meta	7 млрд 13 млрд 70 млрд	Не изданы 2 тыс. токенов	8.72 10.99 18.25	4,096	32 000	Пользовательская
Llama 3	Meta	8 млрд 70 млрд	Не изданы 15 тыс. токенов	13.41 26.37	8,192	128 256	Пользовательская
Llama 3.1	Meta	8 млрд 70 млрд 405 млрд	Не изданы 15 тыс. токенов	13.78 25.91 N/A	131,072	128 256	Пользовательская
Mistral	Mistral	7 млрд	Не изданы	14.5	8,192	32 000	Apache 2.0
Mixtral	Mistral	8x7 млрд ⁶ 8x22 млрд	Не изданы	19.23 25.49	32,768 65,536	32 000	Apache 2.0
Qwen 2	Alibaba	500 млн 1.5 млрд 7 млрд 72 млрд	Не изданы	7.06 10.32 23.66 35.13	131,072	~150 000	Пользовательская
Phi (1, 1.5, 2)	Microsoft	1.42 млрд 1.42 млрд 2.78 млрд	54 млрд токенов 150 млрд токенов 1,4 трлн токенов	5.52 7.06 15.45	2,048	51 200	MIT

⁶ В модели Mixtral индекс «8x7» означает, что модель представляет собой смесь экспертов (Mixture of Experts, MoE) — особую архитектуру, о которой вы подробнее узнаете в главе 10. Она подразумевает использование нескольких более мелких моделей внутри. Механизм основной модели определяет, какие из них использовать для каждого токена. Сравнение ряда параметров между MoE и обычными компактными моделями — непростая задача, о которой вы узнаете позже.

◆ *IFEval* (следование инструкциям)

Этот бенчмарк помогает оценивать, может ли модель следовать инструкциям, например «упоминуть ключевое слово Y не менее трех раз» или «написать, уложившись в десять слов или меньше».

Из этих шести бенчмарков только *IFEval* ориентирован на разговорные модели (мы обсудим их позже в этой главе). Поскольку наша текущая цель — генерация текста в определенном стиле, сосредоточимся на ней. В табл. 6.2 представлены несколько популярных готовых моделей с открытым доступом.

Эта таблица является не полной, существует множество других открытых LLM, например Google Gemma, Mosaic MPT и Cohere Command R+. К моменту публикации этой книги их, вероятно, будет еще больше. Кроме того, таблица не охватывает модели, генерирующие код. Если вам интересно, мы рекомендуем ознакомиться с таблицей «*Big Code Models Leaderboard*», где представлены такие модели, как CodeLlama (популярная модель от Meta) и BigCode (модель, обученная с помощью лицензированного разрешенного кода).

Также стоит отметить, что таблица состоит преимущественно из моделей, обученных по большей части на англоязычных данных. Однако существуют мощные модели, обученные на китайском языке, например InternLM, ChatGLM и Baichuan. Они вносят заметный вклад в расширение области предобученных языковых моделей. Эта информация служит лишь руководством по выбору для экспериментов, а не исчерпывающим списком вариантов с открытым исходным кодом.

Обучение генеративной модели

Поскольку у вас может не быть в распоряжении мощного GPU, обучение должно основываться на небольшом количестве данных и быть быстрым. Поэтому мы будем использовать самый маленький вариант модели — SmolLM — и тонко настроим его. Однако рекомендуем поэкспериментировать с более крупными моделями и различными наборами данных, если у вас достаточно вычислительной мощности. Далее в этой главе мы рассмотрим методы, которые позволяют использовать более крупные модели для вывода и обучения.

Как и прежде, для начала загрузим модель и токенизатор. Особенность SmolLM заключается в том, что в ней не указывается заполняющий токен, но он требуется при токенизации, поскольку используется для обеспечения одинаковой длины всех выборок. Мы можем установить токен заполнения таким образом, чтобы он совпадал с токеном конца текста.

```
from transformers import AutoModelForCausalLM

model_id = "HuggingFaceTB/SmolLM-135M"
tokenizer = AutoTokenizer.from_pretrained(model_id)
tokenizer.pad_token = (
    tokenizer.eos_token
) #Эта инструкция необходима, т. к. SmolLM не указывает токен заполнения
model = AutoModelForCausalLM.from_pretrained(model_id).to(device)
```

Далее токенизируем набор данных.

```
def tokenize_function(batch):
    return tokenizer(batch["text"], truncation=True)

tokenized_datasets = filtered_datasets.map(
    tokenize_function,
    batched=True,
    remove_columns=["text"], # Нам нужны только input_ids и attention_mask
)

tokenized_datasets

DatasetDict({
  test: Dataset({
    features: ['input_ids', 'attention_mask'],
    num_rows: 1900
  })
  train: Dataset({
    features: ['input_ids', 'attention_mask'],
    num_rows: 30000
  })
})
```

Если вы помните, в примере классификации тем мы дополняли и усекали все выборки, чтобы обеспечить одинаковую длину. Помимо этого, на этапе токенизации можно использовать *сборщиков данных (Data collator)*. Эти утилиты собирают выборки в пакеты. Библиотека `transformers` предоставляет несколько готовых сборщиков для таких задач. Они динамически дополняют пакет до максимальной длины. Помимо этого, они также структурируют входные данные в задачах моделирования языка, которые имеют большую сложность. В них мы сдвигаем входные данные на один элемент и используем его в качестве метки. Например, если входные данные — «I love Hugging Face», метка будет love Hugging Face. Модель стремится предсказывать следующий токен по предыдущим. На практике сборщики создают столбец меток с копией входных данных. Позже модель позаботится о сдвиге ввода и меток.

В коде ниже показано, как создать сборщика данных для моделирования причинно-следственной связи.

```
from transformers import DataCollatorForLanguageModeling
#mlm соответствует моделированию замаскированного языка, и мы устанавливаем
#его в значение False, поскольку мы обучаем не модель
#замаскированного языка, а причинно-следственную модель
data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=False)
```

Проверим, как это работает, на примере трех выборок. Как показано ниже, каждая выборка имеет разную длину (37, 55 и 51).

```

samples = [tokenized_datasets["train"][i] for i in range(3)]

for sample in samples:
    print(f"input_ids shape: {len(sample['input_ids'])}")

input_ids shape: 37
input_ids shape: 55
input_ids shape: 51

```

Благодаря сборщику данных выборки дополняются до максимальной длины в пакете (55), а также добавляется столбец `label`.

```

out = data_collator(samples)
for key in out:
    print(f"{key} shape: {out[key].shape}")

input_ids shape: torch.Size([3, 55])
attention_mask shape: torch.Size([3, 55])
labels shape: torch.Size([3, 55])

```

Далее нам нужно определить аргументы обучения. Изменяя любой из нескольких ключевых параметров `TrainingArguments` (например, один из перечисленных ниже), мы можем контролировать обучение модели:

◆ *Снижение веса*

Это метод регуляризации, который предотвращает переобучение модели, добавляя штрафную составляющую к функции потерь. Он препятствует назначению больших весов алгоритмом обучения. Алгоритмом можно управлять, настраивая его в `TrainingArguments`, что, в свою очередь, будет влиять на возможности модели касательно генерации.

◆ *Темп обучения*

Это ключевой гиперпараметр, который определяет размер шага оптимизации. В `TrainingArguments` можно указать темп обучения, который будет влиять на скорость сходимости и устойчивость процесса обучения. Осторожная настройка этого параметра может существенно повлиять на качество генерации модели.

◆ *Тип графика для темпа обучения*

График темпа обучения определяет, как темп обучения меняется в процессе обучения. Для разных задач и архитектур выигрышными и полезными будут разные типы графиков. `TrainingArguments` предоставляет возможность определять тип графика для темпа обучения. Это позволяет экспериментировать с различными сценариями (постоянная скорость обучения, косинусный отжиг и др.).

Изменим в нашем примере несколько параметров, чтобы продемонстрировать гибкость обучения.

```

training_args = TrainingArguments(
    "business-news-generator",
    push_to_hub=True,

```

```

per_device_train_batch_size=8,
weight_decay=0.1,
lr_scheduler_type="cosine",
learning_rate=5e-4,
num_train_epochs=2,
eval_strategy="steps",
eval_steps=200,
logging_steps=200,
)

```

После всей настройки, как и в задаче классификации, на последнем шаге нам необходимо создать экземпляр `Trainer` со всеми компонентами. Но на этот раз мы будем использовать сборщик данных и 5000 выборок.

```

trainer = Trainer(
    model=model,
    tokenizer=tokenizer,
    args=training_args,
    data_collator=data_collator,
    train_dataset=tokenized_datasets["train"].select(range(5000)),
    eval_dataset=tokenized_datasets["test"],
)
trainer.train()

```

В табл. 6.3 приведены потери при обучении и оценке в процессе тонкой настройки.

Таблица 6.3. Результаты обучения для тонкой настройки SmolLM на наборе данных AG News

Эпоха	Шаг	Потери	grad_norm	learning_rate	eval_loss	eval_runtime	eval_samples_per_second	eval_steps_per_second
0.32	200	3.2009	2.99705	0.0004690	3.31005	18.6024	102.137	12.794
0.64	400	2.8833	2.46037	0.0003839	3.21182	18.8513	100.789	12.625
0.96	600	2.7102	2.35531	0.0002656	3.09971	18.953	100.248	12.557
1.28	800	1.722	2.55815	0.0001435	3.24014	18.7631	101.262	12.684
1.6	1000	1.5371	1.89922	4.774e-05	3.224	18.7509	101.328	12.693
1.92	1200	1.4841	2.78178	1.971e-06	3.22884	18.5468	102.444	12.832

Как и прежде, отправим модель в Hub.

```
trainer.push_to_hub()
```

Теперь мы можем использовать `pipeline()` и указать тип задачи (`text-generation`), чтобы загрузить модель и запустить вывод.

```
from transformers import pipeline
```

```

pipe = pipeline(
    "text-generation",

```

```

    model="genaibook/business-news-generator",
    device=device,
)
print(
    pipe("Q1", do_sample=True, temperature=0.1, max_new_tokens=30)[0][
        "generated_text"
    ]
)
print(
    pipe("Wall", do_sample=True, temperature=0.1, max_new_tokens=30)[0][
        "generated_text"
    ]
)
print(
    pipe("Google", do_sample=True, temperature=0.1, max_new_tokens=30)[0][
        "generated_text"
    ]
)

('Q1: China #39;s Airline Pilots Union Says Unions May Block Planes '
 '(Update1) China #39;s Air')
('Wall Street Seen Flat After Jobless Data NEW YORK (Reuters) - '
 'Wall Street was expected to see a slightly lower open on Friday')
('Google IPO Imminent Google #39;s long-awaited stock sale is '
 'imminent, and the company is already considering whether to sell '
 'its')

```

Как вы можете заметить, сгенерированный текст имеет структуру, схожую со структурой бизнес-компонента набора AG News. Однако сгенерированный контент не всегда может быть связным. Это вполне нормально, учитывая, что мы использовали малую базовую модель невысокого качества и небольшой объем обучающих данных. Использование Mistral (обученной на 7 млрд данных) или другой очень крупной модели, например Llama 3.1 (обученной на 70 млрд данных), несомненно, позволило бы получить гораздо более связный текст, сохранив при этом тот же формат.

Инструкции

В начале главы мы обсуждали тонкую настройку модели-энкодера под конкретные задачи вроде классификации текста по темам. Однако такой подход требует, чтобы для каждого сценария использования мы обучали новую модель. Если мы сталкиваемся с невиданной ранее задачей (например, определить, является ли текст спамом), у нас не будет готовой предварительно обученной модели, и потребуются тонкая настройка. Это подводит нас к другим методам, поэтому мы кратко обсудим их преимущества, ограничения и применение:

◆ Тонкая настройка нескольких моделей

Мы можем каждый раз настраивать базовую модель, чтобы она могла эффективно решать задачу. В этом случае все веса обновляются во время тонкой настройки. Это означает, что, если мы хотим решить, например, пять различных задач, мы получим пять тонких настроек модели.

◆ Адаптеры

Вместо того чтобы изменять все веса модели, можно «заморозить» базовую модель и обучить ее небольшую вспомогательную версию, которая называется адаптером. В этом случае для каждой задачи у нас по-прежнему будет отдельная версия модели, но они будут значительно меньше в размере, что позволяет легко использовать несколько адаптеров без дополнительных затрат. В данный момент в ИИ-сообществе ведутся активные исследования по управлению сотнями (или даже тысячами) адаптеров в рабочей среде. Они широко популярны и используются как отдельными специалистами, так и в промышленном масштабе. Вы узнаете об адаптерах в следующем разделе.

◆ Промпты

Как известно из первой главы, можно использовать надежную предварительно обученную модель с нулевым или несколькими выстрелами для решения различных задач. При нулевом выстреле пишется промпт, подробно объясняющий задачу. При нескольких выстрелах также добавляются примеры решений (помеченные данные) и улучшается производительность модели. Результативность этих подходов зависит от мощности базовой модели. Достаточно мощная модель, например Llama 3.1, может дать впечатляющие результаты при нулевом выстреле, что отлично подходит для решения самых разных задач, таких как написание длинных и связных электронных писем или реферирование глав книги.

◆ Тонкая настройка с учителем (*Supervised fine-tuning, SFT*)

SFT, также известная как настройка с помощью инструкций (*Instruct-tuning*), — это альтернативный и простой способ улучшить производительность LLM с нулевым выстрелом⁷. Метод формулирует задачи в виде инструкций, например: «Тема этой публикации — бизнес или спорт?» или «Переведите “как дела” на испанский». Подход в основном включает в себя создание набора данных с инструкциями для множества задач и последующую тонкую настройку предварительно обученной языковой модели с использованием смеси наборов данных и инструкций, как показано на рис. 6.4. Создание наборов для настройки через инструкции — задача управляемой сложности. Например, мы могли бы использовать набор AG News и структурировать входные данные и метки в виде инструкций, создав промпт, подобный этому:

To which of the "World", "Sports", "Business" or "Sci/Tech" categories does the text correspond to? Answer with a single word:

⁷ Несмотря на то что изначально этот метод носил название *Instruct-tuning*, ИИ-сообщество после публикации статьи про *InstructGPT* все же предпочло называть его *Supervised fine-tuning*, особенно в контексте моделей с интерфейсом в виде чата.

Text: Wall St. Bears Claw Back Into the Black (Reuters)

Reuters - Short-sellers, Wall Street's dwindling\\band of ultra-cynics, are seeing green again.

Тонкая настройка через инструкции



Рис. 6.4. С помощью настройки через инструкции можно форматировать множество маркированных наборов данных как задачи генерации
(изображение адаптировано из статьи «Scaling Instruction-Finetuned Language Models»)

Создав достаточно большой набор данных из разнообразных инструкций, мы можем получить универсальную модель с тонкой настройкой через инструкции, способную решать множество задач (даже новых благодаря обобщению между ними). Эта идея лежит в основе Flan — модели, способной решать 62 задачи прямо «из коробки». Данная концепция была расширена в Flan T5 — семействе моделей T5, способных решать более 1000 задач, с тонкой настройкой через инструкции и открытым исходным кодом. Стоит отметить, что модель обучается на входных (инструкциях) и выходных (ответах) текстах. В отличие от примера с тонкой настройкой SmolLM, этот метод обучения является контролируемым (с учителем). Настройка через инструкции была очень популярна в архитектурах типа энкодер-декодер, например T5 или BART, благодаря структуре входов-выходов для набора данных. Впоследствии эта идея была распространена на большинство LLM.

Вы можете задаться вопросом: как понять, когда следует использовать тонкую настройку, инструкции или промпты? Все зависит от задачи, доступных ресурсов, желаемой скорости экспериментирования и многих других факторов. Обычно модели с тонкой настройкой под конкретные задачи или предметные области работают лучше. С другой стороны, они не могут решать задачи сразу «из коробки». Использование инструкций — более универсальный метод, но определение набора

данных и структуры требует дополнительных усилий. Промпты — наиболее гибкий подход для быстрых экспериментов, поскольку он не требует, чтобы модель была уже обучена. Тем не менее ему нужно, чтобы базовая модель была более мощной, а контроль над генерацией ограничен.

Мы не будем рассматривать настройку через инструкции от начала до конца, поскольку в основном эта задача касается набора данных, а не моделирования. Однако вы можете изучить несколько отличных статей, если хотите глубже погрузиться в эту тему:

- ◆ Авторы статьи «*Finetuned Language Models Are Zero-Shot Learners*» (февраль 2022 г.) обучили модель Flan, используя настройку через инструкции, которая превосходит базовую модель по производительности как при нулевом, так и при нескольких выстрелах.
- ◆ После Flan появилась новая волна статей, касающихся различных наборов данных. В работе «*Cross-Task Generalization via Natural Language Crowdsourcing Instructions*» (март 2022 г.) представлен *Natural Instructions* — набор данных из 61 задачи с инструкциями для человека и 193 000 пар «ввод-вывод», сгенерированных с помощью сопоставления существующих NLP-наборов с унифицированной схемой. Идея заключается в том, что люди могут следовать инструкциям для решения новых задач, обучаясь (с учителем) на примерах. Авторы использовали настроенную через инструкции модель BART (энкодер-декодер), что привело к повышению производительности на 19% при обобщении между задачами по сравнению с вариантом без использования инструкций. Чем большему количеству задач обучена модель, тем лучше она работает.
- ◆ В статье «*Multitask Prompted Training Enables Zero-Shot Task Generalization*» (март 2022 г.) продемонстрирована аналогичная концепция унифицированных схем данных для различных задач. Авторы доработали T5 и создали T0 — модель типа энкодер-декодер, которая обучена на многозадачном наборе и может обобщаться на большее количество задач. Одним из интересных моментов является то, что чем больше задач представлено в данных, тем выше медианная производительность модели без снижения вариативности.
- ◆ В статье «*Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks*» (октябрь 2022 г.) представлен новый набор данных, содержащий более 1600 задач с 5 миллионами примеров. Разница между ним и вариантом из предыдущего пункта заключается в том, каким образом были сгенерированы наборы данных. T0 ретроактивно (задним числом) формирует инструкции на основе уже имеющихся экземпляров задач, в то время как *Natural Instructions* позволяет исследователям NLP создавать инструкции и экземпляры наборов данных.

Существует альтернативный подход. Он заключается в том, чтобы создавать выходные данные с использованием LLM:

- ◆ *Unnatural Instructions* (декабрь 2022 г.) — это набор с автоматически сгенерированными данными. Авторы собрали 64 000 примеров: они запросили модель с тремя исходными примерами инструкций и попросили сгенерировать четвер-

тый. Затем этот набор дополнили тем, что запросили модель перефразировать каждую инструкцию, создав в общей сложности примерно 240 000 примеров.

- ♦ Набор *Self-Instruct* (май 2023 г.) использует собственную генерацию языковых моделей. Идея заключается в том, чтобы модель генерировала инструкцию, затем входные данные (обусловленные инструкцией) и, наконец, выходные данные⁸. Синтетически сгенерированные наборы, как правило, содержат больше шума. Они могут привести к созданию модели, которая менее надежна, чем та, что обучена на меньшем количестве данных (сгенерированных человеком), но примеры здесь подобраны лучше.
- ♦ *LIMA* (май 2023 г.) — набор данных с англоязычными инструкциями очень малого размера. Несмотря на то что он содержит всего тысячу примеров, авторам удалось тонко настроить модель Llama с его помощью. Этого удалось достичь благодаря сильному предварительному обучению и тщательному отбору обучающих данных.

Это лишь некоторые примеры из огромного количества моделей, настроенных через инструкции. *Flan-T5* — тонко настроенная модель семейства T5, использующая набор данных FLAN. *Alpaca* — модель Llama, тонко настроенная на наборе данных с инструкциями, сгенерированными с помощью *InstructGPT*. *WizardLM* — модель Llama, настроенная через инструкции на наборе *Evol-Instruct*. *ChatGLM2* — тонко настроенная двуязычная модель, обученная с помощью инструкций на английском и китайском языках. ИИ-сообщество сейчас находится в поисках единой формулы, которая объединила бы сильную базовую модель с разнообразными наборами инструкций (которые могут быть созданы человеком или моделью).

В статье «*Learning to Generate Task-Specific Adapters from Task Description*» (июнь 2021 г.) представлен иной подход к улучшению способности обобщать области, в которых модели были бы полезными. Вместо того чтобы стремиться создать общую нейросеть для всех задач, авторы создали *адаптеры* — отдельные версии моделей, параметры которых специфично настроены под конкретные сценарии использования. Несмотря на то что они существуют уже много лет, в последнее время их применение получило широкое распространение в области генерации естественного языка и изображений. В языковых моделях с миллиардами параметров важно, чтобы модель была тонко настроена для своей области или задачи, поэтому в следующем разделе речь пойдет об адаптерах.

Подводя итоги этого раздела, можно отметить, что двумя основными компонентами настройки через инструкции являются надежная базовая модель и высококачественный набор данных. Качество последнего, что неудивительно, играет ключевую роль для модели. Он может быть сгенерирован синтетически (например, с помощью *Self-Instruct*), вручную или представлять собой комбинацию этих двух методов. Исследования неизменно показывают, что чем больше инструкций представлено в обучающих данных, тем лучше работает модель. Шаблон инструкций может существенно влиять на конечную производительность. Существующие на-

⁸ На практике все гораздо сложнее. Авторы предоставили восемь случайно выбранных инструкций и попросили модель сгенерировать их еще больше. Авторы также удалили дублирующиеся и похожие инструкции.

боры в итоге представляют собой компромисс между качеством и разнообразием данных.

Краткое введение в адаптеры

До этого момента мы рассматривали примеры с тонкой настройкой DistilBERT для классификации текста и предварительной подготовкой SmolLM для генерации текста в заданном стиле. В обоих случаях все веса моделей изменялись в процессе работы с ними. Первый подход гораздо эффективнее второго, поскольку не требует большого количества данных или огромной вычислительной мощности. Однако по мере роста популярности более крупных моделей традиционная тонкая настройка становится невыполнимой на потребительском оборудовании. Кроме того, если возникает необходимость тонкой настройки модели-энкодера для различных задач, приходится использовать несколько моделей, что увеличивает требования к объему памяти и вычислительным ресурсам.

Здесь нам на помощь приходит *PEFT (Parameter-efficient fine-tuning)*. Подход представляет собой группу методов, которые позволяют адаптировать предварительно обученные модели без необходимости тонкой настройки всех параметров. Обычно мы добавляем небольшое количество дополнительных параметров, называемых *адаптерами*, а затем настраиваем их, одновременно «замораживая» исходную предварительно обученную модель. Это дает следующие преимущества:

- ◆ *Более быстрое обучение и более низкие требования к оборудованию*

При традиционной тонкой настройке происходит обновление множества параметров. При использовании PEFT обновляется только адаптер, у которого процент параметров невелик по сравнению с базовой моделью. Следовательно, обучение происходит гораздо быстрее и может выполняться на графических процессорах с меньшей мощностью.

- ◆ *Низкие затраты на хранение*

После тонкой настройки хранить нужно только адаптер, а не всю модель целиком. Поскольку некоторые модели могут содержать более 100 ГБ информации, масштабирование в этом случае будет непрактичным, если для каждой последующей модели потребуется повторное сохранение всех параметров. Адаптер может занимать 1% от размера исходной модели. Например, если у нас есть 100 тонких настроек модели объемом 100 ГБ, традиционная тонкая настройка займет 10 000 ГБ памяти, в то время как PEFT — 200 ГБ (исходная модель и 100 адаптеров по 1 ГБ каждый).

- ◆ *Сопоставимая производительность*

Производительность PEFT-моделей, как правило, сопоставима с производительностью полных тонко настроенных моделей.

- ◆ *Отсутствие задержек*

После обучения адаптер можно объединить с предварительно обученной моделью. Это означает, что конечный размер и задержка вывода будут одинаковыми.

Все это звучит слишком хорошо, чтобы быть правдой. Поэтому возникает вопрос: как это работает? Существует несколько методов PEFT. Среди наиболее популярных — *настройка префикса*, *настройка промпта* и *адаптация низкого ранга* (**Low-rank adaptation, LoRA**). Мы рассмотрим их дальше в этой главе. LoRA позволяет обновлять веса с помощью двух матриц меньшего размера, называемых *матрицами обновлений*, с использованием декомпозиции низкого ранга. Несмотря на то что это можно применить ко всем блокам в моделях-трансформерах, они обычно применяются только к блокам внимания.

PEFT — это простая библиотека для использования описанных выше методов с библиотеками `transformers` и `diffusers`. Для начала обсудим, как создать адаптер для модели `SmolLM` из предыдущего раздела. В случае LoRA мы можем управлять несколькими параметрами:

♦ Ранг r

Этот параметр управляет размером матриц обновления. Бóльший ранг позволяет адаптеру изучать более сложные шаблоны, но требует больше параметров.

♦ `lora_alpha`

Этот параметр масштабирует матрицы обновления. Например, если `lora_alpha = 32`, а $r = 8$, обновления градиента будут масштабированы в 4 раза. Это аналогично использованию более высокой скорости обучения во время обучения.

♦ `lora_dropout`

Вероятность отсеечения для слоев LoRA, которая может помочь в борьбе с переобучением.

♦ `task_type`

Тип задачи, например `SEQ_CLS` (классификация последовательностей) или `CAUSAL_LM` (модель причинно-следственной связи). Этот параметр определяет архитектуру адаптера.

♦ `use_dora`

DoRA — это вариант LoRA, который особенно хорошо подходит для достижения производительности, сопоставимой с полной тонкой настройкой. Мы не будем использовать его в этом примере, но вам полезно знать, что он существует.

Матрица обновления в LoRA — это не просто матрица. Она представляет собой особый вид, называемый *матрицей низкого ранга*. Представьте, что у вас есть огромная матрица с большим количеством информации. Идея матриц низкого ранга заключается в том, чтобы суммировать их, используя меньше строк и столбцов, без потери важной информации. Для тех, кто интересуется (на очень высоком уровне) математикой, лежащей в основе LoRA, матрица обновления представлена в виде разложения низкого ранга, где W_0 — исходный вес, а x — входные данные:

$$h = W_0 x + \Delta W = W_0 x + \frac{\alpha}{r} B A x.$$

В LoRA обновление ΔW выражается как произведение двух матриц низкого ранга B и A , причем B имеет меньше строк, а A — меньше столбцов, чем исходная мат-

рица весов. Коэффициент масштабирования $\alpha \times r$ определяет величину этого обновления. Размеры B составляют $d \times r$, а размеры A — $r \times k$, где d и k — строки и столбцы исходных весов соответственно.

Перейдем к коду. Первый шаг — создать конфигурацию метода PEFT.

```
from peft import LoraConfig, get_peft_model

peft_config = LoraConfig(
    r=8, lora_alpha=32, lora_dropout=0.05, task_type="CAUSAL_LM"
)

model = AutoModelForCausalLM.from_pretrained("HuggingFaceTB/SmolLM-135M")
peft_model = get_peft_model(model, peft_config)
peft_model.print_trainable_parameters()

trainable params: 460,800 || all params: 134,975,808 || trainable%: 0.3414
```

Исходная модель содержит почти 135 миллионов параметров, но обучению подлежат только около 460 000 из них. Это всего лишь 0,34% от размера исходной модели. Идея заключается в том, чтобы обучить этот небольшой адаптер и получить производительность, аналогичную модели, которая в 300 раз больше.

Как работает PEFT изнутри? При тонкой настройке базовой модели обновляются все слои. Как уже обсуждалось, LoRA аппроксимирует матрицы обновления с помощью двух других матриц меньшего размера. Предположим, что есть одна матрица обновления с 10 000 строками и 20 000 столбцами. А значит, она содержит 200 миллионов значений. Допустим, мы используем LoRA с рангом 8. Первая матрица (A) будет иметь 10 000 строк и 8 столбцов, а вторая (B) — 8 строк и 20 000 столбцов, чтобы обеспечить одинаковый размер входных и выходных данных. У модели A будет 80 000 значений, а у модели B — 160 000. Таким образом мы перешли от 200 миллионов значений к 240 000, что в 800 раз меньше исходной модели. LoRA предполагает, что эти матрицы достаточно хорошо аппроксимируют матрицы обновления весов.

Параметр r , как уже упоминалось, управляет размерностью матриц LoRA, что приводит к компромиссу между возможностями и переобучением. Слишком высокое его значение приведет к созданию сложных адаптеров, склонных к переобучению. Слишком маленькое значение ранга может привести к низкой производительности, поскольку модель не сможет охватить достаточное количество данных.

Второй ключевой параметр — α . Он контролирует степень влияния адаптеров на исходную модель. Чем он выше, тем важнее адаптер. Выбор значений r и α зависит от задачи и модели. Для LLM хорошей отправной точкой будет использование ранга, равного 8, и последовательное использование α , вдвое превышающего ранг. После тонкой настройки мы можем объединить веса LoRA обратно в исходную модель, как показано на рис. 6.5. Это означает, что задержка и объем вычислений, необходимых для выполнения вывода с использованием модели, будут абсолютно одинаковыми как с объединенным адаптером LoRA, так и без него:

$$\text{Вес} = \text{Вес} + \text{Масштабирование} \times (B \times A).$$

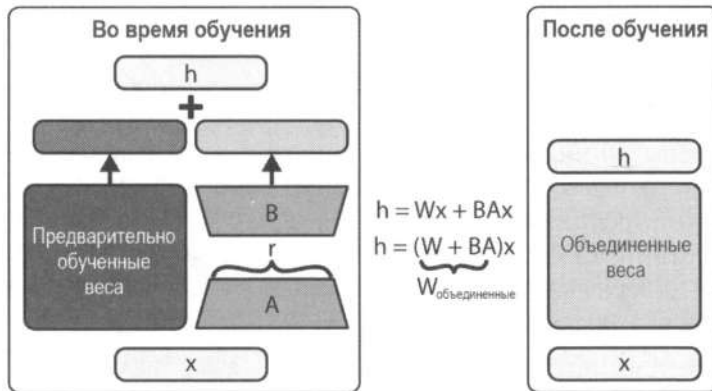


Рис. 6.5. LoRA сокращает количество обучаемых весов.
 После обучения веса LORA можно объединить с исходной моделью

Одна из интересных особенностей LoRA заключается в том, что подход портативен и практичен в использовании на производстве. Представьте себе сценарий, в котором пользователи ожидают, что чат-бот или генератор изображений будет генерировать данные в 10 разных стилях, неизвестных исходной модели. Вместо того чтобы десять раз настраивать исходную модель и загружать ее версии, мы можем загрузить и выгрузить адаптер по мере необходимости. Современные технологии, такие как LoRAX, позволяют обслуживать более сотни тонко настроенных адаптеров с помощью одного графического процессора.

В некоторых случаях может потребоваться объединение адаптеров. Так же, как мы обновили один адаптер, мы можем сделать это и с несколькими:

$$\text{Вес} = \text{Вес} + \text{Масштабирование}_1 \times (B_1 \times A_1);$$

$$\text{Вес} = \text{Вес} + \text{Масштабирование}_2 \times (B_2 \times A_2);$$

$$\text{Вес} = \text{Вес} + \text{Масштабирование}_3 \times (B_3 \times A_3).$$

Последний вопрос, который осталось рассмотреть, — какие параметры можно обновлять с помощью LoRA? Использование этого подхода в большом количестве блоков часто приводит к малому повышению производительности из-за увеличения требований к памяти во время обучения, что является разумным компромиссом и может быть реализовано с помощью параметра `target_modules`. LoRA можно использовать в блоках внимания для быстрых экспериментов (обычно эта настройка включена по умолчанию в библиотеке PEFT). Также можно использовать параметр `target_modules="all-linear"`, чтобы выбрать все линейные модули, за исключением выходных.

Несмотря на то что в этой главе мы акцентируем внимание на тонкой настройке для генерации текста, PEFT широко используется и в других областях, например генерации изображений (которую мы рассмотрим в главе 7), сегментации изображений и т. д.

Краткое введение в квантование

Технология PEFT позволяет тонко настраивать модели, сокращая вычислительные затраты и дисковое пространство. Однако их размер не уменьшается. Если реализовать вывод модели с 30 миллиардами параметров, вам все равно понадобится мощный графический процессор для ее запуска. Например, для Llama с 405-байтовым объемом данных потребуется более 8 GPU типа A100, которые являются довольно мощными и дорогими (каждый стоит более 15 000 долларов США). В этом разделе мы обсудим метод, который позволит запускать модели на менее мощных GPU без снижения производительности.

Каждый параметр модели имеет тип данных, также называемый *точностью* (рис. 6.6). Например, тип `float32` (FP32, также называемый полной точностью) хранит число с плавающей точкой длиной 32 бита. Он позволяет представлять широкий диапазон чисел с высокой точностью, что важно для предварительного обучения моделей. Однако часто применение такого широкого диапазона не требуется. В этих случаях мы можем использовать `float16` (FP16, также называемый половинной точностью). Он имеет меньшую точность и меньший диапазон чисел (максимально возможное число — 64 000), что влечет за собой некоторые риски, а именно модель может переполниться (если число не попадает в диапазон представимых чисел). Однако во время вывода использование FP16 допустимо, поскольку риски, характерные для половинной точности, существенны лишь во время обучения. Третий тип данных — это *brain floating-point*, он же `bfloat16` (или BF16). Он использует 16 бит, как и FP16, но распределяет их иначе, чтобы добиться большей точности для меньших чисел (например, тех, которые обычно встречаются в весах нейросетей), при этом охватывая тот же общий диапазон, что и FP32.

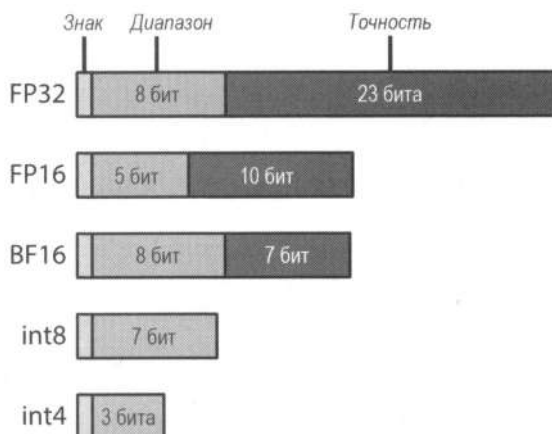


Рис. 6.6. Представление различных степеней точности

Предположим, у нас есть модель с 7 миллиардами параметров, каждый из которых по 32 бита. Итого мы имеем 224 миллиарда бит, а это — 28 миллиардов байт или примерно 26 ГБ. При использовании половинной точности нам потребовалось бы всего 13 ГБ. Это значительно сокращает потребление памяти, что может привести

к ускорению вывода и снижению затрат. В *приложении В* к этой книге есть обсуждения требований к памяти для различных моделей и уровней точности. Проверьте свои знания:

- ◆ Сколько памяти займет модель объемом 135 млн параметров в половинной точности?⁹
- ◆ Сколько памяти займет модель объемом 405 млрд параметров в половинной точности?¹⁰

Полная точность для обучения и вывода обычно приводит к наилучшим результатам, но она значительно медленнее половинной. Поэтому для обучения часто используется смешанный тип точности, что обеспечивает значительное ускорение в работе. В этом случае веса поддерживаются как при полной точности в качестве эталонных, но операции выполняются с половинной точностью. Затем обновления с половинной точностью используются для обновления весов с полной точностью.

Точность не оказывает существенного влияния на вывод, поэтому мы можем загрузить модель с половинной точностью. PyTorch по умолчанию загружает все модели с полной точностью, поэтому нам нужно указать тип при загрузке модели, передав параметр `torch_dtype`, если мы хотим использовать `float16` или `bfloat16`.

```
model = AutoModelForCausalLM.from_pretrained("gpt2", torch_dtype=torch.float16)
```

Загрузка модели (7 млрд параметров) с 16 битами вместо 32 на параметр потребует наличия 13 ГБ видеопамати, что вполне приемлемо для некоторых пользовательских видеокарт. Подобные модели (например, Llama, Mistral или Gemma) стали использоваться в качестве популярных решений для потребительских видеокарт, но существуют модели с еще большим количеством параметров. Например, если мы хотим использовать модель с 32 млрд параметров с половинной точностью, нам понадобится видеокарта на 64 ГБ, чем не могут похвастаться обычные потребительские видеокарты. Возникает вопрос: как можно использовать эти модели?

На первый взгляд, можно было бы попробовать просто уменьшить диапазон или точность чисел до четверти от полных значений (8 бит на параметр). К сожалению, это привело бы к значительному снижению производительности. Помочь в этом нам может *8-битное квантование*. Его идея заключается в том, чтобы преобразовывать значения одного типа (например, `fp16`) в `int8`, который будет представлять значения в диапазоне `[-127, 127]` или `[0, 256]`.

Существуют различные 8-битные методы квантования. Для начала рассмотрим простое *absmax-квантование*. Сначала для заданного вектора вычисляется его максимальное абсолютное значение. Затем на него делится число 127 (максимально возможное значение). Таким образом, мы находим *коэффициент квантования*. При умножении вектора на этот коэффициент мы получим обратно максимально возможное значение 127. Далее мы можем *деквантовать* массив, чтобы получить

⁹ Она займет около 260 МБ. Это достаточно малый размер, который позволяет работать даже локально в веб-браузере.

¹⁰ Она займет 405 млрд × 16 ~ 800 ГБ. Для ее работы потребуются как минимум два узла с 8 графическими процессорами A100 на каждом.

исходные числа, но часть информации при этом будет потеряна. Эту мысль легче понять, выполнив код.

```
import numpy as np

def scaling_factor(vector):
    #Получаем наибольшее значение вектора
    m = np.max(np.abs(vector))

    #Возвращаем коэффициент масштабирования
    return 127 / m

array = [1.2, -0.5, -4.3, 1.2, -3.1, 0.8, 2.4, 5.4, 0.3]
alpha = scaling_factor(array)
quantized_array = np.round(alpha * np.array(array)).astype(np.int8)
dequantized_array = quantized_array / alpha

print(f"Scaling factor: {alpha}")
print(f"Quantized array: {quantized_array}")
print(f"Dequantized array: {dequantized_array}")
print(f"Difference: {array - dequantized_array}")

Scaling factor: 23.518518518518515
Quantized array: [ 28 -12 -101 28 -73 19 56 127 7]
('Dequantized array: [ 1.19055118 -0.51023622 -4.29448819 1.19055118
'-3.10393701 0.80787402 2.38110236 5.4 0.2976378 ]')
('Difference: [ 0.00944882 0.01023622 -0.00551181 0.00944882 0.00393701
'-0.00787402 0.01889764 0. 0.0023622 ]')
```

Различия между квантованным и деквантованным вектором приводят к снижению производительности. Из-за этого классические методы квантования являются неэффективными при масштабировании модели с миллиардами параметров. Метод `LLM.int8()` позволяет выполнять 8-битное квантование без ухудшения качества. Его идея заключается в том, чтобы извлечь выбросы (т. е. значения, выходящие за определенные границы) и вычислить их матричное произведение с точностью `fp16` (при этом для остальных вычислений используется `int8`). Такая структура смешанной точности позволяет обрабатывать 99,9% значений в 8-битном формате и 1% в формате полной или половинной точности без снижения производительности.

Основная цель `LLM.int8()` — снизить требования к графическим процессорам при выполнении вывода модели. Учитывая дополнительные накладные расходы на преобразование, вывод будет медленнее (на 15–30%), чем при использовании `fp16`. Стоит также отметить, несмотря на то, что все GPU последних лет оснащены тензорными ядрами для `int8`, некоторые старые графические процессоры могут не поддерживать эту функцию.

Границы низкоточного вывода расширяются благодаря новым методам 4-битного и 2-битного квантования. Существуют даже исследования, касающиеся применения суб-1-битного квантования. Достижение квантования без ухудшения качества — область исследований, представляющая огромный интерес, учитывая тенденцию к увеличению размеров моделей. В начале этого раздела нам требовался графиче-

ский процессор с 28 ГБ памяти для загрузки модели с 7 млрд параметров. Теперь мы можем загрузить ту же модель, используя 7 ГБ памяти без ухудшения качества, но при этом скорость вывода будет снижена (незначительно).

Библиотека `transformers` располагает различными методами квантования, например AWQ, GPTQ, а также 4-битным и 8-битным квантованием с помощью библиотеки `bitsandbytes`.

Загрузить модель в 8-битном формате можно, создав объект `BitsAndBytesConfig` и указав параметр `load_in_8bit`. Затем вы можете передать их модели при ее загрузке.

```
from transformers import BitsAndBytesConfig

quantization_config = BitsAndBytesConfig(load_in_8bit=True)
model = AutoModelForCausalLM.from_pretrained(
    "gpt2", quantization_config=quantization_config
)
```

Помимо квантования, мы можем сделать еще несколько вещей для удобства в работе с очень большими моделями. Один из популярных методов вывода называется *разгрузкой (offloading)*, он показан на рис. 6.7. Если модель слишком велика для GPU, ее можно разделить на несколько контрольных фрагментов, которые автоматически обрабатываются с помощью библиотеки `transformers`. Какие преимущества это дает? Если модель слишком большая, мы можем загрузить только те слои или фрагменты, которые нам нужны, и выгрузить остальные в оперативную память, которая намного медленнее. Это позволяет работать с моделями любого размера, но при этом будут увеличены затраты на скорость вывода, что может быть неприемлемо для множества больших моделей. Если модель настолько велика, что не помещается в оперативную память вашего компьютера, ее можно выгрузить на диск, что сделает весь процесс еще медленнее, но позволяет работать с моделями любого размера (при условии, что они помещаются на диске).

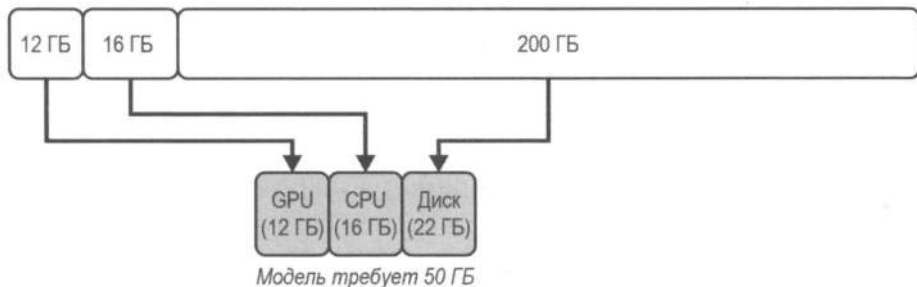


Рис. 6.7. Разгрузка модели

Собираем все вместе

Рассмотрим PEFT и квантование:

- ♦ PEFT позволяет тонко настраивать модели, используя гораздо меньше вычислительных ресурсов, добавляя адаптеры и фиксируя базовые веса. Это ускоряет обучение, поскольку обновляются лишь некоторые веса.

- ◆ Квантование позволяет загружать модель, используя меньше бит, чем требуется для хранения. Это снижает требования к графическому процессору для загрузки и выполнения вывода.

Возникает вопрос: почему бы не использовать оба варианта? Предположим, мы обучаем модель в 8-битном формате. С одной стороны, высокая точность важна для предварительного обучения или тонкой настройки больших моделей. С другой — PEFT «замораживает» базовую модель и использует только адаптер меньшего размера. Таким образом, мы могли бы использовать более низкую точность, достигая той же производительности.

Модель QLoRA позволяет тонко настраивать большие модели на графических процессорах меньшей мощности. Этот метод очень похож на LoRA, но он также применяет квантование. Сначала базовая модель квантуется до 4-битного формата и замораживается. Затем добавляются адаптеры LoRA (две матрицы) и сохраняются в формате `bfloat16`. При тонкой настройке QLoRA использует базовую модель с 4-битным форматом хранения и 16-битную модель с половинной точностью для выполнения вычислений.

Для загрузки модели в 4-битном формате достаточно изменить параметр `load_in_4bit` при создании объекта `BitsAndBytesConfig`. Попробуем реализовать это на модели Mistral с 7 млрд параметров, которая хорошо подходит в качестве базовой модели. Поскольку она довольно большая по сравнению с предыдущими, мы также укажем настройку `device_map="auto"`. В этом случае модель автоматически попытается заполнить все пространство GPU, а затем передаст веса на центральный процессор, если модель не поместится в видеокарту (это значительно замедлит работу, но модель все равно загрузится).

И последнее замечание перед использованием Mistral: ее репозиторий является закрытым. Авторы требуют явного согласия с условиями лицензии. Для этого необходимо посетить ее страницу на Hugging Face, войти в свой аккаунт, прочитать условия лицензии и нажать кнопку «Принять», если вы согласны с условиями. Для программного доступа к модели (как во фрагменте кода ниже) необходимо пройти аутентификацию с тем же аккаунтом. Проще всего это сделать, установив пакет `Python huggingface_hub` (он входит в состав библиотеки `transformers`) и выполнив команду `huggingface-cli login` в терминале. Вам будет предложено ввести токен доступа, который можно создать на странице настроек (Setting). Если вы загружаете модель из Google Colab, то можете настроить секретный ключ `HF_TOKEN` и разрешить своему компьютеру использовать его.

```
quantization_config = BitsAndBytesConfig(load_in_4bit=True)
```

```
model = AutoModelForCausalLM.from_pretrained(
    "mistralai/Mistral-7B-v0.3",
    quantization_config=quantization_config,
    device_map="auto",
)
```

Объект `BitsAndBytesConfig` обеспечивает более детальное управление методами квантования, используя дополнительные аргументы, чтобы изменять типы вычислений, применять вложенное квантование и многое другое.

Модель QLoRA — это всего лишь инструмент в нашем арсенале, а не панацея. Она значительно снижает требования к графическому процессору, сохраняя ту же производительность, но при этом увеличивает время обучения. При этом все преимущества PEFT сохраняются, что делает ее предпочтительной для быстрой тонкой настройки больших моделей.

Далее мы выполним тонкую настройку QLoRA, чтобы создать генеративную модель, способную вести простые диалоги. Рассмотрим каждый компонент отдельно.

◆ Базовая модель

Мы будем использовать модель Mistral с 7 млрд параметров. Мы загрузим ее с настройками `load_in_4bit` и `device_map="auto"`, чтобы применить 4-битное квантование.

◆ Набор данных

Мы будем использовать набор Guanaco, содержащий 10 000 высококачественных диалогов между людьми и моделью OpenAssistant.

◆ Конфигурация PEFT

Мы укажем файл конфигурации `LoraConfig` с хорошими начальными значениями по умолчанию: ранг (`r`) равен 8, а альфа (`alpha`) — его удвоенному значению.

◆ Аргументы обучения

Как и прежде, мы можем настроить параметры обучения (например, частоту оценки и количество эпох), а также гиперпараметры модели (скорость обучения, снижение веса или количество эпох).

В предыдущих примерах мы использовали `TrainingArguments` и `Trainer` — два универсальных инструмента из библиотеки `transformers`. При тонкой настройке LLM для авторегрессионных методов нам будут полезны классы `SFTConfig` и `SFTTrainer` из библиотеки `trl`. Они представляют собой обертки вокруг `TrainingArguments` и `Trainer`, оптимизированные для генерации текста. Их возможности включают в себя следующее:

- ◆ Простые инструменты загрузки и обработки наборов данных. Вместо того чтобы выполнять обработку самостоятельно, мы можем использовать параметр `dataset_text_field`, чтобы указать поле, содержащее обучающие данные. Кроме того, мы можем использовать «упаковку», чтобы объединить несколько последовательностей, что полезно для эффективной пакетной обработки.
- ◆ Встроенная поддержка стандартных шаблонов промптов для диалогов и инструкций.
- ◆ Возможность напрямую передавать любой `config`-файл (`PeftConfig`) в `SFTTrainer` для использования методов PEFT.

Как и прежде, мы передадим квантованную модель и набор данных (для быстрого обучения достаточно всего 300 выборок). `SFTTrainer` уже имеет полезные стандартные сортировщики и утилиты для работы с наборами, поэтому токенизация и предварительная обработка данных не требуются.

```

from trl import SFTConfig, SFTTrainer

dataset = load_dataset("timdettmers/openassistant-guanaco", split="train")

peft_config = LoraConfig(
    r=8,
    lora_alpha=16,
    lora_dropout=0.05,
    task_type="CAUSAL_LM",
)

sft_config = SFTConfig(
    "fine_tune_e2e",
    push_to_hub=True,
    per_device_train_batch_size=8,
    weight_decay=0.1,
    lr_scheduler_type="cosine",
    learning_rate=5e-4,
    num_train_epochs=2,
    eval_strategy="steps",
    eval_steps=200,
    logging_steps=200,
    gradient_checkpointing=True,
    max_seq_length=512,
    # Новые параметры
    dataset_text_field="text",
    packing=True,
)

trainer = SFTTrainer(
    model,
    args=sft_config,
    train_dataset=dataset.select(range(300)),
    peft_config=peft_config,
)

trainer.train()

trainer.push_to_hub()

```

Выполнение этого кода может занять около часа или больше, поскольку, как вы помните, QLoRA приводит к более медленному обучению. Пока идет процесс, это хорошая возможность узнать больше о наборе данных. Если вы посетите его официальную страницу, то можете заметить, что он имеет следующий формат.

```

### Human: Can you write a short introduction ....### Assistant: "Monopsony"
refers to a market ...### Human: Now explain it to a dog

```

Немного поясним:

- ◆ Каждый ход начинается с обозначения «### Human:», за которым следует пробел, а затем ответ человека.
- ◆ Ответ модели начинается с обозначения «### Assistant:», за которым следует пробел и сам ответ.
- ◆ Ходов может быть много.

При тонкой настройке модели для разговорных задач обычно используется шаблон *чата*, поэтому важно учитывать многие детали. Добавление новой строки, удаление пробела или добавление дополнительного символа «#» может ухудшить генерацию. Такие ожидания обусловлены форматом обучения. Если модель обучена только на однократных диалогах (сначала идет один вопрос от человека, потом один ответ модели, и на этом диалог завершается), ей будет сложно генерировать качественные многотактные диалоги. Знание формата промптов важно, т. к. нам нужно использовать его для генерации качественных диалогов.

После завершения обучения перейдем к использованию модели. Ранее мы якобы ее использовали для вывода, но на самом деле это был просто ее адаптер. Но теперь мы выполним вывод и с помощью модели, и с помощью адаптера, а затем сравним.

```
#Загружаем базовую модель, как и раньше
tokenizer = AutoTokenizer.from_pretrained("mistralai/Mistral-7B-v0.3")
model = AutoModelForCausalLM.from_pretrained(
    "mistralai/Mistral-7B-v0.3",
    torch_dtype=torch.float16,
    device_map="auto",
)

#Загрузить адаптер можно с помощью «load_adapter»
model.load_adapter("genaibook/fine_tune_e2e") #меняем название нашего адаптера

#В качестве альтернативы вы можете просто использовать
#«from_pretrained» с именем адаптера, и он автоматически позаботится
#о загрузке базовой модели и адаптера.
#model = AutoModelForCausalLM.from_pretrained("genaibook/fine_tune_e2e"...

pipe = pipeline("text-generation", model=model, tokenizer=tokenizer)
pipe("### Human: Hello!### Assistant:", max_new_tokens=100)
```

Вывод данного кода будет примерно таким¹¹.

```
### Human: Hello
### Assistant: Hello! How can I help you?

### Human: I want to know how to make a website.
### Assistant: Sure! Here are some steps to help you get started...
```

¹¹ Для удобства чтения мы добавили новую строку между человеком и моделью, а также дополнительную строку между каждым поворотом.

Результат впечатляет. Мы только что тонко настроили модель с 7 млрд параметров, сделав ее диалоговой. При этом нам не потребовалось мощного графического процессора.

По мере того как диалоговые модели становились все более распространенными, сотрудники из Hugging Face добавили в библиотеку transformers возможность указывать настройку chat_template. Благодаря ей конечным пользователям не нужно сильно беспокоиться о шаблонах сообщений, и они могут сосредоточиться непосредственно на содержании диалогов. Например, можно просто передавать сообщения в модель, а токенизатор автоматически их отформатирует.

```
pipe = pipeline(
    "text-generation", "HuggingFaceTB/SmolLM-135M-Instruct", device=device
)
messages = [
    {
        "role": "system",
        "content": """"You are a friendly chatbot who always responds
        in the style of a pirate""",
    },
    {
        "role": "user",
        "content": "How many helicopters can a human eat in one sitting?",
    },
]
print(pipe(messages, max_new_tokens=128)[0]["generated_text"][-1])

{'content': 'The number of helicopters that can be eaten in one '
            'sitting depends on the number of people in the room. If '
            'there are 10 people, then there are 10 helicopters that '
            'can be eaten in one sitting. If there are 15 people, '
            'then there are 15 helicopters that can be eaten in one '
            'sitting. If there are 20 people, then there are 20 '
            'helicopters that can be eaten in one sitting.\n'
            '\n'
            'The number of helicopters that can be eaten in one '
            'sitting depends on the number of people in the room. If '
            'there are 10 people, then there are 10 helicopters',
 'role': 'assistant'}
```

Если вам нужно просто применить шаблон чата, но не передавать его в модель, вы можете напрямую использовать метод `tokenizer.apply_chat_template()`.

```
tokenizer = AutoTokenizer.from_pretrained("HuggingFaceTB/SmolLM-135M-Instruct")

chat = [
    {"role": "user", "content": "Hello, how are you?"},
    {
        "role": "assistant",
```

```

    "content": "I'm doing great. How can I help you today?",
  },
  {
    "role": "user",
    "content": "I'd like to show off how chat templating works!",
  },
]
tokenizer.apply_chat_template(chat, tokenize=False)

```

Мы можем использовать `print()`, чтобы получить полный промпт, но стоит учитывать, что исходная строка, переданная в модель, содержит такие символы, как «`\n`», для обозначения новых строк.

```
print(tokenizer.apply_chat_template(chat, tokenize=False))
```

```

<|im_start|>user
Hello, how are you?<|im_end|>
<|im_start|>assistant
I'm doing great. How can I help you today?<|im_end|>
<|im_start|>user
I'd like to show off how chat templating works!<|im_end|>

```

Это будет работать для моделей, где `chat_template` указан в файле `tokenizer_config.json`. Чтобы узнать больше о шаблонах чата, рекомендуем ознакомиться с официальной документацией.

Более глубокое погружение в оценку

Как оценить качество сгенерированного текста? Мы рассматривали популярные бенчмарки, оценивающие общие знания и способность к рассуждению, но вам может понадобиться оценить качество сгенерированного текста в более общем контексте. Первое, что следует различать, — это оценка базовых моделей и тонко настроенных (например, чат-моделей). Их результаты различаются, поэтому не следует оценивать их одинаково. Например, не стоит ожидать, что базовая модель будет «из коробки» обладать возможностями следовать инструкциям или вести диалоги в чате, поэтому оценивать ее с точки зрения этих задач было бы неправильно¹².

Рассмотрим некоторые способы оценки базовых моделей:

◆ Сложность (*Perplexity*)

Сложность измеряет, насколько хорошо языковая модель предсказывает заданный набор данных. Более низкое значение указывает на более высокую производительность и меньшую неопределенность при генерации текста. Это позволяет предположить, что модель может точнее предсказать следующее слово.

¹² В последнее время ситуация меняется. Некоторые базовые модели добавляют инструкции в свою смесь обучающих данных, чтобы они могли следовать им сразу после установки.

Сложность особенно важна на этапе обучения базовых моделей, поскольку она отражает их способность усваивать эффективные распределения вероятностей для последовательностей слов.

◆ BLEU

BLEU измеряет сходство между сгенерированным и эталонным текстами. Это достигается с помощью вычисления доли n -грамм в сгенерированном выводе, которые также присутствуют в эталонном тексте. Учитывая, что BLEU в значительной степени опирается на точные совпадения n -грамм, этот метод может не отражать все разнообразие естественного языка, а также не обладать семантическим пониманием.

◆ ROUGE

Подобно BLEU, ROUGE измеряет перекрытие между двумя текстами. Однако метод фокусируется на полноте, а не на точности, что делает его очень полезным для таких задач, как обобщение текста. Тем не менее семантического понимания ему также не хватает, и он склонен к более длинным выводам.

Как видите, оценка базовых моделей — непростая задача. В процессе обучения обычно отслеживаются потери и сложность и ожидается, что оба показателя со временем будут уменьшаться. ROUGE и BLEU часто используются с наборами данных, содержащими справочный текст. Однако обе метрики основаны на точных совпадениях и не учитывают семантическое сходство, что ограничивает их применение во время оценки более креативной или разнообразной генерации текста.

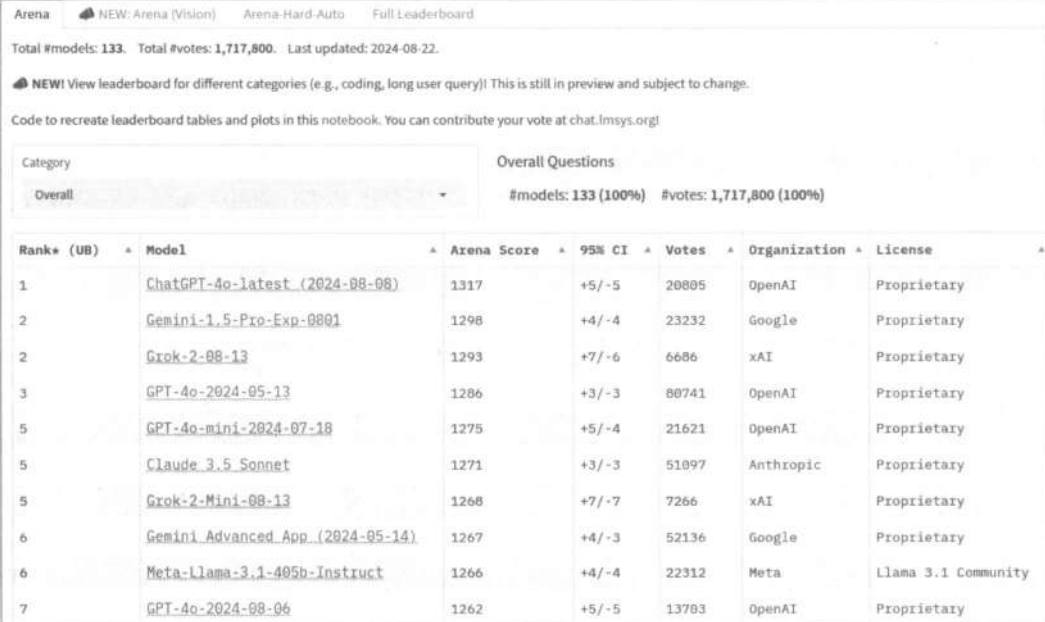


В недавней работе «*The Unlocking Spell on Base LLMs: Rethinking Alignment via In-Context Learning*» утверждается, что большая часть преимуществ, которые мы наблюдаем при использовании настройки через инструкции, на самом деле обусловлена базовой моделью. Авторы проанализировали распределение токенов между ней и моделью, настроенной через инструкции, и обнаружили малозначительные различия в большинстве токенов. Имеющиеся расхождения являются в основном стилистическими и касаются таких токенов, как приветствие и дисклеймеры, которые свойственны разговорным моделям. Это открытие предполагает, что базовые модели уже обладают значительной частью знаний, необходимой для выполнения инструкций, что подчеркивает важность предварительного обучения. Существует также потенциал для методов, не требующих какой-либо настройки (отсутствие тонкой настройки) для достижения эффективности, аналогичной настройке через инструкции, несмотря на то, что эта область все еще находится в стадии активных исследований.

Несмотря на то что эти метрики дают количественную оценку, качественная оценка, включая человеческое суждение, также имеет решающее значение для определения общей согласованности и релевантности сгенерированного текста относительно поставленной задачи. Количественные метрики полезны для масштабных сравнений, но они могут упускать из виду такие тонкости, как плавность, креативность или соответствие контексту. Именно здесь человеческая оценка играет важнейшую роль. Баланс количественных и качественных метрик обеспечивает более полную оценку моделей генерации текста.

Для генеративных моделей, ориентированных на конечного пользователя, один из лучших способов — это поэкспериментировать с ними. Существуют популярные

платформы, например LMSYS, где пользователи взаимодействуют с различными анонимизированными моделями и выбирают наилучшие результаты, которые затем публикуются в таблице лидеров, а модели ранжируются (рис. 6.8). Эти *арены* (так называются таблицы с лидерами сравнения) более показательны, чем автоматизированные таблицы лидеров, поскольку они отражают производительность, более близкую к реальному использованию. К сожалению, получение оценок в рамках арен является дорогостоящим и зачастую невозможным для совершенно новой модели. В таких случаях бенчмарки вроде MT Bench, IFEval, EQ Bench и AGIEval имеют определенную корреляцию с результатами на арене и, следовательно, могут быть полезны для приблизительного представления о том, насколько хорошо модель будет работать в реальных условиях. Новые бенчмарки появляются часто, поэтому важно быть в курсе последних исследований.



Arena NEW: Arena (Vision) Arena-Hard-Auto Full Leaderboard

Total #models: 133. Total #votes: 1,717,800. Last updated: 2024-08-22.

NEW! View leaderboard for different categories (e.g., coding, long user query)! This is still in preview and subject to change.

Code to recreate leaderboard tables and plots in this notebook. You can contribute your vote at chat.lmsys.org!

Category: Overall

Overall Questions: #models: 133 (100%) #votes: 1,717,800 (100%)

Rank* (UB)	Model	Arena Score	95% CI	Votes	Organization	License
1	ChatGPT-4o-latest (2024-08-08)	1317	+5/-5	20805	OpenAI	Proprietary
2	Gemini-1.5-Pro-Exp-0801	1298	+4/-4	23232	Google	Proprietary
2	Grok-2-08-13	1293	+7/-6	6686	xAI	Proprietary
3	GPT-4o-2024-05-13	1286	+3/-3	80741	OpenAI	Proprietary
5	GPT-4o-mini-2024-07-18	1275	+5/-4	21621	OpenAI	Proprietary
5	Claude 3.5 Sonnet	1271	+3/-3	51097	Anthropic	Proprietary
5	Grok-2-Mini-08-13	1268	+7/-7	7266	xAI	Proprietary
6	Gemini Advanced App (2024-05-14)	1267	+4/-3	52136	Google	Proprietary
6	Meta-Llama-3.1-405b-Instruct	1266	+4/-4	22312	Meta	Llama 3.1 Community
7	GPT-4o-2024-08-06	1262	+5/-5	13783	OpenAI	Proprietary

Рис. 6.8. Таблица лидеров LMSYS

Бенчмарки, хотя и являются ценными инструментами, также имеют свои ограничения. Например, те, что основаны на знаниях, часто демонстрируют предвзятость в отношении США, включая вопросы по истории и законодательству. Кроме того, большинство из них основаны на английском, и лишь немногие доступны для других языков. ИИ-сообщество прилагает большие усилия по их переводу, но прогресс идет достаточно медленно, и конечные результаты могут быть несовершенными. Более того, бенчмарки для чат-моделей (а часто и арен) имеют тенденцию больше фокусироваться на однократных, нежели на многотактных диалогах. Оценка очень длинных контекстных моделей остается сложной задачей, и это та область, которую еще предстоит урегулировать.

Самый важный вывод из всего этого заключается в том, что тестировать модель важно именно на задаче, которую вы хотите решить. Бенчмарки полезны, чтобы можно было выбрать исходную базовую модель для тонкой настройки или в генеративных целях, но они не заменят тестирования в реальных условиях.

Время проекта: поисково-дополненная генерация

LLM могут использовать только информацию, основанную на собственном контексте и данных, использованных для обучения. Если вы хотите запросить у LLM информацию по конкретной теме, она сможет ответить, только если эта информация является частью ее данных. Например, если вы попытаетесь спросить Llama о новых вышедших фильмах, ей будет сложно предоставить точный ответ.

Поисково-дополненная генерация (Retrieval-augmented generation, RAG) — это метод, при котором модель может получать доступ к информации (например, к текстам или документам), хранящейся где-либо. С помощью RAG можно сделать так, чтобы LLM могла использовать как пользовательский ввод, так и сохраненную информацию, чтобы сгенерировать ответ. Данный подход эффективен, поскольку позволяет модели получать доступ к большому объему информации, что упрощает ее обновление, которое легче осуществить, чем переобучить модель.

К сожалению, информации может быть огромное количество, поэтому невозможно просто передать ее всю в модель. Для решения этой проблемы можно использовать варианты с эмбедингами (например, трансформеры из главы 2), которые будут кодировать документы с информацией в векторы, а затем хранить их (обычно в так называемой векторной базе данных). Затем выполняется поиск ближайшего соседа, чтобы найти документы, наиболее похожие на введенные пользователем данные. И после введенные пользователем данные и полученные документы передаются в LLM. Этот подход привлекателен тем, что он позволяет модели получать доступ к большому объему информации по мере необходимости.

Ваша цель в этом проекте — построить конвейер (рис. 6.9), в котором происходит следующее:

1. Пользователь вводит вопрос.
2. Конвейер извлекает документы, наиболее похожие на вопрос.
3. Конвейер передает вопрос и найденные документы в LLM.
4. Конвейер генерирует ответ.

Для этой задачи вам не потребуется обучать какую-либо модель. Для поиска данных мы рекомендуем использовать предварительно обученный экземпляр *sentence_transformers*. Вы можете использовать наиболее предпочтительную модель, например Mistral или Llama, для генераций. Обратите внимание, решение задач 1 и 2 находится в задании 3 главы 2, а решение задач 3 и 4 — в этой главе. Цель — собрать все эти части воедино. В качестве руководства вы можете использовать функции ниже.

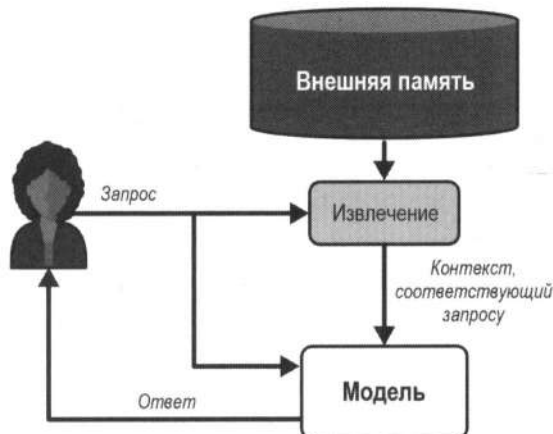


Рис. 6.9. Конвейер RAG

```

def embed_documents(documents: List[str]):
    #Используем модель трансформера предложений для кодирования документов

    #Сохраняем документы в удобном месте

def retrieve_documents(query: str):
    #Используем сохраненные документы для извлечения документов,
    #наиболее похожих на запрос

def generate_response(query: str, documents: List[str]):
    #Используем LLM для генерации ответа

def pipeline(query: str):
    documents = retrieve_documents(query)
    response = generate_response(query, documents)
    return response
  
```

Может возникнуть вопрос: какие документы использовать? Это решать вам, но мы рекомендуем начать с какого-либо минимального набора (например, выбрать 5–10 предложений или абзацев, возможно, составленных вами же), а затем масштабировать и использовать больше документов.



В приложении С представлен пример построения минимального конвейера RAG. Мы рекомендуем сначала попробовать построить конвейер самостоятельно, а затем ознакомиться с полным примером в приложении.

Заключение

В этой главе были рассмотрены методы тонкой настройки. Мы начали с обсуждения традиционной тонкой настройки моделей-энкодеров для классификации текста. Однако такой подход может быть использован и для других задач, например для ответов на вопросы по заданному тексту и идентификации сущностей в тексте. За-

тем мы рассмотрели, как выполнить тонкую настройку модели-декодера для генерации текста. Обсудили преимущества и ограничения тонкой настройки по сравнению с генерацией с нулевым или несколькими выстрелами. Мы также показали, как контролируемая тонкая настройка может позволить генеративной модели решать множество задач «из коробки».

Несмотря на мощь этих методов, их масштабирование до новейших, все более крупных моделей представляет собой проблему. Чтобы решить ее, мы исследовали, как можно использовать квантование, чтобы выполнить вывод с помощью больших моделей на графических процессорах меньшей мощности, и обсудили методы PEFT (Parameter-Efficient Fine-Tuning) для тонкой настройки моделей с меньшими вычислительными затратами и требованиями к дисковому пространству. Комбинируя эти подходы, мы успешно настроили модель с 7 млрд параметров, сделав ее диалоговой. Благодаря этим основам теперь вы можете выполнять тонкую настройку больших моделей для ваших конкретных задач.

Несмотря на то что эта глава в основном посвящена архитектуре моделей и методам тонкой настройки, важно помнить, что успех также зависит от качества и разнообразия обучающих данных. Не всегда очевидно, сколько данных необходимо: это зависит от размера модели, сложности задачи и качества самих данных. Несколько сотен высококачественных обучающих выборок часто могут быть эффективнее тысяч низкокачественных.

Для дополнительной информации рекомендуем использовать следующие ресурсы:

- ♦ Чтобы узнать больше о данных, мы рекомендуем прочесть публикацию *«FineWeb: decanting the web for the finest text data at scale»*. Это подробное введение в наборы данных объемом 15 триллионов токенов, включая тщательное исследование предварительной обработки, а также объяснение того, как создавать высококачественные веб-наборы и автоматические аннотации.
- ♦ Что касается оценки, мы рекомендуем прочитать статью *«Let's Talk About LLM Evaluation»*, где представлен общий обзор, как оценивать модели и связанные с этим сложности. Мы также рекомендуем прочитать публикацию *«Performances are plateauing, let's make the leaderboard steep again»* в блоге *Open LLM Leaderboard*, где представлен подробный обзор оценки моделей, ориентированных на ИИ-сообщество. Юджин Ян (Eugene Yan) написал отличную публикацию в блоге на эту тему.
- ♦ Чтобы узнать больше о LoRA, мы рекомендуем прочитать *«Practical Tips for Fine-tuning LLMs Using LoRA»*, а также публикацию *«Making LLMs even more accessible with bitsandbytes, 4-bit quantization and QLoRA»* о запуске QLoRA.
- ♦ Чтобы узнать больше о квантовании, мы рекомендуем изучить наглядное руководство *«A Visual Guide to Quantization»*, а также книгу *«Hands-On Large Language Models»* Джея Аламмара (Jay Alammar) и Маартена Гроотендорста (Maarten Grootendorst) издательства O'Reilly.
- ♦ Чтобы узнать больше обо всех компонентах, используемых в производстве LLM, мы рекомендуем прочитать публикацию *«Building a Generative AI Platform»*.

Вопросы

1. В чем разница между базовой и тонко настроенной моделями? Какие модели являются разговорными?
2. В каких случаях для тонкой настройки следует выбрать базовую модель на основе энкодера?
3. Объясните различия между тонкой настройкой, настройкой через инструкции и QLoRA.
4. Приводит ли использование адаптеров к увеличению размера модели?
5. Сколько памяти GPU требуется для загрузки модели размером 70 млрд параметров в режимах половинной точности, 8-битного и 4-битного квантования?
6. Почему QLoRA приводит к более медленному обучению?
7. В каких случаях мы замораживаем веса модели во время тонкой настройки?

Ответы на эти вопросы и решение следующей задачи можно найти в репозитории книги на GitHub.

Задачи

Классификация изображений. Несмотря на то что эта глава посвящена тонкой настройке моделей-трансформеров для задач обработки естественного языка (NLP), их также можно использовать для других модальностей, например аудио и компьютерного зрения. Цель этого задания — тонкая настройка трансформеров для классификации изображений. Мы предлагаем следующее:

- ♦ Используйте предварительно обученную модель ViT, например *google/vit-base-patch16-224-in21k*.
- ♦ Используйте набор данных изображений и меток, например *food101*.

Логика здесь практически такая же, но с некоторыми ключевыми отличиями. Например, используйте класс `AutoImageProcessor` вместо `AutoTokenizer`. Рекомендуем ознакомиться с документацией, чтобы лучше разобраться в этом процессе.

Тонкая настройка Stable Diffusion

В предыдущей главе мы рассмотрели, как тонкая настройка может помочь в обучении языковых моделей для генерации текста в определенном стиле или изучения концепций в конкретной области. Те же принципы могут быть применены к моделям преобразования текста в изображение (text-to-image), что позволит настраивать их даже при наличии единственного GPU (в отличие от узлов с несколькими графическими процессорами, необходимыми для предварительного обучения моделей, вроде Stable Diffusion).

В этой главе мы будем использовать базовую предварительно обученную модель Stable Diffusion, которую рассматривали в *главе 5*. Мы расширим ее, чтобы обучить стилям и концепциям, о которых она могла не знать (например, понятию «ваш питомец» или определенному стилю рисования). Также мы рассмотрим, как добавить ей новые возможности (например, *inpainting*) и использовать новые обуславливания в качестве входных данных.

Вместо того чтобы писать код с нуля, в этом разделе мы применим существующие скрипты, созданные для тонкой настройки моделей. Для этого мы рекомендуем вам использовать библиотеку *diffusers*, т. к. в большинстве примеров будет применена именно она.

Тонкая настройка полной модели Stable Diffusion

*Полная модель (Full model)*¹ — это необходимое условие для тонкой настройки, появившейся в результате разработки специальных методов настройки, наподобие LoRA, Textual Inversion и DreamBooth. Они не обеспечивают полную тонкую настройку всей модели, а скорее либо предоставляют эффективный способ для этого (как мы узнали на примере LoRA для LLM в *главе 6*), либо предлагают новые способы обучить модель новым концепциям. Мы обсудим эти методы далее.

До появления таких методов понятия «полная модель» не существовало, поскольку это называлось просто тонкой настройкой. В данном контексте она означает даль-

¹ *Full model* — это математическая структура (или алгоритм), обученная на большом количестве данных для выполнения конкретных задач без явного программирования каждого шага. Такие модели имитируют когнитивные функции человека, включая обучение, распознавание, принятие решений и решение проблем. — Пер.

нейшее обучение модели диффузии, но с тем отличием, что наша цель теперь — направить ее в область конкретных знаний, которые мы хотели бы добавить. Вы можете заставить Stable Diffusion изучить какую-либо тему или область, которые вы не можете реализовать с помощью промптов или которые появились уже после выпуска модели (рис. 7.1). После тонкой настройки модель будет хорошо работать с заданным стилем или темой и может специализироваться преимущественно на создании контента только этого типа данных. В этом разделе мы будем использовать готовый скрипт для полной тонкой настройки, который вы можете найти в файле `diffusers/examples/text_to_image/train_text_to_image.py` из библиотеки `diffusers`.

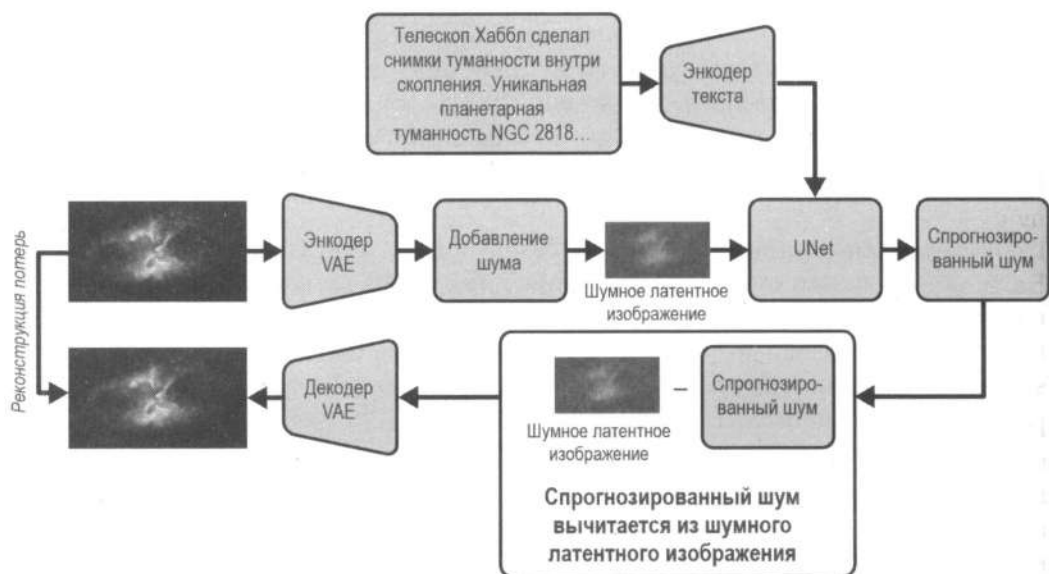


Рис. 7.1. Архитектура тонкой настройки Stable Diffusion

Подготовка набора данных

Важнейший аспект данных — это качество. Фильтрация набора с целью оставить только выборки с высоким качеством, а также удаление низкокачественных выборок могут существенно повлиять на качество тонкой настройки.

Чтобы обеспечить высококачественную тонкую настройку полной модели, потребуется относительно большой набор данных, содержащий более 500 изображений. Несмотря на то что этого может быть более чем достаточно (по сравнению с миллиардами изображений, необходимых для обучения полной модели Stable Diffusion), набор из сотен изображений — это все еще капля в море. В конкретных методах настройки, которые мы обсудим далее в этой главе, вы научитесь настраивать модель, используя всего четыре изображения.

Вернемся к тонкой настройке полной. Управляя моделью `text-to-image`, мы должны показать ей набор данных, содержащий изображения и соответствующие подписи к ним, точно так же, как это было сделано во время ее предварительного обучения.

Вот несколько примеров, которые вы можете попробовать самостоятельно:

- ◆ Картины эпохи Возрождения, чтобы создавать точные копии.
- ◆ Фотографии зданий в вашем любимом архитектурном стиле для создания архитектурной модели.
- ◆ Набор фотографий с пейзажами для тонкой настройки модели и создания реалистичных пейзажей с «безмятежными лесами», «величественными горами» и «тихими берегами озер».



Несмотря на то что подобные исследования приветствуются, использование тонкой настройки в любых целях, кроме образовательных, требует осторожности. Если используемые вами изображения соответствуют стилю какого-либо художника или на них изображены лица людей или материалы, защищенные правами интеллектуальной собственности, то запросить разрешение на их использование у автора или правообладателя будет не просто разумным шагом, а соблюдением законодательства вашей страны.

В нашем примере мы будем использовать снимки телескопа Хаббл, которые NASA публикует в открытом доступе. Создание набора и тонкая настройка модели Stable Diffusion на этих снимках были впервые предприняты исследователем Максвеллом Вайнцирлом (Maxwell Weinzierl) из Техасского университета в Далласе. Он создал набор данных *esa-hubble* и модель Hubble Diffusion версий 1 и 2. В этом примере мы попытаемся воссоздать модель Hubble Diffusion 1, доработав Stable Diffusion v1.5 с помощью *esa-hubble*.

Набор был создан путем сканирования изображений с веб-сайта Европейского космического агентства. К счастью, подписи, описывающие астрономические явления на этих снимках, также доступны, поэтому их можно сохранить и преобразовать в формат, совместимый с библиотекой *datasets*, для последующей тонкой настройки. Сбор данных (с помощью скраппинга или иным способом) выходит за рамки этой книги, но вы можете воспользоваться такими Python-инструментами, как *Scrapy* или *Beautiful Soup*. Но хотим предупредить, что вам следует учитывать политики в отношении сканирования или скраппинга данных каждого веб-сайта.

Теперь, имея пары «изображение-текст», мы можем загрузить их в библиотеку *datasets*. В следующем разделе показано, как это сделать. Если у вас нет доступного набора изображений, можете использовать предоставленный набор *esa-hubble*.

```
from datasets import load_dataset
dataset = load_dataset("imagefolder", data_dir="/path/to/folder")
```

Аргумент *imagefolder* задает специальный режим для директивы *load_dataset*, позволяющий загрузить каталог изображений и файл метаданных с подписями к каждому изображению. Данный режим требует, чтобы все изображения находились в папке */path/to/folder*, а файл *metadata.csv* содержал подпись к каждому изображению. Таким образом, папка может иметь следующую структуру.

```
folder/metadata.csv
folder/0001.png
folder/0002.png
folder/0003.png
```

Файл `metadata.csv` должен выглядеть следующим образом.

```
file_name,text
0001.png,This is a golden retriever playing with a ball
0002.png,A german shepherd
0003.png,One chihuahua
```

После загрузки набора отправьте его в Hugging Face Hub, чтобы поделиться им с сообществом.

```
dataset.push_to_hub("my-hf-username/my-incredible-dataset")
```

Теперь набор данных сохранен и готов к настройке модели. Может случиться так, что у вас будет отличный набор с изображениями, но для них могут отсутствовать подписи. В этом случае вы можете использовать модели `text-to-image`, чтобы создать необходимые описания, которые затем можно использовать для настройки. Модели вроде BLIP-2 от компании Salesforce или Florence 2 от компании Microsoft очень популярны для подобных задач. Если у вас нет подходящего набора данных, не волнуйтесь: вы можете использовать набор `esa-hubble`.

Тонкая настройка модели

Чтобы выполнить тонкую настройку, помимо набора данных нам также понадобятся обучающий скрипт и веса предварительно обученной модели, которую мы и будем настраивать. К счастью, это легко сделать с помощью библиотек `diffusers`, которая предоставляет примеры обучающих скриптов, и `accelerate`, которая позволяет обеспечить эффективную процедуру, позволяющую обучающим циклам PyTorch работать на нескольких GPU, TPU², а также с различной точностью, например `bf16`. Обучающий скрипт полезен тем, что он содержит весь необходимый код для эффективного обучения сети UNet в Stable Diffusion, при этом он предоставляет пользователю управление с помощью гиперпараметров, которые мы изучим далее.

Для выполнения тонкой настройки вам понадобится видеокарта с объемом видеопамяти не менее 16 ГБ. Или вы можете использовать облачные сервисы, например Google Colab Pro. Методы настройки, которые мы изучим в этой главе, позволяют проводить обучение на более «скромных» видеокартах или в бесплатной версии Google Colab.

Вы можете использовать свои собственные данные или набор `esa-hubble`, чтобы воспроизвести модель Hubble Diffusion.

Для начала скопируйте обучающие скрипты из репозитория `diffusers`.

```
git clone https://github.com/huggingface/diffusers.git
cd diffusers/examples/text_to_image/
```

² TPU (Tensor Processing Unit) — специализированный ускоритель машинного обучения, разработанный компанией Google для рабочих нагрузок нейронных сетей. — Пер.

Затем примените скрипт `train_text_to_image.py`. Для конфигураций с ограниченным объемом видеопамати используется параметр `use_8bit_adam`, который требует совместного применения метода квантования `bitsandbytes` и параметра `gradient_accumulation_steps`, чтобы можно было использовать пакеты большего размера, чем обычно помещается в памяти GPU.

```
accelerate launch train_text_to_image.py \
--pretrained_model_name_or_path="stable-diffusion-v1-5/stable-diffusion-v1-5" \
--dataset_name="Supermation/esa-hubble" \
--use_ema \
--mixed_precision="fp16" \
--resolution=512 \
--center_crop \
--random_flip \
--train_batch_size=1 \
--gradient_checkpointing \
--gradient_accumulation_steps=4 \
--use_8bit_adam \
--checkpointing_steps=1000 \
--num_train_epochs=50 \
--validation_prompts \
    "Hubble image of a colorful ringed nebula: \
A new vibrant ring-shaped nebula was imaged by the \
NASA/ESA Hubble Space Telescope." \
    "Pink-tinted plumes in the Large Magellanic Cloud: \
The aggressively pink plumes seen in this image are extremely uncommon, \
with purple-tinted currents and nebulous strands reaching out into \
the surrounding space." \
--validation_epochs 5 \
--learning_rate=1e-05 \
--output_dir="sd-hubble-model" \
--push_to_hub
```

Прежде чем углубляться в параметры, рекомендуем вам запустить скрипт и вернуться к прочтению книги, т. к. обучение займет некоторое время (от одного до трех часов в зависимости от вашей видеокарты).

Подробное описание работы скрипта `train_text_to_image.py` выходит за рамки этой книги. Однако основы, которые вы изучили в *главе 4*, где мы рассматривали основные концепции обучения моделей диффузии, остаются прежними. Ключевые этапы включают подготовку набора данных с изображениями, определение графика шума и создание сети UNet для прогнозирования. Напомним: цикл обучения итеративно добавляет шум к чистым изображениям, затем модель прогнозирует шум, вычисляет разницу между предсказанным и фактическим шумом и обновляет веса модели с помощью обратного распространения. Далее мы подробнее изучим ключевые гиперпараметры (настройки, которые вы задаете перед началом тонкой настройки), которые по-прежнему очень важны. А именно мы рассмотрим все настройки из скрипта обучения выше.

Наиболее важные параметры, которые следует изучить, — это `learning_rate` и `num_train_epochs`:

◆ `learning_rate`

Обозначает частоту обновления весов модели на каждом этапе обучения. Если вы стремитесь достичь более высокой скорости обучения, процесс оптимизации для тонкой настройки может не стабилизироваться, а слишком низкое значение приведет к недостаточному обучению (или ваша модель может вообще не обучиться). Мы рекомендуем экспериментировать в диапазоне от $1e-04$ (0,0001) до $1e-06$ (0,000001).

◆ `num_train_epochs`

Обозначает, сколько проходов модель выполнит по всему набору данных. Обычно модели требуется всего несколько проходов, чтобы выучить концепцию. Другой способ задать, сколько раз модель будет запускать цикл обучения, — настроить переменную `max_train_steps`. В ней можно указать точное количество эпох, которые пройдет модель (даже если она завершается в середине эпохи).

Мы также рассмотрим и другие гиперпараметры, но менее подробно:

◆ `use_ema`

Обозначает использование *экспоненциальной скользящей средней*. Она помогает стабилизировать обучение модели по эпохам за счет усреднения весов.

◆ `mixed_precision`

Определяет, будет ли модель обучаться со смешанной точностью. Если установлено значение `FP16` (как в предыдущем коде), все необучаемые веса будут приведены к половинной точности. Данные веса используются только для вывода, поэтому нам не нужна полная точность. Это позволит использовать меньше видеопамяти и ускорить обучение.

◆ `resolution`

Указывает разрешение изображения, которое используется для обучения. Размер будет изменен на основе таких параметров, как `center crop` (если изображения больше целевого разрешения, они будут обрезаны по центру) и `random flip` (некоторые изображения будут перевернуты во время обучения для повышения надежности).

◆ `train_batch_size`

Указывает, сколько примеров отображается для модели одновременно. Чем больше размер пакета, тем больше требуется видеопамяти, но тем меньше время обучения.

◆ `gradient_checkpointing` и `gradient_accumulation_steps`

Позволяют проводить обучение, используя меньше видеопамяти. Параметр `gradient_checkpointing` расходует градиентную память на дополнительные вычисления, а `gradient_accumulation_steps` позволяет использовать более крупные эффективные пакеты, накапливая результаты из нескольких обучающих мини-пакетов.

◆ `use_8bit_adam`

Определяет, следует ли использовать 8-битный оптимизатор *Adam Optimizer* из *bitsandbytes* для уменьшения требуемой памяти графического процессора. Этот параметр ускоряет обучение и использует меньше памяти, обеспечивая более низкую точность, чем FP16 или FP32 для накопления градиента (суммирование градиентов по нескольким мини-пакетам перед обновлением весов модели).

◆ `checkpointing_steps`

Определяет, после скольких шагов обучения (пакетов, которые увидела модель) нужно сохранять снимок модели. Сохранение промежуточных моделей полезно, если вы задаете большое количество эпох (`num_train_epochs`) или шагов (`max_train_steps`) обучения. Наиболее эффективное обучение модели происходит после 30 эпох, а после 50 происходит переобучение.

◆ `validation_prompts`

Содержит промпты, которые помогут проверить, как модель работает во время обучения. Для каждой эпохи в `validation_epochs` ваша модель будет генерировать изображения с промптами из `validation_prompts`, и вы сможете визуально проанализировать, как она обучается.

◆ `output_dir`

Локальный каталог, в котором будет сохранена модель.

◆ `push_to_hub`

Определяет, отправлять ли вашу модель в Hugging Face Hub после обучения.

Обучающий скрипт `train_text_to_image.py` имеет гораздо больше параметров и настроек. Модель Stable Diffusion XL также можно настроить аналогичным образом. После обучения вы можете запустить вывод модели.

Вывод модели

После тонкой настройки нашу модель уже можно использовать для вывода, как и обычную Stable Diffusion в *главе 5*, но на этот раз она обучена заново. Если у вас нет достаточной вычислительной мощности для обучения, можете попробовать в действии готовую модель, обученную на том же наборе данных телескопа Хаббл — *Supermaxman/hubble-diffusion-1*. Вы также можете поделиться своей моделью с другими, используя платформу Hugging Face. Ниже вы увидите результат тонкой настройки Stable Diffusion на наборе *esa-hubble* (рис. 7.2).

```
import torch
from diffusers import StableDiffusionPipeline

from genaibook.core import get_device

model_id = "Supermaxman/hubble-diffusion-1"
device = get_device()
```

```

pipe = StableDiffusionPipeline.from_pretrained(
    model_id, # your-hf-username/your-finetuned-model
    torch_dtype=torch.float16,
).to(device)

prompt = (
    "Hubble reveals a cosmic dance of binary stars: In this stunning new image "
    "from the Hubble Space Telescope, a pair of binary stars orbit each other "
    "in a mesmerizing ballet of gravity and light. The interaction between "
    "these two stellar partners causes them to shine brighter, offering "
    "astronomers crucial insights into the mechanics of dual-star systems."
)

pipe(prompt).images[0]

```



Рис. 7.2. Результат тонкой настройки модели Stable Diffusion на наборе снимков телескопа Хаббл

Поэкспериментировав с моделью, вы можете заметить, что она дает отличные генерации изображений, которые могли бы быть получены с телескопа Хаббл (или с любого другого телескопа). Однако стоит отметить, что модель стала хороша только в этом. Если вы попросите ее создать что-то другое, она выдаст либо что-то похожее на галактику, либо нечто невнятное. Это связано с тем, что при тонкой настройке с полной моделью происходит *катастрофическое забывание* (**Catastrophic forgetting**)³, из-за которого вся модель настраивается строго в заданном направлении. Кроме того, для обучения модели нам потребовалось довольно много изображений. Такие методы, как DreamBooth и LoRA, могут помочь преодолеть эти ограничения.

³ Это явление, при котором нейросеть, обучаясь решению новой задачи, утрачивает способность справляться с предыдущими. — *Нер.*

DreamBooth

DreamBooth (рис. 7.3) — это техника управления тонкой настройкой модели Stable Diffusion, впервые представленная в статье «*DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation*» в Google Research. Она работает за счет полной тонкой настройки сети UNet в модели Stable Diffusion с использованием нескольких выборочных изображений, связанных с *триггерным словом*. Чтобы сохранить предыдущие знания модели, DreamBooth также использует выборочные изображения, репрезентативные для типа объекта, который мы настраиваем, используя метод *Prior Preservation Loss (PPL)*⁴. Далее в этой главе на примере мы объясним, что для тонкой настройки генерации конкретных изображений, например с вашей собакой, следует использовать данные именно этой собаки, а также изображения других собак.

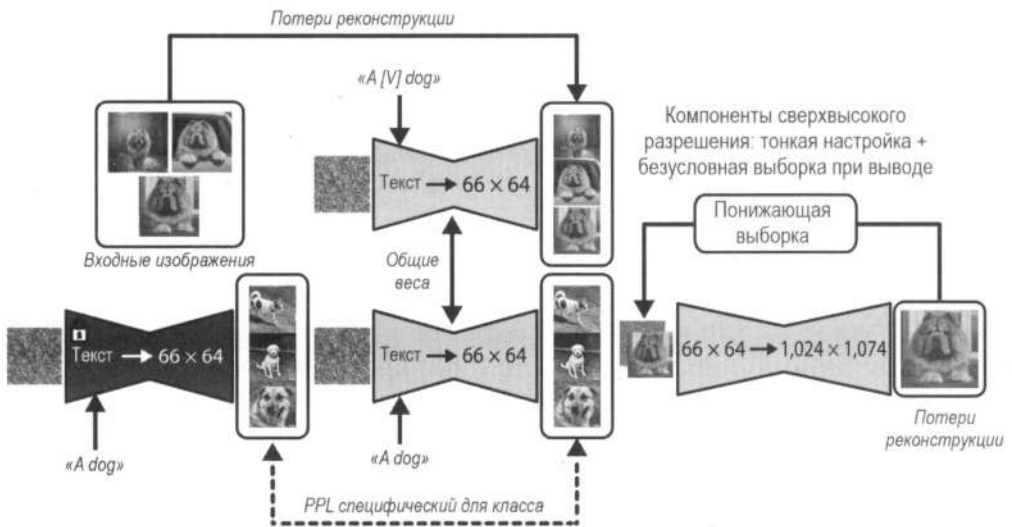


Рис. 7.3. Архитектурный поток DreamBooth

DreamBooth предлагает три интересных улучшения по сравнению с тонкой настройкой полных моделей типа text-to-image:

- ◆ *Настройка модели диффузии через обучение новому понятию с сохранением всех предыдущих знаний* (избегая катастрофическое забывание, упомянутое в предыдущем разделе). Как показано на рис. 7.3, DreamBooth настраивает конкретный уникальный токен или набор токенов на добавляемое понятие. Таким образом, если вы хотите включить объект «собака» («A dog») в модель, нужно обучить ее предложению *a [T] dog*, и каждый раз, когда вы будете упоминать *a [T] dog* в своей модели, DreamBooth сможет выполнять генерацию с конкретной собакой, пока сохраняются ее характеристики. Этого можно добиться, применив

⁴ *Prior preservation loss* — термин, который означает дополнительный член регуляризации в функции потерь для моделей диффузии, наподобие Stable Diffusion. Его цель — предотвратить переобучение новой модели, например при дообучении на небольшом наборе данных. — *Пер.*

семантические знания, которыми уже обладает модель (например, она знает, что такое собака), совместно с новой PPL, специфичной для объекта. Это позволяет модели сохранять имеющиеся знания при изучении нового понятия. Такая комбинация позволяет создавать объект в различных сценах, позах, ракурсах и условиях освещения, которые отсутствуют на исходных изображениях. Без уникального токена модель может смешивать новые знания, которым вы пытаетесь ее обучить, с уже существующей концепцией, связанной с этим токеном.

- ◆ *Настройка модели диффузии всего с 3–5 примерами.* Вместо использования 500+ изображений из полной тонкой настройки, модель может хорошо обучаться и обобщать данные даже при небольшом количестве примеров. Это возможно, поскольку модель может использовать свои внутренние знания для того же типа контента, который вы хотите настроить. Например, а [T] dog содержит слово «dog», которое затем используется во внутренних представлениях понятия «собака» («dog») в модели, чтобы настроить ее под конкретную собаку на вашем изображении. Меньшее количество примеров также приводит к быстрому обучению, поскольку модели нужно обрабатывать меньше данных.
- ◆ *Создание подписи для каждого изображения допустимо*, но необязательно, поскольку для DreamBooth достаточно использовать токен без подписей к изображениям.

DreamBooth в оригинальной статье был применен к модели диффузии Imagen от компании Google. Однако один из участников сообщества разработчиков ПО с открытым исходным кодом по имени Ксавье Сяо (Xavier Xiao) адаптировал этот метод к модели Stable Diffusion. С тех пор появилось множество реализаций, разработанных различными членами сообщества, например TheLastBen и khoya-ss. Библиотека `diffusers` также содержит обучающие скрипты DreamBooth.

В целом результаты, полученные сообществом с помощью метода проб и ошибок, а также с помощью децентрализованного опыта, свидетельствуют о следующем:

- ◆ Для изучения общего понятия касательно какого-либо объекта или стиля модели Stable Diffusion достаточно 3–5 изображений.
- ◆ Использование от 8 до 20 изображений наиболее эффективно для изучения уникальных стилей или редких объектов.
- ◆ PPL полезна для изучения лиц, но для других объектов или стилей может не потребоваться.
- ◆ Тонкая настройка текстового энкодера и сети UNet может дать хорошие результаты.
- ◆ Большинство этих идей были включены в обучающие скрипты ИИ-сообщества.

Мы будем использовать скрипт `diffusers/examples/dreambooth/train_dreambooth.py` из библиотеки `diffusers`.



DreamBooth — не первый метод настройки, выполняющий аналогичные задачи. Textual Inversion, представленный в статье «An Image is Worth One Word: Personalizing Text-to-Image Generation using Textual Inversion», демонстрирует, как можно помочь модели изучить новые эмбединги для текстового энкодера в Stable

Diffusion, чтобы он содержал новый объект. Этот метод актуален и по сей день, поскольку изучаемые эмбединги чаще всего небольшого размера (всего несколько килобайт). Однако существует компромисс между размером и качеством, и качество DreamBooth позволило выйти ему на первый план. Эксперименты с методами, сочетающими Textual Inversion и DreamBooth, которые носят название *Pivotal Tuning*⁵, также показали хорошие результаты. В DreamBooth необходимо найти уникальное *слово-триггер*. Однако, используя Textual Inversion, мы можем создавать слова-триггеры как новые токены, что приводит к более эффективному внедрению новых концепций.

Подготовка набора данных

Как правило, для изучения нового объекта или лица достаточно 5–20 примеров. Что касается стилей, если модель испытывает трудности с обучением в этой области, добавление дополнительных примеров может помочь. Поскольку DreamBooth не требует добавлять подписи к изображениям, вам достаточно просто сохранить обучающие изображения в выбранной папке, на которую можно ссылаться с помощью кода. Например, вы можете загрузить фотографии своего питомца.

В нашем примере мы обучим модель на примере лица одного из авторов книги. Если вы хотите повторить эти действия, можете обучить модель на своем собственном лице (или на лице питомца).

Сохранение предыдущих знаний

В DreamBooth при желании можно воспользоваться классом, который позволяет сохранять уже имеющиеся знания. Он работает за счет PPL во время обучения, чтобы модель понимала, что генерирует элементы того же класса. Например, при изучении вашего собственного лица наличие коллекции изображений с другими лицами поможет модели понять, что класс, которому вы пытаетесь ее обучить, — это лица. Поэтому, даже если вы предоставите всего несколько примеров, класс будет основан на изображениях лиц. Если у модели уже есть информация о классе, который вы собираетесь создать, можно самостоятельно сгенерировать обучающие изображения (в коде обучения есть флаг, позволяющий это сделать) или загрузить их в определенную папку.

Чтобы включить сохранение знаний, нужно настроить несколько параметров в скрипте обучения:

◆ `with_prior_preservation`

Определяет, использовать сохранение данных или нет. Если установлено значение `True`, модель будет генерировать изображения на основе параметров `class_prompt` и `num_class_images`. Если указать папку `class_data_dir`, изображения внутри нее будут использоваться в качестве изображений классов.

⁵ *Pivotal Tuning* — термин, который означает метод настройки генератора изображений с учетом латентного пространства. Он используется в задачах редактирования изображений на основе генеративных моделей (GAN) и семантических манипуляций. — Пер.

◆ `class_prompt`

При включенной настройке сохранения данных позволяет описывать класс сгенерированных изображений, которые будут использоваться для обучения (например, `the face of a Brazilian man`). Параметр не включает триггерное слово.

◆ `num_class_images`

Определяет количество генерируемых изображений в классе. Если указать папку `class_data_dir`, изображения внутри нее будут использоваться в качестве изображений класса. Если в папке меньше изображений, чем `num_class_images`, оставшиеся изображения будут сгенерированы с помощью `class_prompt`.

Подготовка модели с помощью DreamBooth

Как и при тонкой настройке полной модели, наиболее важными переменными являются `learning_rate` и `num_train_epochs`. Низкая скорость обучения с последующим медленным увеличением количества эпох или шагов обучения может стать хорошей отправной точкой для достижения высокого качества. Но есть и другой путь — фиксировать количество эпох или шагов обучения и увеличивать скорость обучения. Обе стратегии можно комбинировать для поиска оптимальных гиперпараметров.

Рассмотрим некоторые параметры, доступные только в DreamBooth:

◆ `instance_prompt`

Это промпт, с помощью которого модель будет пытаться изучить концепцию. Попробуйте найти редкий токен или их комбинацию для наименования вашего объекта и окружите его контекстом, например «в стиле mybtfuart», «фото plstps» или «игрушка sckpto». Редкая комбинация будет триггерным словом для конкретного экземпляра, который вы настраиваете.

◆ `train_text_encoder`

Определяет, обучать ли дополнительно текстовый энкодер или нет. Эта настройка может дать хорошие результаты, но будет потреблять больше видеопамати. Причина, по которой обучение текстового энкодера совместно с UNet может быть полезным, заключается в том, что вы также внедряете знания о новой концепции в текстовую интерпретацию промпта.

◆ `with_prior_preservation`

Определяет, использовать PPL или нет.

◆ `class_prompt` или `class_data_dir`

Содержит промпт для генерации изображений класса с целью сохранить знания. Они будут взяты из каталога `class_data_dir`, если он указан. В противном случае модель сгенерирует несколько изображений, используя промпт из переменной `class_prompt`.

◆ `prior_loss_weight`

Управляет влиянием PPL на модель.

Далее мы применим скрипт `train_dreambooth.py`, чтобы обучить модель на лице одного из авторов книги, как показано на рис. 7.4. Если вы хотите повторить это, можете обучить модель на своем собственном лице.

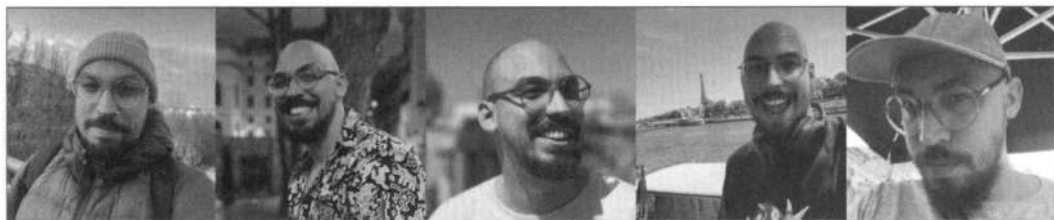


Рис. 7.4. Набор изображений лица Аполиналиро Пассоса (Apolinário Passos) для обучения с помощью DreamBooth

```
accelerate launch train_dreambooth.py \
--pretrained_model_name_or_path="stable-diffusion-v1-5/stable-diffusion-v1-5" \
--instance_data_dir="my-pictures" \
--instance_prompt="a photo of plstps" \ ❶
--resolution=512 \
--train_batch_size=1 \
--with_prior_preservation \
--class_prompt="an ultra realistic portrait of a man" \ ❷
--gradient_accumulation_steps=1 \
--train_text_encoder \
--learning_rate=5e-6 \
--num_train_epochs=100 \
--output_dir="myself-model" \
--push_to_hub
```

Немного поясним код:

- ❶ `plstps` является уникальным токеном, по которому модель будет изучать концепцию.
- ❷ Здесь вы можете добавить общее описание того, чему обучается модель.

После обучения модель можно использовать для генерации новых изображений. Рассмотрим, как это сделать.

Вывод

Несмотря на то что общая структура сохраняется, а изменяется только новый токен, основанный на `instance_prompt`, новая DreamBooth-модель по-прежнему является «весовым коэффициентом» для полной модели Stable Diffusion. Поэтому ее можно загрузить для вывода:

```
model_id = "your-hf-profile/your-custom-dreambooth-model"
pipe = StableDiffusionPipeline.from_pretrained(
    model_id,
```

```
torch_dtype=torch.float16,
).to(device)
# Сюда можно вставить ваш промпт и некоторые индивидуальные настройки
prompt = "a photo of plstps speaking on a microphone"
pipe(prompt).images[0]
```

На рис. 7.5 показаны некоторые результаты работы модели.



Рис. 7.5. Изображения, созданные моделью с помощью DreamBooth

Обучение LoRA

Касательно полной тонкой настройки и DreamBooth у нас возникла небольшая «звездка». Как только мы закончили настройку модели, мы получили новые веса, такие же большие, как и у исходной модели Stable Diffusion. Проблема состоит в том, что они не подходят для совместного использования, локального размещения, объединения моделей в стек, обслуживания моделей в облаке и других вариантов использования. Чтобы решить вопрос, мы можем использовать LoRA (рис. 7.6), как мы делали в главе 6 для LLM.

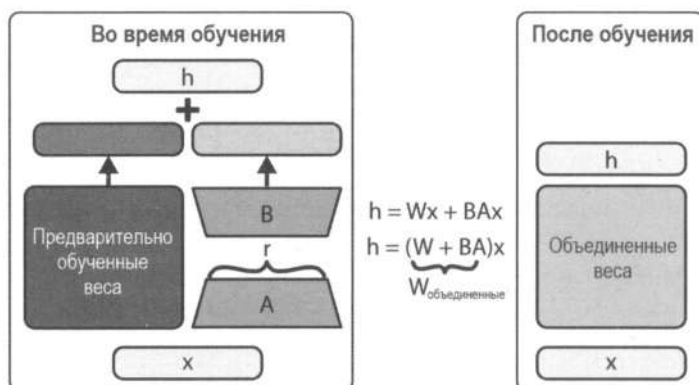


Рис. 7.6. Архитектурный поток LoRA

LoRA позволяет замораживать веса предварительно обученной модели и вводить *ранговую декомпозицию матриц (Rank decomposition matrix)*⁶, что значительно сокращает количество параметров для обучения. Обученные с помощью LoRA ранги также можно использовать в качестве артефактов, которые можно объединить в модель без дополнительной задержки вывода.

Звучит неплохо. Но мы находимся в главе, посвященной тонкой настройке моделей диффузии, а оригинальные LoRA были сосредоточены на трансформерах. Поэтому самое время рассмотреть модель *Stable Diffusion LoRA* от автора по имени Симо Рю (Simo Ryu). Понимание того, что ранги LoRA можно прикрепить к UNet и текстовому энкодеру точно так же, как их можно добавить к трансформерам LLM, раскрыло потенциал LoRA для моделей диффузии.

На помощь снова приходит библиотека *diffusers*, включающая в себя скрипт обучения LoRA, который служит для тонкой настройки как полной модели, так и DreamBooth. Обучение весов LoRA с помощью *diffusers*, по сути, то же самое, что и тонкая настройка полной модели или DreamBooth, но с некоторыми ключевыми отличиями:

- ◆ Формат набора данных тот же, что используется для тонкой настройки полной модели и DreamBooth.
- ◆ Скрипты обучения различаются, несмотря на то, что гиперпараметры остаются теми же. Мы будем использовать `examples/text_to_image/train_text_to_image_lora.py` и `examples/dreambooth/train_dreambooth_lora.py` из библиотеки *diffusers*⁷.
- ◆ Процесс вывода включает загрузку базовой модели в конвейер и последующее добавление адаптера LoRA. Этот подход удобен, поскольку позволяет быстро загружать различные адаптеры и переключаться между ними, сохраняя при этом одну и ту же базовую модель. Например, можно использовать предварительно обученную тонкую настройку, предоставленную другим пользователем (предварительно настроенные варианты можно найти на Hugging Face Hub). Шаги будут следующими: выбрать базовую модель (к которой будет подключен адаптер LoRA), загрузить конвейер из библиотеки *diffusers*, загрузить веса в модель и при необходимости объединить их для повышения эффективности и скорости.

Далее рассмотрим, как загрузить настроенную модель LoRA и выполнить с ее помощью вывод (рис. 7.7). Основные отличия заключаются в определении базовой модели и загрузке весов LoRA в модель.

```
from diffusers import DiffusionPipeline
from huggingface_hub import model_info
```

⁶ *Rank decomposition matrix* — термин, который означает метод анализа геометрии и топологии паттернов, сформированных активациями нейронов в нейронной сети. Цель — выявить низкоразмерные топологические структуры (например, петли или дыры) в высокоразмерном пространстве нейронных представлений. — Пер.

⁷ Обучающие скрипты LoRA, вроде Kohya, TheLastBen и Advanced LoRA Trainer, созданные на основе скриптов библиотеки *diffusers*, предлагают гораздо больше экспериментального функционала.

```
#Мы будем использовать классический мультяшный стиль, нарисованный от руки
lora_model_id = "alvdansen/littlelinies"

#Определяем базовую модель
#Данная информация часто содержится в карточке модели
#В нашем случае это "stabilityai/stable-diffusion-xl-base-1.0"
info = model_info(lora_model_id)
base_model_id = info.card_data.base_model

#Загружаем базовую модель
pipe = DiffusionPipeline.from_pretrained(
    base_model_id, torch_dtype=torch.float16
)
pipe = pipe.to(device)

#Добавляем LoRA в модель
pipe.load_lora_weights(lora_model_id)

#Объединяем LoRA с базовой моделью
pipe.fuse_lora()

image = pipe(
    "A llama drinking boba tea", num_inference_steps=25, guidance_scale=7.5
).images[0]

Image
```



Рис. 7.7. Результат работы LoRA совместно с базовой моделью

Предоставление для Stable Diffusion новых возможностей

Тонкая настройка для обучения модели новым стилям или темам — это само по себе удивительно. Но что, если бы мы могли использовать ее, чтобы дать Stable Diffusion больше возможностей, чем обычно? Используя специальные техники, можно наделить модель способностью к методу *inpainting*⁸ или включить дополнительные виды обусловливания.

Inpainting

Inpainting подразумевает маскирование определенной области изображения, которую требуется заменить. Этот процесс аналогичен маскированию изображения с помощью шума, но с той лишь разницей, что модель в этом случае добавляет шум только к замаскированной области, стремясь изменить или удалить ее с изображения, сохраняя остальную часть нетронутой.

Метод inpainting можно включить в предварительно обученную модель диффузии типа text-to-image. Для этого нужно добавить дополнительные входные каналы для UNet. В случае Stable Diffusion v1 было добавлено около 400 тыс. шагов за счет пяти обнуленных входных каналов для UNet: четырех для кодированного изображения с маской и одного для самой маски. В процессе обучения генерируются синтетические маски, которые маскируют 25% всего изображения. Поскольку у нас есть исходные данные изображения за маской, модель учится заполнять скрытые области с помощью промптов, становясь мощным инструментом для редактирования изображений. В более продвинутых моделях, таких как Stable Diffusion XL, некоторые возможности inpainting доступны сразу, без дополнительной настройки, что вызывает у некоторых людей вопрос о том, сможет ли специализированная, тонко настроенная модель улучшить этот потенциал. Однако на данный момент уже существуют специализированные модели SDXL, реализующие метод inpainting с некоторыми дополнительными возможностями, демонстрирующими потенциал этой техники даже в более крупных и сложных моделях. Несмотря на то что эта техника недоступна для обучения на домашнем оборудовании и требует полной тонкой настройки в сотни тысяч шагов, обычные тонко настроенные модели доступны всем. В следующей главе мы более подробно рассмотрим inpainting (с использованием кода).

Дополнительные входы для специальных обусловливаний

Подобно inpainting, мы можем добавлять в модель и другие возможности (с помощью добавления в UNet новых входных каналов). Одним из примеров является модель Stable Diffusion 2 Depth, созданная на основе модели `stable-diffusion-2-base` и

⁸ *Inpainting* (или *Image Inpainting*) — технология машинного обучения, предназначенная для восстановления поврежденных или закрытых участков изображений. Простыми словами, модель «дорисовывает» недостающие части, чтобы восстановить изображение, основываясь на окружающих областях.

тонко настроенная на 200 тыс. шагах с дополнительным входным каналом, который служит для обработки как промпта пользователя, так и изображения, содержащего монокулярное предсказание глубины (расстояние относительно камеры), полученное с помощью *MiDaS*. Пример представлен на рис. 7.8.



Водное изображение

Измерение глубины с помощью *MiDaS*

«плавающий меха-робот»

Рис. 7.8. Пример монокулярного предсказания глубины с помощью *MiDaS*

Несмотря на то что этот метод хорошо работает, тонкая настройка базовой модели на сотнях или тысячах шагов для получения новых обусловливаний ограничивает этот процесс из-за лицензионных прав некоторых компаний и лабораторий. Однако сейчас появились методы (например, *ControlNets*, *ControlLoras* и *T2I*), добавляющие к модели адаптеры, которые делают процесс обучения и вывода более эффективным. Подробнее мы рассмотрим приложения *text-to-image* в следующей главе.

Время проекта: самостоятельное обучение модели SDXL с помощью *DreamBooth* и *LoRA*

Тонкая настройка — отличный способ расширить знания о моделях диффузии типа *text-to-image*. *DreamBooth* позволяет проводить тонкую настройку всего на нескольких примерах, а обучение с помощью *LoRA* позволяет работать с небольшими моделями и снижать нагрузку на графический процессор по сравнению с тонкой настройкой всей модели. В этом проекте вы будете настраивать модель с использованием *DreamBooth* и *LoRA*. Освоив основы, вы сможете использовать более продвинутые скрипты из библиотеки *diffusers*. Если у вас в распоряжении нет видеокарты с объемом памяти не менее 16 ГБ, мы рекомендуем использовать *Google Colab* или *Hugging Face Spaces* для этого проекта.

Ваша цель — научиться добавлять новый, еще не существующий объект или стиль в модель *Stable Diffusion* и успешно генерировать с ее помощью новые изображения. Проект включает в себя два этапа.

1. Создание набора данных:

- Найдите объект или стиль, который вы хотите включить в модель. Это может быть уникальный предмет, которым вы владеете (например, деревянная игрушка для кошек), или стиль мебели/картин/ковров в вашем доме.

- Сделайте несколько снимков этих объектов с разных ракурсов и на разных фонах. Достаточно 3–8 снимков.
- Придумайте подпись к каждому изображению и используйте уникальный токен для описания вашего объекта (например, `cttoy`), наподобие:
 - «Фотография передней части `cttoy`, белый фон»
 - «Фотография боковой части `cttoy`, цветочный горшок на заднем плане»
- Загрузите набор данных в Hugging Face Datasets Hub или сохраните его в локальной папке.

2. Обучение модели:

- Откройте любой подходящий вам скрипт обучения (рекомендуемый или тот, который вы можете найти).
- Укажите в качестве папки с изображениями либо локальную папку, либо созданный вами набор данных на Hugging Face.
- Запустите обучение. Как известно, вы можете экспериментировать с `learning_rate`, `batch_size` и другими гиперпараметрами, пока не будете удовлетворены результатами своей модели. Чтобы узнать, как загрузить обученную модель LoRA в Stable Diffusion для тестирования, обратитесь к разделу «Обучение LoRA».
- С помощью параметра `validation_prompts` проверьте, как генерируются выборки во время обучения. После этого загрузите модель с помощью `load_lora_weights`, чтобы понять, как она обучалась.

Заключение

Обучение большой модели преобразования текста в изображение (text-to-image) с нуля требует значительных вычислительных ресурсов, поэтому тонкая настройка позволяет использовать компьютеры с одним GPU для настройки уже существующих моделей. В этой главе мы показали, как тонкая настройка моделей диффузии может привести к расширению знаний и настройке модели под конкретные задачи, сохраняя при этом ее общий набор знаний. Вы узнали, как выполнить полную тонкую настройку, использовать DreamBooth для изучения новых стилей и LoRA для повышения эффективности обучения. Вы также убедились, что тонкая настройка моделей диффузии может предоставить им новые возможности. В целом тонкая настройка — мощный инструмент.

В этой главе были рассмотрены методы тонкой настройки модели Stable Diffusion для обучения ее новым стилям, темам и возможностям. Начав с тонкой настройки полной модели, мы продвинулись до изменения поведения модели для генерации изображений в желаемом стиле или в рамках конкретной темы.

Также мы рассмотрели такие методы, как DreamBooth и LoRA, которые позволяют настраивать модель с меньшим количеством примеров и меньшим риском катаст-

рофического забывания. Мы обсудили потенциал тонкой настройки для добавления в модель новых возможностей, вроде inpainting и специальных обусловливающих, продвигающих полезность Stable Diffusion за пределы ее первоначальной конфигурации.

Для дополнительного чтения мы предлагаем ознакомиться со следующими материалами:

- ♦ *«How to Fine Tune Stable Diffusion: How We Made the Text-to-Pokemon Model at Lambda».*
- ♦ *«How I Train a LoRA: m3lt Style Training Overview».*
- ♦ *«Create an Infinite Icon Library by Fine-Tuning Stable Diffusion».*
- ♦ Расширенное руководство по обучению диффузии.
- ♦ Статья о DreamBooth.
- ♦ Подробное введение в LoRA на моделях диффузии.

Вопросы

1. Объясните основные различия между тонкой настройкой полной модели и DreamBooth.
2. Каковы преимущества использования LoRA по сравнению с тонкой настройкой полной модели с точки зрения вычислительных ресурсов и адаптивности модели?
3. Почему важно использовать уникальный токен при обучении с помощью DreamBooth?
4. Помимо обучения новым концепциям, тонкая настройка также позволяет добавить в модель новые возможности. Приведите два примера, которые модель может освоить, применяя методы тонкой настройки.
5. Обсудите, как выбор гиперпараметров влияет на результат тонкой настройки модели диффузии.
6. Опишите потенциальные риски тонкой настройки моделей text-to-image на смещенных наборах данных.

Вы можете найти ответы на эти вопросы и решение следующей задачи в репозитории этой книги на GitHub.

Задачи

Сравнение LoRA и полной тонкой настройки. Обучите модель с помощью DreamBooth с использованием LoRA и полной тонкой настройкой и сравните результаты. Попробуйте изменить гиперпараметр `rank` для LoRA, чтобы увидеть, насколько он влияет на результаты.

ЧАСТЬ III

Двигаемся дальше

Творческое применение моделей text-to-image

В этой главе представлены примеры, использующие модели text-to-image. Они расширяют возможности ИИ за пределы простого использования текста для управления генерацией. Сначала мы рассмотрим самые простые приложения, а затем перейдем к более сложным.

Преобразование изображения в изображение

Как нам уже известно, генеративные модели диффузии могут создавать изображения с помощью текста из полностью зашумленных данных. Но они также могут выполнять генерацию, используя в качестве ввода уже существующие изображения, а не чистый шум. То есть мы можем дополнить исходное изображение шумом и позволить модели частично его модифицировать. Этот процесс называется *преобразованием изображения в изображение (image-to-image)*, поскольку одно изображение преобразуется в другое в зависимости от степени его зашумленности и промпта.

Библиотека `diffusers` позволяет загрузить конвейер `image-to-image`, чтобы подключить необходимый класс. В качестве примера рассмотрим, как использовать SDXL для этой задачи, но с некоторыми отличиями:

- ◆ Будем использовать конвейер `StableDiffusionXLImg2ImgPipeline` вместо обычного `StableDiffusionXLPipeline`.
- ◆ Помимо промпта, будем передавать в конвейер исходное изображение.

Чтобы применить эти улучшения, в качестве базовой модели мы можем использовать либо `stabilityai/stable-diffusion-xl-base-1.0`, либо `stabilityai/stable-diffusion-xl-refiner-1.0`. Базовая модель необходима, если вы хотите стилизовать изображение или создать новый контекст на основе имеющегося. Если вы хотите улучшить или добавить детали изображения без значительных преобразований, вам может быть полезна модель рефайнера, специализирующаяся на проработке мелких деталей изображений.

```
import torch
from diffusers import StableDiffusionXLImg2ImgPipeline
from genaibook.core import get_device
```

```
device = get_device()
```

```
#Загружаем конвейер
img2img_pipeline = StableDiffusionXLImg2ImgPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-refiner-1.0",
    torch_dtype=torch.float16,
    variant="fp16",
)
```

Затем перенесем конвейер на наше устройство (обычно для GPU это `cuda`). Поскольку некоторые примеры могут требовать слишком много ресурсов, в качестве альтернативы можно использовать функцию `img2img_pipeline.enable_model_cpu_offload()`, которая переносит подмодули на GPU по мере необходимости. Это замедлит вывод, но позволит запустить модель на устройстве меньшей мощности.

```
#Перемещаем конвейер на устройство
#В качестве альтернативы используем img2img_pipeline.enable_model_cpu_offload()
img2img_pipeline.to(device)
```

Теперь, когда у нас есть конвейер, выполним пример.

```
from genaibook.core import SampleURL, load_image, image_grid
```

```
#Загружаем изображение
url = SampleURL.ToyAstronauts
init_image = load_image(url)
```

```
prompt = "Astronaut in a jungle, cold color palette, muted colors,
detailed, 8k"
```

```
#Пропускаем промт и изображение через конвейер
image = img2img_pipeline(prompt, image=init_image, strength=0.5).images[0]
image_grid([init_image, image], rows=1, cols=2)
```

Конвейер `StableDiffusionXLImg2ImgPipeline` принимает те же входные данные, что и обычный конвейер `Stable Diffusion`, который мы использовали до этого, а также два дополнительных параметра:

- ◆ `init_image`

Исходное изображение, которое нужно изменить.

- ◆ `strength`

Интенсивность шума, который добавляется к изображению. Значение 0 вернет исходные данные, поскольку шум не был добавлен. Значение 1 полностью зашумит изображение, как обычный конвейер `text-to-image`.

На рис. 8.1 показан эксперимент, где к одному и тому же изображению применены разные значения интенсивности от 0 до 1.

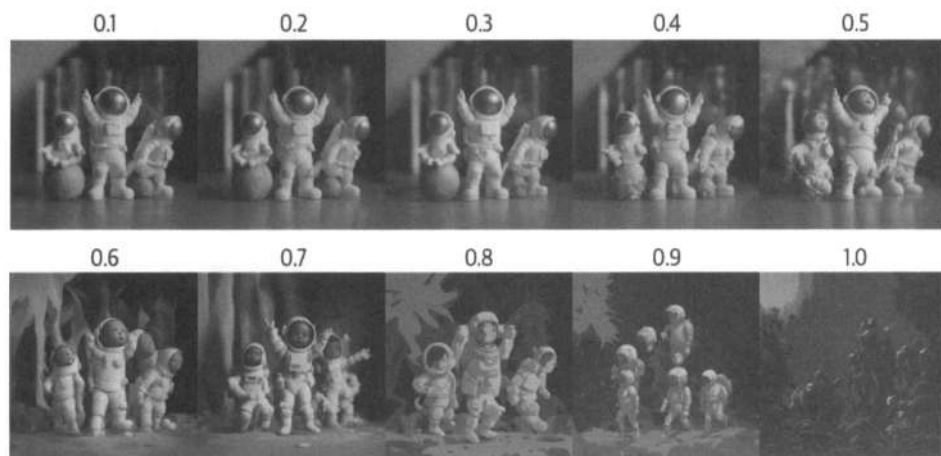


Рис. 8.1. Изменение степени подавления шума на изображении от 0,1 до 1,0 с помощью модели image-to-image

Inpainting

Inpainting — это процесс заполнения недостающих частей изображения на основе окружающего контекста. Как известно из предыдущей главы, для применения этого метода можно использовать как готовую модель, так и настроить модель text-to-image.

Прежде чем углубиться в особенности моделей диффузии для выполнения *inpainting*, стоит отметить различия между этим подходом и классическими методами обработки изображений. Традиционные варианты обычно основаны на анализе окружающих пикселей и использовании различных алгоритмов для заполнения замаскированной области на основе локальной статистики изображения или выборки на основе определенных фрагментов. Несмотря на то что они могут быть эффективны для сравнительно «простых» фонов или небольших областей, они часто становятся бесполезными при применении со сложными текстурами или в семантическом понимании самого содержимого изображения. В отличие от этого, *inpainting* обладает рядом преимуществ. Используя его, модели могут понимать и генерировать контент на основе визуального и семантического контекста, что позволяет добиваться более последовательных и креативных результатов. Однако методы машинного обучения, как правило, требуют больших вычислительных ресурсов.

Рассмотрим, как выполнять *inpainting*, а также какие творческие приложения можно реализовать с помощью этой техники. Как и прежде, для этого мы будем использовать конвейер `StableDiffusionXLInpaintPipeline`.

```
from diffusers import StableDiffusionXLInpaintPipeline
```

```
#Загружаем конвейер
```

```
inpaint_pipeline = StableDiffusionXLInpaintPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0",
```

```

    torch_dtype=torch.float16,
    variant="fp16",
).to(device)

img_url = SampleURL.DogBenchImage
mask_url = SampleURL.DogBenchMask

init_image = load_image(img_url).convert("RGB").resize((1024, 1024))
mask_image = load_image(mask_url).convert("RGB").resize((1024, 1024))

#Пропускаем изображение и промт через конвейер
prompt = "A majestic tiger sitting on a bench"
image = inpaint_pipeline(
    prompt=prompt,
    image=init_image,
    mask_image=mask_image,
    num_inference_steps=50,
    strength=0.80,
    width=init_image.size[0],
    height=init_image.size[1],
).images[0]

```

В полученной сетке (рис. 8.2) левая панель обозначает исходное изображение, средняя — маску, а правая панель — выходное изображение.

```
image_grid([init_image, mask_image, image], rows=1, cols=3)
```

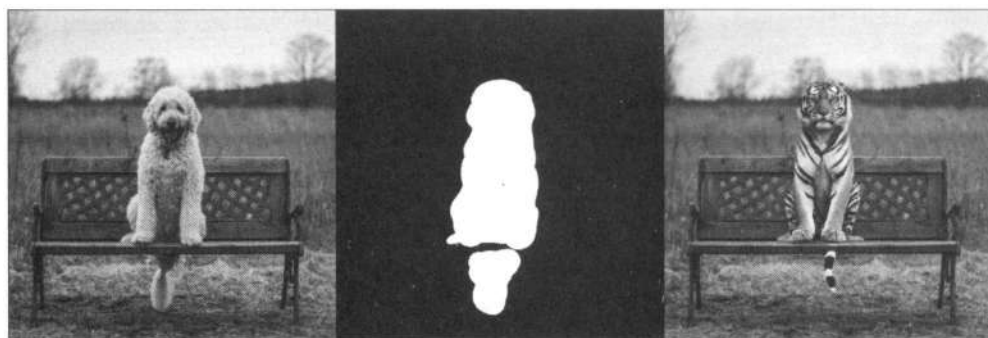


Рис. 8.2. Пример выполнения inpainting: в результате левая панель обозначает исходное изображение, средняя — маску, а правая — выходное изображение

Ниже приведены некоторые из наиболее важных параметров, которые принимает конвейер `StableDiffusionXLInpaint`:

◆ `init_image`

Изображение, над которым будет выполняться преобразование.

◆ `mask_image`

Двоичное цветное изображение маски. Черный цвет обозначает места, которые должны остаться неизменными, а белый — области, где изображение будет заменено при генерации.

◆ Strength

Количество шума, которое добавляется к маске. Параметр аналогичен параметру `strength` для моделей *image-to-image*, но применяется только к замаскированной области. Значение 0 вернет изображение без шума. Значение 1 полностью зашумит замаскированную область, но это не лучший вариант. Поэкспериментируйте со значениями от 0,6 до 0,8.

Помимо базовых моделей диффузии, есть и другие, которые обладают функционалом *inpainting*, например `diffusers/stablediffusion-xl-inpainting`. Они были специально тонко настроены для данной задачи, что позволяет нам использовать более высокое значение `strength` при выводе.

Взвешивание промптов и редактирование изображений

Модели диффузии используют механизмы внимания, подобные тем, что есть в трансформерах. Они позволяют гибко фокусироваться на наиболее важных частях входных данных. В частности, перекрестное внимание используется для обусловливания трансформеров внутри слоев UNet с помощью текстовых промптов, чтобы формировать изображения.

Однако в некоторых случаях требуется больший контроль над генерацией изображения. Например, нам может потребоваться следующее:

- ◆ Обновить вес каждого слова в промпте, изменив масштаб эмбеддингов текста.
- ◆ Объединить несколько промптов для генерации изображения.
- ◆ Изменить генерации, сохраняя структуру для редактирования изображения.

Для этого в статье *«Prompt-to-Prompt Image Editing with Cross Attention Control»*, посвященной методу *Prompt-to-Prompt* для редактирования изображений, была предложена идея модифицировать диффузию, чтобы достигнуть управляемости за счет изменения и контроля перекрестного внимания. В этой публикации также был представлен неофициальный релиз библиотеки `diffusers`.

Помимо *Prompt-to-Prompt*, существуют и другие методы редактирования сгенерированных изображений, например *Attend-and-Excite* и *Semantic Guidance* (с официальными релизами в библиотеке `diffusers`). В этой главе мы подробнее рассмотрим редактирование с помощью *Semantic Guidance*, поскольку эта техника обеспечивает баланс между управляемостью и качеством.

Взвешивание и слияние промптов

Библиотека *Compel* реализует ключевые аспекты взвешивания и слияния промптов и легко сочетается с библиотекой `diffusers`. Она работает с помощью предварительной обработки строк и улучшения соответствующих эмбеддингов в пространстве эмбеддингов модели CLIP. Поскольку *Stable Diffusion XL* использует два текстовых энкодера, это усложняет процесс взвешивания промптов. Такая сложность возникает из-за необходимости согласовывать веса между обоими энкодерами, а

выходные данные второго энкодера к тому же необходимо *объединять в пул*¹. Библиотека Compel абстрагирована от этой сложности.

Есть два простых способа управлять промптом:

- ◆ Увеличить или уменьшить вес с помощью знаков «+» и «-» после слова, чтобы сделать его более заметным на изображении, изменив масштаб текстового эмбединга. Чем больше знаков вы добавите, тем сильнее вы увеличите или уменьшите вес слова. Несмотря на то что мы можем сильно уменьшить заметность слова, это не всегда приводит к полному удалению объекта с изображения.
- ◆ Объединить два промпта (заклучив их в скобки) и затем указать вес для каждого из них.

Рассмотрим код. Как обычно, начнем с загрузки конвейера.

```
from diffusers import DiffusionPipeline

pipeline = DiffusionPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0",
    torch_dtype=torch.float16,
    variant="fp16",
).to(device)
```

Далее инициализируем класс Compel, который требует предоставить ему токенизатор и энкодеры текста из модели диффузии. Класс также требует указать, какие текстовые эмбединги будут объединены в пул.

```
from compel import Compel, ReturnedEmbeddingsType

#Используем предпоследний слой CLIP, т. к. он более выразителен
embeddings_type = (
    ReturnedEmbeddingsType.PENULTIMATE_HIDDEN_STATES_NON_NORMALIZED
)

compel = Compel(
    tokenizer=[pipeline.tokenizer, pipeline.tokenizer_2],
    text_encoder=[pipeline.text_encoder, pipeline.text_encoder_2],
    returned_embeddings_type=embeddings_type,
    requires_pooled=[False, True],
)
```

Теперь сгенерируем изображения (рис. 8.3) с помощью различных промптов, усиленных классом compel.

```
from genaibook.core import image_grid

#Подготавливаем промпты
prompts = []
```

¹ Объединение в пул подразумевает преобразование эмбедингов для токенов в единый эмбединг фиксированной длины, отражающий всю последовательность, так же, как мы это делали с эмбедингами для предложений.

```

prompts.append("a humanoid robot eating pasta")
prompts.append(
    "a humanoid+++ robot eating pasta"
) #Делаем гуманоидные характеристики робота более выраженными
prompts.append(
    '["a humanoid robot eating pasta", "a van gogh painting"].and(0.8, 0.2)'
) #Делаем стиль похожим на стиль Ван Гога

images = []
for prompt in prompts:
    #Используем одно и то же начальное значение для всех поколений
    generator = torch.Generator(device=device).manual_seed(1)

    #Библиотека compel возвращает как векторы обусловливания,
    #так и эмбединги промпта, объединенные в пул
    conditioning, pooled = compel(prompt)

    #Мы передаем обусловленные и объединенные эмбединги промпта в конвейер
    image = pipeline(
        prompt_embeds=conditioning,
        pooled_prompt_embeds=pooled,
        num_inference_steps=30,
        generator=generator,
    ).images[0]
    images.append(image)
image_grid(images, rows=1, cols=3)

```

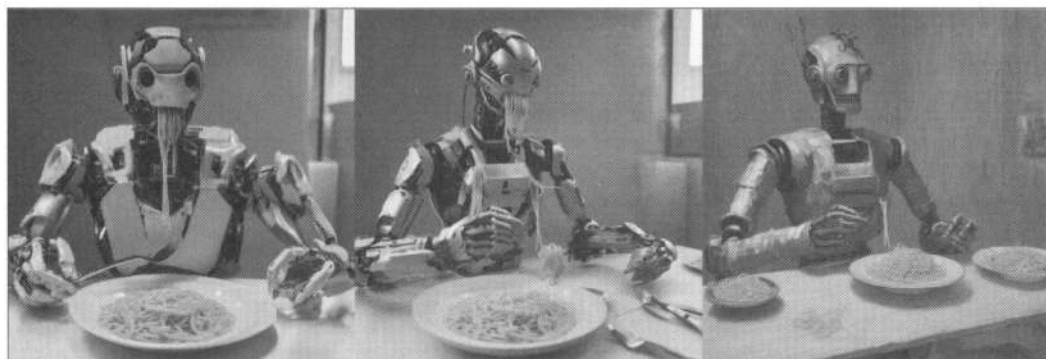


Рис. 8.3. Результат работы модели по промпту `a humanoid+++ robot eating pasta`

Знак «+» эквивалентен умножению веса слова на коэффициент 1,1, а «-» — на 0,9. Помимо использования этих знаков, также можно взвешивать токены следующим образом: `a robot eating (pasta)1.2`. Дополнительную информацию о библиотеке `Compel` можно найти в ее официальном справочном руководстве.

Редактирование диффузионных изображений с помощью Semantic Guidance

Как уже упоминалось, существует множество методов редактирования изображений, созданных с помощью диффузии. В то время как управление перекрестным вниманием с помощью метода Prompt-to-Prompt является популярным выбором, *Semantic Guidance* (SEGA) обеспечивает более детальный контроль и точное редактирование.

SEGA манипулирует оценками шума модели на каждом этапе процесса обратной диффузии. Такая динамическая корректировка позволяет выполнять семантическое редактирование в латентном пространстве на основе текстовых описаний. Динамически корректируя прогнозируемый шум, SEGA направляет изменения в семантическую сторону, полученную из текстовых эмбеддингов. Метод вычисляет градиенты текстовых эмбеддингов относительно латентного пространства, эффективно направляя генерацию или модификацию изображения к желаемым результатам. Этот процесс осуществляется без необходимости переобучать или модифицировать исходную архитектуру модели, что позволяет вносить динамические и направленные изменения исключительно на основе текстовых входных данных.

Продemonстрируем работу конвейера `SemanticStableDiffusionPipeline` и сгенерируем изображение с лицом мужчины, как показано на рис. 8.4 (a photo of the face of a man).

```
from diffusers import SemanticStableDiffusionPipeline

semantic_pipeline = SemanticStableDiffusionPipeline.from_pretrained(
    "CompVis/stable-diffusion-v1-4", torch_dtype=torch.float16, variant="fp16"
).to(device)

generator = torch.Generator(device=device).manual_seed(100)
out = semantic_pipeline(
    prompt="a photo of the face of a man",
    negative_prompt="low quality, deformed",
    generator=generator,
)
out.images[0]
```

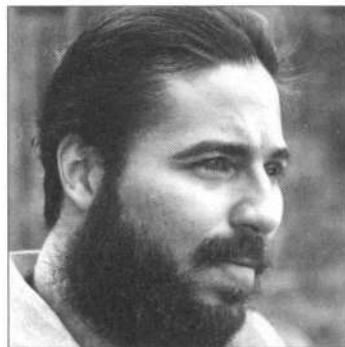


Рис. 8.4. Результат работы конвейера `SemanticStableDiffusionPipeline` по промπτу a photo of the face of a man

Как вы можете увидеть в последующих примерах, SEGA содержит несколько важных параметров для управления возможностями редактирования:

◆ `edit_guidance_scale`

Определяет, насколько сильно модель должна следовать правкам (изменениям).

◆ `edit_warmup_steps`

Определяет, сколько шагов шумоподавления должно быть в модели перед применением SEGA.

◆ `edit_threshold`

Определяет, какой процент пикселей исходного изображения должен быть сохранен.

◆ `reverse_editing_direction`

Определяет, должно ли редактирование добавлять (`False`) или удалять (`True`) какой-либо объект.

Попробуем направить промпт с помощью параметра `editing_prompt`, чтобы заставить человека на изображении улыбнуться, добавив слова «smiling» и «smile» (рис. 8.5).

```
generator = torch.Generator(device=device).manual_seed(100)
out = semantic_pipeline(
    prompt="a photo of the face of a man",
    negative_prompt="low quality, deformed",
    editing_prompt="smiling, smile",
    edit_guidance_scale=4
    edit_warmup_steps=10,
    edit_threshold=0.99,
    edit_momentum_scale=0.3,
    edit_mom_beta=0.6,
    reverse_editing_direction=False,
)
out.images[0]
```

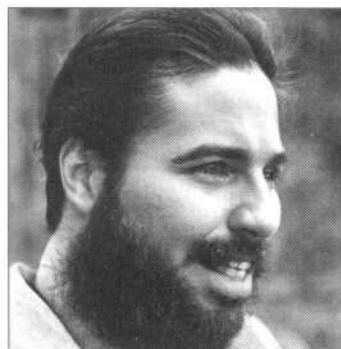


Рис. 8.5. Теперь человек на фото улыбается

Еще раз отредактируем фото. На этот раз мужчина будет носить очки (рис. 8.6).

```
generator = torch.Generator(device=device).manual_seed(100)
out = semantic_pipeline(
    prompt="a photo of the face of a man",
    negative_prompt="low quality, deformed",
    editing_prompt="glasses, wearing glasses",
    reverse_editing_direction=False,
    edit_warmup_steps=10,
    edit_guidance_scale=4,
    edit_threshold=0.99,
    edit_momentum_scale=0.3,
    edit_mom_beta=0.6,
    generator=generator,
)
out.images[0]
```



Рис. 8.6. Теперь мужчина на фото носит очки

Попробуем выполнить несколько правок одновременно (рис. 8.7). Обратите внимание на код ниже: параметр `editing_prompt` теперь представляет собой список промптов. Ключевые параметры (`edit_warmup_steps` и др.) также должны быть представлены списками.

```
generator = torch.Generator(device=device).manual_seed(100)
out = semantic_pipeline(
    prompt="a photo of the face of a man",
    negative_prompt="low quality, deformed",
    editing_prompt=[
        "smiling, smile",
        "glasses, wearing glasses",
    ],
    reverse_editing_direction=[False, False],
    edit_warmup_steps=[10, 10],
    edit_guidance_scale=[6, 6],
    edit_threshold=[0.99, 0.99],
    edit_momentum_scale=0.3,
```

```

edit_mom_beta=0.6,
generator=generator,
)
out.images[0]

```

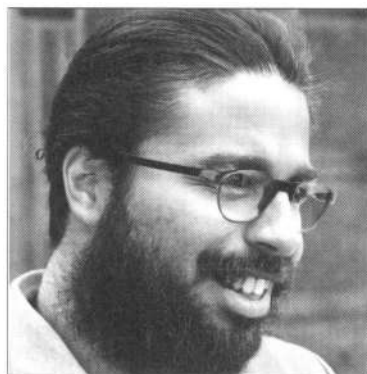


Рис. 8.7. Мы выполнили несколько правок изображения одновременно

Редактирование реальных изображений с помощью инверсии

Инверсия (Inversion) — это метод переноса реального изображения обратно в латентное пространство предварительно обученной генеративной модели. Метод показал многообещающие результаты в сетях GAN и был успешно реализован в моделях управляемой диффузии.

Инверсия позволяет редактировать реальные фото. Для этого можно было бы использовать технику image-to-image, представленную ранее, однако она дает очень ограниченные возможности и не всегда приводит к ожидаемым результатам, поскольку вывод может существенно меняться без какого-либо контроля. Для решения этой проблемы можно объединить технику диффузионного редактирования изображений (например, Prompt-to-Prompt или SEGA) с методом инверсии, обеспечив тем самым более детальный результат.

Процесс инверсии в моделях управляемой диффузии предполагает использование *инверсного графика*. Первым представленным экземпляром стал *DDIMInverseScheduler* — шумоподаватель с инверсией, который может прогнозировать выборки из временных шагов в латентное пространство, выполняя DDIM-выборку в обратном порядке (начиная с реального изображения и постепенно добавляя к нему шум). После подавления шума на выходе мы получаем исходное изображение, как и ожидали. Сам по себе этот процесс не очень интересен: мы просто использовали шумные латентные компоненты для реконструкции уже имеющегося изображения. Однако инверсия может стать мощным инструментом, если нужно внести изменения в изображение.

Самый простой способ изменить изображение с помощью инверсии — это выполнить следующий алгоритм:

1. Получить *DDIM-инверсию* реального промпта и описания (например, «Фотография лошади в поле»).
2. Изменить промпт на целевое изображение (например, «Фотография зебры в поле»).
3. Реконструировать изображение с помощью измененного промпта.

Результаты показаны на рис. 8.8.



Рис. 8.8. Примеры редактирования с помощью техники *DDIM-инверсии* (изображения адаптированы из статьи «An Edit Friendly DDPM Noise Space: Inversion and Manipulations»)

Несмотря на то что результаты получились впечатляющими, они не подходят для редактирования реальных изображений. Более продвинутые методы, например улучшенная *DDPM-инверсия*, способны обеспечить более качественные реконструкции, но они все еще не могут предоставить наиболее широкий спектр возможностей.

Чтобы обеспечить наиболее широкий функционал для модификации изображений, необходимо комбинировать метод инверсии с методами редактирования. Например, в статье «Null-text Inversion for Editing Real Images using Guided Diffusion Models» и технике *LEDITS++* используются методы *Prompt-to-Prompt* и *SEGA* соответственно.

Эти техники задействуют методы, рассмотренные ранее, но вместо того, чтобы выполнять редактирование в латентном пространстве с изображением, которое было бы сгенерировано с помощью промпта, они делают это в латентном пространстве реконструкции изображения после его инвертирования. Далее мы рассмотрим, как выполнять редактирование реальных изображений с помощью *LEDITS++*, используя уже изученный метод *SEGA* совместно с инверсией.

Редактирование с помощью *LEDITS++*

LEDITS++ объединяет два только что изученных вами метода: *SEGA* и инверсию. Кроме того, он реализует метод, позволяющий обосновать редактирование с помощью масок перекрестного внимания и шума, создаваемых моделью. Такая комбинация позволяет редактировать реальные изображения с теми же параметрами, которые мы изучили для *SEGA*.

Алгоритм работы *LEDITS++*:

1. Применяем инверсию, чтобы преобразовать интересующее нас изображение в формат, который может обработать модель.

2. Определяем список редактирования с помощью `editing_prompt` и направление редактирования (добавлять или удалять объект) с помощью `reverse_editing_direction`.
3. Применяем те же параметры `edit_guidance_scale` и `edit_threshold`, которые мы изучили в SEGA, для наших правок.

Попробуем применить конвейер LEDITS++ на практике (результат представлен на рис. 8.9).

```
from diffusers import LEditsPPPipelineStableDiffusion

#Загружаем модель как обычно
pipe = LEditsPPPipelineStableDiffusion.from_pretrained(
    "stable-diffusion-v1-5/stable-diffusion-v1-5",
    torch_dtype=torch.float16,
    variant="fp16"
)
pipe.to(device)

image = load_image(SampleURL.ManInGlasses).convert("RGB")
#Инвертируем изображение, постепенно добавляя к нему шум,
#чтобы его можно было устранить с помощью измененных направлений,
#фактически осуществляя редактирование
pipe.invert(image=image, num_inversion_steps=50, skip=0.2)

#Редактируем изображение в помощь промпта
edited_image = pipe(
    editing_prompt=["glasses"],
    # Сообщаем модели, что нужно удалить очки (изменяем направление)
    reverse_editing_direction=[True],
    edit_guidance_scale=[1.5],
    edit_threshold=[0.95],
).images[0]

image_grid([image, edited_image], rows=1, cols=2)
```



Рис. 8.9. Модель убрала очки на изображении

Редактирование реальных изображений с помощью тонкой настройки через инструкции

Другой способ обеспечить редактирование изображений для моделей диффузии — это тонко настроить их исключительно под эту задачу. Такой подход впервые был предложен в статье, посвященной модели *InstructPix2Pix*. В нем для обучения использовался набор данных с инструкциями-правками, содержащими исходное изображение, инструкции для редактирования и измененное изображение.

Затем к модели *Stable Diffusion* были добавлены дополнительные входные каналы к первому сверточному слою, что позволило ей принимать входные изображения. Модель обучалась с использованием того же механизма обработки текста, который был предусмотрен для подписей в исходной модели *Stable Diffusion*, модифицированной для использования инструкций-правок в качестве промпта (рис. 8.10).



Входное изображение



«Добавь клоунский грим на лицо»



«Преврати ее в бородатого мужчину»



«Сделай ей широкую улыбку»



«Преврати ее в зомби»



«Сделай серьгу более нарядной»

Рис. 8.10. Редактирование с использованием техники *InstructPix2Pix*

После публикации оригинальной статьи появились новые обученные и улучшенные модели *InstructPix2Pix*. Наиболее известной из них является *CosXL Edit* от компании *StabilityAI*, обученная на экземпляре *Stable Diffusion XL* для выполнения высококачественного редактирования.

Репозиторий модели CosXL является закрытым, поэтому вам необходимо посетить ее страницу на Hugging Face, ознакомиться с лицензией и принять условия, если вы согласны с ними. Или вы можете выполнить команду `huggingfacecli login` в терминальном сеансе для входа. Вам будет предложено ввести токен доступа, который можно создать на странице настроек. Если загрузить модель из сеанса Google Colab, то можно настроить секретный токен `hf_token` или переменную окружения и предоставить своему компьютеру разрешение на ее использование. Ниже на рис. 8.11 приведен пример редактирования, выполненный с помощью CosXL.

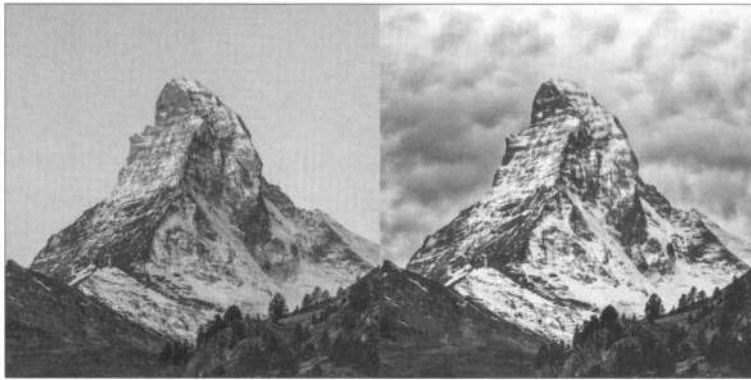


Рис. 8.11. Пример редактирования, выполненный с помощью CosXL

```
from diffusers import (
    EDMERulerScheduler,
    StableDiffusionXLInstructPix2PixPipeline,
)
from huggingface_hub import hf_hub_download

edit_file = hf_hub_download(
    repo_id="stabilityai/cosxl", filename="cosxl_edit.safetensors"
)

#Параметр from_single_file позволяет загрузить модель диффузии из единого
#файла библиотеки diffusers
pipe_edit = StableDiffusionXLInstructPix2PixPipeline.from_single_file(
    edit_file, num_in_channels=8, is_cosxl_edit=True, torch_dtype=torch.float16
)

#Модель была обучена таким образом, чтобы EDMERulerScheduler
#имел корректный график шума при шумоподавлении
pipe_edit.scheduler = EDMERulerScheduler(
    sigma_min=0.002,
    sigma_max=120.0,
    sigma_data=1.0,
    prediction_type="v_prediction",
    sigma_schedule="exponential",
)
```

```
pipe_edit.to(device)

prompt = "make it a cloudy day"
image = load_image(SampleURL.Mountain)
edited_image = pipe_edit(
    prompt=prompt, image=image, num_inference_steps=20
).images[0]

image_grid([image, edited_image], rows=1, cols=2)
```

ControlNet

ControlNet — это модель, которая служит для управления моделями диффузии путем задания дополнительных условий помимо текстового промпта. В отличие от прямой тонкой настройки, ControlNet полностью сохраняет исходную модель и добавляет новые условия в ее обучаемую копию. Это позволяет сохранить возможности модели, даже если ваша ControlNet обучена на относительно небольшом количестве выборок.

Эти модели обучены учитывать различные условия. Два из них показаны на рис. 8.12: четкие границы (canny edges) и человеческая открытая поза (human pose или OpenPose). Помимо этого, существуют также карты глубины (depth maps), каракули (scribble), сегментация (segmentation), линейный рисунок (lineart) и многое другое. Ознакомиться со всеми официальными моделями ControlNet для Stable Diffusion v1-5 можно на сайте Hugging Face.

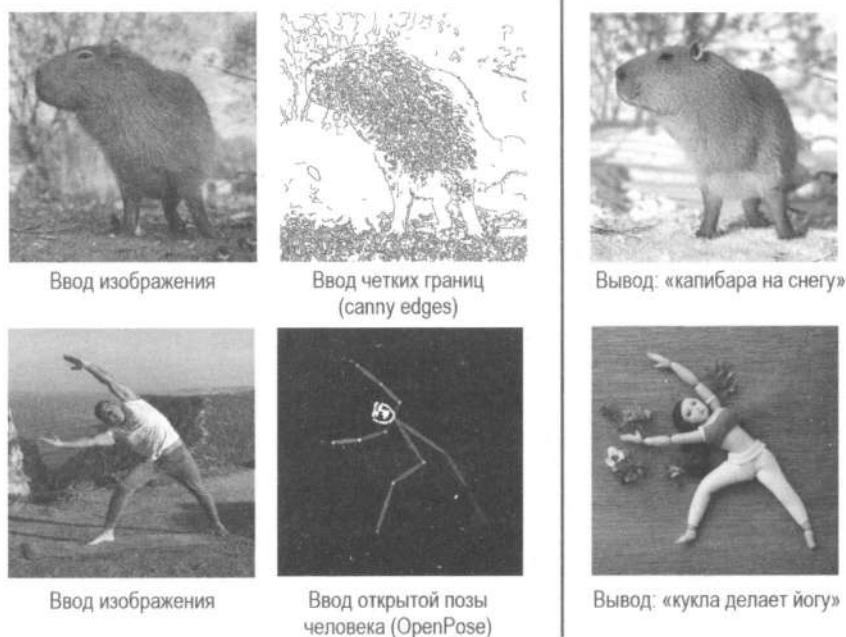


Рис. 8.12. Примеры ControlNet с входными изображениями (с применением «четких границ» и «открытых поз»), а также сгенерированный вывод

Универсальность и эффективность ControlNet делают ее мощным инструментом для различных задач в области генерации и обработки изображений. Обеспечивая детальный контроль над выходными данными с сохранением возможностей исходной модели, она открывает новые возможности для творческого и практического применения, как показано на рис. 8.13. Например, модель можно использовать в таких областях, как дизайн одежды (чтобы визуализировать наряд в различных позах), в архитектуре (чтобы создавать проекты зданий на основе черновых набросков) или в процессе подготовки к съемкам (чтобы быстро создать раскадровки на основе простых линейных рисунков). Возможность использовать различные типы управления входом вроде границ, оценок поз или карт глубины предоставляет разработчикам гибкую структуру, которая позволяет управлять процессом генерации изображений в соответствии с их конкретными потребностями.



Рис. 8.13. Пример ControlNet с четкими границами на изображении утки

Рассмотрим пример, где мы будем использовать вспомогательную библиотеку `controlnet_aux`, чтобы предварительно обработать входные изображения в желаемом формате в качестве условия для официальных предварительно обученных моделей ControlNet. Авторы библиотеки `diffusers` также обучили ControlNet для Stable Diffusion XL. Вы можете найти эти модели на Hugging Face.

Сначала загрузим основную модель с помощью конвейера `StableDiffusionXLControlNetPipeline`. Он также ожидает параметр `ControlNetModel` вместе с моделью, которая обеспечивает дополнительную обработку UNet во время шумоподавления.

```
from diffusers import ControlNetModel, StableDiffusionXLControlNetPipeline
```

```
controlnet = ControlNetModel.from_pretrained(
    "diffusers/controlnet-depth-sdxl-1.0",
    torch_dtype=torch.float16,
    variant="fp16",
)
```

```
controlnet_pipeline = StableDiffusionXLControlNetPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0",
    controlnet=controlnet,
```

```

torch_dtype=torch.float16,
variant="fp16",
)
#Опционально, экономит видеопамять
controlnet_pipeline.enable_model_cpu_offload()
controlnet_pipeline.to(device)

```

Затем воспользуемся функционалом `controlnet_aux` для предварительной обработки изображения в желаемом формате. В данном случае мы будем применять детектор глубины `MidasDetector`. Он принимает на вход изображение и выводит предполагаемую карту глубины, которая как раз и нужна для нашей модели `diffusers/controlnet-depthsdxl-1.0`.

```

from controlnet_aux import MidasDetector
from PIL import Image

original_image = load_image(SampleURL.WomanSpeaking)
original_image = original_image.resize((1024, 1024))

#загружаем модель детектора глубины MiDAS
midas = MidasDetector.from_pretrained("llyasviel/Annotators")

#Применяем определение глубины с помощью MiDAS
processed_image_midas = midas(original_image).resize(
    (1024, 1024), Image.BICUBIC
)

```

Теперь можно передать запрос и обработанное изображение в конвейер, чтобы сгенерировать новое изображение (рис. 8.14). Параметр `controlnet_conditioning_scale` определяет, насколько сильно условие будет влиять на конечный результат.

```

image = controlnet_pipeline(
    "A colorful, ultra-realistic masked super hero singing a song",
    image=processed_image_midas,
    controlnet_conditioning_scale=0.4,
    num_inference_steps=30,
).images[0]
image_grid([original_image, processed_image_midas, image], rows=1, cols=3)

```

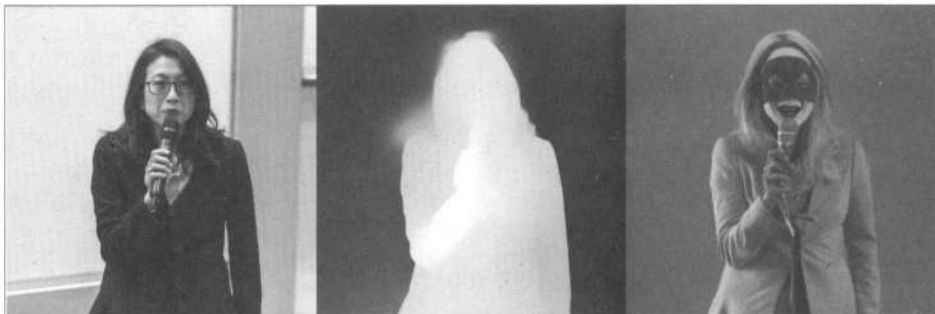


Рис. 8.14. Пример ControlNet: мы выполнили определение глубины с помощью MiDAS

Обучение моделей ControlNet выходит за рамки этой книги, поэтому если вас интересует эта тема, рекомендуем ознакомиться со статьей «*Train Your ControlNet with diffusers*» в блоге Hugging Face.

Изображения в качестве промптов и вариации изображений

Текстовые промпты — отличный инструмент, но иногда нам может потребоваться нечто большее, чтобы выразить намерение, которое мы хотим от модели. Использование изображений в качестве промптов в моделях диффузии позволяет расширить диапазон входных данных до визуальной области.

Вариации изображений

Чтобы раскрыть творческий потенциал чего-либо, иногда нужно взглянуть на это под другим углом. Именно в этом и заключается суть вариаций: взять заданное изображение и реинтерпретировать («переосмыслить») его, создав похожий, но в то же время отличающийся его вариант. Рассмотрим два подхода — использование эмбедингов изображений из модели CLIP и использование IP-адаптеров:

♦ Использование эмбедингов изображений из модели CLIP

Как известно из главы 5, Stable Diffusion использует CLIP в качестве текстового энкодера. Помимо этого, модель также может быть применена для создания эмбедингов изображений. Некоторые модели диффузии обучены таким образом, чтобы они могли использовать эмбединги изображений в качестве входных данных для генерации новых изображений (например, модели Karlo и Kandinsky). В Stable Diffusion это не работает «из коробки». Однако данный функционал можно реализовать с помощью тонкой настройки. Stable Diffusion Image Variations — это тонко настроенная модель Stable Diffusion v1-5, которая принимает эмбединги изображений из модели CLIP в качестве входных данных. Вы можете попробовать ее демоверсию на Hugging Face.

♦ Использование IP-адаптеров

Иной подход, не требующий тонкой настройки, заключается в использовании предварительно обученных IP-адаптеров (**Image Prompt Adapter**). Они позволяют использовать изображения в качестве промптов, допуская их вариации и широкий спектр других способов использовать изображения в качестве промптов, например перенос стиля, сохранение идентичности субъекта и управление структурой.

Как показано на рис. 8.15, IP-адаптер состоит из двух компонентов: энкодера, который извлекает признаки изображения, и разделенных модулей перекрестного внимания, которые прикрепляются к предварительно обученной сети UNet модели Stable Diffusion.

Использование IP-адаптера требует минимальных изменений по сравнению с базовым конвейером SDXL:

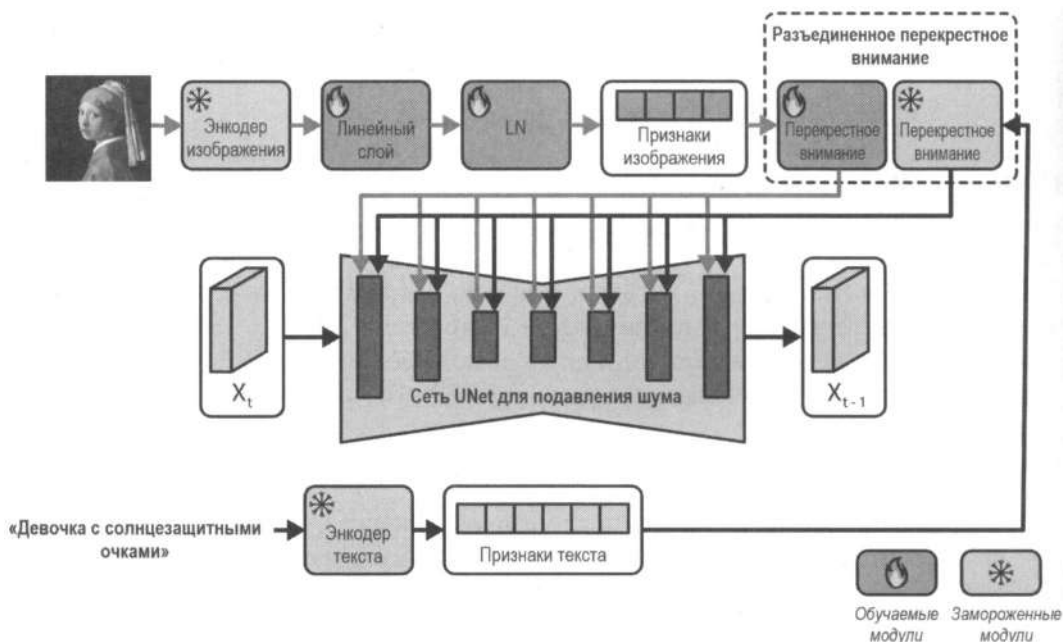


Рис. 8.15. Архитектура IP-адаптера (адаптировано из изображения в статье «IP-Adapter: Text Compatible Image Prompt Adapter for Text-to-Image Diffusion Models»)

- ◆ Использование функции `load_ip_adapter()` для загрузки модели IP-адаптера.
- ◆ Указание масштаба IP-адаптера с помощью функции `set_ip_adapter_scale()`.

Чтобы получить вариации изображений с помощью IP-адаптера, достаточно предоставить эталонное изображение и пустой промпт. Результаты показаны на рис. 8.16.

```
from diffusers import StableDiffusionXLPipeline
```

```
sdxl_base_pipeline = StableDiffusionXLPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0",
    torch_dtype=torch.float16,
    variant="fp16",
)
sdxl_base_pipeline.to(device)
```

```
#Мы также загружаем IP-адаптер
sdxl_base_pipeline.load_ip_adapter(
    "h94/IP-Adapter", subfolder="sdxl_models", weight_name="ip-adapter_sdxl.bin"
)
```

```
#Мы можем задать масштаб, определяющий, насколько сильно
#наш IP-адаптер будет влиять на общий результат
sdxl_base_pipeline.set_ip_adapter_scale(0.8)
image = load_image(SampleURL.ItemsVariation)
original_image = image.resize((1024, 1024))
```



```
#Создаем вариации изображения
generator = torch.Generator(device=device).manual_seed(1)
variation_image = sdxl_base_pipeline(
    prompt="",
    ip_adapter_image=original_image,
    num_inference_steps=25,
    generator=generator,
).images

image_grid([original_image, variation_image[0]], rows=1, cols=2)
```



Рис. 8.16. Пример работы IP-адаптера

С помощью IP-адаптера мы создали новую интерпретацию нашего изображения (как вы можете убедиться, он убрал взбитые сливки), что действительно удивляет и открывает интересные возможности. IP-адаптеры гораздо мощнее всего, что вы делали до этого момента. Изображения в качестве промптов позволяют комбинировать IP-адаптеры с текстовыми промптами и другими элементами управления, о которых вам уже известно.

Изображения в качестве промптов

IP-адаптеры могут не только генерировать вариации изображений, но и использовать их в качестве промптов, что позволяет применять различные техники (например, *перенос стилей*), рассмотренные в этой главе.

Перенос стиля

Несмотря на то что IP-адаптеры отлично работают с переносом стилей «из коробки», исследования показывают, что, если их применять только к определенным блокам сети UNet модели Stable Diffusion, они могут влиять исключительно на стиль изображения. Идея заключается в том, чтобы передать в модель промпт и стиль, и она сгенерирует соответствующее им изображение. Ниже в примере (рис. 8.17) мы продемонстрируем стиль работы бразильской художницы Тарсилы

ду Амарал (Tarsila do Amaral) на примере картины «О Mamoeiro». Основные отличия будут заключаться в масштабе IP-адаптера и входном промпте, который теперь не является пустым.

```
#Загружаем модель и IP-адаптер, как и раньше
pipeline = StableDiffusionXLPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0", torch_dtype=torch.float16
).to(device)

#Загружаем IP-адаптер в модель
pipeline.load_ip_adapter(
    "h94/IP-Adapter", subfolder="sdxl_models", weight_name="ip-adapter_sdxl.bin"
)

#Применяем IP-адаптер только к среднему блоку,
#где он должен быть сопоставлен со стилем в SDXL
scale = {"up": {"block_0": (0.0, 1.0, 0.0)}}
pipeline.set_ip_adapter_scale(scale)

image = load_image(SampleURL.Mamoeiro)
original_image = image.resize((1024, 1024))

#Запускаем вывод для генерации изображения в заданном стиле
generator = torch.Generator(device=device).manual_seed(0)
variation_image = pipeline(
    prompt="a cat inside of a box",
    ip_adapter_image=original_image,
    num_inference_steps=25,
    generator=generator,
).images

image_grid([original_image, variation_image[0]], rows=1, cols=2)
```



Рис. 8.17. Пример работы IP-адаптера для переноса стиля изображения

Дополнительный контроль

В завершение мы покажем, как все изученные в этой главе техники можно объединить в единое целое. Для примера добавим стиль IP-адаптера из «O Mameoieiro» к предыдущему примеру с замаскированным певцом из ControlNet (рис. 8.18).

```
controlnet = ControlNetModel.from_pretrained(
    "diffusers/controlnet-depth-sdxl-1.0", torch_dtype=torch.float16
)

#Загружаем конвейер ControlNet
controlnet_pipeline = StableDiffusionXLControlNetPipeline.from_pretrained(
    "stabilityai/stable-diffusion-xl-base-1.0",
    controlnet=controlnet,
    torch_dtype=torch.float16,
    variant="fp16",
)
controlnet_pipeline.to(device)

#Загружаем IP-адаптер
controlnet_pipeline.load_ip_adapter(
    "h94/IP-Adapter", subfolder="sdxl_models", weight_name="ip-adapter_sdxl.bin"
)

#Применяем IP-адаптер только к среднему блоку,
#где он должен быть сопоставлен со стилем в SDXL
scale = {
    "up": {"block_0": [0.0, 1.0, 0.0]},
}
controlnet_pipeline.set_ip_adapter_scale(scale)

#Загружаем исходное изображение
original_image = load_image(SampleURL.WomanSpeaking)
original_image = original_image.resize((1024, 1024))

#Загружаем стиль изображения
style_image = load_image(SampleURL.Mameoieiro)
style_image = style_image.resize((1024, 1024))

#Применяем оценку глубины с помощью MiDAS
processed_image_midas = midas(original_image).resize(
    (1024, 1024), Image.BICUBIC
)

image = controlnet_pipeline(
    "A masked super hero singing a song",
    image=processed_image_midas,
    ip_adapter_image=style_image,
```

```
controlnet_conditioning_scale=0.5,
).images[0]

image_grid(
    [original_image, style_image, processed_image_midas, image], rows=1, cols=4
)
```



Рис. 8.18. Объединяем все изученное в единое целое

Время проекта: ваш творческий холст

В этой главе мы представили множество инструментов для творческого применения и самовыражения. Теперь вам пора попрактиковаться. Ваша задача — попытаться объединить как минимум два представленных здесь метода в своих собственных интерпретациях. Вот несколько идей для вашего творческого эксперимента:

- ◆ Используйте четкие края (canny edges) ControlNet, чтобы «переосмыслить» (реинтерпретировать) свое жилое пространство, используя стиль IP-адаптера, взятый на основе симпатичного вам референса.
- ◆ Нарисуйте набросок города на листе бумаги, сфотографируйте его и примените технику image-to-image, чтобы превратить его в утопию в стиле Solarpunk. Затем примените стиль вашего наброска в качестве референса и сгенерируйте здания, транспортные системы и жителей вашего нового города.
- ◆ Найдите свои собственные способы исследовать латентное пространство.

Заключение

В этой главе мы рассмотрели различные творческие приложения, которые обеспечивают больший контроль над генерациями, чем рассмотренные ранее методы, и расширяют возможности моделей преобразования текста в изображение благодаря детальному контролю и увеличению диапазона входных данных, которые могут принимать эти модели. Поскольку данные методы являются komponуемыми и совместимыми друг с другом, это позволяет сложным творческим процессам и художественным разработкам вывести генерации за рамки простого ввода текста и получения изображения на выходе. Используя сочетание преобразований изображений, вариаций, стилей или структурных референсов, а также более детальное

управление промптами, творческие люди могут позволить себе включить рабочие процессы машинного обучения в свой инструментарий.

Однако такое применение методов, особенно при переносе реальных изображений в латентное пространство модели, влечет за собой новые проблемы. К основным этическим ограничениям относятся дезинформация, обман и право собственности. Доступность редактирования изображений для всех (а не только, например, для специалистов по Adobe Photoshop) создает новые возможности для манипуляций с изображениями, которые могут быть использованы для распространения дезинформации и обмана людей с помощью дипфейков или несуществующего контента. Вопрос права собственности также имеет ключевое значение: художественные стили можно видоизменить, но правовые и этические границы, определяющие, когда такое изменение является справедливым, все еще остаются открытыми. Стратегии снижения рисков, такие как нанесение водяных знаков, уже существуют в различных библиотеках вроде *diffusers*. Однако на общественном уровне предстоит еще обсудить, как использовать подобные новые возможности, предоставляемые машинным обучением.

Тем не менее творческий потенциал новых моделей является захватывающим. При этическом и ответственном использовании они, безусловно, предоставляют творческим людям инструменты, превосходящие все ожидания.

Для дополнительной информации мы рекомендуем следующие ресурсы:

- ◆ «*IP-Adapter: All You Need to Know*».
- ◆ «*Train Your ControlNet with diffusers*».
- ◆ Руководство по технике *inpainting* в библиотеке *diffusers*.
- ◆ «*Instruction-tuning Stable Diffusion with InstructPix2Pix*».

Вопросы

1. Объясните, чем *inpainting* отличается от преобразования изображения в изображение? Приведите пример практического применения.
2. Как взвешивание промптов может помочь преодолеть ограничения моделей диффузии?
3. Каковы ключевые различия между редактированием Prompt-to-Prompt и SEGA?
4. Как ControlNet расширяет возможности моделей диффузии? Приведите примеры условий, которые можно использовать с ControlNet.
5. Что такое инверсия в контексте моделей преобразования текста в изображение и что она позволяет делать?

Ответы на эти вопросы можно найти в репозитории книги на GitHub.

Генерация аудио

В главе 1 мы кратко упомянули про потенциал генерации звука с помощью конвейера трансформеров, основанного на модели MusicGen от Meta. В этой главе мы более подробно остановимся на генерации аудиоданных, используя методы как трансформеров, так и диффузии. Представьте, что мы могли бы во время звонка удалить весь фоновый шум в режиме реального времени, получить высококачественные транскрипции и краткие обзоры записи конференций или сгенерировать известные песни на других языках. Это большая область для исследований.

С помощью машинного обучения и набора аудиоданных можно решать многие задачи. Две наиболее распространенные из них:

- ♦ *Автоматическое распознавание речи (Automatic Speech Recognition, ASR).*
- ♦ *Генерация речи из текста (Text To Speech, TTS).*

В ASR модель получает на вход аудиозапись речи одного или нескольких человек и выдает ее содержание в виде текста. Некоторые выходные данные также могут содержать дополнительную информацию: например, кто именно говорит и в какое время. Такие системы широко используются, от виртуальных речевых помощников до генераторов субтитров. Благодаря тому, что многие модели имеют открытый доступ (особенно много их стало в последние годы), исследования в этом направлении позволили сделать их многоязычными и дали возможность запускать их непосредственно на пользовательских устройствах.

В TTS модель генерирует синтетическую и, как ожидается, реалистичную речь. Как и в случае с ASR, здесь также наблюдается значительный интерес к запуску моделей непосредственно на устройстве и поддержке многоязычности. TTS имеет свои особенности, например возможность генерировать аудиозаписи с несколькими участниками разговора, обеспечение более естественного звучания голосов, а также учет интонаций, пауз, маркеров эмоций, управления высотой тона, акцентов и других характеристик в генерациях.

Несмотря на то что оба направления являются наиболее популярными, существует множество и других вещей (рис. 9.1), которые можно делать с помощью машинного обучения и набора аудиоданных:

- ♦ *Преобразование текста в аудио*

TTS можно обобщить до преобразования текста в аудио (**Text To Audio, TTA**), где на основе промпта модель может генерировать мелодии, звуковые эффекты и песни.

◆ *Клонирование голоса*

Голос человека, включая тон, высоту и просодию, может быть использован для создания новых генераций его речи.

◆ *Классификация аудио*

Модель способна классифицировать предоставленные аудиоданные в соответствии с принадлежностью к какой-либо группе. Типичными примерами являются распознавание команд и идентификация говорящего.

◆ *Улучшение голоса*

Модель удаляет шум на записи и очищает голос, делая его более четким.

◆ *Перевод аудио*

Модель получает файл с исходным языком X и выводит его на целевой язык Y .

◆ *Диаризация говорящего¹*

Модель идентифицирует говорящего в определенное время.

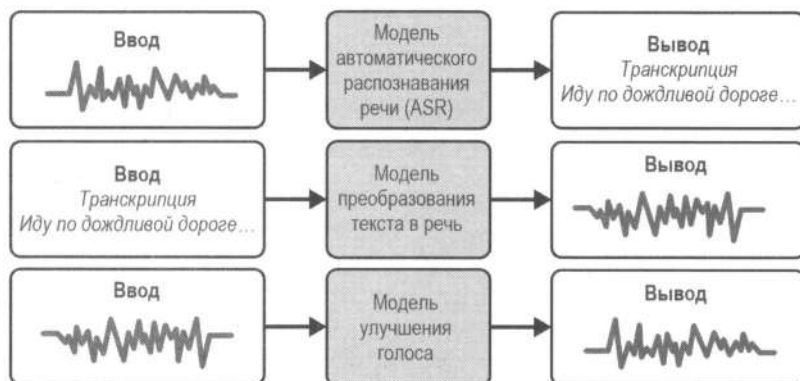


Рис. 9.1. Примеры использования аудиогенераций

Задачи, связанные с аудио, сложны по нескольким причинам. Во-первых, работа с необработанным сигналом сложна сама по себе и интуитивно менее понятна, чем работа с текстом. Во-вторых, реализация многих приложений предполагает, что аудиомодели должны работать в режиме реального времени или быть размещены на локальных устройствах, что может ограничить их размер и скорость вывода. Например, существующие модели диффузии являются слишком медленными для интерактивного перевода. В-третьих, оценка генеративных аудиомоделей может быть сложной. Как оценить качество сгенерированной песни?

Существуют сотни моделей и наборов данных с открытым доступом, которые могут в этом помочь. Например, Common Voice — популярный краудсорсинговый²

¹ Диаризация (или разделение дикторов) — процесс разделения входящего аудиопотока на однородные сегменты в соответствии с принадлежностью тому или иному говорящему. Диаризация повышает качество текстов при автоматическом транскрибировании, а также может использоваться совместно с системой распознавания речи, значительно ее улучшая. — Пер.

набор данных от Mozilla Foundation — содержит более 2000 часов аудиозаписей и соответствующий им текст на более чем ста языках. Открытый доступ есть у множества и других популярных наборов, например LibriSpeech, VoxPopuli и GigaSpeech. Каждый из них имеет свою собственную область применения и варианты использования. Поскольку они являются открытыми, это позволяет использовать их совместно с моделями, которые также имеют открытый доступ. В этой главе мы рассмотрим модели на основе трансформеров: Wav2Vec2 от Meta, Whisper от OpenAI, SpeechT5 от Microsoft и Bark от Suno. Мы также познакомимся с интересными моделями диффузии, которые могут генерировать песни, например Stable Diffusion (но для песен), Dance Diffusion и AudioLDM. Несмотря на то что переход к другой модальности (аудио) может показаться сложным, многие из инструментов, рассмотренных нами по ходу чтения книги, вы скоро сможете применить.

Аудиоданные

Сначала стоит изучить, как структурированы аудиоданные и как их использовать. Для примера рассмотрим набор данных LibriSpeech, содержащий более 1000 часов чтения книг вслух. Он полезен при обучении и оценке систем распознавания речи. Одно из важных ограничений касательно аудио заключается в том, что они, как правило, имеют большой объем, поэтому одновременная загрузка всех данных может быть нецелесообразной. Наборы могут содержать терабайты информации и не помещаться на жесткий диск.

Чтобы получить наиболее полное представление о структуре наборов без загрузки всех данных, можно использовать инструмент `load_dataset_builder()`.

```
from datasets import load_dataset_builder

ds_builder = load_dataset_builder(
    "openslr/librispeech_asr", trust_remote_code=True
)
ds_builder.info.splits

{'test.clean': SplitInfo(name='test.clean',
                          num_bytes=368449831,
                          num_examples=2620,
                          shard_lengths=None,
                          dataset_name=None),
 'test.other': SplitInfo(name='test.other',
                          num_bytes=353231518,
                          num_examples=2939,
                          shard_lengths=None,
                          dataset_name=None),
```

² **Краудсорсинг (Crowdsourcing)** — это метод организации работы, при котором компания привлекает множество людей со стороны для решения какой-то задачи. Вместо того чтобы поручать задачу ограниченному числу специалистов, компания обращается к «толпе» — неограниченному кругу пользователей. — *Пер.*

```
'train.clean.100': SplitInfo(name='train.clean.100',
                             num_bytes=6627791685,
                             num_examples=28539,
                             shard_lengths=None,
                             dataset_name=None),
'train.clean.360': SplitInfo(name='train.clean.360',
                             num_bytes=23927767570,
                             num_examples=104014,
                             shard_lengths=None,
                             dataset_name=None),
'train.other.500': SplitInfo(name='train.other.500',
                              num_bytes=31852502880,
                              num_examples=148688,
                              shard_lengths=None,
                              dataset_name=None),
'validation.clean': SplitInfo(name='validation.clean',
                               num_bytes=359505691,
                               num_examples=2703,
                               shard_lengths=None,
                               dataset_name=None),
'validation.other': SplitInfo(name='validation.other',
                               num_bytes=337213112,
                               num_examples=2864,
                               shard_lengths=None,
                               dataset_name=None)}
```

Авторы набора LibriSpeech обнаружили, что размер корпуса данных делает непрактичной работу с ним, поэтому они решили разбить его на подмножества по 100, 360 и 500 часов. Это вполне целесообразно, ведь только сами обучающие данные занимают более 60 ГБ. Мы можем начать изучать признаки данных без ожидания их полной загрузки, используя инструмент `.info.features`.

```
ds_builder.info.features
```

```
{'file': Value(dtype='string', id=None),
 'audio': Audio(sampling_rate=16000, mono=True, decode=True, id=None),
 'text': Value(dtype='string', id=None),
 'speaker_id': Value(dtype='int64', id=None),
 'chapter_id': Value(dtype='int64', id=None),
 'id': Value(dtype='string', id=None)}
```

У нас есть все данные, необходимые, чтобы построить начальный конвейер распознавания речи с помощью признаков. Наиболее ценными из них здесь являются `'text'` и `'audio'`. Каждый признак имеет свой тип. Например, `'text'` — это тип `Value`, содержащий данные в виде `string`. А `'audio'` — это тип `Audio`, содержащий аудиоинформацию. Как и изображения, звук может быть представлен несколькими каналами. Атрибут `mono` указывает, является ли звучание моноканальным (один канал, обеспечивающий равномерное звучание) или стерео (два канала, обеспечивающие

ощущение направленности). Что означают `sampling_rate` и `decode`, мы обсудим в следующем разделе.

Учитывая большой размер набора данных, нам необходимо найти способы эффективной работы с ним. Вместо того чтобы загружать его полностью, можно с помощью библиотеки `datasets` использовать потоковый режим, по одному примеру за раз. Это позволит сэкономить дисковое пространство и использовать данные из набора по мере их загрузки. Таким образом, получим объект `IterableDataset`, который можно использовать как любой из итераторов Python. Рассмотрим первый пример 100-часового отрезка.

```
from datasets import load_dataset

ds = load_dataset(
    "openslr/librispeech_asr",
    split="train.clean.360",
    streaming=True,
)

sample = next(iter(ds))

{'audio': {'array': array([9.15527344e-05, 4.57763672e-04, 5.18798828e-04, ...,
        -4.57763672e-04, -5.49316406e-04, -4.88281250e-04]),
        'path': '1487-133273-0000.flac',
        'sampling_rate': 16000},
 'chapter_id': 133273,
 'file': '1487-133273-0000.flac',
 'id': '1487-133273-0000',
 'speaker_id': 1487,
 'text': 'THE SECOND IN IMPORTANCE IS AS FOLLOWS SOVEREIGNTY MAY BE '
        'DEFINED TO BE THE RIGHT OF MAKING LAWS IN FRANCE THE KING '
        'REALLY EXERCISES A PORTION OF THE SOVEREIGN POWER SINCE '
        'THE LAWS HAVE NO WEIGHT'}
```

В нем представлены аудиозаписи и соответствующие им тексты. Признак `'audio'` содержит следующее:

- ◆ Массив `Array` с декодированными аудиоданными. Признак имеет настройку `decode` со значением `True`, а значит, что аудиоданные уже декодированы. В противном случае они будут представлены в формате байтов, и вам необходимо декодировать их самостоятельно.
- ◆ Путь к загруженному аудиофайлу.
- ◆ Данные `sampling_rate`, необходимые для корректной загрузки аудио.

Если эти концепции кажутся вам странными, не волнуйтесь. Это прекрасная возможность узнать, как устроены аудиоданные.

Аудио — это бесконечный набор значений, изменяющихся во времени. Компьютеры не могут работать с непрерывными данными, поэтому необходимо обработать

аудиосигнал и получить его цифровое дискретное (конечное) представление. Для этого мы делаем заданное количество снимков в секунду. Это называется *частотой дискретизации*. Например, для нашего аудиофрагмента в признаке 'audio' можно обнаружить, что частота дискретизации составляет 16 000 Герц. Это означает, что в секунду делается 16 000 выборок. Для файла длительностью в минуту это составляет почти миллион значений, неудивительно, что наборы аудиоданных имеют огромные объемы и размеры. На рис. 9.2 показана форма аудиосигнала, полученная с использованием частоты дискретизации со значением 6.

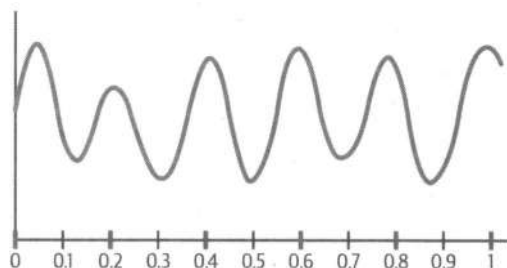


Рис. 9.2. В этой форме сигнала дискретизация происходит с интервалом в 1/6 секунды

Частота дискретизации — важный параметр. При работе с аудио в ML необходимо обеспечить одинаковую частоту дискретизации всех выборок. Модели предварительно обучаются на данных, дискретизированных с определенной частотой, поэтому при тонкой настройке или выполнении вывода необходимо использовать одинаковую частоту дискретизации. Несмотря на то что некоторые из самых популярных наборов аудиоданных имеют частоту дискретизации 16 000 Герц, такое встречается нечасто, поэтому на этапе предварительной обработки необходимо выполнить повторную выборку данных.

```
array = sample["audio"]["array"]
sampling_rate = sample["audio"]["sampling rate"]

#Получаем первые 5 секунд аудиозаписи
array = array[: sampling_rate * 5]
print(f"Number of samples: {len(array)}. Values: {array}")
```

```
('Number of samples: 80000. Values: [9.15527344e-05 4.57763672e-04 '
 '5.18798828e-04 ... 7.05261230e-02\n'
 ' 5.92041016e-02 6.50329590e-02]')
```

Конечно, мы не можем напечатать аудиофайл в книге, но вы можете запустить код, чтобы его прослушать. Для этого можно использовать функцию `IPython.display.Audio()`. Кроме того, вы можете посетить официальную интерактивную демостраницу, чтобы прослушать все аудиофрагменты из этой главы. Попробуем прослушать первый аудиофрагмент из 100-часового отрезка.

```
import IPython.display as ipd
ipd.Audio(data=array, rate=sampling_rate)
```



Возможно, вас заинтересовали атрибуты `decode` и `mono` в признаке `'audio'`. Первый определяет, следует ли декодировать данные (возвращать их в виде массива чисел с плавающей точкой) или нет (возвращать в виде байтов). Второй определяет, является ли аудио моно- или многоканальным. Мы рассмотрим примеры в следующих разделах.

Осциллограмма

Для работы с аудио используется цифровое дискретное представление. По сути, сигнал — это всего лишь массив значений. Поскольку массивы содержат информацию, необходимую, чтобы обучить модели решать различные задачи, нам стоит уделить немного времени их изучению, прежде чем перейти сразу к применению.

Каждое значение в массиве представляет собой амплитуду, которая описывает силу звуковой волны и измеряется в децибелах (дБ).³ Амплитуда показывает громкость звука относительно опорного значения. Опорное значение 0 дБ соответствует самому низкому уровню звука, воспринимаемому человеческим ухом. Например, громкость дыхания составляет около 10 дБ, громкий концерт может достигать 120 дБ (достаточно болезненно), а извержение вулкана Кракатау (колоссальное событие в 1883 году) можно было услышать даже на расстоянии 4800 километров (3000 миль) с предполагаемой громкостью 310 дБ, как показано на рис. 9.3.



Рис. 9.3. Амплитуда некоторых звуков в дБ

Амплитуда обычно измеряется в децибелах, но массив часто бывает нормализован, поэтому числа находятся в диапазоне от -1 до 1 . Для визуализации звука можно использовать *осциллограмму* — график изменения амплитуды во времени. Построим его (рис. 9.4) с помощью `librosa` — популярной библиотеки для работы с аудиоданными.

```
import librosa.display
librosa.display.waveshow(array, sr=sampling_rate);
```

Осциллограмма помогает провести первоначальное исследование сигнала. Прислушав запись, можно заметить, что первые волны соответствуют словам диктора: «Второй по важности сигнал — это следующий», за которым следует короткая пауза. В более общем смысле осциллограммы — это интуитивно понятный способ выявить нерегулярности в аудиоданных и получить общее представление о сигнале и его закономерностях.

³ Точнее говоря, вибрации вызывают колебания в воздухе (или других средах), приводящие к возникновению звука. Эти вибрации создают звуковую волну, которая, в свою очередь, вызывает изменение давления в ушах.

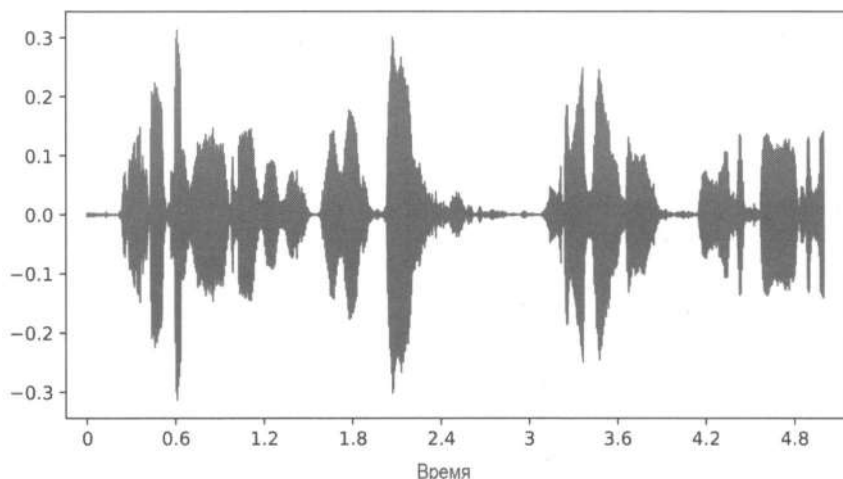


Рис. 9.4. Осциллограмма, построенная с помощью библиотеки librosa

Спектрограммы

Как и осциллограммы, *спектрограммы* (рис. 9.5) — это тоже способ представления аудиосигналов. В этом разделе мы объясним, что такое спектрограммы и когда их следует использовать.

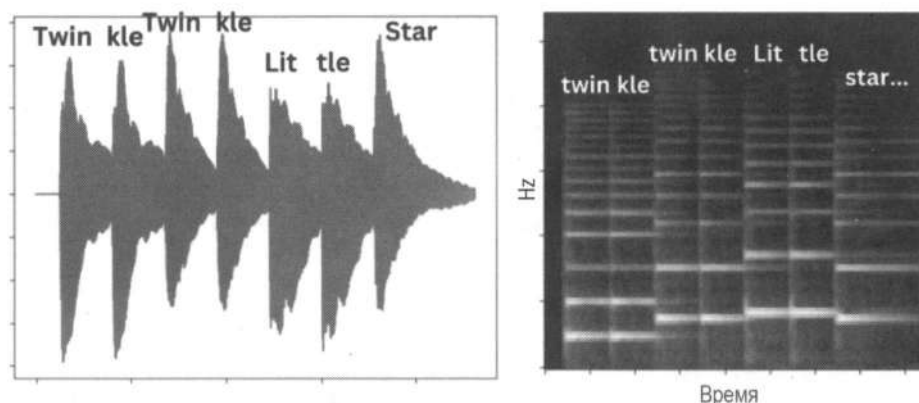


Рис. 9.5. Осциллограмма (слева) и мел-спектрограмма (справа), представляющие одну и ту же аудиозапись

Допустим, мы хотим обучить генеративную модель создавать новый звук или синтетическую речь на основе входного сигнала. Простейший способ сделать это — использовать необработанную форму аудиозаписи в качестве входных данных. Осциллограммы — это прямое и интуитивно понятное представление звука, содержащее всю информацию, необходимую для воспроизведения. Вы можете легко убедиться в этом, прослушав несколько фрагментов из набора, будь то голоса, музыка или другие звуки. Итак, если осциллограммы могут точно представлять звук, почему бы не использовать их для обучения моделей?

Здесь существуют некоторые ограничения. Одно из них заключается в размерности. Несмотря на то что осциллограммы одномерны, они состоят из очень большого количества точек данных. Каждая секунда звука содержит десятки тысяч фрагментов, которые модель должна обработать. Такая высокая размерность затрудняет эффективное изучение моделями закономерностей и структур данных.

Кроме того, человеческая слуховая система очень чувствительна к небольшим различиям в таких характеристиках, как высота или тембр, которые связаны с частотными характеристиками звука. Несмотря на то что эти атрибуты закодированы в форме сигнала, их сложно идентифицировать даже человеку, если он просто взглянет на него. Наиболее очевидная информация, которую мы можем извлечь из сигнала, — это изменение амплитуды во времени, часто называемое *временной областью*. Однако атрибуты, связанные с частотой (то, что отличает звучание, например, фортепиано от звучания скрипки), гораздо сложнее определить только по форме сигнала (рис. 9.6).

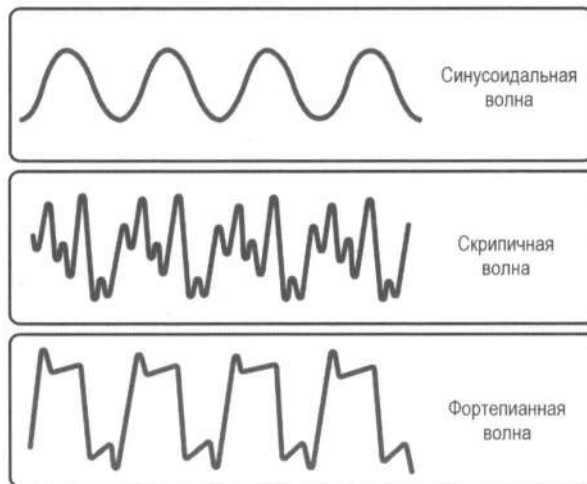


Рис. 9.6. Одна и та же нота, сыгранная несколькими инструментами, имеет одинаковую высоту (частоту) и амплитуду, но звучит по-разному из-за различий в форме волны; чистая нота — это идеальная синусоида

Несмотря на то что обучение модели на необработанных данных возможно (мы рассмотрим это далее в этой главе), огромное количество задействованных выборок представляет собой серьезную проблему. Например, одна минута аудиозаписи с частотой дискретизации 16 000 Гц содержит почти миллион выборок. Если мы попытаемся уменьшить эту размерность через *усреднение* или *объединение* выборок, мы рискуем потерять детали, которые критически важны в частотной области. Даже частота дискретизации в 16 000 Гц, которой хватает для восприятия голоса, является недостаточной для высококачественной музыки. Поэтому потребительские аудиостандарты, от компакт-дисков до потоковых сервисов, обычно используют частоту дискретизации 44,1 кГц или выше.

Учитывая эти сложности, более эффективный подход часто заключается в преобразовании аудиосигнала в другое представление, называемое *спектрограммой*. Оно

показывает, как частота и амплитуда звука изменяются с течением времени. Благодаря явному захвату частотных характеристик спектрограммы обеспечивают более структурированное и информативное представление аудиосигнала, что упрощает обучение моделей. Спектрограммы переводят задачу из временной области в *частотно-временную*, что лучше подходит для задач машинного обучения, связанных со звуком. Однако важно отметить, что спектрограммы как представления имеют потери, т. е. они не сохраняют всю информацию из исходного сигнала. Несмотря на это, потери обычно являются приемлемыми, поскольку наиболее значимые для восприятия характеристики сохраняются.

Прежде чем перейти к работе со спектрограммами, рассмотрим частоты. Мы построим графики (рис. 9.7) четырех волн с одинаковыми диапазонами амплитуд, но с разными частотами.

```
import numpy as np
from matplotlib import pyplot as plt

def plot_sine(freq):
    sr = 1000 # samples per second
    ts = 1.0 / sr # sampling interval
    t = np.arange(0, 1, ts) # time vector
    amplitude = np.sin(2 * np.pi * freq * t)

    plt.plot(t, amplitude)
    plt.title("Sine wave with frequency {}".format(freq))
    plt.xlabel("Time")

fig = plt.figure()

plt.subplot(2, 2, 1)
plot_sine(1)

plt.subplot(2, 2, 2)
plot_sine(2)

plt.subplot(2, 2, 3)
plot_sine(5)

plt.subplot(2, 2, 4)
plot_sine(30)

fig.tight_layout()
plt.show()
```

Как вы можете заметить, волны имеют одинаковые амплитуды, но демонстрируют разные значения частот. Несмотря на то что на графиках изображены простые синусоиды, реальные звуки имеют более сложные формы волн. Обычно звук представляет собой композицию нескольких объединенных волн, каждая из которых

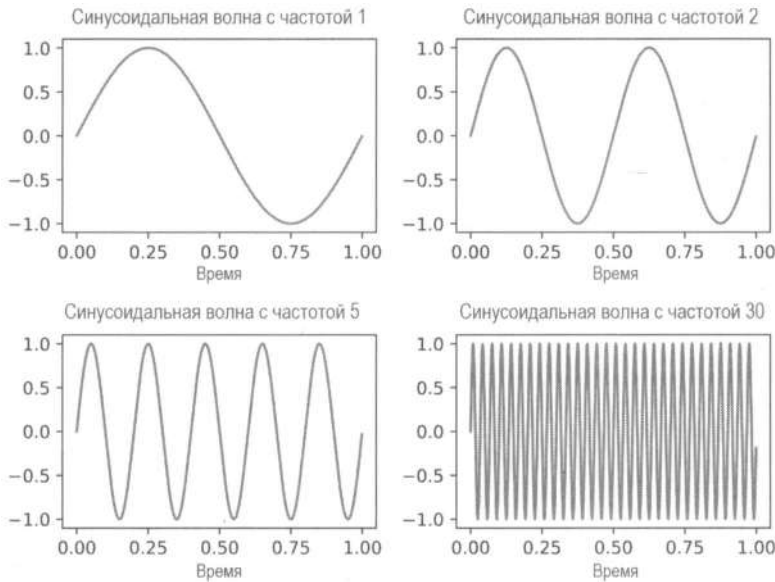


Рис. 9.7. Графики четырех волн с одинаковыми диапазонами амплитуд, но разными частотами

имеет свою частоту и амплитуду. Поэтому наша первая задача — разбить звуковую волну на несколько более простых компонентов. Это позволит извлечь ценную информацию, которую может использовать модель. А именно — как амплитуда изменяется со временем на разных частотах и как мы можем разложить звук? Мы будем использовать *преобразование Фурье* (ПФ) — математический инструмент, позволяющий разложить одну функцию на множество других.

Начнем с нескольких простых функций, как показано на рис. 9.8. В первом столбце находятся синусоидальные функции. Во втором столбце — график ПФ, т. е. функции в частотной области.

Проанализируем первую строку рисунка. Слева показан исходный сигнал — синусоида с частотой один цикл в секунду. Справа график ПФ отображает частоту по оси X и амплитуду в частотной области по оси Y . Обратите внимание на пик в области единицы, который совпадает с частотой исходного сигнала. В следующих строках показаны примеры с разными частотами. Они имеют одинаковое поведение: график ПФ достигает пика на частоте исходного сигнала. Значение по оси Y равно половине числа отсчетов (в этом примере частота дискретизации составляет 2000, и это всего одна секунда, поэтому у нас 2000 отсчетов), умноженного на амплитуду исходного сигнала (1 в первых пяти строках)⁴.

Далее рассмотрим последнюю строку. Сигнал сочетает в себе три синусоидальные функции с различными амплитудами (1, 3 и 1,5) и частотами (2, 5 и 14). Сложно определить состав сигнала, просто глядя на форму волны, поэтому здесь на помощь

⁴ Здесь есть некоторые нюансы. При вычислении преобразования Фурье действительного сигнала его абсолютное значение симметрично. Это приводит к зеркальному отображению графика в частотной области. Для пояснения мы построили первую половину графика.

приходит частотная область. В ней можно наблюдать три пика, соответствующие частотам исходной функции: 2, 5 и 14. Следовательно, мы можем провести реверс-инжиниринг и описать начальную форму трех функций осциллограммы.

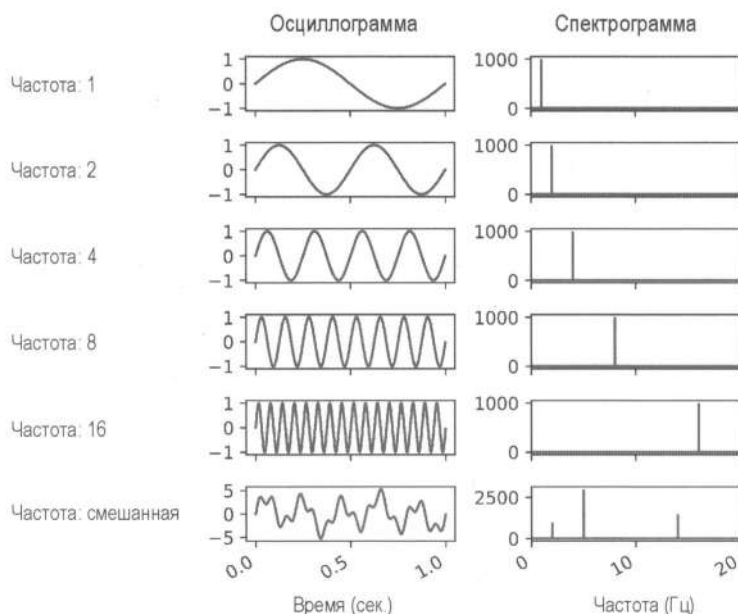


Рис. 9.8. Некоторые синусоидальные функции и их частотные спектры: чистые ноты (синусоидальные волны) имеют один пик в периоде синуса, тогда как более сложные звуки демонстрируют несколько пиков в частотной области

На частоте 2 Гц амплитуда в частотной области (ось Y справа) составляет 1000 единиц. Выполнив преобразование $1000 \times 2 / 2000$, мы получим значение 1 — амплитуду первой синусоидальной функции, составляющей сигнал. Аналогично частота 3 Гц дает амплитуду в частотной области 3000 единиц, соответственно, выполнив преобразование $3000 \times 2 / 2000$, мы получим значение 3. Разложенные синусоиды изображены на рис. 9.9. Преобразование Фурье дает механизм, который позволяет анализировать сложные формы сигналов и понимать частоты, которые их составляют. Теперь мы знаем, что в звуке есть скрытые сложности — информация, которой, как мы думали, там нет, но она есть. Это дает гораздо больше информации и

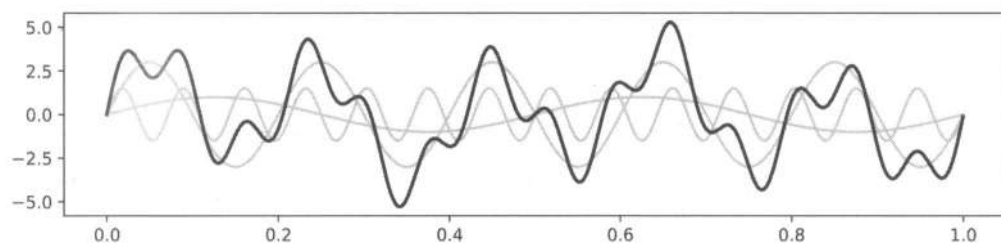


Рис. 9.9. Сложную звуковую волну можно разложить на синусоидальные частоты, анализируя спектральное представление

является ключом к моделям, которые позволяют транскрибировать речь или генерировать музыку.

Как и в случае с формами сигналов на рис. 9.9, функцию можно разбить на несколько синусоид с их амплитудами и частотами. Посмотрим на график в частотной области (рис. 9.10).

```
#Вычисляем быстрое преобразование Фурье (БПФ) входного сигнала
```

```
X = np.fft.fft(array)
```

```
#Длина результата БПФ (которая совпадает с длиной входного сигнала)
```

```
N = len(X)
```

```
#Вычислить частотные интервалы, соответствующие результату БПФ
```

```
n = np.arange(N)
```

```
T = N / sampling_rate
```

```
freq = n / T
```

```
#Строим амплитудный спектр для первых 8000 частотных интервалов
```

```
#Мы могли бы построить все интервалы, но тогда получим зеркальное отражение спектра
```

```
plt.stem(freq[:8000], np.abs(X[:8000]), "b", markerfmt=" ", basefmt="-b")
```

```
plt.xlabel("Frequency (Hz)")
```

```
plt.ylabel("Amplitude in Frequency Domain")
```

```
plt.show()
```

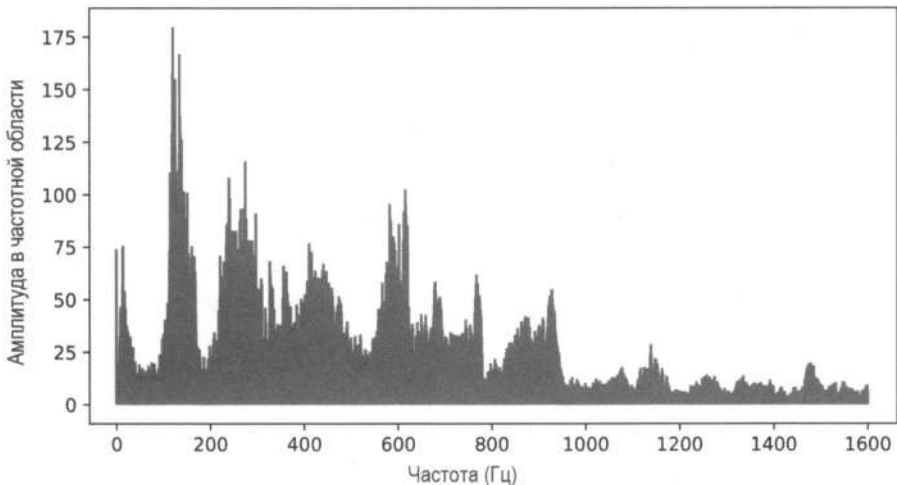


Рис. 9.10. График аудиосигнала, разбитый на составляющие в частотной области

Этот аудиосигнал сложнее интерпретировать, чем из предыдущих примеров. Видно, что большая часть звука находится в диапазоне 0–800 Гц. Также видно, что около 170 Гц присутствуют громкие шумы. Несмотря на то что график полезен, мы теряем информацию во временной области. Неизвестно, в какое время были слышны звуки с определенными частотами. Осциллограммы содержат информацию об

амплитуде и времени, а графики преобразования Фурье — об амплитуде и частоте. Можно ли объединить эти три параметра вместе?

Спектрограммы отображают изменение частоты и амплитуды сигнала во времени. Они являются очень информативными и визуализируют время, частоту и амплитуду на одном графике. Чтобы создать спектрограмму, нужно переместить *окно* по исходному сигналу и вычислить ПФ для всего сегмента, чтобы зафиксировать изменение частот во времени. Затем окна можно наложить друг на друга и сформировать спектрограмму. Такой подход (перемещение окна по аудиосигналу) называется *кратковременным преобразованием Фурье* (КПФ).

В библиотеке Librosa есть функция `stft()`, позволяющая получить КПФ. Помимо вычисления спектрограммы, мы также преобразуем амплитуду в децибелы, которые являются логарифмическими, и они гораздо более удобны для визуализации. Помните, амплитуда — это разность звукового давления. Следовательно, числовой диапазон очень широк. Используя логарифмическую шкалу, мы сужаем масштаб, делаем графики более информативными и получаем информацию, более приближенную к восприятию звука человеком.



Шкала децибел является логарифмической, т. е. каждое увеличение на 10 дБ соответствует десятикратному увеличению относительной интенсивности звука. Однако наше восприятие громкости не соответствует напрямую этим физическим изменениям. Несмотря на то что и шкала децибел, и восприятие громкости подчиняются логарифмическим закономерностям, они делают это по-разному. В частности, увеличение интенсивности звука на 10 дБ воспринимается человеческим ухом примерно как удвоение громкости, а не как десятикратное увеличение. Это различие возникает потому, что слуховая система человека «сжимает» изменения интенсивности в более приемлемый диапазон воспринимаемой громкости.

Рассмотрим спектрограмму нашего примера (рис. 9.11).

#Вычисляем кратковременное преобразование Фурье (КПФ)

#Мы берем абсолютное значение КПФ, чтобы

#получить амплитуду каждого частотного интервала

```
D = np.abs(librosa.stft(array))
```

#Преобразуем амплитуду в децибелы, которые являются логарифмическими

```
S_db = librosa.amplitude_to_db(D, ref=np.max)
```

#Генерируем изображение спектрограммы

```
librosa.display.specshow(S_db, sr=sampling_rate, x_axis="time", y_axis="hz")
```

```
plt.colorbar(format="%+2.0f dB");
```

Ось *X*, как и на осциллограмме, отображает время. Ось *Y* отображает частоту (в герцах), а цвет — интенсивность (децибелы) частоты в каждой точке. Черный цвет соответствует областям без энергии (тишина). Как и прежде, мы можем наблюдать некоторый шум в первые 2,4 и последние 1,6 секунды. Самые громкие точки приходятся на низкую частоту (яркий цвет и низкое значение по оси *Y*). Это соответствует осциллограмме и графику в частотной области, где мы наблюдаем высокую амплитуду на низких частотах.

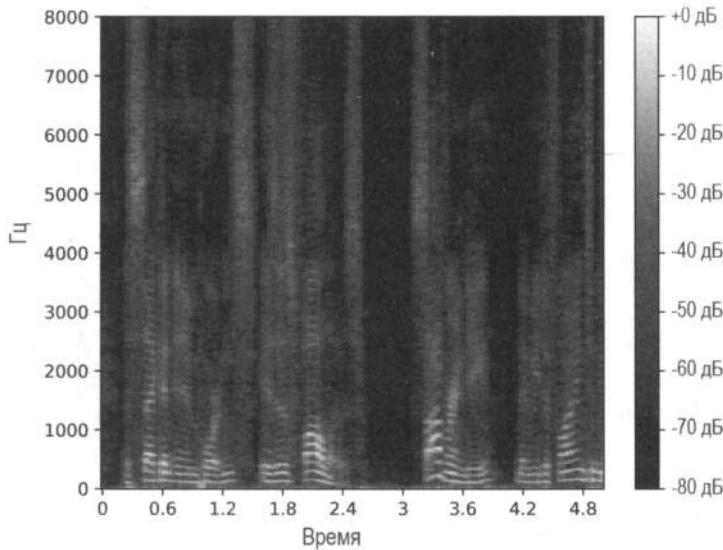


Рис. 9.11. Спектрограмма аудиосигнала



Вы можете задаться вопросом, как значения в децибелах могут быть отрицательными. Поскольку мы использовали функцию `amplitude_to_db()` с параметром `ref=np.max`, максимальное значение спектрограммы составляет 0 дБ. Остальные значения указаны относительно него. Например, «-20 дБ» означает, что амплитуда на 20 дБ ниже максимального значения.

Одна из вариаций спектрограммы (наиболее популярная) называется *мел-спектрограммой*. В то время как в обычной спектрограмме единица измерения частоты линейна, мел-спектрограмма использует шкалу, подобную той, как мы воспринимаем звук, а именно логарифмически. То есть мы более чувствительны к изменениям на низких частотах, чем на высоких. Разница между 500 и 1000 Гц гораздо заметнее, чем между 5000 и 5500. Библиотека `librosa` имеет удобный метод вычисления мел-спектрограмм. В мел-спектрограммах равные расстояния по частоте (ось Y) соответствуют одинаковому воспринимаемому расстоянию. Построим мел-спектрограмму (рис. 9.12).

```
#Генерируем спектрограмму по шкале Мел из аудиосигнала
#Результат – матрица, каждый элемент которой соответствует мощности
#частотной полосы (в шкале Мел) в определенный момент времени
S = librosa.feature.melspectrogram(y=array, sr=sampling_rate)
```

```
#Преобразуем спектрограмму мощности в шкалу децибел
S_db = librosa.power_to_db(S, ref=np.max)
```

```
#Отображаем мел-спектрограмму
librosa.display.specshow(S_db, sr=sampling_rate, x_axis="time", y_axis="mel")
plt.colorbar(format="%+2.0f dB");
```

Мел-спектрограмма имеет схожие паттерны с обычной спектрограммой, но можно заметить некоторые различия. Во-первых, шкала Y — нелинейна: расстояние между

512 и 1024 такое же, как между 2048 и 4096. Во-вторых, области с большей энергией (большей интенсивностью) на низких частотах гораздо более заметны на мел-спектрограмме. Это соответствует тому, как воспринимает звук человек.

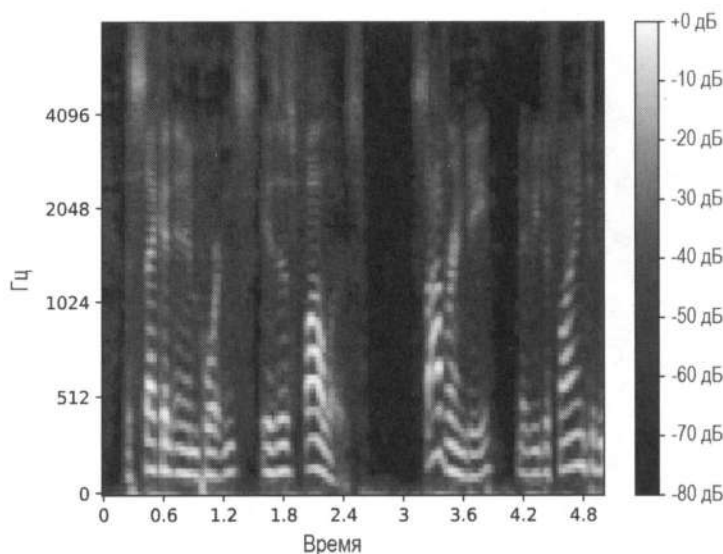


Рис. 9.12. Мел-спектрограмма

Помимо того что спектрограммы являются отличным визуальным представлением для понимания аудиосигналов, они часто напрямую используются моделями машинного обучения. Например, спектрограмму песни можно использовать в качестве входных данных для классификации жанра. Аналогично модель может получать на вход несколько слов и выдавать спектрограмму, представляющую человека, произносящего их.

Преобразование речи в текст с использованием архитектур на основе трансформеров

Рассмотрим транскрибирование аудиофайла в текст подробнее. Как и во многих других задачах, мы можем использовать функцию `pipeline()` из библиотеки `transformers`, которая берет на себя всю предварительную и постобработку и является полезной оберткой для вывода. Чтобы лучше понять, мы будем использовать минимальную версию модели Whisper от компании OpenAI. Она достаточно популярна и имеет открытый исходный код.

```
from transformers import pipeline

pipe = pipeline(
    "automatic-speech-recognition",
    model="openai/whisper-small",
```

```
max_new_tokens=2
)
pipe(array)
```

```
('text': ' The second in importance is as follows. Sovereignty may be defined to be')
```

Результаты данной минимальной версии (244 миллиона параметров, что позволяет эффективно запускать ее на устройстве) впечатляют. Еще более удивительно, что исходный звук обрезан на середине слова «be». Модель может предсказать все слово, даже если оно не было закончено. Кроме того, Whisper может предсказать знаки препинания (например, точку).

В следующих разделах мы рассмотрим, как работают модели, выполняющие автоматическое распознавание речи (ASR). Прежде чем использовать Whisper, сначала обсудим, как модели, использующие только энкодер, могут быть применены к ASR.

Техники на основе энкодеров

Один из способов понять задачу ASR — это взглянуть на нее как на классификацию текстовых токенов. Идея аналогична использованию BERT в обработке естественного языка (NLP). Сначала модель предварительно обучается с помощью MLM, используя трансформер только с энкодером. То есть модель обучается на большом объеме немаркированных данных, а часть входных данных маскируется. Цель — предсказать, какое слово должно заполнить маску. Мы можем применить тот же принцип к аудио. Вместо текста мы маскируем звук (точнее, латентное представление речи), и модель обучается контекстуализированным представлениям. После этого ее можно настроить с помощью маркированных данных для различных аудиозадач, таких как идентификация говорящего или ASR. Рассмотрим данную реализацию подробнее.

Первая проблема, о которой мы уже упоминали, заключается в том, что аудиофрагмент может быть очень длинным. 30-секундная дорожка с частотой дискретизации 16 кГц дает массив из 480 000 значений. Их использование для одной входной выборки в трансформере с энкодером потребовало бы огромных объемов памяти и вычислений. Для решения этого можно использовать сверточную нейронную сеть (CNN) в качестве энкодера признаков. Она проходит по входным данным (сигналу) и выдает латентное речевое представление. Например, предположив, что окно длительностью 20 миллисекунд проходит через аудио длиной в 1 секунду, мы получим 50 латентных представлений (при этом предполагается отсутствие перекрывающихся окон; в терминах CNN мы используем шаг 20). Эти латентные представления затем передаются в классический трансформер с энкодером, который выдает эмбединги для каждого из 50 представлений. Во время предварительного обучения интервалы латентных представлений маскируются, и модель учится предсказывать, как заполнить недостающие части.

Для тонкой настройки ASR-модели к энкодеру добавляется простая линейная классификационная сеть, цель которой состоит в том, чтобы оценивать текст, соответ-

ствующий каждому аудиокну, обрабатываемому энкодером. Количество классов, которые она будет классифицировать, определяется проектным решением. Можно было бы классифицировать целые слова, слоги или только символы, но, учитывая, что мы используем окно в 20 миллисекунд, одно слово не поместится в него. Классификация символов в данном контексте представляется весьма целесообразной, и ее дополнительное преимущество заключается в возможности использовать очень небольшой словарь. Например, для английского языка можно использовать словарь из 26 английских символов плюс несколько специальных токенов (например, для тишины, для чего-то неизвестного и т. д.). Чтобы сохранить минимальный словарь, обычно текст следует предварительно обработать, преобразовав его полностью в верхний регистр и преобразовав числа в слова (например, «14» в «четырнадцать»).



В первой части этой главы мы использовали спектрограммы для захвата амплитудных и частотных характеристик входных данных в кратком двумерном визуальном представлении. Теперь мы исследуем архитектуры, которые обрабатывают данные иначе, используя CNN для извлечения признаков непосредственно из сигнала без преобразования его в спектрограмму. Выбор в пользу какого-либо из этих подходов зависит от конкретной задачи и способа разработки модели.

Вскоре вы увидите, что некоторые модели принимают спектрограммы в качестве входных данных, в то время как другие работают непосредственно с необработанным сигналом. Трансформеры с их механизмами внимания особенно эффективны для обработки последовательных данных, поэтому важно учитывать темпоральную структуру ввода.

Кратко рассмотрим весь процесс распознавания с использованием моделей на основе энкодера:

1. В модель поступают необработанные аудиоданные (одномерный массив), представляющие амплитуды.
2. Данные нормализуются до нулевого среднего значения и одновариантности для стандартизации по разным амплитудам.
3. Небольшая CNN преобразует аудиоданные в латентное представление. Это сокращает длину входной последовательности.
4. Затем представления передаются в модель с энкодером, которая выводит эмбединги для каждого представления.
5. Каждый эмбединг проходит через классификатор, который предсказывает соответствующий символ для каждого из них.

Выходные данные такой модели будут примерно следующими:

```
CHAAAAAPTERRRRSSIXTEEEENIMMIIIGHT...
```

Казалось бы вывод похож на текст, но очевидно, что это не так. Почему так происходит? Если звучание символа распространяется на период, превышающий одно окно, он может появиться в выходных данных несколько раз. Помните, что модель не знает, когда каждый символ встречался во время обучения, поэтому невозможно напрямую сопоставить аудио с текстом.

Решение этой проблемы, изначально применявшееся в рекуррентных нейронных сетях (RNN), называется *Connectionist Temporal Classification (CTC)*. Идея его использования в области аудио заключается в том, чтобы добавлять токен-заполнитель (для визуализации мы будем использовать символ «*»), который служит границей между группами символов, и токен-разделитель (символ «/»), разделяющий слова. Модель научится предсказывать и такие токены. В процессе вывода выходные данные могут выглядеть следующим образом:

```
CHAAAAA*PTT*ERRR/SS*IX*T*EE*EEN/I/MMM*II*GHT
```

С помощью этого вывода мы можем выполнить *дедупликацию*⁵, объединив одинаковые последовательные символы в группы, что приведет к желаемому результату.

CHAPTER SIXTEEN I MIGHT

Все эти идеи (использование модели, работающей только с энкодером, обработка формы сигнала с помощью CNN и использование CTC для классификации) составляют основу архитектур, основанных на таких энкодерах, как *Wav2Vec2* (2020 год) и *HuBERT* (2021 год) от Meta. *Wav2Vec2* предварительно обучена с использованием Librispeech и LibriVox (оба набора данных имеют немаркированные данные). Ее достаточно настроить всего на 10 минутах маркированных данных, чтобы она смогла превзойти модели, обученные на значительно большем объеме информации. Это очень интересно, поскольку базовую модель можно легко настроить для определенных областей, не имея большого объема данных. Нижестоящей задачей, решаемой здесь, является ASR, но ту же предварительно обученную модель можно настроить и для других задач, таких как распознавание говорящего и определение языка. Код ниже демонстрирует каждый этап выполнения вывода с помощью модели *Wav2Vec2* (обратите внимание, что вы также можете использовать `pipeline()` в качестве высокоуровневого API).

```
import torch
from transformers import Wav2Vec2ForCTC, Wav2Vec2Processor

from genaibook.core import get_device

device = get_device()

#Wav2Vec2Processor имеет встроенную предварительную и постобработку
wav2vec2_processor = Wav2Vec2Processor.from_pretrained(
    "facebook/wav2vec2-base-960h"
)
wav2vec2_model = Wav2Vec2ForCTC.from_pretrained(
    "facebook/wav2vec2-base-960h"
).to(device)
```

⁵ Дедупликация (или дедубликация) — это процесс обнаружения и устранения повторяющихся фрагментов информации. Вместо хранения нескольких копий дедупликация обеспечивает сохранение только одной копии уникальных данных. Избыточные копии заменяются ссылками или указателями на единственный экземпляр, хранящийся в системе. — *Пер.*

```
#Выполняем прямой проход, убедившись, что
#частота дискретизации установлена на 16 кГц
inputs = wav2vec2_processor(
    array, sampling_rate=sampling_rate, return_tensors="pt"
)
with torch.inference_mode():
    outputs = wav2vec2_model(**inputs.to(device))

#Транскрибируем
predicted_ids = torch.argmax(outputs.logits, dim=-1)
transcription = wav2vec2_processor.batch_decode(predicted_ids)
print(transcription)

['THE SECOND IN IMPORTANCE IS AS FOLLOWS SOVEREIGNTY MAY BE DEFINED TO']
```



Поскольку модели предварительно обучаются на данных с определенной частотой дискретизации, в выводе необходимо использовать аудио с той же частотой. Этого можно добиться с помощью повторной выборки данных (например, с помощью функции `dataset.cast_column("audio", Audio(sampling_rate=16_000))`) или через указание частоты дискретизации в объекте `processor`, как это сделано в предыдущем фрагменте кода.

Модель HuBERT следует той же концепции предварительного обучения для изучения полезных латентных представлений, но изменяет процесс обучения, чтобы использовать исходную MLM-цель из модели BERT. В то время как Wav2Vec2 предсказывает символы, HuBERT обрабатывает сигнал методами кластеризации, чтобы изучить дискретные латентные речевые единицы, которые можно рассматривать как эквивалент токенов в обработке естественного языка. Модель предсказывает эти скрытые единицы на втором этапе в случайно замаскированных местах.

Обратите внимание, Wav2Vec2 и HuBERT работают только для английского языка. Через несколько недель после выпуска Wav2Vec2 компания Meta выпустила модель XLSR-53, имеющую ту же архитектуру, что и Wav2Vec2, но предварительно обученную на 56 000 часах речи на 53 языках. XLSR изучает речевые единицы, общие для нескольких языков. Следовательно, даже на языках с небольшим количеством цифровых данных можно добиться неплохих результатов. В 2021 году компания Meta выпустила XLS-R (отличается от XLSR) — модель с 2 миллиардами параметров, обученную по аналогичной схеме, но с использованием почти в 10 раз большего количества немаркированных данных (436 000 часов) на 128 языках.

Важно отметить, что эти модели являются акустическими: их выходные данные основаны исключительно на звуковом сигнале и не содержат внутренней языковой информации. Например, они могут часто допускать орфографические ошибки, выводить что-то не похожее на слова или путать омофоны (например, «beag» и «bage»). Один из способов решить эту проблему — включить языковую информацию на этапе генерации.

Обычно на этапе классификации мы вычисляем функцию `argmax(logits)` для прогнозирования наиболее вероятного символа. Однако есть иной подход. Он заключается

во внедрении языковой модели, которая может прогнозировать наиболее вероятное слово по последовательности символов. Модель n -грамм — это тип языковой модели, которая предсказывает вероятность появления слова на основе $n - 1$ предыдущего слова в заданном корпусе текстов. Например, модель-биграмм ($n = 2$) рассматривает пары слов, а модель-триграмм ($n = 3$) — триплеты. Такая вероятностная модель помогает понять контекст и структуру языка и может быть очень компактной и эффективной. Также может быть использована полноценная языковая модель на основе трансформеров, подобная описанной в главе 2. Однако модели n -грамм помогают достигать большинства улучшений качества, которые они могли бы дать, при значительно меньших затратах на вычисления, память и время вывода.

Оценка n -грамм может быть добавлена к лучевому поиску для генерации k наиболее вероятных текстовых последовательностей. Интеграция языковой модели в лучевой поиск позволяет улучшить процесс декодирования. Лучевой поиск оценивает как акустическую, так и языковую модели, что помогает исправлять орфографические ошибки и отфильтровывать бессмысленные слова. Такой комбинированный подход значительно повышает точность и связность сгенерированного текста.



Возможно, вам может потребоваться увеличить вероятность появления определенных слов (например, если некоторые из них отсутствуют в языковой модели или вам нужно увеличить объем данных, специфичных для предметной области). Для этого можно подсчитать количество «горячих» слов в результатах поиска и увеличить вероятность появления.

Техники на основе энкодер-декодера

Использование моделей-энкодеров с CTC — один из популярных подходов к распознаванию речи. Как уже обсуждалось, акустические модели могут генерировать ошибки, для которых необходимо использовать модели n -грамм. Эти проблемы приводят к более тщательному исследованию архитектур энкодеров и декодеров.

Мы можем сформулировать ASR как задачу последовательного распознавания, а не классификации. Именно это и выполняет модель Whisper, представленная в начале этого раздела. В отличие от Wav2Vec2 или HuBERT, Whisper (рис. 9.13) обучалась в контролируемой среде на огромном объеме размеченных данных (более 680 000 часов аудио с соответствующим текстом). Для сравнения: Wav2Vec2 обучалась на менее чем 60 000 часов неразмеченных данных. Около трети данных являются многоязычными, что позволяет Whisper выполнять распознавание для 96 языков. Учитывая, что модель обучалась на размеченных данных, она осваивает преобразование речи в текст непосредственно во время предварительного обучения, не требуя тонкой настройки. Еще одна особенность Whisper заключается в том, что она обучена работать без маски внимания, она может напрямую определять, где следует игнорировать входные данные.

Как Whisper осуществляет вывод? В отличие от Wav2Vec2, она работает со спектрограммами. Сначала пакет выборок дополняется и/или усекается, чтобы обеспечить одинаковую длину входных данных, которые затем преобразуются в логариф-

мические мел-спектрограммы, а входные данные обрабатываются с помощью CNN перед передачей их в энкодер. Затем выходные данные из энкодера передаются декодеру, который предсказывает последующий токен (по одному за раз) авторегрессионным методом (как это делает, например, Llama), пока не будет сгенерирован конечный токен. Несмотря на то что архитектура энкодер-декодер может быть медленнее, чем подходы, использующие только энкодер, Whisper может обрабатывать длинные аудиофрагменты, предсказывать пунктуацию и не требует дополнительной языковой модели при выводе.

В моделях, использующих только энкодер, включение дополнительной языковой модели необходимо, чтобы устранить орфографические ошибки, генерируемые акустическими моделями, что часто требует использования внешней модели n-грамм. В случае Whisper декодер выполняет двойную функцию: генерирует текстовые транскрипции, одновременно выполняя функции языковой модели⁶. Возникает вопрос, почему декодер работает как языковая модель? Обучаясь предсказывать последующий токен в последовательности транскрипции на основе контекстной информации из энкодера, Whisper устраняет необходимость во внешней языковой модели во время вывода.



результату. Это похоже на методы обусловливания, которые мы рассматривали в предыдущих главах. Вот некоторые из наиболее важных токенов:

- ◆ Речь начинается с токена `start of transcript`.
- ◆ Если язык не английский, используется токен языкового тега (например, `hi` для хинди).
- ◆ С помощью языкового тега мы можем выполнить идентификацию языка, транскрипцию или перевод на английский.
- ◆ Если токена `speech` нет, Whisper используется для обнаружения голосовой активности.

Рассмотрим пример на испанском языке в соответствующем формате.

```
from transformers import WhisperTokenizer

tokenizer = WhisperTokenizer.from_pretrained(
    "openai/whisper-small", language="Spanish", task="transcribe"
) ❶

input_str = "Hola, ¿cómo estás?"
labels = tokenizer(input_str).input_ids ❷
decoded_with_special = tokenizer.decode(
    labels, skip_special_tokens=False
) ❸

decoded_str = tokenizer.decode(labels, skip_special_tokens=True) ❹

print(f"Input: {input_str}")
print(f"Formatted input w/ special: {decoded_with_special}")
print(f"Formatted input w/out special: {decoded_str}")
'Input: Hola, ¿cómo estás?'
'Formatted input w/ special: '
'<|startoftranscript|><|es|><|transcribe|><|notimestamps|>Hola, '
'¿cómo estás?<|endoftext|>'
'Formatted input w/out special: Hola, ¿cómo estás?'
```

Рассмотрим код подробнее:

- ❶ Загружаем предварительно обученный токенизатор. Поскольку Whisper требует добавления некоторых токенов вроде идентификатора языка или задачи, нам необходимо указать эти параметры.
- ❷ Токенизируем входную строку.
- ❸ Декодируем идентификаторы токенов обратно в исходную строку, но с учетом специальных токенов.
- ❹ Декодируем идентификаторы токенов обратно в исходную строку, но без учета специальных токенов.

Создание транскрипций с помощью Whisper не сильно отличается от использования Wav2Vec2:

- ◆ Мы используем процессор для подготовки аудио к ожидаемому моделью формату. В этом случае мел-спектрограммы извлекаются из исходной речи и обрабатываются для подготовки к использованию моделью.
- ◆ Модель генерирует идентификаторы токенов, соответствующие транскрипциям.
- ◆ Процессор декодирует идентификаторы и преобразует их в понятные человеку строки.

Компания OpenAI выпустила девять вариантов модели Whisper, содержащих от 39 миллионов до 1,5 миллиарда параметров, с контрольными точками для многоязычных и англоязычных конфигураций. В нашем примере мы будем использовать промежуточную, небольшую многоязычную модель, которая может работать с 2 ГБ видеопамати и в шесть раз быстрее самой большой модели.



Сейчас продолжают выпускаться новые версии большой модели Whisper. В рамках проекта Distil Whisper разработаны высококачественные уменьшенные версии оригинальных моделей, которые работают до шести раз быстрее и являются на 49% компактнее.

```
from transformers import WhisperForConditionalGeneration, WhisperProcessor
```

```
whisper_processor = WhisperProcessor.from_pretrained("openai/whisper-small")
```

```
whisper_model = WhisperForConditionalGeneration.from_pretrained(
    "openai/whisper-small"
).to(device)
```

```
inputs = whisper_processor(
    array, sampling_rate=sampling_rate, return_tensors="pt"
)
```

```
with torch.inference_mode():
    generated_ids = whisper_model.generate(**inputs.to(device))
```

```
transcription = whisper_processor.batch_decode(
    generated_ids, skip_special_tokens=False
)[0]
print(transcription)
```

```
('<|startoftranscript|><|en|><|transcribe|><|notimestamps|> The '
 'second in importance is as follows. Sovereignty may be defined to '
 'be<|endoftext|>')
```

Это хорошая возможность углубиться в объект `WhisperProcessor`. Чтобы лучше понять предыдущий код, мы предлагаем следующее:

- ◆ Ознакомиться с его документацией.
- ◆ Определить два компонента процессора.
- ◆ Определить и изучить выходные данные процессора (входные данные в предыдущем коде).

От модели к конвейеру

В предыдущих разделах вы узнали о различных архитектурах и подходах к реализации ASR. Однако при их использовании в производственных сценариях все еще существуют три проблемы, которые необходимо решить:

◆ *Длинная транскрипция аудиоданных*

Трансформеры обычно имеют конечную длину входных данных, которые они могут обработать. Например, Wav2vec2 использует механизм внимания, который имеет квадратичную сложность. Whisper не имеет механизма внимания, но она разработана для работы с записями длительностью до 30 секунд и обрезает более длинные файлы. Простой подход решить эту проблему — разбить аудиофайлы на более короткие фрагменты, выполнить на них вывод, а затем реконструировать выходной сигнал. Несмотря на эффективность, это может привести к ухудшению качества на границе разбиения. Чтобы сделать это, мы можем разбить аудиофайлы на фрагменты с определенным шагом (т. е. у нас будут перекрывающиеся фрагменты), а затем объединить их в цепочку. Результат будет отличаться от того, который модель предсказала бы для полного аудиофайла, но он, скорее всего, будет очень на него похож. Мы можем объединить фрагменты в пакет и пропустить через модель параллельно, что является более эффективным, чем последовательная транскрипция всего аудиофайла. Разделение на фрагменты и их пакетирование можно быстро выполнить с помощью параметров `chunk_length_s` и `batch_size` конвейера ASR.

◆ *Вывод в реальном времени*

Выполнение автоматического распознавания речи в реальном времени приветствуется во многих приложениях. Теперь, когда вы научились разбиению, мы можем использовать модель с небольшими фрагментами (например, 5 секунд) и шагом в 1 секунду. Вывод в реальном времени с использованием моделей CTC будет быстрее, поскольку эта архитектура выполняет один проход по сравнению с моделями, включающими декодеры. Несмотря на то что Whisper будет работать медленнее, вы можете использовать ту же логику разделения для транскрибирования фрагментов по мере их поступления и получения надежных результатов.

◆ *Временные метки*

Наличие временных меток, указывающих время начала и окончания коротких аудиофрагментов, может быть полезно, чтобы выровнять транскрипцию с входным аудио. Например, если вы генерируете субтитры во время видеозвонка, вам нужно знать, к какому временному сегменту относится каждая транскрипция. Это легко сделать с помощью параметра `return_time`. «Под капотом» мы знаем контекстное окно каждого выведенного токена и частоту дискретизации для моделей CTC.

Объединим все это с более длинным (1 минута) аудио.

```
from genaibook.core import generate_long_audio

long_audio = generate_long_audio()
```

```

device = get_device()

pipe = pipeline(
    "automatic-speech-recognition", model="openai/whisper-small", device=device
)

pipe(
    long_audio,
    generate_kwargs={"task": "transcribe"},
    chunk_length_s=5,
    batch_size=8,
    return_timestamps=True,
)

{'chunks': [{'text': ' the second in importance is as follows.',
              'timestamp': (0.0, 3.0)},
            {'text': ' Sovereignty may be defined to be the right of '
                    'making laws.',
              'timestamp': (3.0, 6.33)},
            {'text': ' In France, the king really exercises a '
                    'portion of the sovereign power, since the laws '
                    'have no weight till he has given his assent to '
                    'them.',
              'timestamp': (6.33, 16.89)},
            {'text': ' He is moreover the executor of the laws, but '
                    'he does not really cooperate in their '
                    'formation since the refusal of his assent does '
                    'not annul them. He is therefore merely to be '
                    'considered as the agent of the sovereign '
                    'power.',
              'timestamp': (16.89, 36.61)},
            {'text': ' But not only does the king of France exercise '
                    'a portion of the sovereign power, He also '
                    'contributes to the nomination of the '
                    'legislature, which exercises the other '
                    'portion. He has the privilege of appointing '
                    'the members of one chamber and of dissolving '
                    'the United States has no share in the '
                    'formation of the legislative body',
              'timestamp': (36.61, 59.75)},
            {'text': ' and cannot dissolve any part of it. The king '
                    'has the same right of bringing forward '
                    'measures as the chambers.',
              'timestamp': (59.75, 67.09)},
            {'text': ' A right which the president does not possess.',
              'timestamp': (67.09, 70.43)}],
 'text': ' the second in importance is as follows. Sovereignty may '
         'be defined to be the right of making laws. In France, the '

```


'king really exercises a portion of the sovereign power, '
 'since the laws have no weight till he has given his assent '
 'to them. He is moreover the executor of the laws, but he '
 'does not really cooperate in their formation since the '
 'refusal of his asset does not annul them. He is therefore '
 'merely to be considered as the agent of the sovereign '
 'power. But not only does the king of France exercise a '
 'portion of the sovereign power, He also contributes to the '
 'nomination of the legislature, which exercises the other '
 'portion. He has the privilege of appointing the members of '
 'one chamber and of dissolving the United States has no '
 'share in the formation of the legislative body and cannot '
 'dissolve any part of it. The king has the same right of '
 'bringing forward measures as the chambers. A right which '
 'the president does not possess

Вы можете заметить, что расшифровка в целом является точной, но одно или два предложения, присутствующие в аудиозаписи, в ней отсутствуют. Это распространенная проблема для небольших моделей Whisper. Поскольку они являются генеративными (генерируют текст, а не напрямую классифицируют звуки по токенам), то иногда могут пропускать слова или даже искажать содержание. Далее обсудим некоторые стратегии оценки этих моделей.

Оценка

При таком большом количестве моделей, способных выполнять аудиозадачи, выбор подходящего экземпляра может быть сложным. Для предварительно обученных моделей мы обычно оцениваем производительность нескольких задач по нисходящей. Хотя наиболее распространенной задачей для оценки здесь является ASR, мы также можем оценивать предварительно обученные модели, доработанные для других задач, таких как обнаружение ключевых слов, классификация намерений или идентификация говорящего. Помимо производительности, также необходимо учитывать другие факторы, такие как размер модели, скорость вывода, языки, для которых она была обучена, и близость обучающих данных к выводу. Например, если вы работаете с определенным акцентом, вам может потребоваться тонкая настройка на основе данных, которые его содержат. Если вы хотите сделать вывод в режиме реального времени, вам потребуется уменьшенная версия.

В этом разделе мы проведем общую оценку различных моделей распознавания английской речи. Несмотря на то что это не обеспечит сквозной оценочной структуры для выбора лучшей модели с вашим вариантом использования, зато даст некоторое представление о практических способах анализа производительности. Далее мы оценим две небольшие многоязычные модели: версию многоязычной модели Whisper с 74 миллионами параметров и версию модели Wav2Vec2 с 94 миллионами параметров. Мы можем сравнить скорость вывода и пиковую мощность графического процессора, используемого для запуска.

```

from genaibook.core import measure_latency_and_memory_use

wav2vec2_pipe = pipeline(
    "automatic-speech-recognition",
    model="facebook/wav2vec2-base-960h",
    device=device,
)

whisper_pipe = pipeline(
    "automatic-speech-recognition", model="openai/whisper-base", device=device
)

with torch.inference_mode():
    measure_latency_and_memory_use(
        wav2vec2_pipe, array, "Wav2Vec2", device, nb_loops=100
    )
    measure_latency_and_memory_use(
        whisper_pipe, array, "Whisper", device=device, nb_loops=100
    )

```

Wav2Vec2 execution time: 0.009195491333007812 seconds

Wav2Vec2 max memory footprint: 1.7330821120000002 GB

Whisper execution time: 0.092218232421875 seconds

Whisper max memory footprint: 1.6933248 GB

Неудивительно, что максимальный объем памяти (объем используемой видеопамати) обеих моделей очень схож, что вполне логично, учитывая схожее количество параметров. Потребляемая мощность в 1,7 ГБ — относительно невелика, поскольку модель может работать на ноутбуках и даже на некоторых мощных смартфонах. Wav2Vec2 значительно быстрее, что вполне ожидаемо, учитывая, что декодер Whisper генерирует текст по одному токenu за раз.

Посмотрим, насколько хороши обе модели с точки зрения качества предсказаний. Наиболее распространенной метрикой для оценки является *коэффициент ошибок в словах* (**Word Error Rate, WER**), который рассчитывает количество ошибок, анализируя различия между предсказанием и исходной меткой. Ошибка определяется на основе количества замен, вставок и удалений, необходимых для перехода от предсказания к метке. Например, если на входе будет фраза «how can the llama jump», а на выходе предсказание «can the lama jump up», мы получим следующую картину:

- ◆ Одно удаление, например отсутствует слово «how».
- ◆ Одна замена, например слово «llama» заменено на «lama».
- ◆ Одна вставка, например слово «up» присутствует только в прогнозе.

В WER вычисляется сумма ошибок, деленная на количество слов в метке. Таким образом, коэффициент равен 0,6 (три ошибки, деленные на пять предсказаний). Обратите внимание, несмотря на то, что между словами «llama» и «lama» различается всего один символ, это считается полной ошибкой. Многие альтернативные метри-

ки, такие как *частота ошибок в символах (Character Error Rate, CER)*, оценивают разницу на основе каждого символа. Тем не менее WER широко используется в качестве основной метрики для оценки ASR. Библиотека `evaluate` предоставляет высокоуровневый API-интерфейс для применения этих метрик. Далее мы разберемся на примерах, как загрузить и рассчитать WER.

```
from evaluate import load

wer_metric = load("wer")

label = "how can the llama jump"
pred = "can the lama jump up"
wer = wer_metric.compute(references=[label], predictions=[pred])

print(wer)

0.6
```

Второй аспект, который следует учитывать перед оценкой, заключается в том, что разные модели ASR имеют разные форматы вывода, основанные на обучающих данных. Например, Whisper обучалась с учетом регистра и знаков препинания, поэтому ее транскрипции их также содержат. Справедливости ради при оценке моделей можно нормализовать метки и прогнозы перед вычислением WER. Этот способ неидеален, поскольку модель, учитывающая регистр и знаки препинания, даст ошибку меньше, чем модель, которая их не учитывает, но это является хорошей отправной точкой.

Библиотека `transformers` предлагает нормализаторы (`BasicNormalizer`, `EnglishTextNormalizer` и др.). `BasicTextNormalizer` удаляет подряд идущие пробелы и основные знаки препинания, а также переводит текст в нижний регистр. `EnglishNormalizer` — более продвинутый инструмент, стандартизирующий числа (от «миллиона» до «1 000 000»), управляющий сокращениями и другими функциями. Мы воспользуемся `BasicNormalizer`. Он отлично справится с нашей задачей. Важно, чтобы мы единообразно нормализовали весь текст и транскрипции.

```
from transformers.models.whisper.english_normalizer import BasicTextNormalizer

normalizer = BasicTextNormalizer()
print(normalizer("I'm having a great day!"))

i m having a great day
```

Мы будем оценивать модели, используя *Common Voice*, — популярный многоязычный набор данных, собранный с помощью краудсорсинга. Для демонстрации мы будем использовать часть тестовых сплитов набора данных на английском и французском языках, но мы могли бы сделать это и на других языках (хотя нам нужно быть осторожными с токенизацией). Мы оценим WER и CER на 200 примерах для каждого языка.



Набор Common Voice является общедоступным, но вам все равно необходимо принять условия его использования и сообщить свое имя и адрес электронной почты его создателю — Mozilla Foundation. Вы можете легко выполнить это, посетив официальную страницу набора и подтвердив свое согласие. Если вы не согласны с условиями, можете использовать другой набор или свои собственные данные для оценки.

Начнем с реализации конвейера оценки.

#Этот пример кода оптимизирован для наглядности

#Например, вывод можно выполнять пакетно для ускорения

```
from datasets import Audio
```

```
def normalize(batch): ❶
    batch["norm_text"] = normalizer(batch["sentence"])
    return batch

def prepare_dataset(language="en", sample_count=200):
    dataset = load_dataset(
        "mozilla-foundation/common_voice_13_0",
        language,
        split="test",
        streaming=True,
    ) ❷
    dataset = dataset.cast_column("audio", Audio(sampling_rate=16000)) ❸
    dataset = dataset.take(sample_count) ❹
    buffered_dataset = [sample for sample in dataset.map(normalize)] ❺

    return buffered_dataset

def evaluate_model(pipe, dataset, lang="en", use_whisper=False):
    predictions, references = [], []

    for sample in dataset:
        if use_whisper:
            extra_kwargs = {
                "task": "transcribe",
                "language": f"<|{lang}|>",
                "max_new_tokens": 100,
            } ❻
            transcription = pipe(
                sample["audio"]["array"],
                return_timestamps=True,
                generate_kwargs=extra_kwargs,
            )
        else:
            transcription = pipe(sample["audio"]["array"])
```

```

predictions.append(normalizer(transcription["text"]))
references.append(sample["norm_text"])
return predictions, references

```

В этом коде мы делаем следующее:

- ❶ Реализуем функцию для нормализации пакета с использованием нормализации Whisper English.
- ❷ Загружаем набор данных Common Voice в потоковом режиме.
- ❸ Изменяем частоту дискретизации аудиоданных до 16 кГц.
- ❹ Делаем выборку 200 элементов из набора данных.
- ❺ Нормализуем набор данных. Мы также буферизируем его, чтобы он хранился в списке, а не в потоковом режиме.
- ❻ В Whisper мы добавляем дополнительные параметры для его генерации (например, указываем язык и задачу).

Теперь, когда конвейер оценки готов к работе, попробуем его в деле с двумя моделями и двумя языками. Сначала укажем набор.

```

eval_suite = [
    ("Wav2Vec2", wav2vec2_pipe, "en"),
    ("Wav2Vec2", wav2vec2_pipe, "fr"),
    ("Whisper", whisper_pipe, "en"),
    ("Whisper", whisper_pipe, "fr"),
]

```

Теперь, когда все компоненты на месте, проведем оценку.

```
cer_metric = load("cer")
```

#Предварительная обработка английского и французского наборов данных

```

processed_datasets = {
    "en": prepare_dataset("en"),
    "fr": prepare_dataset("fr"),
}

```

```
for config in eval_suite:
```

```
    model_name, pipeline, lang = config[0], config[1], config[2]
```

```
    dataset = processed_datasets[lang]
```

```

    predictions, references = evaluate_model(
        pipeline, dataset, lang, model_name == "Whisper"
    )

```

```
#Вычисляем оценочные метрики
```

```

wer = wer_metric.compute(references=references, predictions=predictions)
cer = cer_metric.compute(references=references, predictions=predictions)

```

```
print(f"(model_name) metrics for lang: {lang}. WER: {wer}, CER: {cer}")
```

```
Reading metadata...: 16372it [00:00, 26197.69it/s]
```

```
Reading metadata...: 16114it [00:00, 36235.41it/s]
```

```
Wav2Vec2 metrics for lang: en. WER: 0.44012772751463547, CER: 0.22138
```

```
Wav2Vec2 metrics for lang: fr. WER: 1.0099113197704748, CER: 0.57450
```

```
Whisper metrics for lang: en. WER: 0.2687599787120809, CER: 0.14674
```

```
Whisper metrics for lang: fr. WER: 0.5477308294209703, CER: 0.27584
```

Посмотрим на результаты:

- ◆ Whisper явно превосходит Wav2Vec2 как на английском, так и на французском языке (чем меньше ошибка, тем лучше).
- ◆ Несмотря на то что показатель WER в Wav2Vec2 для английского языка высок, можно заметить, что CER значительно ниже. Как уже упоминалось, Wav2Vec2 — это акустическая модель, и она может генерировать орфографические ошибки. Whisper же, напротив, использует глубокое языковое моделирование, поэтому с большей вероятностью генерирует правильные слова.
- ◆ Превосходство Whisper над Wav2Vec2 у французского языка вполне закономерно, поскольку первая модель обучалась на многоязычных данных, в то время как вторая — на англоязычных.

Можно было бы поэкспериментировать с более крупным вариантом модели Whisper и еще больше снизить показатель WER для французского языка (самый большой вариант в 20 раз больше базовой версии). Еще один интересный аспект заключается в том, что Whisper показала себя лучше, чем Wav2Vec2 на английском языке. Whisper многоязычна, поэтому логично ожидать, что на английском языке она будет работать хуже, чем модель, настроенная исключительно на английских данных. Кроме того, OpenAI также выпустила экземпляр модели Whisper того же размера, но полностью обученный на английских данных. Если бы мы хотели ввести Whisper в эксплуатацию и знали, что все пользователи будут говорить по-английски, то было бы целесообразнее перейти на англоязычный вариант.

Помните, что Whisper — это авторегрессионная модель. Из-за этого она может «галлюцинировать», а иногда и продолжать генерировать токены. Это одна из основных проблем по сравнению с подходами, основанными только на энкодере. Учитывая, что показатель WER не имеет верхней границы, одна галлюцинация может значительно его увеличить. Один из вариантов решений — ограничить максимальное количество генерируемых токенов. Поскольку Whisper предназначена для транскрибирования 30-секундных сегментов, можно предположить, что это примерно 60 слов или около 100 токенов. Но есть и другой способ — настроить модель на подавление галлюцинаций, принудительно возвращая временные метки. В первоначальном эксперименте со 100 выборками показатель WER снизился с 1,72 до 0,84 только благодаря применению этого метода⁷. Если галлюцинации являются

⁷ Опытным путем было установлено, что настройка `return_timestamps=True` помогает уменьшить галлюцинации при оценке длинных форм. Подробное объяснение этого есть на форуме Hugging Face.

проблемой, то почему Whisper превзошла Wav2Vec2 в предыдущей оценке? Ответ кроется в сочетании языкового моделирования модели, размеченных данных и огромного объема информации, использованной для предварительного обучения.

Мы рекомендуем потратить немного времени на следующие эксперименты:

- ♦ Попробуйте разные размеры моделей (например, Whisper Tiny с 39 миллионами параметров, Large V2 с 1,5 миллиарда параметров или дистиллированные варианты).
- ♦ Попробуйте разные модели, например Wav2Vec2, настроенную на французский язык, или вариант Whisper, обученный только на английских данных.
- ♦ Попробуйте использовать разные параметры генерации.



Если вас интересует тема оценки ASR, рекомендуем ознакомиться со статьей «ESB: A Benchmark For Multi-Domain End-to-End Speech Recognition». Бенчмарк, описанный в ней, предлагает сравнение нескольких сквозных систем с использованием единой оценки, основанной на множестве наборов данных. Также существует общедоступная таблица лидеров с открытым исходным кодом для моделей распознавания речи, которая регулярно обновляется.

Это было увлекательное погружение в технологии распознавания речи. Далее мы рассмотрим, как сделать обратное: преобразовать текст в речь, а затем обобщить эти знания для генерации звука.

Генерация аудио

Как делать высококачественные транскрипции с помощью моделей на основе трансформеров, мы уже изучили. В этой части главы пришло время подробно рассмотреть методы генерации аудио, их оценку и связанные с ними сложности. Вы познакомитесь с двумя популярными моделями text-to-speech (TTS) — SpeechT5 и Bark. Затем мы кратко обсудим модели, которые могут выходить за рамки речи и способны генерировать другие формы аудио (например, музыку): например, MusicGen, AudioLDM и AudioGen. В завершение мы рассмотрим, как использовать модели диффузии для генерации аудио с помощью моделей Riffusion и Dance Diffusion⁸.

Обучение и оценка моделей text-to-audio (TTA) с нуля могут быть довольно дорогостоящим и сложным процессом. В отличие от ASR, верных прогнозов в модели TTA может быть множество. Представьте себе модель TTS⁹. Ее генерации могут иметь разные произношения, акценты и стили речи, и все они могут быть правильными. Кроме того, популярные наборы данных, используемые для ASR, такие как Common Voice, часто содержат шум. К сожалению, шум в наборах данных — нежелательное свойство для TTA, поскольку модели также учатся генерировать и

⁸ Выбор моделей для этого раздела был основан на многих факторах, включая их популярность, размер и качество. SpeechT5 является универсальной и подходит для решения различных задач. Bark — одна из лучших открытых моделей для создания выразительной речи. Методы диффузии, хотя и используются не так часто, имеют периоды высокой популярности.

⁹ Напомним, что TTS — это разновидность TTA.

его. Представьте, что ваша сгенерированная речь звучит на фоне лая собаки или автомобильного гудка. Поэтому обучающие наборы для ТТА должны быть высокого качества.

Генерация звука с помощью моделей Sequence-to-Sequence

SpeechT5 — это предварительно обученная открытая модель от Microsoft, способная выполнять задачи speech-to-text, например ASR и *преобразование голоса* (**Speaker conversion**), задачи speech-to-speech, например улучшение и преобразование голоса, а также задачи TTS. SpeechT5, архитектура которой представлена на рис. 9.14, использует схему энкодер-декодер, где входные и выходные данные могут быть как речью, так и текстом. Для управления вводом различных модальностей сети предварительной обработки (предварительные сети) речи и текста преобразуют входные данные в скрытое представление, которое впоследствии поступает в энкодер. Иначе говоря, предварительные сети энкодера преобразуют входные сигналы и тексты в общее скрытое представление, используемое энкодером. Аналогичным образом ввод и вывод декодера подвергаются предварительной и постобработке с помощью предварительных сетей и сетей постобработки (постсетей) речи и текста. Это дает нам шесть дополнительных сетей, которые обеспечивают необходимую гибкость при выполнении нескольких задач с использованием одной и той же модели:

◆ *Предварительная сеть энкодера текста*

Это слой эмбедингов текста, который сопоставляется со скрытыми представлениями, ожидаемыми энкодером.

◆ *Предварительная сеть энкодера речи*

Тот же принцип, что и у экстрактора признаков из Wav2Vec2. Это сверточная нейросеть (CNN), которая предварительно обрабатывает входной сигнал.

◆ *Предварительная сеть декодера текста*

Во время предварительного обучения эта сеть работает аналогично предварительной сети текстового энкодера. При тонкой настройке она может модифицироваться.

◆ *Предварительная сеть декодера речи*

Берет логарифмическую спектрограмму и сжимает ее в скрытое представление.

◆ *Постсеть декодера текста*

Один линейный слой, проецирующий вероятности на словарный запас.

◆ *Постсеть декодера речи*

Предсказывает выходную спектрограмму и уточняет ее с помощью дополнительных сверточных слоев.

Рассмотрим пример где мы хотим выполнить ASR: на входе будет аудио, а на выходе — текст. В этом случае мы используем предварительную сеть энкодера речи,

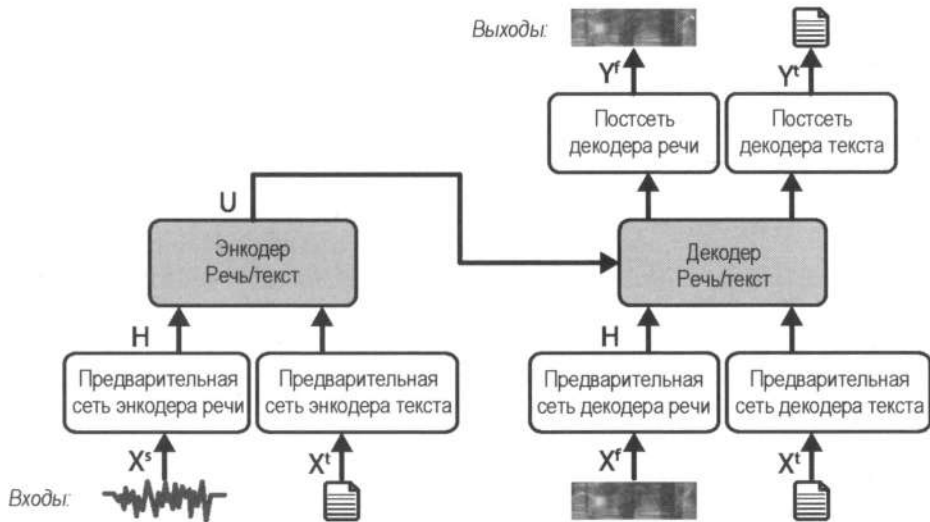


Рис. 9.14. Архитектура модели SpeechT5 (адаптирована из изображения в оригинальной статье)

а также обе сети декодера текста (предварительную и постсеть). Библиотека `transformers` содержит класс процессора (`SpeechT5Processor`), который предоставляет функции для обработки входов и выходов как для аудио, так и для текста. Структура очень похожа на ту, что представлена в разделе «Техники на основе энкодеров» в этой главе.

```
from transformers import SpeechT5ForSpeechToText, SpeechT5Processor

processor = SpeechT5Processor.from_pretrained("microsoft/speecht5_asr")
model = SpeechT5ForSpeechToText.from_pretrained("microsoft/speecht5_asr")

inputs = processor(
    audio=array, sampling_rate=sampling_rate, return_tensors="pt"
)

with torch.inference_mode():
    predicted_ids = model.generate(**inputs, max_new_tokens=70)

transcription = processor.batch_decode(predicted_ids, skip_special_tokens=True)
print(transcription)

['chapter sixteen i might have told you of the beginning i might '
 'have told you of the beginning of the beginning of the beginning '
 'of the beginning of the beginning chapter sixteen']
```

Далее нам нужно реализовать TTS. Ожидается, что ввод будет текстовый, а вывод — речевой, поэтому мы используем предварительную сеть энкодера текста и обе сети декодера речи (предварительную и постсеть). Чтобы TTS поддерживала множество голосов, ожидается, что на вход SpeechT5 поступят *голосовые эмбединги*. Они представляют информацию о говорящем, например его голос и акцент,

что впоследствии позволяет генерировать речь в этом стиле. Эмбединги извлекаются с помощью *x*-векторов. Это техника, которая сопоставляет входные аудиосигналы любой длины в эмбединг фиксированной размерности. *SpeechT5* эффективно использует *x*-вектор, объединяя его с выходом предварительной сети декодера речи, тем самым включая информацию о говорящем при декодировании.

Мы могли бы использовать эмбединг случайного говорящего (например, с помощью `torch.zeros(1, 512)`), но результаты будут очень зашумленными. К счастью, мы можем использовать некоторые существующие онлайн-эмбединги говорящих. Результат показан на рис. 9.15.

```
from transformers import SpeechT5ForTextToSpeech

from genaibook.core import get_speaker_embeddings

processor = SpeechT5Processor.from_pretrained("microsoft/speecht5_tts")
model = SpeechT5ForTextToSpeech.from_pretrained("microsoft/speecht5_tts")

inputs = processor(text="There are llamas all around.", return_tensors="pt")
speaker_embeddings = torch.tensor(get_speaker_embeddings()).unsqueeze(0)

with torch.inference_mode():
    spectrogram = model.generate_speech(inputs["input_ids"], speaker_embeddings)

plt.figure()
plt.imshow(np.rot90(np.array(spectrogram)))
plt.show()
```

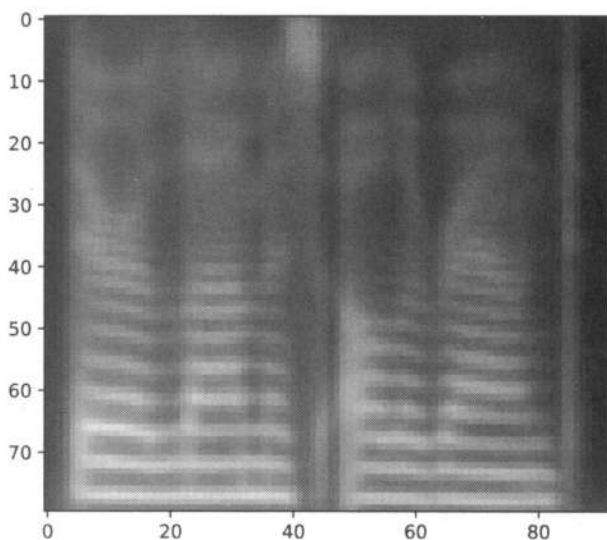


Рис. 9.15. Мел-спектрограмма при реализации задачи TTS

Как видите, выход модели представляет собой логарифмическую мел-спектрограмму, а не осциллограмму. Спектрограммы — мощный инструмент, но у них есть свои ограничения. Преобразование сигналов в спектрограммы с помощью КПФ (кратковременное преобразование Фурье) — простой процесс, чего нельзя сказать об обратном преобразовании спектрограмм в осциллограмму сигнала. К сожалению, спектрограммы не содержат всей информации, необходимой для восстановления исходного звука. Чтобы понять это ограничение, рассмотрим общую формулу синусоиды:

$$F(t) = A \sin(2\pi ft + \Phi).$$

Коэффициент A сообщает амплитуду, f — частоту, а t — время входного сигнала. Вся эта информация присутствует в спектрограмме, за исключением одного элемента — Φ . Он представляет фазу, которая предоставляет дополнительную информацию о сигнале. Несмотря на то что амплитуда и высота являются более важными характеристиками, информация о фазе необходима для точной реконструкции звука. Поэтому нам нужны методы реконструкции исходной формы сигнала (включая информацию о фазе) из спектрограммы. Это отличная возможность познакомиться с *вокодерами* (**Vocoder**).

Классическим подходом к реконструкции является *алгоритм Гриффина–Лима* (Griffin–Lim) — итерационный алгоритм, который восстанавливает форму сигнала из предсказанной спектрограммы. Он популярен благодаря своей простоте и скорости, но его применение может привести к низкому качеству звука. Алгоритм достаточно хорош для некоторых генераций спектрограмм, но, к сожалению, если взглянуть на спектрограмму, сгенерированную в предыдущем примере, можно заметить, что качество изображения может улучшиться. Использование классических методов для преобразования спектрограмм в сигналы приведет к появлению большого количества шума и артефактов, поэтому нам придется прибегнуть к более сложным методам, включающим нейронные сети.

Получение обучающих данных для реконструкции спектрограмм в сигнал является простым процессом. Он способствует исследованиям обучаемых моделей, называемых *нейронными вокодерами*, которые получают некоторые представления признаков или спектрограммы и преобразуют их в сигнал. WaveNet стала одной из первых. Это знаменитая модель от DeepMind, которая обеспечивает высококачественные реконструкции. WaveNet, несмотря на высокое качество, является авторегрессионной моделью с медленными результатами, что делает ее непригодной для использования в реальных условиях. Попытки повысить скорость WaveNet, к сожалению, требуют значительного улучшения оптимизации и мощных графических процессоров.

Подходы на основе сетей GAN являются популярной высококачественной альтернативой для реконструкции спектрограмм в сигнал в режиме реального времени. На высоком уровне они используют *технику состязательного обучения*, в которой модель *генератора* получает мел-спектрограммы и выводит сигналы, а модель *дискриминатора* определяет, достаточно ли близко качество звука к истинному значению. Это позволяет улучшить работу как генератора, так и дискриминатора. Модели MelGAN и HiFiGAN — популярные инструменты, основанные на сетях GAN.

Они быстро работают и поддаются распараллеливанию, выдают качество, сопоставимое с WaveNet, и реализованы с открытым исходным кодом. Мы будем использовать HiFiGAN в качестве вокодера для SpeechT5. Он представляет собой сверточную нейросеть с двумя дискриминаторами, которые помогают оценивать различные аспекты звука, помогая сверточной сети генерировать аудио более высокого качества.

Мы можем применить модель SpeechT5HiFiGan с трансформерами, передавая спектрограмму напрямую, или указать параметр `vocoder` при генерации речи.

```
from transformers import SpeechT5HiFiGan

vocoder = SpeechT5HiFiGan.from_pretrained("microsoft/speecht5_hifigan")
with torch.inference_mode():
    # В качестве альтернативы
    # model.generate_speech(
    #     inputs["input_ids"],
    #     speaker_embeddings,
    #     vocoder=vocoder)
    speech = vocoder(spectrogram)
```

Помните, что вы можете воспроизвести аудио через функцию `Audio(array, rate=sampling_rate)` или изучить на Hugging Face Hub некоторые скомпилированные нами генерации.

Еще раз напомним, мы генерируем спектрограмму с помощью SpeechT5 и преобразуем ее в сигнал с помощью HiFiGAN. SpeechT5 обучена на английском языке, поэтому на других языках она будет работать плохо. Ее можно тонко настроить для других языков, но она будет иметь некоторые ограничения, например поддержку символов за пределами английского. Тонкая настройка для других языков также требует получения эмбеддингов, говорящих для тех, кто не является носителем английского языка, поэтому ожидается, что модель будет работать хуже. Если вы экспериментировали с разными эмбеддингами голосов, то знаете, что качество результатов сильно зависит от них. Возможно, мы сможем добиться большего.

Идеальная конфигурация TTS — это единая модель (а не генератор спектрограмм и вокодер), которая имеет сквозное обучение, гибкость для работы с несколькими голосами, генерацию длинных аудиофайлов и быстрый вывод. Этим требованиям соответствует модель VITS, которая представляет собой параллельный сквозной метод. На высоком уровне VITS можно рассматривать как обусловленный VAE. Модель сочетает в себе несколько приемов, некоторые из которых нам уже знакомы. Она использует трансформер с одним энкодером в качестве основного и генератор HiFiGAN в качестве декодера. Другие компоненты помогают повысить качество и гибкость для решения задачи «один ко многим» в TTS. Во время обучения мы также сравнивали мел-спектрограммы, а не конечные необработанные формы сигнала при вычислении потерь реконструкции. Это помогает сосредоточиться на улучшении качества восприятия. Если вы помните, мел-спектрограммы приблизительно отражают то, как человек воспринимает звук. Модель стремится генерировать более различимую речь, включая ее в потери реконструкции.

VITS была выпущена в 2021 году компанией Kakao Enterprise и входила в число новейших моделей. В 2023 году компания Meta выпустила Massively Multilingual Speech (MMS) — массивный многоязычный набор данных. Он привел ко многим интересным результатам. Во-первых, набор содержит данные для идентификации среди 4000 разговорных языков (задача аудиоклассификации). Meta также опубликовала данные TTS и ASR для более чем 1100 языков. Авторы создали новую предварительно обученную модель Wav2Vec2 и выпустили многоязычную тонкую настройку ASR для 1100 языков. Но можно задаться вопросом, какое отношение все это имеет к TTS? Авторы MMS также обучили отдельные модели VITS для каждого языка, что привело к созданию множества высококачественных моделей TTS для большого количества языков, таких как вьетнамский и голландский. Модели показали лучшие результаты, чем исходные модели VITS. Это отличный пример того, как усовершенствование обучающих данных может привести к улучшению результатов. Далее мы будем использовать эту модель для генерации речи.

```
from transformers import VitsModel, VitsTokenizer, set_seed

tokenizer = VitsTokenizer.from_pretrained("facebook/mms-tts-eng")
model = VitsModel.from_pretrained("facebook/mms-tts-eng")

inputs = tokenizer(text="Hello - my dog is cute", return_tensors="pt")

set_seed(555) # make deterministic
with torch.inference_mode():
    outputs = model(inputs["input_ids"])

outputs.waveform[0]
```

Выход за рамки генерации речи с помощью Bark

Мы только что рассмотрели, как генерировать речь с помощью SpeechT5 и VITS. Эти модели применимы ко многим сценариям, но существуют и такие, которые выходят за рамки речевых задач. Например, вам может понадобиться модель, способная генерировать, например, смех или плач. Или вам также может быть необходима модель, способная генерировать песни.

Еще одна генеративная модель, которую мы рассмотрим, — это Bark (рис. 9.16) от компании Suno AI. Она построена на базе трансформера. Это одна из самых популярных открытых генеративных аудиомоделей. Она может генерировать не только речь, но и другие звуки. Например, можно добавить смех или вздох в промпт, и модель интегрирует соответствующие звуки в речь. Мы даже можем сделать ее немного музыкальной, используя символ «♪» в промптах. Bark — многоязычна и включает в себя библиотеку голосовых пресетов (аналогичных голосовым эмбедингам), что позволяет генерировать речь разными голосами.

Прежде чем подробно рассмотреть Bark, мы изучим еще одно понятие — *аудиокодеки*. При сжатии аудио цель состоит в уменьшении размера файла (битрейта) при условии сохранения максимально высокого качества звука. Исследователи годами пытались использовать нейросети для этого, что привело к появлению таких пере-

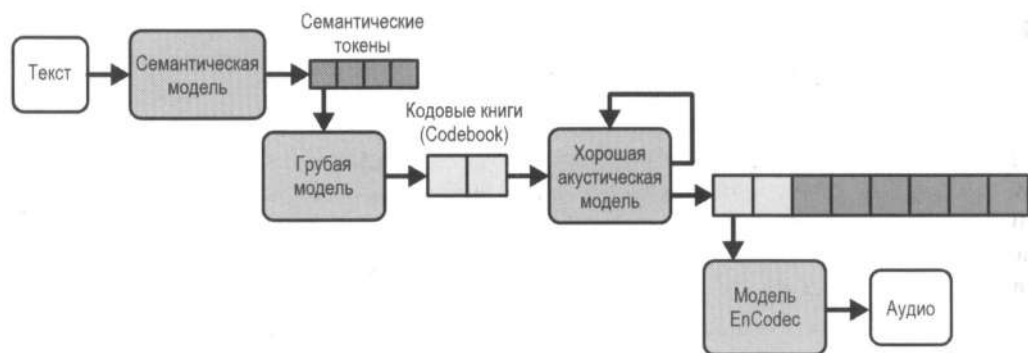


Рис. 9.16. Конвейер Bark использует три компонента: токены преобразования текста в семантику (text-to-semantic), токены преобразования семантических данных в грубые данные (semantic-to-coarse) и токены преобразования грубых данных в точные данные (coarse-to-fine)

довых инструментов, как SoundStream и EnCodec. EnCodec (разработан компанией Meta) — популярный нейронный кодек с открытым исходным кодом, способный сжимать аудио в реальном времени с высоким качеством.

EnCodec использует трехэтапный процесс. Сначала энкодер сжимает аудио в латентное представление. Затем слой квантования преобразует его в компактный, более эффективный формат (сжатое представление)¹⁰. Далее декодер восстанавливает аудио из сжатых данных. Квантованное латентное пространство представлено *кодowymi книгами (Codebook)*, каждая из которых содержит несколько возможных векторов. Например, входной аудиосигнал может быть представлен 32 векторами (каждый по 1024 элемента) в квантованном латентном пространстве.

Имея кодеки в нашем распоряжении, вернемся к Bark. Цель модели — получить текст и преобразовать его в кодовые книги. Затем она с помощью декодера нейронного кода преобразует кодовые книги в аудио. Это возможно благодаря четырем компонентам:

◆ *Текстовая модель*

Авторегрессивный трансформер-декодер с сетью моделирования языка, который преобразует текстовые промпты в высокоуровневые семантические токены. Благодаря этому Bark может обобщать звуки за пределами обучающих данных, например звуковые эффекты и тексты песен.

◆ *Грубая акустическая модель*

Имеет ту же архитектуру, что и у текстовой модели, но семантические токены из текстовой модели преобразуются в первые две кодовые книги с аудио.

◆ *Точная акустическая модель*

Это некаузальный автоэнкодер-трансформер, который итеративно предсказывает следующие кодовые книги на основе суммы исходных кодовых книг. Всего на выходе получаются две грубые кодовые книги и шесть сгенерированных.

¹⁰ Способ квантования выходных данных энкодера — это проектное решение. Авторы EnCodec использовали метод, называемый *остаточным векторным квантованием (Residual Vector Quantization)*.

◆ Кодек

После того как все восемь каналов кодовой книги предсказаны, часть декодера модели EnCodec декодирует выходной аудиомассив.

Существует множество настраиваемых параметров. Например, мы можем изменить количество кодовых книг, генерируемых грубой и точной акустическими моделями. Мы также можем изменить размер каждой кодовой книги, которая в официальной реализации составляет 1024. Такая архитектура позволяет генерировать новые звуки, речь на нескольких языках и многое другое. Попробуем это сделать.

```
from transformers import AutoModel, AutoProcessor

processor = AutoProcessor.from_pretrained("suno/bark-small")
model = AutoModel.from_pretrained("suno/bark-small").to(device)

inputs = processor(
    text=[
        ""Hello, my name is Suno. And, uh - and I like pizza. [laughs]
        But I also have other interests such as playing tic tac toe.""
    ],
    return_tensors="pt",
).to(device)

speech_values = model.generate(**inputs, do_sample=True)
```

Результаты получились отличными. Предположим, мы хотим настроить вывод звука заданным голосом. В этом случае мы также можем использовать эмбединги голосов из официальной библиотеки, предоставленной авторами. Кроме того, можно обучить эти эмбединги на вашем собственном голосе, что позволит генерировать синтетический звук, соответствующий вашей просодии. Попробуем использовать один из определенных голосов.

```
voice_preset = "v2/en_speaker_6"

inputs = processor("Hello, my dog is cute", voice_preset=voice_preset)

audio_array = model.generate(**inputs.to(device))
audio_array = audio_array.cpu().numpy().squeeze()
```

AudioLM и MusicLM

Управление генерацией речи с помощью дополнительных звуков — это интересная задача, но гораздо интереснее генерировать целые мелодии. AudioLM и MusicLM — две модели от Google, выпущенные в 2023 году, которые могут генерировать звуки и музыку. Они могут быть использованы в самых разных приложениях для создания шумовых эффектов в видео, добавления фоновой музыки в подкасты или создания звуков для игр.

AudioLM может получать на вход аудиозапись длительностью несколько секунд, а затем генерировать высококачественные продолжения, которые способны сохра-

нять индивидуальность и манеру речи говорящего. Модель обучается без транскрипций и аннотаций, что делает ее весьма впечатляющей. Рассмотрим, как AudioLM достигает этого. Концептуально модель похожа на Bark, но у нее другая задача — продолжить звук, а не выполнить ТТА.

AudioLM (рис. 9.17) сначала использует w2v-BERT — предварительно обученную модель, которая сопоставляет сигнал с расширенными семантическими токенами (аналогично тому, как мы использовали языковую модель для генерации семантических токенов из текста в Bark). Затем семантическая модель предсказывает будущие токены, моделируя высокоуровневую структуру аудиопоследовательности. Затем другая модель (грубая акустическая) использует сгенерированные семантические и предыдущие акустические токены для генерации новых. Прошлые акустические токены генерируются с помощью передачи входного сигнала кодеку и извлечения кодовых книг (квантованное латентное представление). Это помогает сохранять характеристики говорящего и генерировать более когерентный звук. Далее точная акустическая модель добавляет больше деталей к звуку, очищая его, улучшая качество и удаляя артефакты сжатия при потерях, возникшие на предыдущих этапах. В завершение токены передаются в нейронный кодек для реконструкции сигнала.

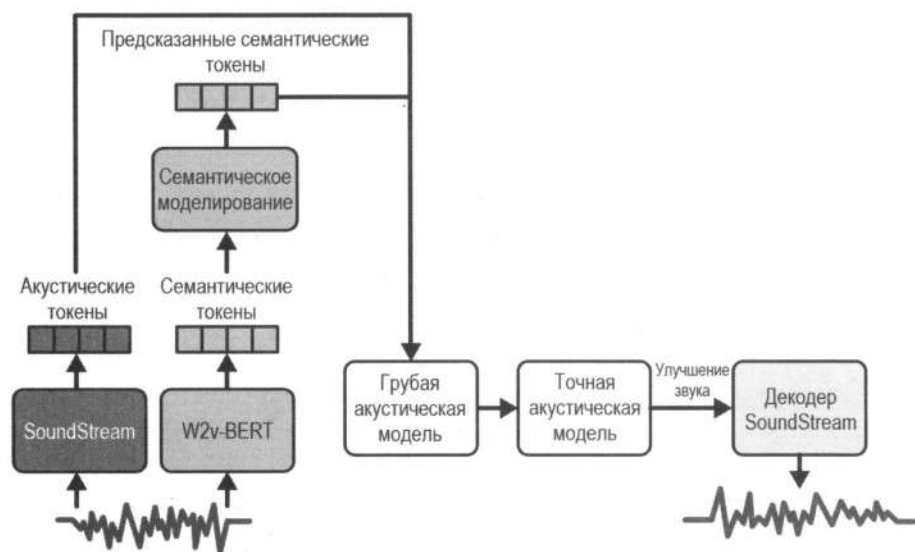


Рис. 9.17. В конвейере AudioLM модель преобразует входной аудиосигнал в последовательность токенов и выполняет генерацию аудиосигнала, используя методы моделирования языка (адаптировано из изображения в оригинальной статье)

AudioLM также можно обучить музыке (например, мелодии на фортепиано) для создания продолжений, сохраняющих ритм и мелодию. Несмотря на то что базовые модели различаются по сравнению с Bark, вы можете заметить, что этапы в целом схожи: мы обучаем ряд моделей, что приводит к созданию высококачественных кодовых книг, а затем используем нейронный кодек (EnCodec в Bark и SoundStream в AudioLM), чтобы создавать окончательный аудиосигнал.

MusicLM (рис. 9.18) имеет еще больше возможностей, фокусируясь на генерации высококачественной музыки, точно соответствующей текстовым описаниям¹¹. В качестве промпта можно использовать текстовое описание, например, «напряженный рок-концерт со скрипками». MusicLM использует многоступенчатую настройку AudioLM и включает в себя обработку текста.

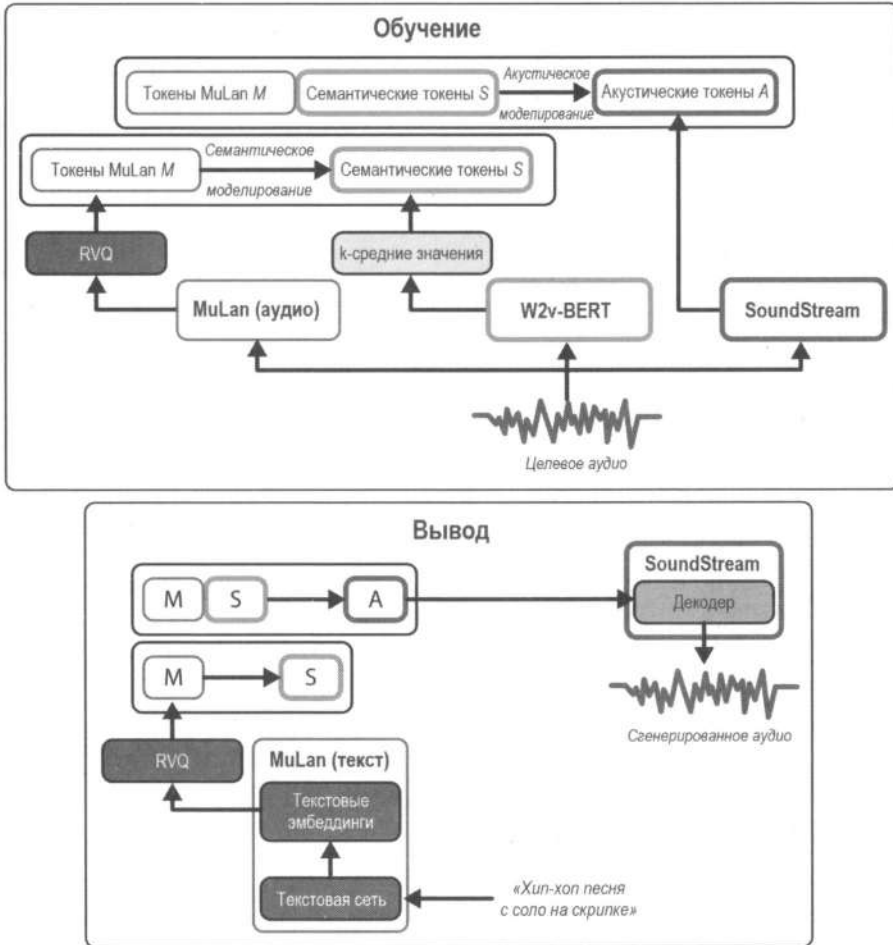


Рис. 9.18. MusicLM включает обусловливание текста для архитектуры AudioLM, чтобы генерировать звук на основе промпта (адаптировано из изображения в оригинальной статье)

Если получение высококачественных размеченных данных в задачах TTS представляет сложность, размеченные данные в ТТА могут быть гораздо сложнее. Последние могут включать гораздо более широкий спектр типов аудио, включая звуки окружающей среды и музыку. Аннотирование разнообразного спектра звуков с высокой точностью может быть более сложным и трудоемким процессом. Для реше-

¹¹ MusicCaps – оценочный набор данных, используемый для MusicLM. Он имеет открытый исходный код и может быть доступен в сети Интернет.

ния этой проблемы MusicLM использует дополнительную модель MuLan, которая, подобно CLIP для пар «изображение-текст», может сопоставлять тексты и соответствующие им аудиоданные с одним и тем же пространством эмбедингов. Благодаря этому MuLan устраняет необходимость в описаниях во время обучения и позволяет проводить обучение на больших объемах данных. Если говорить более конкретно, во время обучения MusicLM использует эмбединги, вычисленные на основе аудиоданных, для настройки моделей, а во время вывода она использует эмбединги текста.



На момент написания книги ни одна из этих моделей не имела открытого исходного кода. LAION выпустила альтернативу MuLan под названием CLAP. Несмотря на то что для обучения использовалось в 20 раз меньше данных, чем в MuLan, она может генерировать разнообразные музыкальные семплы. EnCodec — это альтернатива SoundStream, но с открытым исходным кодом.

AudioGen и MusicGen

В 2022 и 2023 годах компания Meta выпустила несколько открытых моделей для генерации звука на основе текста. AudioGen может генерировать звук и эффекты окружающей среды (например, лай собаки или стук в дверь). Модель работает по принципу, аналогичному Bark и AudioLM. Она использует текстовый энкодер из T5 для генерации текстовых эмбедингов и обуславливает генерацию с их помощью. Декодер авторегрессионно генерирует аудиотокены, основанные на тексте и аудиотокенах, полученных на предыдущих этапах. Готовые аудиотокены можно декодировать с помощью нейронного кодера.

Версия AudioGen с открытым исходным кодом представляет собой вариацию оригинальной архитектуры. Для нейронного кодера они переобучили EnCodec на данных из окружающей среды. Возможно, вам это уже знакомо, т. к. мы снова выполняем весь процесс, аналогичный Bark и AudioLM. Итак, вот что мы можем сделать с помощью AudioGen:

- ◆ Благодаря полной архитектуре мы можем создавать аудио, обусловленное текстом, например «собака лает, а кто-то играет на трубе».
- ◆ Если убрать энкодер текста, мы можем генерировать звуки без каких-либо условий.
- ◆ Мы можем продолжить аудио, используя входные аудиотокены из существующего звука.

Развивая AudioGen, компания Meta выпустила модель MusicGen, которая обучена генерировать музыку, обусловленную текстом, и достигает результатов по различным показателям лучше, чем MusicLM. MusicGen пропускает текстовые описания через энкодер текста (например, из T5 или Flan T5), чтобы получить эмбединги. Затем она использует языковую модель для генерации кодовых книг с аудио, обусловленных эмбедингами. Наконец, аудиотокены декодируются с помощью EnCodec, чтобы получить сигнал. Было выпущено несколько вариаций модели MusicGen.



Модель AudioGen обучена на общедоступных наборах данных (AudioSet, AudioCaps и т. д.). MusicGen, в свою очередь, обучена на 20 тысячах часов музыки, которые принадлежат Meta и лицензированы специально для этой цели, а также с использованием данных из Shutterstock и Pond5.

Попробуем применить наименьший вариант MusicGen из 300 миллионов параметров и загрузим каждый компонент.

```
from transformers import AutoProcessor, MusicgenForConditionalGeneration

model = MusicgenForConditionalGeneration.from_pretrained(
    "facebook/musicgen-small"
).to(device)
processor = AutoProcessor.from_pretrained("facebook/musicgen-small")
inputs = processor(
    text=["an intense rock guitar solo"],
    padding=True,
    return_tensors="pt",
).to(device)

audio_values = model.generate(
    **inputs, do_sample=True, guidance_scale=3, max_new_tokens=256
)
```

Теперь вы знаете, как использовать некоторые модели генерации звука: Bark, SpeechT5 и MusicGen. В зависимости от ожидаемых API-абстракций вместо того, чтобы загружать модель и процессор по отдельности и выполнять весь код вывода самостоятельно, мы можем использовать конвейеры text-to-audio и text-to-speech, доступные в библиотеке transformers. Они абстрагируют логику и отлично подходят для вывода.

```
from transformers import pipeline

pipe = pipeline("text-to-audio", model="facebook/musicgen-small", device=device)
data = pipe("electric rock solo, very intense")
```

Audio Diffusion и Riffusion

Рассмотрим методы генерации звука, основанные на диффузии. Как мы помним, спектрограммы дают информативное визуальное представление об аудио, которое может служить шаблоном для обратного преобразования в звук. В главе 4 мы рассказывали, что можно использовать конвейеры диффузии для создания изображений (как обусловленных, так и безусловных). Применив для генерации спектрограмм модели диффузии, можно делать удивительные вещи.

В этом может помочь модель Audio Diffusion. Ее можно обучить на тысячах мел-спектрограмм генерировать изображения, которые затем можно преобразовать в аудио. Кроме того, что эта идея звучит удивительно просто, она также дает неплохие результаты. Рассмотрим пример с моделью teticio/audio-diffusion-ddim-256,

которая обучена на 20 000 изображений из любимых песен одного из авторов этой книги. Сгенерируем песню (спектрограмма на рис. 9.19) с помощью этой модели.

```
from diffusers import AudioDiffusionPipeline

pipe = AudioDiffusionPipeline.from_pretrained(
    "teticio/audio-diffusion-ddim-256"
).to(device)

output = pipe()
```

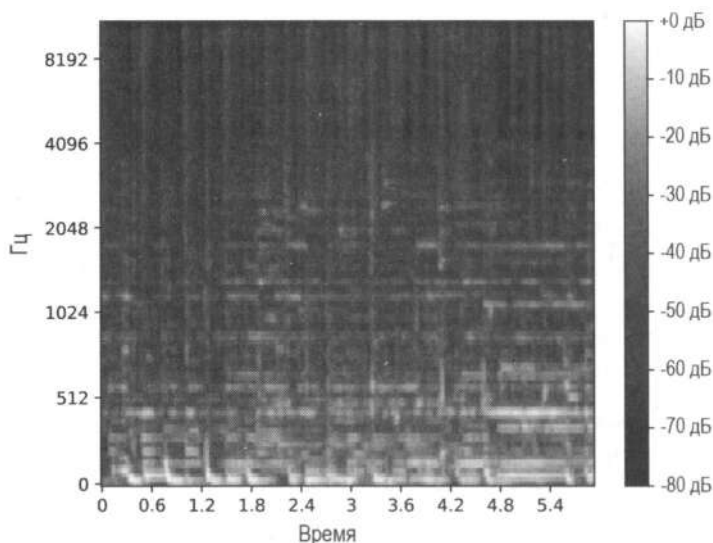


Рис. 9.19. Спектрограмма, сгенерированная с помощью модели диффузии

Получить доступ к спектрограмме можно, выполнив команду `AudioDiffusionPipeline(output.images)`. К нашему удобству конвейер `AudioDiffusionPipeline` преобразует спектрограмму в аудио (используя алгоритм Гриффина–Лима, который работает благодаря качеству спектрограммы) и возвращает соответствующий аудиофайл (`output.audios`). Обратите внимание, эта модель напрямую удаляет шум из спектрограмм. В качестве альтернативы мы могли бы использовать автоэнкодер, чтобы кодировать изображения и работать в латентном пространстве (как это было сделано в главе 5), что значительно ускорило бы обучение модели и вывод. Это может стать отличным упражнением для тех, кто хочет углубиться в тему.



При построении спектрограммы из `output.images[0]` вы увидите различия по сравнению с предыдущей спектрограммой, полученной из аудиовыходов. Модель обучена генерировать изображения в оттенках серого, тогда как мел-спектрограмма — цветная. Кроме того, сгенерированная спектрограмма перевернута по горизонтали из-за структуры обучающих данных. Отображаемая спектрограмма перевернута для соответствия другим спектрограммам, представленным в этой главе.

Разовьем эту идею дальше и используем модель, обусловленную текстом, для генерации спектрограмм, обусловленных текстом. Например, модель `Riffusion` — это

усовершенствованная версия Stable Diffusion, которая может генерировать изображения спектрограмм на основе текстового промпта. Несмотря на то что эта идея звучит странно, она работает на удивление хорошо (рис. 9.20).

```
from diffusers import StableDiffusionPipeline

pipe = StableDiffusionPipeline.from_pretrained(
    "riffusion/riffusion-model-v1", torch_dtype=torch.float16
)
pipe = pipe.to(device)
prompt = "slow piano piece, classical"
negative_prompt = "drums"
spec_img = pipe(
    prompt, negative_prompt=negative_prompt, height=512, width=512
).images[0]
```

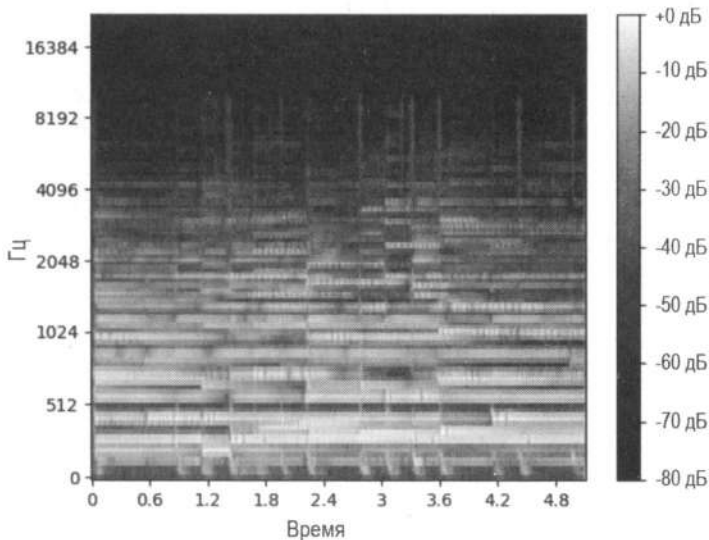


Рис. 9.20. Спектрограмма, сгенерированная моделью диффузии

Простота применения Stable Diffusion дает множество преимуществ. Такие инструменты, как преобразование изображения в изображение (image-to-image), inpainting, негативные промпты и интерполяция, работают без проблем. Например, вы можете преобразовать акустическое соло в соло на электрогитаре или плавно выполнить переход между жанрами, например от эмбиент к джазу, создавая уникальные звуковые эффекты. Возьмем ранее сгенерированную спектрограмму и применим конвейер image-to-image с новым промптом, чтобы преобразовать звуки фортепиано в гитарную композицию.

```
from diffusers import StableDiffusionImg2ImgPipeline

pipe = StableDiffusionImg2ImgPipeline.from_pretrained(
    "riffusion/riffusion-model-v1", torch_dtype=torch.float16
)
```

```

pipe = pipe.to(device)

prompt = "guitar, acoustic, calmed"
generator = torch.Generator(device=device).manual_seed(1024)
image = pipe(
    prompt=prompt,
    image=spec_img,
    strength=0.7,
    guidance_scale=8,
    generator=generator,
).images[0]

```

Dance Diffusion

Несмотря на то что генерация спектрограмм и их преобразование в аудио работают вполне отлично, использование изображений для обучения аудиомодели интуитивно понятно лишь отчасти. Вместо использования спектрограмм мы можем исследовать процесс обучения модели, которая способна работать непосредственно с необработанными аудиоданными (массивом чисел), а не с изображениями. Как известно из главы 4, сеть UNet представляет собой CNN с серией слоев понижения частоты дискретизации, за которыми следует серия слоев повышения частоты дискретизации. Мы ранее рассматривали UNet, работающую исключительно с двумерными данными (UNet2DModel), но мы также можем использовать ее вариацию, работающую с необработанным аудио, а именно заставить сеть работать с массивом чисел с плавающей точкой (UNet1DModel)¹².

Dance Diffusion — это семейство моделей с открытым исходным кодом для безусловной генерации звука, которые напрямую генерируют аудиосигналы. Существуют различные вариации, обученные на разных наборах данных. Например, *harmonai/maestro-150k* — это модель, обученная на 200 часах записей фортепиано, поэтому она может отлично генерировать звуки этого инструмента.

```

from diffusers import DanceDiffusionPipeline

pipe = DanceDiffusionPipeline.from_pretrained(
    "harmonai/maestro-150k", torch_dtype=torch.float16
)
pipe = pipe.to(device)
audio = pipe(audio_length_in_s=5, num_inference_steps=50).audios[0]

```

Настройка Dance Diffusion аналогична той, что мы рассматривали в главе 4. Изменив модель UNet2DModel и обучающие данные, вы можете получить неплохие результаты сразу «из коробки». Настройка минимального цикла обучения Dance Diffusion с использованием ваших данных может быть относительно простой и не требовать

¹² Если вы взглянете на сгенерированные аудиоданные, то увидите два массива. Это связано с тем, что Dance Diffusion обучена на стереозвуке.

больших усилий. Качество этих моделей приемлемое, но, очевидно, могло бы быть и лучше.

Подробнее о моделях диффузии для генерации звука

Одно из существенных ограничений моделей диффузии заключается в очень низкой скорости вывода. Вдохновленные Stable Diffusion, авторы моделей DiffSound и AudioLDM разработали их для работы с латентным пространством. Оно представляет собой, такое же пространство эмбедингов, как в модели CLAP. Подобно MuLan в MusicLM и CLIP в области изображений, CLAP — это модель, которая отображает тексты и аудио в общее латентное пространство. Это значительно ускоряет процесс диффузии и устраняет необходимость в размеченных данных.

Глобально модель похожа на Stable Diffusion, но работает с другой модальностью:

- ◆ Мы используем CLAP вместо CLIP.
- ◆ Мы по-прежнему используем магистраль UNet (фактически те же характеристики, что и Stable Diffusion).
- ◆ Мы используем VAE для декодирования латентного пространства и генерации мел-спектрограмм.

Как и в наших ранних экспериментах с TTS, здесь мы также используем вокодер HiFiGAN для синтеза аудиосигнала из спектрограммы. Такая архитектура предоставляет для AudioLLM платформу, способную выполнять различные аудиоманипуляции с использованием текста. Например, она может выполнять *inpainting*, генерировать сверхвысокое разрешение и переносить стили.

MusicLDM является модификацией AudioLDM, которая полностью сосредоточена на музыке. CLAP не работала бы сразу, поскольку она предварительно обучена на наборах данных, в которых преобладают звуковые эффекты и прочие звуки, и в них не так много музыки. Авторы переобучили CLAP на наборах данных, состоящих из пар «текст-музыка», что улучшило понимание музыкальных данных. Авторы также переобучили вокодер HiFiGAN на музыкальных данных, чтобы гарантировать соответствие реконструированных сигналов музыке.

Один из рисков, связанных с генерацией текста в музыку, заключается в том, что, учитывая дефицит допустимых данных, процесс диффузии с большой вероятностью будет восстанавливать примеры из исходных обучающих данных. Чтобы избежать этого, MusicLDM использует методы аугментации, чтобы побудить модель исследовать больше пространства в музыкальных данных. Например, они используют *mixup* — метод смешивания данных, который объединяет два аудиосемпла с помощью интерполяции и создает новый. Это помогает предотвратить переобучение и улучшить обобщение.

Оценка систем генерации звука

Модели, решающие задачи TTS и TTA, как и генерацию изображений, сложно оценить, поскольку не существует единственно правильного метода. Кроме того, качество звука во многом зависит от человеческого восприятия. Представьте, что вы используете разные модели для создания песни на основе промпта. Разные люди будут иметь разные предпочтения, что делает сравнение моделей сложным и затратным.

Существуют различные подходы к определению объективных метрик. Например, *Frechet Audio Distance (FAD)* может использоваться для оценки качества моделей при отсутствии референтного аудио. Эта метрика коррелирует с человеческим восприятием и позволяет идентифицировать правдоподобный материал, но не всегда позволяет оценить соответствие сгенерированных аудиоданных промптам. Другой подход при наличии референтных аудиоматериалов заключается в использовании классификатора для вычисления классов предсказаний как для самих предсказаний, так и для референтного аудиоматериала, а затем в измерении KL-дивергенции между двумя распределениями вероятностей. В случае отсутствия референтных данных CLAP может сопоставить входной промпт и вывод с одним и тем же пространством эмбедингов и вычислить степень сходства между этими эмбедингами. Такой подход будет сильно зависеть от данных предварительного обучения, используемых для CLAP.

В конечном счете для систем TTS и TTA всегда необходима человеческая оценка. Как упоминалось ранее, качество генераций является субъективным, поэтому в идеале его должны оценивать несколько человек с разным опытом. Один из способов сделать это — вычислить *средний балл мнения (Mean Opinion Score, MOS)*, который требует, чтобы люди оценивали генерации по определенной шкале. Еще один подход заключается в том, чтобы показать людям входной промпт и две разные генерации из разных моделей. Затем людям предлагается определить, какая из них предпочтительнее.

Что дальше?

Мир генеративного аудио переживает период беспрецедентных инноваций и роста. Недавние достижения в области генерации аудио и речи весьма существенны, но область продолжает постоянно развиваться, открывая новые горизонты для исследований.

Например, разработка высококачественных аудиотехнологий для работы в реальном времени — это многообещающая перспективная область. Возможности создания высококачественного звука открывают множество перспектив для приложений в различных областях, от инструментов обеспечения доступа до разработки игр. Недавние релизы, такие как XTTS от Coqui, TTS-модели от ElevenLabs и Moshi от Kyutai, служат яркими примерами.

Еще одна новая тема — унифицированное моделирование. Оно позволяет создавать модели, которые могут быть гибкими для различных задач. Например,

SeamlessM4T представляет собой масштабируемую унифицированную систему перевода речи и является отличным примером в этой области. Она поддерживает множество аудиозадач, например TTS, перевод речи в речь, перевод текста в речь (генерация синтетической речи на другом языке) и пр. В этой главе были рассмотрены популярные высококачественные модели генерации речи, аудио и музыки. Различия между этими тремя областями создают трудности при обучении унифицированной модели для генерации звука во всех трех форматах. Однако современные методы моделирования, такие как AudioLDM 2, продвигают исследования в этом направлении. Возможности позволяют создать единый аудиоязык, помогающий реализовывать все эти модели и получать достойные результаты, сопоставимые или даже превосходящие результаты других специализированных моделей.

Проблемы, связанные с аудио, выходят за рамки моделирования. Ключевые этические вопросы включают происхождение данных, законы об авторском праве, хранение и право собственности. Несмотря на то что клонирование голоса впечатляет, оно поднимает серьезные этические вопросы при применении к голосам других людей без их согласия. Обучение музыкальной модели может быть захватывающим интеллектуальным и творческим экспериментом, но добросовестное использование наборов данных остается открытым вопросом. Например, создание синтетической версии голоса знаменитости для рекламы без ее согласия может быть расценено как нарушение личных прав.

Кроме того, последние исследования в области аудио сосредоточены преимущественно на английском языке, что подразумевает значительный прогресс и в области многоязычия. Если модели распознавания речи разрабатываются преимущественно для определенных языков, носители других языков могут получить некачественные сервисы или недоступные инструменты. Такие этические соображения подчеркивают важность ответственной разработки и применения технологий машинного обучения. Проактивное решение этих проблем может способствовать укреплению доверия и обеспечению справедливого распределения преимуществ, которые дают эти технологии.

Время проекта: сквозная диалоговая система

На данный момент вы уже научились использовать модели трансформера для генерации текста (главы 2 и 6), модели диффузии для генерации изображений (главы 4 и 5), модели для транскрибирования и генерации речи (текущая глава), а также создавать демо-модели с помощью Gradio (глава 5). Цель этого задания — создать комплексное приложение с помощью Gradio, в котором:

- ◆ Пользователь может задать промпт текстом или голосом.
- ◆ Промпт может быть в виде диалога или содержать просьбу создать изображение.
- ◆ На основе промпта модель должна генерировать ответ, который может быть в виде изображения или текста.
- ◆ Модель должна выводить изображение и текст, а также соответствующую сгенерированную речь в случае, если вывод является текстом.

Данный проект потребует от вас принятия множества решений. Какую модель вы выберете? Как сбалансировать качество и скорость? Как определить, требуется ли в промпт добавить просьбу сгенерировать изображение?¹³ В этом проекте необходимо применить навыки, полученные в предыдущих главах. Мы рекомендуем начать с простой версии приложения, а затем развивать его дальше. Для этого проекта вам не нужно обучать модели, но при желании вы можете их доработать. Цель — создать увлекательную и интерактивную диалоговую систему, способную генерировать изображения, текст и аудио на основе пользовательских промптов.

Заключение

Сфера генеративного аудио переживает период подъема: новые модели появляются каждые несколько недель, выпускаются более качественные наборы данных, новые лаборатории выходят на рынок. Вполне нормально чувствовать себя в замешательстве от такого количества моделей, используемых в этой области, которая развивается невероятно быстро. Понимание других моделей (таких как языковые или диффузионные) вдохновило многих авторов на создание инструментов, которые мы использовали. Сложность, присущая аудио, привела к открытию новых компонентов — вокодеров для реконструкции спектрограмм в сигнал и нейронных кодеков для сжатия и декомпрессии аудио. Несмотря на то что эта глава содержит лишь введение в модальность аудио, она дает фундаментальные знания для более глубокого понимания последних исследований.

Если вы хотите глубже изучить эту область, рекомендуем ознакомиться со следующими темами:

◆ CTC

Чтобы узнать больше об алгоритме CTC, используемом в модели Wav2Vec2, рекомендуем прочитать пост «Sequence Modeling With CTC».

◆ ParlerTTS

Это библиотека для обучения и вывода (а также семейство моделей) для TTS. Она мало весит и может генерировать высококачественную и настраиваемую речь. Рекомендуем изучить репозиторий ParlerTTS на GitHub и ознакомиться с примерами вывода и обучения.

◆ Вокодеры

В этой главе мы кратко рассказали о вокодерах на примере HiFiGAN. Они преобразуют мел-спектрограмму в речь. Для более подробного обзора рекомендуем прочитать о вокодерах WaveNet и MelGAN. Постарайтесь понять, в чем их различия и как они оцениваются.

¹³ Подсказка: например, вы можете попробовать эвристику, классификацию с нулевым выстрелом или сходство предложений.

◆ Оптимизация модели

Различные методы оптимизации могут привести к значительно более быстрой генерации звука с минимальным ухудшением качества. Рекомендуем прочитать пост «AudioLDM 2, but Faster 3» и статью «Speculative Decoding for 2x Faster Whisper Inference» на Hugging Face.

◆ Другие популярные модели

В этой главе мы рассмотрели множество моделей. Помимо них, мы рекомендуем изучить и другие популярные экземпляры, такие как Tacotron 2, FastSpeech, FastSpeech 2, TorToiSe TTS и VALL-E. Общее понимание этих моделей позволит получить полную картину.

Поскольку мы столкнулись с исчерпывающим множеством наборов данных и моделей, в табл. 9.1 и 9.2 кратко суммирована информация по ключевым ресурсам для дальнейшего изучения.

Таблица 9.1. Общая информация о наборах данных

Набор данных	Описание	Часы обучения	Рекомендуемый сценарий использования
LibriSpeech	Озвученные аудиокниги	Английский: 960	Бенчмарки и предварительная подготовка моделей
Multilingual LibriSpeech	Многоязычный эквивалент LibriSpeech	Английский: 44 659 Всего: 65 000	Бенчмарки и предварительная подготовка моделей
Common Voice 13	Многоязычный контент, собранный с помощью краудсорсинга, с различным качеством	Английский: 2400 Всего: 17 600	Многоязычные системы
VoxPopuli	Запись Европейского парламента	Английский: 543 Всего: 1800	Многоязычные системы, предметно-ориентированное использование, не носители языка
GigaSpeech	Многодоменный английский из аудиокниг, подкастов и YouTube	Английский: 10 000	Надежность в множестве доменов
FLEURS	Параллельный многоязычный корпус	10 часов для каждого из 102 языков	Оценка в многоязычных условиях (включая условия «низкого уровня цифровых ресурсов»)
MMS-labeled	Новый Завет читается вслух	37 000 часов для 1100 языков	Многоязычные системы
MMS-unlabeled	Записи рассказов и песен	7700 часов для 3800 языков	Многоязычные системы

Таблица 9.2. Общая информация о моделях

Модель	Задача	Тип модели	Примечания
Wav2Vec2	Английский язык (ASR)	Трансформер-энкодер с CTC	Обучена на немаркированных данных на английском языке. Легко настраивается

Таблица 9.2 (окончание)

Модель	Задача	Тип модели	Примечания
hUBERT	Английский язык (ASR)	Трансформер-энкодер с CTC	Обучена на немаркированных данных на английском языке. Легко настраивается.
XLS-R	Многоязычный ASR	Трансформер-энкодер с CTC	Обучена на немаркированных данных для 128 языков
Whisper	Многоязычный ASR	Трансформер энкодер-декодер	Обучена на огромном объеме многоязычных размеченных данных
SpeechT5	ASR, TTS и S2S	Трансформер энкодер-декодер	Добавляет предварительные и постсети для отображения речи и текста в одном пространстве
HiFiGAN	Преобразование спектрограммы в речь	GAN с множеством дискриминаторов	Это один из типов вокодера
EnCodec и SoundStream	Сжатие аудио	Энкодер-декодер	Использует квантованное латентное пространство
Bark	Многоязычный TTA	Многоступенчатая авторегрессия	Предсказывает кодовые книги и использует EnCodec для реконструкции
MuLan и CLAP	Сопоставляет текст и аудио в одном пространстве	Трансформер-энкодер для текста и CNN для звука	CLAP — это репликация MuLan с открытым исходным кодом
AudioLM	Продолжение аудио-дооржки	Многоступенчатая авторегрессия	Концептуально похож на Bark, но на входе получает аудио
MusicLM	Текст в музыку (Text-to-music, TTM)	Объединяет MuLan и AudioLM	Включает в себя MuLan, чтобы устранить необходимость в маркированных данных
AudioGen	TTA	Многоступенчатая авторегрессия	EnCodec проходит переобучение на основе данных об окружающем шуме
MusicGen	TTM	То же, что и AudioGen	Несколько вариаций имеют открытый исходный код
AudioLDM	TTA	Диффузия патентного пространства (то же самое, что Stable Diffusion)	Использует CLAP для латентного пространства
MusicLDM	TTM	То же, что и AudioLDM	Переобучает CLAP и HiFiGAN в области музыки

Вопросы

1. Каковы плюсы и минусы использования осциллограмм и спектрограмм?
2. Что такое спектрограмма и мел-спектрограмма? Какая из них используется в моделях?
3. Объясните, как работает алгоритм CTC. Зачем он нужен в ASR-моделях на основе энкодера?

4. Что произойдет, если данные вывода будут иметь частоту дискретизации 8 кГц, а модель будет обучена с частотой 16 кГц?
5. Как работает добавление модели n-грамм к модели на основе энкодера?
6. Каковы плюсы и минусы моделей на основе энкодера и моделей на основе энкодера-декодера для задач ASR?
7. В каком случае стоит предпочесть использовать метрику CER вместо WER для оценки ASR-моделей?
8. Какие шесть сетей использует модель SpeechT5? Какая конфигурация потребуется для преобразования голоса?
9. Что такое вокодер? В каких случаях он используется?
10. Для чего используется модель EnCodec?
11. Как модели TTA используют модели MuLan и CLAP, чтобы снизить необходимость в размеченных данных?

Ответы на эти вопросы и решения следующих задач можно найти в репозитории книги на GitHub.

Задачи

1. *Исследование Whisper*. Следующий фрагмент кода создает случайный массив и экстрактор признаков Whisper с нуля.

```
import numpy as np
from transformers import WhisperFeatureExtractor

array = np.zeros((16000, ))
feature_extractor = WhisperFeatureExtractor(feature_size=100)
features = feature_extractor(
    array, sampling_rate=16000, return_tensors="pt"
)
```

2. Изучите, как влияет изменение параметров `feature_size`, `hop_length` и `chunk_length` на форму входных признаков. Затем изучите значения по умолчанию для `WhisperFeatureExtractor` в его документации и назначение каждого из них. Попробуйте рассчитать, сколько признаков будет сгенерировано для аудиофрагмента.
3. *Преобразование голоса*. Реализуйте преобразование голоса с помощью модели SpeechT5, чтобы входной звук воспроизводился другим диктором.
4. *Обучение Dance Diffusion*. Реализуйте небольшой обучающий конвейер для модели Dance Diffusion. В качестве отправной точки можно использовать код из главы 4.

Быстро развивающиеся направления в области генеративного ИИ

Сфера генеративного ИИ развивается очень быстро. С начала работы над этой книгой мы стали свидетелями появления новых моделей, таких как GPT-4, Llama 3, Gemini и Sora. Кроме того, появилось множество новых базовых LLM, аудиомоделей и методов диффузии. Как упоминалось в предисловии, эта книга посвящена общим принципам и основам, которые дают навыки и понимание, позволяющие следить за развитием этой области.

Прежде чем завершить книгу, мы хотим кратко осветить некоторые из самых интересных и быстро развивающихся областей генеративного ИИ. В этой главе представлен общий обзор тем и ресурсов, чтобы вы могли углубиться в них, если они вам интересны. Вместо того чтобы стремиться досконально разбираться в чем-либо, воспринимайте эту главу как руководство, которое помогает продолжать развиваться по мере изучения генеративного ИИ.

Оптимизация предпочтений

В главе 6 мы обучали чат-модель на основе набора данных Open Assistant с использованием контролируемого обучения и традиционной тонкой настройки. Сейчас появились новые мощные модели, учитывающие предпочтения. Они могут обучаться, чтобы генерировать ответы, соответствующие определенным ожиданиям. Например, кому-то могут понадобиться «услужливые» модели, которые всегда стараются помочь, независимо от запроса. Другим могут быть нужны модели, которые являются более нейтральными и стараются избегать генерации токсичных результатов.

Когда модель заявляет, что не может помочь, это происходит из-за *оптимизации предпочтений*. Она используется для того, чтобы направить LLM к желаемому поведению, которое может быть любым: от генерации наименее ошибочного кода или текста в разговорном стиле до отказа от создания контента по определенным темам.

Обучение с подкреплением и обратной связью от человека (Reinforcement Learning with Human Feedback, RLHF) — один из методов, используемых для настройки предпочтений, который немного меняет традиционный процесс тонкой настройки. Как и в главе 6, первым шагом является тонкая настройка модели с помощью контролируемой тонкой настройки. При RLHF тонкая настройка затем используется для обучения *модели вознаграждений*. Для каждого промпта она гене-

рирует несколько потенциальных вариантов. Затем эксперт ранжирует их, а модель вознаграждения обучается предсказывать итоговую оценку. Эксперты обычно являются людьми, но существует тенденция использовать в качестве экспертов другие, более крупные модели (например, в статье «*RLAIF vs. RLHF: Scaling Reinforcement Learning from Human Feedback with AI Feedback*» для ранжирования используются LLM). Заключительный этап — использование модели вознаграждения для дальнейшей тонкой настройки исходной контролируемой тонкой настройки модели, чтобы она научилась генерировать результаты, похожие на примеры с высокими (по мнению экспертов) результатами. RLHF был критически важным компонентом как для Llama 2, так и для ChatGPT. Несмотря на то что концепция внедрения модели вознаграждения в процесс тонкой настройки интригует, она также добавляет сложности, как показано на рис. 10.1. Это стимулирует значительное количество исследований, направленных на замену модели вознаграждения новой функцией потерь. Недавние исследования, такие как «*Direct Preference Optimization: Your Language Model is Secretly a Reward Model*», «*A General Theoretical Paradigm to Understand Learning from Human Preferences*», и «*KTO: Model Alignment as Prospect Theoretic Optimization*», являются яркими примерами этих продолжающихся исследований. Данные методы полностью исключают компонент обучения с подкреплением (RL), который, как известно, нестабилен и сложен в обучении.

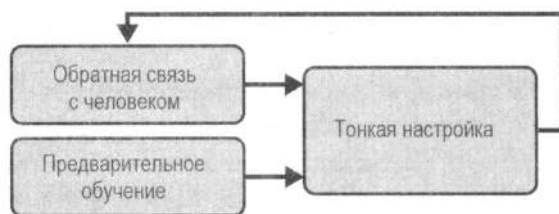


Рис. 10.1. RLHF состоит из трех основных компонентов: предварительной подготовки, тонкой настройки и обратной связи с человеком

RLHF также может применяться к моделям диффузии. *Оптимизация политики диффузии шумоподавления (Denoising diffusion policy optimization, DDPO)* — это способ дополнить модели диффузии с помощью обучения с подкреплением для тонкой настройки модели и повышения качества генерируемых изображений. RLHF также может применяться к моделям диффузии, как показано на рис. 10.2.

Вот несколько дополнительных материалов по теме:

- ◆ Вводная статья «*Illustrating Reinforcement Learning from Human Feedback (RLHF)*» в блоге Hugging Face.
- ◆ Пост «*LLM Training: RLHF and Its Alternatives*» Себастьяна Рашки (Sebastian Rashka) содержит хороший обзор на RLHF и его альтернативы.
- ◆ Чтобы узнать больше о RLHF в контексте моделей диффузии, рекомендуем прочитать статью «*Finetune Stable Diffusion Models with DDPO via TRL*» или блог Танишка Абрахама (Tanishq Abraham).

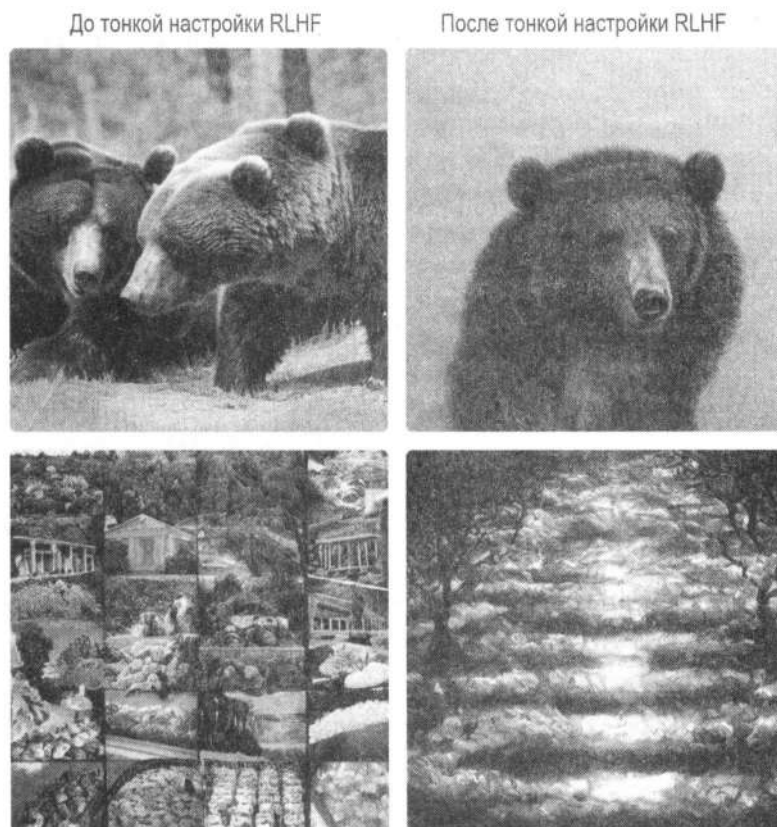


Рис. 10.2. RLHF применяется для моделей диффузии

- ◆ Чтобы узнать больше о DPO, IPO и KTO, рекомендуем прочитать соответствующие публикации в блоге Hugging Face.
- ◆ Конституционный ИИ — еще один подход к согласованию LLM с набором значений. Публикация «*Constitutional AI with Open LLMs*» в блоге Hugging Face — хороший источник информации для более глубокого изучения.

Длинные контексты

Большинство LLM-моделей, обсуждаемых в книге, способны обрабатывать контекст длиной до нескольких тысяч токенов. Например, Llama 3.1 способна учитывать 128 000 токенов, а какие-то — до нескольких сотен тысяч. В некоторых проприетарных моделях удалось достичь гораздо более длинных контекстов. Например, Gemini обрабатывает до 2 миллионов токенов, а Anthropic Claude — 200 000. Обработка очень длинных контекстов может быть крайне полезна для систем RAG (например, реализованной в проекте в главе 6) или для систем, генерирующих или понимающих код, где в качестве контекста может использоваться большая кодовая база.

По мере увеличения длины входных данных возникает ряд проблем:

- ◆ LLM-моделям требуется больше видеопамати для обработки длинных контекстов.
- ◆ Качество генерации, как правило, ухудшается по мере роста контекста. Поскольку модели не обучались на таких длинных контекстах, они могут не улавливать зависимости между токенами.
- ◆ Генерация замедляется. Традиционный механизм внимания является узким местом, поскольку имеет квадратичную сложность.

Одним из решений является использование оконного внимания, которое ограничивает количество токенов, передаваемых в LLM. *Окно внимания* можно представить как «скользящее окно», проходящее по входному тексту. Оно ограничивает использование графического процессора, но качество все равно снижается, поскольку могут встречаться токены, несущие важную информацию до начала окна. Окно внимания можно адаптировать для учета начальных токенов с помощью таких методов, как *Attention Sinks*, что отлично подходит для многораундовых диалогов. Альтернативный подход заключается в повышении эффективности механизма внимания. Существует множество предложений по субквадратичному масштабированию, например *Longformer* (который сочетает оконные методы с глобальными функциями внимания) и *Flash Attention* (который оптимизирует передачу данных в память за счет объединения операций). *Flash Attention* является популярным решением, поскольку делает требования к пространству линейными во время вывода.

Несмотря на то что упомянутые подходы направлены на ускорение алгоритма внимания или повышение его вычислительной эффективности, также существуют исследования по расширению предварительно обученных LLM с помощью масштабирования *Rotary Position Embeddings (RoPE)*. Можно ли взять, например, *Llama* и заставить ее обрабатывать больше токенов, чем было предварительно обучено? *RoPE* изменяет способ включения позиционной информации в трансформеры, что позволяет им улавливать зависимости на больших расстояниях. Выполняя минимальную тонкую настройку, мы можем расширить контекстное окно. Например, компании Meta удалось расширить контекстное окно исходной модели *LLaMA* с 2048 токенов до 32 768.

Исследования, направленные на контексты произвольной длины, продолжаются и по сей день. *Ring Attention* и *Infini-Attention* — два примера, направленных на обработку бесконечного контекста. Исследования по оценке моделей длинного контекста все еще находятся на ранней стадии. Метод иголки в стоге сена, показанный на рис. 10.3, служит примером извлечения длинного контекста.

Сейчас параллельно ведутся работы по использованию альтернативных архитектур. Одна из них — рекуррентные нейронные сети (*RNN*). *RWKV* — это семейство моделей *RNN* с открытым исходным кодом, которые достигают эффективного вывода благодаря линейному вниманию, сохраняя при этом высокую эффективность и возможность распараллеливать обучение. Другой альтернативой является использование *моделей пространства состояний (State Space Model, SSM)*. Например,

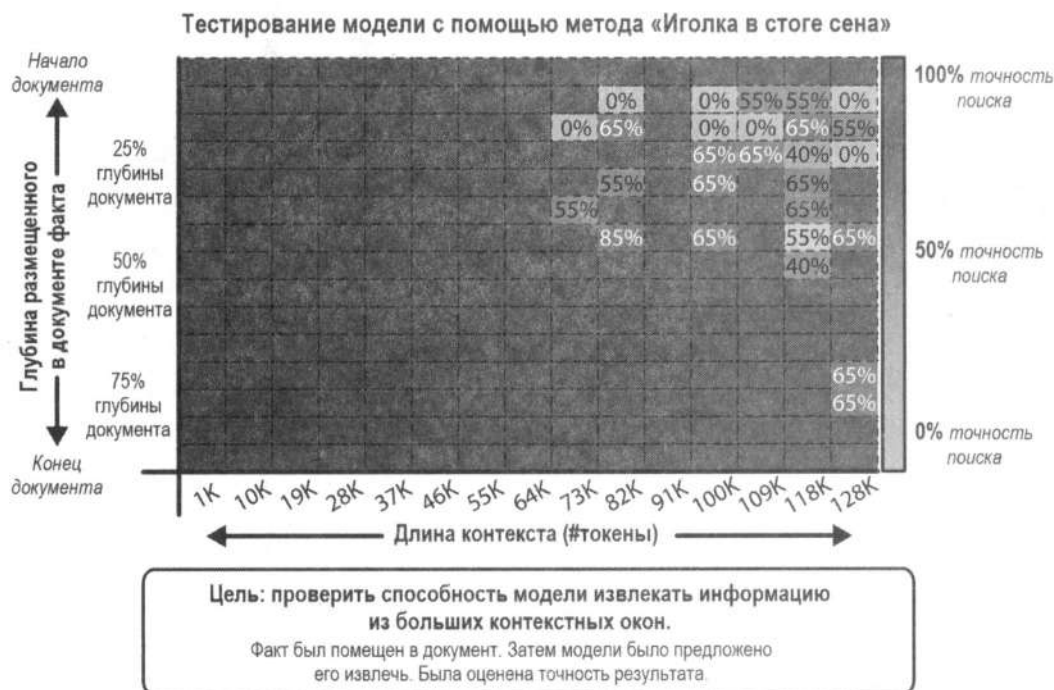


Рис. 10.3. Метод иголки в стоге сена оценивает контекстный поиск в длинных контекстах

модель Mamba использует SSM для линейного масштабирования памяти по количеству токенов и обеспечивает очень быстрый вывод.

Вот несколько рекомендуемых материалов по этой теме:

- ◆ Публикация «*Attention Sinks in LLMs for endless fluency*» в блоге Hugging Face объясняет, как работает механизм Attention Sinks, и содержит результаты различных экспериментов.
- ◆ Документация по Flash Attention и Flash Attention 2.
- ◆ Статьи, касающиеся расширения контекста с помощью RoPE: «*Extending Context is Hard...but not Impossible*» и «*Extending Context Window of Large Language Models via Positional Interpolation*».
- ◆ Чтобы узнать больше о RWKV, рекомендуем прочитать публикацию «*Introducing RWKV — An RNN with the advantages of a transformer*», статью «*RWKV: Reinventing RNNs for the Transformer Era*» или работу «*Eagle and Finch: RWKV with Matrix-Valued States and Dynamic Recurrence*» с улучшениями в архитектуре.
- ◆ Чтобы узнать больше о SSM и Mamba, рекомендуем прочитать «*The annotated S4*» и «*Mamba: The Hard Way*».

Смесь экспертов

В последние годы метод *смесь экспертов* (**Mixture of Experts, MoE**) стал привлекательным подходом для LLM, наиболее заметной из которых стала модель **Mixtral 8x7B** от команды **Mistral** (релиз в декабре 2023 года)¹. В контексте моделей-трансформеров MoE очень похожа на плотные (традиционные) трансформеры (рис. 10.4), но обладает преимуществами в эффективности обучения и масштабируемости производства. При наличии фиксированного объема вычислений для обучения модели методы MoE могут продвинуть вас дальше по кривой обучения, чем плотные модели. При крупномасштабном использовании они могут быть более эффективны в обработке большого количества запросов в секунду.

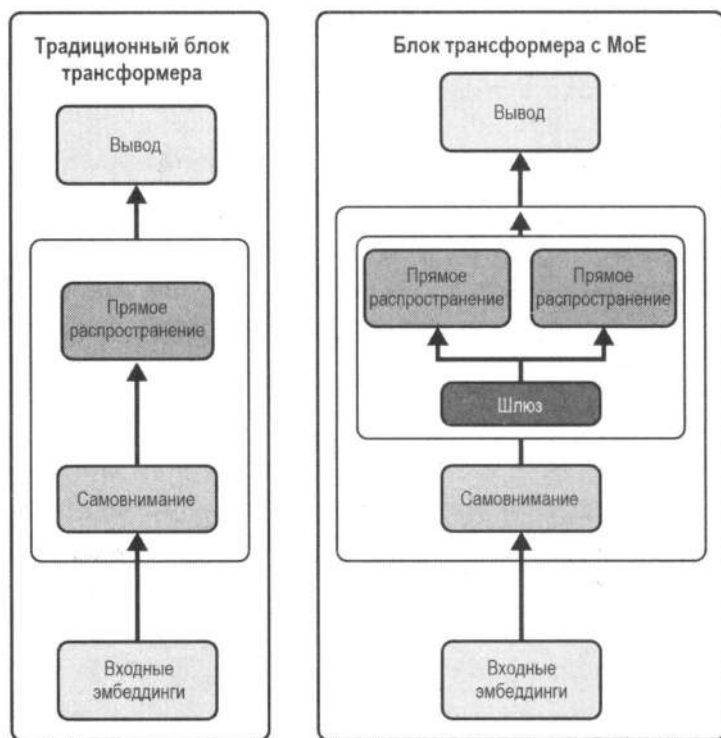


Рис. 10.4. Упрощенный традиционный блок трансформера и его эквивалент с методом MoE с двумя экспертами

Одним из ключевых элементов моделей MoE является замена некоторых или всех сетей прямого распространения в блоках трансформеров разреженными блоками MoE. Каждый блок MoE представляет собой набор «экспертных» сетей, где каждый эксперт представляет собой отдельную модель (обычно другую сеть прямого распространения). При наличии токена динамически активируются только некото-

¹ Ходят слухи, что GPT-4 также использует архитектуру MoE.

рые эксперты², а остальные будут бездействовать. Чтобы определить, каких экспертов активировать, MoE используют сеть шлюзов (или маршрутизатор), которая динамически назначает токены различным экспертам во время обучения и вывода. Сеть шлюзов действует как контроллер трафика, обеспечивая эффективное распределение токенов между экспертами. Это делает шлюз критически важным компонентом MoE, и многие исследования направлены на обучение более эффективных сетей шлюзов.

Важно уточнить, что количество экспертов в MoE не приводит напрямую к линейному увеличению параметров. Например, в случае Mixtral 8x7B название модели может предполагать восемь экспертов по 7 миллиардов параметров в каждом (в сумме 56 миллиардов параметров), но в действительности она включает 47 миллиардов параметров из-за наличия общих компонентов, а цифра «8» относится к количеству экспертов в каждом блоке MoE. Помните, что блоками MoE заменяются только сети прямого распространения. Остальная часть, например блоки внимания, по-прежнему используется совместно.

Это означает, что для загрузки модели вам потребуется графический процессор для хранения 47 миллиардов параметров. С другой стороны, для одного токена активируются только некоторые эксперты (2 для Mixtral). Следовательно, количество активированных параметров значительно меньше (12 миллиардов для Mixtral). Это делает методы MoE менее интересными для локального вывода (из-за большого требуемого объема памяти GPU), но привлекательными для производственных конфигураций, где вы можете получать много запросов одновременно. Учитывая, что для каждого параллельно обрабатываемого запроса будет активировано меньше параметров на токен, MoE должны быть способны обрабатывать больше запросов, чем плотные модели.

Второе заблуждение относительно MoE заключается в том, что эксперты специализируются на разных задачах или подмножествах данных. Они обучаются с помощью функции потерь, которая обеспечивает равномерное распределение нагрузки по генерации токенов между экспертами, гарантируя задействование всех экспертов. Эксперты в шлюзе работают как своего рода балансировщики нагрузки, и нет никаких свидетельств высокоуровневой специализации задач.

Методы MoE стали широко популярны благодаря своему впечатляющему качеству и производительности. Помимо широко известной модели Mixtral 8x7B, существуют также Snowflake Arctic Instruct, Databricks DBRX, Mixtral 8x22B, DeepSeekMoE и Qwen 1.5 MoE.

Чтобы узнать больше по этой теме, рекомендуем ознакомиться со следующими ресурсами:

- ◆ Вводный блог на Hugging Face по методам MoE содержит общий обзор исследований и объяснение принципов их работы.
- ◆ В статье «*Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*» подробно рассматриваются многие проблемы и проектные

² Количество активированных экспертов — это параметр конфигурации, который можно изменить.

решения для создания и обучения методов MoE. Это отличная статья, которая поможет вам понять вопросы относительно проектирования, возникающие при работе с MoE.

- ♦ В статье «*DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models*» представлены некоторые новые идеи для методов MoE, такие как сегментация экспертов на более мелкие группы и использование общих экспертов, которые всегда активируются. Эта статья отлично подходит для понимания передовых исследований в этой области.
- ♦ Статья «*Mixtral of Experts*» — отличный материал о том, как обучалась модель Mixtral 8x7B. Она не представляет новую архитектуру или идеи обучения, но полезна для понимания того, как обучаются MoE на практике. Это особенно интересно, учитывая, что Mixtral была первой высококачественной общедоступной моделью с MoE.

Оптимизации и квантование

Методы оптимизации традиционно востребованы по двум причинам:

- ♦ Они максимизируют производительность модели в сценариях с высокой нагрузкой (например, чаты или генеративные API). Обслуживание большего количества пользователей на сервере в единицу времени минимизирует затраты и позволяет большему числу людей пользоваться сервисом.
- ♦ Они сокращают время или ресурсы на обучение. Для больших моделей умеренное сокращение объема используемой памяти или времени, необходимого для завершения этапа обучения, может привести к значительному ускорению работы и уменьшению времени использования оборудования. Даже для небольших лабораторий или индивидуальных специалистов, обучающих небольшие модели, более быстрое обучение позволяет сократить итерационные циклы и провести больше экспериментов.

Кроме того, исследования за последние несколько лет продемонстрировали огромный интерес общества к запуску всех видов моделей, включая LLM, на потребительском оборудовании. Причин этому множество: эксперименты с моделями без привязки к API, понимание, как работают модели, с помощью анализа внутренней активации, запуск задач в частном порядке на компьютере и даже без подключения к Интернету, тонкая настройка данных без траты множества денег и усилий на облачные сервисы и т. д.

В результате возникло множество методов оптимизации и «хитрых» трюков. Мы уже кратко рассмотрели квантование — набор методов, направленных на снижение точности параметров модели (с минимальным влиянием на качество), чтобы уменьшить объем памяти и повысить точность моделей на потребительском оборудовании. Также мы познакомились с Flash Attention, который не только позволяет использовать более длинные контекстные окна, но и снижает потребление памяти всеми типами моделей-трансформеров (включая языковые и многие другие генеративные модели, которые мы рассмотрели в главе 4).

Другой интересный метод ускорения — *спекулятивное декодирование*, также называемое *вспомогательной генерацией*, которое применяется к регрессивным моделям LLM. Этот метод использует две модели с одинаковой архитектурой для одной и той же задачи. Одна из них — небольшая и быстрая, а другая — большая и обладает гораздо лучшим качеством. Маленькая модель генерирует несколько токенов как обычно, и после того, как некоторые из них собраны, они передаются большой модели для подтверждения. Большая модель проверяет их за один проход, а не через цикл генерации. Используются только токены, подтвержденные большой моделью (остальные отбрасываются), но при условии, что малая и большая модели часто совпадают. Это работает быстрее, чем использование только большой модели. Подход особенно полезен для структурированного текста вроде кода. При этом потребление памяти увеличится, поскольку задействуются две модели вместо одной, но пропускная способность при этом увеличивается.

Несколько похожий метод — *декодирование медузы*. Это вариация тонкой настройки, которая добавляет новые *головы*³ к существующей модели, чтобы можно было генерировать последовательность из нескольких токенов одновременно. Как и в предыдущем случае, здесь выполняется проверка нескольких кандидатов, чтобы сгенерированный текст был таким же, как если бы медуза не использовалась.

Вот несколько ресурсов, которые помогут узнать больше об этой теме:

- ◆ Публикация Мерв Нойан (Merve Noyan) на Hugging Face представляет собой краткое введение в квантование и, в свою очередь, предоставляет дополнительные источники для более глубокого изучения темы.
- ◆ Кодовая база llama.cpp на GitHub содержит множество методов квантования и оптимизации, включая собственные ядра, для ускорения вывода на потребительском оборудовании, включая Windows и Apple Silicon.
- ◆ Компания TheBloke (Том Джоббинс, Tom Jobbins) подготовила квантованные, готовые к использованию версии нескольких LLM. Модели, подготовленные ею, были скачаны миллионы раз.
- ◆ Документация Hugging Face содержит исчерпывающую информацию о спекулятивном декодировании и способах его использования.
- ◆ Публикация «Assisted Generation: a new direction toward low-latency text generation» Жоао Ганте (Joao Gante) — отличное введение во вспомогательную генерацию / спекулятивное декодирование.
- ◆ Также возможны варианты спекулятивного декодирования с потерями. В публикации «An Optimal Lossy Variant of Speculative Decoding» Вивьен Тран-Тьен (Vivien Tran-Thien) проделала фантастическую работу по исследованию этого направления.

³ Так называются сети, которые дополняют работу базовой модели. Например, классификатор (классификационная сеть) — это голова для декодера. — Пер.

Данные

Несмотря на то что большая часть этой книги посвящена моделям и их применению, данные являются ключевым аспектом машинного обучения. Большинство моделей тренируются на огромных объемах веб-данных. Тем не менее, фильтруя или генерируя синтетическую информацию с помощью LLM, модели меньшего размера могут достигать производительности, сопоставимой с гораздо более крупными моделями. Высококачественные данные могут значительно повысить производительность даже небольших моделей, вроде Microsoft Phi-3. FineWeb — это высококачественный отфильтрованный набор открытых веб-данных, выпущенный с полностью открытым исходным кодом. Он имеет технический отчет, который демонстрирует, как токены были отфильтрованы для достижения высокого качества.

Несмотря на то что синтетические данные не являются чем-то новым, Phi открыла двери для исследования больших наборов (миллиардов токенов), созданных исключительно с помощью моделей LLM. Первая вариация Phi, модель с 1,3 миллиарда параметров, была обучена на 6 миллиардах токенов высококачественных веб-данных и 1 миллиарде токенов из учебников и упражнений, сгенерированных с помощью GPT-3.5. Кроме того, Cosmopedia — это первый проект, выпустивший набор данных из 25 миллиардов токенов синтетических данных, охватывающих различные темы, и созданный с помощью Mixtral 8x7B.

Такие инструменты, как distilabel от Argilla, упрощают создание синтетических данных и получение обратной связи от ИИ, которые можно использовать как для предварительного обучения, так и для RLHF. В то время как Phi и Cosmopedia фокусируются на больших объемах синтетической информации, модели машинного обучения все чаще используются для создания наборов предпочтений и масштабирования оценки моделей.

Мы также рекомендуем прочитать публикацию Юджина Яна (Eugene Yan) о создании и использовании синтетических данных, статью про TinyStories и оригинальную статью Phi «*Textbooks Are All You Need*», а также отчеты по Phi-1.5 и Phi-3.

Для работы с другими модальностями (например, изображения или аудио) нам нужны не только соответствующие наборы, но и текстовые наборы, которые содержат описания или транскрибирование данных. Существует множество наборов с парами «текст-изображение», например LAION-2B (использовался для обучения ранних моделей Stable Diffusion), COYO 700M и DataComp 1B. Однако извлечение миллиардов неотфильтрованных изображений из Интернета может включать неприемлемый или незаконный контент, поэтому модели, обученные на неотфильтрованных данных, могут оставлять вопросы открытыми относительно авторских прав и добросовестного использования. Для смягчения этих проблем Creative Commons выпустила альтернативные наборы с открытыми лицензированными парами «изображение-текст», например Common-Canvas. Для аудио широко используются открытые речевые наборы, такие как Common Voice от Mozilla, а для звуковых эффектов — FreeSound и Free Music Archive. Для аудио также существуют наборы, извлеченные из Интернета и имеющие лицензию, но в области изображений нет централизованного индекса наборов данных, эквивалентного COYO или DataComp.

Одна модель, чтобы править всеми

Существуют три основных подхода к использованию LLM и других генеративных моделей для вашего конкретного сценария. Они перечислены в порядке убывания сложности:

1. Обучение новой модели с нуля.
2. Тонкая настройка предварительно обученной модели для конкретного сценария.
3. Использование существующей модели с промптами или RAG.

Выбор между этими подходами зависит от ресурсов и приоритетов создателя. Обучение модели с нуля в настоящее время обходится большинству компаний непомерно дорого. К счастью, появление высококачественных моделей с открытым исходным кодом упрощает получение отличных результатов с помощью тонкой настройки. Такие модели, как BloombergGPT (для финансов), AstroLLaMA (для астрономии) и BioMistral (для медицины), демонстрируют, что тонкая настройка сильной предварительно обученной модели с использованием данных, специфичных для конкретной области применения, иногда может дать результаты более высокого качества, чем использование готовой модели.

Тем не менее по мере того, как возможности моделей с нулевым выстрелом улучшаются, их производительности может быть достаточно для многих вариантов использования без какой-либо тонкой настройки. В зависимости от требований проекта время, затраченное на качество данных и итерации модели, можно было бы эффективнее направить на конечный продукт. Например, когда Meta выпустила Llama 3 и ее высококачественную версию с инструкциями, многим исследователям было сложно добиться лучших результатов с помощью тонкой настройки, поскольку модель уже была очень мощной изначально.

Другим важным фактором является способ развертывания и использования модели. Варианты включают в себя самостоятельное размещение, использование готовых решений облачного провайдера или применение API, предоставляемого авторами модели (например, OpenAI или Cohere). Оптимальный выбор зависит от ваших конкретных потребностей. ИИ-сообщество все чаще использует инструменты (например, langchain и llamaindex), которые упрощают переключение между решениями, убирая зависимость от одного поставщика. Мы рекомендуем разрабатывать систему таким образом, чтобы она позволяла легко оценивать и заменять модели по мере необходимости.

Компьютерное зрение

Компьютерное зрение — обширная область с богатой историей, которая появилась еще до методов машинного обучения (ML). Оно направлено на извлечение содержательной информации из изображений и ее применение для принятия решений или выполнения действий, таких как управление автономным транспортным средством, обнаружение дефектов на заводской линии, подсчет объектов или мониторинг дорожного движения.

Методы ML произвели революцию в компьютерном зрении, и, в свою очередь, постоянное совершенствование задач в этой области стимулировало исследования все более мощных методов обучения представлением, что привело к взрывному развитию генеративных технологий распознавания изображений.

Традиционно компьютерное зрение рассматривается как набор отдельных задач, поскольку их проще разбить на более мелкие, чем, например, пытаться распознавать зрительные образы за один раз. Задачи компьютерного зрения включают в себя следующее:

◆ *Классификация изображений*

Прогресс в решении этой задачи значительно ускорился с 2009 года, когда был опубликован набор данных ImageNet на конкурсе ImageNet Large Scale Visual Recognition Challenge (ILSVRC). В 2012 году глубокая нейронная сеть, созданная Алексом Крижевским (Alex Krizhevsky), Ильей Суцкевером (Ilya Sutskever) и Джефффри Э. Хинтоном (Geoffrey E. Hinton), выиграла соревнование того года, показав после многих лет постепенного прогресса невероятную производительность — на 41% выше, чем у второго претендента. Это событие считается началом революции в глубоком обучении, поскольку коренным образом изменило подход к задачам машинного обучения и компьютерного зрения.

◆ *Обнаружение объектов*

Эта задача связана с поиском объектов или субъектов определенного типа на изображении (рис. 10.5). Модель генерирует прямоугольники (ограничивающие рамки) вокруг объектов, принадлежащих классам, и выдает оценку, насколько она уверена в том, что объект внутри каждой рамки принадлежит предсказанному классу.



Рис. 10.5. Некоторые объекты, обнаруженные моделью YOLOv10, из набора распознаваемых ею классов объектов

◆ Сегментация

Эта задача идет следом за обнаружением объектов. Она касается классификации каждого отдельного пикселя изображения в соответствии с набором предопределенных классов. Например, модель сегментации, обученная на городских фотографиях, будет предсказывать пиксели на изображении, соответствующие, например, дороге, дереву, людям, идущим по улице, или автомобилям. Это называется семантической сегментацией, когда нет различия между несколькими объектами одного класса (всем пикселям, принадлежащим людям, будет присвоен один и тот же идентификатор метки), или паноптической сегментацией, когда модель способна различать отдельные экземпляры объектов, принадлежащих одному классу.

◆ Оценка глубины

В этой задаче происходит оценка расстояния между объектами с помощью одного лишь изображения без дополнительной информации (рис. 10.6). Ее называют монокулярной оценкой глубины, чтобы отличить ее от других систем, использующих стереовходы (два изображения) или другие типы дополнительных данных. Монокулярная оценка глубины полезна для компьютерной графики, 3D, игр, фотографии и художественных задач.

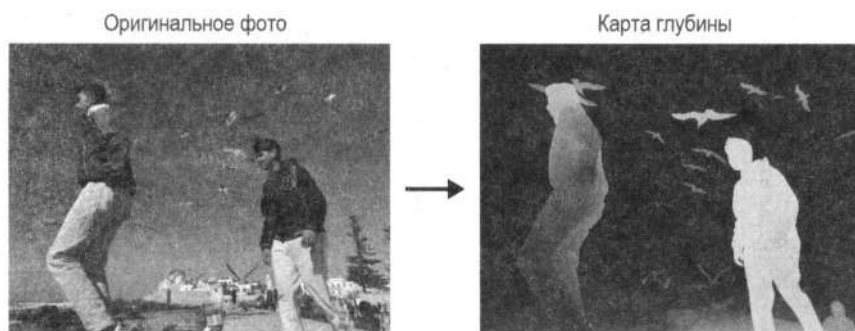


Рис. 10.6. Модель Depth Pro от Apple используется для оценки карты глубины по фотографии

Модели классификации, обнаружения и сегментации изображений традиционно обучаются на определенном наборе известных классов. Например, проект ImageNet проводился на 1000 категориях (хотя исходный набор данных содержит около 22 000 классов). Это создает проблему масштабирования, когда для обучения моделей, понимающих мир, необходимо использовать достаточное количество данных, чтобы охватить все возможные классы и нюансы различий между ними. Задачи с нулевым выстрелом (Zero-shot) относятся к способности модели выполнять эти задачи без предварительной подготовки на определенном наборе классов. Например, модель CLIP обучена на парах «изображение-текст», загруженных из Интернета, поэтому она хорошо понимает признаки изображения и текста, которые обычно сочетаются друг с другом. Как мы видели в главе 3, обученная модель CLIP может использоваться для классификации изображений по набору произвольных классов, указанных пользователем, без предварительной подготовки на изображениях, классифицированных по этим категориям.

По мере того как более крупные модели обучаются на всё больших объемах данных, они могут решать задачи с нулевым выстрелом без предварительной подготовки. Также эти крупные модели обучаются использовать богатые и описательные представления, что упрощает их тонкую настройку под конкретные задачи. Более того, мультимодальные модели, сочетающие текстовое и визуальное представления (также известные как *зрительные языковые модели*), могут отвечать на вопросы на естественном языке, что позволяет использовать их в различных рабочих процессах вместо специализированных моделей, ориентированных на конкретные задачи. Возникает вопрос: находимся ли мы на пороге новой революции компьютерного зрения, когда обучение на огромных наборах данных может оказаться более плодотворным, чем сосредоточение на конкретных задачах? Или же более мелкие, тонко настроенные модели сохранят преимущество перед моделями общего профиля?

Компьютерное зрение 3D

В то время как традиционное компьютерное зрение в основном фокусируется на двумерных данных (изображения и видео), в обществе растет интерес к пониманию, интерпретации и созданию трехмерных объектов. Компьютерное зрение 3D широко применяется в робототехнике, дополненной реальности, здравоохранении, видеопроизводстве, играх и автономном вождении. Традиционно трехмерные данные представляются с использованием сеток — совокупности вершин, ребер и граней, определяющих форму объекта (рис. 10.7). Однако методы машинного обучения часто испытывают трудности с этим, что побуждает к исследованию альтернативных представлений, таких как нейронные поля излучения NeRF (Neural Radiance Fields) и *гауссовское разбрызгивание*.

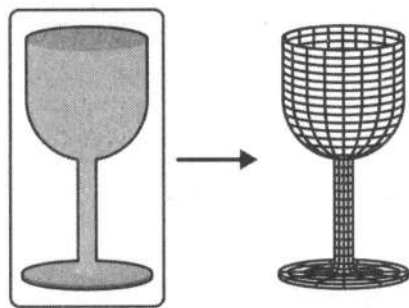


Рис. 10.7. Сетку можно создать даже из одного изображения

В отличие от обработки естественного языка (NLP), экосистема машинного обучения 3D невелика и в значительной степени ориентирована на исследования. В результате многие открытые инструменты остаются экспериментальными и находятся на ранних стадиях разработки. Однако эта область быстро развивается. Например, первая статья по NeRF была опубликована в 2020 году, а гауссовское разбрызгивание появилось в 2023 году. Для тех, кто заинтересован в дальнейшем изучении этой области, можем порекомендовать несколько ресурсов:

- ♦ Фрэнк Деллаэрт (Frank Dellaert) опубликовал посты в блоге в 2020–2022 годах с обзором событий в экосистеме NeRF. Рекомендуем читать их в хронологическом порядке.
- ♦ Hugging Face предлагает бесплатный курс по машинному обучению для 3D. Он представляет собой общий практический обзор методов генеративного машинного обучения, которые можно применять на различных этапах конвейера 3D-рендеринга.

Генерация видео

Поскольку модели генерации изображений зарекомендовали себя как перспективные в генеративном машинном обучении, логичным продолжением исследований стала область видео. Оно, по сути, представляет собой последовательность изображений, движущихся достаточно быстро, чтобы создать у человеческого мозга иллюзию движения. Таким образом, одним из подходов в генерации видео, вполне ожидаемо, стало использование предварительно обученных моделей, которые генерируют изображения и объединяют их в визуально согласованные последовательности (рис. 10.8). Фреймворки вроде AnimateDiff позволяют создавать анимированные видео из текстовых описаний или изображений. Конвейеры вроде Deforum могут создавать анимацию с контролем камеры, используя вариации кадров как часть эстетики видео.

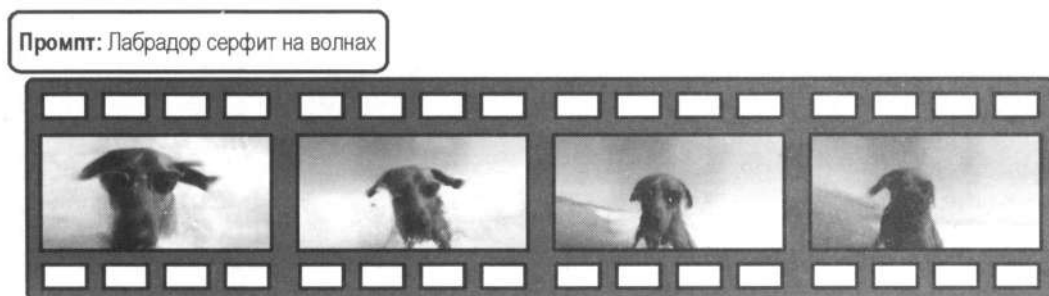


Рис. 10.8. Mochi-1 Preview от Genmo — это модель преобразования текста в видео, которая позволяет создавать короткие клипы на основе текстовых промптов

Методы преобразования видео в видео (video-to-video) также являются способом использовать генеративные модели в этой сфере. Используя уже существующую видеозапись, можно придавать ей новые стили или сюжеты. Модель будет преобразовывать ее или превращать наброски в анимацию. В статье «*Structure and Content-Guided Video Synthesis with Diffusion Models*» компании RunwayML представлен эффективный и производительный метод преобразования видео в видео.

Однако нативные техники для более эффективного создания новых видео находятся еще в стадии исследований и активно развиваются. Одним из ограничений нативных моделей является сложность создания пар «видео-текст», которые могли бы быть семантически значимыми и эффективными для обучения.

Для решения этой проблемы такие модели, как CogVideo от Университета Цинхуа (первая модель преобразования текста в видео с открытым доступом) и Make-A-Video, выпущенная Meta AI в 2022 году, используют методы для сопряжения видео без описаний с наборами данных и методами text-to-image. В частности, модель Make-A-Video преодолевает ограничение на данные, связанные с парами «видео-текст», обучаясь пониманию визуального восприятия на основе пар «изображение-текст» и объединяя эти знания с пониманием движения, полученным из неконтролируемых и неподписанных видео. Таким образом, она может изучать внешний вид объектов и их движение без явного сопоставления видео с текстом, что позволяет легко выполнять задачи text-to-video и image-to-video.

Эта методика, примененная к Stable Diffusion, позволила провести дальнейшие исследования для более эффективных открытых моделей text-to-video вроде ModelScope от Alibaba. Масштабирование видеонаборов для обучения подобных моделей также было достигнуто с помощью Stable Video Diffusion, которая используется для обучения видеоверсии Latent Diffusion и объединяет ее со Stable Diffusion. Вариант Stable Video Diffusion для преобразования изображения в видео был выпущен с открытыми весами и улучшенным качеством по сравнению с моделями Make-A-Video и ModelScope. Также были анонсированы дополнительные масштабируемые подходы к генерации видео, начиная с Sora — сверхвысококачественной, реалистичной и когерентной по времени модели text-to-video от OpenAI. Несмотря на то что результаты модели впечатляют, в ее техническом отчете раскрыто мало подробностей. В этой же области Google DeepMind представил Veo — своего конкурента Sora. Kuai выпустили Kling — первую общедоступную модель уровня Sora, а RunwayML спустя несколько недель выпустила Gen-3.

Что касается открытого исходного кода и открытых весовых коэффициентов, группа THUDM из Университета Цинхуа выпустила CogVideoX 2B и CogVideoX 5B — модели одного семейства с открытыми весами, способные генерировать реалистичный и согласованный по времени текст в видео наравне с ранее упомянутыми закрытыми альтернативами. Также активно развивается проект по созданию открытого исходного кода для репликации Sora с помощью модели Open-Sora, которая демонстрирует стремительный прогресс.

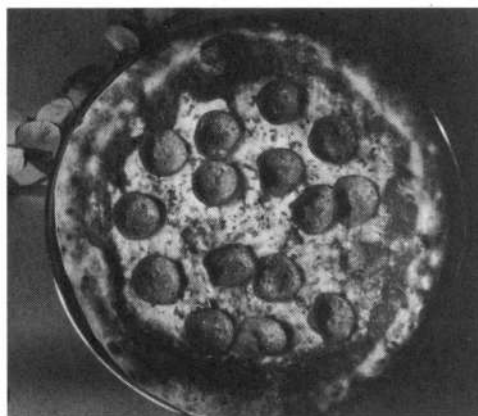
Мультимодальность

В предыдущих главах мы рассматривали использование генеративных моделей для нескольких модальностей: текст, изображение и аудио. Некоторые из них, например Stable Diffusion, имеют ввод одной модальности (текст) и генерируют вывод в другой (изображения). Однако некоторые модели, например для преобразования текста в речь и речи в текст, обычно не относятся к тому, что сообщество называет *мультимодальными моделями*. Мультимодальность обычно означает, что одна модель может либо обрабатывать входные данные, либо выдавать выходные данные в более чем одной модальности одновременно. Рассмотрим некоторые из последних достижений в этой области.

CLIP — это архитектура моделей, представленная OpenAI в 2021 году, обученная на миллионах изображений и описаний к ним. После обучения модель может при-

нимать на вход как изображения, так и текст, причем эти входные данные кодируются для работы в одном и том же семантически релевантном векторном пространстве. Данная способность позволяет выполнять широкий спектр задач с нулевым выстрелом, например классификацию изображений и семантическое сравнение изображений и текста. Введение в CLIP было представлено в главе 3. В главе 5 мы показали, как энкодер текста CLIP может быть компонентом Stable Diffusion.

Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation (BLIP) — это и фреймворк, и архитектура модели, представленные компанией Salesforce в 2022 году. Как и CLIP, BLIP обучена на парах «изображение-текст» и выполняет дальнейшее декодирование выходных данных в текст. Мультимодальный ввод (текст, изображение или все вместе) позволяет ей выполнять задачи с нулевым выстрелом, например создание подписей к изображениям и визуальные ответы на вопросы (рис. 10.9). Следующие версии (BLIP-2 и далее) дополнительно усовершенствовали эту концепцию, объединив в себе замороженный (необучаемый) энкодер изображений и LLM.



Пицца с пепперони и сыром на тарелке

Рис. 10.9. Подписи с помощью BLIP

Визуальные языковые модели (Visual Language Model, VLM), иногда называемые большими визуальными языковыми моделями (*Visual Large Language Model, VLLM*), могут принимать как изображения, так и текст в качестве ввода и выдавать текстовый вывод (рис. 10.10). Основополагающей работой в этой области является статья «*Flamingo: a Visual Language Model for Few-Shot Learning*», опубликованная DeepMind в 2022 году, но сама модель так и не была выпущена в релиз. Существуют ее репликации с открытым исходным кодом, например IDEFICS. Обучение VLM с нуля оказалось дорогостоящим. Однако подходы, в которых уже существующая предварительно обученная LLM дополнительно настраивается для приема выходных изображений с помощью замороженного энкодера, доказали свою эффективность и производительность. Архитектуры вроде BLIP-2 стали пионерами данного подхода, но они все еще имели относительно узкую предметную область применения (описания, ответы на вопросы и т. д.) и не сохраняли всех возможностей LLM.

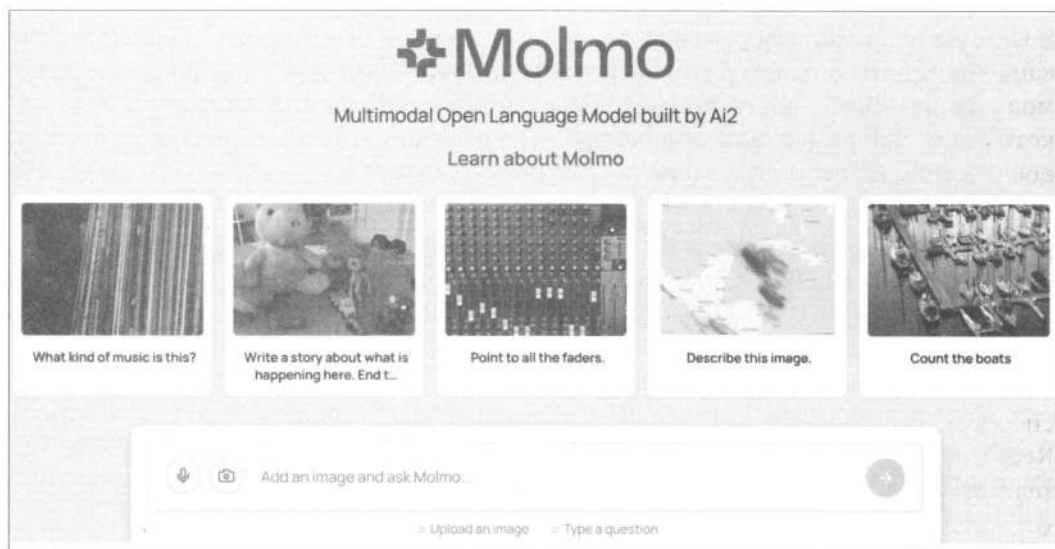


Рис. 10.10. Пример интерфейса VLM

LLaVA — архитектура, представленная Microsoft Research в 2023 году, позволяет энкодеру зрения и LLM объединяться, сохраняя все возможности рассуждения. Этот подход привел к взрывному развитию технологий и моделей в сообществе разработчиков ПО с открытым исходным кодом: от миниатюрных и эффективных моделей до современных новейших VLM, конкурентоспособных относительно коммерческих моделей. Чтобы быть в курсе последних тенденций, вы можете следить за рейтингом Open VLM Leaderboard или за трендами относительно image-to-text на сайте Hugging Face. Чтобы получить практический опыт в этой области, можно начать с вывода VLM на основе трансформеров. Коммерческие модели, такие как GPT-4 от OpenAI или Claude от Anthropic, также имеют компонент машинного зрения VLM.

Используя ту же логику, что и VLM, можно реализовать другие модальности, используя замороженный энкодер. Например, Gazelle — это совместная модель речи и языка, использующая предварительно обученный LLM (Mistral 7B) и замороженный аудиоэнкодер (Wav2Vec2). Модель способна обрабатывать и строить выводы непосредственно на основе аудиовхода.

Мультимодальные модели вывода — это следующий рубеж. Исследования и коммерческие решения в этой области развиваются, и в скором времени ожидается появление новой волны открытых мультимодальных моделей вывода. Одним из основополагающих исследований в этой области является архитектура Chameleon от Meta AI, которая может не только принимать текстовый и графический ввод, но и генерировать текстовый и графический вывод. Она обладает возможностями для выполнения таких задач, как следование инструкциям по изображениям, редактирование изображений, создание подписей к изображениям, ответы на вопросы и многое другое. На сегодняшний день уже представлены архитектуры с еще более широкими возможностями мультимодального ввода/вывода, например Unified-IO

и Unified-IO 2 от Института искусственного интеллекта Аллена, но их масштабирование пока не получило существенного значения. Что касается коммерческих моделей, то в мае 2024 года OpenAI выпустила мультимодальную модель GPT-4o, которая может принимать изображения, текст и аудио и выводить те же модальности (изображения, текст и аудио).

Сообщество

Как вы, вероятно, заметили на протяжении как всей главы, так и книги, область машинного обучения развивается очень быстро. Лучший способ быть в курсе последних исследований и разработок — стать частью ИИ-сообщества. Есть много способов сделать это, например присоединиться к Discord или сообществам на Reddit, делиться своими работами с другими, читать статьи и следить за исследователями и практиками в X (ранее Twitter).

Удивительно, что мы перешли от мира, где все исследования проводились в традиционных лабораториях, к весьма впечатляющим исследованиям в децентрализованных системах. EleutherAI, Nous Research, BigCode и LAION — это все примеры актуальных сообществ, и присоединиться к их работе можно так же просто, как зайти на их серверы Discord. Многие из них также проводят сессии, увлекательные обсуждения в чатах и хакатоны.

Некоторые из этих сообществ основаны отдельными участниками, которые начали свой путь с экспериментов над открытыми моделями, чтобы удовлетворить собственные потребности и решить важные для себя задачи. Несмотря на высокую стоимость обучения крупных моделей с нуля, для всех, кто интересуется этой областью, существует множество возможностей для исследований и разработок. Сейчас самое подходящее время начать.

Инструменты с открытым исходным кодом

Эта книга была бы невозможна без открытого исходного кода. Большинство обсуждаемых тем и большая часть исследований в области машинного обучения опираются именно на разработки с открытым исходным кодом, не говоря уже о наборе инструментов для практики, который мы использовали, включая программное обеспечение: Jupyter Notebook, Quarto, nbdev и многое другое.

В этом приложении мы рассмотрим различные инструменты для специалистов по машинному обучению. Некоторые из них мы уже использовали в книге, а о других полезно просто знать. Ознакомившись с ними, вы будете хорошо подготовлены и сможете расширить возможности приложений и методов, которые вы только что изучили.

Стек Hugging Face

В этой книге вы познакомились с основными библиотеками Hugging Face. Мы использовали две основные:

◆ transformers

Основная библиотека для обучения и вывода с использованием моделей на основе трансформеров в различных модальностях. Она предоставляет несколько уровней абстракции, от высокоуровневого конвейера и класса `Trainer` до поддержки запуска собственных обучающих циклов PyTorch.

◆ Diffusers

Аналогично transformers библиотека diffusers позволяет запускать предварительно обученные модели диффузии. Несмотря на то что она в основном известна своими возможностями генерации изображений, библиотека также поддерживает аудио, видео и 3D.

Обе библиотеки имеют продуманный дизайн, в котором приоритет отдается удобству использования, простоте и настраиваемости. Что это означает для конечных пользователей? Во-первых, обе библиотеки стремятся предоставить единообразные спецификации для всех моделей. Независимо от того, используете ли вы Llama или Gemma, переключение между ними в идеале должно осуществляться одной строкой кода. Несмотря на то что обе библиотеки обладают множеством функций для быстрого вывода, модели по умолчанию всегда загружаются с максимальной точностью и минимальной оптимизацией. Это обеспечивает удобство использования

на разных платформах и позволяет избежать сложных установок, но также означает, что модели будут работать медленнее «из коробки», если не настроены необходимые параметры оптимизации.

Также широко используются две дополнительные библиотеки Hugging Face:

◆ `accelerate`

Позволяет запускать код PyTorch в распределенных условиях как для обучения, так и для вывода. Независимо от того, хотите ли вы запустить модель на одном GPU, нескольких GPU, GPU с разгрузкой CPU или полностью на CPU, библиотека абстрагируется от всех сложностей. Он используется как в `transformers`, так и в `diffusers`, поэтому большинству пользователей не нужно много знать о ее API-интерфейсах.

◆ `peft`

Эта библиотека предоставляет эффективные с точки зрения параметров методы тонкой настройки для моделей с меньшими вычислительными затратами и возможностями хранения. Она хорошо интегрирована с `transformers` и `diffusers`. Несмотря на то что книга в основном рассматривает методы LoRA, существует множество других подходов вроде р-настройки, префиксной настройки, IA3, OFT и DoRA.

Данные

Экосистема с открытым исходным кодом для обработки, маркировки и генерации наборов данных значительно выросла за последние годы. Ниже приведен краткий список некоторых инструментов, разрабатываемых для работы с данными:

◆ `datasets`

Эта книга в значительной степени опирается на `datasets` — популярную библиотеку для доступа, обмена и обработки касательно открытых наборов данных для различных модальностей. Подобно `transformers` и `diffusers`, `datasets` предоставляет согласованный API, позволяющий пользователям легко менять наборы данных для заданной модальности.

◆ `argilla`

Это инструмент для создания высококачественных наборов данных. Он предоставляет простой пользовательский интерфейс, с помощью которого вы можете оценивать данные, что является полезным для таких задач, как сравнение генераций (важно для RLHF), создание наборов данных для классических задач обработки естественного языка (например, распознавания сущностей) или создание наборов данных для оценки.

◆ `distilabel`

С развитием генерации синтетических данных появились новые инструменты вроде `distilabel`. Он позволяет создавать конвейеры для генерации синтетических данных.

Обертки

По мере развития экосистемы вокруг моделей-трансформеров были созданы различные инструменты для сообщества и исследователей:

◆ `axolotl`

Этот инструмент упрощает тонкую настройку модели. Вам достаточно создать файл конфигурации, чтобы выполнить тонкую настройку. Он поддерживает распространенные форматы наборов данных и архитектуры моделей.

◆ `unsloth`

Инструмент обеспечивает чрезвычайно быструю тонкую настройку LLM-моделей на основе трансформеров, используя класс `FastLanguageModel`, включающий оптимизированные ядра.

◆ `sentence-transformers`

Как обсуждалось в *главе 2*, модели-трансформеры также можно использовать при вычислениях векторных представлений для целого предложения, абзаца или документа. Библиотека `sentence-transformers` предоставляет простые API-интерфейсы для вычисления векторных представлений с помощью предварительно обученных моделей или для тонкой настройки ваших собственных моделей.

◆ `trl`

Библиотека `trl` предоставляет простой API для тонкой настройки и согласования моделей-трансформеров и моделей диффузии. В *главе 6* мы показали, как выполнять тонкую настройку с учителем (SFT), но `trl` также содержит множество других методов, например моделирование вознаграждения и DPO.

Локальный вывод

Одним из основных преимуществ открытых моделей является возможность запускать многие из них локально на вашем собственном оборудовании. Это обеспечивает множество преимуществ: конфиденциальность, настраиваемость и локальную интеграцию (например, использование модели кода в качестве локального расширения IDE). В зависимости от вашего варианта использования можно выбрать различные инструменты:

◆ `llama.cpp`

Позволяет выполнять вывод LLM на различном оборудовании. Поддерживает разные методы квантования (от 1,5 до 8 бит) и пользуется широкой популярностью в ИИ-сообществе. Обычно применяется либо для локального общения с LLM, либо для настройки локальной конечной точки, чтобы можно было использовать ее другим локальным сервисом.

◆ `Transformers.js`

Позволяет запускать модели непосредственно в браузере без необходимости использовать сервер. Это может быть очень полезно при простом развертывании

сервисов с низкой задержкой и без затрат на вывод, а также для случаев, когда конфиденциальность играет первостепенную роль, например для создания транскрипций звонков или субтитров в режиме реального времени.

Инструменты развертывания

Выполнение вывода для одного промпта может быть простым, однако развертывание LLM в рабочей среде — более сложная задача. Для этого доступно множество инструментов, включая два популярных:

◆ **vLLM**

Простая библиотека для обслуживания LLM, гибкая и хорошо интегрированная с популярными моделями.

◆ **TGI**

Готовый к использованию набор инструментов для развертывания LLM.

Существуют и другие инструменты, например `lmdeploy` и `TensorRT-LLM` от NVIDIA. Учитывая такое разнообразие альтернатив, вы можете задаться вопросом, какой из них выбрать. Мы рекомендуем изучить их все и выбрать наиболее подходящий для ваших сценариев использования. Все они находятся в активной разработке, обладают разным уровнем охвата моделей, принятия сообществом, масштабируемости, интеграции с облачными сервисами и локальными средами и т. д.

Требования к памяти для моделей LLM

Модели бывают самых разных размеров. Например, Llama 3.1 имеет варианты 8B, 70B и 405B (B означает «миллиард»). Для загрузки и использования LLM требуется достаточно памяти, чтобы хранить модель. Количество параметров и их точность среди прочих факторов влияют на требования к памяти.

Что делать, если памяти недостаточно? Попробуйте следующие варианты:

- ◆ Уменьшите точность используемой модели. Вместо float16 можно использовать int8.
- ◆ Используйте модель меньшего размера. Существует множество высококачественных моделей малого размера.
- ◆ Выгрузите неиспользуемые части модели. Это можно сделать через *разгрузку оперативной памяти процессора (CPU RAM offloading)* — распространенного метода снижения требований модели к памяти за счет снижения скорости вывода. Что произойдет, если памяти не хватит? Оставшиеся части модели можно сохранить на диске и подгружать по мере необходимости. К счастью для нас, библиотека accelerate «заботится» об этом с помощью параметра `device_map="auto"`, который автоматически выгружает части модели по мере необходимости.

Требования к памяти вывода

Приблизительно оценить требования к памяти можно по формуле

Требуемая память GPU = Количество параметров × Байтов на параметр.

Количество байтов на параметр зависит от используемой точности. Не вдаваясь в подробности, табл. В.1 показывает объемы памяти, необходимые для загрузки моделей размером 2B, 8B, 70B и 405B с использованием различных уровней точности (float32, float16, int8, int4 и int2).

Таблица В.1. Требования к памяти вывода для моделей и уровни точности

Размер модели	float32	float16	int8	int4	int2
2B	8 ГБ	4 ГБ	2 ГБ	1 ГБ	512 МБ
8B	32 ГБ	16 ГБ	8 ГБ	4 ГБ	2 ГБ
70B	280 ГБ	140 ГБ	70 ГБ	35 ГБ	17.5 ГБ
405B	1.62 ТБ	810 ГБ	405 ГБ	202.5 ГБ	101.25 ГБ

Для справки: GPU H100 от NVIDIA имеет 80 ГБ памяти, поэтому для загрузки Llama 3.1 405B потребуется как минимум один полный узел (из 8 экземпляров H100), чтобы загрузить модель в 8-битные целые числа. Имейте в виду, что это грубая оценка. Также нужно учитывать объем памяти, необходимый для входных и выходных тензоров, а также объем памяти, необходимый для промежуточных вычислений. Например, длинные последовательности требуют больше памяти, чем короткие, особенно когда мы переходим к более чем 100 000 токенам.

На момент написания книги мы можем квантовать модели до int8 с минимальной потерей производительности. Методы, использующие менее 8 бит на параметр, приводят к снижению производительности и являются областью активных исследований.

Требования к памяти для обучения

Расчет требований к памяти для обучения может оказаться более сложным, чем кажется на первый взгляд, поскольку он зависит от деталей реализации модели и сценария обучения. Требования к памяти также могут существенно меняться в зависимости от размера пакета, количества токенов в выборках набора данных, метода обучения (например, полная тонкая настройка или PEFT) и настройки параллелизма при обучении.

Подробное описание выходит за рамки этой книги. Тем не менее мы можем дать приблизительные оценки требований для обучения LLM. В табл. В.2 показаны приблизительные требования к GPU для тонкой настройки Llama.

Таблица В.2. Требования к памяти для моделей и методов обучения

Размер модели	Полная тонкая настройка	LoRA	QLoRA
8B	60 ГБ	16 ГБ	6 ГБ
70B	500 ГБ	160 ГБ	48 ГБ
405B	3.25 ТБ	950 ГБ	250 ГБ

Для дополнительного чтения

Если вам интересно узнать больше о требованиях к памяти для LLM, вы можете обратиться к следующим ресурсам:

- ♦ «*Transformer Math 101*» от EleutherAI, где подробно объясняются требования к памяти для обучения модели в различных конфигурациях.
- ♦ «*Breaking Down GPU VRAM Consumption*» — короткий пост, в котором описываются компоненты, потребляющие память GPU.

Сквозная генерация, дополненная поиском

LLM-модели часто применяются для генерации контента как на основе входных промптов, так и с помощью полученной извне информации. В этом приложении мы покажем, как построить конвейер, использующий предварительно обученные модель LLM и преобразователь предложений, чтобы генерировать контент на основе пользовательского ввода и набора документов. Мы рассматривали этапы этого процесса на протяжении всей книги. В *главе 2* обсуждалась генерация текста и использование преобразователей предложений для кодирования текста, а *глава 6* содержала проект, в котором мы создавали минимальный конвейер RAG.

Компоненты системы RAG (схематически показаны на рис. С.1):

1. Пользователь вводит вопрос.
2. Конвейер извлекает документы, наиболее похожие на вопрос.
3. Конвейер передает вопрос и извлеченные документы в LLM.
4. Конвейер генерирует ответ.



Рис. С.1. Упрощенный конвейер RAG

Обработка данных

Как и в любом проекте машинного обучения, первым шагом является загрузка и обработка данных. Для простоты мы сосредоточимся на одной теме. Представим, что нам нужно, чтобы модель генерировала контент, относящийся к Закону Евро-

пейского союза об искусственном интеллекте. Сам Закон не является частью обучающих данных LLM, поскольку модель, которую мы используем, обучена до начала его разработки. Сначала загрузим документ.

```
import urllib.request

#Определяем имя файла и URL
file_name = "The-AI-Act.pdf"
url = https://artificialintelligenceact.eu/wp-content/uploads/2021/08/The-AIAct.pdf

#Загружаем файл
urllib.request.urlretrieve(url, file_name)
print(f"{file_name} downloaded successfully.")
The-AI-Act.pdf downloaded successfully.
```

Документ, вероятно, слишком длинный для обработки за один раз, поэтому разделим его на более мелкие части и внедрим каждую из них отдельно. Каждый фрагмент представляет собой отдельный документ, который мы должны сравнить с пользовательским вводом. Для простоты воспользуемся инструментами предварительной обработки из библиотеки langchain, предоставляющей вспомогательные функции для создания систем RAG. Например, в ней есть удобный класс PyPDFLoader, который может извлекать текст из PDF-файлов и обрабатывать фрагменты.

Сначала установим необходимые зависимости.

```
!pip install langchain_community pypdf langchain-text-splitters
```

Далее загрузим и предварительно обработаем документ с помощью PyPDFLoader.

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader(file_name)
docs = loader.load()
print(len(docs))
```

108

Класс PyPDFLoader разбил наш PDF-файл на отдельные документы. В результате получилось 108 документов. Мы разобьем их на еще более мелкие фрагменты. Библиотека langchain предоставляет классы, которые помогают с различными типами разбиения текста. Мы будем использовать рекурсивный класс CharacterTextSplitter с двумя ключевыми параметрами:

◆ `chunk_size`

Определяет количество символов в каждом фрагменте. Как правило, рекомендуется связать параметр с максимальным количеством токенов, которое может обработать модель эмбедингов. Как правило, для большинства преобразователей предложений он имеет небольшой размер. В противном случае есть риск, что часть документа будет обрезана.

◆ `chunk_overlap`

Определяет количество символов, на которое каждый фрагмент перекрывает предыдущий. Он полезен, когда нужно избежать разбиения предложений по середине. Мы произвольно установим его равным 100 символам (одна пятая от выбранного нами размера фрагмента).

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

```
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=100
)
chunks = text_splitter.split_documents(docs)
print(len(chunks))
```

854

Далее сохраняем текстовые фрагменты в массив.

```
chunked_text = [chunk.page_content for chunk in chunks]
```

Один из фрагментов выглядит следующим образом.

```
chunked_text[404]
('user or for own use on the Union market for its intended '
 'purpose; \n'
 '(12) 'intended purpose' means the use for which an AI system is '
 'intended by the provider, \n'
 'including the specific context and conditions of use, as '
 'specified in the information \n'
 'supplied by the provider in the instructions for use, promotional '
 'or sales materials \n'
 'and statements, as well as in the technical documentation; \n'
 '(13) 'reasonably foreseeable misuse' means the use of an AI system '
 'in a way tha t is not in')
```

Эмбединги документов

Теперь, когда у нас есть документы (фрагменты), нужно создать их эмбединги. Мы будем использовать модель преобразователя предложений в качестве *извлека-теля (Retriever)*, который действует как поисковая система, находя наиболее релевантные фрагменты для заданного вопроса. Этот процесс основан на вычислении сходства между эмбедингами пользовательского запроса и эмбедингами документов в нашей коллекции. Для вычисления всех эмбедингов документов мы воспользуемся предварительно обученной моделью преобразователя предложений (BAAI/bge-small-en-v1.5), используя пример из главы 2. Код ниже загружает ее и использует для кодирования двух предложений.

```
from sentence_transformers import SentenceTransformer, util

sentences = ["I'm happy", "I'm full of happiness"]
model = SentenceTransformer("BAAI/bge-small-en-v1.5")
```

#Вычисляем эмбединги для обоих предложений

```
embedding_1 = model.encode(sentences[0], convert_to_tensor=True)
```

```
embedding_2 = model.encode(sentences[1], convert_to_tensor=True)
```

Преобразователи возвращают один эмбединг для всего предложения. Несмотря на то что модели-трансформеры обычно выводят один эмбединг для каждого токена, преобразователи обучены объединять эмбединги токенов в одно предложение, отражающее семантическое значение текста.

```
embedding_1.shape
```

```
torch.Size([384])
```

Затем мы можем сравнить документы, используя косинусное сходство.

```
util.pytorch_cos_sim(embedding_1, embedding_2)
```

```
tensor([[0.8367]], device='cuda:0')
```

Косинусное сходство (глава 3) представляет собой просто скалярное произведение двух векторов с эмбедингами.

```
embedding_1 @ embedding_2
```

```
tensor(0.8367, device='cuda:0')
```

Есть так же альтернативный вариант.

```
import torch
```

```
torch.dot(embedding_1, embedding_2)
```

```
tensor(0.8367, device='cuda:0')
```

Теперь, когда мы знаем, как создать эмбединг предложения, можем сделать это для всех документов.

```
chunk_embeddings = model.encode(chunked_text, convert_to_tensor=True)
```

Код ниже возвращает 384-мерный вектор с эмбедингами для каждого фрагмента.

```
chunk_embeddings.shape
```

```
torch.Size([854, 384])
```

Извлечение

Используя эмбединги документов, мы можем извлечь из них наиболее релевантные документы для заданного вопроса. Мы будем использовать тот же подход, что и раньше, для вычисления косинусного сходства между вопросом и каждым документом¹. К счастью, вычисление сходства не требует итераций, его можно эффективно выполнить с помощью встроенных примитивов умножения матриц PyTorch.

¹ Для этого случая вы также можете использовать удобный метод `semantic_search` из библиотеки `sentence_transformers`.

```
def search_documents(query, top_k=5):
    #Кодируем запрос в вектор
    query_embedding = model.encode(query, convert_to_tensor=True)

    #Вычисляем косинусное сходство между запросом и
    #всеми фрагментами документа
    similarities = util.pytorch_cos_sim(query_embedding, chunk_embeddings)

    #Получаем k наиболее похожих фрагментов
    top_k_indices = similarities[0].topk(top_k).indices

    #Извлекаем соответствующие фрагменты документа
    results = [chunked_text[i] for i in top_k_indices]

    return results
```

Рассмотрим пример. В книге мы сократили вывод для краткости изложения, но вы можете запустить код в своей локальной среде и изучить его полностью.

```
search_documents("What are prohibited ai practices?", top_k=2)

('TITLE II \n'
 'PROHIBITED ARTIFICIAL INTELLIGENCE PRACTICES \n'
 'Article 5 \n'
 '1. The following artificial intelligence practices shall be '
 'prohibited: \n'
 '(a) the placing on the market, putting into service o')
('low or minimal risk. The list of prohibited practices in Title II '
 'comprises all those AI systems \n'
 'whose use is considered unacceptable as contravening Unio n '
 'values, for instance by violating \n'
 'fundame')
```

Модель верно извлекла соответствующую информацию из входного вопроса.

Генерация

Следующий шаг — сгенерировать ответ на основе вопроса и найденных документов. Воспользуемся нашим старым добрым другом — версией модели SmolLM с инструкциями. Вы можете смело экспериментировать с другими моделями.

```
from transformers import pipeline

from genaibook.core import get_device

device = get_device()
generator = pipeline(
    "text-generation", model="HuggingFaceTB/SmolLM-135M-Instruct", device=device
)
```

Мы применив шаблон чата совместно с SmolLM. Как обсуждалось в *главе 6*, библиотека `transformers` имеет утилиты, которые форматируют промпт в соответствии с ожиданиями модели. В случае RAG нам нужно добавить извлеченные документы в промпт.

```
def generate_answer(query):
    #Извлекаем релевантные фрагменты
    context_chunks = search_documents(query, top_k=2)

    #Объединяем фрагменты в одну строку контекста
    context = "\n".join(context_chunks)

    #Генерируем ответ, используя контекст
    prompt = f"Context:\n{context}\n\nQuestion: {query}\nAnswer:"

    #Определяем контекст, который будет передан в модель
    system_prompt = {
        "You are a friendly assistant that answers questions about the AI Act."
        "If the user is not making a question, you can ask for clarification"
    }
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": prompt},
    ]

    response = generator(messages, max_new_tokens=300)
    return response[0]
```

Рассмотрим еще один пример.

```
answer = generate_answer("What are prohibited ai practices in the EU act?")

print(answer)

('The EU Act prohibits the use of artificial intelligence practices '
 'that are harmful to individuals, such as:\n'
 '\n'
 '* The placing on the market, putting into service or use of an A I '
 'system that is subliminal, that is, it is not intended to be used '
 'for any purpose other than to deceive or manipulate individuals.\n'
 '* The use of A I systems that are designed to deceive or '
 'manipulate individuals, such as those used in advertising, '
 'marketing, or customer service.\n')
```

Модель генерирует ответ на основе входного вопроса и корректно извлекает информацию. Используемая генеративная модель имеет небольшой размер, поэтому масштабирование до более крупного экземпляра позволит нам получать более качественные генерации и увеличить длину контекста, что, в свою очередь, позволит увеличить количество извлеченных документов, которые мы можем передать в контекст.

RAG на уровне производства

Приведенный код — это простой пример системы RAG. В системе промышленного уровня необходимо учитывать несколько дополнительных факторов:

◆ *Разбиение на фрагменты*

Одна из проблем с реальными данными заключается в том, что документы могут быть очень длинными. Поиск правильного размера фрагмента — это проектное решение, которое зависит от данных и модели. Слишком короткие фрагменты будут обрезать входные данные, а слишком большие — разбавлять их. Подробнее о разбиении можно узнать в статье «5 Levels Of Text Splitting».

◆ *Эмбединги меньшего размера*

Большие эмбединги могут требовать большого объема памяти. В настоящее время ведутся активные исследования, направленные на уменьшение размера эмбедингов при сохранении их качества. Если вам интересны эти темы, рекомендуем прочитать статью «Introduction to Matryoshka Embedding Models» и «Binary and Scalar Embedding Quantization for Significantly Faster & Cheaper Retrieval».

◆ *Повторное ранжирование*

Этап извлечения играет ключевую роль в системах RAG. Модели извлечения необходимы для сравнения тысяч и миллионов документов, но они не обязательно могут иметь большую точность. После извлечения `top_k` документов можно повторно ранжировать их, используя более медленную, но более качественную модель. Подробнее о повторном ранжировании можно узнать в документации к библиотеке `sentence_transformers` или в статье «Deep Dive into Cross-encoders and Re-ranking».

◆ *Оценка модели эмбедингов*

Существуют десятки преобразователей предложений. Чтобы выбрать наиболее подходящий для вашего случая, рекомендуем изучить бенчмарк Massive Text Embedding Benchmark, который содержит такую информацию, как размер модели, размерности эмбедингов и качество для десятков задач. Обратите внимание, для задач извлечения обычно требуются небольшие и быстрые модели, поэтому это должно стать ключевым фактором при принятии решения.

◆ *Компоненты производства*

В реальном использовании может потребоваться интеграция и других компонентов, таких как переписывание запросов, редактирование персонально идентифицируемой информации, кеширование и контроль входных данных, чтобы предотвратить ненадлежащее использование вашей модели. Подробнее об этом можно узнать в статье «Building A Generative AI Platform» Чипа Хуэна (Chip Huyen).

И в завершение надо сказать, что существует множество инструментов с открытым исходным кодом для создания систем RAG. Вот несколько рекомендаций:

◆ *ColBERT*

Быстрая и точная модель поиска на основе BERT.

◆ *RAGatouille*

Система для применения и обучения моделей поиска.

Открытые векторные базы данных вроде Milvus, Weaviate и Qdrant могут оказаться полезными при работе с большими наборами данных. Подробное рассмотрение векторных баз данных выходит за рамки этой книги, но является быстро развивающейся областью. Вы можете увидеть сравнение с 2023 годом в публикации «*Picking a vector database: a comparison and guide for 2023*».

Предметный указатель

8

8-битное квантование 235

A

absmax-квантование 235

B

BART (Bidirectional and Auto-Regressive Transformers) 57

Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation (BLIP) 371

Brain floating-point 234

C

CLAP (Contrastive Language-Audio Pretraining) 113

Classification head 203

CLIP (Contrastive Language-Image Pre-training) 111

D

DDIM-инверсия 284

DDPM-инверсия 284

F

Flow Matching 185

Frechet Audio Distance (FAD) 348

H

Holistic Evaluation of Text-to-Image Models (HEIM) 142

I

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 118

Inception Score 142

IP-адаптер (Image Prompt Adapter) 291

K

Kernel Inception Distance (KID) 142

M

Mixup 347

N

NeRF (Neural Radiance Fields) 368

P

PEFT (Parameter-efficient fine-tuning) 230

Pivotal Tuning 260

Prior Preservation Loss (PPL) 259

R

Rectified flow matching 185

Reinforcement Learning with Human Feedback (RLHF) 217

Rotary Position Embedding (RoPE) 205, 358

S

Semantic Guidance (SEGA) 280

State of the art (SOTA) 17

T

Top-p sampling *См. Выборка ядра (Nucleus sampling)*

U

ULMFiT (Universal Language Model Fine-tuning for Text Classification) 61

V

Variance preserving, VP 150

А

Автоматическое распознавание речи (Automatic Speech Recognition, ASR) 299
 Авторегрессивная языковая модель *См.*
 Казуальная языковая модель (Casual Language Model)
 Автоэнкодер (AutoEncoder) 26, 75
 Адаптация низкого ранга (Low-rank adaptation, LoRA) 231
 Адаптер 226
 Алгоритм Гриффина–Лима 335
 Аугментация 132

Б

Базовая модель 52
 Башня зрения 115
 Бенчмарк (Benchmark) 218
 Блок внимания 136
 Большая визуальная языковая модель (Visual Large Language Model, VLLM) *См.*
 Визуальная языковая модель (Visual Language Model, VLM)
 Большая языковая модель (Large Language Model, LLM) 33

В

Вариационный автоэнкодер (Variational AutoEncoder, VAE) 77, 97
 Вейвлет-преобразование (Wavelet transform) 158
 Взрывная дисперсия (Variance exploding, VE) 150
 Визуальная языковая модель (Visual Language Model, VLM) 371
 Вокодер (Vocoder) 335
 Временная область 307
 Вспомогательная генерация *См.*
 Спекулятивное декодирование
 Выборка (Sampling) 43
 ♦ Top-K (Top-K Sampling) 46
 ♦ Top-p (Top-p sampling) 43, 47
 ♦ ядра (Nucleus sampling) *См.* Выборка Top-p (Top-p sampling)

Г

Гармоническое среднее значение 207
 Гауссовский шум 143
 Гауссовское разбрызгивание 368
 Генеративно-сопоставительная сеть (Generative Adversarial Networks, GAN) 29, 127

Генеративный ландшафт 111
 Генерация речи из текста (Text To Speech, TTS) 299
 Глубокая сверточная
 генеративно-сопоставительная сеть (DCGAN) 29
 Глубокое обучение (Deep Learning) 29
 Глубокое слияние 320
 График шума (Noise schedule) 134

Д

Декодер (Decoder) 55
 Декодирование Медузы 363
 Дивергенция Кульбака–Лейблера (Kullback–Leibler Divergence, KLD) 102
 Динамический порог 187
 Дипфейк (Deepfake) 23
 Дискриминатор (Discriminator) 178
 Долгая краткосрочная память (Long Short-term Memory, LSTM) 55

Ж

Жадное декодирование 41

З

Заморозка весов 208
 Заполнение (Padding) 167

И

Извлекатель (Retriever) 383
 Измерение (Dimension) 38
 Инверсия (Inversion) 283

К

Карта признаков 67
 Катастрофическое забывание (Catastrophic forgetting) 258
 Казуальная языковая модель (Causal Language Model) 37
 Классификация с нулевым выстрелом 68, 118
 Кодирование байтовых пар на уровне байтов (Byte-level Byte-Pair Encoding, BPE) 37
 Кодовая книга (Codebook) 338
 Компьютерное зрение (Computer Vision) 61
 Контекстное окно 65
 Контрастная потеря (Contrastive loss) 111
 Контрастный поиск 48
 Контролируемое обучение *См.* Обучение с учителем (Supervised learning)
 Косинусное сходство 112

Коэффициент

- ◊ квантования 235
- ◊ ошибок в словах (Word Error Rate, WER) 326

Кратковременное преобразование Фурье 312

Краудсорсинг (Crowdsourcing) 301

Л

Латентное пространство (Latent space) 76

Линейная интерполяция 144

Логит (Logit) 38

Лучевой поиск 41

М

Маскировка случайных токенов 144

Масштабирование 152

Матрица

- ◊ низкого ранга 231

- ◊ обновлений 231

- ◊ ошибок 214

- ◊ путаницы См. Матрица ошибок

Машинное обучение (Machine Learning, ML) 23

Мел-спектрограмма 313

Моделирование замаскированного языка (Masked Language Modeling, MLM) 58

Модель

- ◊ VITS 336

- ◊ вознаграждений 355

- ◊ диффузии (Diffusion Model) 127

- ◊ мультимодальная 370

- ◊ пространства состояний (State Space Model, SSM) 358

Мультимодальное репрезентативное обучение (Multimodal representation learning) 77

Н

Настройка с помощью инструкций (Instruct-tuning) См. Тонкая настройка с учителем (Supervised fine-tuning, SFT)

Начальное расстояние Фреше (Frechet Inception Distance, FID) 141

Нейронная сеть прямого распространения (Feed-Forward Neural Network, FFNN) 54

Несколько выстрелов (Few-shot) 51

Нормализатор 327

Нулевой выстрел (Zero-shot) 49

О

Обработка естественного языка (Natural Language Processing, NLP) 30

Обусловливание (Conditioning) 165

Обучение

- ◊ с подкреплением и обратной связью от человека (Reinforcement Learning with Human Feedback, RLHF) 355

- ◊ с учителем (Supervised learning) 61

Окно внимания 358

Оптимизация

- ◊ политики диффузии шумоподавления (Denoising diffusion policy optimization, DDPO) 356

- ◊ предпочтений 355

Остаточное векторное квантование (Residual Vector Quantization) 338

Остаточный блок 136

Осциллограмма 305

Относительная энтропия 102

Оценка F1 (F1 score) 207

П

Паноптическая сегментация 367

Параметр температура 117

Перекрестная энтропия (Cross-entropy) 117

Перекрестное внимание (Cross-attention) 57

Переобучение (Overfitting) 157

Поверхностное слияние 320

Повышающая дискретизация 135

Поисково-дополненная генерация (Retrieval-Augmented Generation, RAG) 73, 246

Полная модель (Full model) 251

Полнота (Recall) 206

Понижающая дискретизация 135

Последовательность-последовательность (sequence-to-sequence) 55

Потери дискриминатора на основе патчей 179

Преобразование

- ◊ голоса (Speaker conversion) 332

- ◊ изображения в изображение (image-to-image) 273

- ◊ текста в аудио (Text To Audio, TTA) 299

- ◊ Фурье 309

Прецизионность (Precision) 206

Промпт (Prompt) 25

Пропускное соединение (Skip connection) 135

Пулер (Pooler) 116

Пулинг (Pooling) 153

- ◊ максимального значения 153

- ◊ среднего значения 153

Р

- Разгрузка (Offloading) 237
- ◊ оперативной памяти процессора (CPU RAM offloading) 379
- Разреженный автоэнкодер (Sparse AutoEncoder) 66
- Ранговая декомпозиция матриц (Rank decomposition matrix) 265
- Рекуррентная нейронная сеть (Recurrent Neural Networks, RNN) 29
- Рефайнер (Refiner) 184
- Руководство без классификатора (Classifier-free guidance, CFG) 185

С

- Самовнимание (Self-attention) 34
- Самообучение (Self-supervised learning) 78
- Сборщик данных (Data collator) 222
- Сверточная нейронная сеть (Convolutional Neural Networks, CNN) 29
- Семантическая сегментация 367
- Скользящее окно (Sliding window) 204
- Скраппинг (Scrapping) 190
- Скрытое состояние энкодера 177
- Смесь экспертов (Mixture of Experts, MoE) 220, 360
- Спектрограмма 306
- Спекулятивное декодирование 363
- Среднеквадратичная ошибка (Mean Squared Error, MSE) 103
- Средний балл мнения (Mean Opinion Score, MOS) 348

Т

- Текстовый энкодер 176
- Техника состязательного обучения 335
- Токен (Token) 33
- Токенизация (Tokenizing) 34
- Тонкая настройка (Fine-tuning) 52
- Тонкая настройка с учителем (Supervised fine-tuning, SFT) 226
- Точность (Accuracy) 206

- Транспонированная свертка 83
- Трансферное обучение (Transfer learning) 61
- Трансформер (Transformer) 33
- ◊ зрения (Vision Transformer, ViT) 67

У

- Управляемый рекуррентный блок (Gated Recurrent Unit, GRU) 55
- Условная модель 165
- Условный вариационный автоэнкодер (Conditional Variational Autoencoder, C-VAE) 98

Ф

- Фильтрация качества на основе модели 216
- Фолдинг белков (Protein folding) 33
- Функция потерь (Loss function) 102
- ◊ дискриминатора на основе локальных фрагментов 178

Ц

- Центральное кадрирование 114

Ч

- Частота
- ◊ дискретизации 304
- ◊ ошибок в символах (Character Error Rate, CER) 327

Э

- Экспоненциальное скользящее среднее (Exponential Moving Average, EMA) 184, 256
- Эмбединг (Embedding) 54
- Энкодер (Encoder) 56
- Энкодер-декодер (Encoder-Decoder) 55
- Эпоха 210

Я

- Языковая башня 116
- Языковая модель (Language model, LM) 31, 33

Омар Сансевьеро (Omar Sanseviero) был главным директором Llama и руководителем платформы Hugging Face, возглавлял команды по защите интересов разработчиков, команды по разработке приложений и команды в компании Moonshot. Омар имеет обширный опыт работы в Google, включая Google Assistant и TensorFlow Graphics. Его деятельность в Hugging Face находилась на стыке сообществ с открытым исходным кодом, продуктового, исследовательского и технического сообществ.

Педро Куэнка (Pedro Cuenca) — инженер машинного обучения в Hugging Face, работающий над программным обеспечением, моделями и приложениями для диффузии. У него более 20 лет опыта разработки программного обеспечения в области веб- и iOS-приложений. Будучи соучредителем и техническим директором LateNiteSoft, он работал над технологией Camera+. Это успешное приложение для iPhone, использующее пользовательские модели машинного обучения для улучшения фотографий. Он создал модели глубокого обучения для таких задач, как улучшение фотографий и получение сверхвысокого разрешения. Он также участвовал в разработке и эксплуатации DALL·E mini. Омар обладает практическим видением, как интегрировать исследования в области ИИ в реальные сервисы, а также связанные с этим проблемы и возможные оптимизации.

Аполинарио Пассос (Apolinário Passos) — инженер по машинному обучению в сфере искусства в компании Hugging Face. Он работает во многих командах над различными вариантами использования машинного обучения в сфере искусства и творчества. Аполинарио обладает более чем 10-летним профессиональным опытом, успешно балансируя между организацией художественных выставок, программированием и управлением продуктами. Также он занимается разработкой продуктов в World Data Lab. Он стремится достичь того, чтобы экосистема машинного обучения имела поддержку и смысл для художественных вариантов использования.

Джонатан Уитакер (Jonathan Whitaker) — специалист по данным и исследователь в области глубокого обучения, специализирующийся на генеративном моделировании. Ранее он работал над несколькими курсами, связанными с темами, затронутыми в этой книге, включая класс моделей диффузии в Hugging Face и курс «*From Deep Learning Foundations to Stable Diffusion*» в Fast.AI, созданный им совместно с Джереми Ховардом (Jeremy Howard) в 2022 году. Он также применял эти методы в индустрии, работая консультантом, и сейчас полностью посвятил себя исследованиям и разработкам в области ИИ в Answer.AI.

Об изображении на обложке

Животное, изображенное на обложке данной книги, — гигантская африканская бабочка-парусник (*Papilio antimachus*).

Это один из крупнейших видов бабочек с размахом крыльев до 9–10 дюймов (примерно с размер тарелки или виниловой пластинки). Тем не менее, несмотря на их внушительные размеры, об этих колоссальных насекомых известно относительно мало.

Впервые обнаруженные в 1782 году, парусники обитают в тропических лесах Западной и Центральной Африки, где проводят большую часть времени в пологе леса. Самцов иногда можно увидеть валяющимися в грязи на лесной органике — бабочки собираются на влажной поверхности вроде почвы или навоза в поисках питательных веществ.

Из-за своего рациона гигантские африканские бабочки-парусники очень ядовиты, и у них нет известных врагов среди хищников. Несмотря на новости о сокращении популяции в результате разрушения среды обитания и браконьерства (особи очень ценятся и могут стоить более 1000 долларов), Международный союз охраны природы отнес гигантского африканского парусника к категории видов, о которых недостаточно данных: для оценки его природоохранной значимости требуется больше информации. Многие животные на обложках издательства O'Reilly находятся под угрозой исчезновения, все они важны для мира.

Иллюстрация на обложке выполнена художницей Карен Монтгомери (Karen Montgomery). Дизайн серии разработан Эди Фридманом (Edie Freedman), Элли Фолькхаузен (Ellie Volckhausen) и Карен Монтгомери (Karen Montgomery).

Базовая математика для искусственного интеллекта



Прочитав книгу, вы сможете:

- уверенно пользоваться языками ИИ, машинного обучения, науки о данных, математики;
- объединять модели машинного обучения и модели естественного языка в рамках одной математической структуры;
- легко работать с графовыми и сетевыми данными;
- изучать реальные данные, визуализировать преобразования пространства, уменьшать размерность, обрабатывать изображения;
- решать, какие модели использовать для проектов, основанных на данных;
- изучать различные последствия и ограничения ИИ.

Сегодня многие сферы бизнеса стремятся внедрять новые технологии на основе ИИ и управления данными.

Однако для того чтобы создать действительно успешные системы ИИ, требуются прочные знания математики, лежащей в основе их работы. Книга представляет собой всеобъемлющее руководство, способное устранить существующий разрыв в представлении между неограниченным потенциалом и возможностями применения ИИ и соответствующими математическими основами.

Автор книги Хала Нельсон не углубляется в сложные академические теории, она рассказывает о математике, необходимой для успешной работы в области ИИ, уделяя особое внимание реальным приложениям и современным моделям.

В книге обсуждаются такие темы, как регрессия, нейронные сети, свертка, оптимизация, вероятность, марковские процессы, дифференциальные уравнения и многое другое, исключительно в контексте ИИ. Она будет интересна инженерам, специалистам по обработке данных, математикам, ученым в качестве прочной базы для успешной работы в различных областях ИИ и математики.

Хала Нельсон — доцент кафедры математики в Университете Джеймса Мэдисона (James Madison University), специализируется на математическом моделировании, консультирует официальные органы по вопросам чрезвычайных ситуаций и инфраструктуры. Получила докторскую степень по математике в Курантовском институте математических наук (Courant Institute of Mathematical Sciences) при Нью-Йоркском университете.

Прикладное машинное обучение и искусственный интеллект для инженеров



Эта книга поможет вам:

- узнать, что такое машинное обучение и глубокое обучение;
- понять, как работают популярные алгоритмы машинного обучения и когда их следует применять;
- построить модели машинного обучения на языке Python с помощью библиотеки Scikit-Learn, а также создать нейронные сети, используя библиотеки Keras и TensorFlow;
- обучать и оценивать регрессионные модели, а также модели бинарной и многоклассовой классификации;
- создавать модели распознавания лиц и обнаружения объектов;
- строить языковые модели, отвечающие на естественно-языковые вопросы и переводящие текст на другие языки;
- использовать набор облачных API Cognitive Services для внедрения ИИ в создаваемые вами приложения.

В то время как многие руководства по искусственному интеллекту (ИИ) представляют собой скорее учебники по математике, в этой книге математики практически нет. Вместо этого автор помогает инженерам и разработчикам программного обеспечения интуитивно понять и использовать ИИ для решения технических и бизнес-задач. Эта книга научит вас практическим навыкам, необходимым для внедрения ИИ и машинного обучения в вашей компании.

В книге приводятся примеры и иллюстрации из курсов по искусственному интеллекту и машинному обучению, которые автор преподает в компаниях и исследовательских институтах по всему миру. Здесь нет пустых слов и страшных уравнений — только полезная информация для инженеров и разработчиков программного обеспечения, дополненная практическими примерами.

Джеф Просиз — инженер, чья страсть — знакомить других инженеров и разработчиков программного обеспечения с чудесами искусственного интеллекта и машинного обучения. Соучредитель компании Wintellect, он написал девять книг и сотни статей, обучил тысячи разработчиков в Microsoft и выступал на крупнейших мировых конференциях. До этого Джеф занимался лазерными системами и исследованиями в области термоядерной энергии в Национальной лаборатории Оук-Ридж и Ливерморской национальной лаборатории имени Лоуренса. После продажи своей фирмы в 2021 г. занимает должность директора по обучению в компании Atmosera, где он помогает клиентам внедрять искусственный интеллект в их продукты.

Программирование с помощью искусственного интеллекта



В книге рассматриваются:

- Основные возможности ИИ-инструментов для разработки
- Плюсы, минусы и сценарии использования популярных систем, включая GitHub Copilot
- Применение универсальных языковых моделей (LLM), таких как ChatGPT, Gemini, Claude и других, для решения задач программирования
- Использование ИИ-инструментов в жизненном цикле разработки ПО: от разработки требований до тестирования
- Инженерия промптов для разработки
- Автоматизация рутинных задач, таких как создание регулярных выражений, с помощью ИИ
- Низкокодové и бескодové инструменты на основе ИИ

Получите практические советы, как использовать инструменты искусственного интеллекта для всех этапов создания кода: от разработки требований и планирования до проектирования, написания, отладки и тестирования. Начинающие и опытные разработчики узнают, как применять широкий спектр инструментов ИИ — от универсальных языковых моделей (ChatGPT, Gemini, Claude) до специализированных систем (GitHub Copilot, Tabnine, Cursor, Amazon CodeWhisperer).

Вы также познакомитесь с узкоспециализированными генеративными ИИ-инструментами для таких задач, как, например, преобразование текста в изображение.

Автор предлагает методологию модульного программирования, которая эффективно сочетается с подходом генерации кода с помощью ИИ. Также описаны лучшие способы использования универсальных языковых моделей для изучения языков программирования, объяснения кода или его преобразования с одного языка на другой.

Том Таулли — автор, консультант и инвестор, автор многочисленных книг, в том числе «Основы искусственного интеллекта: нетехническое введение». Публикуется в изданиях AIBusiness.com, Inc.com, Barrons.com, eSecurity Planet и Kiplinger.com, а также создает обучающие курсы для O'Reilly и Pluralsight, посвященные генеративному ИИ, базам данных и языку Python.

Ситунаяке Д., Планкетт Дж.
**Искусственный интеллект
для периферийных устройств**



В этой книге:

- Как наработать опыт в области машинного обучения и искусственного интеллекта для работы с периферийными устройствами
- Как понять, какие проскты эффективнее всего выполняются с привлечением периферийного искусственного интеллекта
- Какие ключевые паттерны применяются при проектировании приложений для периферийных вычислений на основе искусственного интеллекта
- Как внедрить итеративный подход к разработке систем с искусственным интеллектом
- Как собрать команду, обладающую всем спектром описанных в книге навыков и способную решать реальные задачи
- Как ответственно подходить к разработке устройств, оснащенных искусственным интеллектом

Парадигма периферийного, или пограничного, искусственного интеллекта (Edge AI) заставляет прямо сейчас пересматривать привычные принципы взаимодействия компьютеров с окружающей средой. Устройства, объединенные в Интернет вещей (IoT), самостоятельно принимают решения, опираясь на те 99% сенсорных данных, которые ранее просто отбрасывались ради экономии средств, полосы передачи данных или из-за ограничений питания. При помощи таких технологий, как машинное обучение для встраиваемых систем, можно учитывать информацию о человеческом поведении и развертывать приложения на любой платформе — от исключительно маломощных микроконтроллеров до встраиваемых устройств, работающих под Linux.

Изучив это практическое руководство, разработчики-профессионалы, менеджеры по продукту и руководители смогут подобрать инструментарий для решения разнообразных промышленных, коммерческих и научных задач с применением периферийных вычислений. В книге исследованы все этапы рабочего процесса: сбор данных, оптимизация модели, тонкая настройка и тестирование модели. Изучив предложенный материал, можно уверенно проектировать и поддерживать устройства с искусственным интеллектом и программировать встраиваемые модули с возможностями машинного обучения. Авторы подробно разъясняют, как сориентироваться в этой формирующейся отрасли и приступить к коммерческим разработкам.

Дэниел Ситунаяке — руководитель по машинному обучению в компании Edge Impulse, занимается научными исследованиями и разработками в сфере машинного обучения для встраиваемых систем.

Джени Планкетт — старший инженер по связям с разработчиками в Edge Impulse, евангелист, технический писатель, докладчик на технических конференциях.

Генеративный ИИ на практике: трансформеры и диффузионные модели

Перевод с английского А. Морозова

ТОО "АЛИСТ"
010000, Республика Казахстан,
г. Астана, пр. Сарыарка, д. 17, ВП 30

Подписано в печать 04.02.26.
Формат 70×100^{1/16}. Печать офсетная. Усл. печ. л. 32,25.
Тираж 1200 экз. Заказ № 16788.

Отпечатано с готового оригинал-макета
ООО "Принт-М", 142300, РФ, М.О., г. Чехов, ул. Полиграфистов, д. 1

«Фундаментальное руководство для разработчиков, позволяющее овладеть инструментами и концепциями самой значимой революции в области ИИ за последнее десятилетие».

Льюис Танстолл, инженер по машинному обучению в компании Hugging Face, соавтор книги «Natural Language Processing with Transformers»

«В этой книге есть все необходимое для старта в генеративном ИИ: от всеобъемлющих объяснений до продуманных советов и практических упражнений».

Люба Эллиотт, специалист по искусству, созданному с помощью искусственного интеллекта, elluba.com

Генеративный ИИ на практике: трансформеры и диффузионные модели

Это практическое руководство научит вас применять методы генеративного искусственного интеллекта для создания текстов, изображений, аудио и даже музыки. Вы поймете, как работают современные генеративные модели, как дообучать и адаптировать их под свои задачи, а также как комбинировать готовые компоненты для создания новых моделей и творческих приложений в различных областях. Книга сочетает теоретические концепции с практическими примерами, содержит рабочие листинги и наглядные иллюстрации. Вы научитесь использовать библиотеки с открытым исходным кодом для работы с трансформерами и диффузионными моделями, исследовать код и анализировать готовые проекты.

- Создавайте и настраивайте модели для генерации текста и изображений
- Сравнивайте подходы: использование предобученной модели и дообучение собственной
- Создавайте и применяйте модели для генерации, редактирования и стилизации изображений
- Адаптируйте трансформеры и диффузионные модели для различных творческих задач
- Обучайте модели для отражения вашего уникального стиля

Омар Сансевьеро — ведущий специалист по LLAMA и руководитель направления в компании Hugging Face. Имеет богатый опыт в области разработки продуктов с открытым исходным кодом, проведения исследований и управления командами.

Педро Куэнка — инженер по машинному обучению в Hugging Face.

Аполинарио Пассос — художник и инженер по машинному обучению в Hugging Face, создающий методы и инструменты для взаимодействия творческого сообщества с ИИ-моделями.

Джонатан Уитакер — специалист по данным и исследователь в области глубокого обучения в компании answer.ai. Ведет YouTube-канал [DataScienceCastnet](https://www.youtube.com/channel/UC8v33333333333333333333) и поддерживает бесплатные онлайн-ресурсы.

ISBN 978-601-12-6514-0



9 786011 265140

olist

O'REILLY®